Р. БЛЕЙХУТ

ТЕОРИЯ И ПРАКТИКА КОДОВ, КОНТРОЛИРУЮЩИХ ОШИБКИ

THEORY AND PRACTICE OF ERROR CONTROL CODES

RICHARD E. BLAHUT

ADDISON-WESLEY PUBLISHING COMPANY
READING, MASSACHUSETTS
MENLO PARK, CALIFORNIA · LONDON
AMSTERDAM · DON MILLS, ONTARIO
SYDNEY
REPRINTED WITH CORRECTION
1984

Р. БЛЕЙХУТ

ТЕОРИЯ И ПРАКТИКА КОДОВ, КОНТРОЛИРУЮЩИХ ОШИБКИ

ПЕРЕВОД С АНГЛИЙСКОГО
И. И. ГРУШКО и В. М. БЛИНОВСКОГО

ПОД РЕДАКЦИЕЙ
К. Ш. ЗИГАНГИРОВА

МОСКВА „МИР“ 1986

ББК 32.811
Б68
УДК 519.725

Блейхут Р.

Б68 Теория и практика кодов, контролирующих ошибки: Пер. с англ. — М.: Мир, 1986. — 576 с., ил.

Монография известного американского специалиста, адресованная тем, кто непосредственно разрабатывает программы и аппаратуру помехоустойчивого кодирования. В ней впервые излагается разработанный автором единый подход к кодированию и декодированию, основанный на дискретном преобразовании Фурье.

Для чтения книги достаточно знать математику в объеме втузовских программ (первые главы содержат необходимые сведения по алгебре). Она может служить и основой для курсов лекций, и пособием при первоначальном ознакомлении с предметом.

Для математиков-прикладников, программистов и инженеров, а также для аспирантов и студентов вузов.

Б $\frac{1502000000-183}{041\ (01)-86}$ 2—86, ч. 1

ББК 32.811

Редакция литературы по математическим наукам

Copyright © 1983 by Addison-Wesley Publishing Company, Inc.
© перевод на русский язык, «Мир», 1986

ОТ РЕДАКТОРА ПЕРЕВОДА

Книга Р. Блейхута посвящена разделу науки, за которым традиционно закреплено название «теория кодов, исправляющих ошибки». В последнее время, однако, в зарубежной литературе все чаще используется более точный термин «коды, контролирующие ошибки», поскольку эта теория изучает не только исправление, но и обнаружение ошибок. В отличие от традиционных курсов теории кодирования, адресованных в первую очередь специалистам в области передачи информации, данная книга ориентирована на проектировщиков цифровых комплексов обработки данных независимо от того, для чего предназначены эти комплексы: для передачи данных, для их хранения или для других операций над ними.

Это условие наложило свой отпечаток на структуру книги. В ней почти не рассматриваются вопросы оптимальности предлагаемых методов кодирования, а вопросы их схемной реализации исследуются более подробно, чем это обычно принято. Хотя книга в первую очередь адресована читателю с инженерным образованием и формально не требует от него предварительного знакомства с высшей алгеброй, ее математический уровень достаточно высок (отметим, что в начале книги имеются специальные главы, излагающие сведения из алгебры).

Центральное место в книге занимает спектральная теория циклических кодов, в развитие которой автор внес существенный вклад. Достаточно отметить модификацию автором процедуры Берлекэмпа—Месси, позволившую сократить число операций декодирования с $O(n^2)$ до $O(n \log n)$. Эта теория излагается в гл. 8—11 и частично в гл. 13 на основе дискретного преобразования Фурье (ДПФ) в конечных полях. Хотя связь ДПФ с циклическими кодами была известна и ранее, Р. Блейхут был первым, кто заметил большие потенциальные возможности такого подхода. Этот подход не только позволяет упростить процедуру декодирования, но и способствует более глубокому пониманию теории циклических кодов.

В основном книга посвящена теории блоковых кодов, и автор излагает ее превосходно. Теория сверточных кодов излагается

несколько бегло и не всегда достаточно четко. В частности, минимальное расстояние сверточных кодов, введенное в гл. 12, мало связано с реальными характеристиками этих кодов, а их теоретическое описание неточно. Некоторые неточности содержит и гл. 15, посвященная связи модуляции и кодирования.

В списке литературы неполно отражен вклад в теорию кодирования советских ученых, так что в отдельных случаях переводчики (гл. 1, 2, 4—6 и 8—15 переведены И. И. Грушко, гл. 3 и 7 — В. М. Блиновским) сочли необходимым дать соответствующие комментарии. При переводе учтены исправления, внесенные автором в первоначальный вариант книги, и исправлены замеченные опечатки.

Переводчики и редактор выражают признательность автору книги за сотрудничество в процессе перевода книги, в частности за любезно присланное им исправленное ее издание. Соответствующие изменения внесены в перевод в корректуре.

К. Ш. Зигангиров

ПРЕДИСЛОВИЕ К РУССКОМУ ИЗДАНИЮ

Автор переведенной на иностранный язык книги чувствует себя как отец, сын которого покидает дом, чтобы начать собственную жизнь: к чувству гордости примешивается сознание, что книга начинает новую жизнь, которую ты никогда не узнаешь. Русское издание готовилось специалистами высокого класса: редактором Камилем Зигангировым и переводчиками Инной Грушко и Владимиром Блиновским. Я очень благодарен им за большую работу по переводу книги «Теория и практика кодов, контролирующих ошибки» на русский язык.

Используя возможность, предоставленную мне издательством «Мир», я обращаюсь к советскому читателю. Он несомненно заметит, что книга адресована как инженерам, так и математикам, работающим в области приложений, и что в ней декодерам и алгоритмам декодирования уделяется гораздо больше внимания, чем в других книгах. Это связано отчасти с моими научными интересами и отчасти с широким распространением декодеров, исправляющих ошибки, за последнее десятилетие. Найти хороший алгоритм декодирования сейчас так же важно, как и найти хороший код. Читатель обнаружит также, что в книге недостаточно отражен крупный вклад советских исследователей в эту область. Частично это объясняется моей неосведомленностью, частично тем, что во многих советских работах принят более высокий уровень математической строгости, чем принятый в данной книге, и, наконец, тем, что советские исследователи не столь интенсивно работают в области алгоритмов декодирования, которая интересует меня больше всего.

Я хотел бы поблагодарить переводчиков и редактора за исправление многочисленных ошибок, которые они нашли, и принимаю ответственность за оставшиеся ошибки.

Р. Э. Блейхут

ОДНИ ЛИШЬ СЛОВА — ЭТО НИЧТО
— девиз Королевского общества

ПРЕДИСЛОВИЕ

В настоящее время невозможно представить себе инженера-конструктора цифровых систем, который бы не был знаком с кодами, контролирующими ошибки. Сегодняшний интерес к этому предмету резко контрастирует с тем, что в прежние годы такие коды считались практически применимыми лишь в самых дорогих системах связи. Необходимость в контроле ошибок сейчас так велика, а возможности электроники столь развиты, что интерес к этой тематике непрерывно растет. Умение применять кодирование стало важным для любого специалиста, создающего современные системы связи или большие цифровые системы, и это умение ценится все больше. По этой теме написаны превосходные книги, но в них основное внимание уделяется математическим исследовательским аспектам. Традиционно превалируют вопросы, связанные с построением лучших кодов. Хотя эти вопросы важны для дальнейшего развития теории, конструктору в принципе интересно лишь то, что он может построить.

Эта книга написана для студентов и инженеров, которые интересуются кодами, контролирующими ошибки, и намереваются использовать их в различных приложениях. Сказав это, мы должны, однако, также сказать, что отделить практику от теории невозможно. Не располагая необходимой математической базой и опираясь лишь на поверхностное знакомство с материалом, конструктор не сможет удовлетворительно работать, хотя ему нет необходимости столь же мастерски владеть всеми аспектами теории, как исследователю.

Книга возникла из конспекта курса лекций, посвященного контролирующим ошибки кодам, который автор много раз читал как в Корнеллском университете, так и в корпорации ИБМ. От слушателей этих лекций нельзя было ожидать какой-либо подготовки по современной алгебре. Поэтому одно из требований к курсу состояло в изложении необходимых основ алгебры, которое обеспечило бы достаточную математическую строгость, и в то же время умещалось в считанное число лекционных часов. Учитывая это ограничение, я попытался включить в книгу все те основы алгебры, которые требуются для введения в теорию контролирую-

щих ошибки кодов и позволяют проводить доказательства или убедительную аргументацию. Это тот минимум, без которого инженер не может быть уверен в своих построениях.

На протяжении тех лет, что читался курс, аудитория изменилась: сначала это были аспиранты и студенты старших курсов, а затем появились и студенты младших курсов; это заставило заботиться о более простых объяснениях, излагавшихся, по возможности, на языке, понятном инженерам. Математические рассуждения проводились на возможно более низком уровне, хотя в некоторых вопросах уровень оставался достаточно высоким.

Материал излагается в форме «теорема—доказательство», хотя в инженерной литературе чаще используется описательная форма. Принятая нами форма позволяет читателю при желании пропускать доказательства и выделять необходимые главные факты. Кроме того, те, кто заботится о строгости, могут разбить теорию на легче усваиваемые части.

Я пытался выдвинуть на первый план преобразование Фурье в конечных полях, так как оно интуитивно понятнее инженерам и быстро усваивается людьми с техническим образованием. Применение преобразования Фурье проясняет также тот факт, что теория контролирующих ошибки кодов является одной из ветвей теории дискретной обработки сигналов. Хотя для студентов инженерных специальностей предпочтительнее другой способ изложения того же круга идей, который основан на использовании многочленов Мэттсона—Соломона, такое изложение вызвало бы необходимость изучать новый для них язык в то время, как они уже владеют одним, столь же употребительным.

По возможности подчеркивалась тесная связь теории кодов, контролирующих ошибки, и теории обработки дискретных сигналов. Во многих работах эта связь оставалась невыявленной; объяснение заключается в том, что данные дисциплины развивались совершенно различными путями: одна разрабатывалась в основном алгебраистами, а другая — в основном инженерами. Однако, если не считать различия в числовых системах — поле Галуа в одном случае и поле комплексных чисел в другом — используемые методы аналогичны. Обе дисциплины основаны на преобразовании Фурье, фильтрах с конечным импульсным откликом, циклических свертках и соотношениях между свойствами последовательностей во временной и частотной областях.

В книге на первый план выдвигается реализация кодеров и декодеров с помощью цепей, содержащих регистры сдвигов, причем по возможности используется техническая терминология, принятая в теории фильтров. Для максимально возможной ясности изложения описывается построение регистров сдвига и часто выявляются модификации, позволяющие уменьшить число элементов устройства. Даже в тех случаях, когда описывается про-

граммная реализация кодеров и декодеров, приводятся некоторые соображения о том, как использование регистров сдвига может упростить программу. Моя точка зрения состоит в том, что окончательным критерием качества кода или алгоритма является стоимость кодера и декодера. Инженер не будет интересоваться кодами с наибольшим минимальным расстоянием, если неизвестны хорошие алгоритмы их декодирования. Хорошие коды нуждаются в хорошем декодере, а хорошие алгоритмы декодирования найти трудно. По-видимому, поиск теоретиками новых кодов, допускающих использование известных алгоритмов декодирования, может оказаться более плодотворным, нежели поиск новых алгоритмов декодирования для известных кодов.

При выборе обозначений и терминологии всегда возникает вопрос, использовать ли традиционные или ввести новые, более удобные. В выборе обозначений предпочтение отдается традиционным, хотя в некоторых случаях я считал, что важнее добиться методологической ясности и согласованности. Например, при обсуждении сверточных кодов я выбрал обозначения, подчеркивающие их аналогию с блоковыми кодами, хотя иногда они отличаются от обозначений, принятых в литературе по сверточным кодам.

Благодарности. Было бы очень трудно перечислить все беседы и источники, оказавшие существенное влияние на эту книгу: любой список оказался бы неполным, и поэтому я упомяну лишь главные.

Профессор Тоби Бергер был моим другом и консультантом на протяжении всех лет работы над книгой: его советы всегда были очень полезны. Профессор Д. Л. Сервейт внимательно прочитал большую часть рукописи и спас меня от многих ошибок и неточностей. Полезные советы и критические замечания сделали также К. Л. Чинь, А. ЭльГамаль, М. Р. Бест, Н. М. Блечман, Т. Хасимото, К. Кобаяси, М. Симада, Г. Унгербёк, В. Вандеркулк, С. Виноград и С. К. Вест. Книги и статьи, прямо или косвенно оказавшие существенное влияние на нашу книгу, перечислены в списке литературы; список статей, оказавших меньшее влияние, был бы необъятным.

Я должен выразить признательность корпорации ИБМ за поддержку при подготовке этой книги и Корнеллскому университету за предоставление лекционных помещений, в которых ее текст прошел апробацию. Текст книги шлифовался также в процессе лекций, прочитанных в Технологическом институте Южного Китая.

Наиболее важное участие в подготовке книги приняла моя жена Барбара. Она помогала и морально поддерживала меня, разделяя все трудности и удачи. Наконец, эта книга посвящается Эдварду Дж. Блейхуту, Эндрю С. Чамеру и Карлу А. Краченфелсу; частицы их душ живут в ней.

ГЛАВА 1
ВВЕДЕНИЕ

Обработка дискретных сигналов является инженерной дисциплиной со многими разветвлениями. Сюда относится и теория кодов, контролирующих ошибки, — отдельный предмет со своими собственными задачами и собственными арифметическими системами. Однако наиболее эффективными из этих арифметических систем являются известные операции обработки сигналов, в том числе свертки, преобразования Фурье, фильтры и регистры сдвигов. Теория кодов, контролирующих ошибки, — предмет со своей собственной историей и своими прелестями; его различные грани смыкаются со многими другими дисциплинами.

Рассматриваемая в теории кодов, контролирующих ошибки, техническая задача состоит в защите цифровых данных от появляющихся в процессе передачи по каналам связи ошибок. Многие хитроумные способы защиты от ошибок, развитые на основе богатой математической теории, превратились в зрелые важные инженерные методы с многочисленными приложениями.

Большие объемы данных в современных системах связи и хранения данных, большинство из которых очень чувствительно к ошибкам, приводят к необходимости контроля ошибок. Зрелая теория хороших кодов и хороших кодовых алгоритмов способна удовлетворить эту потребность. Кроме того, быстрые успехи в создании интегральных цифровых схем открывают возможность реализации этих алгоритмов.

1.1. ДИСКРЕТНЫЙ КАНАЛ СВЯЗИ

Система связи соединяет источник данных с получателем данных посредством канала; примерами каналов являются микроволновые линии, коаксиальные кабели, телефонные сети и даже магнитные ленты. При проектировании системы связи разрабатываются устройства, подготавливающие вход и обрабатывающие выход каналов. Уже стало традицией подразделять основные функции

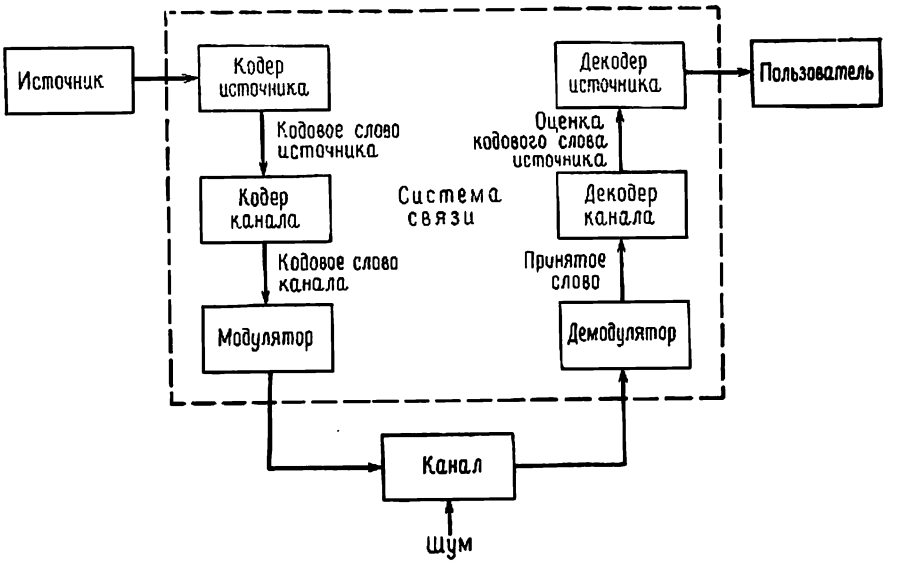


Рис. 1.1. Блок-схема цифровой системы связи.

цифровой системы связи так, как показано на блок-схеме на рис. 1.1.

Данные, поступающие в систему связи от источника данных, прежде всего обрабатываются кодером источника, предназначенным для более компактного представления данных источника. Это промежуточное представление является последовательностью символов, которая называется *кодовым словом источника*. Затем данные обрабатываются кодером канала, преобразующим последовательность символов кодового слова источника в другую последовательность символов, называемую *кодовым словом канала*. Кодовое слово канала представляет собой новую, более длинную последовательность с большей, чем у кодового источника, избыточностью. Каждый символ кодового слова канала может быть представлен битом или, возможно, группой битов.

Далее модулятор преобразует каждый символ кодового слова канала в соответствующий аналоговый символ из конечного множества допустимых аналоговых символов. Последовательность аналоговых символов передается по каналу. Так как в канале возникают различного типа шумы, искажения и интерференция, то выход канала отличается от его входа. Демодулятор преобразует каждый полученный на выходе канала сигнал в последовательность символов одного из кодовых слов канала. Каждый принятый символ является лучшей оценкой переданного символа, но из-за шума в канале демодулятор делает ошибки. Демодулиро-

ванная последовательность символов называется *принятым словом*. Из-за ошибок символы принятого слова не всегда соответствуют символам кодового слова канала.

Декодер канала использует избыточность кодового слова канала для того, чтобы исправить ошибки в принятом слове, и затем выдает оценку кодового слова источника. Если все ошибки исправлены, то оценка кодового слова источника совпадает с исходным кодовым словом источника. Декодер источника выполняет операцию, обратную операции кодера источника, результат которой поступает к получателю.

В данной книге рассматривается только конструкция кодера и декодера канала — дисциплина, известная как *кодирование, контролирующее ошибки*. Сжатие данных или функции более компактной записи данных, выполняемые кодером и декодером источника, а также устройства модулятора и демодулятора в ней не рассматриваются. Кодер и декодер канала будут в дальнейшем называться просто кодером и декодером соответственно.

1.2. ИСТОРИЯ КОДИРОВАНИЯ, КОНТРОЛИРУЮЩЕГО ОШИБКИ

История кодирования, контролирующего ошибки, началась в 1948 г. публикацией знаменитой статьи Клода Шеннона. Шеннон показал, что с каждым каналом связано измеряемое в битах в секунду и называемое *пропускной способностью* канала число C, имеющее следующее значение. Если требуемая от системы связи скорость передачи информации R (измеряемая в битах в секунду) меньше C, то, используя коды, контролирующие ошибки, для данного канала можно построить такую систему связи, что вероятность ошибки на выходе будет сколь угодно мала. В самом деле, из шенноновской теории информации следует тот важный вывод, что построение слишком хороших каналов является расточительством; экономически выгоднее использовать кодирование. Шеннон, однако, не указал, как найти подходящие коды, а лишь доказал их существование. В пятидесятые годы много усилий было потрачено на попытки построения в явном виде классов кодов, позволяющих получить обещанную сколь угодно малую вероятность ошибки, но результаты были скудными. В следующем десятилетии решению этой увлекательной задачи уделялось меньше внимания; вместо этого исследователи кодов предприняли длительную атаку по двум основным направлениям.

Первое направление носило чисто алгебраический характер и преимущественно рассматривало *блоковые коды*. Первые блоковые коды были введены в 1950 г., когда Хэмминг описал класс блоковых кодов, исправляющих одиночные ошибки. Коды Хэмминга были разочаровывающе слабы по сравнению с обещанными

Шенноном гораздо более сильными кодами. Несмотря на усиленные исследования, до конца пятидесятых годов не было построено лучшего класса кодов. В течение этого периода без какой-либо общей теории были найдены многие коды с малой длиной блока. Основной сдвиг произошел, когда Боуз и Рой-Чоудхури [1960] и Хоквингем [1959] нашли большой класс кодов, исправляющих кратные ошибки (коды БЧХ), а Рид и Соломон [1960] нашли связанный с кодами БЧХ класс кодов для недвоичных каналов. Хотя эти коды остаются среди наиболее важных классов кодов, общая теория блоковых кодов, контролирующих ошибки, с тех пор успешно развивалась, и время от времени удавалось открывать новые коды.

Открытие кодов БЧХ привело к поиску практических методов построения жестких или мягких реализаций кодеров и декодеров. Первый хороший алгоритм был предложен Питерсоном. Впоследствии мощный алгоритм выполнения описанных Питерсоном вычислений был предложен Берлекэмпом и Месси, и их реализация вошла в практику как только стала доступной новая цифровая техника.

Второе направление исследований по кодированию носило скорее вероятностный характер. Ранние исследования были связаны с оценками вероятностей ошибки для лучших семейств блоковых кодов, несмотря на то что эти лучшие коды не были известны. С этими исследованиями были связаны попытки понять кодирование и декодирование с вероятностной точки зрения, и эти попытки привели к появлению последовательного декодирования. В последовательном декодировании вводится класс неблоковых кодов бесконечной длины, которые можно описать деревом и декодировать с помощью алгоритмов поиска по дереву. Наиболее полезными древовидными кодами являются коды с тонкой структурой, известные под названием *сверточных кодов*. Эти коды можно генерировать с помощью цепей линейных регистров сдвига, выполняющих операцию свертки информационной последовательности. В конце 50-х годов для сверточных кодов были успешно разработаны алгоритмы последовательного декодирования. Интересно, что наиболее простой алгоритм декодирования — алгоритм Витерби — не был разработан для этих кодов до 1967 г. Применительно к сверточным кодам умеренной сложности алгоритм Витерби пользуется широкой популярностью, но для более мощных сверточных кодов он не практичен.

В 70-х годах эти два направления исследований опять стали переплетаться. Теорией сверточных кодов занялись алгебраисты, представившие ее в новом свете. В теории блоковых кодов за это время удалось приблизиться к кодам, обещанным Шенноном: были предложены две различные схемы кодирования (одна Юстесеном, а другая Гоппой), позволяющие строить семейства кодов,

которые одновременно могут иметь очень большую длину блока и очень хорошие характеристики. Обе схемы, однако, имеют практические ограничения, и надо ждать дальнейших успехов. Между тем к началу 80-х годов кодеры и декодеры начали появляться в новейших конструкциях цифровых систем связи и цифровых систем памяти.

1.3. ПРИЛОЖЕНИЯ

Так как развитие кодов, контролирующих ошибки, первоначально стимулировалось задачами связи, терминология теории кодирования проистекает из теории связи. Построенные коды, однако, имеют много других приложений. Коды используются для защиты данных в памяти вычислительных устройств и на цифровых лентах и дисках, а также для защиты от неправильного функционирования или шумов в цифровых логических цепях. Коды используются также для сжатия данных, и теория кодирования тесно связана с теорией планирования статистических экспериментов.

Приложения к задачам связи носят самый различный характер. Двоичные данные обычно передаются между вычислительными терминалами, между летательными аппаратами и между спутниками. Коды могут быть использованы для получения надежной связи даже тогда, когда мощность принимаемого сигнала близка к мощности тепловых шумов. И поскольку электромагнитный спектр все больше и больше заполняется создаваемым человеком сигналами, коды, контролирующие ошибки, становятся еще более важным инструментом, так как позволяют линиям связи надежно работать при наличии интерференции. В военных приложениях такие коды часто используются для защиты против намеренно организованной противником интерференции.

Во многих системах связи имеется ограничение на передаваемую мощность. Например, в системах ретрансляции через спутники увеличение мощности обходится очень дорого. Коды, контролирующие ошибки, являются замечательным средством снижения необходимой мощности, так как с их помощью можно правильно восстановить полученные ослабленные сообщения.

Передача в вычислительных системах обычно чувствительна даже к очень малой доле ошибок, так как одиночная ошибка может нарушить программу вычисления. Кодирование, контролирующее ошибки, становится в этих приложениях весьма важным. Для некоторых носителей вычислительной памяти (магнитных лент, например) использование кодов, контролирующих ошибки, позволяет добиться более плотной упаковки битов.

Другим типом систем связи является система с многими пользователями и разделением по времени, в которой каждому из данного числа пользователей заранее предписаны некоторые вре-

менные окна (интервалы), в которых ему разрешается передача. Длинные двоичные сообщения разделяются на пакеты, и один пакет передается в отведенное временное окно. Из-за нарушения синхронизации или дисциплины обслуживания некоторые пакеты могут быть утеряны. Подходящие коды, контролирующие ошибки, защищают от таких потерь, так как утерянные пакеты можно восстановить по известным пакетам.

Связь важна также внутри одной системы. В современных сложных цифровых системах могут возникнуть большие потоки данных между подсистемами. Цифровые автопилоты, цифровые системы управления процессами, цифровые переключательные системы и цифровые системы обработки радарных сигналов — все это системы, содержащие большие массивы цифровых данных, которые должны быть распределены между многими взаимно связанными подсистемами. Эти данные должны быть переданы или по специально предназначенным для этого линиям, или посредством более сложных систем с шинами передачи данных и с разделением по времени. В любом случае важную роль играют методы кодирования, контролирующего ошибки, так как они позволяют гарантировать соответствующие характеристики.

Со временем коды, контролирующие ошибки, и устройства кодирования и декодирования достигли такого уровня развития, на котором они могут обрабатывать большие массивы данных. Можно ожидать, что в системах связи будущего методы кодирования, контролирующего ошибки, будут играть центральную роль. Представляется правдоподобным, что в будущем грамофонные пластинки, ленты и телевизионные сигналы будут содержать цифровые сообщения, защищенные кодами, контролирующими ошибки. Влияние повреждений пластинки или интерференции в принятом сигнале может быть полностью устранено кодированием, если число ошибок не выходит за пределы возможностей заложенного в конструкцию кода, контролирующего ошибки.

1.4. ОСНОВНЫЕ ПОНЯТИЯ

Предмет кодирования, контролирующего ошибки, одновременно прост и сложен. Он прост в том смысле, что его задачи легко объяснить любому технически грамотному человеку, и сложен в том смысле, что для того, чтобы получить решение задачи (и даже частное ее решение), потребуется весь объем данной книги, причем изложению должно предшествовать отступление, посвященное некоторым разделам современной алгебры.

Предположим, что все представляющие интерес данные могут быть представлены в виде двоичной (закодированной двоично) информации, т. е. в виде последовательности нулей и единиц. Эта двоичная информация подлежит передаче по каналу, подвержен-

ному случайным ошибкам. Задача кодирования состоит в таком добавлении к информационным символам дополнительных символов, чтобы на приемнике эти искажения могли быть найдены и исправлены. Иначе говоря, последовательность символов данных представляется в виде некоторой более длинной последовательности символов, избыточность которой достаточна для защиты данных.

Двоичный код мощности M и длины n представляет собой множество из M двоичных слов длины n, называемых кодовыми словами. Обычно $M = 2^k$, где k — некоторое целое число; такой код называется двоичным (n, k)-кодом.

Например, можно построить следующий код:

$$\mathscr{C} = \begin{Bmatrix} 1\,0\,1\,0\,1 \\ 1\,0\,0\,1\,0 \\ 0\,1\,1\,1\,0 \\ 1\,1\,1\,1\,1 \end{Bmatrix}.$$

Это очень бедный (и очень маленький) код с $M = 4$ и $n = 5$, но он удовлетворяет приведенному выше определению. Данный код можно использовать для представления 2-битовых двоичных чисел, используя следующее (произвольное) соответствие:

$$\begin{aligned} 00 &\leftrightarrow 10101, \\ 01 &\leftrightarrow 10010, \\ 10 &\leftrightarrow 01110, \\ 11 &\leftrightarrow 11111. \end{aligned}$$

Если получено одно из четырех 5-битовых кодовых слов, то полагаем, что соответствующие ему два бита являются правильной информацией. Если произошла ошибка, то мы получим 5-битовое слово, отличающееся от кодовых слов. Тогда попытаемся найти наиболее вероятно переданное слово и возьмем его в качестве оценки исходных двух битов информации. Например, если мы приняли 01100, то полагаем, что передавалось 01110, и, следовательно, информационное слово равнялось 10.

В приведенном примере код не является хорошим, так как он не позволяет исправлять много конфигураций ошибок. Желательно выбирать коды так, чтобы каждое кодовое слово по возможности больше отличалось от каждого другого кодового слова; в частности, это желательно в том случае, когда длина блока велика.

Первой целью данной книги является построение хороших кодов. Хотя с внешней стороны эта задача может показаться простой, на самом деле она чрезвычайно трудна, и многие хорошие коды пока еще не найдены.

По неопытности может показаться, что достаточно определить требования к хорошему коду и затем предпринять машинный поиск

по множеству всех возможных кодов. Но сколько кодов существует для данных n и k? Каждое кодовое слово представляет собой последовательность n двоичных символов, и всего имеется 2^k таких слов; следовательно, код описывается $n \cdot 2^k$ двоичными символами. Всего существует $2^{n \cdot 2^k}$ способов выбора этих двоичных символов; следовательно, число различных (n, k)-кодов равно $2^{n \cdot 2^k}$. Конечно, очень многие из этих кодов не представляют интереса (как в случае, когда два кодовых слова равны), так что надо либо проверять это по ходу поиска, либо развить некоторую теорию, позволяющую исключить такие коды.

Например, выберем $(n, k) = (40, 20)$ — код, весьма умеренный по современным стандартам. Тогда число таких кодов превзойдет величину $10^{10\,000\,000}$ — невообразимо большое число! Следовательно, неорганизованные процедуры поиска бессильны.

В общем случае блоковые коды определяются над произвольным конечным алфавитом, скажем над алфавитом из q символов $\{0, 1, 2, \ldots, q-1\}$. На первый взгляд введение алфавитов, отличных от двоичного, может показаться излишним обобщением. Из соображений эффективности, однако, многие современные каналы являются недвоичными, и коды для этих каналов должны быть недвоичными. На самом деле коды для недвоичных каналов часто оказываются достаточно хорошими, и сам этот факт может служить причиной для использования недвоичных каналов. Двоичные данные источника тривиальным образом представляются символами q-ичного алфавита, особенно если q равно степени двойки, как это обычно и бывает на практике.

Определение 1.4.1. *Блоковый код* мощности M над алфавитом из q символов определяется как множество из M q-ичных последовательностей длины n, называемых *кодовыми словами*.

Если $q = 2$, то символы называются *битами*. Обычно $M = q^k$ для некоторого целого k, и мы будем интересоваться только этим случаем, называя код (n, k)-кодом. Каждой последовательности из k q-ичных символов можно сопоставить последовательность из n q-ичных символов, являющуюся кодовым словом.

Имеются два основных класса кодов: блоковые коды и древовидные коды; они иллюстрируются рис. 1.2. Блоковый код задает блок из k информационных символов n-символьным кодовым словом. *Скорость* R блокового кода [1]) определяется равенством

$$R = k/n.$$

[1]) Скорость — величина безразмерная или, возможно, измеряемая в единицах бит/бит или символ/символ. Ее следует отличать от другого называемого тем же термином *скорость* понятия, измеряющего канальную скорость в бит/с. Используется и другое определение скорости: $R = (k/n) \log_e q$, единицей которой является нат/символ, где один нат равен $\log_2 e$ битов. Принято также определение $R = (k/n) \log_2 q$, в котором скорость измеряется в единицах бит/символ,

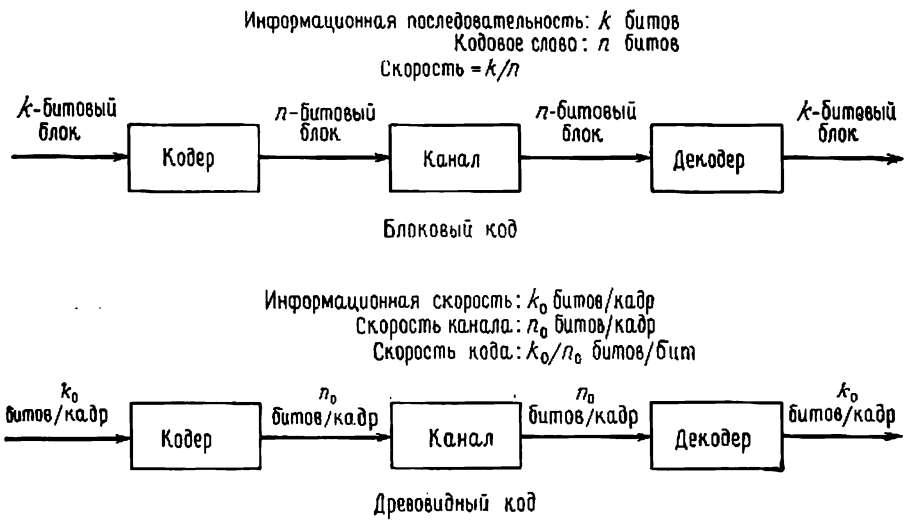


Рис. 1.2. Основные классы кодов.

Древовидный код более сложен. Он отображает бесконечную последовательность информационных символов, поступающую со скоростью k_0 символов за один интервал времени, в непрерывную последовательность символов кодового слова со скоростью n_0 символов за один интервал времени. Мы отложим рассмотрение древовидных кодов до гл. 12 и сначала сосредоточим внимание на блоковых кодах.

Если сообщение состоит из большого числа битов, то в принципе лучше использовать один кодовый блок большой длины, чем последовательность кодовых слов из более короткого кода. Природа статистических флуктуаций такова, что случайная конфигурация ошибок обычно имеет вид серии ошибок. Некоторые сегменты этой конфигурации содержат больше среднего числа ошибок, а некоторые меньше. Следовательно, при одной и той же скорости более длинные кодовые слова гораздо менее чувствительны к ошибкам, чем более короткие кодовые слова, но, конечно, соответствующие кодер и декодер могут быть более сложными. Например, предположим, что 1000 информационных битов передаются с помощью (воображаемого) 2000-битового двоичного кода, способного исправлять 100 ошибок. Сравним такую возможность с передачей одновременно 100 битов с помощью 200-битового кода, исправляющего 10 ошибок на блок. Для передачи 1000 битов необходимо 10 таких блоков. Вторая схема также может исправлять 100 ошибок, но лишь тогда, когда они распределены частным образом — по 10 ошибок в 200-битовых подблоках. Первая схема может исправлять 100 ошибок независимо от того, как они рас-

положены внутри 2000-битового кодового слова. Она существенно эффективнее.

Эти эвристические рассуждения можно обосновать теоретически, но здесь мы к этому не стремимся. Мы только хотим обосновать тот факт, что хорошими являются коды с большой длиной блока и что очень хорошими кодами являются коды с очень большой длиной блока. Такие коды может быть очень трудно найти, а будучи найденными, они могут потребовать сложных устройств для реализации операций кодирования и декодирования.

О блоковом коде судят по трем параметрам: длине блока n, информационной длине k и минимальному расстоянию d^*. Минимальное расстояние является мерой различия двух наиболее похожих кодовых слов. Минимальное расстояние вводится двумя следующими определениями.

Определение 1.4.2. *Расстоянием по Хэммингу* между двумя q-ичными последовательностями $\mathbf{x}$ и $\mathbf{y}$ длины n называется число позиций, в которых они различны. Это расстояние обозначается через $d(\mathbf{x}, \mathbf{y})$.

Например, возьмем $\mathbf{x} = 10101$ и $\mathbf{y} = 01100$; тогда имеем $d(10101, 01100) = 3$. В качестве другого примера возьмем $\mathbf{x} = 30102$ и $y = 21103$; тогда $d(30102, 21103) = 3$.

Определение 1.4.3. Пусть $\mathscr{C} = \{\mathbf{c}_i,\ i = 0, \ldots, M-1\}$ — код. Тогда *минимальное расстояние* кода $\mathscr{C}$ равно наименьшему из всех расстояний по Хэммингу между различными парами кодовых слов, т. е.

$$d^* = \min_{\substack{\mathbf{c}_i, \mathbf{c}_j \in \mathscr{C} \\ i \neq j}} d(\mathbf{c}_i, \mathbf{c}_j).$$

(n, k)-код с минимальным расстоянием d^* называется также (n, k, d^*)-кодом.

В коде $\mathscr{C}$, выбранном в первом примере данного параграфа,

$$d(10101, 10010) = 3, \quad d(10010, 01110) = 3,$$
$$d(10101, 01110) = 4, \quad d(10010, 11111) = 3,$$
$$d(10101, 11111) = 2, \quad d(01110, 11111) = 2;$$

следовательно, для этого кода $d^* = 2$.

Предположим, что передано кодовое слово и в канале произошла одиночная ошибка. Тогда принятое слово находится на равном 1 расстоянии по Хэммингу от переданного слова. В случае когда расстояние до каждого другого кодового слова больше чем 1, декодер исправит ошибку, если положит, что действительно переданным словом было ближайшее к принятому кодовое слово.

В более общем случае если произошло t ошибок и если расстояние от принятого слова до каждого другого кодового слова больше t, то декодер исправит эти ошибки, приняв ближайшее к принятому кодовое слово в качестве действительно переданного. Это всегда будет так, если

$$d^* \geqslant 2t + 1.$$

Иногда удается исправлять конфигурацию из t ошибок даже тогда, когда это неравенство не удовлетворяется. Однако если $d^* < 2t + 1$, то исправление любых t ошибок не может быть гарантировано, так как тогда оно зависит от того, какое слово передавалось и какова была конфигурация из t ошибок внутри блока.

Геометрическая иллюстрация дается на рис. 1.3. В пространстве всех q-ичных n-последовательностей выбрано некоторое множество n-последовательностей, объявленных кодовыми словами. Если d^* — минимальное расстояние этого кода, а t — наибольшее целое число, удовлетворяющее условию $d^* \geqslant 2t+1$, то вокруг каждого кодового слова можно описать непересекающиеся сферы радиуса t. Принятые слова, лежащие внутри сфер, декодируются как кодовое слово, являющееся центром соответствующей сферы. Если произошло не более t ошибок, то принятое слово всегда лежит внутри соответствующей сферы и декодируется правильно.

Некоторые принятые слова, содержащие более t ошибок, попадут внутрь сферы, описанной вокруг другого кодового слова, и будут декодированы неправильно. Другие принятые слова,

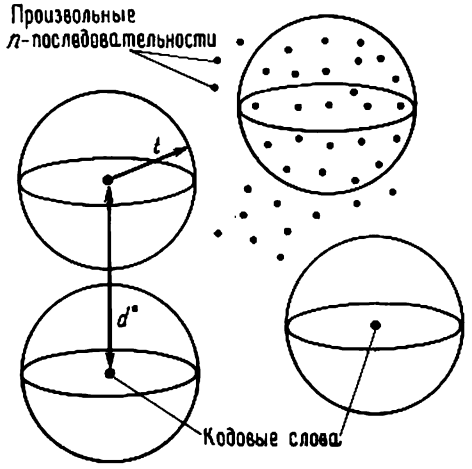


Рис. 1.3. Сферы декодирования.

содержащие более t ошибок, попадут в промежуточные между сферами декодирования области. В зависимости от применения последний факт можно интерпретировать одним из двух способов.

Неполный декодер декодирует только те принятые слова, которые лежат внутри сфер декодирования, описанных вокруг кодовых слов. Остальные принятые слова, содержащие более допустимого числа ошибок, декодер объявляет нераспознаваемыми. Такие конфигурации ошибок при неполном декодировании называются *неисправляемыми*. Большинство используемых декодеров являются неполными декодерами.

Полный декодер декодирует каждое принятое слово в ближайшее кодовое слово. Геометрически это представляется следующим образом: полный декодер разрезает промежуточные области на куски и присоединяет их к сферам так, что каждая точка попадает в ближайшую сферу. Обычно некоторые точки находятся на равных расстояниях от нескольких сфер; тогда одна из этих сфер произвольно объявляется ближайшей. Если происходит более t ошибок, то полный декодер часто декодирует неправильно, но бывают и случаи попадания в правильное кодовое слово. Полный декодер используется в тех случаях, когда лучше угадывать сообщение, чем вообще не иметь никакой его оценки.

Мы будем также рассматривать каналы, в которых кроме ошибок происходят и стирания. Это значит, что конструкцией приемника предусмотрено объявление символа стертым, если он получен ненадежно или если приемник распознал наличие интерференции или сбой. Такой канал имеет входной алфавит мощности q и выходной алфавит мощности $q + 1$; дополнительный символ называется *стиранием*. Например, стирание символа 3 в сообщении 12345 приводит к слову 12-45. Это не следует путать с другой операцией, называемой *выбрасыванием*, которая дает 1245.

В таких каналах могут использоваться коды, контролирующие ошибки. В случае когда минимальное расстояние кода равно d^*, любая конфигурация из ρ стираний может быть восстановлена, если $d^* \geqslant \rho + 1$. Далее, любая конфигурация из ν ошибок и ρ стираний может быть декодирована при условии, что

$$d^* \geqslant 2\nu + 1 + \rho.$$

Для доказательства выбросим из всех кодовых слов те ρ компонент, в которых приемник произвел стирания. Это даст новый код, минимальное расстояние которого не меньше $d^* - \rho$; следовательно, ν ошибок могут быть исправлены при условии, что выполняется выписанное выше неравенство. Таким образом, можно восстановить укороченное кодовое слово с ρ стертыми компонентами. Наконец, так как $d^* > \rho + 1$, существует только

одно кодовое слово, совпадающее с полученным в нестертых компонентах; следовательно, исходное кодовое слово может быть восстановлено.

1.5. ПРОСТЕЙШИЕ КОДЫ

Некоторые коды настолько просты, что их можно описать в самом начале.

Простые коды с проверкой на четность. Это высокоскоростные коды с плохими корректирующими характеристиками. К заданным k информационным битам дописывается $(k+1)$-й бит так, чтобы полное число единиц в кодовом слове было четным [1]). Таким образом, например, для $k=4$

$$0\,0\,0\,0 \leftrightarrow 0\,0\,0\,0\,0,$$
$$0\,0\,0\,1 \leftrightarrow 0\,0\,0\,1\,1,$$
$$0\,0\,1\,0 \leftrightarrow 0\,0\,1\,0\,1,$$
$$0\,0\,1\,1 \leftrightarrow 0\,0\,1\,1\,0$$

и т. д. Этот код является $(k+1, k)$-кодом или $(n, n-1)$-кодом. Минимальное расстояние кода равно двум, и, следовательно, никакие ошибки не могут быть исправлены. Простой код с проверкой на четность используется для обнаружения (но не исправления) одной ошибки.

Простые коды с повторением. Это низкоскоростные коды с хорошими корректирующими характеристиками. Один заданный информационный символ повторяется n раз (обычно n нечетно). Таким образом,

$$0 \leftrightarrow 0\,0\,0\,0\,0,$$
$$1 \leftrightarrow 1\,1\,1\,1\,1.$$

Это $(n, 1)$-код. Для него минимальное расстояние равно n, и при предположении, что большинство принятых битов совпадает с переданным информационным битом, может быть исправлено $(n-1)/2$ ошибок.

Коды Хэмминга. Эти коды позволяют исправлять одну ошибку. Сейчас мы введем эти коды с помощью непосредственного описания. Для каждого m существует $(2^m-1,\ 2^m-1-m)$-код Хэмминга. При больших m скорость кода близка к 1, но доля общего числа битов, которые могут быть искажены, очень мала. (7, 4)-код Хэмминга можно описать с помощью приведенной на рис. 1.4, *а* реализации. При заданных четырех информационных битах (i_1, i_2, i_3, i_4) полагаем первые четыре бита кодового слова

[1]) В дальнейшем этот дополнительный бит называется битом проверки на четность. — *Прим. ред.*

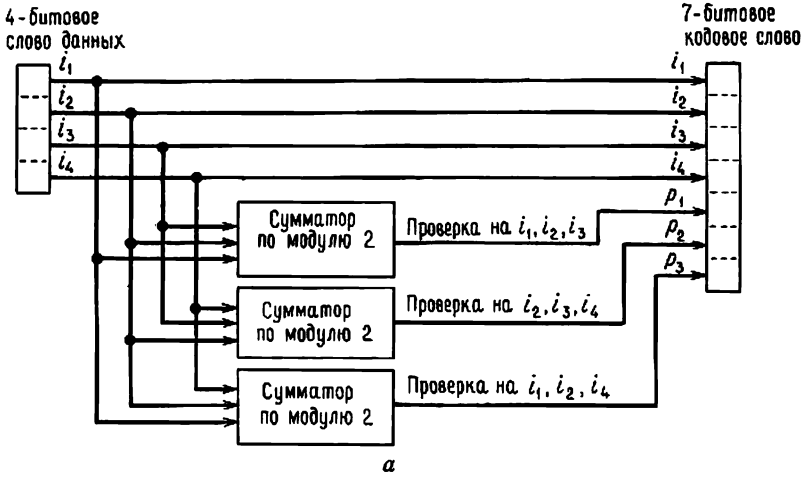


Рис. 1.4. Кодер/декодер для простого (7, 4)-кода Хэмминга. *а* — кодер; *б* — декодер.

равными этим четырем информационным битам. Дополняем тремя *проверочными битами*, задавая их равенствами

$$p_1 = i_1 + i_2 + i_3,$$
$$p_2 = i_2 + i_3 + i_4,$$
$$p_3 = i_1 + i_2 + i_4.$$

Здесь $+$ обозначает сложение по модулю 2: $0 + 0 = 0$, $0 + 1 = 1$, $1 + 0 = 1$, $1 + 1 = 0$. Шестнадцать кодовых слов (7, 4)-кода Хэмминга выписаны в табл. 1.1. Декодер принимает 7-битовое

Таблица 1.1

(7, 4)-код Хэмминга

0 0 0 0 0 0 0
0 0 0 1 0 1 1
0 0 1 0 1 1 0
0 0 1 1 1 0 1
0 1 0 0 1 1 1
0 1 0 1 1 0 0
0 1 1 0 0 0 1
0 1 1 1 0 1 0
1 0 0 0 1 0 1
1 0 0 1 1 1 0
1 0 1 0 0 1 1
1 0 1 1 0 0 0
1 1 0 0 0 1 0
1 1 0 1 0 0 1
1 1 1 0 1 0 0
1 1 1 1 1 1 1

слово $\mathbf{v} = (i_1', i_2', i_3', i_4', p_1', p_2', p_3')$. При передаче в этом слове произошло не более одной ошибки. Изображенный на рис. 1.4, *б* декодер вычисляет биты

$$s_1 = p_1' + i_1' + i_2' + i_3',$$
$$s_2 = p_2' + i_2' + i_3' + i_4',$$
$$s_3 = p_3' + i_1' + i_2' + i_4'.$$

Трехбитовая последовательность (s_1, s_2, s_3) называется *синдромом*. Она зависит не от истинных информационных битов, а только от конфигурации ошибок. Всего имеется восемь возможных синдромов: один для случая отсутствия ошибки и по одному для каждой из семи возможных одиночных ошибок. Простая проверка показывает, что каждая из этих ошибок имеет свой единственный синдром. Таким образом, не составляет труда сконструировать цифровую логику, которая по синдрому локализует

соответствующий бит. После внесения исправления проверочные символы можно опустить. Две или более ошибки превышают возможности кодовой конструкции, и код будет ошибаться. Это означает, что он будет вносить неправильные исправления и выдавать искаженные информационные биты.

Идея этой кодовой конструкции, естественно, не меняется при перестановке позиций битов в кодовых словах. Все такие варианты называются (7, 4)-кодом Хэмминга.

ЗАДАЧИ

1.1. Исправляющие одну ошибку коды Хэмминга содержат $2^m - 1$ битов, m из которых являются проверочными битами.

а. Выписать (n, k) для первых пяти нетривиальных кодов Хэмминга (начиная с $m = 3$).

б. Вычислить скорости этих кодов.

в. Выписать выражение для вероятности p_e ошибки декодирования при условии, что код используется в двоичном канале с вероятностью ошибки q. Как ведет себя вероятность p_e с ростом n?

1.2. Руководствуясь рис. 1.4, сконструировать кодер/декодер для (15,11)-кода Хэмминга. Опустить повторяющиеся детали (т. е. в принципе показать, как это можно сделать).

1.3.а. Методом проб и ошибок найти множество из четырех двоичных слов длиной 3, таких, что каждое слово находится от каждого другого слова на расстоянии, не меньшем 2.

б. Найти множество из 16 двоичных слов длиной 7, такое, что каждое слово находится от каждого другого слова на расстоянии, не меньшем 3.

1.4.а. Описать, как вырезать 88 кругов диаметром 1 дюйм ($\approx$2,5 см) из листа бумаги шириной 8,5 дюйма и длиной 11 дюймов. Доказать, что нельзя вырезать более 119 кругов диаметром 1 дюйм.

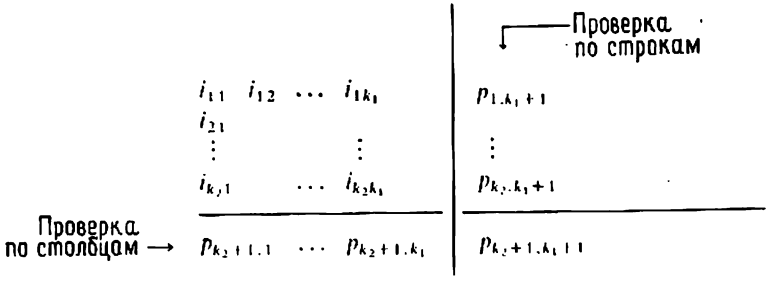
б. Доказать, что нельзя найти 32 двоичных слова длиной 8, таких, что каждое слово отличается от каждого другого слова не менее чем тремя компонентами.

1.5. Доказать, что число k информационных символов любого блокового (n, k)-кода с минимальным расстоянием, не меньшим $2t + 1$, удовлетворяет неравенству

$$n - k \geqslant \log_q \left[1 + \binom{n}{1}(q-1) + \binom{n}{2}(q-1)^2 + \cdots + \binom{n}{t}(q-1)^t \right]$$

(граница Хэмминга для алфавита из q элементов).

1.6. Простейший пример кодовой конструкции, известной под названием «код-произведение», имеет вид



где k_1k_2 символов верхнего левого блока являются двоичными информационными символами. Конструкция дает $((k_1+1)(k_2+1),\ k_1k_2)$-код.

а. Показать, что $p_{k_2+1,\,k_1+1}$ является проверочным символом и по строкам, и по столбцам.

б. Показать, что данный код является кодом, исправляющим одну ошибку.

в. Показать, что данный код является кодом, обнаруживающим две ошибки. Привести пример двух двойных ошибок, которые этот код не может отличить одну от другой.

г. Чему равно минимальное расстояние данного кода?

1.7. Доказать, что расстояние по Хэммингу обладает следующими тремя свойствами:

(i) $d(\mathbf{x}, \mathbf{y}) \geqslant 0$, причем равенство достигается тогда и только тогда, когда $\mathbf{x} = \mathbf{y}$:

(ii) $d(\mathbf{x}, \mathbf{y}) = d(\mathbf{y}, \mathbf{x})$;

(iii) выполняется неравенство треугольника $d(\mathbf{x}, \mathbf{y}) \leqslant d(\mathbf{y}, \mathbf{z}) + d(\mathbf{x}, \mathbf{z})$.

Функция расстояния, обладающая этими тремя свойствами, называется *метрикой*.

1.8.а. Показать, что код $\mathscr{C}$ позволяет обнаруживать любую конфигурацию не более чем из d ошибок тогда и только тогда, когда минимальное расстояния кода $\mathscr{C}$ больше d.

б. Показать, что код способен исправлять любую конфигурацию не более чем из t ошибок, если минимальное расстояние кода равно по меньшей мере $2t+1$.

в. Показать, что код можно использовать для исправления всех конфигураций не более чем из t ошибок и одновременного обнаружения всех конфигураций не более чем из d ошибок ($d \geqslant t$), если минимальное расстояние кода не меньше $d+t+1$.

г. Показать, что код можно использовать для восстановления ρ стертых символов, если минимальное расстояние кода не меньше $\rho+1$.

1.9.а. Показать, что если двоичный (15, 11)-код Хэмминга используется в канале, который вносит две ошибки, то выход декодера всегда будет ошибочным.

б. Показать, как используя проверку на четность, расширить (15, 11)-код Хэмминга до (16, 11)-кода, исправляющего все одиночные и обнаруживающего все двойные ошибки. Чему равно минимальное расстояние этого кода?

ГЛАВА 2

ВВЕДЕНИЕ В АЛГЕБРУ

Поиск хороших кодов, контролирующих ошибки, в значительной мере связан с мощными и изящными структурами современной алгебры. Многие найденные важные коды основаны на структурах колец многочленов и полей Галуа. Кроме того, эти алгебраические понятия и методы являются необходимым рабочим инструментом для конструирования кодеров и декодеров. Данная глава и гл. 4 посвящены изложению тех разделов алгебры, которые важны для теории кодов, контролирующих ошибки. Изложение ведется строго, но ограничивается лишь используемым в дальнейшем материалом.

2.1. 2-ПОЛЕ И 6-10-ПОЛЕ

Вещественные числа образуют известное множество математических объектов, которые можно складывать, умножать, вычитать и делить. Аналогично комплексные числа образуют множество объектов, которые можно складывать, вычитать, умножать и делить. Обе эти арифметические системы являются важнейшей основой всех инженерных дисциплин. Нам необходимо построить другие, менее известные арифметические системы, полезные при исследовании кодов, контролирующих ошибки. Такие новые арифметические системы состоят из множеств и операций над элементами этих множеств. Хотя мы будем называть операции «сложением», «вычитанием», «умножением» и «делением», они не обязательно будут теми же операциями, что в элементарной арифметике.

Изучаемые в современной алгебре арифметические системы классифицируются в соответствии с усложнением их математической структуры. Такая формальная классификация будет приведена ниже в данной главе. Пока мы дадим следующие неформальные определения:

1) *абелева группа* [1]: множество математических объектов, которые можно «складывать» и «вычитать»;

2) *кольцо*: множество математических объектов, которые можно «складывать», «вычитать» и «умножать»;

3) *поле*: множество математических объектов, которые можно «складывать», «вычитать», «умножать» и «делить».

Заметим, что названия этих операций взяты в кавычки потому, что, вообще говоря, они не являются принятыми арифметическими операциями; эти названия употребляются из-за их сходства с принятыми.

Прежде чем переходить к формальным понятиям, выполним некоторые вычисления в простейшем из всех возможных полей, а именно в поле, состоящем только из двух элементов. (Поле вещественных чисел содержит бесконечное число элементов.) Обозначим через 0 и 1 два элемента поля и определим операции сложения и умножения равенствами

$$\begin{aligned} 0+0&=0, & 0\cdot 0&=0,\\ 0+1&=1, & 0\cdot 1&=0,\\ 1+0&=1, & 1\cdot 0&=0,\\ 1+1&=0, & 1\cdot 1&=1. \end{aligned}$$

Так определенные сложение и умножение называются *сложением по модулю* 2 и *умножением по модулю* 2. Заметим, что из равенства $1+1=0$ следует, что $-1=1$, а из равенства $1\cdot 1=1$ — что $1^{-1}=1$. Используя это, легко проверить, что, за исключением деления на нуль, вычитание и деление всегда определены. Алфавит из двух символов 0 и 1 вместе со сложением по модулю 2 и умножением по модулю 2 называется полем из двух элементов и по приведенным в гл. 4 соображениям обозначается через *GF* (2).

В указанной арифметической системе можно осуществлять известные алгебраические операции. Пример дается следующей системой уравнений над полем *GF* (2):

$$\begin{aligned} X+Y+Z&=1,\\ X+Y\quad\ &=0,\\ X\quad\ +Z&=1. \end{aligned}$$

Эту систему можно решить вычитанием третьего уравнения из первого, что дает $Y=0$. Тогда из второго уравнения получаем $X=0$, а из первого уравнения $Z=1$. Подстановкой получен-

[1]) Абелева группа является частным случаем группы. Арифметическая операция группы общего вида слишком слаба для того, чтобы называться сложением.

ного решения в исходную систему уравнений проверяем, что оно правильно.

Чтобы получить другой способ решения, предположим, что можно доказать возможность применения обычных методов линейной алгебры над полем $GF(2)$. Определитель системы вычисляется следующим образом:

$$D = \det\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix} = 1\cdot\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} - 1\cdot\begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} + 1\cdot\begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} =$$

$$= 1\cdot 1 - 1\cdot 1 - 1\cdot 1 = -1 = 1.$$

Эту систему уравнений можно решить по правилу Крамера:

$$X = D^{-1}\cdot\begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix} = 0, \quad Y = D^{-1}\cdot\begin{bmatrix} 1 & 1 & 1 \\ 1 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix} = 0,$$

$$Z = D^{-1}\cdot\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix} = 1.$$

Вторым примером поля является 6-10-поле. Это поле содержит 16 элементов, которые мы обозначим символами 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F. Таблицы сложения и умножения в этом поле выписаны на рис. 2.1. Отметим, что здесь правила сложения и умножения значительно отличаются от известных правил сложения и умножения вещественных чисел; в то же время эти таблицы обладают внутренней закономерностью и позволяют осуществлять вычитание и деление. Для деления надо взять $x : y = x\cdot(y^{-1})$, где y^{-1} — элемент поля, удовлетворяющий условию $y\cdot y^{-1} = 1$. Просмотр таблицы умножения показывает, что каждый ненулевой элемент имеет обратный, и, следовательно, деление всегда определено, за исключением деления на нуль.

Большинство методов линейной алгебры, так же как и матричные операции, переносится на случай произвольного поля. Именно поэтому поля с конечным числом элементов оказались очень полезными. Мы будем изучать эти поля и найдем способы построения таблиц сложения и умножения, которые порождают поле даже для большого числа элементов. Со временем мы увидим, что поля с q элементами можно построить тогда и только тогда, когда q равно p^m, где p — простое, а m — произвольное положительное целое число. Но прежде мы должны ввести понятия групп и колец.

+	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
1	1	0	3	2	5	4	7	6	9	8	B	A	D	C	F	E
2	2	3	0	1	6	7	4	5	A	B	8	9	E	F	C	D
3	3	2	1	0	7	6	5	4	B	A	9	8	F	E	D	C
4	4	5	6	7	0	1	2	3	C	D	E	F	8	9	A	B
5	5	4	7	6	1	0	3	2	D	C	F	E	9	8	B	A
6	6	7	4	5	2	3	0	1	E	F	C	D	A	B	8	9
7	7	6	5	4	3	2	1	0	F	E	D	C	B	A	9	8
8	8	9	A	B	C	D	E	F	0	1	2	3	4	5	6	7
9	9	8	B	A	D	C	F	E	1	0	3	2	5	4	7	6
A	A	B	8	9	E	F	C	D	2	3	0	1	6	7	4	5
B	B	A	9	8	F	E	D	C	3	2	1	0	7	6	5	4
C	C	D	E	F	8	9	A	B	4	5	6	7	0	1	2	3
D	D	C	F	E	9	8	B	A	5	4	7	6	1	0	3	2
E	E	F	C	D	A	B	8	9	6	7	4	5	2	3	0	1
F	F	E	D	C	B	A	9	8	7	6	5	4	3	2	1	0

а

×	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
2	0	2	4	6	8	A	C	E	3	1	7	5	B	9	F	D
3	0	3	6	5	C	F	A	9	B	8	D	E	7	4	1	2
4	0	4	8	C	3	7	B	F	6	2	E	A	5	1	D	9
5	0	5	A	F	7	2	D	8	E	B	4	1	9	C	3	6
6	0	6	C	A	B	D	7	1	5	3	9	F	E	8	2	4
7	0	7	E	9	F	8	1	6	D	A	3	4	2	5	C	B
8	0	8	3	B	6	E	5	D	C	4	F	7	A	2	9	1
9	0	9	1	8	2	B	3	A	4	D	5	C	6	F	7	E
A	0	A	7	D	E	4	9	3	F	5	8	2	1	B	6	C
B	0	B	5	E	A	1	F	4	7	C	2	9	D	6	8	3
C	0	C	B	7	5	9	E	2	A	6	1	D	F	3	4	8
D	0	D	9	4	1	C	8	5	2	F	B	6	3	E	A	7
E	0	E	F	1	D	3	2	C	9	7	6	8	4	A	B	5
F	0	F	D	2	9	6	4	B	1	E	C	3	8	7	5	A

б

Рис. 2.1. 6-10-поле. *а* — таблица сложения; *б* — таблица умножения.

2.2. ГРУППЫ

Группа — это собирательное название некоторых алгебраических структур. Хотя существуют многие конкретные примеры интересных групп, в математике введено абстрактное понятие группы, так как легче одновременно исследовать все матема-

тические системы с общей структурой, чем исследовать каждую из них по отдельности.

Определение 2.2.1. *Группой* G называется множество элементов с определенной для каждой пары элементов операцией (обозначаемой $*$), обладающее следующими четырьмя свойствами:

1) *замкнутость:* для каждой пары a и b из множества элемент $c = a * b$ принадлежит множеству;

2) *ассоциативность:* для всех a, b и c из множества

$$a * (b * c) = (a * b) * c;$$

3) *существование единицы:* в множестве существует элемент e, называемый *единичным элементом* и такой, что

$$a * e = e * a = a$$

для любого элемента a множества;

4) *существование обратных элементов:* для любого a из множества существует некоторый элемент b из множества, называемый обратным *элементу* a и такой, что

$$a * b = b * a = e.$$

Если группа G содержит конечное число элементов, то она называется *конечной группой*, а число элементов в G называется *порядком* G.

Некоторые группы обладают тем дополнительным свойством, что для любых a и b из группы

$$a * b = b * a.$$

Это свойство называется *коммутативностью*. Группы, обладающие этим дополнительным свойством, называются *коммутативными* или *абелевыми группами*. За исключением некоторого материала этого параграфа, мы всегда будем иметь дело с абелевыми группами.

В случае абелевых групп групповая операция обозначается символом $+$ и называется сложением (даже тогда, когда она не является обычным арифметическим сложением). В этом случае единичный элемент называется нулем и обозначается 0, а обратный элементу a элемент записывается в виде $-a$, так что

$$a + (-a) = (-a) + a = 0.$$

Иногда групповая операция обозначается символом $\cdot$ и называется умножением (даже тогда, когда она не является обычным арифметическим умножением). В этом случае единичный элемент называется единицей и обозначается 1, а обратный элементу a элемент записывается в виде a^{-1}, так что

$$a \cdot a^{-1} = a^{-1} \cdot a = 1.$$

Теорема 2.2.2. *Единичный элемент в каждой группе является единственным. Для каждого элемента группы обратный элемент также является единственным, и* $(a^{-1})^{-1} = a$.

Доказательство. Предположим, что e и e' — единичные элементы группы; тогда $e = e*e' = e'$. Далее, предположим, что b и b' — элементы, обратные элементу a; тогда

$$b = b*(a*b') = (b*a)*b' = b'.$$

Наконец, $a^{-1}a = aa^{-1} = 1$, так что a—обратный элементу a^{-1}. Но в силу единственности обратного элемента $(a^{-1})^{-1} = a$.□

Имеется бесконечно много примеров групп. Многие группы содержат бесконечное число элементов. Примерами являются целые числа относительно сложения, положительные рациональные числа относительно умножения [1]), множество вещественнозначных (2×2)-матриц относительно сложения. Многие другие группы содержат только конечное число элементов. Примерами являются двухэлементное множество $\{0, 1\}$ относительно операции «исключительного или» (сложения по модулю 2), множество $\{0, 1, \ldots, 8, 9\}$ относительно сложения по модулю 10 и т. д.

В качестве более сложного примера построим конечную неабелеву группу, т. е. менее известную структуру. Одним из способов построения групп с интересной алгебраической структурой является исследование преобразований простых геометрических фигур и алгебраическая интерпретация этих преобразований. Например, равносторонний треугольник с вершинами A, B и C (занумерованными по часовой стрелке) можно вращением или отражением относительно оси отобразить на себя точно шестью различными способами, причем каждое из этих вращений и отражений имеет обратное преобразование. Используя некоторые очевидные факты, можно быстро построить алгебраическую группу. Обозначим эти шесть преобразований символами 1, a, b, c, d и e следующим образом:

$1 = (ABC \to ABC)$ (нет изменений),
$a = (ABC \to CAB)$ (вращение против часовой стрелки),
$b = (ABC \to BCA)$ (вращение по часовой стрелке),
$c = (ABC \to ACB)$ (отражение относительно биссектрисы угла A)
$d = (ABC \to CBA)$ (отражение относительно биссектрисы угла B),
$e = (ABC \to BAC)$ (отражение относительно биссектрисы угла C),

[1]) Этот пример дает удобный повод предостеречь относительно терминологии. В случае произвольной абелевой группы групповая операция обычно называется сложением, но не обязательно является обычным сложением. В данном примере она является обычным умножением.

где преобразование $ABC \to BCA$ означает, что вершина A переходит в вершину B, вершина B переходит в вершину C, а вершина C переходит в вершину A. Таким образом, треугольник поворачивается на 120°. Пусть группа $(G, *)$ определяется множеством

$$G = \{1, a, b, c, d, e\}$$

и $y*x$ является элементом группы, обозначающим преобразование, которое получается последовательным выполнением сначала преобразования x, а затем преобразования y; например,

$$a*d = (ABC \to BCA)*(ABC \to CBA) = (ABC \to BAC) = e.$$

Поступая таким образом, можно построить таблицу для $x*y$:

x \ y	1	a	b	c	d	e
1	1	a	b	c	d	e
a	a	b	1	d	e	c
b	b	1	a	e	c	d
c	c	e	d	1	b	a
d	d	c	e	a	1	b
e	e	d	c	b	a	1

Поскольку таблица построена, можно забыть о ее геометрическом происхождении. Таблица сама определяет группу. Подчеркнем, что это пример неабелевой группы, так как $a*c \neq c*a$. Заметим также, что каждый элемент появляется один раз в каждом столбце и в каждой строке. Для конечных групп это выполняется всегда.

Нашим последним примером группы является группа перестановок n букв. Пусть X представляет собой множество $\{1, 2, \ldots, n\}$. Взаимно-однозначное отображение этого множества на самого себя называется перестановкой. Всего имеется $n!$ таких перестановок, и можно определить группу, называемую симметрической группой и обозначаемую через S_n, элементами которой являются перестановки на множестве X. (Сначала может несколько смущать то обстоятельство, что элементами группы являются операторы — операторы перестановок на множестве X. На самом деле в примере преобразований равностороннего треугольника речь также идет о группе перестановок.) Если взять перестановку на выбранных целых числах и переставить их еще раз, то получится другая перестановка на этих целых числах. Выберем в качестве групповой операции $*$ такую композицию

перестановок и возьмем, например, $n = 4$. Всего имеется $4! = 24$ перестановок в группе S_4. Типичный элемент группы S_4 равен

$$a = [(1\ 2\ 3\ 4) \rightarrow (3\ 1\ 4\ 2)]$$

и является перестановкой, заменяющей 1 на 3, 2 на 1, 3 на 4 и 4 на 2. Другой такой перестановкой является

$$b = [(1\ 2\ 3\ 4) \rightarrow (4\ 1\ 3\ 2)].$$

Тогда произведение $b * a$ в группе S_4 равно перестановке, получающейся в результате применения сначала a, а затем b:

$$b * a = [(1\ 2\ 3\ 4) \rightarrow (2\ 3\ 4\ 1)],$$

что является элементом группы S_4. С таким определением умножения группа перестановок S_4 является неабелевой группой, содержащей 24 элемента.

Пусть G — группа, и пусть H — некоторое подмножество в G. Тогда H называется *подгруппой* группы G, если оно является группой относительно ограничения операции $*$ на H. Для того чтобы проверить, что непустое множество H является подгруппой группы G, необходимо только проверить, что для всех a и b из H элемент $a * b$ принадлежит H (замкнутость) и что элемент, обратный к a из H, также принадлежит H. Остальные групповые свойства наследуются из группы G. Как вскоре мы увидим при рассмотрении циклических подгрупп, в случае конечных групп из свойства замкнутости автоматически вытекает даже свойство существования обратного элемента.

Например, множество всех четных чисел и множество чисел, кратных 3, являются подгруппами в множестве всех целых чисел (положительных, отрицательных и нуля) относительно операции сложения.

Один из путей построения подгруппы H конечной группы G состоит в выборе произвольного элемента h группы H и формировании H как множества элементов, образованных умножением h на самого себя произвольное число раз. Таким образом, строим последовательность элементов

$$h,\ h * h,\ h * h * h,\ h * h * h * h,\ \ldots,$$

обозначая их для простоты через h, h^2, h^3, h^4, ... Так как G конечна, то только конечное число этих элементов различно, так что с некоторого момента последовательность начнет повторяться. Первым повторяющимся элементом должен быть сам элемент h, так как если два различных элемента h^i и h^j равны, то их можно умножить на элемент, обратный h, и получить, что h^{i-1} и h^{j-1} также равны. Далее заметим, что если $h^j = h$, то $h^{j-1} = 1$, единичному элементу группы. Множество H называется подгруппой, порожденной элементом h. Число c элементов в H называется

порядком элемента h. Множество элементов h, h^2, h^3, ..., $h^c = 1$ называется *циклом*. Цикл является подгруппой, так как произведение двух элементов такого вида снова является элементом этого вида, а элемент, обратный элементу h^i, равен h^{c-i} и, следовательно, является одним из элементов цикла. Группа, состоящая из всех степеней одного из ее элементов, называется *циклической группой*.

Для заданных конечной группы G и подгруппы H существует важная операция, которая устанавливает некоторые взаимосвязи между G и H и называется *разложением группы G на смежные классы по H*. Обозначим через h_1, h_2, h_3, ... элементы из H, причем через h_1 обозначим единичный элемент. Построим таблицу следующим образом. Первая строка состоит из элементов подгруппы H, причем первым слева выписан единичный элемент h_1 и каждый элемент из H записан в строке один и только один раз. Выберем произвольный элемент группы G, не содержащийся в первой строке. Назовем его g_2 и используем в качестве первого элемента второй строки. Остальные элементы второй строки получаются умножением слева элементов подгруппы на этот первый элемент. Аналогично строим третью, четвертую и пятую строки: каждый раз в качестве элемента первого столбца выбираем не использованный на предыдущих шагах элемент группы G. Построение заканчивается тогда, когда после некоторого шага оказывается, что каждый элемент группы записан в некотором месте таблицы. Процесс обрывается в силу конечности G. В результате получается следующая таблица:

$$
\begin{array}{llllll}
 & h_1 = 1 & h_2 & h_3 & \dots & h_n \\
g_2 * & h_1 = g_2 & g_2 * h_2 & g_2 * h_3 & \dots & g_2 * h_n \\
g_3 * & h_1 = g_3 & g_3 * h_2 & g_3 * h_3 & \dots & g_3 * h_n \\
\cdot & & \cdot & \cdot & \cdot & \\
\cdot & & \cdot & \cdot & \cdot & \\
\cdot & & \cdot & \cdot & \cdot & \\
g_m * & h_1 = g_m & g_m * h_2 & g_m * h_3 & \dots & g_m * h_n
\end{array}
$$

Первый элемент слева в каждой строке называется *лидером смежного класса*. Каждая строка таблицы называется *левым смежным классом*, а в случае абелевой группы — просто *смежным классом*. Если при построении разложения группы на смежные классы использовать правое умножение на элементы группы G вместо левого, то строки называются *правыми смежными классами*. В силу указанных выше правил построения разложение на смежные классы всегда представляется прямоугольной таблицей, все строки которой полностью заполнены. Докажем теперь, что всегда получается таблица, в которой каждый элемент группы встречается точно один раз.

Теорема 2.2.3. *В разложении группы G на смежные классы каждый элемент из G встречается один и только один раз.*

Доказательство. Каждый элемент появится хотя бы один раз, так как в противном случае процесс не остановится. Докажем теперь, что каждый элемент не может появиться дважды в одной и той же строке и что один и тот же элемент не может появиться в двух разных строках.

Предположим, что два элемента одной и той же строки, $g_i * h_j$ и $g_i * h_k$, равны. Тогда умножение [слева. — *Перев.*] каждого из них на g_i^{-1} дает равенство $h_j = h_k$. Это противоречит тому, что каждый элемент подгруппы выписан в первой строке только один раз.

Предположим, что два элемента различных строк $g_i * h_j$ и $g_k * h_l$ равны и что $k < i$.[1] Умножение справа на h_j^{-1} приводит к равенству $g_i = g_k * h_l * h_j^{-1}$. Тогда g_i порождает k-й смежный класс, так как элемент $h_l * h_j^{-1}$ принадлежит подгруппе. Это противоречит указанному выше правилу выбора лидеров смежных классов. □

Следствие 2.2.4. *Если H — подгруппа группы G, то число элементов в H делит число элементов в G. Таким образом,*

(Порядок H)·(Число смежных классов G по H) = = (Порядок G). □

Доказательство следует непосредственно из прямоугольности таблицы разложения на смежные классы.

Теорема 2.2.5. *Порядок конечной группы делится на порядок любого из ее элементов.*

Доказательство. Группа содержит циклическую подгруппу, порожденную любым из ее элементов; таким образом, утверждение теоремы вытекает из следствия 2.2.4. □

2.3. КОЛЬЦА

Следующей необходимой нам алгебраической структурой является кольцо. Кольцо представляет собой абстрактное множество, которое является абелевой группой и наделено дополнительной структурой.

Определение 2.3.1. *Кольцом* R называется множество с двумя определенными на нем операциями: первая называется *сложением* (обозначается $+$), вторая называется *умножением* (обозначается соседним расположением), причем имеют место следующие аксиомы:

1) относительно сложения $(+)$ R является абелевой группой;

2) *замкнутость:* произведение ab принадлежит R для любых a и b из R;

3) *закон ассоциативности:*
$a(bc) = (ab)c$;
4) *закон дистрибутивности:*
$a(b + c) = ab + ac$, $(b + c)a = ba + ca$.

Сложение в кольце всегда коммутативно, а умножение не обязательно должно быть коммутативным. *Коммутативное кольцо* — это кольцо, в котором умножение коммутативно, т. е. $ab = ba$ для всех a и b из R.

Закон дистрибутивности в определении кольца связывает операции сложения и умножения. Этот закон имеет несколько непосредственных следствий, как, например, приведенная ниже теорема.

Теорема 2.3.2. *Для произвольных элементов a и b в кольце R*
(i) $a0 = 0a = 0$,
(ii) $a(-b) = (-a)b = -(ab)$.
Доказательство.

(i). $a0 = a(0 + 0) = a0 + a0$.
Вычитая из обеих частей равенства $a0$, получаем $0 = a0$.
Вторая часть утверждения (i) доказывается аналогично.
(ii). $0 = a0 = a(b - b) = ab + a(-b)$.
Следовательно, $a(-b) = -(ab)$.
Вторая часть утверждения (ii) доказывается аналогично. □

Операция сложения в кольце имеет единичный элемент, называемый нулем. Операция умножения не обязательно имеет единичный элемент, но если он есть, то является единственным. Кольцо, обладающее единственным элементом относительно умножения, называется *кольцом с единицей*. Этот единичный элемент называется единицей и обозначается символом 1. Тогда для всех a из R имеет место равенство

$$1a = a1 = a.$$

Относительно операции сложения каждый элемент кольца имеет обратный. Относительно операции умножения элемент, обратный данному элементу, не обязательно существует, но в кольце с единицей обратные элементы могут существовать. Это означает, что для данного элемента a может существовать элемент b, такой, что $ab = 1$. Если это так, то b называется *правым обратным к a*. Аналогично если существует элемент c, такой, что $ca = 1$, то c называется *левым обратным к a*.

Теорема 2.3.3. *В кольце с единицей*
(i) *единица единственна;*
(ii) *если элемент a имеет как правый обратный b, так и левый обратный c, то элемент a называется обратимым,*

причем обратный ему элемент является единственным (и обозначается через a^{-1});

(iii) $(a^{-1})^{-1} = a$.

Доказательство. Рассуждения аналогичны проведенным при доказательстве теоремы 2.2.2. □

Обратимый элемент кольца называется *единицей*. Множество всех единиц в кольце замкнуто относительно умножения, так как если a и b — единицы, то $c = ab$ имеет обратный элемент, равный $c^{-1} = b^{-1}a^{-1}$.

Теорема 2.3.4.

(i) *Множество единиц кольца образует группу относительно умножения в кольце.*

(ii) *Если* $c = ab$ *и* c *— единица, то* a *имеет правый обратный,* a b *— левый обратный элемент.*

Доказательство. Непосредственная проверка. □

Имеется много известных примеров колец, и ниже приводятся некоторые из них. Представляется поучительным проиллюстрировать этими примерами теоремы 2.3.3 и 2.3.4.

1. Множество всех вещественных чисел образует коммутативное кольцо с единицей относительно обычных сложения и умножения. Каждый ненулевой элемент кольца является единицей.

2. Множество всех целых чисел (положительных, отрицательных и нуля) образует коммутативное кольцо с единицей относительно обычных сложения и умножения. Это кольцо принято обозначать через Z; его единицами являются только ± 1.

3. Множество всех квадратных $(n \times n)$-матриц, элементами которых являются вещественные числа, образует некоммутативное кольцо с единицей относительно матричного сложения и умножения. Единицей является единичная $(n \times n)$-матрица. Единицами в кольце служат все невырожденные матрицы.

4. Множество всех квадратных $(n \times n)$-матриц, элементами которых являются целые числа, образует некоммутативное кольцо с единицей относительно матричного сложения и умножения.

5. Множество всех многочленов от x с вещественными коэффициентами образует коммутативное кольцо с единицей относительно сложения и умножения многочленов. Единицей кольца является многочлен нулевой степени $p(x) = 1$.

2.4. ПОЛЯ

Нестрого говоря, абелевой группой является множество, в котором можно складывать и вычитать, а кольцом — множество, в котором можно складывать, вычитать и умножать. Более силь-

ной алгебраической структурой, называемой полем, является множество, в котором можно складывать, вычитать, умножать и делить.

Определение 2.4.1. *Полем* называется множество с двумя определенными на нем операциями — *сложением* и *умножением*, причем имеют место следующие аксиомы:

1) множество образует абелеву группу по сложению;

2) поле замкнуто относительно умножения, и множество ненулевых элементов образует абелеву группу по умножению;

3) закон дистрибутивности:

$$(a + b)\,c = ac + bc \text{ для любых } a,\ b,\ c \text{ из поля.}$$

Единичный элемент относительно сложения принято обозначать через 0 и называть нулем, аддитивный обратный элементу a элемент — через $-a$; единичный элемент относительно умножения обозначать через 1 и называть единицей, мультипликативный обратный к элементу a элемент — через a^{-1}. Под вычитанием $(a - b)$ понимается $a + (-b)$; под делением (a/b) понимается $b^{-1}a$.

Широко известны следующие примеры полей:

1) $\mathbb{R}$: множество вещественных чисел,
2) $\mathbb{C}$: множество комплексных чисел,
3) $\mathbb{Q}$: множество рациональных чисел.

Все эти поля содержат бесконечное множество элементов. Мы интересуемся полями, содержащими конечное число элементов. Поле с q элементами, если оно существует, называется *конечным полем* или *полем Галуа* и обозначается через $GF(q)$.

Что представляет собой наименьшее поле? Оно обязательно содержит нулевой элемент и единичный элемент. На самом деле этого уже достаточно при следующих таблицах сложения и умножения:

+	0	1
0	0	1
1	1	0

•	0	1
0	0	0
1	0	1

Это поле $GF(2)$, с которым мы уже встречались в § 2.1. Проверка показывает, что не существует другого поля с двумя элементами.

В гл. 4 конечные поля будут изучены более детально. Сейчас мы приведем два простых примера и опишем их таблицами сложения и умножения (вычитание и деление неявно определяются этими же таблицами).

Поле $GF(3) = \{0, 1, 2\}$ с операциями

+	0	1	2
0	0	1	2
1	1	2	0
2	2	0	1

·	0	1	2
0	0	0	0
1	0	1	2
2	0	2	1

Поле $GF(4) = \{0, 1, 2, 3\}$ с операциями

+	0	1	2	3
0	0	1	2	3
1	1	0	3	2
2	2	3	0	1
3	3	2	1	0

·	0	1	2	3
0	0	0	0	0
1	0	1	2	3
2	0	2	3	1
3	0	3	1	2

Отметим, что умножение в поле $GF(4)$ не является умножением по модулю 4 и сложение не является сложением по модулю 4.

Существуют многие другие поля Галуа. Даже для этих примеров очень маленьких полей не так легко с помощью простой проверки установить, что они обладают указанной структурой. Структура этих и бо́льших полей будет разъясняться в гл. 4.

Прежде чем расстаться с этими примерами, заметим, что поле $GF(2)$ содержится в $GF(4)$, так как в поле $GF(4)$ два элемента 0 и 1 складываются и умножаются точно так же, как они складываются и умножаются в поле $GF(2)$. Однако $GF(2)$ не содержится в $GF(3)$.

Определение 2.4.2. Пусть F — некоторое поле. Подмножество в F называется *подполем*, если оно само является полем относительно наследуемых из F операций сложения и умножения. В этом случае исходное поле F называется *расширением поля*.

Для того чтобы доказать, что подмножество конечного поля является подполем, необходимо доказать только, что оно содержит ненулевой элемент и что оно замкнуто относительно сложения и умножения. Все остальные необходимые свойства наследуются из F. Обратные элементу β по сложению или умножению элементы содержатся в порожденной β циклической группе относительно операции сложения или умножения.

Поле обладает всеми свойствами кольца, а также важным дополнительным свойством — в нем всегда возможно сокращение.

Сокращение представляет собой слабую форму деления и означает, что если $ab = ac$, то $b = c$.

Теорема 2.4.3. *Если в произвольном поле* $ab = ac$ *и* $a \neq 0$, *то* $b = c$.

Доказательство. Умножить на a^{-1}. □

Некоторые кольца могут также удовлетворять этому условию сокращения, но все-таки не быть полями. Простым примером служит кольцо целых чисел. В этом кольце сокращение возможно, но приведенное для теоремы 2.4.3 доказательство не проходит, так как в этом кольце не существует элемента a^{-1}. Кольца, в которых всегда возможно сокращение, имеют специальное название.

Определение 2.4.4. Коммутативное кольцо, в котором $b = c$, если $ab = ac$ и элемент a отличен от нуля, называется *областью целостности*.

2.5. ВЕКТОРНЫЕ ПРОСТРАНСТВА

Известный пример векторного пространства дает трехмерное евклидово пространство, фигурирующее во многих физических задачах. Его обобщением является n-мерное векторное пространство над полем вещественных чисел. Понятие n-мерного пространства тесно связано с идеями линейной алгебры и теории матриц и играет важную роль во многих приложениях.

Для произвольного поля можно дать абстрактное определение векторных пространств.

Определение 2.5.1. Пусть F — некоторое поле. Назовем элементы из F *скалярами*. Множество V называется *векторным пространством*, и его элементы называются *векторами*, если для пар элементов из V определена такая операция *векторного сложения* (обозначается плюсом), а для элементов из V и элементов из F определена такая операция *умножения на скаляры* (обозначается приписыванием), что результат выполнения операции дает элемент из V, причем имеют место следующие аксиомы:

1) V является абелевой группой относительно векторного сложения;

2) *закон дистрибутивности*: для каждой пары векторов $\mathbf{v}_1$, $\mathbf{v}_2$ и скаляра c выполняется равенство

$$c(\mathbf{v}_1 + \mathbf{v}_2) = c\mathbf{v}_1 + c\mathbf{v}_2;$$

3) *закон дистрибутивности*: для произвольного вектора $\mathbf{v}$ и произвольных скаляров c_1 и c_2 выполняются равенства $1\mathbf{v} = \mathbf{v}$ и $(c_1 + c_2)\mathbf{v} = c\mathbf{v}_1 + c\mathbf{v}_2$;

4) *закон ассоциативности*: для произвольного вектора $\mathbf{v}$ и произвольных скаляров c_1 и c_2 выполняется равенство

$$(c_1c_2)\,\mathbf{v} = c_1\,(c_2\mathbf{v}).$$

Нулевой элемент из V называется *началом координат* пространства V и обозначается через $\mathbf{0}$.

Отметим, что мы использовали символ $+$ двумя различными способами: для векторного сложения и для сложения в поле. Отметим также, что мы использовали символ 0 для обозначения нулевого элемента поля и символ $\mathbf{0}$ для обозначения начала координат векторного пространства.

В качестве менее известного примера векторного пространства V можно указать множество многочленов от x с коэффициентами из $GF(q)$. Векторами этого пространства служат многочлены. Векторное сложение совпадает со сложением многочленов, а умножение на скаляр — с умножением многочленов на элементы поля.

В векторном пространстве V сумма вида

$$\mathbf{u} = a_1\mathbf{v}_1 + a_2\mathbf{v}_2 + \cdots + a_k\mathbf{v}_k,$$

где a_i — скаляры, называется *линейной комбинацией* векторов $\mathbf{v}_1, \ldots, \mathbf{v}_k$. Множество векторов $\mathbf{v}_1, \ldots, \mathbf{v}_k$ называется *линейно зависимым*, если существует множество не всех равных нулю скаляров $\{a_1, \ldots, a_k\}$, такое, что

$$a_1\mathbf{v}_1 + a_2\mathbf{v}_2 + \cdots + a_k\mathbf{v}_k = \mathbf{0}.$$

Множество векторов, которое не является линейно зависимым, называется *линейно независимым*. Никакой вектор из множества линейно независимых векторов не может быть представлен в виде линейной комбинации остальных векторов этого множества. Отметим, что нулевой вектор $\mathbf{0}$ не может принадлежать линейно независимому множеству; каждое множество, содержащее $\mathbf{0}$, является линейно зависимым.

О множестве векторов говорят, что оно *порождает* векторное пространство, если каждый вектор пространства равен хотя бы одной линейной комбинации векторов из этого множества. Векторное пространство, порождаемое конечным множеством векторов, называется *конечномерным векторным пространством*. Мы в первую очередь интересуемся конечномерными векторными пространствами.

Теорема 2.5.2. *Если векторное пространство V порождено конечным множеством из k векторов $A = \{\mathbf{v}_1, \ldots, \mathbf{v}_k\}$ и V содержит m линейно независимых векторов $B = \{\mathbf{u}_1, \ldots, \mathbf{u}_m\}$, то $k \geqslant m$.*

Доказательство. Мы опишем правило построения последовательности множеств A_0, A_1, A_2, ..., A_m, таких, что каждое из множеств порождает V, каждое из множеств содержит k элементов, выбираемых из A и B, и множество A_r содержит $\mathbf{u}_1$, ..., $\mathbf{u}_r$. Таким образом, среди k элементов множества A_m будут содержаться $\mathbf{u}_1$, ..., $\mathbf{u}_m$, и, следовательно, $k \geqslant m$.

Так как никакая линейная комбинация векторов из B с ненулевыми коэффициентами не равна нулю, то никакой элемент из B не может быть представлен в виде линейной комбинации других элементов из B. Если множество A_{r-1} не содержит $\mathbf{u}_r$ и порождает V, то должен быть способ представления $\mathbf{u}_r$ в виде линейной комбинации элементов из A_{r-1}, включающий хотя бы один вектор из A (скажем, $\mathbf{v}_j$), не принадлежащий множеству B. Уравнение, задающее эту линейную комбинацию, можно разрешить относительно $\mathbf{v}_j$, представив $\mathbf{v}_j$ в виде линейной комбинации из $\mathbf{u}_r$ и других элементов из A_{r-1}.

Это построение осуществляется следующим образом. Пусть $A_0 = A$. Если A_{r-1} содержит $\mathbf{u}_r$, то полагаем $A_r = A_{r-1}$; в противном случае $\mathbf{u}_r$ не принадлежит множеству A_{r-1}, но может быть представлен в виде линейной комбинации элементов из A_{r-1}, содержащей некоторый элемент $\mathbf{v}_j$ из A, не принадлежащий B. Множество A_r образуем из множества A_{r-1} заменой $\mathbf{v}_j$ на $\mathbf{u}_r$.

Произвольный вектор $\mathbf{v}$ равен некоторой линейной комбинации элементов из A_{r-1} и, следовательно, также элементов из A_r, если исключить вектор $\mathbf{v}_j$, используя линейную комбинацию, связывающую $\mathbf{v}_j$ и $\mathbf{u}_r$ с другими векторами из A_{r-1}. Следовательно, множество A_r порождает V, и из A_{r-1} мы построили A_r с желаемыми свойствами. Таким образом, множество A_m может быть построено, и доказательство закончено. □

Теорема 2.5.3. *Два множества линейно независимых векторов, порождающие одно и то же векторное пространство, содержат одинаковое количество векторов.*

Доказательство. Если одно множество содержит m векторов а другое k векторов, то по теореме 2.5.2 $m \geqslant k$ и $k \geqslant m$, и, следовательно, $m = k$. □

Число линейно независимых векторов в множестве, порождающем конечномерное векторное пространство V, называется *размерностью* пространства V. Множество k линейно независимых векторов, порождающее k-мерное векторное пространство, называется *базисом* этого пространства. Согласно теореме 2.5.2, в k-мерном векторном пространстве каждое множество, содержащее более k векторов, является линейно зависимым.

Теорема 2.5.4. *В k-мерном векторном пространстве V любые k линейно независимых векторов образуют базис пространства V.*

Доказательство. Пусть $\{\mathbf{v}_1, \ldots, \mathbf{v}_k\}$ — произвольное множество k линейно независимых векторов из V. Если оно не порождает V, то в V найдется такой вектор $\mathbf{v}$, что он не равен никакой линейной комбинации векторов $\mathbf{v}_1, \ldots, \mathbf{v}_k$. Множество $\{\mathbf{v}, \mathbf{v}_1, \ldots, \mathbf{v}_k\}$ содержит $k + 1$ линейно независимых векторов из V, что противоречит теореме 2.5.3. Следовательно, множество $\{\mathbf{v}_1, \ldots, \mathbf{v}_k\}$ порождает V и является базисом. □

Если множество линейно независимых векторов k-мерного пространства не является базисом, то оно должно содержать меньше k векторов. Дополнение такого множества векторами так, чтобы оно превратилось в базис, называется *пополнением* базиса.

Теорема 2.5.5. *Заданное множество линейно независимых векторов в конечномерном векторном пространстве всегда может быть дополнено до множества, образующего базис.*

Доказательство. Если данное множество не является базисом, то некоторый вектор пространства не является линейной комбинацией векторов данного множества. Выберем такой произвольный вектор и присоединим его к исходному множеству, увеличив объем множества на единицу. Если полученное множество все еще не является базисом, повторим процесс. Процесс обязательно оборвется, так как число линейно независимых векторов в множестве не может превосходить размерность пространства. Полученное по завершении процесса множество векторов удовлетворяет условиям теоремы. □

Непустое подмножество векторного пространства называется *векторным подпространством*, если оно также является векторным пространством относительно исходных операций векторного сложения и умножения на скаляр. Относительно операции векторного сложения векторное пространство является группой, а векторное подпространство подгруппой. Чтобы установить, что непустое подмножество векторного пространства образует подпространство, достаточно проверить только замкнутость подмножества относительно векторного сложения и умножения на скаляры. Замкнутость относительно умножения на скаляры гарантирует, что нулевой вектор принадлежит подмножеству; другие необходимые свойства наследуются из исходного пространства.

Теорема 2.5.6. *Множество всех линейных комбинаций множества векторов* $\{\mathbf{v}_1, \ldots, \mathbf{v}_k\}$ *произвольного векторного пространства* V *образует подпространство в* V.

Доказательство. Каждая линейная комбинация векторов $\mathbf{v}_1, \ldots, \mathbf{v}_k$ является вектором из V, и поэтому множество W всех линейных комбинаций образует подмножество пространства V. Оно не пусто, так как $\mathbf{0}$ принадлежит W. Мы должны показать, что

W является подпространством. Если $\mathbf{w} = b_1\mathbf{v}_1 + \ldots + b_k\mathbf{v}_k$ и $\mathbf{u} = c_1\mathbf{v}_1 + \ldots + c_k\mathbf{v}_k$ — два произвольных элемента подмножества W, то $\mathbf{w} + \mathbf{u} = (b_1 + c_1)\,\mathbf{v}_1 + \ldots + (b_k + c_k)\,\mathbf{v}_k$ также принадлежит W. Далее, для произвольного вектора $\mathbf{w}$ скалярное кратное вектора $\mathbf{w}$, $a\mathbf{w} = ab_1\mathbf{v}_1 + \ldots + ab_k\mathbf{v}_k$, также принадлежит W. Так как W замкнуто относительно векторного сложения и умножения на скаляр, то оно является векторным подпространством. □

Теорема 2.5.7. *Если размерность векторного подпространства W конечномерного векторного пространства V равна размерности V, то $W = V$.*

Доказательство. Обозначим размерность обоих пространств через k. Выберем в W базис. Он образует множество k линейно независимых векторов пространства V, поэтому является базисом в V. Следовательно, каждый вектор из W принадлежит также V. □

Для заданного поля F величина $(a_1, a_2, \ldots, a_k)$, составленная из элементов поля, называется *n-последовательностью* элементов поля. Относительно операций покомпонентного сложения и покомпонентного умножения на скаляры множество n-последовательностей элементов поля F образует векторное пространство, которое обозначается через F^n. С помощью выбора базиса $\mathbf{v}_1, \ldots, \mathbf{v}_k$ любое конечномерное векторное пространство можно превратить в пространство n-последовательностей, представляя каждый вектор $\mathbf{v} = a_1\mathbf{v}_1 + \ldots + a_n\mathbf{v}_n$ n-последовательностью его коэффициентов $(a_1, \ldots, a_n)$. Следовательно, мы можем ограничиться рассмотрением только векторных пространств n-последовательностей.

Скалярное произведение двух n-последовательностей из F^n:

$$\mathbf{u} = (a_1, \ldots, a_n) \text{ и } \mathbf{v} = (b_1, \ldots, b_n)$$

равно скаляру, определяемому так:

$$\mathbf{u}\cdot\mathbf{v} = (a_1, \ldots, a_n)\cdot(b_1, \ldots, b_n) = a_1b_1 + \cdots + a_nb_n.$$

Можно сразу же проверить, что $\mathbf{u}\cdot\mathbf{v} = \mathbf{v}\cdot\mathbf{u}$, $(c\mathbf{u})\cdot\mathbf{v} = c\,(\mathbf{u}\cdot\mathbf{v})$ и $\mathbf{w}\cdot(\mathbf{u} + \mathbf{v}) = (\mathbf{w}\cdot\mathbf{u}) + (\mathbf{w}\cdot\mathbf{v})$. Если скалярное произведение двух векторов равно нулю, то они называются *ортогональными*. Ненулевые векторы над $GF(q)$ могут быть ортогональны сами себе. Вектор, ортогональный к каждому элементу множества, называется ортогональным к множеству.

Теорема 2.5.8. *Пусть V — векторное пространство n-последовательностей над некоторым полем F, и пусть W — некоторое его подпространство. Множество векторов, ортогональных к W, также образует подпространство.*

Доказательство. Обозначим через U множество всех векторов, ортогональных к W. Так как $\mathbf{0}$ принадлежит U, то U не пусто. Пусть $\mathbf{w}$ — произвольный вектор из W, а $\mathbf{u}_1$ и $\mathbf{u}_2$ — два произвольных вектора из U. Тогда $\mathbf{w}\cdot\mathbf{u}_1 = \mathbf{w}\cdot\mathbf{u}_2 = 0$ и $\mathbf{w}\cdot\mathbf{u}_1 + \mathbf{w}\cdot\mathbf{u}_2 = 0 = \mathbf{w}\cdot(\mathbf{u}_1 + \mathbf{u}_2)$, так что $\mathbf{u}_1 + \mathbf{u}_2$ принадлежит U. Также $\mathbf{w}\cdot(c\mathbf{u}_1) = c(\mathbf{w}\cdot\mathbf{u}_1) = 0$, и, следовательно, $c\mathbf{u}_1$ принадлежит U. Таким образом, U является подпространством. □

Множество векторов, ортогональных к W, называется *ортогональным дополнением* W и обозначается через $W^\perp$. В случае конечномерных векторных пространств над полем вещественных чисел пересечение W и $W^\perp$ содержит только нулевой вектор, но над полем $GF(q)$ подпространство $W^\perp$ может иметь нетривиальное пересечение с W или может даже принадлежать W либо содержать W. В действительности можно даже построить примеры подпространств, которые сами являются своими ортогональными дополнениями. Например, в $GF^2(2)$ подпространство $\{00, 11\}$ совпадает со своим ортогональным дополнением.

Теорема 2.5.9. *Вектор, ортогональный к каждому вектору множества, порождающего W, принадлежит ортогональному дополнению пространства W.*

Доказательство. Предположим, что множество $\{\mathbf{w}_1, \ldots, \mathbf{w}_n\}$ порождает W. Вектор $\mathbf{w}$ из W можно записать в виде $\mathbf{w} = c_1\mathbf{w}_1 + \cdots + c_n\mathbf{w}_n$. Тогда

$$\mathbf{w}\cdot\mathbf{u} = (c_1\mathbf{w}_1 + \cdots + c_n\mathbf{w}_n)\cdot\mathbf{u} = c_1\mathbf{w}_1\cdot\mathbf{u} + \cdots + c_n\mathbf{w}_n\cdot\mathbf{u}.$$

Если $\mathbf{u}$ ортогонален к каждому $\mathbf{w}_i$, то он ортогонален к каждому $\mathbf{w}$ из W. □

Если размерность подпространства W в векторном пространстве n-последовательностей равна k, то размерность ортогонального дополнения $W^\perp$ равна $n - k$. Этот факт часто используется в дальнейшем и поэтому будет доказан в виде теоремы 2.6.9. Мы сошлемся на этот факт при доказательстве следующего результата.

Теорема 2.5.10. *Пусть W — подпространство в пространстве n-последовательностей, и пусть $W^\perp$ — его ортогональное дополнение. Тогда W представляет собой ортогональное дополнение подпространства $W^\perp$.*

Доказательство. Пусть размерность W равна k. Тогда, согласно теореме 2.6.9, размерность $W^\perp$ равна $n - k$, а размерность ортогонального дополнения пространства $W^\perp$ равна k. Но каждый вектор из W ортогонален к $W^\perp$. Следовательно, W содержится в ортогональном дополнении к $W^\perp$ и имеет ту же самую размерность, так что эти подпространства совпадают. □

2.6. ЛИНЕЙНАЯ АЛГЕБРА

Широко используемые разделы прикладной математики — линейная алгебра, в частности теория матриц, — обычно изучаются только для поля вещественных чисел и поля комплексных чисел, однако большинство известных операций линейной алгебры справедливо также для произвольного поля. Мы кратко изложим этот материал отчасти в порядке обзора и отчасти для доказательства того, что известные методы остаются справедливыми над произвольным полем (а иногда даже над произвольным кольцом).

Определение 2.6.1. *($n \times m$)-матрицей* $\mathbf{A}$ над кольцом R называется прямоугольная таблица, состоящая из n строк и m столбцов и содержащая nm элементов из кольца R:

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1m} \\ a_{21} & a_{22} & \dots & a_{2m} \\ \cdot & & \cdot & \cdot \\ \cdot & & \cdot & \cdot \\ \cdot & & \cdot & \cdot \\ a_{n1} & a_{n2} & \dots & a_{nm} \end{bmatrix} = [a_{ij}],$$

В большинстве приложений кольцо R в действительности является полем, и мы ограничимся этим случаем. Как правило, мы будем рассматривать матрицы над конечным полем $GF(q)$.

Множество элементов a_{ii}, для которых номер строки совпадает с номером столбца, называется *главной диагональю*. Если n равно m, то матрица называется *квадратной матрицей*. ($n \times n$)-матрица, все элементы главной диагонали которой равны единичному элементу поля, а остальные элементы равны нулевому элементу поля, называется *единичной* ($n \times n$)*-матрицей*. Единичная матрица обозначается через $\mathbf{I}$. Примерами единичных матриц являются

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad \text{и} \quad \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

Две ($n \times m$)-матрицы $\mathbf{A}$ и $\mathbf{B}$ над полем $GF(q)$ можно складывать по правилу

$$\mathbf{A} + \mathbf{B} = \begin{bmatrix} a_{11} + b_{11} & a_{12} + b_{12} & \dots & a_{1m} + b_{1m} \\ \cdot & \cdot & & \cdot \\ \cdot & \cdot & & \cdot \\ a_{n1} + b_{n1} & a_{n2} + b_{n2} & \dots & a_{nm} + b_{nm} \end{bmatrix}.$$

$(n \times m)$-матрицу $\mathbf{A}$ можно умножать на элемент поля β по правилу

$$\beta\mathbf{A} = \begin{bmatrix} \beta a_{11} & \beta a_{12} & \dots & \beta a_{1m} \\ \cdot & \cdot & & \cdot \\ \cdot & \cdot & & \cdot \\ \cdot & \cdot & & \cdot \\ \beta a_{n1} & \beta a_{n2} & \dots & \beta a_{nm} \end{bmatrix}.$$

$(l \times n)$-матрицу $\mathbf{A}$ можно умножить на $(n \times m)$-матрицу $\mathbf{B}$, получив результирующую $(l \times m)$-матрицу $\mathbf{C}$, по правилу

$$c_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}, \quad i = 1, \dots, l, \quad j = 1, \dots, m.$$

Это произведение матриц обозначается через

$$\mathbf{C} = \mathbf{AB}.$$

Как легко проверить, множество квадратных $(n \times n)$-матриц образует кольцо относительно так определенных умножения и сложения матриц. Это кольцо не коммутативно, но обладает единицей, а именно единичной $(n \times n)$-матрицей.

Матрицу можно разбить на блоки по правилу

$$\mathbf{A} = \left[\begin{array}{c|c} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \hline \mathbf{A}_{21} & \mathbf{A}_{22} \end{array}\right]$$

где $\mathbf{A}_{11}$, $\mathbf{A}_{12}$, $\mathbf{A}_{21}$ и $\mathbf{A}_{22}$ — меньшие матрицы, размеры которых очевидным образом дополняют друг друга до размеров матрицы $\mathbf{A}$. А именно сумма числа строк матрицы $\mathbf{A}_{11}$ (или $\mathbf{A}_{12}$) и числа строк матрицы $\mathbf{A}_{21}$ (или $\mathbf{A}_{22}$) равна числу строк матрицы $\mathbf{A}$; аналогичное утверждение выполняется для столбцов. Матрицы можно перемножить поблочно, а именно если

$$\mathbf{A} = \left[\begin{array}{c|c} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \hline \mathbf{A}_{21} & \mathbf{A}_{22} \end{array}\right], \qquad \mathbf{B} = \left[\begin{array}{c|c} \mathbf{B}_{11} & \mathbf{B}_{12} \\ \hline \mathbf{B}_{21} & \mathbf{B}_{22} \end{array}\right]$$

и $\mathbf{C} = \mathbf{AB}$, то при условии корректного выбора размеров блоков (корректного в том смысле, что все произведения и суммы матриц определены)

$$\mathbf{C} = \left[\begin{array}{c|c} \mathbf{A}_{11}\mathbf{B}_{11} + \mathbf{A}_{12}\mathbf{B}_{21} & \mathbf{A}_{11}\mathbf{B}_{12} + \mathbf{A}_{12}\mathbf{B}_{22} \\ \hline \mathbf{A}_{21}\mathbf{B}_{11} + \mathbf{A}_{22}\mathbf{B}_{21} & \mathbf{A}_{21}\mathbf{B}_{12} + \mathbf{A}_{22}\mathbf{B}_{22} \end{array}\right].$$

Такое разложение можно получить как простое следствие аксиом ассоциативности и дистрибутивности основного поля.

Транспонированной к $(n \times m)$-матрице $\mathbf{A}$ называется $(m \times n)$-матрица $\mathbf{A}^T$, такая, что $a^T_{ij} = a_{ji}$. Таким образом, строками матрицы $\mathbf{A}^T$ служат столбцы матрицы $\mathbf{A}$, а столбцами матрицы $\mathbf{A}^T$ служат строки матрицы $\mathbf{A}$. *Обратной* к квадратной матрице $\mathbf{A}$ называется квадратная матрица $\mathbf{A}^{-1}$ (если она существует), такая что $\mathbf{A}^{-1}\mathbf{A} = \mathbf{A}\mathbf{A}^{-1} = \mathbf{I}$. Как можно сразу проверить, множество всех обратимых $(n \times n)$-матриц образует группу относительно операции умножения. Следовательно, если матрица имеет обратную, то обратная матрица единственна, так как в силу теоремы 2.2.2 это свойство выполняется в каждой группе. Матрица, имеющая обратную, называется *невырожденной;* в противном случае она называется *вырожденной*. Если $\mathbf{C} = \mathbf{AB}$, то при условии, что $\mathbf{A}$ и $\mathbf{B}$ обратимы, $\mathbf{C}^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1}$, так как $(\mathbf{B}^{-1}\mathbf{A}^{-1})\,\mathbf{C} = \mathbf{I} = \mathbf{C}\,(\mathbf{B}^{-1}\mathbf{A}^{-1})$. Впоследствии мы увидим, что если у $\mathbf{A}$ или у $\mathbf{B}$ нет обратной матрицы, то и у $\mathbf{C}$ нет обратной матрицы.

Определение 2.6.2. Пусть задано поле F. *Определитель* квадратной $(n \times n)$-матрицы $\mathbf{A}$ для каждого n является функцией на множестве всех $(n \times n)$-матриц над F со значениями в поле F, обозначается через $\det(\mathbf{A})$ и задается формулой

$$\det(\mathbf{A}) = \sum \xi_{i_1 \dots i_n} a_{1i_1} a_{2i_2} \cdots a_{ni_n},$$

где $i_1, i_2, \dots, i_n$ — перестановка на множестве целых чисел $\{1, 2, \dots, n\}$, $\xi_{i_1 \dots i_n}$ равно ± 1 в зависимости от четности или нечетности перестановки, а суммирование ведется по всем перестановкам.

Нечетная перестановка определяется как произведение нечетного числа транспозиций (транспозицией называется перестановка двух членов). Четная перестановка определяется как перестановка, которая не может быть получена нечетным числом транспозиций.

Один из способов сделать это определение наглядным состоит в рассмотрении множеств всех матриц, которые можно получить из матрицы $\mathbf{A}$ перестановкой строк. Для каждой из таких матриц возьмем произведение всех членов, лежащих на главной диагонали (если перестановка была нечетной, то изменим знак произведения), и сложим все полученные таким образом произведения. Конечно, вычислять таким образом определитель не следует, но это дает хороший способ установления свойств определителей.

В приведенной ниже теореме перечислены свойства функции $\det(\mathbf{A})$, вытекающие непосредственно из ее определения.

Теорема 2.6.3.

(i) *Если все элементы некоторой строки квадратной матрицы равны нулю, то определитель этой матрицы равен нулю.*

(ii) *Определитель матрицы равен определителю транспонированной матрицы.*

(iii) *Если две строки матрицы поменять местами, то ее определитель изменит знак.*

(iv) *Если две строки матрицы равны, то ее определитель равен нулю.*

(v) *Если все элементы одной строки матрицы умножить на элемент поля c, то определитель новой матрицы будет равен определителю исходной матрицы, умноженной на c.*

(vi) *Если матрицы* **A** *и* **B** *отличаются только i-й строкой, то сумма их определителей равна определителю матрицы* **C**, *i-я строка которой равна сумме i-х строк матриц* **A** *и* **B**, *а остальные строки равны соответствующим строкам матрицы* **A** *или* **B**.

(vii) *Если к элементам некоторой строки матрицы k раз прибавить соответствующие элементы некоторой другой ее строки, то определитель матрицы не изменится.* Доказательство: *использовать свойства* (iv), (v) *и* (vi).

(viii) *Определитель матрицы отличен от нуля тогда и только тогда, когда ее строки (столбцы) линейно независимы.*

Доказательство предоставляется читателю в качестве упражнения. *Замечание*: в общем случае утверждение (iv) не может быть доказано с помощью перестановки двух строк и применения свойства (iii). Почему? □

Если в квадратной матрице удалить строку и столбец, содержащие элемент a_{ij}, то определитель оставшейся квадратной таблицы размера $n - 1$ называется *минором* элемента a_{ij} и обозначается через M_{ij}. *Алгебраическое дополнение*, обозначаемое здесь через C_{ij}, определяется равенством

$$C_{ij} = (-1)^{i+j} M_{ij}.$$

Из способа задания определителя матрицы следует, что алгебраическое дополнение элемента a_{ij} является коэффициентом при a_{ij} в разложении определителя:

$$\det(\mathbf{A}) = \sum_{k=1}^{n} a_{ik} C_{ik}.$$

Это известная формула Лапласа для разложения определителей. Она дает выражение определителя $(n \times n)$-матрицы через определители $(n - 1) \times (n - 1)$-матриц. Формула разложения Лапласа лежит в основе рекуррентного способа вычисления определителей.

Если a_{ik} заменить на a_{jk}, то получится сумма $\sum_{k=1}^{n} a_{jk} C_{ik}$, равная определителю новой матрицы, полученной из старой за-

меной элементов i-й строки элементами j-й строки; этот определитель равен нулю, если $j \neq i$. Таким образом,

$$\sum_{k=1}^{n} a_{jk}C_{ik} = \begin{cases} \det(\mathbf{A}), & i = j, \\ 0 & i \neq j. \end{cases}$$

Поэтому если $\det(\mathbf{A}) \neq 0$, то матрица $\mathbf{A} = [a_{ij}]$ имеет обратную, равную

$$\mathbf{A}^{-1} = \left[\frac{C_{ji}}{\det(\mathbf{A})}\right].$$

Если $\det(\mathbf{A}) = 0$, то обратной матрицы не существует.

Строки $(n \times m)$-матрицы $\mathbf{A}$ над $GF(q)$ можно рассматривать как множество m-мерных векторов над $GF(q)$. *Пространство строк* матрицы $\mathbf{A}$ определяется как множество всех линейных комбинаций векторов-строк матрицы $\mathbf{A}$. Размерность пространства строк называется *рангом матрицы по строкам*. Аналогично столбцы матрицы $\mathbf{A}$ можно рассматривать как множество n-мерных векторов над $GF(q)$. *Пространство столбцов* матрицы $\mathbf{A}$ определяется как множество всех линейных комбинаций векторов-столбцов матрицы $\mathbf{A}$, а размерность пространства столбцов называется *рангом матрицы по столбцам*. Множество всех векторов $\mathbf{v}$, таких, что $\mathbf{A}\mathbf{v}^{\mathrm{T}} = 0$, называется *нулевым пространством* матрицы $\mathbf{A}$. Ясно, что нулевое пространство является подпространством в $GF^n(q)$. В частности, нулевое пространство является ортогональным дополнением пространства строк матрицы $\mathbf{A}$, так как нулевое пространство можно задать как множество всех векторов, ортогональных ко всем векторам пространства строк.

Элементарными операциями над строками матрицы называются следующие действия:

1) перестановка двух произвольных строк;
2) умножение произвольной строки на нулевой элемент поля;
3) замена произвольной строки на сумму ее самой и некоторого кратного любой другой строки.

Каждая элементарная операция над строками обратима, и обратная операция имеет такой же вид. Каждая элементарная операция над строками $(n \times n)$-матрицы $\mathbf{A}$ может быть выполнена путем левого умножения $\mathbf{A}$ на соответствующим образом подобранную так называемую *элементарную* $(n \times n)$*-матрицу* $\mathbf{F}$.

Элементарные матрицы определяются как одна из следующих модификаций единичной матрицы:

$$\begin{bmatrix} 1 & & & & \\ & \ddots & & & \\ & & 0 & 1 & \\ & & \ddots & \ddots & \\ & & 1 & 0 & \\ & & & & \ddots \\ & & & & & 1 \end{bmatrix}, \quad \begin{bmatrix} 1 & & & & \\ & \ddots & & & \\ & & \ddots & & \\ & & & a & \\ & & & & \ddots \\ & & & & & 1 \end{bmatrix} \text{ или } \begin{bmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & & \ddots & \\ & & a & 1 & \\ & & & & \ddots \\ & & & & & 1 \end{bmatrix}$$

Элементарные операции над строками используются для приведения матрицы к стандартному виду, называемому *каноническим ступенчатым видом* и определяемому следующим образом:

1) ведущий ненулевой элемент каждой ненулевой строки равен единице;

2) все остальные элементы каждого столбца, содержащего такой ведущий элемент, равны нулю;

3) ведущий элемент любой строки находится правее любого ведущего элемента любой расположенной выше строки. Нулевые строки расположены ниже всех ненулевых строк.

Примером матрицы, приведенной к каноническому ступенчатому виду, является

$$\mathbf{A} = \begin{bmatrix} 1 & 1 & 0 & 1 & 3 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}.$$

Заметим, что нулевая строка расположена снизу и что если удалить последнюю строку, то все столбцы единичной (3×3)-матрицы появятся среди столбцов матрицы, но в разбросанном виде. В общем случае если имеется k ненулевых строк и по меньшей мере такое же количество столбцов, то матрица в каноническом ступенчатом виде всегда будет содержать все столбцы единичной матрицы размера k. Частным случаем канонической ступенчатой формы матрицы является матрица вида

$$\mathbf{A} = [\mathbf{I} \vdots \mathbf{P}],$$

где $\mathbf{I}$ — единичная матрица. С помощью элементарных операций над строками каждая матрица, содержащая по меньшей мере столько же столбцов, сколько и строк, может быть приведена к каноническому ступенчатому виду, но не к указанному выше его частному случаю.

Теорема 2.6.4. *Если матрицы* A *и* A′ *получаются одна из другой с помощью элементарных операций, то пространства строк этих матриц совпадают.*

Доказательство. Каждая строка из A′ является некоторой линейной комбинацией строк матрицы A. Следовательно, каждая линейная комбинация строк матрицы A′ также является линейной комбинацией строк матрицы A, и, таким образом, пространство строк матрицы A содержит пространство строк матрицы A′. Но матрица A получается из матрицы A′ с помощью обратных элементарных операций, и, следовательно, пространство строк матрицы A′ содержится в пространстве строк матрицы A. Таким образом, A и A′ имеют одно и то же пространство строк. □

Теорема 2.6.5. *Если матрицы* A *и* A′ *связаны между собой последовательностью элементарных операций над строками, то любое множество линейно независимых столбцов в* A *линейно независимо и в* A′.

Доказательство. Так как для элементарных операций над строками первого и второго вида теорема очевидна, то достаточно доказать ее только для элементарной операции третьего вида. Итак, предположим, что A′ получается из A прибавлением кратного строки α к строке β. В любой линейно зависимой комбинации столбцов матрицы A′ элементы строки α должны давать нуль и, следовательно, не оказывать никакого воздействия на строку β. Таким образом, рассматриваемое множество столбцов в матрице A также является линейно зависимым.

Теорема 2.6.6. *($k \times n$)-матрица* A, *k строк которой линейно независимы, содержит k линейно независимых столбцов.*

Доказательство. Приведем A к каноническому ступенчатому виду A′. Так как строки линейно независимы, то ни одна из них не является нулевой. Следовательно, для каждой строки существует столбец, элемент которого в этой строке равен единице, а в каждой другой строке равен нулю. Множество из k таких столбцов матрицы A′ линейно независимо, и, следовательно, в силу теоремы 2.6.5 это же множество столбцов линейно независимо в A. □

Теорема 2.6.7. *Ранг матрицы* A *по строкам равен ее рангу по столбцам и равен размеру наибольшей квадратной подматрицы, определитель которой отличен от нуля. (Поэтому данная величина называется просто рангом матрицы.)*

Доказательство. Достаточно показать, что ранг матрицы A по строкам равен размеру наибольшей квадратной подматрицы с ненулевым определителем. То же самое доказательство применительно к транспонированной матрице дает доказательство

утверждения для ранга матрицы по столбцам, и, таким образом, доказывается, что ранг по строкам равен рангу по столбцам.

Подматрица матрицы **A** получается выбрасыванием из **A** некоторого числа строк и столбцов. Пусть **M** — невырожденная квадратная подматрица матрицы **A** наибольшего размера. Согласно теореме 2.6.3 (viii), строки матрицы **M** линейно независимы, и, следовательно, их продолжения до строк матрицы **A** также должны быть линейно независимыми. Следовательно, ранг матрицы **A** по строкам не меньше размера матрицы **M**.

С другой стороны, выберем произвольное множество из k линейно независимых строк матрицы **A**. Согласно теореме 2.6.7, матрица, образованная такими строками, имеет k линейно независимых столбцов. Таким образом, определитель матрицы, составленной из расположенных на пересечении этих k столбцов и этих k строк элементов, будет отличен от нуля. Следовательно, размер наибольшей невырожденной подматрицы не меньше ранга матрицы **A** по строкам. Это завершает доказательство. □

Пусть **A** — квадратная $(n \times n)$-матрица с ненулевым определителем. Тогда, согласно теоремам 2.6.4 и 2.6.7, ее каноническая ступенчатая форма является $(n \times n)$-матрицей, все строки которой отличны от нулевой, и, следовательно, представляет собой единичную матрицу. Так как **A** может быть получена из **I** обращением последовательности элементарных операций над строками, то **A** можно записать через элементарные матрицы:

$$\mathbf{A} = \mathbf{F}_1\mathbf{F}_2 \cdots \mathbf{F}_r.$$

Теорема 2.6.8. *Если в кольце $(n \times n)$-матриц над полем F* $\mathbf{C} = \mathbf{AB}$, *то*

$$\det(\mathbf{C}) = \det(\mathbf{A}) \det(\mathbf{B}).$$

Доказательство. *Шаг 1.* Сначала покажем, что если $\det(\mathbf{A})$ или $\det(\mathbf{B})$ равны нулю, то и $\det(\mathbf{C})$ равен нулю. Предположим, что $\det(\mathbf{B})$ равен нулю; тогда по теореме 2.6.3 (viii) строки матрицы **B** линейно зависимы. Но строки матрицы **C** являются линейными комбинациями строк матрицы **B**. Следовательно, строки матрицы **C** линейно зависимы и $\det \mathbf{C}$ равен нулю. Аналогично исследуется случай, когда $\det(\mathbf{A})$ равен нулю.

Шаг 2. Предположим, что $\det(\mathbf{A})$ не равен нулю. Тогда матрицу **A** можно записать в виде произведения элементарных матриц:

$$\mathbf{A} = \mathbf{F}_1\mathbf{F}_2 \ldots \mathbf{F}_r.$$

Каждая из матриц $\mathbf{F}_i$ соответствует элементарной операции над строками матрицы $\mathbf{A}$, и, следовательно, согласно пп. (iii), (v) и (vii) теоремы 2.6.3, имеем

$$\det(\mathbf{AB}) = \det[(\mathbf{F}_1\mathbf{F}_2 \ldots \mathbf{F}_r)\mathbf{B}] = \det[\mathbf{F}_1(\mathbf{F}_2 \ldots \mathbf{F}_r\mathbf{B})] =$$

$$= (\det \mathbf{F}_1)\det(\mathbf{F}_2 \ldots \mathbf{F}_r\mathbf{B}) = (\det \mathbf{F}_1)(\det \mathbf{F}_2) \ldots$$

$$\ldots (\det \mathbf{F}_r)(\det \mathbf{B}).$$

При $\mathbf{B} = \mathbf{I}$ это дает

$$\det(\mathbf{A}) = (\det \mathbf{F}_1)(\det \mathbf{F}_2) \ldots (\det \mathbf{F}_r).$$

Подставляя последнее равенство в формулу для случая произвольной $\mathbf{B}$, получаем $\det(\mathbf{AB}) = \det(\mathbf{A}) \cdot \det(\mathbf{B})$, что и требовалось доказать. □

Одно из следствий этой теоремы состоит в том, что если $\mathbf{C} = \mathbf{AB}$, то матрица $\mathbf{C}$ обратима тогда и только тогда, когда обе матрицы $\mathbf{A}$ и $\mathbf{B}$ обратимы, так как квадратная матрица имеет обратную тогда и только тогда, когда ее определитель отличен от нуля.

Заканчивая данный параграф, завершим работу, оставшуюся от предыдущего параграфа.

Теорема 2.6.9. *Если размерность подпространства W векторного пространства всех n-последовательностей равна k, то размерность его ортогонального дополнения $W^\perp$ равна $n - k$.*

Доказательство. Пусть $\{\mathbf{g}_1, \ldots, \mathbf{g}_k\}$ — базис подпространства W; определим матрицу $\mathbf{G}$ равенством

$$\mathbf{G} = \begin{bmatrix} \mathbf{g}_1 \\ \cdot \\ \cdot \\ \cdot \\ \mathbf{g}_k \end{bmatrix},$$

где строками являются базисные векторы. Ранг этой матрицы равен k, и размерность пространства столбцов матрицы $\mathbf{G}$ равна k. Вектор $\mathbf{v}$ принадлежит $W^\perp$, если

$$\mathbf{G}\mathbf{v}^T = \mathbf{0},$$

так как в этом случае он ортогонален к каждому базисному вектору. Пусть $\{\mathbf{h}_1, \ldots, \mathbf{h}_r\}$ — базис подпространства $W^\perp$. Пополним этот базис до базиса всего пространства $\{\mathbf{h}_1, \ldots, \mathbf{h}_r, \mathbf{f}_1, \ldots, \ldots, \mathbf{f}_{n-r}\}$. Теперь вектор $\mathbf{v}$ из пространства столбцов матрицы $\mathbf{G}$ запишется в виде $\mathbf{v} = \mathbf{G}\mathbf{b}^T$, где вектор $\mathbf{b}$ представляет собой некоторую линейную комбинацию базисных векторов. Следовательно, каждый вектор в пространстве столбцов матрицы $\mathbf{G}$

должен записываться в виде линейной комбинации векторов $\{\mathbf{Gh}_1^T, \mathbf{Gh}_2^T, \ldots, \mathbf{Gh}_r^T, \mathbf{Gf}_1^T, \ldots, \mathbf{Gf}_{n-r}^T\}$.

Покажем теперь, что векторы $\{\mathbf{Gf}_1^T, \ldots, \mathbf{Gf}_{n-r}^T\}$ образуют базис пространств столбцов матрицы $\mathbf{G}$. Так как $\mathbf{Gh}_i^T = \mathbf{0}$, то это множество порождает пространство столбцов матрицы $\mathbf{G}$. Далее, эти векторы линейно независимы, так как если

$$a_1(\mathbf{Gf}_1^T) + \cdots + a_{n-r}(\mathbf{Gf}_{n-r}^T) = \mathbf{0},$$

то

$$\mathbf{G}(a_1\mathbf{f}_1^T + \cdots + a_{n-r}\mathbf{f}_{n-r}^T) = \mathbf{0},$$

и поэтому $a_1 = a_2 = \cdots = a_{n-r} = 0$, поскольку единственной линейной комбинацией векторов $\mathbf{f}_1, \ldots, \mathbf{f}_{n-r}$, принадлежащей нулевому пространству матрицы $\mathbf{G}$, является нулевой вектор $\mathbf{0}$. Следовательно, $\{\mathbf{Gf}_1^T, \ldots, \mathbf{Gf}_{n-r}^T\}$ образуют базис пространства столбцов матрицы $\mathbf{G}$. Таким образом, $n - r = k$, что и доказывает теорему. □

ЗАДАЧИ

2.1. Используя вращения и отражения, отображающие на себя пятиугольник, можно построить группу.

а. Сколько элементов содержит эта группа?

б. Является ли эта группа абелевой?

в. Построить эту группу.

г. Найти в этой группе подгруппу, содержащую пять элементов, и подгруппу, содержащую два элемента.

д. Существует ли подгруппа этой группы, содержащая четыре элемента? Почему?

2.2.а. Доказать, что существует только одна группа, содержащая три элемента. Построить эту группу и доказать, что она абелева.

б. Доказать, что существуют только две группы, содержащие четыре элемента. Построить эти группы и доказать, что они абелевы. Доказать, что одна из этих групп с четырьмя элементами не содержит элемента четвертого порядка (эта группа называется четвертой группой Клейна).

2.3. Пусть групповая операция в группах из задачи 2.2 называется сложением.

а. Определить умножение так, чтобы превратить эту трехэлементную группу в кольцо. Единственно ли оно?

б. Для каждой из четырехэлементных групп определить умножение так, чтобы превратить их в кольца. Единственны ли эти кольца?

2.4. Какие из трех колец задачи 2.3 являются также и полями? Можно ли задать умножение другим способом и получить при этом поле?

2.5. Доказать, что множество всех целых чисел (положительных, отрицательных и нуля) не образует группу относительно операции вычитания.

2.6. Привести пример кольца без единицы.

2.7. Рассмотрим множество S $\{0 = 1, 2, 3\}$ с операциями

+	0	1	2	3
0	0	1	2	3
1	1	2	3	0
2	2	3	0	1
3	3	0	1	2

·	0	1	2	3
0	0	0	0	0
1	0	1	2	3
2	0	2	3	1
3	0	3	1	2

Является ли оно полем?

2.8. Пусть G — произвольная группа (не обязательно конечная). Для удобства назовем групповую операцию умножением, а единичный элемент единицей. Пусть g — произвольный элемент, а ν (если оно существует) — наименьшее целое число, такое, что $g^\nu = 1$, где g^ν означает $g \cdot g \cdot \ldots \cdot g$ (ν раз). Тогда ν называется порядком элемента g. Доказать, что подмножество $\{g, g^2, \ldots, g^{\nu-1}, g^\nu\}$ образует подгруппу группы G. Доказать, что эта подгруппа абелева, даже если G не является абелевой.

2.9. Доказать теоремы 2.3.3 и 2.3.4.

2.10. Пусть задано некоторое кольцо с единицей 1, и пусть $a \cdot b = 1$. Доказать эквивалентность следующих утверждений:

(i) b — левый обратный к a;
(ii) если $ax = 0$, то $x = 0$;
(iii) если $yb = 0$, то $y = 0$.

Замечание. В некоторых кольцах условие (ii) не выполняется. В таких кольцах элемент может иметь только правый обратный или только левый обратный.

2.11. Поле из четырех элементов GF (4) задается арифметическими таблицами

+	0	1	2	3
0	0	1	2	3
1	1	0	3	2
2	2	3	0	1
3	3	2	1	0

·	0	1	2	3
0	0	0	0	0
1	0	1	2	3
2	0	2	3	1
3	0	3	1	2

Решить в GF (4) систему уравнений

$$2x + y = 3,$$
$$x + 2y = 3.$$

2.12. Поле с тремя элементами GF (3) задается арифметическими таблицами

+	0	1	2
0	0	1	2
1	1	2	0
2	2	0	1

·	0	1	2
0	0	0	0
1	0	1	2
2	0	2	1

Вычислить определитель матрицы

$$\begin{bmatrix} 2 & 1 & 2 \\ 1 & 1 & 2 \\ 1 & 0 & 1 \end{bmatrix}$$

и доказать, что ее ранг равен 3.

2.13. Привести матрицу

$$\mathbf{A} = \begin{bmatrix} 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}$$

к каноническому ступенчатому виду. Можно ли решить задачу, не конкретизируя поле? Почему?

2.14. Сколько векторов содержит векторное пространство GF^n (2)?

2.15. Верно ли, что если векторы $\mathbf{x}$, $\mathbf{y}$ и $\mathbf{z}$ — линейно независимы над $GF(q)$, то и векторы $\mathbf{x}+\mathbf{y}$, $\mathbf{y}+\mathbf{z}$ и $\mathbf{z}+\mathbf{x}$ линейно независимы?

2.16. Существует ли над полем $GF(q)$ векторное пространство, содержащее 24 элемента и имеющее размерность, бóльшую 1?

2.17. Пусть S и T — различные двумерные подпространства трехмерного векторного пространства; доказать, что их пересечение образует одномерное подпространство.

2.18. Пусть S — произвольное конечное множество, и пусть G — множество подмножеств из элементов S. Для двух подмножеств A и B обозначим через $A \cup B$ подмножество, состоящее из элементов, принадлежащих хотя бы одному из A и B, через $A \cap B$ множество элементов, принадлежащих как A, так и B, а через $A - B$ множество элементов, принадлежащих A и не принадлежащих B.

а. Доказать, что относительно операции объединения множеств $\cup$ множество G не является группой.

б. Теоретико-множественная операция симметрической разности Δ определяется равенством

$$A \Delta B = (A - B) \cup (B - A).$$

Доказать, что если в качестве групповой операции в G выбрать симметрическую разность, то G на самом деле будет группой. Является ли эта группа абелевой?

в. Доказать, что относительно операций Δ и $\cap$ множество G образует кольцо. Является ли это кольцо коммутативным? Имеется ли в нем единица?

2.19. Поле с бесконечным числом элементов, содержащее конечное поле, также называется полем Галуа. Пусть F — множество всех формальных выражений вида

$$F = \left\{ \left(\sum_{i=0}^{I-1} a_i x^i \right) \Big/ \left(\sum_{j=0}^{J-1} b_j x^j \right) \right\},$$

где I и J — некоторые положительные целые числа, а a_i и b_i — элементы поля GF (3) Предложить свои собственные определения операций сложения и умножения, такие, чтобы множество F превратилось в поле Галуа с бесконечным числом элементов.

ЗАМЕЧАНИЯ

В этой главе рассматривался обычный материал современной алгебры. Можно указать много учебников, в которых этот материал излагается более детально. В качестве легко усваиваемого вводного курса, по уровню достаточного для данной книги, мы рекомендуем книгу Биркгофа и Маклейна [1953]. Монография Ван дер Вардена [1950, 1953] представляет собой курс более высокого уровня, адресованный в основном математикам и углубленно излагающий многие вопросы. Материал по линейной алгебре и теории матриц можно найти также и в учебниках, специально посвященных этим разделам алгебры. Особенно подходящей является книга Тралля и Торнгейма [1957], так как она не предполагает никакой предварительной подготовки [1]).

Поля Галуа названы в честь Эвариста Галуа (1811—1832). Абелевы группы названы в честь Нильса Хенрика Абеля (1802—1829).

---

[1]) Не требующее предварительной подготовки хорошее изложение начал линейной алгебры и теории матриц можно найти также в книге: Головина Л. И. Линейная алгебра и некоторые ее приложения. — 4-е изд. — М.: Наука, 1985. Более углубленное изложение содержится в следующих книгах: Курош А. Г. Курс высшей алгебры. — 11-е изд. — М.: Наука, 1975; Мальцев А. И. Основы линейной алгебры. — 4-е изд. — М: Наука, 1975. — *Прим. перев.*

ГЛАВА 3

ЛИНЕЙНЫЕ БЛОКОВЫЕ КОДЫ

Большинство известных хороших кодов принадлежит классу *линейных кодов*. Этот класс кодов определяется специально выбираемой структурой кода. Такая структура облегчает поиск хороших кодов, а также помогает создать хорошие кодеры и декодеры. Следует подчеркнуть, что мы изучаем линейные коды не потому, что это лучшие коды, а скорее потому, что в настоящее время мы располагаем весьма скудной информацией о построении нелинейных кодов. В то же время хорошие линейные коды существуют и некоторые из них известны. Большинство наиболее эффективных теоретических методов применимо только к линейным кодам, и поэтому при поисках новых кодов обычно ограничиваются классом линейных кодов.

3.1. СТРУКТУРА ЛИНЕЙНЫХ БЛОКОВЫХ КОДОВ

Напомним, что векторное пространство $GF^n(q)$ представляет собой множество n-последовательностей элементов из $GF(q)$ с покомпонентным сложением и умножением на скаляр. Наиболее важным частным случаем является $GF^n(2)$ — векторное пространство двоичных слов длины n, в котором при сложении двух векторов в каждой компоненте происходит сложение по модулю 2.

Определение 3.1.1. *Линейный код* есть подпространство в в $GF^n(q)$.

Таким образом, линейный код есть непустое множество n-последовательностей над $GF(q)$, называемых кодовыми словами, такое, что сумма двух кодовых слов является кодовым словом, и произведение любого кодового слова на элемент поля также является кодовым словом. В любом линейном коде нулевое слово как начало координат векторного пространства всегда есть кодовое слово. Точнее, если $\mathbf{c}$ — кодовое слово, то $-\mathbf{c}$ также кодо-

вое слово и, следовательно, $\mathbf{c} + (-\mathbf{c})$ тоже будет кодовым словом.

Каждое слово в линейном коде связано с остальными словами кода так же, как любое другое кодовое слово. Расположение соседних кодовых слов вокруг нулевого слова есть типичное расположение слов вокруг любого другого кодового слова. Например, предположим, что $\mathbf{c}$ — некоторое кодовое слово и $\mathbf{c}_1, \ldots, \mathbf{c}_r$ — кодовые слова, находящиеся на некотором расстоянии d от $\mathbf{c}$; тогда $\mathbf{c} - \mathbf{c}$ — нулевое слово и $\mathbf{c}_1 - \mathbf{c}$, $\mathbf{c}_2 - \mathbf{c}$, $\ldots$, $\mathbf{c}_r - \mathbf{c}$ — кодовые слова, находящиеся на расстоянии d от нулевого слова. Таким образом, для определения минимального расстояния линейного кода достаточно определить расстояние между нулевым словом и ближайшим к нему другим кодовым словом.

Определение 3.1.2. *Вес Хэмминга* $w(\mathbf{c})$ кодового слова $\mathbf{c}$ равен числу его ненулевых компонент. *Минимальный вес* кода w^* есть минимальный вес ненулевого кодового слова.

Теорема 3.1.3. *Для линейного кода минимальное расстояние* d^* *находится из равенства*

$$d^* = \min_{\mathbf{c} \neq 0} w(\mathbf{c}) = w^*,$$

где минимум берется по всем кодовым словам, кроме нулевого.

Доказательство.

$$d^* = \min_{\substack{\mathbf{c}_i, \mathbf{c}_j \in \mathscr{C} \\ i \neq j}} d(\mathbf{c}_i, \mathbf{c}_j) = \min_{\substack{\mathbf{c}_i, \mathbf{c}_j \in \mathscr{C} \\ i \neq j}} d(0, \mathbf{c}_i - \mathbf{c}_j) = \min_{\substack{\mathbf{c} \in \mathscr{C} \\ \mathbf{c} \neq 0}} w(\mathbf{c}).$$

□

Следовательно, для нахождения линейного кода, исправляющего t ошибок, необходимо найти линейный код с минимальным весом, удовлетворяющим неравенству

$$w^* \geqslant 2t + 1.$$

Изучение метрических свойств линейных кодов много проще, чем нелинейных.

3.2. МАТРИЧНОЕ ОПИСАНИЕ ЛИНЕЙНЫХ БЛОКОВЫХ КОДОВ

Линейный код $\mathscr{C}$ образует подпространство в $GF^n(q)$. Для изучения этих кодов будет полезна теория линейных пространств. Любое множество базисных векторов может быть использовано в качестве строк для построения $(k \times n)$-матрицы $\mathbf{G}$, которая называется *порождающей матрицей кода*. Пространство строк матрицы $\mathbf{G}$ есть линейный код $\mathscr{C}$, любое кодовое слово есть линейная комбинация строк из $\mathbf{G}$. Множество q^k кодовых слов называется линейным (n, k)-кодом.

Строки матрицы $\mathbf{G}$ линейно независимы, и число строк k равно размерности подпространства. Размерность всего пространства $GF^n(q)$ равна n. Всего существует q^k кодовых слов, и q^k различных k-последовательностей над $GF(q)$ могут быть отображены на множество кодовых слов.

Любое взаимно однозначное соответствие k-последовательностей и кодовых слов может быть использовано для кодирования, но наиболее естественный способ кодирования использует отображение

$$\mathbf{c} = \mathbf{iG},$$

где $\mathbf{i}$ — информационное слово, представляющее собой k-последовательность кодируемых информационных символов, а $\mathbf{c}$ — образующая кодовое слово n-последовательность. Задаваемое последним равенством соответствие между информационным и кодовым словами определяет кодер и зависит от выбора базисных векторов в качестве строк матрицы $\mathbf{G}$; в то же время само множество кодовых слов не зависит от этого выбора.

В качестве простого примера двоичного блокового кода рассмотрим порождающую матрицу

$$\mathbf{G} = \begin{bmatrix} 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 \end{bmatrix}.$$

В этом случае информационное слово

$$\mathbf{i} = [0 \quad 1 \quad 1]$$

будет кодироваться в кодовое слово

$$\mathbf{c} = [0 \ 1 \ 1] \begin{bmatrix} 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 \end{bmatrix} = [0 \ 1 \ 1 \ 1 \ 0].$$

Порождающая матрица является сжатым описанием линейного кода. Сравните двоичный линейный (100,50)-код, для описания которого требуется $100 \times 50 = 5000$ битов, определяющих элементы матрицы $\mathbf{G}$, с произвольным (100, 50)-кодом, имеющим 2^{50} кодовых слов и требующим для описания примерно 10^{17} битов.

Поскольку $\mathscr{C}$ является подпространством, оно имеет ортогональное дополнение $\mathscr{C}^{\perp}$, которое состоит из всех векторов, ортогональных к $\mathscr{C}$. Ортогональное дополнение также является подпространством и, таким образом, может рассматриваться как код. Когда $\mathscr{C}^{\perp}$ само рассматривается как код, оно называется кодом, *дуальным* к $\mathscr{C}$.

Ортогональное дополнение $\mathscr{C}^{\perp}$ имеет размерность $n - k$, и любой его базис также состоит из $n - k$ векторов. Пусть $\mathbf{H}$ — матрица со строками, являющимися этими базисными векторами. Тогда n-последовательность $\mathbf{c}$ является кодовым словом в том и только том случае, когда она ортогональна каждой вектор-строке матрицы $\mathbf{H}$, т. е. когда

$$\mathbf{c}\mathbf{H}^{\mathrm{T}} = \mathbf{0},$$

Это равенство позволяет проверить, является ли данное слово кодовым. Матрица $\mathbf{H}$ называется *проверочной матрицей* кода. Она является $(n - k) \times n$ матрицей; поскольку равенство $\mathbf{c}\mathbf{H}^{\mathrm{T}} = \mathbf{0}$ выполняется при подстановке вместо $\mathbf{c}$ любой строки матрицы $\mathbf{G}$, то

$$\mathbf{G}\mathbf{H}^{\mathrm{T}} = \mathbf{0}.$$

Для порождающей матрицы $\mathbf{G}$, рассмотренной в предыдущем примере, получаем

$$\mathbf{H} = \begin{bmatrix} 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 \end{bmatrix},$$

что является одним из возможных выборов матрицы $\mathbf{H}$. Заметим, что точно так же, как существует много способов выбора матрицы $\mathbf{G}$, существует много способов выбора матрицы $\mathbf{H}$.

Теорема 3.2.1. *Порождающая матрица кода $\mathscr{C}$ является проверочной матрицей дуального кода $\mathscr{C}^{\perp}$.*

Доказательство непосредственно следует из предыдущих рассуждений. □

Связь минимального веса кода и проверочной матрицы устанавливается следующей теоремой.

Теорема 3.2.2. *Код $\mathscr{C}$ содержит ненулевое кодовое слово веса Хэмминга не более w тогда и только тогда, когда $\mathbf{H}$ содержит множество из w линейно зависимых столбцов.*

Доказательство. Для любого кодового слова $\mathbf{c}$ выполняется равенство $\mathbf{c}\mathbf{H}^{\mathrm{T}} = \mathbf{0}$. Пусть $\mathbf{c}$ имеет вес w. Исключим из рассмотрения нулевые компоненты вектора $\mathbf{c}$. Полученное равенство будет соотношением линейной зависимости w столбцов из $\mathbf{H}$. Следовательно, $\mathbf{H}$ содержит множество из w линейно зависимых столбцов.

Обратно, если $\mathbf{H}$ содержит множество из w линейно зависимых столбцов, то найдется равная нулю линейная комбинация, составленная не более чем из w столбцов. Соответствующие w ненулевых коэффициентов определяют вектор веса не более w, для которого $\mathbf{c}\mathbf{H}^{\mathrm{T}} = \mathbf{0}$. □

Следствие 3.2.3. *Код имеет минимальный вес не менее w тогда и только тогда, когда каждое множество из $w-1$ столбцов матрицы* $\mathbf{H}$ *линейно независимо.*

Отсюда следует, что для того чтобы найти (n, k)-код, исправляющий t ошибок, достаточно найти $(n-k) \times n$-матрицу $\mathbf{H}$, в которой любые $2t$ столбцов линейно независимы.

Из данного (n, k)-кода с минимальным расстоянием d^* можно получить новый код с теми же параметрами, если в каждом кодовом слове выбрать две позиции и поменять местами символы в этих позициях. Однако в этом случае мы получим код, который лишь тривиальным образом отличается от исходного. Говорят, что такой код *эквивалентен* исходному. Вообще два линейных кода, одинаковых с точностью до перестановки координат, называются эквивалентными.

Порождающие матрицы $\mathbf{G}$ и $\mathbf{G}'$ эквивалентных кодов просто связаны друг с другом. Сам код является пространством строк матрицы $\mathbf{G}$ и поэтому остается неизменным при элементарных операциях над строками. Перестановка координат кода эквивалентна перестановке столбцов в матрице $\mathbf{G}$. Таким образом, два кода эквивалентны тогда и только тогда, когда их порождающие матрицы получаются одна из другой посредством:

1) перестановки столбцов и

2) элементарных операций над строками.

Каждая порождающая матрица $\mathbf{G}$ эквивалентна некоторой матрице канонического ступенчатого вида, и, поскольку строки $\mathbf{G}$ линейно независимы, все строки эквивалентной матрицы являются ненулевыми. Следовательно, с точностью до перестановки столбцов любая порождающая матрица эквивалентна матрице, которая в первых k столбцах содержит единичную подматрицу размером $k \times k$. Эквивалентную матрицу можно записать в виде

$$\mathbf{G} = [\mathbf{I} \vdots \mathbf{P}],$$

где $\mathbf{P}$ есть $k \times (n-k)$-матрица. Любая порождающая матрица может быть приведена к такому частному виду при помощи последовательности элементарных операций над строками и последующей перестановки столбцов. Такую порождающую матрицу будем называть *порождающей матрицей в систематическом виде.*

Предположим, что $\mathbf{G} = [\mathbf{I} \vdots \mathbf{P}]$. Тогда естественным определением проверочной матрицы в систематическом виде, очевидно, является равенство $\mathbf{H} = [-\mathbf{P}^T \vdots \mathbf{I}]$, поскольку

$$\mathbf{G}\mathbf{H}^T = [\mathbf{I} \vdots \mathbf{P}] \begin{bmatrix} -\mathbf{P} \\ \cdots \\ \mathbf{I} \end{bmatrix} = -\mathbf{P} + \mathbf{P} = 0.$$

Определение 3.2.4. *Систематическим* [1]) кодом называется код, у которого каждое кодовое слово начинается с информационных символов. Оставшиеся символы называются *проверочными символами.*

Принято говорить о систематическом коде, хотя это всегда означает систематическое кодирование соответствующего кода.

Теорема 3.2.5. *Каждый линейный код эквивалентен систематическому линейному коду.*

Доказательство. Систематический линейный код кодируется умножением информационного вектора на порождающую матрицу **G** в систематическом виде, а каждая порождающая матрица **G** эквивалентна некоторой систематической матрице. □

В качестве примера выберем

$$\mathbf{G} = \left[\begin{array}{ccc|cc} 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 \end{array}\right], \qquad \mathbf{H} = \left[\begin{array}{ccc|cc} 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 \end{array}\right];$$

тогда $\mathbf{i} = [0 \;\; 1 \;\; 1]$ при систематическом кодировании отобразится в слово $\mathbf{c} = [0\ 1\ 1\ 1\ 0]$.

Рассматривая код в систематическом виде, можно получить простое неравенство, связывающее параметры кода. Эта граница является очень неточной, и большинство хороших кодов имеет минимальное расстояние, существенно меньшее, чем следует из границы, но бывают случаи, когда она оказывается полезной. Пусть задан некоторый систематический (n, k)-линейный код с минимальным расстоянием d^*. Следующая теорема накладывает ограничение на возможные значения (n, k, d^*).

Теорема 3.2.6 (граница Синглтона). *Минимальное расстояние (минимальный вес) любого линейного (n, k)-кода удовлетворяет неравенству*

$$d^* \leqslant 1 + n - k.$$

Доказательство. Ненулевое слово минимального веса в коде имеет вес d^*. Существуют слова систематического кода с одним ненулевым информационным символом и $n - k$ проверочными символами. Такое кодовое слово не может иметь вес, больший $1 +$

[1]) В гл. 5 будут рассматриваться циклические коды и будет удобно по соглашению считать, что кодовое слово **c** начинается с координаты с максимальным номером c_{n-1} и кончается c_0. В этом случае будем использовать эквивалентные матрицы вида $\mathbf{G} = [\mathbf{P} \vdots \mathbf{I}]$ и $\mathbf{H} = [\mathbf{I} \vdots -\mathbf{P}^T]$ для некоторой новой **P**. Эти матрицы получаются обращением всех столбцов и строк матриц, записанных в систематическом виде. Такие матрицы также называются систематическими.

$+ n - k$. Таким образом, минимальный вес кода не может быть больше $1 + n - k$. □

Определение 3.2.7. Любой код с минимальным расстоянием, удовлетворяющим равенству

$$d^* = 1 + n - k,$$

называется *кодом с максимальным расстоянием*.

Граница Синглтона показывает, что для исправления t ошибок код должен иметь не менее $2t$ проверочных символов (2 проверочных символа на ошибку). Большинство кодов, даже оптимальных, имеют намного больше проверочных символов, чем требует граница Синглтона, но параметры некоторых кодов удовлетворяют границе с равенством. Коды с максимальным расстоянием имеют точно $2t$ проверочных символов.

3.3. СТАНДАРТНОЕ РАСПОЛОЖЕНИЕ

Поскольку разность двух кодовых слов линейного кода является кодовым словом, нулевое слово всегда будет кодовым. Если мы знаем, какое множество принятых слов линейного кода лежит ближе всего к нулевому слову, то с помощью простого сдвига начала координат нетрудно выяснить, какие из принятых слов лежат ближе всего к любому другому кодовому слову.

Пусть d^* нечетно и $d^* = 2t + 1$. Внутри сферы радиуса t с центром в нулевом слове находится множество точек

$$S_0 = \{\mathbf{v} \mid d(0, \mathbf{v}) \leqslant t\}.$$

Эта сфера содержит все принятые слова, которые декодируются в нулевое кодовое слово. Внутри сферы радиуса t с центром в кодовом слове $\mathbf{c}$ находится множество точек

$$S_{\mathbf{c}} = \{\mathbf{v} \mid d(\mathbf{c}, \mathbf{v}) \leqslant t\};$$

тогда

$$S_{\mathbf{c}} = S_0 + \mathbf{c} = \{\mathbf{v} + \mathbf{c} \mid \mathbf{v} \in S_0\}.$$

Таким образом, мы можем выписать точки, лежащие внутри каждой сферы декодирования или (что эффективнее) внутри сферы декодирования с центром в нулевом слове, а точки, лежащие внутри других сфер получать по мере необходимости с помощью простого сдвига.

Стандартное расположение представляет собой способ описания всех этих сфер. Пусть $\mathbf{0}$, $\mathbf{c}_2$, $\mathbf{c}_3$, ..., $\mathbf{c}_{q^k}$ суть q^k кодовых слов (n, k)-кода. Построим таблицу, изображенную на рис. 3.1, следующим образом. В первой строке выпишем все кодовые слова. Из оставшихся в $GF^n(q)$ слов, лежащих на расстоянии 1 от нулевого слова, выберем любое и обозначим его $\mathbf{v}_1$. Во второй строке

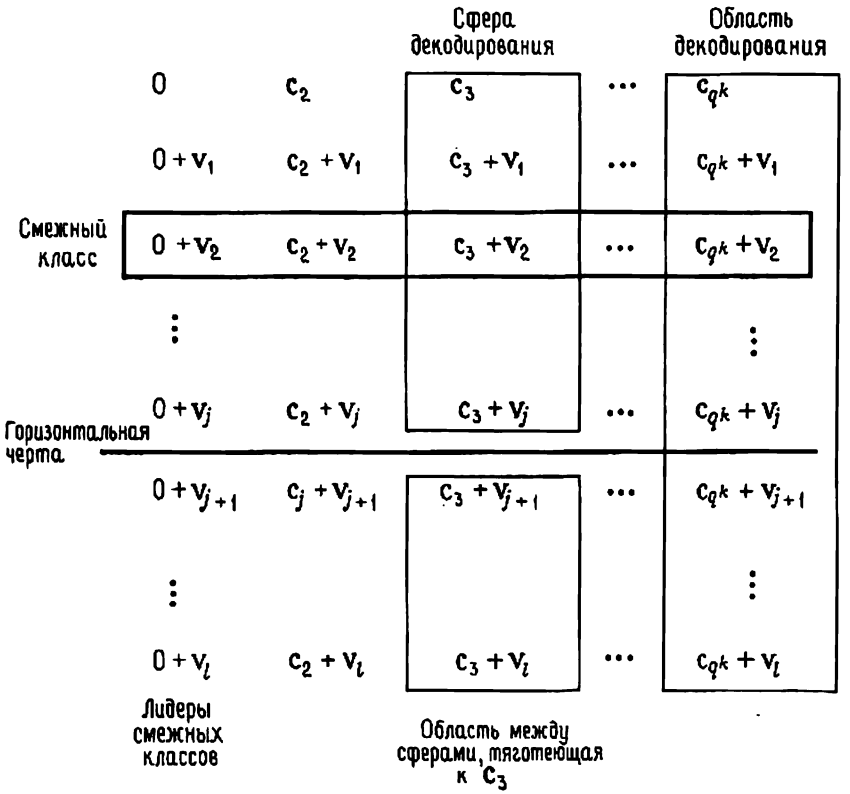


Рис. 3.1. Стандартное расположение.

запишем $0 + v_1$, $c_2 + v_1$, $c_3 + v_1$, ..., $c_{qk} + v_1$. Таким же, образом строятся следующие строки. На j-м шаге выберем слово v_j, которое является ближайшим к нулевому и отсутствует в предыдущих строках, и запишем в j-й строке $0 + v_j$, $c_2 + v_j$ $c_3 + v_j$, ..., $c_{qk} + v_j$. Эта процедура закончится, когда после очередного шага не останется неиспользованных слов.

Если рассматривать код как подгруппу, то описанная процедура порождает смежные классы по этой подгруппе. Следовательно, процесс закончится, когда каждое слово в таблице встретится ровно один раз. По следствию 2.2.4 всего в таблице будет q^{n-k} строк. Слова из первого столбца называются *лидерами смежных классов*.

Поскольку строки таблицы построены подобно чешуйкам лука, начиная с нулевого слова, первый столбец должен содержать все слова, лежащие внутри сферы декодирования с центром в нулевом слове, т. е. все слова, находящиеся на расстоянии не более

t от нулевого. Когда чешуйки заполнят всю внутренность сферы радиуса t вокруг нулевого слова, проведем в таблице горизонтальную черту. Могут остаться векторы, не занесенные в таблицу выше этой черты, и далее в стандартном расположении эти векторы будут сопоставляться ближайшим, но отчасти произвольным кодовым словам.

Существуют два основных класса декодеров, которые могут быть описаны с помощью стандартного расположения: полные декодеры и неполные декодеры. Полный декодер сопоставляет принятому слову ближайшее кодовое слово. Принятое слово необходимо найти в стандартном расположении, определить столбец, в котором оно находится, и декодировать его в кодовое слово в верхней позиции этого столбца.

Неполный декодер сопоставляет каждому принятому слову кодовое слово, находящееся от него на расстоянии не более t, если такое существует, в противном же случае отказывается от декодирования. Принятое слово необходимо найти в стандартном расположении. Если оно лежит выше горизонтальной черты, то декодируем его в кодовое слово в вершине столбца, в котором оказывается принятое слово. Если же оно лежит ниже горизонтальной черты, то помечаем принятое слово как не поддающееся декодированию. В таком слове имеется более t ошибок.

В качестве примера рассмотрим (5, 2)-код с порождающей матрицей

$$\mathbf{G} = \begin{bmatrix} 1\,0\,1\,1\,1 \\ 0\,1\,1\,0\,1 \end{bmatrix}.$$

Этот код позволяет исправлять одну ошибку. Стандартное расположение выглядит следующим образом:

0 0 0 0 0	1 0 1 1 1	0 1 1 0 1	1 1 0 1 0
0 0 0 0 1	1 0 1 1 0	0 1 1 0 0	1 1 0 1 1
0 0 0 1 0	1 0 1 0 1	0 1 1 1 1	1 1 0 0 0
0 0 1 0 0	1 0 0 1 1	0 1 0 0 1	1 1 1 1 0
0 1 0 0 0	1 1 1 1 1	0 0 1 0 1	1 0 0 1 0
1 0 0 0 0	0 0 1 1 1	1 1 1 0 1	0 1 0 1 0
0 0 0 1 1	1 0 1 0 0	0 1 1 1 0	1 1 0 0 1
0 0 1 1 0	1 0 0 0 1	0 1 0 1 1	1 1 1 0 0

Сферы радиуса 1 не пересекаются. Имеются всего четыре сферы, шесть точек внутри каждой сферы и восемь точек вне каждой сферы.

Очевидно, что ценность стандартного расположения относительна. Для больших n и k составлять такую таблицу было бы не практично.

Таблицу можно упростить, если мы сможем помнить только первый столбец и находить остальные столбцы по мере необходимости. Это можно сделать, введя понятие синдрома ошибок.

Для любого принятого вектора $\mathbf{v}$ определим *синдром* равенством

$$\mathbf{s} = \mathbf{v}\mathbf{H}^{\mathrm{T}}.$$

Теорема 3.3.1. *Все векторы из одного смежного класса имеют одинаковый синдром, присущий только этому смежному классу.*

Доказательство. Если $\mathbf{v}$ и $\mathbf{v}'$ лежат в одном смежном классе, то $\mathbf{v} = \mathbf{c}_i + \mathbf{y}$, $\mathbf{v}' = \mathbf{c}_j + \mathbf{y}$ для некоторого $\mathbf{y}$ и кодовых слов $\mathbf{c}_i$ и $\mathbf{c}_j$. Для любого кодового слова $\mathbf{c}$ выполняется равенство $\mathbf{c}\mathbf{H}^{\mathrm{T}} = \mathbf{0}$. Отсюда получаем

$$\mathbf{s} = \mathbf{v}\mathbf{H}^{\mathrm{T}} = \mathbf{y}\mathbf{H}^{\mathrm{T}},$$

$$\mathbf{s}' = \mathbf{v}'\mathbf{H}^{\mathrm{T}} = \mathbf{y}\mathbf{H}^{\mathrm{T}},$$

и, следовательно, $\mathbf{s} = \mathbf{s}'$. Обратно, допустим, что $\mathbf{s} = \mathbf{s}'$; тогда $(\mathbf{v} - \mathbf{v}')\,\mathbf{H}^{\mathrm{T}} = \mathbf{0}$, и поэтому разность $\mathbf{v} - \mathbf{v}'$ является кодовым словом. Следовательно, $\mathbf{v}$ и $\mathbf{v}'$ принадлежат одному смежному классу. □

Любые два вектора из одного смежного класса имеют одинаковый синдром. Следовательно, нам необходимо только свести в таблицу синдромы и лидеры смежных классов. Затем мы можем производить декодирование следующим образом. По принятому вектору $\mathbf{v}$ вычисляем синдром и находим соответствующий этому синдрому лидер смежного класса. Этот лидер является разностью между принятым словом и кодовым словом, которое одновременно является центром сферы кодирования. Таким образом, мы исправим ошибку, вычтя из слова $\mathbf{v}$ лидер смежного класса.

В примере, приведенном выше, проверочной матрицей будет

$$\mathbf{H} = \begin{bmatrix} 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 1 \end{bmatrix},$$

и новая таблица запишется в виде

Лидеры смежных классов	Синдромы
0 0 0 0 0	0 0 0
0 0 0 0 1	0 0 1
0 0 0 1 0	0 1 0
0 0 1 0 0	1 0 0
0 1 0 0 0	1 0 1
1 0 0 0 0	1 1 1
0 0 0 1 1	0 1 1
0 0 1 1 0	1 1 0

Эта таблица проще, чем стандартное расположение. Предположим, что принят вектор $\mathbf{v} = 10010$. Тогда $\mathbf{s} = \mathbf{v}\mathbf{H}^{\mathrm{T}} = 101$. Лидер смежного класса будет равен 01000. Отсюда получим, что переданное слово равно $10010 - 01000 = 11010$, а информационное слово равно 11.

3.4. КОДЫ ХЭММИНГА

Код, у которого минимальное расстояние не менее трех, в силу следствия 3.2.3 имеет проверочную матрицу, в которой все столбцы ненулевые и различные. Если проверочная матрица двоичного кода имеет m строк, то каждый столбец оказывается двоичным числом длины m. Существует всего $2^m - 1$ возможных столбцов. Следовательно, если $\mathbf{H}$ — матрица двоичного кода с $d^* \geqslant 3$— имеет m строк, то она может иметь не более $2^m - 1$ столбцов. В результате получается $(2^m - 1, 2^m - 1 - m)$-код. Простейший нетривиальный пример соответствует $m = 3$. В этом случае матрицы $\mathbf{H}$ и $\mathbf{G}$ в систематическом виде запишутся следующим образом:

$$\mathbf{H} = \begin{bmatrix} 1 & 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 1 \end{bmatrix}$$

и

$$\mathbf{G} = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{bmatrix}.$$

Такие $(2^m - 1, 2^m - 1 - m)$-коды называются кодами Хэмминга.

Ясно, что каждая пара столбцов матрицы **H** линейно независима (поскольку никакая пара различных двоичных векторов не дает в сумме нуля), а некоторые множества из трех столбцов будут линейно зависимы. Следовательно, согласно теореме 3.2.2, минимальный вес кода равен 3 и код исправляет одиночные ошибки.

Определение кодов Хэмминга легко обобщить на случай бо́льших алфавитов. Достаточно просто заметить, что главная идея построения таких кодов состоит в определении матрицы **H**, любая пара столбцов которой линейно независима. Для задания проверочной матрицы нельзя использовать все ненулевые m-последовательности над $GF(q)$, $q \neq 2$, поскольку некоторые из них попарно линейно зависимы. Чтобы обеспечить линейную независимость, выберем в качестве столбцов матрицы все m-последовательности, у которых первая ненулевая компонента равна единице. Тогда все столбцы будут попарно линейно независимыми, а некоторые тройки столбцов могут оказаться линейно зависимыми, и минимальный вес в коде будет равен трем.

Всего существует $(q^m - 1)/(q - 1)$ таких различных столбцов. Следовательно, получившийся код будет $((q^m - 1)/(q - 1), (q^m - 1)/(q - 1) - m)$-кодом. Код Хэмминга, исправляющий одиночные ошибки, существует для каждого q, для которого существует поле $GF(q)$, и для любого m. Для примера в табл. 3.1 приведены параметры нескольких кодов Хэмминга.

Таблица 3.1

Параметры (n, k) некоторых кодов Хэмминга

$GF(2)$	$GF(4)$	$GF(8)$	$GF(16)$	$GF(27)$
(7, 4)	(5, 3)	(9, 7)	(17, 15)	(28, 26)
(15, 11)	(21, 18)	(73, 70)	(273, 270)	(757, 754)
(31, 26)	(85, 81)	(585, 581)		
(63, 57)	(341, 336)			
(127, 120)				

Например (13,10)-код Хэмминга над $GF(3)$ задается проверочной матрицей

$$\mathbf{H} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 2 & 2 & 2 & 1 & 1 & 0 & 1 & 0 \\ 1 & 2 & 0 & 1 & 2 & 0 & 1 & 2 & 1 & 2 & 0 & 0 & 1 \end{bmatrix}$$

и порождающей матрицей

$$\mathbf{G} = \begin{bmatrix} 1&0&0&0&0&0&0&0&0&0&1&0&1 \\ 0&1&0&0&0&0&0&0&0&0&1&0&2 \\ 0&0&1&0&0&0&0&0&0&0&1&1&0 \\ 0&0&0&1&0&0&0&0&0&0&1&1&1 \\ 0&0&0&0&1&0&0&0&0&0&1&1&2 \\ 0&0&0&0&0&1&0&0&0&0&1&2&0 \\ 0&0&0&0&0&0&1&0&0&0&1&2&1 \\ 0&0&0&0&0&0&0&1&0&0&1&2&2 \\ 0&0&0&0&0&0&0&0&1&0&0&1&1 \\ 0&0&0&0&0&0&0&0&0&1&0&1&2 \end{bmatrix}$$

Второй пример носит более практический характер. Предположим, что ЭВМ и периферийное устройство связаны кабелем, по которому параллельно передаются 4 бита. Четыре бита представляют один 6-10-символ, и передается последовательность таких информационных символов. Нам бы хотелось объединить символы в блоки и защитить каждый блок от одиночных ошибок. Взяв $q = 16$ и $m = 2$, мы видим, что можно было бы получить (17, 15)-код Хэмминга над полем GF (16) при условии, что такое поле существует. Но поле GF (16) уже было приведено на рис. 2.1 (хотя мы еще не проверили, что оно удовлетворяет аксиомам поля). Используя это поле, можно построить порождающую матрицу (17, 15)-кода Хэмминга над GF (16) и с ее помощью выписать следующие соотношения для проверочных символов:

$$p_1 = i_1 + i_2 + i_3 + \cdots + i_{14} + i_{15},$$

$$p_2 = i_1 + 2i_2 + 3i_3 + \cdots + Ei_{14} + Fi_{15}.$$

После каждого блока из 15 информационных символов ставятся эти два проверочных символа. Используя это, декодер может исправлять одиночные ошибки в блоке из 17 символов. Конечно, все это имеет смысл только в том случае, когда поле GF (16) существует и задано так, как на рис. 2.1. Прежде чем проводить только что описанное построение в некотором поле GF (q), мы должны доказать, что такое поле существует, и задать в нем сложение и умножение.

3.5. СОВЕРШЕННЫЕ И КВАЗИСОВЕРШЕННЫЕ КОДЫ

Представьте себе маленькие сферы с центрами во всех кодовых словах; все эти сферы имеют один и тот же (целочисленный) радиус. Пусть теперь радиус сфер увеличивается (оставаясь целыми чис-

лами) до тех пор, пока дальнейшее увеличение радиуса будет невозможно без пересечения сфер. Значение этого радиуса равно числу исправляемых кодом ошибок. Этот радиус называется *радиусом сферической упаковки* кода. Теперь позволим радиусу увеличиваться далее (оставаясь при этом целым числом) до тех пор, пока каждая точка пространства не окажется внутри хотя бы одной сферы. Такой радиус называется *радиусом покрытия* кода.

Радиус упаковки и радиус покрытия кода могут совпадать; если это так, то построение стандартного расположения закончится в тот момент, когда исчерпаются все векторы внутри сфер радиуса t. Все точки пространства содержатся внутри этих сфер, и ни одна точка не остается вне сфер.

Определение 3.5.1. *Совершенный код* есть код, для которого сферы некоторого одинакового радиуса вокруг кодовых слов, не пересекаясь, покрывают все пространство.

Совершенный код удовлетворяет границе Хэмминга из задачи 1.5 с равенством. Код Хэмминга, имеющий длину $n = (q^m - 1)/(q - 1)$, является совершенным. Это происходит потому, что внутри каждой сферы радиуса 1 содержится $1 + n(q - 1) = q^m$ точек, и число точек в пространстве, деленное на число сфер, равно $q^n/q^k = q^m$, поскольку $n - k = m$. Совершенные коды (в случаях, когда они существуют) обладают замечательными свойствами, и с ними приятно иметь дело, но они столь редки, что имеют ограниченное практическое значение.

Определение 3.5.2. *Квазисовершенный код* — это код, у которого сферы радиуса t вокруг каждого кодового слова не пересекаются и все слова, не лежащие внутри какой-либо из этих сфер, находятся на расстоянии $t + 1$ хотя бы от одного кодового слова.

Квазисовершенные коды встречаются чаще, чем совершенные коды. Когда для заданных n и k существует такой код (и не существует совершенного кода), то для этих n и k не существует кода с бо́льшим значением d^*. Однако мы должны снова отметить, что квазисовершенные коды редки и по всей видимости не являются особо важными в практических приложениях.

3.6. ПРОСТЫЕ ПРЕОБРАЗОВАНИЯ ЛИНЕЙНОГО КОДА

Существует набор простых преобразований, которые, незначительно изменяя исходный линейный код, приводят к новому коду. Если новый код также оказывается линейным, то эти преобразования соответствуют небольшим изменениям порождающей ма-

трицы **G**; например, можно добавить в эту матрицу столбец или строку, выбросить из нее столбец или строку, добавить одновременно столбец и строку, выбросить одновременно столбец и строку. Все эти изменения кода легко описать, хотя может оказаться, что непросто найти такое преобразование, которое необходимо в каждом конкретном случае.

Длину кода n можно увеличить за счет увеличения k или $n - k$. Мы будем называть такие преобразования *удлинением* и *расширением* кода соответственно и объединим оба этих понятия более общим понятием *увеличения* кода. Под увеличением кода мы понимаем увеличение длины или за счет расширения, или за счет удлинения. Шесть основных преобразований кода выглядят следующим образом.

Расширение кода. Увеличение длины путем добавления новых проверочных символов, что приводит к возрастанию большего размера порождающей матрицы.

Удлинение кода. Увеличение длины кода путем добавления новых информационных символов, что приводит к увеличению обоих размеров порождающей матрицы на одно и то же число.

Выкалывание кодовых координат. Уменьшение длины кода удалением проверочных символов, что приводит к уменьшению большего размера порождающей матрицы.

Укорочение кода. Уменьшение длины кода удалением информационных символов, что приводит к уменьшению обоих размеров порождающей матрицы на одно и то же число.

Пополнение кода. Увеличение числа информационных символов без увеличения длины кода, что приводит к увеличению меньшего размера порождающей матрицы.

Код с выбрасыванием. Уменьшение числа информационных символов без изменения длины кода, что приводит к понижению меньшего размера порождающей матрицы.

Указанные преобразования можно использовать для модификации известных кодов, чтобы сделать их подходящими для каких либо конкретных приложений, а также для получения новых классов хороших кодов.

Любой двоичный (n, k, d^*)-код с нечетным минимальным расстоянием можно расширить до $(n + 1, k, d^* + 1)$-кода добавлением к каждому кодовому слову суммы всех его компонент в качестве проверки на четность. Так происходит потому, что в случае, когда исходное слово имеет нечетный вес, к нему будет добавляться единичный символ. Следовательно, все кодовые слова веса d^* становятся кодовыми словами веса $d^* + 1$. Если **H** —

проверочная матрица исходного кода, то расширенный код будет иметь проверочную матрицу

$$\mathbf{H}' = \begin{bmatrix} 1 & 1 & \dots & 1 \\ 0 & & & \\ \vdots & & \mathbf{H} & \\ 0 & & & \end{bmatrix}.$$

В частности, каждый $(2^m - 1, 2^m - 1 - m)$-код Хэмминга может быть расширен до $(2^m, 2^m - m)$-кода, исправляющего одиночные ошибки и обнаруживающего двойные ошибки; обычно этот код также называется кодом Хэмминга.

Можно произвести и редукцию кода, переходя к коду над меньшим полем. Из некоторого кода над полем $GF(q^m)$ выберем все слова с координатами из подполя $GF(q)$. Получившийся код называется *подкодом над подполем* исходного кода. Если исходный код является линейным, то подкод над подполем также линеен, но не является подпространством исходного кода. Это происходит потому, что подпространство должно содержать все возможные произведения кодовых слов на элементы поля $GF(q^m)$. Подкод над подполем является линейным потому, что над подполем $GF(q)$ все линейные комбинации кодовых слов будут оставаться в подкоде над подполем. Любое множество базисных векторов в подкоде над подполем является линейно независимым также и в поле $GF(q^m)$, и поэтому размерность исходного поля не меньше размерности подкода над подполем. Обычно, однако, исходный код имеет бо́льшую размерность; подкод над подполем имеет меньшую скорость, чем исходный код.

3.7. КОДЫ РИДА—МАЛЛЕРА

Коды Рида—Маллера представляют собой класс линейных кодов над $GF(2)$ с простым описанием и декодированием, осуществляемым методом простого голосования. По этим причинам коды Рида—Маллера играют важную роль в кодировании [1]), хотя если судить об этих кодах по минимальному расстоянию, то, за некоторыми исключениями, они не заслуживают особого внимания. Для любых целых m и $r < m$ существует код Рида—Маллера длины 2^m, который называется кодом Рида—Маллера r-го порядка длины 2^m.

Код Рида—Маллера является линейным кодом. Мы определим этот код через порождающую матрицу; эту матрицу будем строить

[1]) Коды Рида—Маллера были использованы при передаче фотографий Марса космическим кораблем Маринер в 1972 г. В настоящее время для этой цели можно было бы использовать более эффективное кодирование.

в удобной для декодирования несистематической форме. Прежде всего определим *покомпонентное произведение* двух векторов **a** и **b**; если

$$\mathbf{a} = (a_0, a_1, \ldots, a_{n-1}) \quad \text{и} \quad \mathbf{b} = (b_0, b_1, \ldots, b_{n-1}),$$

то их произведение равно вектору

$$\mathbf{ab} = (a_0b_0, a_1b_1, \ldots, a_{n-1}b_{n-1}).$$

Порождающая матрица кода Рида—Маллера r-го порядка длиной 2^m определяется как совокупность блоков

$$\mathbf{G} = \begin{bmatrix} \mathbf{G}_0 \\ \mathbf{G}_1 \\ \mathbf{G}_r \end{bmatrix},$$

где $\mathbf{G}_0$ — вектор размерности $n = 2^m$, состоящий из одних единиц; $\mathbf{G}_1$ есть $(m \times 2^m)$-матрица, содержащая в качестве столбцов все двоичные m-последовательности; строки матрицы $\mathbf{G}_l$ получаются из строк матрицы $\mathbf{G}_1$ как все возможные произведения l строк из $\mathbf{G}_1$. Для определенности будем считать, что первый столбец в $\mathbf{G}_1$ состоит из одних нулей, последний — из одних единиц, а остальные m-последовательности в $\mathbf{G}_1$ расположены в порядке возрастания, считая, что младший бит расположен в нижней строке.

Поскольку существует всего $\binom{m}{l}$ способов выбора l строк, входящих в произведение, то матрица $\mathbf{G}_l$ имеет размер $\binom{m}{l} \times \times 2^m$. Ясно, что для кода Рида—Маллера порядка r

$$k = 1 + \binom{m}{1} + \cdots + \binom{m}{r},$$

$$n - k = 1 + \binom{m}{1} + \cdots + \binom{m}{m-r-1},$$

что обеспечивается линейной независимостью строк в матрице $\mathbf{G}$. Код Рида—Маллера нулевого порядка является $(n, 1)$-кодом. Это просто код с повторением, который тривиально декодируется с помощью мажоритарного метода. Минимальное расстояние такого кода равно 2^m.

В качестве примера положим $m = 4$, $n = 16$, $r = 3$. Тогда

$$\mathbf{G}_0 = [1\ 1\ 1\ 1\ 1\ 1\ 1\ 1\ 1\ 1\ 1\ 1\ 1\ 1\ 1\ 1] = [\mathbf{a}_0],$$

$$\mathbf{G}_1 = \begin{bmatrix} 0\ 0\ 0\ 0\ 0\ 0\ 0\ 0\ 1\ 1\ 1\ 1\ 1\ 1\ 1\ 1 \\ 0\ 0\ 0\ 0\ 1\ 1\ 1\ 1\ 0\ 0\ 0\ 0\ 1\ 1\ 1\ 1 \\ 0\ 0\ 1\ 1\ 0\ 0\ 1\ 1\ 0\ 0\ 1\ 1\ 0\ 0\ 1\ 1 \\ 0\ 1\ 0\ 1\ 0\ 1\ 0\ 1\ 0\ 1\ 0\ 1\ 0\ 1\ 0\ 1 \end{bmatrix} = \begin{bmatrix} \mathbf{a}_1 \\ \mathbf{a}_2 \\ \mathbf{a}_3 \\ \mathbf{a}_4 \end{bmatrix}.$$

Поскольку $\mathbf{G}_1$ имеет 4 строки, матрица $\mathbf{G}_2$ имеет $\binom{4}{2}$ строк:

$$\mathbf{G}_2 = \begin{bmatrix} 0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,1\,1\,1\,1 \\ 0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,1\,1\,0\,0\,1\,1 \\ 0\,0\,0\,0\,0\,0\,0\,0\,0\,1\,0\,1\,0\,1\,0\,1 \\ 0\,0\,0\,0\,0\,0\,1\,1\,0\,0\,0\,0\,0\,0\,1\,1 \\ 0\,0\,0\,0\,0\,1\,0\,1\,0\,0\,0\,0\,0\,1\,0\,1 \\ 0\,0\,0\,1\,0\,0\,0\,1\,0\,0\,0\,1\,0\,0\,0\,1 \end{bmatrix} = \begin{bmatrix} \mathbf{a}_1\mathbf{a}_2 \\ \mathbf{a}_1\mathbf{a}_3 \\ \mathbf{a}_1\mathbf{a}_4 \\ \mathbf{a}_2\mathbf{a}_3 \\ \mathbf{a}_2\mathbf{a}_4 \\ \mathbf{a}_3\mathbf{a}_4 \end{bmatrix},$$

а матрица $\mathbf{G}_3$ имеет $\binom{4}{3}$ строк:

$$\mathbf{G}_3 = \begin{bmatrix} 0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,1\,1 \\ 0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,1\,0\,1 \\ 0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,1\,0\,0\,0\,1 \\ 0\,0\,0\,0\,0\,0\,0\,1\,0\,0\,0\,0\,0\,0\,0\,1 \end{bmatrix} = \begin{bmatrix} \mathbf{a}_1\mathbf{a}_2\mathbf{a}_3 \\ \mathbf{a}_1\mathbf{a}_2\mathbf{a}_4 \\ \mathbf{a}_1\mathbf{a}_3\mathbf{a}_4 \\ \mathbf{a}_2\mathbf{a}_3\mathbf{a}_4 \end{bmatrix}.$$

Таким образом порождающая матрица кода Рида—Маллера третьего порядка длины 16 является (15×16)-матрицей вида

$$\mathbf{G} = \begin{bmatrix} \mathbf{G}_0 \\ \mathbf{G}_1 \\ \mathbf{G}_2 \\ \mathbf{G}_3 \end{bmatrix}.$$

Эта порождающая матрица задает (16, 15)-код над $GF(2)$ (на самом деле это просто код с проверкой на четность). Другой код Рида—Маллера может быть получен с использованием этих матриц, если мы положим $r = 2$. В этом случае порождающая матрица имеет вид

$$\mathbf{G} = \begin{bmatrix} \mathbf{G}_0 \\ \mathbf{G}_1 \\ \mathbf{G}_2 \end{bmatrix}$$

и задает (16, 11)-код над $GF(2)$. (В действительности это (15, 11)-код Хэмминга, расширенный с помощью проверки на четность.)

Из определения порождающей матрицы ясно, что код Рида—Маллера r-го порядка может быть получен пополнением кода Рида—Маллера $(r-1)$-го порядка, а код Рида—Маллера $(r-1)$-го порядка получается из кода r-го порядка с помощью выбрасывания. Поскольку код Рида—Маллера r-го порядка содержит код $(r-1)$-го порядка, ясно что его минимальное расстояние не может быть больше минимального расстояния кода $(r-1)$-го порядка. В дальнейшем мы докажем, что код Рида—Маллера r-го порядка имеет минимальное расстояние $d^* = 2^{m-r}$.

Каждая строка матрицы $\mathbf{G}_i$ имеет вес 2^{m-i}. Таким образом, любая строка матрицы $\mathbf{G}$ имеет четный вес, а сумма двух двоичных векторов четного веса также имеет четный вес (см. задачу 3.11). Следовательно, все линейные комбинации строк матрицы $\mathbf{G}$ имеют четный вес, а это означает, что все кодовые слова имеют четный вес. Матрица $\mathbf{G}_r$ содержит строки весом 2^{m-r}, и, следовательно, минимальный вес кода не превосходит 2^{m-r}.

Мы должны доказать, что строки $\mathbf{G}$ линейно независимы, и найти минимальный вес кода. Мы докажем, что код имеет минимальный вес 2^{m-r} и строки в матрице $\mathbf{G}$ линейно независимы. Для этого мы построим алгоритм декодирования — алгоритм Рида, который позволяет исправлять $\frac{1}{2} \cdot 2^{m-r} - 1$ ошибок и восстанавливать k информационных символов. Отсюда следует, что минимальное расстояние будет не меньше $2^{m-r} - 1$, но оно должно быть четным и поэтому будет не меньше 2^{m-r}.

Алгоритм Рида был разработан специально для кодов Рида — Маллера. Можно, конечно, использовать процедуру синдромного декодирования, описанную в § 3.3, но в данном случае осуществить ее довольно сложно. Алгоритм Рида отличается от большинства алгоритмов декодирования тем, что позволяет восстановить информационные символы прямо из принятого слова и при этом не дает точного значения самой ошибки. В этом алгоритме не используются также промежуточные переменные, например синдром.

Предположим, что у нас имеется декодер для кода Рида—Маллера $(r-1)$-го порядка, исправляющего $\frac{1}{2} \cdot 2^{m-(r-1)} - 1$ ошибок. Мы построим декодер для кода Рида—Маллера r-го порядка, исправляющего $\left(\frac{1}{2} \cdot 2^{m-r} - 1\right)$ ошибку, сведя этот случай к предыдущему. Поскольку мы уже знаем, что код Рида—Маллера нулевого порядка может быть декодирован с помощью мажоритарного метода, мы по индукции получим метод декодирования для кодов высших порядков.

Удобно разбить информационный вектор на $r+1$ сегмент, положив $\mathbf{i} = [\mathbf{I}_0, \mathbf{I}_1, \ldots, \mathbf{I}_r]$, где сегмент $\mathbf{I}_l$ содержит $\binom{m}{l}$ информационных битов. Каждый сегмент будет умножаться на один блок матрицы $\mathbf{G}$. Кодирование можно представить поблочным умножением вектора на матрицу:

$$\mathbf{c} = [\mathbf{I}_0, \mathbf{I}_1, \ldots, \mathbf{I}_r] \begin{bmatrix} \mathbf{G}_0 \\ \mathbf{G}_1 \\ \vdots \\ \mathbf{G}_r \end{bmatrix} = \mathbf{iG}.$$

Предположим, что информационная последовательность разбита на сегменты следующим образом: каждому сегменту соответствует один из r блоков порождающей матрицы, который при кодировании умножается на этот сегмент. Если мы сможем восстановить информационные биты в r-м сегменте, то затем сможем вычислить их вклад в принятое слово и вычесть его из принятого слова. Тогда задача сведется к декодированию кода Рида—Маллера $(r-1)$-го порядка. Процедура декодирования представляет собой последовательность мажоритарных декодирований и начинается с нахождения мажоритарным методом информационных символов в сегменте с номером r.

Принятое слово запишем в виде

$$\mathbf{v} = [\mathbf{I}_0,\ \mathbf{I}_1,\ \ldots,\ \mathbf{I}_r]\begin{bmatrix}\mathbf{G}_0\\ \mathbf{G}_1\\ \vdots\\ \mathbf{G}_r\end{bmatrix} + \mathbf{e}.$$

Алгоритм декодирования сначала по вектору $\mathbf{v}$ восстанавливает $\mathbf{I}_r$, затем вычисляется разность

$$\mathbf{v}' = \mathbf{v} - \mathbf{I}_r\mathbf{G}_r = [\mathbf{I}_0,\ \mathbf{I}_1,\ \ldots,\ \mathbf{I}_{r-1}]\begin{bmatrix}\mathbf{G}_0\\ \mathbf{G}_1\\ \vdots\\ \mathbf{G}_{r-1}\end{bmatrix} + \mathbf{e},$$

которая является искаженным кодовым словом кода Рида—Маллера $(r-1)$-го порядка.

Прежде всего рассмотрим декодирование информационного бита i_{k-1}, который умножается на последнюю строку матрицы $\mathbf{G}_r$. Декодируем этот бит, вычисляя 2^{m-r} проверочных сумм по 2^r бит из 2^m бит принятого слова, так что каждый принятый бит входит лишь в одну сумму. Проверочные суммы строятся таким образом, что символ i_{k-1} вносит вклад только в один бит, а все другие информационные символы вносят вклад в четное число битов в каждой проверочной сумме. Таким образом, при отсутствии ошибок все проверочные суммы равны i_{k-1}. Но, даже если имеется не более $\frac{1}{2}\cdot 2^{m-r} - 1$ ошибок, большинство проверочных сумм по-прежнему будет равняться i_{k-1}.

Первая проверочная сумма представляет собой сумму по модулю 2 первых 2^r битов принятого слова, вторая — сумму по модулю 2 вторых 2^r битов принятого слова и т. д. Всего получается 2^{m-r} проверочных сумм и по предположению имеется $\frac{1}{2}\cdot 2^{m-r} - 1$

ошибок. Таким образом, мажоритарное голосование проверочных сумм дает правильное значение i_{k-1}. Для построенного ранее (16, 11)-кода Рида—Маллера эти четыре суммы выглядят следующим образом:

$$\hat{i}_{10}^{(1)} = v_0 + v_1 + v_2 + v_3,$$
$$\hat{i}_{10}^{(2)} = v_4 + v_5 + v_6 + v_7,$$
$$\hat{i}_{10}^{(3)} = v_8 + v_9 + v_{10} + v_{11},$$
$$\hat{i}_{10}^{(4)} = v_{12} + v_{13} + v_{14} + v_{15}.$$

Если произошла только одна ошибка, то одна из оценок, даваемых проверочными суммами, будет неверна; мажоритарное решение дает значение i_{10}. Если произошло две ошибки, то ни одно значение проверочных сумм не встречается чаще других и обнаруживается двойная ошибка.

Аналогично может быть продекодирован любой информационный символ, который умножается на одну из строк матрицы $\mathbf{G}_r$. Это происходит потому, что любая строка матрицы $\mathbf{G}_r$ ничем не выделяется среди остальных. Перестановкой столбцов мы можем добиться того, что любая строка будет выглядеть так же, как последняя строка матрицы $\mathbf{G}_r$. Следовательно, можно использовать те же самые проверочные суммы, соответственно переставив индексы у символов, входящих в эти суммы. После построения 2^{m-r} проверочных сумм каждый бит декодируется мажоритарным методом.

После того как эти информационные символы найдены, их вклад в кодовое слово вычитается из принятого слова. Это эквивалентно тому, что мы получаем слово кода Рида — Маллера $(r-1)$-го порядка. В свою очередь его последний сегмент информационных битов вычисляется с помощью той же самой процедуры.

Этот процесс повторяется до тех пор, пока не будут найдены все информационные биты.

ЗАДАЧИ

3.1. Порождающая матрица кода над $GF(2)$ задается следующим образом:

$$\mathbf{G} = \begin{bmatrix} 1 & 0 & 1 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 & 0 \end{bmatrix}.$$

а. Найти порождающую и проверочную матрицу эквивалентного систематического кода.

б. Выписать вектор дуального кода.

в. Выписать стандартное расположение для этого кода.

г. Сколько кодовых слов имеет вес 0, ..., 6?

д. Найти кодовое слово, соответствующее информационной последовательности 101. Декодировать принятое слово 111001.

3.2. Для заданного кода с проверочной матрицей **H** показать, что смежный класс, соответствующий синдрому **s**, содержит вектор веса w в том и только том случае, когда некоторая линейная комбинация w столбцов матрицы **H** равна **s**.

3.3. В этой задаче доказывается, что хороший декодер должен производить нелинейные операции (даже если сам код является линейным).

а. Доказать, что процедура вычисления синдрома линейна по отношению к вектору ошибок. Это значит, что если $\mathbf{s} = F(\mathbf{e})$, то

$$F(a\mathbf{e}_1 + b\mathbf{e}_2) = aF(\mathbf{e}_1) + bF(\mathbf{e}_2).$$

б. Линейный декодер — это такой декодер, у которого функция $\hat{\mathbf{e}} = f(\mathbf{s})$, связывающая синдром и вектор ошибок, удовлетворяет равенству

$$f(a\mathbf{s}_1 + b\mathbf{s}_2) = af(\mathbf{s}_1) + bf(\mathbf{s}_2).$$

Доказать, что линейный декодер может исправлять самое большее $(n - k) \times (q - 1)$ из $n(q - 1)$ возможных одиночных ошибок.

в. Доказать, что если мы хотим, чтобы декодер исправлял все одиночные ошибки, то функция $\hat{\mathbf{e}} = f(\mathbf{s})$, связывающая синдром и вектор ошибок, должна быть нелинейной.

3.4. Доказать, что если линейный двоичный код имеет нечетное минимальное расстояние, то расширение кода добавлением проверки на четность увеличивает минимальное расстояние на 1.

3.5. Определим линейный (5, 3)-код над $GF(4)$ с помощью порождающей матрицы

$$\mathbf{G} = \begin{bmatrix} 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 2 \\ 0 & 0 & 1 & 1 & 3 \end{bmatrix}.$$

а. Найти проверочную матрицу.

б. Доказать, что этот код исправляет одиночные ошибки.

в. Доказать, что этот код исправляет два стирания [см. задачу 3.9. — *Перев.*]

г. Доказать, что это совершенный код.

3.6. Чему равна скорость (максимального) двоичного подкода над подполем, лежащего в коде над $GF(4)$ из задачи 3.5? Выписать для этого подкода порождающую матрицу в систематическом виде.

3.7. Сделав необходимый расчет, доказать возможность существования совершенного (11, 6)-кода над $GF(3)$, исправляющего 2 ошибки (Голей построил такой код в 1949 г.).

3.8. Сделав необходимый расчет, доказать возможность существования совершенного $(q^2 + 1, q - 1)$-кода над $GF(q)$, исправляющего одиночные ошибки. (Эти коды являются расширенными кодами Рида—Соломона. Они эквивалентны кодам Хэмминга.)

3.9. Предложить процедуру восстановления двух стираний для двоичного (7, 4)-кода Хэмминга. (Стиранием называется потеря передаваемого символа в некоторой позиции. Оно отличается от ошибки тем, что известно, в какой именно позиции это произошло.)

3.10. Найти проверочную и порождающую матрицы для (21, 18)-кода Хэмминга над $GF(4)$. Разработать кодер и использующий таблицу синдромов декодер, не входя во все детали, а только указывая необходимые для работы декодера блоки. Является ли подлежащая просмотру таблица синдромов чрезмерно большой для практического использования?

3.11. Доказать, что сумма двух двоичных m-мерных векторов четного веса имеет четный вес.

3.12. *Взвешивание шаров.* Пусть имеются весы с двумя чашками и 12 бильярдных шаров, из которых один может оказаться легче или тяжелее остальных.

а. Используя проверочную матрицу укороченного (12, 9)-кода Хэмминга над GF (3), предложить способ, как за три взвешивания определить нестандартный шар (если он имеется) и выяснить, легче или тяжелее он остальных шаров.

б. Можно ли решить задачу таким же способом, если шаров будет 13? Можно ли решить задачу в случае 13 шаров, если можно использовать четырнадцатый, заведомо стандартный шар?

в. Используя сферическую упаковку, доказать, что задачу нельзя решить для 14 шаров, один из которых может оказаться нестандартным.

3.13. Код, совпадающий с дуальным ему кодом, называется *самодуальным*.

а. Доказать, что линейный код с проверочной матрицей $[(-\mathbf{P}) : \mathbf{I}]$ является самодуальным тогда и только тогда, когда $\mathbf{P}$ — квадратная матрица, удовлетворяющая равенству $\mathbf{P}\mathbf{P}^{\mathrm{T}} = \mathbf{I}$.

б. Построить двоичный самодуальный код длины 4 и 8.

3.14. Предложить алгоритм декодирования кодового слова кода Рида—Маллера при наличии $2^{m-r} - 1$ стираний.

3.15. Можно ли с помощью проверки на четность расширить любой недвоичный (n, k, d^*)-код Хэмминга для получения $(n + 1, k, d^* + 1)$-кода? Привести доказательство или контрпример.

3.16. Доказать, что граница Синглтона верна также и для нелинейных кодов.

3.17. Построить двоичный (7, 4)-код Хэмминга с помощью диаграммы Венна: пересечь три круга так, чтобы получилось семь областей, и четырем областям поставить в соответствие информационные символы, а остальным трем — проверочные символы. Обсудить исправление ошибок и стираний, используя полученную диаграмму.

ЗАМЕЧАНИЯ

Изучение линейных кодов восходит к ранним работам Хэмминга [1950] и Голея [1949]. Большинство положений, используемых в настоящее время при изучении линейных кодов, заложено Слепяном [1956, 1960]; первые три параграфа тесно связаны с этими работами. Еще до этого Киясу [1953] обратил внимание на связь между линейными кодами и подпространствами векторных пространств. Коды с максимальным расстоянием впервые изучал Синглтон [1964].

Двоичные коды Хэмминга как коды, исправляющие ошибки, впервые рассматривал Хэмминг, хотя подобные комбинаторные структуры, ранее встречались в задачах статистики. Недвоичные коды Хэмминга были получены Голеем [1958] и Коком [1959].

Понятие совершенного кода впервые встречается у Голея, хотя он не пользовался таким термином. Делалось много попыток найти новые совершенные коды, но лишь отдельные из них увенчались успехом. В ряде работ Тиетявяйнен и Ван Линт (работы завершились в 1974 и 1975 гг. соответственно) доказали, что не существует линейных (нетривиальных) совершенных кодов, отличных от кодов Хэмминга или Голея, и не существует нелинейных (нетривиальных) кодов, за исключением кодов Васильева [1962] и Шёнхейма [1968] [1]).

Коды Рида—Маллера были получены Маллером [1954], и в этом же году Рид разработал алгоритм их декодирования. Этот алгоритм необычен тем, что принимает решение, основанное на мажоритарной логике. Коды Рида—Маллера являются предшественниками обширных семейств мажоритарно декодируемых кодов, с которыми мы встретимся в следующих главах.

---

[1]) Одновременно этот же результат был получен Семаковым и Зиновьевым (см. Семаков Н. В., Зиновьев В. А. Совершенные и квазисовершенные коды. — Проблемы передачи информации, 1969, вып. 2, с. 14—18). — *Прим. перев.*

ГЛАВА 4

АРИФМЕТИКА ПОЛЕЙ ГАЛУА

Важнейшие и наиболее мощные идеи теории кодирования основаны на арифметических системах полей Галуа. Многим из нас эти арифметические системы не известны, и, прежде чем продолжать изложение теории кодирования, нам придется изучить основы этого раздела математики.

В настоящей главе мы возвращаемся к начатому в гл. 2 описанию структуры полей Галуа. Там мы ввели определение поля, но не указали процедур построения полей Галуа, а именно их таблиц сложения и умножения. Мы будем изучать поля Галуа с помощью двух построений, одно из которых основывается на кольце целых чисел, а другое — на кольцах многочленов, и докажем, что таким образом можно построить все поля Галуа.

4.1. КОЛЬЦО ЦЕЛЫХ ЧИСЕЛ

Множество всех целых чисел (положительных, отрицательных и нуля) образуют кольцо относительно обычных операций сложения и умножения. Это кольцо принято обозначать через Z. В данном параграфе мы будем изучать структуру кольца целых чисел.

Говорят, что целое число s *делится* на целое число r или что r *делит* s (или что r является *делителем* s), если $ra = s$, где a — некоторое целое число. Если r и делит s, и делится на s, то $r = \pm s$. Действительно, $r = sa$ и $s = rb$ для некоторых целых чисел a и b; следовательно, $r = rab$ и ab должно равняться 1. Так как a и b — целые, то и a, и b должны быть либо 1, либо -1.

Положительное целое число $p > 1$, которое делится только на $\pm p$ или ± 1, называется *простым*. Положительное целое число, большее 1, не являющееся простым, называется *составным*. *Наибольший общий делитель* двух целых чисел r и s обозначается через НОД (r, s) и определяется как наибольшее положительное число, которое делит оба из них. *Наименьшее общее кратное* двух целых чисел r и s обозначается через НОК (r, s) и определяется как наи-

меньшее положительное число, которое делится на оба из них. Два числа называются *взаимно простыми*, если их наибольший общий делитель равен 1.

В общем случае в кольце целых чисел деление возможно не всегда, зато имеют место две почти столь же важные операции, а именно сокращение и деление с остатком. В силу возможности сокращения кольцо целых чисел является областью целостности. Возможность деления с остатком (известного как алгоритм деления) обычно доказывается с помощью конструктивной процедуры. Мы сформулируем его в виде самоочевидной теоремы.

Теорема 4.1.1 (алгоритм деления). *Для каждой пары целых чисел c и d при отличном от нуля d найдется единственная пара целых чисел Q (частное) и s (остаток), таких, что $c = dQ + s$, где $0 \leqslant s < |d|$.*

Обычно нас будет больше интересовать не частное, а остаток. Мы будем часто записывать остаток в виде равенства

$$s = R_d\,[c],$$

которое читается так: «s является остатком от деления c на d». Другим обозначением является

$$s \equiv c \pmod{d}.$$

Соотношение такого вида называется *сравнением* и читается так: «s сравнимо с c по модулю d». Оно означает, что при делении на d числа s и c имеют один и тот же остаток, но s не обязательно меньше d.

Вычисление остатка от сложного выражения, содержащего сложение и умножение, облегчается тем, что можно менять последовательность выполнения операции вычисления остатка со сложением и умножением. А именно справедливо следующее утверждение.

Теорема 4.1.2.

(i) $R_d\,[a + b] = R_d\,\{R_d\,[a] + R_d\,[b]\}$.

(ii) $R_d\,[a \cdot b] = R_d\,\{R_d\,[a] \cdot R_d\,[b]\}$.

Доказательство предоставляется читателю в качестве упражнения. □

Используя алгоритм деления, можно найти наибольший общий делитель двух целых чисел. Например, НОД (814, 187) находится следующим образом:

$$\begin{aligned} 814 &= 4 \times 187 + 66, \\ 187 &= 2 \times 66 + 55, \\ 66 &= 1 \times 55 + 11, \\ 55 &= 5 \times 11 + 0. \end{aligned}$$

Так как НОД (814, 187) делит и 814, и 187, то он должен делить и остаток 66. Так как он делит и 187, и 66, то он делит и 55. Так как он делит и 66, и 55, то он делит и 11. С другой стороны, 11 делит 55, а поэтому и 66, и 187, и, наконец, также 814. Следовательно, НОД (814, 187) равен 11.

Теперь можно выразить 11 в виде линейной комбинации чисел 814 и 187, начиная снизу выписанной выше последовательности и поступая следующим образом:

$$\begin{aligned} 11 &= 66 - 1 \times 55 = \\ &= 66 - 1 \times (187 - 2 \times 66) = 3 \times 66 - 1 \times 187 = \\ &= 3 \times (814 - 4 \times 187) - 1 \times 187 = 3 \times 814 - 13 \times 187. \end{aligned}$$

Следовательно, мы выразили НОД (814, 187) в виде линейной комбинации чисел 814 и 187 с коэффициентами из кольца целых чисел, а именно

$$\text{НОД}\,(814,\ 187) = 3 \times 814 - 13 \times 187.$$

Эти рассуждения могут быть проведены в общем виде для произвольных целых чисел r и s и позволяют доказать приведенные ниже теорему и следствие.

Теорема 4.1.3 (алгоритм Евклида). *Наибольший общий делитель двух различных ненулевых целых чисел r и s может быть вычислен итеративным применением алгоритма деления. Предположим, что $r < s$ и оба эти числа положительны; тогда алгоритм состоит в следующем:*

$$\begin{aligned} s &= Q_1 r + r_1, \\ r &= Q_2 r_1 + r_2, \\ r_1 &= Q_3 r_2 + r_3, \\ &\vdots \\ r_{n-1} &= Q_{n+1} r_n, \end{aligned}$$

и процесс заканчивается, когда полученный остаток равен нулю. Последний ненулевой остаток r_n равен наибольшему общему делителю.

Наконец, мы приходим к важному и интуитивно не очевидному результату теории чисел.

Следствие 4.1.4. *Для любых целых чисел r и s существуют целые числа a и b, такие, что*

$$\text{НОД}\,(r,\ s) = ar + bs.$$

Доказательство. Последний остаток в теореме 4.1.3 равен НОД (r, s). Воспользуемся множеством выписанных в этой теореме уравнений, чтобы исключить все остальные остатки. Это даст выражение для r_n в виде линейной комбинации r и s с целочисленными коэффициентами. □

4.2. КОНЕЧНЫЕ ПОЛЯ, ОСНОВАННЫЕ НА КОЛЬЦЕ ЦЕЛЫХ ЧИСЕЛ

Имеется очень важная конструкция, позволяющая по заданному кольцу построить новое кольцо, называемое *кольцом отношений*. В случае произвольного кольца для построения кольца отношений строятся смежные классы, однако в случае кольца целых чисел кольцо отношений строится просто. В некоторых случаях это построение приводит к полям (в случае, когда кольцо является областью целостности).

Определение 4.2.1. Пусть q — положительное целое число. Кольцом отношений, именуемым также кольцом целых чисел по модулю q и обозначаемым через $Z/(q)$, называется множество $\{0, 1, \ldots, q-1\}$ с операциями сложения и умножения, определяемыми равенствами $a + b = R_q[a + b]$, $a \cdot b = R_q[ab]$.

Элементы, обозначенные через 0, 1, ..., $q - 1$, принадлежат как Z, так и $Z/(q)$. Пожалуй, лучше под элементами из $Z/(q)$ понимать не первые q элементов из Z, а некоторые другие объекты, обозначенные таким же образом. Произвольный элемент a из Z можно отобразить в $Z/(q)$, полагая $a' = R_q[a]$. Два элемента a и b из Z, отображаемые в один элемент из $Z/(q)$, сравнимы по модулю q и $a = b + mq$ для некоторого целого m.

Теорема 4.2.2. *Кольцо отношений $Z/(q)$ является кольцом.*

Доказательство предоставляется читателю в качестве упражнения. □

Как показывают примеры из § 2.4, арифметику полей $GF(2)$ и $GF(3)$ можно описать как сложение и умножение по модулю 2 и 3 соответственно, а арифметику в поле $GF(4)$ так описать нельзя. Таким образом, в принятой нами символике $GF(2) = Z/(2)$, $GF(3) = Z/(3)$, $GF(4) \neq Z/(4)$. Общий результат дается следующей теоремой.

Теорема 4.2.3. *Кольцо отношений $Z/(q)$ является полем тогда и только тогда, когда q равно простому числу.*

Доказательство. Предположим, что q — простое число. Для доказательства того, что кольцо является полем, надо показать, что каждый ненулевой элемент имеет мультипликативный обрат-

ный. Пусть s — ненулевой элемент кольца; тогда $1 \leqslant s \leqslant q-1$. Так как q просто, то НОД $(s, q) = 1$, и в силу следствия 4.1.4

$$1 = aq + bs$$

для некоторых a и b. Таким образом,

$$1 = R_q[1] = R_q[aq + bs] = R_q\{R_q[aq] + R_q[bs]\} =$$

$$= R_q[bs] = R_q\{R_q[b] \cdot R_q[s]\} = R_q\{R_q[b]\, s\}.$$

Следовательно, элемент $R_q[b]$ является мультипликативным обратным элементу s относительно операции умножения по модулю q.

Теперь допустим, что q — составное число. Тогда $q = rs$. Если данное кольцо представляет собой поле, то r имеет обратный элемент r^{-1}, и поэтому

$$s = R_q[s] = R_q[r^{-1}rs] = R_q[r^{-1}q] = 0.$$

Но $s \neq 0$, так что мы получили противоречие. Следовательно, рассматриваемое кольцо не является полем. □

В случае когда кольцо отношений $Z/(q)$ образует поле, оно также обозначается через $GF(q)$, чтобы подчеркнуть тот факт, что оно является полем.

4.3. КОЛЬЦА МНОГОЧЛЕНОВ

Многочленом над полем $GF(q)$ называется математическое выражение

$$f(x) = f_{n-1}x^{n-1} + f_{n-2}x^{n-2} + \cdots + f_1x + f_0,$$

где символ x называется *неопределенной переменной*, коэффициенты $f_{n-1}, \ldots, f_0$ принадлежат полю $GF(q)$, а индексы и показатели степеней являются целыми числами. *Нулевым многочленом* называется многочлен $f(x) = 0$. *Приведенным многочленом* называется многочлен, старший коэффициент f_{n-1} которого равен 1. Два многочлена равны, если равны все их коэффициенты f_i.

Степенью ненулевого многочлена $f(x)$ называется индекс старшего коэффициента f_{n-1}; степень многочлена $f(x)$ обозначается через $\deg f(x)$. Степень ненулевого многочлена всегда конечна. Степень нулевого многочлена по соглашению полагается равной отрицательной бесконечности $(-\infty)$.

Множество всех многочленов над полем $GF(q)$ образует кольцо относительно сложения и умножения, определяемых по обычным правилам сложения и умножения многочленов. Такое полиномиальное кольцо можно определить для каждого поля Галуа $GF(q)$. Это кольцо обозначается через $GF(q)[x]$. В исследованиях по кольцам $GF(q)[x]$ элементы поля $GF(q)$ иногда называются *скалярами*.

Суммой двух многочленов $f(x)$ и $g(x)$ из $GF(q)[x]$ называется многочлен из $GF(q)[x]$, определяемый равенством

$$f(x) + g(x) = \sum_{i=0}^{\infty} (f_i + g_i) x^i,$$

где, конечно, члены с индексом, бо́льшим наибольшей из степеней многочленов $f(x)$ и $g(x)$, равны нулю. Степень суммы не превосходит наибольшей из этих двух степеней. Например, над $GF(2)$

$$(x^3 + x^2 + 1) + (x^2 + x + 1) =$$
$$= x^3 + (1 + 1) x^2 + x + (1 + 1) = x^3 + x.$$

Произведением двух многочленов из $GF(q)[x]$ называется многочлен из $GF(q)[x]$, определяемый равенством

$$f(x) g(x) = \sum_{i} \left(\sum_{j=0}^{i} f_j g_{i-j} \right) x^i.$$

Например, над $GF(2)$

$$(x^3 + x^2 + 1)(x^2 + x + 1) = x^5 + x + 1.$$

Степень произведения равна сумме степеней множителей.

Кольцо многочленов во многих отношениях аналогично кольцу целых чисел. Чтобы сделать эту аналогию очевидной, в изложении данного параграфа мы следуем § 4.1. Скажем, что многочлен $s(x)$ *делится* на многочлен $r(x)$ или что $r(x)$ *делит* $s(x)$, если существует многочлен $a(x)$, такой, что $r(x)\, a(x) = s(x)$. Многочлен $p(x)$, делящийся только на многочлены $\alpha p(x)$ или α, где α — произвольный ненулевой элемент поля $GF(q)$, называется *неприводимым многочленом*. Приведенный неприводимый многочлен называется *простым многочленом*.

Наибольший общий делитель двух многочленов $r(x)$ и $s(x)$ обозначается через НОД $[r(x), s(x)]$ и определяется как приведенный многочлен наибольшей степени, делящий одновременно оба из них. *Наименьшее общее кратное двух многочленов* $r(x)$ и $s(x)$ обозначается через НОК $[r(x), s(x)]$ и определяется как приведенный многочлен наименьшей степени, делящийся на оба из них. Как мы увидим, наибольший общий делитель и наименьшее общее кратное определены единственным образом, так что наше определение корректно. Если наибольший общий делитель двух многочленов равен 1, то они называются *взаимно простыми*.

Если $r(x)$ одновременно делится на $s(x)$ и делит $s(x)$, то $r(x) = \alpha s(x)$, где α — элемент поля $GF(q)$. Это доказывается следующим образом. Должны существовать многочлены $a(x)$ и $b(x)$, такие, что $r(x) = s(x)\, a(x)$ и $s(x) = r(x)\, b(x)$; следовательно, $r(x) = r(x)\, b(x)\, a(x)$. Но степень правой части равна сумме степеней $r(x)$, $b(x)$ и $a(x)$. Так как эта величина должна быть рав-

ной степени левой части, то $b(x)$ и $a(x)$ должны иметь нулевую степень и, таким образом, являться скалярами.

Для многочленов над полем вещественных чисел очень полезна (и элементарно вводится) операция дифференцирования. В случае многочленов над конечным полем определение дифференцирования в смысле операции предельного перехода невозможно. Тем не менее удобно определить операцию над многочленами, результат которой ведет себя так, как вела бы производная. Такой многочлен называется *формальной производной* от многочлена.

Определение 4.3.1. Пусть $r(x) = r_{n-1}x^{n-1} + r_{n-2}x^{n-2} + \cdots + r_1x + r_0$ — многочлен над $GF(q)$. Формальная производная от $r(x)$ определяется как многочлен вида

$$r'(x) = ((n-1))\,r_{n-1}x^{n-2} + ((n-2))\,r_{n-2}x^{n-3} + \cdots + r_1,$$

где коэффициенты ((i)) называются *числами поля* $GF(q)$ и вычисляются как сумма i членов в поле $GF(q)$:

$$((i)) = 1 + 1 + \cdots + 1.$$

Легко проверить, что сохраняются многие полезные свойства производных, а именно что

$$[r(x)\,s(x)]' = r'(x)\,s(x) + r(x)\,s'(x)$$

и что если $a^2(x)$ делит $r(x)$, то $a(x)$ делит $r'(x)$.

В кольце многочленов, так же как и в кольце целых чисел, деление в общем случае невозможно. Однако для многочленов над полями тоже имеют место сокращение и деление с остатком. Алгоритм деления для многочленов дается следующим утверждением.

Теорема 4.3.2 (алгоритм деления для многочленов). *Для каждой пары многочленов $c(x)$ и $d(x)$, $d(x) \neq 0$, существует единственная пара многочленов $Q(x)$ (частное) и $s(x)$ (остаток), таких, что*

$$c(x) = Q(x)\,d(x) + s(x)$$

и $\deg s(x) < \deg d(x)$.

Доказательство. Частное и остаток находятся по элементарному правилу деления многочленов. Они единственны, так как если

$$c(x) = d(x)\,Q_1(x) + s_1(x) = d(x)\,Q_2(x) + s_2(x),$$

то

$$d(x)\,[Q_1(x) - Q_2(x)] = s_2(x) - s_1(x).$$

В правой части стоит ненулевой многочлен, степень которого меньше $\deg d(x)$, а в левой — ненулевой многочлен, степень которого не меньше $\deg d(x)$. Следовательно, оба многочлена равны нулю, и представление единственно. □

Практическое вычисление частного и остатка выполняется с помощью простого правила деления многочленов «уголком». Обычно мы будем больше интересоваться не частным, а остатком. Остаток можно также записать в виде $s(x) = R_{d(x)}[c(x)]$. Часто остаток называют *вычетом* многочлена $c(x)$ по модулю многочлена $d(x)$. Несколько отличным понятием является сравнение

$$s(x) \equiv c(x) \pmod{d(x)},$$

которое означает, что при делении на $d(x)$ многочлены $s(x)$ и $c(x)$ дают один и тот же остаток, но степень многочлена $c(x)$ не обязательно меньше степени многочлена $d(x)$.

Иногда при вычислении остатка удобнее разбивать деление на этапы. Это можно осуществить с помощью следующей теоремы.

Теорема 4.3.3. *Пусть $d(x)$ кратен многочлену $g(x)$. Тогда для любого $a(x)$*

$$R_{g(x)}[a(x)] = R_{g(x)}[R_{d(x)}[a(x)]].$$

Доказательство. Пусть $d(x) = g(x)h(x)$ для некоторого $h(x)$. Раскрывая правую часть, получаем

$$\begin{aligned} a(x) &= Q_1(x)d(x) + R_{d(x)}[a(x)] = \\ &= Q_1(x)g(x)h(x) + Q_2(x)g(x) + R_{g(x)}[R_{d(x)}[a(x)]], \end{aligned}$$

где степень остатка меньше $\deg g(x)$. Раскрывая левую часть, имеем

$$a(x) = Q(x)g(x) + R_{g(x)}[a(x)],$$

и, согласно алгоритму деления, такая запись однозначна при степени остатка, меньшей $\deg g(x)$. Теорема вытекает из отождествления подобных членов в обоих выражениях. □

Теорема 4.3.4.

(i) $R_{d(x)}[a(x) + b(x)] = R_{d(x)}[a(x)] + R_{d(x)}[b(x)]$,

(ii) $R_{d(x)}[a(x)\cdot b(x)] = R_{d(x)}\{R_{d(x)}[a(x)]\cdot R_{d(x)}[b(x)]\}$.

Доказательство сводится к упражнению: использовать алгоритм деления для выражений в сбеих частях равенства и приравнять остатки. □

Подобно тому как часто бывает полезным представление положительных целых чисел в виде произведения простых сомножителей, часто бывает полезным представление приведенных многочленов в виде произведения простых многочленов.

Теорема 4.3.5 (теорема об однозначном разложении). *Ненулевой многочлен $p(x)$ над некоторым полем однозначно (с точностью до порядка следования множителей) разлагается в произведение элемента поля и простых многочленов над этим полем.*

Доказательство. Ясно, что входящим в произведение элементом поля должен быть коэффициент p_{n-1}, где $n-1$ — степень многочлена $p(x)$. Можно пренебречь этим элементом и доказывать теорему для приведенных многочленов.

Предположим, что теорема не верна. Пусть $p(x)$ — приведенный многочлен наименьшей степени, для которого не верна теорема. Тогда имеются два разложения:

$$p(x) = a_1(x)\, a_2(x) \ldots a_K(x) = b_1(x)\, b_2(x) \ldots b_J(x),$$

где $a_k(x)$ и $b_j(x)$ — простые многочлены.

Все многочлены $a_k(x)$ должны отличаться от всех многочленов $b_j(x)$, так как в противном случае можно было бы сократить общие члены и получить многочлен меньшей степени, который можно разложить двумя разными способами.

Без потери общности предположим, что степень многочлена $b_1(x)$ не больше степени многочлена $a_1(x)$. Тогда

$$a_1(x) = b_1(x)\, h(x) + s(x),$$

где $\deg s(x) < \deg b_1(x) \leqslant \deg a_1(x)$. Далее,

$$s(x)\, a_2(x)\, a_3(x) \ldots a_K(x) = \\ = b_1(x)\, [b_2(x) \ldots b_J(x) - h(x)\, a_2(x) \ldots a_K(x)].$$

Разложим многочлен $s(x)$ и многочлен, стоящий в квадратных скобках, на простые множители, разделив, если надо, на соответствующий элемент поля так, чтобы все множители были приведенными. Поскольку $b_1(x)$ не содержится в левой части, мы получили два различных разложения приведенного многочлена, степень которого меньше степени $p(x)$. Это противоречие доказывает теорему. □

В силу теоремы об однозначном разложении теперь ясно, что для любых двух многочленов $r(x)$ и $s(x)$ НОД $[r(x), s(x)]$ и НОК $[r(x), s(x)]$ являются единственными, так как наибольший общий делитель равен произведению всех общих для $r(x)$ и $s(x)$ простых делителей, причем каждый делитель входит в наименьшей из степеней, в которых он входит в $r(x)$ и в $s(x)$, а наименьшее общее, кратное равно произведению всех простых делителей, входящих либо в $r(x)$, либо в $s(x)$, причем каждый делитель входит в наибольшей из степеней, в которых он входит в $r(x)$ или в $s(x)$. Далее, любой многочлен, делящий как $r(x)$, так и $s(x)$, делит НОД $[r(x), s(x)]$, а любой многочлен, делящийся и на $r(x)$, и на $s(x)$, делится на НОК $[r(x), s(x)]$.

Из алгоритма деления многочленов вытекает важное следствие, известное под названием алгоритма Евклида для многочленов.

Теорема 4.3.6 (алгоритм Евклида для многочленов). *Наибольший общий делитель двух многочленов $r(x)$ и $s(x)$ над*

полем GF (q) можно вычислить с помощью итеративного применения алгоритма деления. Если $\deg s(x) \geqslant \deg r(x) \geqslant 0$, *то последовательность вычислений такова:*

$$\begin{aligned} s(x) &= Q_1(x)\, r(x) + r_1(x), \\ r(x) &= Q_2(x)\, r_1(x) + r_2(x), \\ r_1(x) &= Q_3(x)\, r_2(x) + r_3(x), \\ &\vdots \\ r_{n-1}(x) &= Q_{n+1}(x)\, r_n(x), \end{aligned}$$

и процесс обрывается, как только получается равный нулю остаток. Тогда $r_n(x) = \alpha$ НОД $[r(x), s(x)]$, *где* α *— некоторый скаляр.*

Доказательство. Отправляясь от первого уравнения, видим, что НОД $[r(x), s(x)]$ делит и делимое, и делитель, и, следовательно, остаток. Проводя это рассуждение далее по всем уравнениям, видим, что НОД $[r(x), s(x)]$ делит $r_n(x)$. Отправляясь от последнего уравнения, видим, что $r_n(x)$ делит делитель и остаток, так что он делит и делимое. Проводя это рассуждение по всем уравнениям вплоть до первого, видим, что $r_n(x)$ делит НОД $[r(x), s(x)]$. Так как $r_n(x)$ делит НОД $[r(x), s(x)]$ и делится на него, то получаем утверждение теоремы. □

Следствие 4.3.7. НОД $[r(x), s(x)] = a(x)\, r(x) + b(x)\, s(x)$, *где* $a(x)$ *и* $b(x)$ *— многочлены над GF (q).*

Доказательство. Последнее уравнение с ненулевым остатком в утверждении предыдущей теоремы дает выражение многочлена $r_n(x)$ через $r_{n-1}(x)$ и $r_{n-2}(x)$. Перебирая уравнения снизу вверх, исключаем сначала $r_{n-1}(x)$, затем $r_{n-2}(x)$ и т. д. и в конце концов получим выражение $r_n(x)$ только через $r(x)$ и $s(x)$. □

Для произвольного элемента β поля $GF(q)$ можно вычислить значение многочлена над $GF(q)$ в этой точке, подставив элемент β вместо неопределенной переменной. Например, пусть над GF (3)

$$p(x) = 2x^5 + x^4 + x^2 + 2.$$

Тогда (см. рис. 4.1) получаем

$$\begin{aligned} p(0) &= 2 \cdot 0^5 + 0^4 + 0^2 + 2 = 2, \\ p(1) &= 2 \cdot 1^5 + 1^4 + 1^2 + 2 = 0, \\ p(2) &= 2 \cdot 2^5 + 2^4 + 2^2 + 2 = 2. \end{aligned}$$

В случае поля вещественных чисел известной процедурой является вычисление значений многочлена в расширении этого поля; широко применяется вычисление значений многочлена с вещественными коэффициентами в поле комплексных чисел. Аналогично

GF(2)

+	0	1
0	0	1
1	1	0

·	0	1
0	0	0
1	0	1

GF(3)

+	0	1	2
0	0	1	2
1	1	2	0
2	2	0	1

·	0	1	2
0	0	0	0
1	0	1	2
2	0	2	1

GF(4)

+	0	1	2	3
0	0	1	2	3
1	1	0	3	2
2	2	3	0	1
3	3	2	1	0

·	0	1	2	3
0	0	0	0	0
1	0	1	2	3
2	0	2	3	1
3	0	3	1	2

Рис. 4.1. Примеры конечных полей.

можно вычислять значения многочлена над $GF(q)$ в расширении поля $GF(q)$. Такое вычисление проводится подстановкой элементов из расширения поля вместо неопределенной переменной x и выполнения операций в расширении поля. Например, пусть над $GF(2)$

$$p(x) = x^3 + x + 1.$$

Тогда (см. рис. 4.1) для элементов поля $GF(4)$ имеем

$$p(0) = 0^3 + 0 + 1 = 1,$$
$$p(1) = 1^3 + 1 + 1 = 1,$$
$$p(2) = 2^3 + 2 + 1 = 2,$$
$$p(3) = 3^3 + 3 + 1 = 3.$$

Если $p(\beta) = 0$, то элемент β называется *корнем* многочлена $p(x)$ или *корнем* уравнения $p(x) = 0$. Многочлен не обязательно имеет корни в своем собственном поле. Многочлен $x^3 + x + 1$ не имеет корней в $GF(2)$, а также в $GF(4)$.

Теорема 4.3.8. *Элемент β является корнем многочлена $p(x)$ тогда и только тогда, когда $x - \beta$ делит $p(x)$. Более того, корнями многочлена $p(x)$ степени n являются не более n элементов поля.*

Доказательство. Согласно алгоритму деления,

$$p(x) = (x - \beta)\, Q(x) + s(x),$$

где степень $s(x)$ меньше единицы. Таким образом, $s(x)$ является элементом поля s_0. Следовательно, $0 = p(\beta) = (\beta - \beta)\, Q(\beta) + s_0$, и соответственно $s(x) = s_0 = 0$.

Обратно, если $x - \beta$ делит $p(x)$, то

$$p(x) = (x - \beta)\, Q(x),$$

так что $p(\beta) = (\beta - \beta)\, Q(\beta) = 0$, и, таким образом, β — корень многочлена $p(x)$.

Разложим теперь многочлен $p(x)$ в произведение элемента поля и простых множителей. Степень многочлена $p(x)$ равна сумме степеней простых множителей, и для каждого корня имеется один такой множитель. Следовательно, существует не более n корней. □

4.4. КОНЕЧНЫЕ ПОЛЯ, ОСНОВАННЫЕ НА КОЛЬЦАХ МНОГОЧЛЕНОВ

Конечные поля можно построить из колец многочленов таким же образом, каким были построены поля из кольца целых чисел. Пусть имеется кольцо многочленов $F[x]$ над полем F. Так же, как были построены для кольца $\mathbb{Z}$ кольца отношений, можно построить и кольца отношений для кольца $F[x]$. Выбирая из $F[x]$ произвольный многочлен $p(x)$, можно определить кольцо отношений, используя $p(x)$ в качестве модуля для задания арифметики этого кольца. Мы ограничимся рассмотрением только приведенных многочленов, так как это ограничение снимает ненужную неопределенность построения.

Определение 4.4.1. Для произвольного приведенного многочлена $p(x)$ ненулевой степени над полем F *кольцом многочленов по модулю* $p(x)$ называется множество всех многочленов над F, степень которых не превосходит степени многочлена $p(x)$, с операциями сложения и умножения многочленов по модулю $p(x)$. Это кольцо принято обозначать через $F(x)/(p(x))$.

Произвольный элемент $r(x)$ кольца $F[x]$ можно отобразить в элемент кольца $F[x]/(p(x))$ с помощью соответствия $r(x) \to R_{p(x)}[r(x)]$. Два элемента $a(x)$ и $b(x)$ из $F[x]$, отображаемые в один и тот же элемент из $F[x]/(p(x))$, называются сравнимыми:

$$a(x) \equiv b(x) \pmod{p(x)}.$$

Тогда $b(x) = a(x) + Q(x)\, p(x)$ для некоторого многочлена $Q(x)$.

Теорема 4.4.2. *Множество* $F[x]/(p(x))$ *является кольцом.*

Доказательство предоставляется читателю в качестве упражнения. □

Выберем в кольце многочленов над $GF(2)$, например, многочлен $p(x) = x^3 + 1$. Тогда кольцо многочленов по модулю $p(x)$ равно $GF(2)[x]/(x^3 + 1)$. Оно состоит из элементов $\{0, 1, x,$

$x+1$, x^2, x^2+1, x^2+x, $x^2+x+1\}$. В этом кольце умножение выполняется, например, следующим образом:

$$(x^2+1)\cdot(x^2) = R_{x^3+1}[(x^2+1)\cdot x^2] =$$
$$= R_{x^3+1}[x(x^3+1)+x^2+x] = x^2+x,$$

где использована редукция по правилу $x^4 = x(x^3+1)+x$.

Теорема 4.4.3. *Кольцо многочленов по модулю приведенного многочлена $p(x)$ является полем тогда и только тогда, когда многочлен $p(x)$ прост* [1].

Доказательство. Пусть многочлен $p(x)$ прост. Чтобы доказать, что рассматриваемое кольцо образует поле, достаточно показать, что каждый ненулевой элемент имеет мультипликативный обратный. Пусть $s(x)$ — некоторый ненулевой элемент кольца. Тогда $\deg s(x) < \deg p(x)$. Так как многочлен $p(x)$ прост, то НОД $[s(x), p(x)] = 1$. По следствию 4.3.7

$$1 = a(x)\,p(x) + b(x)\,s(x)$$

для некоторых многочленов $a(x)$ и $b(x)$. Следовательно,

$$1 = R_{p(x)}[1] = R_{p(x)}[a(x)p(x)+b(x)s(x)] =$$
$$= R_{p(x)}\{R_{p(x)}[b(x)]\cdot R_{p(x)}[s(x)]\} = R_{p(x)}\{R_{p(x)}[b(x)]\cdot s(x)\}.$$

Таким образом, в кольце многочленов по модулю $p(x)$ многочлен $R_{p(x)}[b(x)]$ является мультипликативным обратным к $s(x)$.

Предположим теперь, что степень многочлена $p(x)$ равна по меньшей мере 2 и что он не прост. Тогда $p(x) = r(x)\,s(x)$ для некоторых $r(x)$ и $s(x)$, степени которых равны по меньшей мере 1. Если кольцо является полем, то многочлен $r(x)$ имеет обратный $r^{-1}(x)$, и поэтому

$$s(x) = R_{p(x)}[s(x)] = R_{p(x)}[r^{-1}(x)\,r(x)\,s(x)] =$$
$$= R_{p(x)}[r^{-1}(x)\,p(x)] = 0.$$

Но $s(x) \neq 0$, и мы получаем противоречие. Следовательно, такое кольцо не может быть полем. □

Если над полем $GF(q)$ найден простой многочлен степени n, то, используя развитую в данном параграфе теорию, можно построить поле Галуа, содержащее q^n элементов. В этом построении элементы поля представляются многочленами над $GF(q)$ степени не выше $n-1$. Всего существует q^n таких многочленов, и, следовательно, их столько же, сколько элементов в поле.

---

[1] Напомним, что простой многочлен является одновременно неприводимым и приведенным. Для построения поля достаточно только неприводимости $p(x)$, но мы условились рассматривать только приведенные многочлены, так что дальнейшие результаты носят менее общий характер.

Многочленные обозначения	Двоичные обозначения	Целочисленные обозначения	Степенные обозначения
0	00	0	0
1	01	1	x^0
x	10	2	x^1
$x+1$	11	3	x^2

а

+	0	1	x	$x+1$
0	0	1	x	$x+1$
1	1	0	$x+1$	x
x	x	$x+1$	0	1
$x+1$	$x+1$	x	1	0

•	0	1	x	$x+1$
0	0	0	0	0
1	0	1	x	$x+1$
x	0	x	$x+1$	1
$x+1$	0	$x+1$	1	x

б

Рис. 4.2. Структура поля *GF* (4). *а* — представления поля *GF* (4); *б* — арифметические таблицы.

В качестве примера построим поле *GF* (4) по полю *GF* (2), используя примитивный многочлен $p(x) = x^2 + x + 1$. Перебирая все возможные разложения, легко проверить неприводимость этого многочлена. Элементы поля задаются многочленами $\{0, 1, x, x+1\}$. Приведенные на рис. 4.2 таблицы сложения и умножения строятся по готовым правилам. Конечно, после того как арифметические таблицы построены, можно заменить многочленные обозначения на целочисленные или другие желаемые обозначения.

Таблица 4.1 содержит список простых многочленов над *GF* (2). Одним из способов проверки простоты этих многочленов является метод проб и ошибок, т. е. непосредственная проверка всех возможных разложений, хотя для многочленов высоких степеней для этого потребуется ЭВМ. Собранные в табл. 4.1 простые многочлены представляют собой специальные частные случаи простых многочленов, известных под названием примитивных многочленов. Как будет описано в следующем параграфе, они дают наиболее удобное представление расширения поля.

В заключение параграфа подытожим, где мы находимся. Мы разработали необходимые для получения полей построения, которые будут использованы в дальнейшем, но для полного понимания предмета необходимы еще некоторые сведения. В частности, необходимо установить следующие факты: 1) над каждым полем Галуа существуют простые многочлены любой заданной степени; 2) разработанные построения достаточны для получения всех

Таблица 4.1

Простые многочлены над *GF* (2)

Степень	Простые многочлены
2	x^2+x+1
3	x^3+x+1
4	x^4+x+1
5	x^5+x^2+1
6	x^6+x+1
7	x^7+x^3+1
8	$x^8+x^4+x^3+x^2+1$
9	x^9+x^4+1
10	$x^{10}+x^3+1$
11	$x^{11}+x^2+1$
12	$x^{12}+x^6+x^4+x+1$
13	$x^{13}+x^4+x^3+x+1$
14	$x^{14}+x^{10}+x^6+x+1$
15	$x^{15}+x+1$
16	$x^{16}+x^{12}+x^3+x+1$
17	$x^{17}+x^3+1$
18	$x^{18}+x^7+1$
19	$x^{19}+x^5+x^2+x+1$
20	$x^{20}+x^3+1$
21	$x^{21}+x^2+1$
22	$x^{22}+x+1$
23	$x^{23}+x^5+1$
24	$x^{24}+x^7+x^2+x+1$
25	$x^{25}+x^3+1$
26	$x^{26}+x^6+x^2+x+1$
27	$x^{27}+x^5+x^2+x+1$
28	$x^{28}+x^3+1$

Примечание. Все многочлены являются примитивными.

полей Галуа — других полей нет[1]); 3) в каждом поле имеются некоторые предпочтительные, так называемые примитивные элементы.

На рис. 4.3 дается сводка наиболее существенных результатов, относящихся к полям Галуа. Остальная часть главы посвящена

[1]) Математическая строгость требует здесь большего формализма, и надо было бы сказать, что нет других полей с точностью до изоморфизма. Неформально это означает, что любые два поля Галуа с одинаковым числом элементов являются двумя различными представлениями одного и того же поля. Иллюзия другой структуры может быть, например, создана перестановкой тех же самых символов.

1. Число элементов любого поля Галуа равно степени простого числа.
2. Для любого простого p и целого положительного m наименьшим подполем поля $GF(p^m)$ является поле $GF(p)$ Элементы поля $GF(p)$ называются *целыми числами* поля $GF(p^m)$, а число p — его *характеристикой*.
3. В поле Галуа характеристики 2 для каждого элемента β поля выполняется равенство $-\beta=\beta$.
4. Для любого простого p и целого положительного m существует поле Галуа с p^m элементами.
5. Каждое поле Галуа $GF(q)$ содержит хотя бы один примитивный элемент.
6. Над каждым полем Галуа существует хотя бы один примитивный многочлен любой положительной степени.
7. Каждый примитивный элемент имеет над любым подполем простой минимальный многочлен.
8. Два поля Галуа с одним и тем же числом элементов изоморфны.
9. Для любого q, являющегося степенью простого числа, и любого положительного целого m поле $GF(q)$ является подполем в $GF(q^m)$, а $GF(q^m)$ является расширением поля $GF(q)$.
10. Если n не делит m, то $GF(q^n)$ не является подполем поля $GF(q^m)$.
11. Для любого элемента поля $GF(q^m)$ степень минимального многочлена над $GF(q)$ является делителем m.

Рис. 4.3. Некоторые основные свойства полей Галуа.

доказательству большинства из этих результатов и введению новых понятий. Доказательство существования примитивных многочленов мы откладываем до конца § 5.3.

4.5. ПРИМИТИВНЫЕ ЭЛЕМЕНТЫ

В предыдущем параграфе было построено поле $GF(4)$. На рис. 4.2 видно, что элемент поля, представляемый многочленом x, можно выбрать как бы в качестве некоторого основания логарифмов. За исключением нуля, все элементы поля могут быть представлены в виде степени элемента x.

Определение 4.5.1. ***Примитивным элементом*** **поля $GF(q)$ называется такой элемент α, что все элементы поля, за исключением нуля, могут быть представлены в виде степени элемента α.**

Например, в поле $GF(5)$ $2^1 = 2$, $2^2 = 4$, $2^3 = 3$, $2^4 = 1$, так что 2 является примитивным элементом поля $GF(5)$. Примитивные элементы полезны при построении полей, так как если один из них найден, то, перемножая степени этого примитивного элемента, можно построить таблицу умножения в поле. В данном параграфе будет доказано, что каждое поле содержит примитивный элемент.

Поле образует абелеву группу двояким способом. Множество всех элементов поля образует абелеву группу по сложению, и множество всех элементов поля, исключая нуль, образует абелеву группу по умножению. Мы будем работать с группой по умножению. Согласно теореме 2.2.5, порядок этой группы делится на порядок каждого ее элемента.

Теорема 4.5.2. *Пусть* $\beta_1, \beta_2, \ldots, \beta_{q-1}$ — *ненулевые элементы поля* $GF(q)$; *тогда*

$$x^{q-1} - 1 = (x - \beta_1)(x - \beta_2) \ldots (x - \beta_{q-1}),$$

Доказательство. Множество ненулевых элементов поля $GF(q)$ образует конечную группу по умножению. Пусть β — какой-либо ненулевой элемент из $GF(q)$, и пусть h — порядок этого элемента по умножению. Тогда, согласно теореме 2.2.5, h делит $q-1$. Следовательно,

$$\beta^{q-1} = (\beta^h)^{(q-1)/h} = 1^{(q-1)/h} = 1,$$

так что β является корнем многочлена $x^{q-1} - 1$. □

Теорема 4.5.3. *Группа ненулевых элементов поля* $GF(q)$ *по умножению является циклической.*

Доказательство. Если число $q-1$ простое, то теорема тривиальна, ибо порядок всех элементов, за исключением нуля и единицы, равен $q-1$, так что каждый такой элемент примитивен. Доказывать теорему надо только для составного числа $q-1$. Рассмотрим разложение числа $q-1$ на простые множители:

$$q - 1 = \prod_{i=1}^{s} p_i^{\nu_i}.$$

Так как $GF(q)$ — поле, то среди его $q-1$ ненулевых элементов должен найтись хотя бы один, не являющийся корнем многочлена $x^{(q-1)/p_i} - 1$, поскольку этот многочлен может иметь не более $(q-1)/p_i$ корней. Следовательно, для каждого i можно найти такой ненулевой элемент a_i поля $GF(q)$, что $a_i^{(q-1)/p_i} \neq 1$. Пусть $b_i = a_i^{(q-1)/p_i^{\nu_i}}$, и пусть $b = \prod_{i=1}^{s} b_i$ Докажем,что порядок b равен $q-1$ и, следовательно, группа является циклической.

Шаг 1. Порядок элемента b_i равен $p_i^{\nu_i}$. *Доказательство.* Очевидно, что $b_i^{p_i^{\nu_i}} = 1$, так что порядок элемента b_i делит $p_i^{\nu_i}$. Он равен числу вида $p_i^{n_i}$. Если n_i меньше ν_i, то $b_i^{p_i^{\nu_i-1}} = 1$. Но $b_i^{p_i^{\nu_i-1}} = a_i^{(q-1)/p_i} \neq 1$, и, следовательно, порядок элемента b_i равен $p_i^{\nu_i}$.

Шаг 2. Порядок элемента b равен $q-1$. *Доказательство.* Предположим, что $b^n = 1$. Покажем сначала, что из этого следует равенство $n = 0 \pmod{p_i^{\nu_i}}$ для всех $i = 1, \ldots, s$. Для каждого i можно записать

$$b^{\left(n \prod_{j \neq i} p_j^{\nu_j}\right)} = 1.$$

Заменяя b на $\prod_{i=1}^{s} b_i$ и используя равенство $b_j^{p_j^{\nu_j}} = 1$, находим

$$b^{\left(n \prod_{j \neq i} p_j^{\nu_j}\right)} = 1.$$

Таким образом,

$$n \prod_{j \neq i} p_j^{\nu_j} = 0 \pmod{p_i^{\nu_i}}.$$

Поскольку p_i являются различными простыми числами, то $n = 0 \pmod{p_i^{\nu_i}}$ для каждого i. Следовательно, $n = \prod_{i=1}^{s} p_i^{\nu_i}$. Теорема доказана. □

Эта теорема дает важнейший ключ к пониманию структуры полей Галуа, а именно следующее утверждение.

Теорема 4.5.4. *В каждом поле Галуа имеется примитивный элемент.*

Доказательство. Так как ненулевые элементы поля $GF(q)$ образуют циклическую группу, то среди них имеется элемент порядка $q-1$. Этот элемент является примитивным. □

Использование примитивного элемента для умножения в поле иллюстрируется следующими примерами.

В поле $GF(8)$ порядок каждого ненулевого элемента делит 7. Так как 7 — простое число, то каждый элемент, за исключением нуля и единицы, имеет порядок, равный 7, и, следовательно, примитивен. Поле $GF(8)$ можно построить с помощью многочлена

$p(z) = z^3 + z + 1$. Основываясь на примитивном элементе $\alpha = z$, имеем

$$\begin{aligned}
\alpha &= z,\\
\alpha^2 &= z^2,\\
\alpha^3 &= z + 1,\\
\alpha^4 &= z^2 + z,\\
\alpha^5 &= z^2 + z + 1,\\
\alpha^6 &= z^2 + 1,\\
\alpha^7 &= 1 = \alpha^0.
\end{aligned}$$

В таком представлении умножение выполняется легко; например, $\alpha^4 \cdot \alpha^5 = \alpha^7 \cdot \alpha^2 = \alpha^2$.

Порядок каждого элемента в поле $GF(16)$ делит 15. Элемент может иметь порядок 1, 3, 5 или 15. Поле $GF(16)$ можно построить с помощью многочлена $p(z) = z^4 + z + 1$ и примитивного элемента $\alpha = z$; имеем

$$\begin{aligned}
\alpha &= z,\\
\alpha^2 &= z^2,\\
\alpha^3 &= z^3,\\
\alpha^4 &= z + 1,\\
\alpha^5 &= z^2 + z,\\
\alpha^6 &= z^3 + z^2,\\
\alpha^7 &= z^3 + z + 1,\\
\alpha^8 &= z^2 + 1,\\
\alpha^9 &= z^3 + z,\\
\alpha^{10} &= z^2 + z + 1,\\
\alpha^{11} &= z^3 + z^2 + z,\\
\alpha^{12} &= z^3 + z^2 + z + 1,\\
\alpha^{13} &= z^3 + z^2 + 1,\\
\alpha^{14} &= z^3 + 1,\\
\alpha^{15} &= 1.
\end{aligned}$$

В таком представлении поля умножение опять просто; например,

$$\alpha^{11} \cdot \alpha^{13} = \alpha^{15} \cdot \alpha^9 = \alpha^9.$$

При построении расширения поля в виде множества многочленов удобно, чтобы многочлену x соответствовал примитивный

элемент поля. В этом случае в таблице умножения можно использовать x в качестве основания логарифмов, и это самое простое из возможных оснований. Такое построение поля можно осуществить с помощью простых многочленов специального частного вида, называемых примитивными.

Определение 4.5.5. *Примитивным многочленом* $p(x)$ над полем $GF(q)$ называется простой многочлен над $GF(q)$, такой, что в расширении поля, построенном по модулю $p(x)$, соответствующий многочлену x элемент поля является примитивным.

Для каждого поля Галуа существуют примитивные многочлены всех степеней, но доказательство этого результата мы откладываем до конца следующего параграфа. Предваряя этот результат, можно сказать, что примитивный многочлен представляет собой простой многочлен, корнем которого является примитивный элемент поля.

4.6. СТРУКТУРА КОНЕЧНОГО ПОЛЯ

Ранее в данной главе мы изучали, как строить поле. Предполагая, что можно найти простой многочлен степени n над полем $GF(p)$, мы научились строить конечное поле с p^n элементами.

Теперь изменим точку зрения на противоположную. Вместо того чтобы строить поле, предположим, что нам дано конечное поле, и докажем, что независимо от происхождения этого поля всегда можно предполагать, что оно построено соответственно идеям, изложенным в предыдущих параграфах. Никаких других полей так построить нельзя.

В процессе работы над материалом данного параграфа мы углубим наше понимание структуры конечных полей. Структурные свойства будут полезны во многих приложениях. Мы докажем также, что для всех полей Галуа существуют простые многочлены всех степеней.

Определение 4.6.1. Число элементов наименьшего подполя поля $GF(q)$ называется *характеристикой* поля $GF(q)$.

Теорема 4.6.2. *Каждое поле Галуа содержит единственное наименьшее подполе, число элементов которого равно простому числу. Следовательно, характеристика каждого поля Галуа является простым числом.*

Доказательство. Поле содержит элементы 0 и 1. Для задания подполя рассмотрим подмножество $G = \{0, 1, 1+1, 1+1+1, \ldots\}$, обозначая его элементы через $\{0, 1, 2, 3, \ldots\}$. Это подмножество является циклической группой по сложению, которая должна содержать конечное число p элементов. Мы покажем, что p — простое число и что $G = GF(p)$. Сложение в G

является сложением по модулю p, так как G образует циклическую группу по сложению. В силу закона дистрибутивности умножение также должно быть умножением по модулю p, ибо

$$\alpha \cdot \beta = (1 + \cdots + 1) \cdot \beta = \beta + \cdots + \beta,$$

где элемент β суммируется α раз и суммирование ведется по модулю p. Следовательно, умножение также является умножением по модулю p. По умножению каждый элемент обратим, так как последовательность $\{\beta, 2\beta, 3\beta, \ldots\}$ образует циклическую подгруппу в G.

Таким образом, подмножество G содержит единичный элемент, замкнуто относительно операций сложения и умножения и содержит элементы, обратные его элементам и по сложению, и по умножению. Следовательно, оно является подполем, и арифметика в этом подполе есть арифметика по модулю p. Но это в точности поле, описываемое теоремой 4.2.3, и, следовательно, p должно быть простым. □

В поле Галуа $GF(q)$ мы построили подполе $GF(p)$, где p — простое число. В частности, если q, с которого мы начинаем, само является простым числом, то, как мы видим, поле $GF(q)$ можно интерпретировать как поле чисел по модулю q. Следовательно, для заданного простого числа действительно существует только одно поле с данным числом элементов, хотя, конечно, оно может быть представлено многими разными способами. Два поля, различающиеся только представлениями, называются *изоморфными*.

Мы увидели, что исходное поле $GF(q)$ является расширением подполя $GF(p)$. Теперь мы рассмотрим многочлены над $GF(p)$, корнями которых являются некоторые выбранные элементы поля $GF(q)$. Для большей точности введем следующее определение.

Определение 4.6.3. Пусть $GF(q)$ — некоторое поле, пусть $GF(Q)$ — расширение поля $GF(q)$, и пусть β — элемент $GF(Q)$. Простой многочлен $f(x)$ наименьшей степени над $GF(q)$, для которого $f(\beta) = 0$, называется *минимальным многочленом* элемента β над $GF(q)$.

Мы должны доказать, что минимальный многочлен всегда существует и является единственным.

Теорема 4.6.4. *Каждый элемент β из $GF(Q)$ имеет единственный минимальный многочлен над $GF(q)$. Если минимальный многочлен элемента β равен $f(x)$ и β является корнем многочлена $g(x)$, то $f(x)$ делит $g(x)$.*

Доказательство. Прежде всего β всегда является корнем многочлена $x^Q - x$, представляющего собой многочлен над $GF(q)$. Воспользуемся теоремой об однозначном разложении:

$$x^Q - x = f_1(x) f_2(x) \ldots f_k(x),$$

где множители в правой части — все простые многочлены над полем $GF(q)$. Если β — корень левой части, то в правой части должен найтись некоторый член, корнем которого является β. Это может быть только один из членов правой части, так как над расширением $GF(Q)$ простые многочлены дальше разлагаются в произведение линейных членов и констант, и β может быть корнем только одного из линейных членов.

Чтобы доказать вторую часть теоремы, положим

$$g(x) = f(x)\,h(x) + s(x),$$

где степень многочлена $s(x)$ меньше степени $f(x)$, так что β не может быть его корнем. Но

$$0 = g(\beta) = f(\beta)\,h(\beta) + s(\beta) = s(\beta);$$

следовательно, $s(x)$ должен равняться нулю, и теорема доказана. □

Следствие 4.6.5. *Если $f_1(x)$, ..., $f_k(x)$ — все различные многочлены над $GF(q)$, являющиеся минимальными для одного или нескольких элементов из $GF(Q)$, то*

$$x^Q - x = f_1(x)\,f_2(x) \ldots f_k(x).$$

Доказательство следует из теоремы, так как каждый такой элемент β является корнем многочлена $x^Q - x$. □

При $Q = q$ разложение сводится к равенству

$$x^q - x = x(x - \beta_1)(x - \beta_2) \ldots (x - \beta_{q-1}).$$

которое мы уже встречали в теореме 4.5.2. Минимальный многочлен над $GF(q)$ элемента β, принадлежащего $GF(q)$, является многочленом первой степени $f(x) = x - \beta$.

Теорема 4.6.6. *Пусть $g(x)$ — произвольный многочлен над $GF(q)$. Тогда существует расширение $GF(Q)$, в котором $g(x)$ распадается на произведение линейных множителей.*

Доказательство. Без потери общности можно предположить, что $g(x)$ приведен. Построим последовательность расширений

$$GF(q) \subset GF(Q_1) \subset GF(Q_2) \subset \cdots \subset GF(Q)$$

по следующему правилу. На каждом шаге запишем $g(x)$ в виде произведения простых многочленов над $GF(Q_j)$. Если еще имеются нелинейные множители, то выберем один из них, скажем $g_j(x)$, и построим расширение поля $GF(Q_j)$, используя $g_j(y)$ в качестве простого модуля. В этом расширении $g(x)$ можно разложить далее, поскольку новый элемент $\beta = y$ является корнем многочлена $g_j(x)$. Таким образом (в случае необходимости унифицируя обозначения) будем действовать до тех пор, пока все множители не станут линейными. Поскольку степень $g(x)$ конечна, этот процесс закончится после конечного числа шагов. □

Определение 4.6.7. Любое расширение поля $GF(q)$, в котором многочлен $g(x)$ над $GF(q)$ распадается в произведение линейных множителей и констант, называется *полем разложения многочлена* $g(x)$.

Теперь у нас есть все необходимое для описания структуры произвольного поля Галуа.

Теорема 4.6.8. *Пусть* α *— примитивный элемент поля Галуа* $GF(Q)$, *являющегося расширением поля* $GF(q)$, *и пусть* m *— степень минимального многочлена* $f(x)$ *элемента* α *над* $GF(q)$. *Тогда число элементов в поле равно* $Q = q^m$ *и каждый элемент* β *может быть представлен в виде*

$$\beta = a_{m-1}\alpha^{m-1} + a_{m-2}\alpha^{m-2} + \cdots + a_1\alpha + a_0,$$

где $a_{m-1}, \ldots, a_0$ *— элементы поля* $GF(q)$.

Доказательство. Очевидно, что каждый элемент β вида

$$\beta = a_{m-1}\alpha^{m-1} + a_{m-2}\alpha^{m-2} + \cdots + a_1\alpha + a_0$$

принадлежит полю $GF(Q)$. Такое разложение единственно, так как если

$$\beta = b_{m-1}\alpha^{m-1} + b_{m-2}\alpha^{m-2} + \cdots + b_1\alpha + b_0$$

— другое представление элемента β, то

$$0 = (a_{m-1} - b_{m-1})\alpha^{m-1} + \cdots + (a_1 - b_1)\alpha + (a_0 - b_0),$$

и, следовательно, α является корнем многочлена степени $m - 1$, что противоречит выбору числа m. Всего имеется q^m таких элементов β, и, следовательно, число элементов поля Q не меньше q^m.

С другой стороны, каждый ненулевой элемент поля может быть представлен в виде некоторой степени элемента α. Но если $f(x)$ — минимальный многочлен элемента α, то $f(\alpha) = 0$. Следовательно,

$$\alpha^m + f_{m-1}\alpha^{m-1} + \cdots + f_1\alpha + f_0 = 0,$$

Полученное равенство можно использовать для того, чтобы выразить элемент α^m через сумму меньших степеней элемента α:

$$\alpha^m = -f_{m-1}\alpha^{m-1} - \cdots - f_1\alpha - f_0.$$

Полученное соотношение можно повторно применять для редукции любой степени элемента α к линейной комбинации степеней $(\alpha^{m-1}, \ldots, \alpha^1, \alpha^0)$, что дает

$$\alpha^{m+1} = -f_{m-1}(-f_{m-1}\alpha^{m-1} - \cdots - f_1\alpha - f_0) - \\ - f_{m-2}\alpha^{m-1} - \cdots - f_1\alpha^2 - f_0\alpha,$$

и т. д. Следовательно, каждый элемент поля $GF(Q)$ может быть представлен в виде линейной комбинации элементов $\alpha^{m-1}, \alpha^{m-2}, \ldots, \alpha, \alpha^0$, так что Q не может быть больше q^m, и теорема доказана. □

Следствие 4.6.9. *Каждое поле Галуа содержит p^m элементов, где p — некоторое простое, а m — положительное целое число.*

Доказательство. Каждое поле Галуа содержит подполе с p элементами, к которому надо применить теорему 4.6.8. □

Заметим, что теорему 4.6.8 можно использовать для того, чтобы связать с каждым элементом поля некоторый многочлен степени не выше $m - 1$ путем простой замены элемента α на неопределенную переменную x. Эти многочлены можно рассматривать как элементы поля. Складываться и умножаться они будут по модулю минимального многочлена $f(x)$ элемента α. Это в точности то же самое поле, которое получается в теореме 4.4.3, если в качестве простого многочлена выбрать $f(x)$. Следовательно, число элементов в каждом поле Галуа равно степени простого числа, и каждое поле Галуа может быть построено с помощью арифметики по модулю простого многочлена.

Наконец, мы должны доказать и обратное: такое поле существует для каждого простого p и целого положительного числа m. Прежде всего установим некоторые предварительные результаты.

Теорема 4.6.10. *Пусть характеристика поля $GF(q)$ равна p. Тогда для любых элементов α и β из $GF(q)$ и любого положительного целого m*

$$(\alpha \pm \beta)^{p^m} = \alpha^{p^m} \pm \beta^{p^m}.$$

Доказательство. Предположим, что теорема верна для $m = 1$. Тогда

$$(\alpha \pm \beta)^p = \alpha^p \pm \beta^p.$$

Возведем это равенство в p-ю степень:

$$((\alpha \pm \beta)^p)^p = (\alpha^p \pm \beta^p)^p,$$

и снова используем теорему для $m = 1$:

$$(\alpha \pm \beta)^{p^2} = \alpha^{p^2} \pm \beta^{p^2}.$$

Повторяя эту процедуру $m - 1$ раз, получаем

$$(\alpha \pm \beta)^{p^m} = \alpha^{p^m} \pm \beta^{p^m}.$$

Следовательно, теорему надо доказать только для $m = 1$. Воспользовавшись биномиальным разложением

$$(\alpha \pm \beta)^p = \sum_{i=0}^{p} \binom{p}{i} \alpha^i (\pm\beta)^{p-i},$$

видим, что достаточно доказать, что в поле $GF(q)$ выполняется равенство

$$\binom{p}{i} = 0, \quad i = 1, \ldots, p-1.$$

Но для каждого i

$$\binom{p}{i} = \frac{p!}{i!(p-i)!} = \frac{p(p-1)!}{i!(p-i)!}$$

является целым числом, а p — простое. Следовательно, знаменатель делит $(p-1)!$, а число $\binom{p}{i}$ кратно p. Таким образом, $\binom{p}{i} = 0 \pmod{p}$, и поскольку арифметикой целых чисел в поле $GF(q)$ является арифметика по модулю p, то в $GF(q)$ биномиальный коэффициент $\binom{p}{i} = 0$. Наконец, если $p = 2$, то $(\pm\beta)^2 = \beta^2$, а если p нечетно, то $(\pm\beta)^p = \pm\beta^p$. Это завершает доказательство теоремы. □

Теорема 4.6.11. *Пусть p — простое, а m — положительное целое число. Тогда наименьшее поле разложения многочлена $g(x) = x^{p^m} - x$, рассматриваемого над полем $GF(p)$, содержит p^m элементов.*

Доказательство. Каждый многочлен над $GF(p)$ имеет наименьшее поле разложения. Пусть $GF(Q)$ — наименьшее поле разложения многочлена $g(x) = x^{p^m} - x$. Тогда в поле $GF(Q)$ многочлен $g(x)$ имеет p^m корней (возможно, кратных). Мы покажем, что все p^m корней различны и образуют поле. Из этого будет следовать, что $GF(Q)$ содержит p^m элементов.

Для того чтобы доказать, что множество корней образует поле, достаточно показать, что оно замкнуто относительно операций сложения и умножения и содержит обратные для всех ненулевых элементов. Пусть α и β — корни многочлена $g(x)$. Согласно теореме 4.6.10,

$$(\alpha \pm \beta)^{p^m} = \alpha^{p^m} \pm \beta^{p^m} = \alpha \pm \beta,$$

так что $\alpha \pm \beta$ — также корень многочлена, и, следовательно, множество замкнуто относительно операции сложения. Далее,

$$(\alpha\beta)^{p^m} = \alpha^{p^m} \cdot \beta^{p^m} = \alpha\beta,$$

и, таким образом, $\alpha\beta$ тоже является корнем, и множество корней замкнуто относительно операции умножения. Далее, $-\alpha$ является аддитивным обратным элементу α, так что каждый элемент имеет аддитивный обратный. Аналогично легко проверить, что если α — корень многочлена, то α^{-1} — также его корень.

Наконец, проверим, что все p^m корней многочлена $x^{p^m} - x$ различны. Это вытекает из вида формальной производной:

$$d[x^{p^m} - x]/dx = ((p^m))\, x^{p^m-1} - 1 = -1,$$

так как $((p)) = 0$ в поле $GF(Q)$. Следовательно, многочлен $x^{p^m} - x$ не имеет кратных корней. □

Теперь мы получили обращение теоремы 4.6.9.

Следствие 4.6.12. *Для каждого простого p и положительного целого числа m существует поле Галуа с p^m элементами.*

Покажем, наконец, что если q является не простым числом, а степенью простого числа, то $GF(q^m)$ можно построить как расширение поля $GF(q)$. Для этого достаточно доказать существование над $GF(q)$ простых многочленов степени m.

Теорема 4.6.13. *Для каждого целого положительного m над каждым конечным полем $GF(q)$ существует хотя бы один простой многочлен степени m.*

Доказательство. Так как q — степень простого числа, то q^m также является степенью простого числа. Согласно следствию 4.6.12, существует поле с q^m элементами. Это поле содержит примитивный элемент α, и, по теореме 4.6.8, минимальный многочлен этого элемента α над $GF(q)$ является простым многочленом степени m. □

Следствие 4.6.14. *Для каждого целого положительного m над каждым конечным полем $GF(q)$ существует хотя бы один примитивный многочлен степени m.*

Доказательство. Пусть α — примитивный элемент поля $GF(q^m)$, и пусть $f(x)$ — минимальный многочлен элемента α над $GF(q)$. Тогда в поле многочленов по модулю $f(x)$ примитивный элемент $\alpha = x$ является корнем многочлена $f(x)$, так что многочлен x представляет собой примитивный элемент поля. □

В заключение главы рассмотрим вопрос о существовании квадратного корня в поле Галуа.

Теорема 4.6.15. *Каждый элемент поля $GF(2^m)$ имеет в этом поле квадратный корень. Для нечетных простых p половина ненулевых элементов поля $GF(p^m)$ имеет квадратный корень в $GF(p^m)$. Половина ненулевых элементов поля $GF(p^m)$ имеет квадратный корень в расширении $GF(p^{2m})$, а не в $GF(p^m)$.*

Доказательство. Так как нулевой элемент имеет квадратный корень в любом поле, то рассматривать надо только ненулевые элементы поля. Сначала рассмотрим поле $GF(2^m)$ характеристики 2 с примитивным элементом α. Тогда порядок элемента α является

нечетным числом. Каждый элемент поля β может быть записан в виде α^i для некоторого i, и поэтому $\sqrt{\beta} = \alpha^{i/2}$, если i четно, и $\sqrt{\beta} = \alpha^{(i+n)/2}$, если i нечетно. В любом случае $\sqrt{\beta}$ является элементом поля $GF(2^m)$.

Рассмотрим теперь поле $GF(q)$, характеристика которого равна простому нечетному числу p, а примитивным элементом является $\alpha = \gamma^{q+1}$, где γ — примитивный элемент расширения $GF(q^2)$ (порядка $q^2 - 1 = (q+1)(q-1)$) и $q+1$ четно, так как q равно степени нечетного простого числа. Любой элемент β может быть записан в виде α^i или в виде $\gamma^{(q+1)i}$ для некоторого i. Тогда если i четно, то $\sqrt{\beta} = \alpha^{i/2}$ и принадлежит полю $GF(q)$, а если i нечетно, то $\sqrt{\beta} = \gamma^{(q+1)i/2}$ и принадлежит полю $GF(q^2)$, но не полю $GF(q)$, так как $i(q+1)/2$ в этом случае не кратно $q+1$. □

ЗАДАЧИ

4.1. Пусть над $GF(2)$ $p_1(x) = x^3 + 1$ и $p_2(x) = x^4 + x^3 + x + 1$.

а. Найти НОД $[p_1(x), p_2(x)]$.

б. Найти многочлены $A(x)$ и $B(x)$, удовлетворяющие равенству

$$\text{НОД}\,[p_1(x), p_2(x)] = A(x)\,p_1(x) + B(x)\,p_2(x).$$

4.2.а. Сколько различных приведенных многочленов второй степени вида $x^2 + ax + b$, $b \neq 0$, имеется над полем $GF(16)$?

б. Сколько различных многочленов вида $(x-\alpha)(x-\beta)$, $\alpha, \beta \neq 0$, имеется над полем $GF(16)$?

в. Доказывает ли это существование неприводимых многочленов над полем $GF(16)$? Сколько простых многочленов степени два имеется над полем $GF(16)$?

4.3. Доказать теорему 4.1.2, связывая обе части равенств с алгоритмом деления. Этим же способом доказать теорему 4.3.4.

4.4.а. Используя алгоритм Евклида, найти НОД (1573, 308).

б. Найти целые числа A и B, удовлетворяющие равенству

$$\text{НОД}\,(1573{,}308) = 1573A + 308B.$$

4.5. Доказать, что в кольце $Z/(15)$ целых чисел по модулю 15 многочлен $p(x) = x^2 - 1$ имеет более двух корней. Этот же многочлен над полем имеет два корня. В каком месте не проходит доказательство теоремы в случае кольца?

4.6. Сколько различных приведенных многочленов над $GF(2)$ делят многочлен $x^6 - 1$?

4.7. Построить поле $GF(5)$, выписав для него таблицы сложения и умножения.

4.8. Построить таблицы сложения и умножения для полей $GF(8)$ и $GF(9)$.

4.9.а. Доказать, что многочлен $p(x) = x^3 + x^2 + 2$ неприводим над полем $GF(3)$.

б. Каковы возможные (мультипликативные) порядки элементов поля $GF(27)$?

в. Чему равен порядок элемента x в поле $GF(27)$, построенном по модулю этого многочлена $p(x)$?

г. Предполагая, что поле $GF(27)$ построено по модулю данного многочлена $p(x)$, найти $(2x+1)(x^2+2)$.

4.10. Вычислить $3^{100} \pmod 5$.

4.11. Доказать, что кольца отношений $Z/(q)$ и $GF(q)[x]/(p(x))$ являются кольцами.

4.12. Многочлен $p(x) = x^4 + x^3 + x^2 + 1$ неприводим над полем $GF(2)$. Следовательно, кольцо многочленов по модулю $p(x)$ является полем $GF(16)$.

а. Доказать, что в этой конструкции элемент поля, соответствующий многочлену x, не является примитивным.

б. Показать, что многочлену $x + 1$ соответствует примитивный элемент этого поля.

в. Найти минимальный многочлен элемента $x + 1$.

4.13.а. Построить таблицы сложения и умножения поля $GF(2^3)$, используя неприводимый многочлен $x^3 + x^2 + 1$ над $GF(2)$.

б. Повторить задачу п. а для многочлена $x^3 + x + 1$ и доказать, что эти два поля изоморфны. Иначе говоря, доказать, что второе поле можно получить из первого путем переименования его элементов.

4.14. Таблицы сложения и умножения для поля $GF(2^4)$ можно построить по меньшей мере двумя следующими способами:

(i) используя неприводимый многочлен степени 4 над $GF(2)$;

(ii) используя неприводимый многочлен степени 2 над $GF(4)$.

Построить эти таблицы, используя второй способ.

4.15. Многочлен $p(x) = x^{20} + x^3 + 1$ примитивен над полем $GF(2)$ и может быть использован для построения поля $GF(1\,048\,576)$, в котором элемент α, соответствующий многочлену x, будет примитивным.

а. Каковы подполя этого поля?

б. Сколько из этих подполей не имеют никаких собственных подполей, отличных от $GF(2)$?

в. Вычислить в этом поле выражение $ab^2 - 1$, где $a = x^{12} + x^7 + 1$ и $b = x^{13} + x^5 + x^2$.

4.16. Доказать, что для формальной производной многочлена выполняется равенство

$$[r(x)\,s(x)]' = r'(x)\,s(x) + r(x)\,s'(x)$$

и что $a^2(x)$ делит $r(x)$ тогда и только тогда, когда $a(x)$ делит $r'(x)$.

4.17. Доказать, что в любом поле характеристики 2 любой элемент β удовлетворяет условию $-\beta = \beta$.

ЗАМЕЧАНИЯ

Материал данной главы обычен для математической литературы. Свойства полей Галуа описываются во многих книгах по абстрактной алгебре, например в книгах Биркгофа и Маклейна [1953], а также Ван дер Вардена [1950, 1953]. Однако стандартное изложение является формальным, уделяет основное внимание абстрактным свойствам и содержит мало примеров и приложений. В книге Берлекэмпа [1968] подробно рассматриваются те свойства полей Галуа, которые используются в теории кодирования.

ГЛАВА 5

ЦИКЛИЧЕСКИЕ КОДЫ

Циклические коды являются подклассом в классе линейных кодов, удовлетворяющим дополнительному сильному структурному требованию. В силу этой структуры поиск хороших кодов, контролирующих ошибки, в классе циклических кодов оказался наиболее успешным. При этом в качестве математического аппарата, облегчающего поиск хороших кодов, была использована теория полей Галуа. Вне класса циклических кодов теория полей Галуа помогает мало; большинство завершенных построений, использующих идеи этой теории, относится к циклическим кодам.

Важность циклических кодов обусловлена также тем, что заложенные в основу их определения идеи теории полей Галуа приводят к процедурам кодирования и декодирования, эффективным как с алгоритмической, так и с вычислительной точки зрения. В противоположность необходимым для произвольного линейного кода табличным методам декодирования алгоритмические методы декодирования находят важные практические приложения.

5.1. КОД С ТОЧКИ ЗРЕНИЯ РАСШИРЕНИЯ ПОЛЯ

Циклические коды над $GF(q)$ представляют собой класс линейных кодов, допускающих специальное описание, которое, как будет показано, оказывается достаточно мощным. Хотя эти коды являются кодами над $GF(q)$, они часто яснее просматриваются из расширения $GF(q^m)$. Подобно тому как теория функций комплексного переменного используется для более детального исследования функций вещественного переменного, функции над $GF(q)$ можно изучать над $GF(q^m)$. Поэтому не следует рассматривать введение расширения поля как нечто искусственное; вместо этого надо набраться терпения, и вы будете вознаграждены за него.

Мы видели, что линейный код над $GF(q)$ может быть описан посредством матрицы с элементами из $GF(q)$, называемой прове-

рочной матрицей. Вектор $\mathbf{c}$ над $GF(q)$ является кодовым словом тогда и только тогда, когда $\mathbf{cH}^{\mathrm{T}} = \mathbf{0}$. Например, выберем следующую проверочную матрицу (7, 4)-кода Хэмминга:

$$\mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 \end{bmatrix}.$$

Переходя в расширение поля, эту матрицу можно записать более компактно. Столбцы матрицы $\mathbf{H}$ можно отождествить с элементами поля $GF(8)$. В многочленах, представляющих элементы поля, используем элементы первой строки матрицы в качестве коэффициентов при z^0, второй — в качестве коэффициентов при z^1, а третьей — в качестве коэффициентов при z^2. Тогда, используя для построения поля $GF(8)$ многочлен $p(z) = z^3 + z + 1$ и выбирая z в качестве представителя примитивного элемента α, перепишем матрицу $\mathbf{H}$ в виде

$$\mathbf{H} = [\alpha^0\ \alpha^1\ \alpha^2\ \alpha^3\ \alpha^4\ \alpha^5\ \alpha^6].$$

Над расширением поля $GF(8)$ проверочная матрица оказалась (1×7)-матрицей. Используя эту матрицу, кодовое слово можно определить как вектор над $GF(2)$, такой, что в расширении поля $GF(8)$ он дает нулевое произведение с $\mathbf{H}^{\mathrm{T}}$:

$$\mathbf{cH}^{\mathrm{T}} = \mathbf{0},$$

или

$$\sum_{i=0}^{6} c_i \alpha^i = 0.$$

Теперь мы подошли к идее представления кодовых слов в виде многочленов: кодовое слово $\mathbf{c}$ представляется многочленом

$$c(x) = \sum_{i=0}^{n-1} c_i x^i,$$

и операция умножения кодового слова на проверочную матрицу превращается в операцию вычисления многочлена $c(x)$ в точке $x = \alpha$. Условие того, что $c(x)$ представляет кодовое слово, превращается в равенство $c(\alpha) = 0$. Иначе говоря, двоичный многочлен $c(x)$ является кодовым словом тогда и только тогда, когда α является корнем многочлена $c(x)$. Представление многочленами (7,4)-кода Хэмминга образует множество всех многочленов над $GF(2)$ степени не выше 6, таких, что α из $GF(8)$ является корнем каждого из них.

Существует обширный класс линейных кодов, к которому применим тот же метод, который был использован для представления кода Хэмминга в виде множества многочленов. Предположим,

что проверочная матрица линейного кода содержит n столбцов и что число $n-k$ строк этой матрицы делится на m. Каждая группа из m строк такой матрицы может быть представлена в виде одной строки, содержащей элементы из $GF(q^m)$. Таким образом, первые m строк становятся одной строкой $(\beta_{11}, \ldots, \beta_{1n})$, а матрица $\mathbf{H}$ сводится к матрице

$$\mathbf{H} = \begin{bmatrix} \beta_{11} & \cdots & \beta_{1n} \\ \cdot & & \cdot \\ \cdot & & \cdot \\ \cdot & & \cdot \\ \beta_{r1} & \cdots & \beta_{rn} \end{bmatrix}$$

с $r = (n-k)/m$ строками. Вместо $[(n-k) \times m]$-матрицы над $GF(q)$ мы получили $(r \times n)$-матрицу над $GF(q^m)$. Мы, конечно, ничего не изменили при таком представлении, а только сделали его более компактным.

В данной главе будет исследоваться частный случай, когда проверочная матрица может быть записана в виде

$$\mathbf{H} = \begin{bmatrix} \gamma_1^0 & \gamma_1^1 & \cdots & \gamma_1^{n-2} & \gamma_1^{n-1} \\ \gamma_2^0 & \gamma_2^1 & \cdots & \gamma_2^{n-2} & \gamma_2^{n-1} \\ \cdot & & & & \cdot \\ \cdot & & & & \cdot \\ \cdot & & & & \cdot \\ \gamma_r^0 & \gamma_r^1 & \cdots & \gamma_r^{n-2} & \gamma_r^{n-1} \end{bmatrix},$$

где $n = q^m - 1$ и $\gamma_j \in GF(q^m)$ для всех $j = 1, \ldots, r$.

Заменяя каждый элемент поля $GF(q^m)$ коэффициентами представляющего его многочлена над $GF(q)$, эту матрицу можно записать в виде проверочной матрицы над $GF(q)$ с $n = q^m - 1$ столбцами и rm строками. Мы, однако, предпочитаем работать в большем поле.

Каждое кодовое слово $\mathbf{c}$ является вектором над $GF(q)$, таким, что в расширении $GF(q^m)$ выполняется матричное равенство

$$\mathbf{c}\mathbf{H}^{\mathrm{T}} = \mathbf{0},$$

которое для выбранного выше частного вида матрицы $\mathbf{H}$ можно записать в виде

$$\sum_{i=0}^{n-1} c_i \gamma_j^i = 0, \quad j = 1, \ldots, r.$$

Это соотношение в точности совпадает с утверждением, что все элементы $\gamma_1, \ldots, \gamma_r$ являются корнями кодового многочлена $c(x)$. Рассматриваемый код определяется как множество многочленов $c(x) = \sum_{i=0}^{n-1} c_i x^i$ степени не выше $n-1$ и таких, что для всех $j = 1, \ldots, r$ выполняется равенство $c(\gamma_j) = 0$.

Таким образом мы перешли от матричного описания линейных кодов к полиномиальному описанию одного частного подкласса линейных кодов. Мы ограничиваемся рассмотрением этого подкласса потому, что полиномиальная формулировка облегчает поиск хороших кодов и разработку кодеров и декодеров. По причинам, которые выясняются в следующем параграфе, эти коды называются *циклическими кодами*.

Например, выберем некоторый примитивный элемент α поля $GF(16)$ и примем длину кода равной 15. Для построения проверочной матрицы кода над $GF(2)$ выберем $\gamma_1 = \alpha$, $\gamma_2 = \alpha^3$:

$$H = \begin{bmatrix} \alpha^0 & \alpha^1 & \alpha^2 & \alpha^3 & \alpha^4 & \alpha^5 & \alpha^6 & \alpha^7 & \alpha^8 & \alpha^9 & \alpha^{10} & \alpha^{11} & \alpha^{12} & \alpha^{13} & \alpha^{14} \\ \alpha^0 & \alpha^3 & \alpha^6 & \alpha^9 & \alpha^{12} & \alpha^{15} & \alpha^{18} & \alpha^{21} & \alpha^{24} & \alpha^{27} & \alpha^{30} & \alpha^{33} & \alpha^{36} & \alpha^{39} & \alpha^{42} \end{bmatrix}$$

Другой выбор элементов γ_1 и γ_2 приводит к другой проверочной матрице, иногда хорошей, а иногда плохой. Мы надеемся, что сделанный выбор приведет к хорошей проверочной матрице, но пока что еще рано объяснять причины этого.

При желании можно эту проверочную матрицу заменить проверочной матрицей над $GF(2)$, используя приведенное в § 4.5 представление поля $GF(16)$. Для этого надо каждую степень элемента α заменить 4-битовым столбцом, полагая верхний элемент столбца равным коэффициенту при z^0 в полиномиальном представлении этого элемента поля, второй элемент столбца равным коэффициенту при z^1 и т. д. В результате получится матрица

$$H = \left[\begin{array}{ccccccccccccccc} 1 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 1 & 1 \\ \hline 1 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 1 & 1 & 0 & 1 & 1 & 1 & 1 & 0 & 1 & 1 & 1 & 1 \end{array}\right]$$

Хотя этого сразу не видно, строки данной проверочной матрицы линейно независимы, и, следовательно, она определяет двоичный (15,7)-код. Минимальное расстояние этого кода равно 5 (хотя это тоже не очевидно). Этот код можно также представить в виде множества многочленов над $GF(2)$ степени не выше 14, таких, что в расширении $GF(16)$ каждый многочлен удовлетворяет равенствам $c(\alpha) = 0$ и $c(\alpha^3) = 0$. Иначе говоря, кодовые слова являются многочленами с корнями в α и α^3.

Можно показать, что всегда проще иметь дело с матрицей в расширении, и поэтому мы в большинстве случаев будем работать в большом поле. Конечно, ограничиваясь проверочными матрицами такого частного вида, мы исключаем из дальнейшего рассмотрения многие коды.

5.2. ПОЛИНОМИАЛЬНОЕ ОПИСАНИЕ ЦИКЛИЧЕСКИХ КОДОВ

Начнем изложение заново, а затем вернемся к материалу предыдущего параграфа. Линейный код $\mathscr{C}$ называется *циклическим*, если из того, что слово $\mathbf{c} = (c_0, c_1, \ldots, c_{n-1})$ принадлежит $\mathscr{C}$, следует, что слово $\mathbf{c}' = (c_{n-1}, c_0, \ldots, c_{n-2})$ также принадлежит $\mathscr{C}$. Кодовое слово $\mathbf{c}'$ получается из кодового слова $\mathbf{c}$ циклическим сдвигом всех компонент на одну позицию вправо. Примером циклического кода является приведенный в табл. 1.1 код Хэмминга. Каждый линейный код над $GF(q)$ длины n представляет собой подпространство пространства $GF^n(q)$, а циклический код является частным случаем подпространства, так как обладает дополнительным свойством цикличности.

Каждый вектор из $GF^n(q)$ можно представить многочленом от x степени не выше $n-1$. Компоненты вектора отождествляются с коэффициентами многочлена. Множество многочленов обладает структурой векторного пространства, идентичной структуре пространства $GF^n(q)$. Это же множество многочленов обладает структурой определенного в § 4.2 кольца $GF(q)[x]/(x^n-1)$. Как в кольце, в этом множестве определено умножение

$$p_1(x) \cdot p_2(x) = R_{x^n-1}[p_1(x)\, p_2(x)].$$

Заметим, что в это равенство входят произведения двух видов. Произведение в левой части является произведением в кольце $GF(q)[x]/(x^n-1)$, определенным через произведения в кольце $GF(q)[x]$ в правой части.

Циклический сдвиг может быть записан через умножение в этом кольце:

$$x \cdot p(x) = R_{x^n-1}[xp(x)].$$

Итак, если кодовые слова некоторого кода задаются в виде многочленов, то код является подмножеством кольца $GF(q)[x]/(x^n-1)$. Такой код является циклическим, если вместе с каждым кодовым словом $c(x)$ он содержит кодовый многочлен $x \cdot c(x)$.

Теорема 5.2.1. *Подмножество* $\mathscr{C}$ *кольца* $GF(q)[x]/(x^n-1)$ *образует циклический код тогда и только тогда, когда он обладает следующими двумя свойствами* [1]):

1) $\mathscr{C}$ *образует подгруппу кольца* $GF(q)[x]/(x^n-1)$ *по сложению;*

2) *если* $c(x) \in \mathscr{C}$ *и* $a(x) \in GF(q)[x]/(x^n-1)$, *то*

$$R_{x^n-1}[a(x)\,c(x)] \in \mathscr{C}.$$

Доказательство. Предположим, что данное подмножество обладает этими двумя свойствами. Тогда оно замкнуто относительно сложения и умножения на скаляр и, следовательно, является подпространством. Оно замкнуто относительно умножения на любой элемент кольца, в частности на элемент x, и, следовательно, образует циклический код.

Теперь предположим, что рассматриваемое множество является циклическим кодом и поэтому замкнуто относительно сложения и умножения на элемент x. Но тогда оно замкнуто относительно умножения на степени элемента x и на линейные комбинации этих степеней, т. е. относительно умножения на произвольный многочлен. Следовательно, оно удовлетворяет обоим сформулированным условиям, и теорема доказана. □

Выберем теперь в $\mathscr{C}$ ненулевой кодовый многочлен наименьшей степени и обозначим его степень через $n-k$ (она должна быть меньше n). Умножим этот многочлен на элемент поля, чтобы сделать его приведенным. Поскольку код $\mathscr{C}$ линеен, результирующий многочлен тоже принадлежит $\mathscr{C}$. В коде не содержится никакого другого приведенного многочлена этой степени, так как в противном случае разность этих двух приведенных многочленов давала бы кодовый многочлен степени меньше $n-k$, а это противоречит выбору исходного многочлена.

Единственный приведенный ненулевой многочлен наименьшей степени в коде $\mathscr{C}$ называется *порождающим многочленом* кода $\mathscr{C}$ и обозначается через $g(x)$.

Теорема 5.2.2. *Циклический код состоит из всех произведений порождающего многочлена* $g(x)$ *на многочлены степени не выше* $k-1$.

Доказательство. Согласно теореме 5.2.1, все такие многочлены принадлежат коду, так как $g(x)$ принадлежит коду. Далее,

---

[1]) Это подмножество называется *идеалом* кольца. В общем случае подмножество I кольца R называется идеалом в R, если: 1) I является подгруппой аддитивной группы кольца R и 2) из $r \in R$ и $a \in I$ следует, что $ar \in I$.

если некоторый многочлен $c(x)$ принадлежит коду, то, согласно алгоритму деления,

$$c(x) = Q(x)\,g(x) + s(x),$$

где $\deg s(x) < \deg g(x)$, и многочлен

$$s(x) = c(x) - Q(x)\,g(x)$$

является кодовым, так как оба многочлена в правой части принадлежат коду, а код линеен. Но степень многочлена $s(x)$ меньше $n-k$, наименьшей степени ненулевого многочлена в коде. Следовательно, $s(x) = 0$ и $c(x) = Q(x)\,g(x)$. □

Теорема 5.2.3. *Циклический код длины n с порождающим многочленом $g(x)$ существует тогда и только тогда, когда $g(x)$ делит x^n-1.*

Доказательство. Согласно алгоритму деления,

$$x^n - 1 = Q(x)\,g(x) + s(x),$$

где степень многочлена $s(x)$ меньше степени многочлена $g(x)$. Тогда

$$0 = R_{x^n-1}[x^n - 1] = R_{x^n-1}[Q(x)g(x)] + R_{x^n-1}[s(x)],$$

и, следовательно,

$$0 = R_{x^n-1}[Q(x)g(x)] + s(x).$$

Из теоремы 5.2.1 вытекает, что первый многочлен в правой части является кодовым многочленом. Тогда $s(x)$ также является кодовым многочленом степени ниже, чем степень $g(x)$. Единственный такой кодовый многочлен равен $s(x) = 0$. Таким образом, $q(x)$ делит $x^n - 1$. Далее, каждый многочлен, делящий x^n-1, может быть выбран в качестве порождающего многочлена циклического кода. Это завершает доказательство. □

Согласно теореме 5.2.3, для порождающего многочлена $g(x)$ любого циклического кода выполняется равенство

$$x^n - 1 = g(x)\,h(x)$$

при некотором многочлене $h(x)$. Многочлен $h(x)$ называется *проверочным многочленом*. Каждое кодовое слово $c(x)$ удовлетворяет равенству

$$R_{x^n-1}[h(x)\,c(x)] = 0,$$

так как

$$h(x)\,c(x) = h(x)\,g(x)\,a(x) = (x^n - 1)\,a(x)$$

для некоторого многочлена $a(x)$.

Пусть $c(x)$ обозначает переданное кодовое слово. Это значит, что символами переданного слова были коэффициенты многочлена $c(x)$. Пусть многочлен $v(x)$ обозначает принятое слово, и пусть $e(x) = v(x) - c(x)$. Многочлен $e(x)$ называется *многочленом ошибок*. Ненулевые коэффициенты этого многочлена стоят в тех позициях, где в канале произошли ошибки.

Представим информационную последовательность в виде многочлена $i(x)$ степени $k - 1$ [1]). Множество информационных многочленов можно отобразить в кодовые многочлены многими удобными способами. Одним простым правилом кодирования является

$$c(x) = i(x)\, g(x).$$

Такой кодер является несистематическим, так как по многочлену $c(x)$ нельзя сразу установить $i(x)$. Систематическое правило кодирования имеет несколько более сложный вид. Идея состоит в записи информации в виде старших коэффициентов кодового многочлена и подборе младших коэффициентов так, чтобы получить допустимое кодовое слово. Иначе говоря, кодовое слово записывается в виде

$$c(x) = x^{n-k} i(x) + t(x),$$

где $t(x)$ выбирается так, чтобы выполнялось условие

$$R_{g(x)}[c(x)] = 0.$$

Это требование означает, что

$$R_{g(x)}[x^{n-k} i(x)] \mid R_{g(x)}[t(x)] = 0$$

и степень многочлена $t(x)$ меньше $n-k$, степени многочлена $g(x)$. Следовательно,

$$t(x) = -R_{g(x)}[x^{n-k} i(x)],$$

и правило кодирования является взаимно-однозначным, так как k старших коэффициентов многочлена определены однозначно. И систематическое, и несистематическое правила кодирования дают одно и то же множество кодовых слов, но соответствия между $i(x)$ и $c(x)$ различны.

Определим *синдромный многочлен* $s(x)$, который будет использован для декодирования, как остаток от деления многочлена $v(x)$ на $g(x)$:

$$s(x) = R_{g(x)}[v(x)] = R_{g(x)}[c(x) + e(x)] = R_{g(x)}[e(x)].$$

---

[1]) На самом деле надо было бы говорить, что степень $i(x)$ *не выше* $k-1$, но в случаях, подобных данному, когда $i(x)$ является одним из многочленов множества, удобнее указывать максимальную степень в множестве, хотя это и менее точно.

Синдромный многочлен зависит только от $e(x)$ и не зависит ни от $c(x)$, ни от $i(x)$.

Итак, мы ввели следующие многочлены:

порождающий многочлен	$g(x)$,	$\deg g(x) = n - k$,
проверочный многочлен	$h(x)$,	$\deg h(x) = k$,
информационный многочлен	$i(x)$,	$\deg i(x) = k - 1$,
кодовый многочлен	$c(x)$,	$\deg c(x) = n - 1$,
многочлен ошибок	$e(x)$,	$\deg e(x) = n - 1$,
принятый многочлен	$v(x)$,	$\deg v(x) = n - 1$,
синдромный многочлен	$s(x)$,	$\deg s(x) = n - k - 1$.

Теорема 5.2.4. *Пусть d^* — минимальное расстояние циклического кода $\mathscr{C}$. Каждому многочлену ошибок веса меньше, чем $d^*/2$, соответствует единственный синдромный многочлен.*

Доказательство. Предположим, что веса $e_1(x)$ и $e_2(x)$ меньше, чем $d^*/2$, и что им соответствует один и тот же синдромный многочлен. Тогда

$$e_1(x) = Q_1(x)\,g(x) + s(x), \qquad e_2(x) = Q_2(x)\,g(x)\,g(x) + s(x)$$

и

$$e_1(x) - e_2(x) = [Q_1(x) - Q_2(x)]\,g(x).$$

По предположению вес каждого из многочленов $e_1(x)$ и $e_2(x)$ меньше, чем $d^*/2$, так что вес их разности меньше d^*. В правой части последнего равенства стоит кодовое слово. Если это слово отлично от нулевого, то вес его не меньше d^*, минимального веса в коде. Следовательно, в правой части стоит нуль, и многочлены $e_1(x)$ и $e_2(x)$ равны. Это доказывает теорему. □

Таким образом, задача исправления ошибок сводится к однозначному вычислению многочлена $e(x)$ с наименьшим числом ненулевых коэффициентов, удовлетворяющего условию

$$s(x) = R_{g(x)}[e(x)].$$

При не очень большом числе входов эта задача может быть решена путем построения таблиц. Для каждого корректируемого многочлена $e(x)$ вычисляется и табулируется многочлен $s(x)$ так, как это показано в таблице, приведенной на рис. 5.1. Эта таблица называется таблицей значений синдромов. Вычисляя $s(x)$ по принятому $v(x)$, декодер находит $s(x)$ в таблице значений синдромов, а затем соответствующий многочлен $e(x)$. Если таблица значений синдромов не очень велика, то практически ее можно реализовать либо с помощью памяти, либо с помощью цепей комбинаторной логики.

$e(x)$	$s(x)$
1	$R_{g(x)}[1]$
x	$R_{g(x)}[x]$
x^2	$R_{g(x)}[x^2]$
$\vdots$	$\vdots$
$1+x$	$R_{g(x)}[1+x]$
$1+x^2$	$R_{g(x)}[1+x^2]$
$\vdots$	$\vdots$

Рис. 5.1. Таблица значений синдромов.

Временно будем считать эту простую схему декодирования удовлетворительной. В следующих главах будут разработаны более эффективные, чем запоминание заранее вычисленной таблицы, алгоритмы вычисления $e(x)$ по многочлену $s(x)$.

5.3. МИНИМАЛЬНЫЕ МНОГОЧЛЕНЫ И СОПРЯЖЕНИЯ

Мы видели, что циклический код длины n над полем $GF(q)$ существует для каждого многочлена $g(x)$ над $GF(q)$, делящего многочлен $x^n - 1$. Теперь мы хотим более подробно изучить порождающие многочлены. Прежде всего мы хотим найти порождающие многочлены циклических кодов длины n. Наиболее естественный подход состоит в разложении многочлена $x^n - 1$ на простые множители:

$$x^n - 1 = f_1(x) f_2(x) \dots f_s(x),$$

где s — число простых множителей. Произведение произвольного подмножества этих множителей дает порождающий многочлен $g(x)$. Если все простые множители многочлена $x^n - 1$ различны, то всего имеется $2^s - 2$ различных нетривиальных циклических кодов длины n (исключаются тривиальные случаи $g(x) = 1$ и $g(x) = x^n - 1$). Какие из них, если они вообще имеются, дают коды с бóльшим минимальным расстоянием — вопрос, на который сразу ответить нельзя.

Предположим, что $g(x)$ представляет собой порождающий многочлен. Он делит многочлен $x^n - 1$, и, следовательно, $g(x) = \prod f_i(x)$, где произведение берется по некоторому подмножеству множества s простых многочленов. Порождаемый многочленом $g(x)$ циклический код состоит из многочленов, которые делятся на каждый из этих $f_i(x)$. Многочлен $g(x)$ можно найти, найдя все его простые делители.

В данном параграфе мы хотим проследить взаимосвязь простых многочленов с их корнями, лежащими в расширении поля. В частности, мы хотим исследовать, как по заданному множеству корней можно найти простые многочлены и, следовательно, порождающие многочлены. В гл. 7 мы увидим, как надо выбирать корни, чтобы гарантировать хороший код.

Мы начнем с одного предпочитаемого значения n, называемого примитивной длиной кода.

Определение 5.3.1. Для кода над $GF(q)$ длина n вида $n = q^m - 1$ называется *примитивной*. Циклический код примитивной длины над $GF(q)$ называется *примитивным циклическим кодом*.

Поле $GF(q^m)$ является расширением поля $GF(q)$. Согласно теореме об однозначном разложении, разложение

$$x^{q^m-1} - 1 = f_1(x) \ldots f_s(x)$$

над полем $GF(q)$ однозначно. Так как $g(x)$ делит многочлен $x^{q^m-1} - 1$, то он должен быть равен произведению некоторых из этих множителей. С другой стороны, каждый ненулевой элемент поля $GF(q^m)$ является корнем многочлена $x^{q^m-1} - 1$. Следовательно, в поле расширения $GF(q^m)$ многочлен $x^{q^m-1} - 1$ можно представить в виде

$$x^{q^m-1} - 1 = \prod_j (x - \beta_j),$$

где β_j — все ненулевые элементы поля $GF(q^m)$. Отсюда следует, что каждый из многочленов $f_l(x)$ может быть разложен в произведение некоторых из этих линейных множителей и что каждый элемент β_j является корнем точно одного из $f_l(x)$. Этот многочлен $f_l(x)$ является *минимальным многочленом элемента* β_j, и мы также [1]) обозначим его через $f_j(x)$. Он представляет собой многочлен наименьшей степени с коэффициентами из основного поля $GF(q)$, для которого элемент β_j из расширения поля является корнем.

Теперь мы можем связать данное определение циклического кода с использованной в § 5.1 его трактовкой.

Теорема 5.3.2. *Пусть элементы* $\beta_1, \ldots, \beta_r$ *из поля* $GF(q^m)$ *являются корнями порождающего многочлена* $g(x)$ *примитивного кода. Многочлен* $c(x)$ *над* $GF(q)$ *является кодовым тогда и только тогда, когда*

$$c(\beta_1) = c(\beta_2) = \ldots = c(\beta_r) = 0,$$

---

[1]) Мы извиняемся за неопределенность в использовании $f_l(x)$ при $l = 1, \ldots, s$ для обозначения различных минимальных многочленов и $f_j(x)$ при $j = 1, \ldots, q^m - 1$ для обозначения минимальных многочленов элементов β_j. Конкретный смысл всегда будет ясен из контекста.

где $c(\beta_j)$ вычисляется в поле $GF(q^m)$.

Доказательство. Если $c(x) = a(x)\,g(x)$, то имеем $c(\beta_j) = a(\beta_j)\,g(\beta_j) = 0$. Обратно, предположим, что $c(\beta_j) = 0$, и запишем

$$c(x) = Q(x)\,f_j(x) + s(x),$$

где степень $s(x)$ меньше степени $f_j(x)$, а $f_j(x)$ — минимальный многочлен элемента β_j. Но тогда из равенства

$$0 = c(\beta_j) = Q(\beta_j)\,f_j(\beta_j) + s(\beta_j) = s(\beta_j)$$

следует, что $s(x) = 0$. Следовательно, $c(x)$ должен делиться на $f_j(x)$ для всех $j = 1, \ldots, r$, а поэтому и на НОК $[f_1(x), f_2(x), \ldots, f_r(x)] = g(x)$. □

Чтобы проиллюстрировать эти идеи, выберем $n = 15$. Можно найти все циклические коды длины 15, разложив $x^{15} - 1$ на простые многочлены:

$$x^{15} - 1 = (x + 1)(x^2 + x + 1)(x^4 + x + 1)(x^4 + x^3 + 1)(x^4 + x^3 + x^2 + x + 1).$$

Справедливость такого разложения можно проверить перемножением, а простоту множителей — методом проб и ошибок. Всего имеется $2^5 = 32$ подмножества множества этих простых множителей и, следовательно, 32 порождающих многочлена циклических кодов длины 15. Два из них ($g(x) = x^{15} - 1$ с $k = 0$ и $g(x) = 1$ с $k = n$) являются тривиальными, а 30 приводят к нетривиальным циклическим кодам. В качестве примера рассмотрим один из них, положив

$$g(x) = (x^4 + x^3 + 1)(x^4 + x^3 + x^2 + x + 1) = x^8 + x^4 + x^2 + x + 1.$$

Так как степень $g(x)$ равна 8, то $n - k = 8$ и $k = 7$, а так как вес многочлена $g(x)$ равен 5, то минимальное расстояние не превосходит 5. В гл. 7 мы увидим, что минимальное расстояние равно 5. Следовательно, рассматриваемый код является (15, 7, 5)-кодом и позволяет исправлять две ошибки. Проверочные соотношения могут быть записаны в виде

$$c(\alpha^i) = 0, \quad c(\alpha^j) = 0,$$

где α^i и α^j — некоторые корни многочленов $x^4 + x^3 + 1$ и $x^4 + x^3 + x^2 + x + 1$ соответственно из расширения $GF(16)$. В частности, $c(\alpha) = 0$ и $c(\alpha^3) = 0$, и, следовательно, это рассмотренный в § 5.1 пример. (Многочлен $g(x)$ имеет, конечно, и другие корни, но этих двух корней достаточно, чтобы определить $g(x)$, и нет необходимости проверять остальные.)

Предположим теперь, что мы хотим построить порождающий многочлен $g(x)$, имеющий своими корнями элементы $\beta_1, \ldots, \beta_r$. Обозначим через $f_1(x), \ldots, f_r(x)$ минимальные многочлены этих элементов. Тогда

$$g(x) = \text{НОК}\,[f_1(x), f_2(x), \ldots, f_r(x)],$$

и таким образом задача сводится к отысканию минимального многочлена $f(x)$ заданного элемента β.

Пусть степень минимального многочлена $f(x)$ элемента β равна m'. Тогда в $GF(q^m)$ он должен иметь m' корней. Эти дополнительные корни можно описать следующими двумя теоремами.

Теорема 5.3.3. *Пусть характеристика поля $GF(q)$ равна p. Тогда для любого многочлена $s(x)$ над $GF(q)$ и произвольного целого m выполняется равенство*

$$\left[\sum_{i=0}^{I} s_i x^i\right]^{p^m} = \sum_{i=0}^{I} s_i^{p^m} x^{ip^m};$$

это равенство справедливо также при замене p любой его степенью.

Доказательство. Начнем с $m = 1$ и будем рассуждать так же, как при доказательстве теоремы 4.6.10. Чтобы сократить число шагов, определим многочлен $s'(x)$ равенством $s(x) = s'(x)\,x + s_0$; тогда

$$[s(x)]^p = \sum_{i=0}^{I} \binom{p}{i} [s'(x)x]^i\, s_0^{p-i}.$$

Но

$$\binom{p}{i} = \frac{p!}{i!\,(p-i)!} = \frac{p\,(p-1)!}{i!\,(p-i)!},$$

и p является простым числом, которое не входит в знаменатель, за исключением случаев $i = 0$ и $i = p$, так что $\binom{p}{i}$ представляет собой целое число, кратное p, т. е. равное нулю по модулю p. Следовательно,

$$[s(x)]^p = [s'(x)]^p\, x^p + s_0^p.$$

Применим теперь те же рассуждения к многочлену $s'(x)$ и продолжим вычисления. Это даст доказательство утверждения для $m = 1$:

$$[s(x)]^p = \sum_{i=0}^{I} s_i^p x^{ip}.$$

Далее,

$$[s(x)]^{p^2} = [[s(x)]^p]^p = \left[\sum_{i=0}^{I} s_i^p x^{ip}\right]^p = \sum_{i=0}^{I} s_i^{p^2} x^{ip^2}.$$

Эти вычисления можно повторить произвольное число раз, так что теорема верна при замене p любой его степенью. □

Теорема 5.3.4. *Предположим, что $f(x)$ над $GF(q)$ является минимальным многочленом элемента β из $GF(q^m)$. Тогда $f(x)$ является также минимальным многочленом элемента β^q.*

Доказательство. Так как q равно степени характеристики p поля, то теорема 5.3.3 дает

$$[f(x)]^q = \sum_{i=0}^{\deg f(x)} f_i^q (x^q)^i.$$

Но коэффициенты f_i принадлежат полю $GF(q)$, а все элементы этого поля удовлетворяют условию $\gamma^q = \gamma$. Следовательно,

$$[f(x)]^q = \sum_{i=0}^{\deg f(x)} f_i (x^q)^i = f(x^q),$$

и так как $f(\beta) = 0$, то $0 = [f(\beta)]^q = f(\beta^q)$, и, следовательно, β^q является корнем многочлена $f(x)$. Поскольку $f(x)$ — простой многочлен, он является минимальным многочленом элемента β^q, и теорема доказана. □

Определение 5.3.5. Два элемента из поля $GF(q^m)$, являющиеся корнями одного и того же минимального многочлена над $GF(q)$, называются *сопряженными* (относительно поля $GF(q)$).

В общем случае у элемента может быть больше, чем один, сопряженный элемент — на самом деле m. Необходимо также отметить, что отношение сопряженности зависит от основного поля.

Два элемента из $GF(16)$ могут быть сопряжены относительно поля $GF(2)$ и не сопряжены относительно поля $GF(4)$.

Используя теорему 5.3.4, легко выписать все сопряженные с β элементы. Если многочлен $f(x)$ является минимальным для элемента β, то он минимален и для элемента β^q, и для β^{q^2} и т. д. Следовательно, сопряжены все элементы множества

$$\{\beta, \beta^q, \beta^{q^2}, \ldots, \beta^{q^{r-1}}\},$$

где r — наименьшее целое число, такое, что $\beta^{q^r} = \beta$. Заметим, что так как $\beta^{q^m} = \beta$, то $r \leqslant m$. Выписанное выше множество называется *множеством сопряженных элементов*. Все сопряженные элементы являются корнями многочлена $f(x)$, и следующая теорема показывает, что других корней нет.

Теорема 5.3.6. *Минимальный многочлен элемента β равен*

$$f(x) = (x - \beta)(x - \beta^q) \ldots (x - \beta^{q^{r-1}}).$$

Доказательство. Согласно теореме 5.3.4, все эти элементы действительно должны быть корнями минимального многочлена

элемента β, так что степень минимального многочлена не может быть меньше. Поэтому нам надо только показать, что все коэффициенты $f(x)$ принадлежат полю $GF(q)$. Воспользуемся тем фактом, что все корни многочлена $x^q - x$ образуют подполе поля $GF(q^m)$. Прежде всего вычислим $[f(x)]^q$:

$$[f(x)]^q = (x-\beta)^q (x-\beta^q)^q \ldots (x-\beta^{q^{r-1}})^q = \\ = (x^q-\beta^q)(x^q-\beta^{q^2}) \ldots (x^q-\beta),$$

где второе равенство вытекает из теоремы 5.3.3 и того факта, что $\beta^{q^r} = \beta$. Таким образом,

$$[f(x)]^q = f(x^q) = \sum_i f_i x^{iq},$$

в то время как теорема 5.3.3 утверждает, что

$$[f(x)]^q = \left[\sum_i f_i x^i\right]^q = \sum_i f_i^q x^{iq}.$$

Следовательно, $f_i^q = f_i$ для каждого i, и f_i принадлежит подполю $GF(q)$, что и требовалось доказать. □

Например, выберем α равным примитивному элементу поля $GF(256)$. Тогда множество

$$\{\alpha, \alpha^2, \alpha^4, \alpha^8, \alpha^{16}, \alpha^{32}, \alpha^{64}, \alpha^{128}\}$$

представляет собой множество сопряженных элементов. Оно заканчивается элементом α^{128}, так как $\alpha^{255} = 1$, и, следовательно, $\alpha^{256} = \alpha$ — элементу, уже содержащемуся в множестве. Минимальный многочлен элемента α равен многочлену

$$f(x) = (x-\alpha)(x-\alpha^2)(x-\alpha^4) \ldots (x-\alpha^{64})(x-\alpha^{128}),$$

все коэффициенты которого (после раскрытия скобок) будут элементами поля $GF(2)$. Аналогично множеством сопряженных элементов, содержащих элемент α^7, будет

$$\{\alpha^7, \alpha^{14}, \alpha^{28}, \alpha^{56}, \alpha^{112}, \alpha^{224}, \alpha^{193}, \alpha^{131}\},$$

а минимальный многочлен элемента α^7 равен

$$f(x) = (x-\alpha^7)(x-\alpha^{14})(x-\alpha^{28}) \ldots (x-\alpha^{193})(x-\alpha^{131}),$$

все коэффициенты которого (после раскрытия скобок) будут элементами поля $GF(2)$.

Теперь вместо $GF(2)$ выберем в качестве основного поля $GF(4)$, которое также является подполем поля $GF(256)$; тогда множеством сопряженных элементов, содержащим α^7, будет

$$\{\alpha^7, \alpha^{28}, \alpha^{112}, \alpha^{193}\},$$

и минимальный многочлен элемента α^7 над полем $GF(4)$ равен

$$f(x) = (x-\alpha^7)(x-\alpha^{28})(x-\alpha^{112})(x-\alpha^{193}),$$

все коэффициенты которого (после раскрытия скобок) будут элементами поля GF (4). Для того чтобы указать эти элементы из поля GF (4), надо, однако, выделить элементы подполя GF (4) среди элементов поля GF (256). В поле GF (256) подполе GF (4) состоит из элементов $\{0, 1, \alpha^{85}, \alpha^{170}\}$, так как элементы α^{85} и α^{170} являются единственными элементами поля GF (256) порядка 3.

Вернемся теперь к циклическим кодам, которые не являются примитивными. Мы видели, что для примитивных циклических кодов существует тесная связь между порождающим многочленом $g(x)$ и полем $GF(q^m)$, описываемая корнями многочлена $g(x)$. Теперь мы хотим рассмотреть коды длины n, отличной от $q^m - 1$. Следующая теорема устанавливает связь этих кодов с полученными ранее результатами.

Теорема 5.3.7. *Пусть $GF(q)$ — конечное поле. Если n и q взаимно просты, то многочлен $x^n - 1$ делит многочлен $x^{q^m-1} - 1$ при некотором m и многочлен $x^n - 1$ имеет n различных корней в расширении $GF(q^m)$ поля $GF(q)$.*

Доказательство. Необходимо доказать только, что n делит $q^m - 1$ при некотором m, так как в этом случае можно воспользоваться общим разложением

$$(x^r - 1) = (x - 1)(x^{r-1} + x^{r-2} + x^{r-3} + \ldots + x + 1),$$

чтобы показать, что при $q^m - 1 = nb$ для некоторого b имеет место равенство

$$x^{q^m-1} - 1 = (x^n)^b - 1 =$$
$$= (x^n - 1)(x^{n(b-1)} + x^{n(b-2)} + \ldots + x^n + 1).$$

Таким образом, $x^n - 1$ делит $x^{q^m-1} - 1$ и, следовательно, имеет n различных корней в $GF(q^m)$, так как $x^{q^m-1} - 1$ имеет $q^m - 1$ различных корней в этом поле.

Чтобы доказать, что n делит $q^m - 1$ для некоторого m, воспользуемся алгоритмом деления и выпишем $n + 1$ следующих равенств:

$$\begin{aligned} q &= Q_1 n + s_1, \\ q^2 &= Q_2 n + s_2, \\ q^3 &= Q_3 n + s_3, \\ &\vdots \\ q^n &= Q_n n + s_n, \\ q^{n+1} &= Q_{n+1} n + s_{n+1}. \end{aligned}$$

Все остатки расположены между 0 и $n-1$. Так как их всего $n+1$, то по меньшей мере два из них равны между собой, скажем s_i и s_j при i, меньшем j. Тогда

$$q^j - q^i = Q_j n + s_j - Q_i n - s_i,$$

или

$$q^i (q^{j-i} - 1) = (Q_j - Q_i) n.$$

Так как q и n взаимно просты, то n должно делить $q^{j-i} - 1$. Полагая $m = j - i$, завершаем доказательство. □

Используя эту теорему, любой циклический код в случае взаимно простых n и q можно описать в соответствующем расширении поля. Порождающий многочлен $g(x)$ циклического кода длины n делит многочлен $x^n - 1$, который делит $x^{q^m-1} - 1$, так что $g(x)$ делит также $x^{q^m-1} - 1$. Мы всегда будем пользоваться наименьшим m, для которого это выполняется. Пусть α — примитивный элемент поля $GF(q^m)$, пусть $q^m - 1 = nb$, и пусть $\beta = \alpha^b$. Тогда все корни многочлена $x^n - 1$ (а поэтому и многочлена $g(x)$) исчерпываются степенями элемента β. Простые делители многочлена $x^n - 1$ имеют своими корнями только такие элементы.

Подводя итоги, скажем, что если мы используем $\beta = \alpha^b$ вместо α и ограничиваем множество корней порождающего многочлена только степенями β, то мы получаем циклический код длины $n = (q^m - 1)/b$.

5.4. МАТРИЧНОЕ ОПИСАНИЕ ЦИКЛИЧЕСКИХ КОДОВ

Эту главу мы начали кратким обсуждением связи проверочной матрицы с корнями многочленов, лежащими в расширении поля. В следующих параграфах, в которых рассматривались циклические коды, проверочным матрицам уделялось мало внимания. А сейчас самое время заняться связью циклических кодов с их порождающими и проверочными матрицами.

Имеется много способов формирования этих матриц. Прежде всего проверочную матрицу можно формировать в расширении поля так, как это было сделано в § 5.1. Если нулями $g(x)$ являются элементы $\gamma_j \in GF(q^m)$ при $j = 1, \ldots, r$, то

$$\sum_{i=0}^{n-1} c_i \gamma_j^i = 0, \quad j = 1, \ldots, r,$$

что в матричном виде можно переписать как

$$\mathbf{H} = \begin{bmatrix} \gamma_1^0 & \gamma_1^1 & \dots & \gamma_1^{n-1} \\ \gamma_2^0 & \gamma_2^1 & \dots & \gamma_2^{n-1} \\ \cdot & & & \cdot \\ \cdot & & & \cdot \\ \cdot & & & \cdot \\ \gamma_r^0 & \gamma_r^1 & \dots & \gamma_r^{n-1} \end{bmatrix}.$$

Заменим теперь эту $(r \times n)$-матрицу над $GF(q^m)$ на $(rm \times n)$-матрицу над $GF(q)$, заменяя каждый элемент β вектором-столбцом коэффициентов многочлена, представляющего β над $GF(q)$. Это дает проверочную матрицу над $GF(q)$, но некоторые строки этой матрицы могут быть линейно зависимыми и, следовательно, являются излишними. Удалим наименьшее число строк, необходимое для построения матрицы с линейно независимыми строками. Это приведет к проверочной матрице кода.

Хотя описанная процедура проясняет связь между корнями порождающего многочлена и проверочной матрицей кода, она слишком сложна для использования. По порождающему многочлену можно построить необходимые матрицы, даже не переходя в расширение поля. Одним из способов сделать это является построение порождающей матрицы непосредственно по порождающему многочлену. Так как кодовые слова записываются в виде $c(x) = i(x)\, g(x)$, то в матричной форме имеем

$$\mathbf{G} = \begin{bmatrix} 0 & \dots & 0 & g_{n-k} & g_{n-k-1} & \dots & g_2 & g_1 & g_0 \\ 0 & & g_{n-k} & g_{n-k-1} & g_{n-k-2} & \dots & g_1 & g_0 & 0 \\ 0 & & g_{n-k-1} & g_{n-k-2} & g_{n-k-3} & \dots & g_0 & 0 & 0 \\ \cdot & & & & & & & & \cdot \\ \cdot & & & & & & & & \cdot \\ \cdot & & & & & & & & \cdot \\ g_{n-k} & \dots & & & & & & 0 & 0 \end{bmatrix}.$$

Проверочная матрица соответственно равна

$$\mathbf{H} = \begin{bmatrix} 0 & 0 & 0 & \dots & & & & h_{k-1} & h_k \\ \cdot & & & & & & & \cdot & \cdot \\ \cdot & & & & & & & \cdot & \cdot \\ 0 & h_0 & h_1 & \dots & h_{k-1} & h_k & 0 & 0 & 0 \\ h_0 & h_1 & h_2 & \dots & h_k & 0 & 0 \dots & 0 & 0 \end{bmatrix},$$

где $h(x)$ — проверочный многочлен циклического кода. Для проверки равенства

$$\mathbf{G}\mathbf{H}^T = \mathbf{0}$$

рассмотрим величины $u_r = \sum_{i=0}^{r} g_{r-i} h_i$, в силу равенства $h(x) \times g(x) = x^n - 1$ равные нулю при $0 < r < n$. Но тогда

$$\mathbf{G}\mathbf{H}^{\mathrm{T}} = \begin{bmatrix} u_{n-1} & u_{n-2} & \dots & u_k \\ u_{n-2} & u_{n-3} & \dots & u_{k-1} \\ \cdot & \cdot & & \\ \cdot & \cdot & & \\ \cdot & \cdot & & \\ u_{n-k} & u_{n-k-1} & \dots & u_1 \end{bmatrix} = \mathbf{0},$$

и, следовательно, так определенная матрица $\mathbf{H}$ действительно является проверочной матрицей. Теперь видно, что дуальный код $\mathscr{C}^{\perp}$ для циклического кода с порождающим многочленом $g(x)$ также является циклическим, так как $\mathbf{H}$ представляет собой его порождающую матрицу и имеет вид порождающей матрицы циклического кода. Порождающий многочлен дуального кода равен взаимному к $h(x)$ многочлену $\bar{h}(x) = x^k h(x^{-1})$. Иногда при рассмотрении циклических кодов дуальным называют код, порожденный многочленом $h(x)$, но если быть точным, то в этом случае можно говорить только о коде, эквивалентном дуальному коду.

Можно также просто получить порождающую матрицу в систематическом виде. Воспользуемся алгоритмом деления и выпишем для каждой информационной позиции $i = 1, \dots, k$ многочлен

$$x^{n-i} = Q_i(x) g(x) \mid s_i(x), \quad i = 1, \dots, k,$$

где

$$s_i(x) = \sum_{j=0}^{n-k-1} s_{ji} x^j.$$

Тогда многочлен $x^{n-i} - s_i(x)$ является кодовым словом, так как

$$x^{n-i} - s_i(x) = Q_i(x) g(x).$$

Используя коэффициенты многочлена в левой части равенства в качестве элементов порождающей матрицы, получаем

$$\mathbf{G} = \begin{bmatrix} -s_{0,k} & \dots & -s_{(n-k-1),k} & 1 & 0 & \dots & 0 \\ -s_{0,k-1} & & -s_{(n-k-1),k-1} & 0 & 1 & \dots & 0 \\ \cdot & & & & & & \\ \cdot & & & & & & \\ \cdot & & & & & & \\ -s_{0,1} & \dots & -s_{(n-k-1),1} & 0 & 0 & \dots & 1 \end{bmatrix}.$$

В этой систематической порождающей матрице индексы координат информационных символов пробегают числа от $n-k$ до $n-1$. Чтобы выписать такую порождающую матрицу, необ-

ходимо только вычислить все остатки $s_i(x)$ от деления x^i на $g(x)$. Используя описанные в § 3.2 методы, теперь можно выписать и проверочную матрицу кода. Она равна

$$\mathbf{H} = \begin{bmatrix} 1 & 0 & \dots & 0 & s_{0,k} & \dots & s_{0,1} \\ 0 & 1 & \dots & 0 & s_{1,k} & \dots & s_{1,1} \\ \cdot & & & \cdot & & & \cdot \\ \cdot & & & \cdot & & & \cdot \\ \cdot & & & \cdot & & & \cdot \\ 0 & 0 & \dots & 1 & s_{(n-k-1),k} & & s_{(n-k-1),1} \end{bmatrix}.$$

5.5. КОДЫ ХЭММИНГА КАК ЦИКЛИЧЕСКИЕ КОДЫ

Порождающий многочлен (7,4)-кода Хэмминга, использованного в начале главы, равен $g(x) = x^3 + x + 1$. Корнем этого многочлена является примитивный элемент α поля $GF(8)$, и, следовательно, все кодовые слова удовлетворяют равенству $c(\alpha) = 0$. Аналогично, чтобы получить код Хэмминга с примитивной длиной $n = 2^m - 1$, выберем $g(x)$ так, чтобы примитивный элемент α поля $GF(2^m)$ был его корнем, а именно положим $g(x)$ равным минимальному многочлену $p(x)$ элемента α, используемому для построения поля $GF(2^m)$. Тогда так как $c(x) = a(x)\,g(x)$, то $c(\alpha) = 0$ для каждого кодового слова. Это эквивалентно равенству $\mathbf{cH}^T = 0$, где

$$\mathbf{H} = [\alpha^0\ \alpha^1 \dots \alpha^{n-1}].$$

Для бóльших алфавитов также существуют коды Хэмминга. В § 3.4 мы видели, что для каждого m существует код Хэмминга над полем $GF(q)$ с $n = (q^m - 1)/(q - 1)$ и $k = (q^m - 1)/(q - 1) - m$. В данном параграфе мы покажем, что многие из этих кодов (хотя и не все) являются циклическими, выписав их порождающие многочлены.

Пусть α — примитивный элемент поля $GF(q^m)$, и пусть $\beta = \alpha^{q-1}$. Тогда $\beta^{(q^m-1)/(q-1)} = 1$, и, следовательно, β является корнем многочлена $x^{(q^m-1)/(q-1)} - 1$. Таким образом, минимальный многочлен элемента β является делителем многочлена $x^{(q^m-1)/(q-1)} - 1$ и может быть выбран в качестве порождающего многочлена циклического кода длины $n = (q^m - 1)/(q - 1)$. Проверочная матрица равна $\mathbf{H} = [\beta^0\ \beta^1 \dots \beta^{n-1}]$.

Для $q = 2$ и $\beta = \alpha$ легко доказать, что этот код исправляет одну ошибку, указав простую процедуру исправления одной ошибки. Принятое слово записывается многочленом степени $n - 1$ в виде

$$v(x) = a(x)\,g(x) + e(x),$$

где многочлен $e(x)$ содержит не более одного ненулевого коэффициента, так что $e(x) = 0$ или $e(x) = x^i$. Целое число i маркирует позицию, в которой произошла ошибка. Для маркировки ошибочных позиций мы будем также использовать поле $GF(2^m)$. Элемент поля α^i соответствует компоненте с номером i. Так как $g(\alpha) = 0$, то $v(\alpha) = \alpha^i$, и все степени α от 0 до $2^m - 2$ различны. Таким образом, по величине α^i сразу определяется позиция ошибки, за исключением случая $v(\alpha) = 0$, соответствующего отсутствию ошибки. Следовательно, рассматриваемый код является исправляющим одиночную ошибку кодом над $GF(2)$ с $n = 2^m - 1$ и $k = n - m$; на самом деле это код Хэмминга над $GF(2)$.

Мы доказали для случая $q = 2$ формулируемую ниже теорему. Общий случай этого утверждения более сложен, так как для произвольного элемента γ из $GF(q)$ все величины вида $\gamma\beta^i$ должны быть различны. Докажем формально теорему для произвольного q.

Теорема 5.5.1. *Минимальное расстояние кода с проверочной матрицей* $\mathbf{H} = [\beta^0\ \beta^1 \ldots \beta^{n-1}]$, *где* $\beta = \alpha^{q-1}$ *и* $n = (q^m - 1)/(q - 1)$, *равно по меньшей мере* 3 *тогда и только тогда, когда* n *и* $q - 1$ *взаимно просты, что возможно тогда и только тогда, когда* m *и* $q - 1$ *взаимно просты.*

Доказательство. Предположим, что два столбца матрицы $\mathbf{H}$ линейно зависимы: $\beta^i = \gamma\beta^j$, где элемент γ принадлежит $GF(q)$. Тогда элемент β^{i-j} принадлежит полю $GF(q)$ и, следовательно, является корнем уравнения $x^{q-1} - 1 = 0$. Но ненулевые элементы поля $GF(q)$ можно представить через примитивный элемент α поля $GF(q^m)$ в виде первых $q - 1$ степеней элемента $\alpha^{(q^m-1)/(q-1)}$, так как все такие степени различны и

$$\left(\left(\alpha^{(q^m-1)/(q-1)}\right)^k\right)^{q-1} = 1.$$

Тогда для некоторого k, меньшего $q - 1$, имеем

$$\beta^{i-j} = \left(\alpha^{(q^m-1)/(q-1)}\right)^k = \alpha^{nk}.$$

Далее, так как $\beta = \alpha^{q-1}$, то

$$(q - 1)(i - j) = nk.$$

Так как $i - j$ меньше, чем n, то это уравнение разрешимо относительно $i - j$ тогда и только тогда, когда n и $q - 1$ не являются взаимно простыми; таким образом, $\mathbf{H}$ содержит два линейно зависимых столбца тогда и только тогда, когда n и $q - 1$ не являются взаимно простыми.

Покажем теперь, что n и $q - 1$ взаимно просты тогда и только тогда, когда m и $q - 1$ также взаимно просты. Но

$$n = (q^m - 1)/(q - 1) = q^{m-1} + q^{m-2} + \cdots + 1$$

и

$$q^{m-j} = (q-1)\,s_j + 1$$

для некоторого s_j, поскольку $q^{m-j} - 1$ делится на $q - 1$. Следовательно, суммируя по j, получаем

$$n = m + (q-1)\sum_{j=1}^{m} s_j,$$

так что n и $q - 1$ взаимно просты тогда и только тогда, когда m и $q - 1$ взаимно просты. □

Согласно доказанной теореме, коды Хэмминга над $GF(q)$ длины $n = (q^m - 1)/(q - 1)$ являются циклическими, если m и $q - 1$ взаимно просты. В § 3.4 мы рассматривали, однако, некоторые коды Хэмминга, для которых m и $q - 1$ не взаимно просты. Для этих кодов такое циклическое построение не работает. Простейшим их примером является (21,18)-код Хэмминга над $GF(4)$. Этот код нельзя описать через порождающий многочлен. Однако большинство представляющих интерес кодов Хэмминга являются циклическими.

В качестве примера циклического кода Хэмминга рассмотрим (85,81)-код над $GF(4)$, для которого выпишем порождающий многочлен. Это построение реализуется в поле $GF(256)$ и сводится к нахождению минимального многочлена над $GF(4)$ элемента $\beta = \alpha^3$. Элементами порядка 3 в поле $GF(256)$ являются только α^{85} и α^{170}, так что в поле $GF(256)$ поле $GF(4)$ таково:

$$GF(4) = \{0, 1, \alpha^{85}, \alpha^{170}\}.$$

Множеством сопряженных элементов, содержащих β, относительно поля $GF(2)$ является

$$\{\beta, \beta^2, \beta^4, \beta^8, \beta^{16}, \beta^{32}, \beta^{64}, \beta^{128}\}.$$

Но нам нужны сопряженные относительно поля $GF(4)$ элементы, а они образуют множество

$$\{\beta, \beta^4, \beta^{16}, \beta^{64}\}.$$

Следовательно, искомый порождающий многочлен равен

$$g(x) = (x - \beta)(x - \beta^4)(x - \beta^{16})(x - \beta^{64}) =$$
$$= (x - \alpha^3)(x - \alpha^{12})(x - \alpha^{48})(x - \alpha^{192}).$$

Раскрыв скобки, мы получим многочлен, все коэффициенты которого принадлежат полю $GF(4)$. Таким образом, дальше нам уже не понадобится структура поля $GF(256)$. Теперь поступим следующим образом. Используя примитивный многочлен $p(x) = x^8 + x^4 + x^3 + x^2 + 1$, имеем $\alpha^8 = \alpha^4 + \alpha^3 + \alpha^2 + 1$. Неоднократно повторяя эту подстановку или используя таблицу

логарифмов для умножения в поле $GF(256)$ (в обоих случаях вычисления займут примерно час), находим

$$g(x) = x^4 + x^3 + \alpha^{170}x + 1.$$

Наконец, поскольку мы уже оставили работу в поле $GF(256)$, то заменим обозначения на более естественные для поля $GF(4)$ и получим

$$g(x) = x^4 + x^3 + 3x + 1.$$

Это и есть искомый порождающий многочлен (85,81)-кода Хэмминга над полем $GF(4)$.

5.6. ЦИКЛИЧЕСКИЕ КОДЫ, ИСПРАВЛЯЮЩИЕ ДВЕ ОШИБКИ

В предыдущем параграфе было дано описание кодов Хэмминга как циклических кодов, порождающие многочлены которых имеют корень в соответствующем расширении основного поля. Описанные коды исправляют одну ошибку. Сейчас мы возвращаемся к исправляющим две ошибки кодам над полем $GF(2)$. Пусть длина кода имеет вид $n = 2^m - 1$ для некоторого m, и пусть α — примитивный элемент поля $GF(2^m)$. Мы рассмотрим те двоичные коды, для которых α и α^3 являются корнями порождающих многочленов, и покажем, что такие коды позволяют исправлять две ошибки.

Пусть $g(x)$ — многочлен наименьшей степени, для которого α и α^3 из $GF(2^m)$ являются корнями. Описав процедуру декодирования, позволяющую исправлять все одиночные и двойные ошибки, мы докажем, что минимальное расстояние этого кода равно по меньшей мере 5.

Принятое слово записывается многочленом степени $n-1$ вида

$$v(x) = a(x)\,g(x) + e(x),$$

где $e(x)$ содержит не более двух ненулевых коэффициентов, поскольку мы рассматриваем случай исправления не более двух ошибок. Таким образом, или $e(x) = 0$, или $e(x) = x^i$, или $e(x) = x^i + x^{i'}$. Целые числа i и i' маркируют позиции, в которых произошли ошибки. Для маркировки ошибочных позиций мы будем также использовать элементы поля $GF(2^m)$. Элемент поля α^i соответствует компоненте с номером i. В этой роли элементы поля называются *локаторами*. Обозначим $X_1 = \alpha^i$ и $X_2 = \alpha^{i'}$. Так как длина кода n равна порядку элемента α, то локаторы X_1 и X_2 определяются однозначно. Если произошла только одна ошибка, то положим $X_2 = 0$; если ошибок не произошло, то $X_1 = X_2 = 0$.

Пусть $S_1 = v(\alpha)$ и $S_3 = v(\alpha^3)$. Эти элементы, известные также под названием компонент синдрома, вычисляются непосредственно по принятому слову $v(x)$. Так как α и α^3 являются корнями $g(x)$, то $S_1 = e(\alpha)$ и $S_3 = e(\alpha^3)$. Предположим, что произошли две ошибки: $S_1 = \alpha^i + \alpha^{i'}$, $S_3 = \alpha^{3i} + \alpha^{3i'}$. Но это в точности дает систему двух уравнений относительно двух неизвестных X_1 и X_2 в поле $GF(2^m)$:

$$S_1 = X_1 + X_2,$$
$$S_3 = X_1^3 + X_2^3.$$

Если может произойти не более двух ошибок, то величина S_1 равна нулю тогда и только тогда, когда не произошло ни одной ошибки. Декодер должен продолжать работу в том случае, когда S_1 отлично от нуля. Если выписанная система из двух нелинейных уравнений однозначно разрешима относительно X_1 и X_2, то две ошибки могут быть исправлены, и минимальное расстояние рассматриваемого кода равно по меньшей мере 5.

Прямого очевидного способа решения такой системы нет. Один из методов состоит во введении нового многочлена, определяемого так, чтобы локаторы ошибок были его корнями:

$$(x - X_1)(x - X_2) = x^2 + (X_1 + X_2)x + X_1X_2.$$

Если мы сумеем найти коэффициенты этого многочлена, то, разложив его на линейные множители, мы сможем найти X_1 и X_2. Но над расширением поля $GF(2)$

$$S_1^3 + S_3 = X_1^2X_1 + X_2^2X_1 = S_1X_1X_2.$$

Таким образом,

$$(x + X_1)(x + X_2) = x^2 + S_1x + (S_1^3 + S_3)/S_1,$$

и $S_1 \neq 0$, если произошли одна или две ошибки. Многочлен в правой части нам известен, поскольку известны S_1 и S_3. Локаторы ошибок являются корнями этого многочлена, а корни любого многочлена над полем определяются однозначно. Следовательно, код исправляет две ошибки.

Указав процедуру декодирования, мы установили, что выбранный многочлен $g(x)$ порождает код, исправляющий две произвольные ошибки. Практически можно использовать любую удобную процедуру декодирования, и одной из них является процедура вычисления корней выписанного выше квадратного уравнения в $GF(2^m)$. Так как всего имеется 2^m возможностей выбора каждого из корней, то это часто делается методом проб и ошибок. Другие схемы декодирования будут рассмотрены в следующих главах.

Рассмотренные в настоящем параграфе коды, исправляющие две ошибки, служат иллюстрацией того, как построить цикли-

ческие коды над некоторым полем, используя лежащие в большем поле корни порождающих эти коды многочленов. В гл. 7 будет рассмотрен общий случай такого построения для произвольного поля символов $GF(q)$ и произвольного числа исправляемых ошибок.

5.7. ЦИКЛИЧЕСКИЕ КОДЫ, ИСПРАВЛЯЮЩИЕ ПАКЕТЫ ОШИБОК

В большинстве случаев коды строятся для исправления любой случайной конфигурации из t ошибок. Некоторые каналы, однако, более чувствительны к пакетам ошибок. Если необходимо исправлять t ошибок, группирующихся в пределах короткого временного интервала, а не произвольную конфигурацию из t ошибок, то можно воспользоваться этим ослаблением требования для того, чтобы строить более эффективные коды, а именно коды с большей скоростью. В настоящем параграфе мы кратко коснемся данного вопроса и приведем некоторые циклические коды, исправляющие пакеты ошибок. В силу цикличности эти коды обладают дополнительными свойствами, обычно не необходимыми: они будут исправлять не только данный пакет ошибок, но также и все его циклические сдвиги — так называемые циклические пакеты ошибок.

Определение 5.7.1. *Циклическим пакетом* длины t называется вектор, все ненулевые компоненты которого расположены среди t последовательных (по циклу) компонент, первая и последняя из которых отличны от нуля.

Пакет ошибок можно описывать многочленом вида $e(x) = x^i b(x) \pmod{x^n - 1}$, где $b(x)$ — многочлен степени не выше $t-1$ с отличным от нуля коэффициентом b_0, Таким образом, $b(x)$ описывает пакет ошибок, а x^i указывает начальный локатор пакета.

Синдромные многочлены $s(x)$ для исправляющего пакеты ошибок циклического кода должны быть различными. А именно: если многочлены

$$s(x) = R_{g(x)}[e(x)]$$

различны при различных многочленах $e(x)$, задающих циклические пакеты длины t, то данный код обладает способностью исправлять все пакеты длины t. Например, многочлен

$$g(x) = x^6 + x^3 + x^2 + x + 1$$

порождает двоичный код длины 15. Перечислим все циклические пакеты ошибок длины не более 3:

$$e(x) = x^i, \qquad i = 0, \ldots, 14,$$

$$e(x) = x^i(1 + x) \pmod{x^{15} - 1}, \qquad i = 0, \ldots, 14,$$

$$e(x) = x^i(1 + x^2) \pmod{x^{15} - 1}, \qquad i = 0, \ldots, 14,$$

$$e(x) = x^i(1 + x + x^2) \pmod{x^{15} - 1}, \qquad i = 0, \ldots, 14.$$

Непосредственным вычислением легко проверить, что синдромы для каждой из этих 56 ошибок различны, и, следовательно, порождаемый многочленом $g(x)$ циклический код позволяет исправлять все пакеты длины 3.

Заметим, что сумма кодового слова и исправляемого пакета не может быть равна сумме другого кодового слова и исправляемого пакета ошибок. В частности, в данном примере никакой пакет длины 6 не должен являться кодовым словом. В общем случае если линейный код исправляет все пакеты длины t и меньше, то он не может содержать в качестве кодовых слов пакеты длины $2t$ или меньше.

Следующая теорема является аналогом границы Синглтона для кодов, исправляющих случайные ошибки, хотя доказательство ее отличается от доказательства границы Синглтона. Она справедлива не только для циклических кодов.

Теорема 5.7.2 (граница Рейгера). *Каждый линейный блоковый код, исправляющий все пакеты длины t и менее, должен содержать по меньшей мере $2t$ проверочных символов.*

Доказательство. Предположим, что код исправляет все пакеты ошибок длины t и менее. Тогда он не содержит в качестве кодового слова ни одного пакета длины $2t$ или менее. Если два вектора принадлежат одному и тому же смежному классу, то их разность равна кодовому слову. Выберем два произвольных вектора, все компоненты которых, кроме первых $2t$, равны нулю. Если эти два вектора принадлежат одному смежному классу в стандартном расположении, то их разность равна кодовому слову. Но эта разность представляет собой пакет длины $2t$ и, согласно сказанному ранее, не может быть кодовым словом. Поэтому два таких вектора должны лежать в разных смежных классах, а число смежных классов должно быть по меньшей мере равным числу различных таких векторов. Всего имеется q^{2t} различных векторов, все ненулевые компоненты которых содержатся в первых $2t$ позициях; следовательно, число смежных классов равно по меньшей мере q^{2t}, так что код содержит по меньшей мере $2t$ проверочных символов. □

Таблица 5.1

Некоторые двоичные циклические коды исправляющие пакеты ошибок

Порождающий многочлен	Параметры	Длина исправляемого пакета
$x^4+x^3+x^2+1$	(7, 3)	2
$x^5+x^4+x^2+1$	(15, 10)	2
$x^6+x^5+x^4+x^3+1$	(15, 9)	3
$x^6+x^5+x^4+1$	(31, 25)	2
$x^7+x^6+x^5+x^3+x^2+1$	(63, 56)	2
$x^8+x^7+x^6+x^3+1$	(63, 55)	3
$x^{12}+x^8+x^5+x^3+1$	(511, 499)	4
$x^{13}+x^{10}+x^7+x^6+x^5+x^4+x^2+1$	(1023, 1010)	4

Отметим, что предыдущий пример циклического кода, исправляющего пакеты ошибок, удовлетворяет границе Рейгера со знаком равенства.

Наиболее изученными кодами, исправляющими пакеты ошибок, являются циклические коды, и мы ограничимся рассмотрением только этого класса. Для малых t и умеренных длин кода с помощью поиска на ЭВМ было найдено много хороших циклических кодов над $GF(2)$. Некоторые из этих кодов приведены в табл. 5.1.

Из приведенных в табл. 5.1 кодов можно построить более длинные коды методом перемежения. Чтобы из (n, k)-кода получить (jn, jk)-код, выберем из исходного кода j произвольных кодовых слов и укрупним кодовые слова, чередуя их символы. Если исходный код исправлял произвольный пакет ошибок длины t, то, очевидно, результирующий код будет исправлять все пакеты ошибок длины jt. Например, применяя метод перемежения к четырем копиям (31,25)-кода, получаем (124,100)-код. Так как каждый из четырех исходных кодов исправлял пакет ошибок длины 2, то новый код будет исправлять любой пакет ошибок длины 8.

Для циклических кодов метод перемежения приводит к циклическим кодам. Предположим, что исходный код порождается многочленом $g(x)$. Тогда порождающий многочлен получаемого перемежением кода равен $g(x^j)$. Чтобы установить это, заметим, что перемежение символов нескольких информационных многочленов с последующим умножением на $g(x^j)$ дает то же самое кодовое слово, что и умножение каждого из исходных

информационных многочленов на $g(x)$ с последующим перемежением этих слов (n, k)-кода. Точнее, пусть

$$c_1(x) = i_1(x)\, g(x),$$
$$c_2(x) = i_2(x)\, g(x),$$
$$\vdots$$
$$c_j(x) = i_j(x)\, g(x)$$

представляют собой выбранные кодовые слова. Для формирования слова из кода-перемежения каждое из этих выбранных информационных кодовых слов растягивается вставкой между всеми символами слова $j-1$ нулей. Кодовое слово из кода-перемежения получается затем задержкой и сложением этих слов:

$$c(x) = c_1(x^j) + xc_2(x^j) + \cdots + x^{j-1}c_j(x^j) =$$
$$= i_1(x^j)\, g(x^j) + xi_2(x^j)\, g(x^j) + \cdots + x^{j-1}i_j(x^j)\, g(x^j) =$$
$$= [i_1(x^j) + xi_2(x^j) + \cdots + x^{j-1}i_j(x^j)]\, g(x^j).$$

Стоящий в квадратных скобках член можно разложить так, что получится информационное слово, составленное из исходных информационных слов методом перемежения. Это слово можно заменить любым информационным словом $i(x)$; следовательно,

$$c(x) = i(x)\, g(x^j).$$

Замена $g(x)$ на $g(x^j)$ эквивалентна перемежению j копий кода, порождаемого многочленом $g(x)$.

Кроме найденных на ЭВМ кодов и получающихся из них перемежением кодов известны также коды, построенные аналитическими методами. Одним из классов таких кодов являются коды Файра. Параметры некоторых кодов Файра приведены в табл. 5.2.

Таблица 5.2

Параметры некоторых двоичных кодов Файра

(n, k)	$t = m$
(9, 4)	2
(35, 27)	3
(105, 94)	4
(279, 265)	5
(693, 676)	6
(1651, 1631)	7
(3825, 3802)	8
(8687, 8661)	9
(19437, 19408)	10

Определение 5.7.3. *Кодом Файра* называется исправляющий пакеты ошибок циклический код над $GF(q)$ с порождающим многочленом

$$g(x) = (x^{2t-1} - 1)\, p(x),$$

где $p(x)$ — примитивный многочлен над $GF(q)$, степень m которого не меньше длины t исправляемого пакета и который не делит $x^{2t-1} - 1$. Длина n кода Файра равна наименьшему целому n, такому, что $g(x)$ делит $x^n - 1$.

Теорема 5.7.4. *Длина кода Файра равна* $n = e(2t - 1)$, *где* e *— наименьшее целое число, такое, что* $p(x)$ *делит* $x^e - 1$. *Следовательно, если* $p(x)$ *примитивен, то для этого кода*

$$(n, k) = ((q^m - 1)(2t - 1), (q^m - 1)(2t - 1) - m - 2t + 1).$$

Доказательство. При $n = e(2t - 1)$ возможно несколько разложений многочлена $x^n - 1$:

$$x^n - 1 = x^{e(2t-1)} \quad 1 = (x^e - 1) \sum_{k=0}^{2t-2} x^{ek},$$

$$x^n - 1 = x^{e(2t-1)} - 1 = (x^{2t-1} - 1) \sum_{k=0}^{e-1} x^{(2t-1)k}.$$

Так как $p(x)$ делит $x^e - 1$, то он делит и $x^n - 1$. Так как он не делит $x^{2t-1} - 1$, то он должен делить многочлен $\sum_{k=0}^{e-1} x^{(2t-1)k}$. Таким образом,

$$x^n - 1 = (x^{2t-1} - 1)\, p(x)\, a(x)$$

для некоторого $a(x)$, и так как не существует меньшего n, для которого имеет место это разложение, то такое n равно длине кода.

В частности, если $p(x)$ — примитивный многочлен степени m, то $p(x)$ делит $x^e - 1$ при $e = q^m - 1$ и не делит ни при каком меньшем значении e. □

Теорема 5.7.5. *Код Файра исправляет все пакеты ошибок длины* t *и менее.*

Доказательство. Этот код способен исправлять все пакеты длины t и менее, если никакие два таких пакета, $x^i b_1(x)$ и $x^j b_2(x)$, не принадлежат одному и тому же смежному классу. В силу цикличности кода без потери общности можно полагать, что i равно нулю. Предположим, что два пакета длины t или меньше, $b_1(x)$ и $x^j b_2(x)$, принадлежат одному и тому же смежному классу стандартного расположения рассматриваемого кода. Тогда их разность равна кодовому слову и для некоторого $a(x)$ имеем

$$b_1(x) - x^j b_2(x) = a(x)(x^{2t-1} - 1)\, p(x) \pmod{x^n - 1}.$$

Но многочлен $x^{2t-1}-1$ делит $x^{\nu(2t-1)}-1$ для всех неотрицательных ν. Следовательно, можно записать

$$(x^{\nu(2t-1)}-1)\,b_1(x) = Q(x)\,(x^{2t-1}-1) \pmod{x^n-1}.$$

Складывая эти равенства, получаем

$$x^{\nu(2t-1)}\,[b_1(x) - x^{j-\nu(2t-1)}b_2(x)] = a'(x)\,(x^{2t-1}-1) \pmod{x^n-1},$$

или, что эквивалентно,

$$x^{\nu(2t-1)-(t-1)}\,[x^{t-1}b_1(x) - x^{j+(t-1)-\nu(2t-1)}b_2(x)] = \\ = a''(x)\,(x^{2t-1}-1) \pmod{x^n-1}$$

для некоторых $a'(x)$ и $a''(x)$. Но теперь в каждом из последних двух равенств можно выбрать неотрицательное целое ν, меньшее e, так что $b_2(x)$ будет умножаться на x^k, $0 \leqslant k < t$. Таким образом, за счет выбора ν получаем

$$x^{\nu(2t-1)}\,[b_1(x) - x^k b_2(x)] = a'(x)\,(x^{2t-1}-1) \pmod{x^n-1},$$

или

$$x^{\nu(2t-1)-(t-1)}\,[x^{t-1}b_1(x) - x^k b_2(x)] = \\ = a''(x)\,(x^{2t-1}-1) \pmod{x^n-1},$$

где $\deg b_1(x) < t$, $\deg b_2(x) < t$ и $k < t$, так что степень многочлена в квадратных скобках не превышает $2t-2$. Но $x^{2t-1}-1$ должен делить стоящий в квадратных скобках многочлен. Следовательно, или

$$b_1(x) - x^k b_2(x) = 0,$$

или

$$x^{t-1}\,b_1(x) - x^k b_2(x) = 0.$$

По определению пакета ошибок оба коэффициента b_{10} и b_{20} отличны от нуля. Следовательно, $k = 0$ или соответственно $k = t-1$. В любом случае $j = \nu(2t-1)$ и $b_1(x) = b_2(x) = b(x)$.

Остается показать, что $b(x) = 0$. Но исходное соотношение

$$b_1(x) - x^j b_2(x) = a(x)\,(x^{2t-1}-1)\,p(x)$$

теперь приводится к виду

$$-(x^{\nu(2t-1)}-1)\,b(x) = a(x)\,(x^{2t-1}-1)\,p(x).$$

При $\nu \neq 0$ многочлен $p(x)$ не может делить $x^{\nu(2t-1)}-1$, так как ν меньше e и $p(x)$ не делит $x^{\nu(2t-1)}-1$ ни для одного целого положительного ν, меньшего e. Поэтому в левой части равенства многочлен $p(x)$ может делить только $b(x)$. Но степень многочлена $b(x)$ меньше степени многочлена $p(x)$; следовательно, при $\nu \neq 0$ многочлен $b(x)$ равен нулю. Так как, далее, $j = \nu(2t-1)$, то $\nu = 0$ означает, что $j = 0$, а это совместно с уже доказан-

ным равенством $b_1(x) = b_2(x)$ противоречит выбору двух разных пакетов.

Таким образом, два различных пакета $b_1(x)$ и $x^i b_2(x)$ длины t и менее всегда принадлежат разным смежным классам, и, следовательно, код способен исправлять пакеты длины t и менее. □

В качестве примера кода Файра выберем $m = t = 10$ и положим $p(x)$ равным примитивному многочлену степени 10. Тогда $e = 2^m - 1$, и мы получаем (19437, 19408)-код, исправляющий все пакеты ошибок длины 10 и менее.

Коды Файра являются высокоскоростными кодами с малой (при $m = t$) избыточностью $n - k$. В этих случаях избыточность равна $3t - 1$, что превышает границу Рейгера только на $t - 1$. Метод перемежения позволяет строить из кодов Файра более длинные коды, исправляющие более длинные пакеты ошибок. Эти коды являются лучшими известными высокоскоростными кодами, исправляющими пакеты ошибок. В следующей главе мы увидим, что для этих кодов известны очень простые способы построения декодеров.

5.8. ДВОИЧНЫЙ КОД ГОЛЕЯ

Любой, кто займется исследованием таблицы биномиальных коэффициентов, может заметить, что

$$\left[\binom{23}{0} + \binom{23}{1} + \binom{23}{2} + \binom{23}{3}\right] 2^{12} = 2^{23}.$$

Это равенство представляет собой необходимое (но не достаточное) условие существования совершенного исправляющего тройные ошибки (23, 12)-кода над $GF(2)$, так как: 1) число точек внутри сферы декодирования задается записанным в квадратных скобках выражением, 2) всего имеется 2^{12} таких сфер и 3) все пространство содержит 2^{23} точек. Следовательно, можно ожидать существования (23, 12, 7)-кода. Такой код, названный кодом Голея, действительно существует. Он удовлетворяет границе Хэмминга из задачи 1.5 со знаком равенства.

Код Голея занимает уникальное и важное место в теории кодирования. В силу такой хорошей упаковки сфер не удивительно, что код Голея тесно связан со многими математическими объектами, и поэтому его можно использовать для перехода от изучения теории кодирования к глубоким задачам теории групп и других областей математики. Однако встав на практическую точку зрения, вероятно, справедливо было бы заметить, что из-за малой длины код Голея, по-видимому, не найдет дальнейших конкретных приложений.

Определим код Голея как двоичный циклический код через его порождающий многочлен.

Пусть $g(x)$ и $\tilde{g}(x)$ — следующие взаимные многочлены:

$$g(x) = x^{11} + x^{10} + x^6 + x^5 + x^4 + x^2 + 1,$$

$$\tilde{g}(x) = x^{11} + x^9 + x^7 + x^6 + x^5 + x + 1.$$

Простым вычислением проверяется, что

$$(x - 1)\, g(x)\, \tilde{g}(x) = x^{23} - 1,$$

так что в качестве порождающего многочлена циклического (23, 12)-кода можно использовать как $g(x)$, так и $\tilde{g}(x)$. Мы воспользуемся многочленом $g(x)$. Проведенное в начале параграфа комбинаторное рассуждение показывает, что минимальное расстояние этого кода не может быть больше 7. Мы сейчас докажем, что оно равно по меньшей мере 7, однако это доказательство не будет одновременно столь элементарным и кратким, как предыдущее. Можно было бы выписать матрицу **H** и проверить, что никакие шесть ее столбцов не являются линейно зависимыми, но из-за больших размеров матрицы это неудобно. Поэтому мы проведем доказательство непосредственно, хотя оно и окажется длинным (длина этого доказательства показывает, насколько трудным может быть вычисление минимального веса в коде).

Доказательство будет состоять из трех шагов, которые совместно покажут, что минимальный вес не меньше 7. Этими тремя шагами являются доказательства следующих утверждений:

1) в коде нет слов веса 4 или меньше;
2) в коде нет слов веса 2, 6, 10, 14, 18 и 22;
3) в коде нет слов веса 1, 5, 9, 13, 17 и 21.

Совместно эти три утверждения означают, что минимальный вес кода равен по меньшей мере 7. Так как мы уже видели, что он не может быть больше 7, то заключаем, что он в точности равен 7.

Начнем с рассмотрения корней многочленов $g(x)$ и $\tilde{g}(x)$ в соответствующем расширении поля $GF(2)$. Сделаем это не впрямую, а построив минимальные многочлены некоторых элементов из $GF(2048)$, которые затем окажутся равными $g(x)$ и $\tilde{g}(x)$. Если α — примитивный элемент поля, то в силу разложения $2047 = 23 \times 89$ и элемент поля $\beta = \alpha^{89}$, и обратный ему элемент β^{-1} имеют порядок 23. Пусть $f(x)$ и $\bar{f}(x)$ — минимальные многочлены элементов β и β^{-1} соответственно.

Согласно теореме 5.3.6, минимальный многочлен элемента β равен

$$f(x) = (x - \beta)(x - \beta^2)(x - \beta^4) \ldots (x - \beta^{2^{r-1}}),$$

где все показатели степеней приводятся по модулю 23 и $2 \cdot 2^{r-1} = 1 \bmod 23$). Сопряженными являются элементы множества

$$B = \{\beta, \beta^2, \beta^4, \beta^8, \beta^{16}, \beta^9, \beta^{18}, \beta^{13}, \beta^3, \beta^6, \beta^{12}\},$$

которых всего 11, так что степень многочлена $f(x)$ равна 11. Аналогично минимальный многочлен элемента β^{-1} равен

$$\tilde{f}(x) = (x - \beta^{-1})(x - \beta^{-2})(x - \beta^{-4}) \ldots (x - \beta^{-2^{r-1}}),$$

а множество сопряженных элементов равно

$$\tilde{B} = \{\beta^{-1}, \beta^{-2}, \beta^{-4}, \beta^{-8}, \beta^{-16}, \beta^{-9}, \beta^{-18}, \beta^{-13}, \beta^{-3}, \beta^{-6}, \beta^{-12}\}.$$

Вместе множества B и $\tilde{B}$ содержат 22 элемента поля, порядок каждого из которых равен 23; следовательно,

$$(x - 1) f(x) \tilde{f}(x) = x^{23} - 1,$$

причем, согласно теореме о единственности разложения, это разложение единственно. Но мы видели, что такое же разложение имеет место для порождающих многочленов $g(x)$ и $\tilde{g}(x)$ кода Голея. Следовательно, эти порождающие многочлены представляют собой минимальные многочлены элементов α^{89} и α^{-89} из поля $GF(2048)$.

Лемма 5.8.1. *Код Голея не содержит ненулевых кодовых слов веса 4 и менее.*

Доказательство. Так как элементы β, β^2, β^3 и β^4 принадлежат множеству B, то они являются корнями каждого кодового многочлена. Следовательно, каждое кодовое слово удовлетворяет равенству $\mathbf{c}\mathbf{H}^{\mathrm{T}} = 0$, где

$$\mathbf{H} = \begin{bmatrix} 1 & \beta & \beta^2 & \ldots & \beta^{22} \\ 1 & \beta^2 & \beta^4 & \ldots & \beta^{21} \\ 1 & \beta^3 & \beta^6 & \ldots & \beta^{20} \\ 1 & \beta^4 & \beta^8 & \ldots & \beta^{19} \end{bmatrix},$$

и, таким образом, $\mathbf{c}$ является кодовым словом кода с проверочной матрицей $\mathbf{H}$. Но любые четыре столбца матрицы $\mathbf{H}$ образуют ненулевую матрицу, кратную матрице Вандермонда, определитель которой, как известно, отличен от нуля, если все элементы ее первой строки различны (это будет доказано в § 7.2). Следовательно, согласно следствию 3.2.3, $\mathbf{H}$ является проверочной матрицей кода, минимальный вес в котором равен по меньшей мере 5. □

Следующая лемма утверждает, что код Голея не содержит слов веса 2, 6, 10, 14, 18 и 22.

Лемма 5.8.2. *Если вес слова Голея четен, то он кратен 4.*

Доказательство. Пусть $c(x) = a(x) g(x)$ — кодовое слово, вес которого четен:

$$c(x) = \sum_{i=0}^{22} c_i x^i,$$

где число коэффициентов c_i, равных единице, четно. Тогда $c(1) = 0$, и поэтому $x - 1$ делит $c(x)$. Но $x - 1$ не делит $g(x)$; следовательно,

$$c(x) = b(x)(x - 1)\, g(x)$$

для некоторого $b(x)$. Определим взаимный многочлен:

$$\tilde{c}(x) = \sum_{i=0}^{22} c_{22-i} x^i = x^{22} \sum_{i=0}^{22} c_i x^{-i}.$$

Поскольку $\tilde{c}(\beta^{-1}) = 0$, для некоторого $\tilde{a}(x)$ имеем

$$\tilde{c}(x) = \tilde{a}(x)\, \tilde{g}(x).$$

Таким образом,

$$c(x)\, \tilde{c}(x) = b(x)\, \tilde{a}(x)(x - 1)\, g(x)\, \tilde{g}(x) = b(x)\, \tilde{a}(x)(x^{23} - 1).$$

Ниже мы используем это равенство, а сейчас вычислим $c(x)\, \tilde{c}(x)$ отметив, что произведение многочленов может быть записано в виде свертки:

$$c(x)\, \tilde{c}(x) = \sum_{j=0}^{44} \sum_{i=0}^{22} c_i \tilde{c}_{j-i} x^j = \sum_{j=0}^{44} \sum_{i=0}^{22} c_i c_{22+i-j} x^j.$$

Если считать, что коэффициенты с индексами, меньшими нуля и бóльшими 22, равны нулю, то сумму по i можно заменить суммой от $-\infty$ до ∞ (это позволит избежать дальнейших хлопот с пределами суммирования). Сумму можно разбить на части следующим образом:

$$c(x)\, \tilde{c}(x) = \sum_{j=0}^{21} \sum_{i=-\infty}^{\infty} c_i c_{22+i-j} x^j + \sum_{i=-\infty}^{\infty} c_i^2 x^{22} +$$

$$+ \sum_{j=23}^{41} \sum_{i=-\infty}^{\infty} c_i c_{22+i-j} x^j.$$

Второе слагаемое равно нулю, так как $c_i^2 = c_i$ в поле $GF(2)$ и по предположению число коэффициентов c_i, равных единице, четно. В последнем слагаемом сделаем подстановку $i' = i + 22 - j$ и в обоих слагаемых заменим j на j'. Тогда

$$c(x)\, \tilde{c}(x) = \sum_{j'=0}^{21} \sum_{i=-\infty}^{\infty} c_i c_{22+i-j'} x^{j'} +$$

$$+ \sum_{j'=23}^{44} \sum_{i'=-\infty}^{\infty} c_{i'+j'-22} c_{i'} x^{j'}.$$

Далее заменим i' на i во втором слагаемом и положим $j' = j - 1$ в первом слагаемом и $j' = j + 22$ во втором. Это дает

$$c(x)\tilde{c}(x) = \sum_{j=1}^{22}\sum_{i=-\infty}^{\infty} c_i c_{23+i-j}x^{j-1} + \sum_{j=1}^{22}\sum_{i=-\infty}^{\infty} c_i c_{i+j}x^{j-1}x^{23} =$$

$$= \sum_{j=1}^{22}\left[\sum_{i=0}^{22} c_i c_{23+i-j} + c_i c_{i+j}\right]x^{j-1} +$$

$$+\left[\sum_{j=1}^{22}\sum_{i=-\infty}^{\infty} c_i c_{i+j}x^{j-1}\right](x^{23}-1).$$

Но $c(x)\tilde{c}(x)$ кратно многочлену $x^{23} - 1$, и следовательно,

$$\sum_{i=0}^{22} c_i (c_{23+i-j} + c_{i+j}) = 0, \quad j = 1, \ldots, 22.$$

Это равенство должно выполняться при сложении по модулю 2. Следовательно, при целочисленном сложении при каждом j в левой части должно получаться четное число:

$$\sum_{i=0}^{22} c_i (c_{23+i-j} + c_{i+j}) = 2a_j, \quad j = 1, \ldots, 22$$

для некоторых чисел a_j. Левая часть не меняется при замене j на $23 - j$. Следовательно, $a_j = a_{23-j}$. Таким образом, суммирование по j дает

$$\sum_{j=0}^{22}\sum_{i=0}^{22} c_i (c_{23+i-j} + c_{i+j}) = 2\sum_{j=1}^{22} a_j = 4\sum_{j=1}^{11} a_j = 4a$$

для некоторого a. Наконец, левую часть можно переупорядочить. Положим $j' = 23 - j$ в первом слагаемом и $i = i' - j$ во втором; тогда

$$\sum_{j'=1}^{22}\sum_{i=0}^{22} c_i c_{i+j'} + \sum_{j=1}^{22}\sum_{i'=0}^{22} c_{i'}c_{i'-j} = 4a,$$

что вырождается в

$$\sum_{i=0}^{22}\sum_{j\neq i} c_i c_j = 4a.$$

Так как двоичные компоненты c_i отличны от нуля в w позициях, отсюда следует, что

$$w(w-1) = 4a.$$

В силу четности w это означает, что w кратно 4, что и доказывает лемму. □

Теперь можно перейти к доказательству следующей теоремы.

Теорема 5.8.3. *Код Голея является совершенным кодом, исправляющим три ошибки.*

Доказательство. Как уже было указано в начале этого параграфа, необходимо только доказать, что минимальное расстояние равно по меньшей мере 7. В силу леммы 5.8.1 оно равно по меньшей мере 5, а в силу леммы 5.8.2 оно не может быть равно 6. Таким образом, необходимо только доказать, что код не содержит слов веса 5.

Рассмотрим кодовое слово $g(x)\,\tilde{g}(x)$; в действительности $g(x)\,\tilde{g}(x) = \sum_{k=0}^{n-1} x^k$, так как $(x-1)\,g(x)\,\tilde{g}(x) = x^n - 1 = (x-1)\sum_{k=0}^{n-1} x^k$. Таким образом, коду принадлежит слово, все компоненты которого равны 1. Прибавление этого кодового слова к любому кодовому слову веса w дает кодовое слово веса $23 - w$. Тогда, согласно лемме 5.8.2, в коде не содержится слов с весами, равными 21, 17, 13, 9, 5 и 1; в частности, не содержится слов веса 5. Теорема доказана. □

Отметим, что из доказательства теоремы следует также, что все веса слов в коде Голея исчерпываются числами 0, 7, 8, 11, 12, 15, 16 и 23. Просчитанное на ЭВМ число слов каждого веса приведено в табл. 5.3.

Кроме двоичного кода Голея существует также совершенный троичный (11, 6, 5)-код Голея. Этими двумя кодами исчерпываются все нетривиальные примеры совершенных кодов, исправляющих более одной ошибки.

Таблица 5.3

Веса слов кода Голея

Вес	Число слов	
	(23, 12)-код	Расширенный (24, 12)-код
0	1	1
7	253	0
8	506	759
11	1288	0
12	1288	2576
15	506	0
16	253	759
23	1	0
24	—	1
	4096	4096

5.9. КВАДРАТИЧНО-ВЫЧЕТНЫЕ КОДЫ

Класс квадратично-вычетных кодов представляет собой частный подкласс циклических кодов, широко изучаемых потому, что в этом подклассе было найдено несколько хороших кодов и поэтому имеется надежда найти в нем другие хорошие коды. На рис. 5.2 приведены параметры некоторых квадратично-вычетных кодов с известным минимальным расстоянием. Наиболее известным кодом класса является код Голея. Большинство кодов из этого списка обладают бóльшим минимальным расстоянием, чем любой другой известный код с теми же параметрами n и k, что и привлекает внимание исследователей к квадратично-вычетным кодам. Однако не все эти коды хороши, и не известно, существуют ли хорошие квадратично-вычетные коды большой длины. Поэтому неясно, насколько квадратично-вычетные коды полезны в практических приложениях.

Мы рассмотрим только двоичные квадратично-вычетные коды. Название этих кодов отражает их связь с теми элементами простого поля, которые имеют в этом поле квадратный корень. Как показывает теорема 4.6.15, в полях характеристики 2 каждый элемент имеет единственный квадратный корень. В простом поле $GF(p)$, $p \neq 2$, квадратный корень имеет точно половина ненулевых элементов, а именно те $(p-1)/2$ элементов, которые равны четным степеням примитивного элемента. Эти элементы принято называть *квадратичными вычетами* (так как они равны квадратам своих квадратных корней по модулю p). Подчеркнем, что,

n	k	d^*
7	4	3[1]
17	9	5[1]
23	12	7[1]
31	16	7[1]
41	21	9[1]
47	24	11[1]
71	36	11
73	37	13
79	40	15[1]
89	45	17[1]
97	49	15
103	52	19[1]
127	64	19
151	76	19

Рис 5.2. Параметры некоторых двоичных квадратично-вычетных кодов. Знаком сноски [1]) отмечены параметры лучших из известных кодов с данными n и k.

хотя мы рассматриваем коды над $GF(2)$, квадратичные вычеты, входящие в их определение, лежат в поле $GF(p)$, $p \neq 2$. Не следует также путать поле локаторов $GF(2^m)$ с полем $GF(p)$, которое не является подполем поля локаторов.

Определение 5.9.1. *Квадратично-вычетным кодом* называется циклический код над $GF(2)$, длина которого равна простому числу p, делящему $2^m - 1$ при некотором m, а корнями порождающего многочлена являются все элементы α^j из $GF(2^m)$, такие, что j является квадратичным вычетом поля $GF(p)$.

Как следует из этого определения,

$$g(x) = \prod_{j \in QR} (x - \alpha^j),$$

где произведение вычисляется по множеству QR всех квадратичных вычетов по модулю p, причем предполагается, что $g(x)$ представляет собой многочлен над $GF(2)$. Если такой многочлен не является многочленом над $GF(2)$, то для этого значения p кода не существует.

Из определения многочлена $g(x)$ следует также, что

$$x^p - 1 = (x - 1)\, g(x)\, \tilde{g}(x),$$

где многочлен $\tilde{g}(x)$ определяется тем, что все $(p-1)/2$ его корней суть элементы вида α^j из $GF(2^m)$ для всех ненулевых квадратичных невычетов j из $GF(p)$. Многочлен $\tilde{g}(x)$ является взаимным к $g(x)$ и порождает эквивалентный код.

Многочлен

$$g'(x) = (x - 1) \prod_{j \in QR} (x - \alpha^j)$$

порождает другой код. У этого многочлена корней на один больше, чем у многочлена $g(x)$, и, следовательно, порождаемый им код является подкодом квадратично-вычетного кода. Этот подкод тоже называется квадратично-вычетным, но он не так интересен, как исходный код, и в дальнейшем мы не будем его рассматривать.

Квадратично-вычетный код над полем $GF(2)$ существует только тогда, когда все коэффициенты многочлена $g(x)$ лежат в этом поле. Вскоре мы увидим, что квадратично-вычетный код существует только тогда, когда p является простым числом вида $p = 8k \pm 1$. Но для описания одного такого кода надо знать его минимальное расстояние. Обычно задача вычисления минимального расстояния квадратично-вычетного кода является достаточно сложной, и каждый из кодов требует индивидуального подхода. Один из примеров мы встретили в предыдущем параграфе при исследовании (23, 12)-кода Голея. Для всех перечисленных на рис. 5.2 квадратично-вычетных кодов минимальные расстояния

известны. Имеется много других квадратично-вычетных кодов, но их минимальные расстояния не известны.

Мы укажем одну границу для минимального расстояния произвольного квадратично-вычетного кода. Граница не очень хороша, но легко доказывается.

Теорема 5.9.2 (граница квадратного корня). *Минимальное расстояние произвольного квадратично-вычетного кода длины p удовлетворяет неравенству $d^* \geqslant \sqrt{p}$.*

Доказательство. Зафиксируем произвольный ненулевой элемент s поля $GF(p)$, не являющийся квадратичным вычетом. Каждый элемент поля $GF(p)$ можно записать в виде js для некоторого j (так как $GF(p)$ — поле). Если число j является квадратичным вычетом, то, очевидно, js будет квадратичным невычетом, если же j — квадратичный невычет, то js является квадратичным вычетом.

Пусть $c(x)$ представляет собой кодовый многочлен минимального веса d^*. Тогда $c(x) = a(x)\,g(x)$ для некоторого $a(x)$. Определим $\tilde{c}(x) = c(x^s) \pmod{x^p - 1}$. При этом $\tilde{c}(x) = a(x^s) \times g(x^s) \pmod{x^p - 1}$, и вес многочлена $\tilde{c}(x)$ равен самое большее d^*. Но если j — квадратичный вычет, то $g(\alpha^{js}) \neq 0$, а если j — квадратичный невычет, то $g(\alpha^{js}) = 0$. Таким образом, $g(x^s) = \tilde{g}(x)$. Следовательно, $c(x)\,\tilde{c}(x) \pmod{x^p - 1}$ делится как на $g(x)$, так и на $\tilde{g}(x)$. Этот многочлен кратен многочлену $x^{p-1} + x^{p-2} + \ldots + x + 1$, а степень его равна самое большее $p - 1$, и поэтому

$$c(x)\,\tilde{c}(x) = x^{p-1} + x^{p-2} + \cdots + x + 1 \pmod{x^p - 1}.$$

В правой части стоит многочлен веса p, а так как вес многочлена $c(x)$ равен d^*, то возможный вес многочлена в левой части не превосходит $(d^*)^2$. Итак, $d^* \geqslant \sqrt{p}$, что и требовалось доказать. □

Теперь определим, когда коэффициенты определенного выше многочлена $g(x)$ принимают только двоичные значения. Если 2 является квадратичным вычетом, то все степени двойки также являются квадратичными вычетами. Далее, если j — другой квадратичный вычет, то все числа $2j$, $4j$, $8j$, ... также являются квадратичными вычетами. Следовательно, вместе с α^j все его сопряженные относительно поля $GF(2)$ элементы являются корнями порождающего многочлена. В этом случае многочлены $g(x)$ и $\tilde{g}(x)$ представляют собой многочлены над $GF(2)$. Таким образом, квадратично-вычетные коды существуют для всех таких p, для которых 2 является квадратичным вычетом в поле $GF(p)$. Мы увидим, что числа p, для которых 2 является квадратичным вычетом в поле $GF(p)$, исчерпываются простыми числами вида $p = 8k \pm 1$ для произвольного целого k. Это утверждение яв-

ляется достаточно трудной теоремой теории чисел, и чтобы облегчить ее доказательство, мы введем несколько лемм.

Лемма 5.9.3. *Если p — простое число вида $p = 8k \pm 1$, то в поле $GF(p)$ имеет место равенство $2^{(p-1)/2} = 1$. Если p — простое число вида $p = 8k \pm 3$, то в поле $GF(p)$ имеет место равенство $2^{(p-1)/2} = -1$.*

Доказательство. Прежде всего заметим, что в поле $GF(p)$ для любого a справедливо равенство $2a = -(p - 2a)$, и, таким образом,

$$\prod_{a=A_0}^{A_1} 2a = \prod_{a=A_0}^{A_1} [-(p-2a)] = (-1)^{A_1-(A_0-1)} \prod_{a=A_0}^{A_1} (p-2a).$$

Пусть x обозначает наибольшее целое число, меньшее или равное x. Тогда имеем следующую цепочку равенств:

$$2^{(p-1)/2} \prod_{a=1}^{(p-1)/2} a = \prod_{a=1}^{(p-1)/2} (2a) =$$

$$= \prod_{a=1}^{\lfloor (p-1)/4 \rfloor} (2a) \prod_{a=\lfloor (p-1)/4 \rfloor + 1}^{(p-1)/2} (2a) =$$

$$= \left[\prod_{a=1}^{\lfloor (p-1)/4 \rfloor} (2a) \right] (-1)^{[(p-1)/2 - \lfloor (p-1)/4 \rfloor]} \left[\prod_{a=\lfloor (p-1)/4 \rfloor + 1}^{(p-1)/2} (p-2a) \right].$$

Теперь в первом произведении $2a$ пробегает по всем четным целым числам от 2 до $(p-1)/2$, а во втором произведении $p - 2a$ пробегает по всем нечетным целым числам от 1 до $(p-1)/2 - 1$. Это легко проверить, отдельно рассматривая случаи четных и нечетных $(p-1)/2$. Объединяя оба произведения, имеем

$$2^{(p-1)/2} \prod_{a=1}^{(p-1)/2} a = (-1)^{[(p-1)/2 - \lfloor (p-1)/4 \rfloor]} \prod_{a=1}^{(p-1)/2} a.$$

Далее, так как в поле $GF(p)$ все члены произведения отличны от нуля, то можно провести сокращение, что дает

$$2^{(p-1)/2} = (-1)^{[(p-1)/2 - \lfloor (p-1)/4 \rfloor]}.$$

Если $p = 8k \pm 1$, то показатель степени у (-1) равен четному числу и

$$2^{(p-1)/2} = 1.$$

Если $p = 8k \pm 3$, то показатель степени у (-1) равен нечетному числу и

$$2^{(p-1)/2} = -1,$$

что и завершает доказательство леммы. □

Лемма 5.9.4. *В поле GF (p) число r является квадратичным вычетом тогда и только тогда, когда* $r^{(p-1)/2} = 1$.

Доказательство. Предположим, что $r^{(p-1)/2} \neq 1$. Тогда $\sqrt{r}$ не может существовать, так как если он существует, то $(\sqrt{r})^{p-1}$ должно равняться единице, что противоречит предположению.

Предположим теперь, что $r^{(p-1)/2} = 1$, и обозначим через α примитивный элемент поля $GF(p)$. Все четные степени α, очевидно, являются квадратичными вычетами, и так как половина ненулевых элементов поля является квадратичными невычетами, то отсюда вытекает, что все нечетные степени элемента α образуют квадратичные невычеты. Нам надо только показать, что r равно четной степени элемента α. Допустим противное; тогда $r = \alpha^{2i+1}$ и

$$r^{(p-1)/2} = (\alpha^{2i+1})^{(p-1)/2} = \alpha^{i\,(p-1)}\alpha^{(p-1)/2} =$$

$$= \alpha^{(p-1)/2} \neq 1,$$

так как порядок α равен $p - 1$. Итак, из равенства $r^{(p-1)/2} = 1$ вытекает, что r равно четной степени элемента α и, следовательно, является квадратичным вычетом. □

Теорема 5.9.5. *В простом поле GF (p) элемент 2 является квадратичным вычетом, если* $p = 8k \pm 1$ *для некоторого целого k, и квадратичным невычетом, если* $p = 8k \pm 3$.

Доказательство непосредственно вытекает из леммы 5.9.3 и 5.9.4. □

ЗАДАЧИ

5.1. Многочлен $g(x) = x^8 + x^7 + x^6 + x^4 + 1$ порождает циклический код над $GF(2)$ длины 15.

а. Найти проверочную матрицу кода.

б. Сколько ошибок может исправлять этот код?

в. Сколько стираний может исправлять этот код?

г. Найти порождающую матрицу кода в систематическом виде.

5.2. Найти минимальный многочлен для каждого элемента поля $GF(16)$.

5.3. Найти порождающий многочлен двоичного (31, 21)-кода, исправляющего две ошибки.

5.4. Найти порождающий многочлен двоичного циклического (21, 12)-кода, исправляющего две ошибки.

5.5 Многочлен $g(x) = x^6 + 3x^5 + x^4 + x^3 + 2x^2 + 2x + 1$ порождает над $GF(4)$ циклический код длины 15, исправляющий две ошибки.

а. Найти порождающую матрицу кода в систематическом виде.

б. Показать, что каждое кодовое слово кода из задачи 5.1 принадлежит рассматриваемому коду.

5.6. Пусть $g(x)$ — порождающий многочлен циклического кода над $GF(q)$ длины n. Доказать, что если n и q взаимно просты, то слово, все компоненты которого равны 1, принадлежит коду тогда и только тогда, когда 1 не является корнем порождающего многочлена.

5.7. Предположим, что многочлен $g(x) = g_{n-k}x^{n-k} + \ldots + g_0$ порождает циклический код. Доказать, что g_0 не равно нулю. Доказать, что взаимный многочлен $\tilde{g}(x) = g_0x^{n-k} + g_1x^{n-k-1} + \ldots + g_{n-k}$ порождает эквивалентный циклический код.

5.8. Предположим, что двоичный циклический код обладает тем свойством, что если $c(x)$ — кодовый многочлен, то и $\bar{c}(x)$ — кодовый многочлен. Доказать, что $g(x) = \tilde{g}(x)$. Как формулируется соответствующее утверждение для недвоичного кода?

5.9. Найти порождающий многочлен (9, 7)-кода Хэмминга над GF (8).

5.10. Расширить табл. 5.2, включив в нее все коды Файра, основанные на примитивных многочленах $p(x)$ для $12 \geqslant m \geqslant t$.

5.11. Предположим, что $g_1(x)$ и $g_2(x)$ порождают над $GF(q)$ два кода $\mathscr{C}_1$ и $\mathscr{C}_2$ с одинаковой длиной. Доказать, что если все корни многочлена $g_1(x)$ являются корнями многочлена $g_2(x)$ (так что $g_1(x)$ делит $g_2(x)$), то $\mathscr{C}_2$ является подкодом кода $\mathscr{C}_1$.

5.12. Многочлен $g(x) = x^6 + 3x^5 + x^4 + x^3 + 2x^2 + 2x + 1$ над GF (4) порождает циклический (15, 9)-код над GF (4), исправляющий две ошибки.

а. Является ли многочлен $v(x) = x^{10} + 3x^2 + x + 2$ кодовым словом?

б. Чему равен синдромный многочлен для многочлена $v(x)$?

в. Сколько синдромных многочленов нужно затабулировать, чтобы охватить все исправляемые конфигурации ошибок?

5.13. Разложить многочлен $x^8 - 1$ над GF (3). Сколько имеется циклических кодов над GF (3) длины 8?

5.14. Доказать, что все элементы, сопряженные с примитивным элементом, также являются примитивными.

5.15. Построить над GF (256) порождающий многочлен для кода Файра длины $n = 4845$, исправляющего все пакеты не более чем из 10 ошибок.

ЗАМЕЧАНИЯ

Центральная идея, объединяющая всю эту главу, была выдвинута в глубоких работах Прейнджа [1957, 1958]. Прейндж ввел понятие циклического кода и указал на его связь с идеалами алгебр, которые независимо изучали он, Питерсон [1960] и Касами [1960]. Эта работа появилась в конце 50-х годов и заложила фундамент переворота, произошедшего в 60-х годах, когда стало понятно, что циклические коды можно описывать в расширениях полей, и это привело к идеям, которые будут изложены в гл. 7. Большинство материала данной главы использует метод расширения полей.

Исследование кодов, исправляющих пакеты ошибок, было начато Эйбрамсоном [1959]. Большинство из практически используемых кодов было найдено с помощью поиска на ЭВМ, причем многие из них нашел Касами [1963]. Первая таблица из § 5.7 основана на компиляции из книги Питерсона и Уэлдона [1971]. Коды Файра были опубликованы Файром в 1959 г.

Двоичный (23,12,7)-код Голея и троичный (11,6,5)-код Голея были опубликованы в 1949 г. (Голей [1949]). Наше доказательство теоремы о минимальном расстоянии двоичного кода Голея восходит к Мак-Элису [1977]. Циклическую структуру кодов Хэмминга независимо установили Эйбрамсон [1960] и Элспас [1960]. Квадратично-вычетные коды были введены Прейнджем [1958] и широко изучались впоследствии; резюме посвященных им работ дали Лесмус и Мэттсон [1974].

ГЛАВА 6

СХЕМНАЯ РЕАЛИЗАЦИЯ ЦИКЛИЧЕСКОГО КОДИРОВАНИЯ

В этой главе мы временно прерываем поиски циклических кодов с большим минимальным расстоянием. К ним мы вернемся в следующей главе, а сейчас займемся теми конструкциями кодеров и декодеров, которые, как правило, содержат регистры сдвига.

Цифровые логические устройства легко организовать в виде цепей регистров сдвига, имитирующих циклические сдвиги и полиноминальную арифметику, используемые в описании циклических кодов. Поэтому структура циклических кодов тесно связана со структурой цепей регистров сдвига. В частности, эти цепи очень удачно подошли для реализации многих процедур кодирования и декодирования, в которых они часто принимают вид фильтров. В самом деле, многие алгоритмы проще описать, используя символику цепей регистров сдвига. Многие детали легче понять на схеме, чем из формулы. В данной главе мы рассмотрим кодеры и декодеры, основанные на регистрах сдвига.

6.1. ЛОГИЧЕСКИЕ ЦЕПИ ДЛЯ АРИФМЕТИКИ КОНЕЧНОГО ПОЛЯ

Арифметику полей Галуа легко реализовать с помощью логических цепей, особенно если q является степенью двойки. Нам нужны элементы цепи для запоминания элементов поля, называемые *разрядами регистра сдвига*, и элементы цепи для выполнения арифметических операций конечного поля. Мы определим эти элементы для произвольного поля $GF(q)$, хотя обычно они строятся из двоичных элементов.

Регистр сдвига, как показано на рис. 6.1, представляет собой последовательность элементов памяти, называемых *разрядами*. Каждый разряд содержит один элемент поля $GF(q)$. Содержащийся в каждом разряде символ, покидая этот разряд, появляется на выходящей из него линии. Каждый разряд снабжен также входной линией, по которой в него поступает элемент

поля $GF(q)$. Если нет других указаний, то этот входной символ принимается равным нулевому элементу поля. В дискретные моменты времени, называемые *тактами*, содержащиеся в устройствах памяти элементы поля замещаются элементами поля с входных линий. Современные электронные регистры сдвига допускают тактовую частоту, превосходящую 10 миллионов тактов в секунду.

Помимо регистров сдвига мы будем использовать три других элемента, показанных на рис. 6.2, а именно умножитель на скаляр, сумматор и умножитель. Умножитель на скаляр является функцией одной входной переменной; он умножает входную переменную на фиксированный элемент поля $GF(q)$. Сумматор и умножитель являются функциями двух входных переменных, принимающих значения из $GF(q)$. В двоичном случае сумматор также называется элементом «исключительного или», а умножитель — элементом «и».

Все перечисленные выше элементы цепей можно для произвольного $GF(q)$ построить из двоичных элементов. Опишем, как это делается в том случае, когда q представляет собой степень двойки. Элементы поля $GF(2^m)$ записываются совокупностями m битов и могут быть реализованы в цепи или параллельно (в один момент времени один бит на каждом из m проводов), или последовательно (в один момент времени один бит на одном проводе).

Оба способа реализации элементов поля $GF(2^m)$ на двоичных регистрах сдвига демонстрируются на рис. 6.3, где показан частный случай поля $GF(16)$. Каждый элемент поля представляется четырьмя битами, а последовательность элементов поля записывается последовательностью 4-битовых чисел. Элементы поля могут быть выполнены последовательно или параллельно, т. е. с использованием одного или четырех проводов. В случае последовательного исполнения регистр сдвига для $GF(16)$ реализуется в виде двоичного регистра, длина которого в четыре раза больше. Для сдвига элемента поля в следующую ячейку на самом деле потребуется четыре такта.

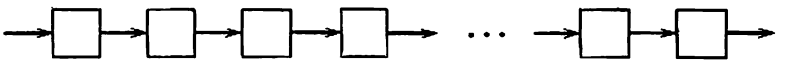


Рис. 6.1. n-разрядный регистр сдвига.

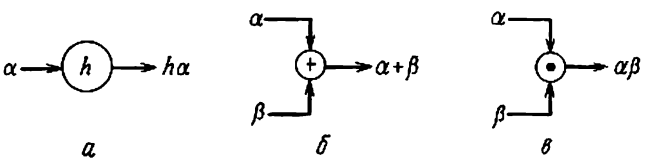


Рис. 6.2. Элементы устройств. *а* — умножитель на скаляр; *б* — сумматор, *в* — умножитель.

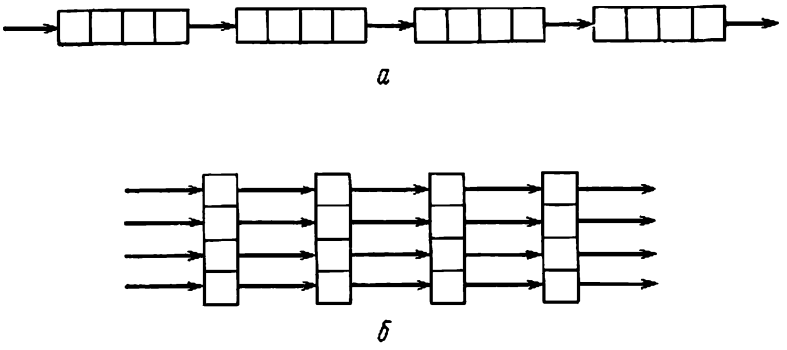


Рис. 6.3. Построенные из двоичных компонентов регистры сдвига для поля из 16 элементов. *а* — последовательное исполнение; *б* — параллельное исполнение.

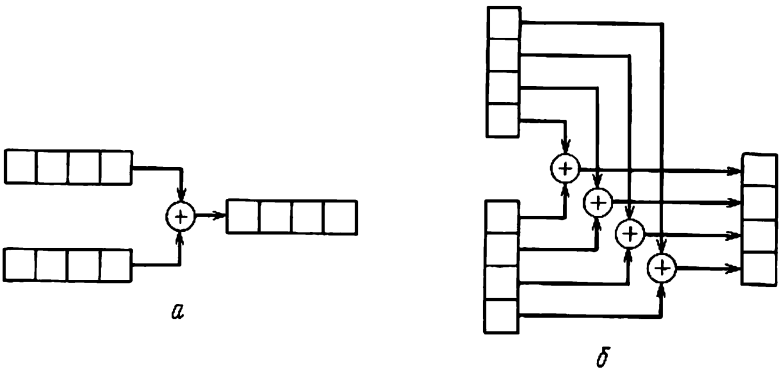


Рис. 6.4. Сложение двух элементов поля. *а* — последовательное исполнение; *б* — параллельное исполнение.

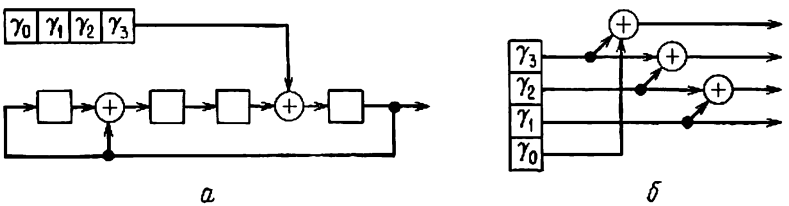


Рис. 6.5. Умножение на константу поля $\beta = z^3$. *а* — последовательное исполнение; *б* — параллельное исполнение.

На рис. 6.4 показаны цепи сложения элементов в поле *GF* (16) как для последовательного, так и для параллельного исполнения. В каждом случае перед выполнением сложения слагаемые вводятся в соответствующие регистры сдвига, а сумма формируется

в третьем регистре сдвига. В схеме последовательного исполнения для выполнения сложения требуются четыре такта, а в схеме параллельного исполнения требуется только один такт, но сумматор содержит больше проводов и сумматоров по модулю 2. При желании слагаемые могут быть возвращены в исходные регистры сдвига для дальнейшего использования в других целях.

На рис. 6.5 изображен умножитель произвольного элемента поля GF (16) на константу $\beta = z^3$ поля GF (16). Для описания этого устройства нам необходимо конкретизировать представление поля GF (16). Предположим, что оно построено с помощью примитивного многочлена $p(z) = z^4 + z + 1$ и что $\gamma = \gamma_3 z^3 + \gamma_2 z^2 + \gamma_1 z + \gamma_0$ — произвольный элемент поля. Тогда

$$\beta\gamma = \gamma_3 z^6 + \gamma_2 z^5 + \gamma_1 z^4 + \gamma_0 z^3 =$$

$$= (\gamma_3 + \gamma_0) z^3 + (\gamma_3 + \gamma_2) z^2 + (\gamma_2 + \gamma_1) z + \gamma_1.$$

Из этого равенства непосредственно вытекает способ параллельного исполнения умножителя на скаляр. Последовательное исполнение этого умножителя менее очевидно. В нем использованы обе строки предыдущего равенства: сначала вычисляется $\gamma_3 z^6 + \gamma_2 z^5 + \gamma_1 z^4 + \gamma_0 z^3$, а затем делается приведение по модулю $z^4 + z + 1$. Эта схема является цепью деления на многочлен $z^4 + z + 1$. Для выполнения умножения на β в ней требуются четыре такта.

6.2. ЦИФРОВЫЕ ФИЛЬТРЫ

Регистры сдвига можно использовать для умножения и деления многочленов над GF (q); этим и объясняется их частое применение в конструкциях кодеров и декодеров. Регистры сдвига полезны и для развития теории, так как играют роль своего рода псевдоматематических обозначений, помогающих лучше понять некоторые действия над многочленами. Цепи регистров сдвига называются также *фильтрами*.

Можно интерпретировать n символов, содержащихся в регистре длины n, как коэффициенты многочлена степени $n - 1$. В качестве обычного соглашения условимся, что регистр всегда сдвигается слева направо. Такое требование приведет иногда к тому, что коэффициенты многочлена будут появляться в регистре сдвига в нисходящем порядке справа налево, а это, к сожалению, противоречит общепринятому правилу записи многочленов.

Для циклического сдвига многочлена используется замкнутый в кольцо регистр сдвига. На рис. 6.6 изображен n-разрядный регистр, используемый для циклического сдвига многочлена степени $n - 1$. Он вычисляет $xv(x) \pmod{x^n - 1}$. Это простейший пример регистра сдвига с линейной обратной связью.

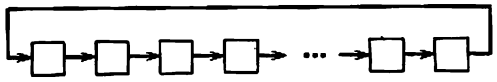
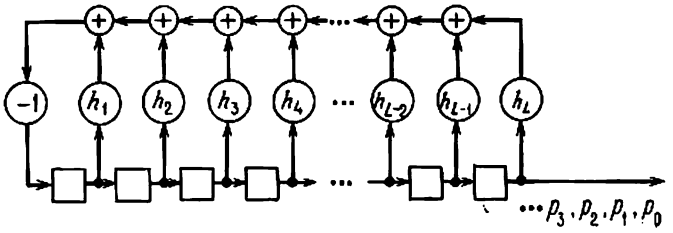


Рис. 6.6. Устройство циклического сдвига многочлена.



$$p_j = -\sum_{i=1}^{L} h_i p_{j-i}$$

Рис. 6.7. Регистр сдвига с линейной обратной связью.

Общий вид регистра сдвига с линейной обратной связью показан на рис. 6.7. Эта цепь реализует вычисление рекурсии

$$p_j = -\sum_{i=1}^{L} h_i p_{j-i}, \quad j \geqslant L.$$

Если в начальный момент регистр загружен L символами (p_0, ..., p_{L-1}), то на выходе регистра появится бесконечная последовательность символов, начинающаяся с p_0 и удовлетворяющая выписанному выше рекуррентному уравнению. Если этот фильтр используется в изображенной на рис. 6.8 цепи, то он называется *авторегрессионным фильтром*. Так как в нем имеется обратная связь, то он относится к обширному классу так называемых *рекуррентных фильтров*.

Вместо того чтобы на вход фильтра подавать по линии обратной связи его выходной сигнал, в качестве входного сигнала можно использовать генерируемую извне последовательность. Такой линейный регистр сдвига без обратной связи показан на рис. 6.9. Он называется также *фильтром с конечным импульсным откликом* (*КИО-фильтром*) или *нерекуррентным фильтром*.

Пусть коэффициенты многочлена $g(x) = g_L x^L + \ldots + g_1 x + g_0$ равны весовым множителям в отводах регистра сдвига без обратной связи, и пусть входная и выходная последовательности записываются соответственно многочленами $a(x) = a_k x^k + \ldots + a_1 x + a_0$ и $b(x) = b_{k+L} x^{k+L} + \ldots + b_1 x + b_0$. Тогда произведение этих многочленов $b(x) = g(x)\, a(x)$ описывает происходящие в изображенном на рис. 6.9 регистре сдвига процессы при условии, что в начальный момент регистр содержал только нули и ввод элемента a_0 сопровождается вводом L нулей. Говорят,

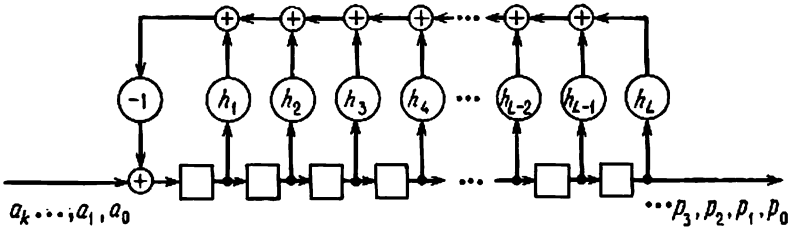
что коэффициенты многочленов $a(x)$ и $g(x)$ свертываются регистром сдвига, так как

$$b_j = \sum_{i=0}^{L} g_i a_{j-i}.$$

Применительно к многочленам КИО-фильтр можно рассматривать как устройство для умножения произвольного многочлена $a(x)$ на фиксированный многочлен $g(x)$. Мы будем называть его также *цепью умножения на* $g(x)$.

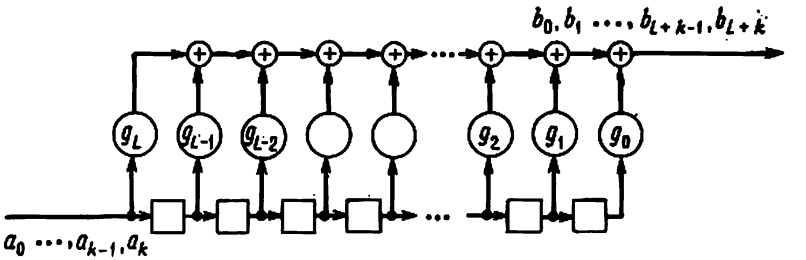
На рис. 6.10, *а* приведен пример цепи умножения на $g(x)$ для $g(x) = x^8 + x^7 + x^4 + x^2 + x + 1$. Эта цепь является КИО-фильтром. Подчеркнем, что внутренние разряды регистра сдвига считываются, но не изменяются. Можно указать другой вариант устройства, в котором внутренние разряды меняются, но, как правило, этот альтернативный вариант оказывается более дорогим. На рис. 6.10, *б* показан такой альтернативный вариант для цепи умножения на $g(x)$. Эта форма КИО-фильтра не является общепринятой.

Регистр сдвига можно также использовать для деления произвольного многочлена на фиксированный многочлен. Выполняющая эту операцию цепь почти повторяет обычную процедуру де-



$$p_j = -\sum_{i=1}^{L} h_i p_{j-i} + a_j$$

Рис. 6.8. Авторегрессионный фильтр.



$$b_j = \sum_{i=0}^{L} g_i a_{j-i}$$

Рис. 6.9. Регистр сдвига без обратной связи.

ления многочленов. Предположим, что делитель является приведенным многочленом. (В противном случае скалярный множитель можно вынести и выполнить соответствующее деление отдельно.) Деление «уголком» записывается в виде

$$
\begin{array}{r|l}
 & a_{n-1}x^{k-1}+(a_{n-2}-a_{n-1}g_{n-k-1})x^{k-2}+\cdots \\
\hline
x^{n-k}+g_{n-k-1}x^{n-k-1}+\cdots+g_0 & a_{n-1}x^{n-1}+a_{n-2}x^{n-2}+\cdots+a_1x+a_0 \\
 & \underline{a_{n-1}x^{n-1}+a_{n-1}g_{n-k-1}x^{n-2}+\cdots} \\
 & \quad (a_{n-2}-a_{n-1}g_{n-k-1})x^{n-2}+\cdots \\
 & \quad \underline{(a_{n-2}-a_{n-1}g_{n-k-1})x^{n-2}+\cdots} \\
 & \quad\quad \cdots
\end{array}
$$

Эти вычисления можно записать в виде системы двух рекуррентных равенств. Пусть $Q^{(r)}(x)$ и $R^{(r)}(x)$ — соответственно частное и остаток на r-м шаге рекурсии с начальными значениями $Q^{(0)}(x) = 0$ и $R^{0}(x) = a(\lambda)$. Тогда рекуррентные равенства записываются в виде

$$Q^{(r)}(x) = Q^{(r-1)}(x) + R^{(r-1)}_{n-r}x^{k-r},$$

$$R^{(r)}(x) = R^{(r-1)}(x) - R^{(r-1)}_{n-r}x^{k-r}g(x),$$

и после k шагов итерации получаются частное $Q^{(k)}(x)$ и остаток $R^{(k)}(x)$.

Изображенная на рис. 6.11 цепь является цепью деления произвольного многочлена на фиксированный многочлен $g(x)$. Это легко понять, обратившись к любому из двух приведенных описаний процедуры деления. Единственной не отраженной на цепи операцией является вычитание члена $R_{n-r}x^{n-r}$ из самого себя, выполнять которую не надо, так как результат всегда равен нулю. После n сдвигов на выходе регистра будет вычислено частное, а в регистре окажется записанным остаток от деления.

На рис. 6.12, *а* приводится пример цепи, реализующей деление произвольного многочлена над $GF(2)$ на многочлен $g(x) = x^8 + x^7 + x^4 + x^2 + x + 1$.

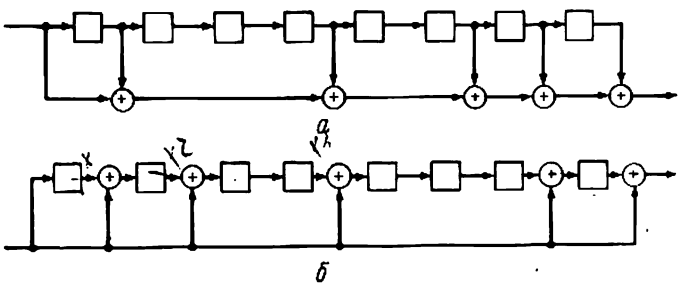


Рис. 6.10. Два устройства умножения на многочлен $x^8 + x^7 + x^4 + x^2 + x + 1$.

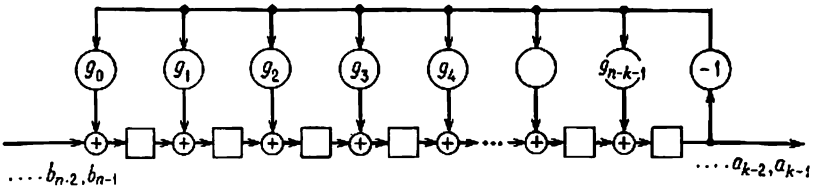


Рис. 6.11. Устройство деления на многочлен $g(x)$.

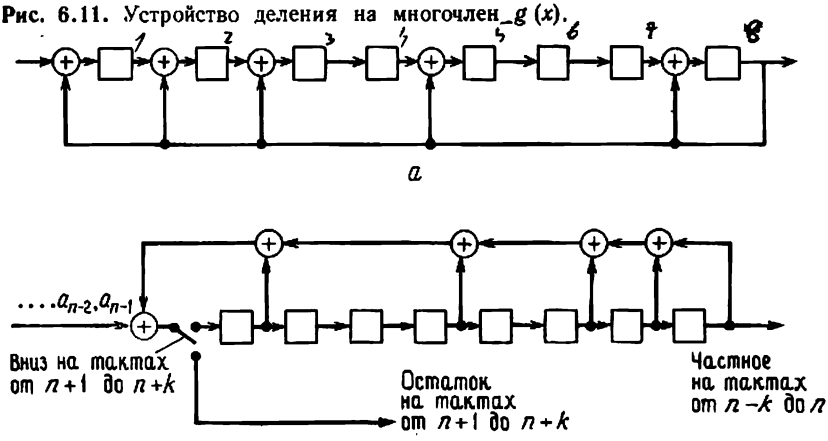


Рис. 6.12. Два устройства деления на многочлен $x^8 + x^7 + x^4 + x^2 + x + 1$.

Отметим, что в этой цепи между внутренними разрядами регистра вставлены сумматоры и это часто усложняет работу цепи. Вместо описанной цепи деления можно использовать цепь, в которой производится только считывание содержимого разрядов регистра сдвига без их изменения. Для построения такой цепи организуем иначе деление многочленов. Идея сводится к одновременному выполнению всех вычитаний одного столбца в описанном выше делении «уголком». Чтобы показать, как это делается, заметим, что можно записать

$$R^{(r)}(x) = a(x) - Q^{(r)}(x)\, g(x),$$

так что

$$R_{n-r}^{(r-1)} = a_{n-r} - \sum_{i=1}^{n-1} g_{n-r-i} Q_i^{(r-1)}$$

и

$$Q^{(r)}(x) = g^{(r-1)}(x) + R_{n-r}^{(r-1)} x^{k-r}.$$

Эти равенства можно реализовать с помощью модификации цепи умножения на $g(x)$. На рис. 6.12, *б* показана соответствующая цепь деления на многочлен $g(x) = x^8 + x^7 + x^4 + x^2 + x + 1$.

6.3. КОДЕРЫ И ДЕКОДЕРЫ НА РЕГИСТРАХ СДВИГА

Целесообразность использования регистров сдвига для построения циклических кодеров и декодеров объясняется структурой циклических кодов. При несистематическом кодировании циклических кодов для формирования кодового слова $c(x)$ надо соответствующий информационный многочлен $i(x)$ умножить на фиксированный порождающий многочлен $g(x)$. Эту операцию можно реализовать на КИО-фильтре над $GF(q)$. Такой кодер для (15, 11)-кода Хэмминга представлен на рис. 6.13. Для кодирования непрерывного потока информационных битов последовательностью слов (15, 11)-кода Хэмминга информационная последовательность просто разбивается на блоки по 11 битов, каждый блок дополняется «прокладкой» из четырех нулей, а результирующий поток битов пропускается через КИО-фильтр. На выходе получается последовательность непересекающихся 15-битовых слов кода Хэмминга. Такой кодер, показанный на рис. 6.14, очень прост, но кодовые слова оказываются несистематическими.

Для получения слов кода в систематическом виде надо воспользоваться другим кодером. Поместим информационные биты в старшие разряды кодового слова и подберем проверочные символы так, чтобы получить допустимое кодовое слово. Кодовое слово записывается в виде

$$c(x) = x^{n-k} i(x) + t(x),$$

где

$$t(x) = -R_{g(x)}\,[x^{n-k} i(x)],$$

так что

$$R_{g(x)}\,[c(x)] = 0.$$

Для реализации систематического кодера используется цепь деления на $g(x)$. Для (15, 11)-кода Хэмминга

$$t(x) = -R_{g(x)}\,[x^4 i(x)];$$

соответствующее устройство показано на рис. 6.15. Одиннадцать информационных битов, занимающих старшие разряды, вводятся слева в цепь деления на $x^4 + x + 1$. Умножение на x^4 учитывается временем работы цепи. Первый бит понимается как коэффициент при x^{15}. Деление не начинается до тех пор, пока не выполнены четыре тактовых сдвига, так что первые четыре бита оказываются записанными в разрядах регистра сдвига. Поэтому ниже цепи деления в устройство включен буфер из четырех разрядов, так, что первые четыре бита начинают поступать в канал одновременно с началом деления. После 11 тактов работы все 11 информационных битов поданы в канал, деление закончено и вы-

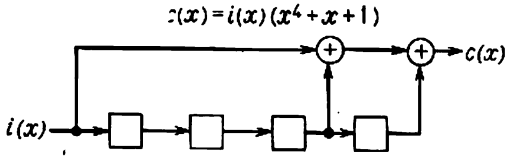


Рис. 6.13. Несистематический кодер для (15, 11)-кода Хэмминга.

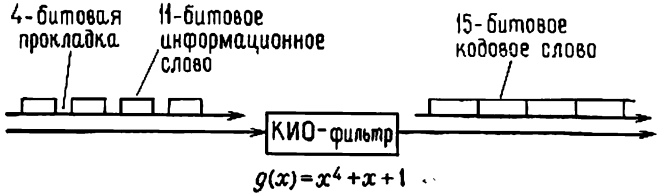


Рис. 6.14. Кодирование длинного потока битов.

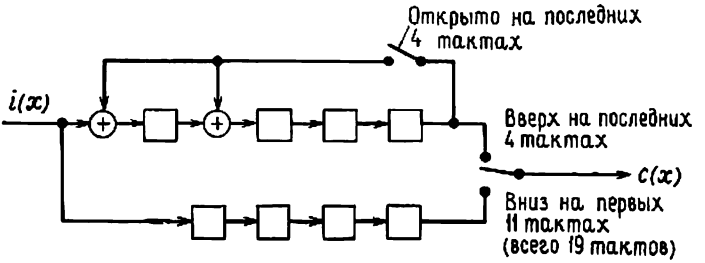


Рис. 6.15. Систематический кодер для (15, 11)-кода Хэмминга.

численный остаток готов для подачи в канал в качестве проверочных символов. В течение этих последних четырех тактов работы цепь обратной связи в устройстве деления разомкнута. В общей сложности полное кодирование занимает 19 тактов.

Можно несколько ускорить кодирование, удалив первые четыре такта. Такой кодер изображен на рис. 6.16. Чтобы понять эту схему, нужно заметить, что поступающие информационные символы не вводятся немедленно для выполнения деления на $g(x)$, а поступают тогда, когда необходимо сформировать сигнал обратной связи. Таким образом, обратная связь в устройстве на рис. 6.16 такова же, как и в устройстве на рис. 6.15. Далее, так как последние биты многочлена $x^4 i(x)$ всегда равны нулю, то ничего не случится, если мы прибавим их к остатку от деления. Таким образом, остаток, вычисляемый устройством на рис. 6.16, равен остатку, вычисляемому устройством на рис. 6.15, но вычисление происходит только за 15 тактов, что, конечно, удобнее.

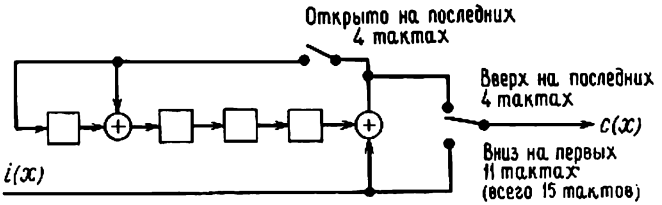


Рис. 6.16. Другой систематический кодер для (15, 11)-кода Хэмминга.

Теперь обратимся к декодеру. В канал поступают коэффициенты многочлена $c(x)$. К ним прибавляется многочлен ошибок $e(x)$. На выходе канала принимается многочлен

$$v(x) \quad c(x) + e(x).$$

В § 5.2 была описана очень простая по идее процедура декодирования, основанная на просмотре таблицы. Принятая последовательность делится на $g(x)$, и остаток от деления полагается равным синдромному многочлену. Синдромный многочлен используется для выбора из таблицы оценки для многочлена ошибки. В двоичном случае синдром можно использовать непосредственно как адрес хранящейся в таблице оценки вектора ошибок $e(x)$.

На рис. 6.17 изображен декодер для несистематического (15, 11)-кода Хэмминга. Для этого кода синдром задается 4 битами, и, следовательно, необходимо постоянное запоминающее устройство (ПЗУ), в котором записаны 15-битовые слова, четыре из которых заняты адресом. Такой подход представляется практичным для длин $n - k$ синдромов вплоть до 12 или 14 битов, но, как

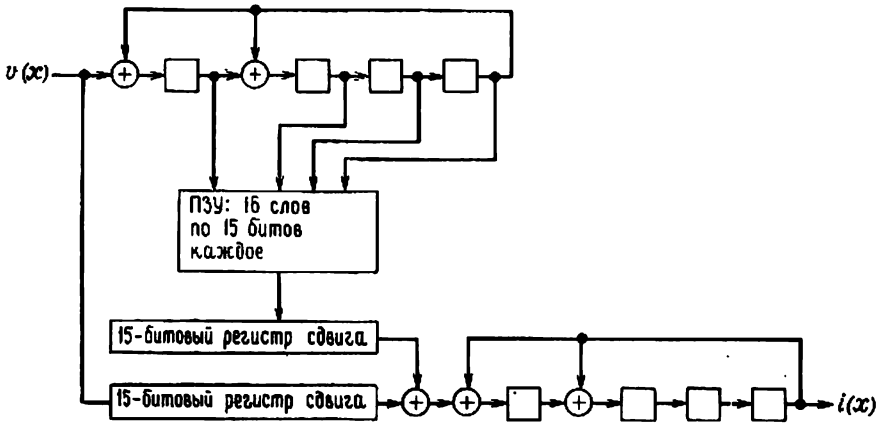


Рис. 6.17. Синдромный декодер для несистематического (15, 11)-кода Хэмминга.

мы увидим в следующем параграфе, возможны и другие технические решения. После исправления принятого слова получаем многочлен $c(x)$, по которому устройство деления на $g(x)$ вычисляет информационные символы согласно правилу

$$i(x) = R_{g(x)}[c(x)],$$

и на этом работа декодера заканчивается.

6.4. ДЕКОДЕР МЕГГИТТА

Наиболее сложной частью описанного в предыдущем параграфе декодера с регистром сдвига является табулирование заранее вычисленных синдромных многочленов и соответствующих им многочленов ошибок. Такой табличный декодер можно значительно упростить, воспользовавшись сильной алгебраической структурой кода для отыскания связей между синдромами. Опираясь на эти связи, можно запомнить только многочлены ошибок, соответствующие некоторым типичным синдромным многочленам. Вычисление остальных необходимых величин осуществляется затем с помощью простых вычислительных алгоритмов.

Простейший декодер такого типа, так называемый *декодер Меггитта*, проверяет синдромы только для тех конфигураций ошибок, которые расположены в старших позициях. Декодирование ошибок в остальных позициях основано на циклической структуре кода и осуществляется позже. Соответственно таблица синдромов содержит только те синдромы, которые соответствуют многочленам ошибок с ненулевым коэффициентом e_{n-1}. Если вычисленный синдром находится в этой таблице, то e_{n-1} исправляется. Затем принятое слово циклически сдвигается и повторяется процесс нахождения возможной ошибки в предшествующей по старшинству позиции ($e_{n-2} \neq 0$). Этот процесс повторяется последовательно для каждой компоненты; каждая компонента проверяется на наличие ошибки, и если ошибка найдена, то она исправляется.

В действительности нет необходимости вычислять синдромы для всех циклических сдвигов принятого слова. Новый синдром можно легко вычислить по уже вычисленному. Основная взаимосвязь описывается следующей теоремой.

Теорема 6.4.1 (Меггитт). *Предположим, что* $g(x)h(x) = x^n - 1$ *и*

$$R_{g(x)}[v(x)] = s(x).$$

Тогда

$$R_{g(x)}[xv(x) \pmod{x^n - 1}] = R_{g(x)}[xs(x)].$$

Доказательство сводится к комбинации трех простых утверждений.

(i) По определению

$$v(x) = g(x) Q_1(x) + s(x),$$

$$xv(x) = xg(x) Q_1(x) + xs(x),$$

(ii) $xv(x) \pmod{x^n - 1} = xv(x) - (x^n - 1) v_{n-1}$.

(iii) Согласно алгоритму деления,

$$xs(x) = g(x) Q_2(x) + t(x)$$

однозначно, причем $\deg t(x) < \deg g(x)$.

Комбинируя эти утверждения, получаем

$$xv(x) \pmod{x^n - 1} = [xQ_1(x) + Q_2(x) + v_{n-1} h(x)] g(x) + t(x) = Q_3(x) g(x) + t(x).$$

Так как $\deg t(x) < \deg g(x)$, то, согласно алгоритму деления, $t(x)$ определяется однозначно и

$$t(x) = R_{g(x)} [xv(x) \pmod{x^n - 1}] = R_{g(x)} [xs(x)],$$

что и требовалось доказать. □

Из этой теоремы, в частности, следует, что многочлены ошибок и соответствующие им синдромные многочлены удовлетворяют равенству

$$R_{g(x)} [xe(x) \pmod{x^n - 1}] = R_{g(x)} [xs(x)].$$

Если $e(x)$ — исправляемая конфигурация ошибок, то многочлен

$$e'(x) = xe(x) \pmod{x^n - 1}$$

получается циклическим сдвигом многочлена $e(x)$. Следовательно, $e'(x)$ также является исправляемой конфигурацией ошибки, и его синдром дается равенством

$$s'(x) = R_{g(x)} [e'(x)] = R_{g(x)} [xs(x)].$$

Это соотношение показывает, как вычислять синдром произвольного циклического сдвига конфигурации ошибки, синдром которой известен. Такое вычисление можно реализовать на простой цепи с регистром сдвига, что, как правило, намного проще, чем просмотр таблицы синдромов.

Предположим, что $s(x)$ и $e(x)$ представляют собой вычисленные синдром и ошибку соответственно и что надо проверить, является ли $\bar{s}(x)$ синдромом для многочлена ошибок $\bar{e}(x)$. Проверим, выполняется ли равенство $s(x) = \bar{s}(x)$; если да, то мы знаем, что $e(x) = \bar{e}(x)$. Если нет, то вычислим $R_{g(x)} [xs(x)]$ и сравним его с $\bar{s}(x)$. Если они совпадают, то

$$\bar{e}(x) = xe(x) \pmod{x^n - 1}$$

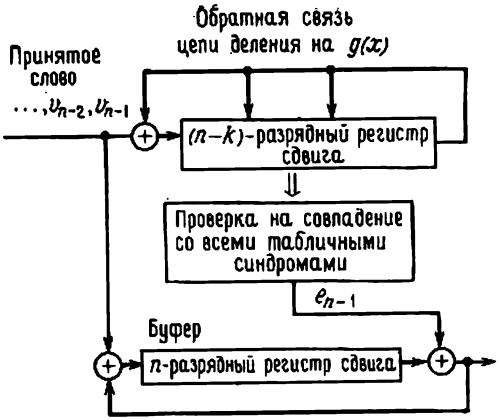


Рис. 6.18. Декодер Меггитта.

представляет собой циклический сдвиг многочлена $e(x)$, так что мы знаем $e(x)$. Продолжая таким образом, вычислим $R_{g(x)}[xR_{g(x)}[xs(x)]]$; в случае совпадения этого многочлена с $\bar{s}(x)$, заключаем, что $\bar{e}(x)$ равен $x^2e(x) \pmod{x^n-1}$, и т. д.

Составим таблицы из таких конфигураций ошибок, чтобы каждая исправляемая конфигурация была циклическим сдвигом одной (или нескольких) из них. Тогда в декодере необходимо запомнить только эту таблицу и таблицу соответствующих синдромных многочленов. Истинный синдром $s(x)$ сравнивается со всеми синдромами, которые содержатся в таблице. Затем вычисляется $R_{g(x)}[xs(x)]$ и тоже сравнивается со всеми содержащимися в таблице синдромами. Повторая этот процесс n раз, находим любую исправляемую кодом ошибку.

Обычно при использовании в декодере цепей с регистром сдвига в качестве табличных конфигураций ошибок выбираются такие, у которых ненулевые коэффициенты расположены в конце регистра сдвига. Это позволяет уменьшить число проводов и использовать регистры, начальные разряды которых не имеют прямого доступа. Такая реализация показана на рис. 6.18.

В качестве табличных выбираются все синдромы, соответствующие исправляемым ошибкам с ненулевыми старшими компонентами. В начале работы декодера содержимое всех разрядов регистра равно нулю. Сначала в декодере срабатывает цепь деления на $g(x)$. В результате такой операции в регистре цепи деления записан синдром $s(x)$, а буферный регистр содержит принятое слово. Вычисленный синдром сравнивается со всеми табличными синдромами, и в случае совпадения хранящийся в старшем разряде буфера символ исправляется. После этого синдромный регистр и буферный регистр один раз циклически сдвигаются. Это

реализует умножение синдрома на x и деление результата на $g(x)$. Содержимое разрядов синдромного регистра теперь равно многочлену $R_{g(x)}[xs(x)]$, представляющему собой синдром ошибки $xe(x) \pmod{x^n - 1}$. Если этот новый синдром совпадает с одним из табличных синдромов, то в предыдущей по старшинству позиции произошла ошибка, которая теперь появится в последнем разряде буфера. Этот бит исправляется, и производится новый циклический сдвиг на одну позицию; синдромный регистр опять готов к проверке ошибки в позиции, отстоящей от старшей на два такта. После повторения этого процесса n раз все биты будут исправлены.

Показанная на рис. 6.18 обратная связь, предназначенная для исправления записанных в буфере символов, не является необходимой; вычисленные символы можно передавать непосредственно пользователю. Мы показали эту связь на схеме для того, чтобы проиллюстрировать идею о том, что только после последнего сдвига в буфере будет записано правильное кодовое слово, а также для того, чтобы иметь отправную точку для некоторых реализаций декодера, которые будут описаны ниже.

Приложения теоремы Меггитта лучше всего пояснить на конкретных примерах. Рассмотрим (15, 11)-код Хэмминга, задаваемый порождающим многочленом $g(x) = x^4 + x + 1$. Так как этот код исправляет только одну ошибку, то имеется только одна конфигурация ошибки с единицей в старшем разряде, а именно $e(x) = x^{14}$, и соответствующий ей синдром равен $s(x) = x^3 + 1$. Декодер Меггитта для этого кода изображен на рис. 6.19. Вычисленный по принятому слову синдром будет содержаться в синдромном регистре после 15 сдвигов. Из них первые 4 сдвига нужны только для того, чтобы ввести в регистр данные, без которых нельзя начать деление. Если синдром равен (1001), то $e(x) = x^{14}$ и соответствующий бит исправляется. Если после одного сдвига синдром равен (1001), то $e(x) = x^{13}$ и соответствующий бит исправляется. Таким же способом проверяется необходимость исправления каждого из битов. После 15 таких сдвигов процесс исправ-

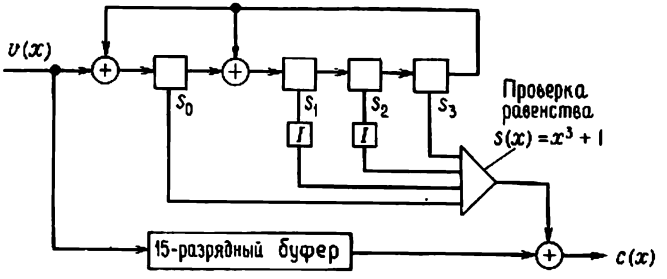


Рис. 6.19. Декодер Меггитта для (15, 11)-кода Хэмминга.

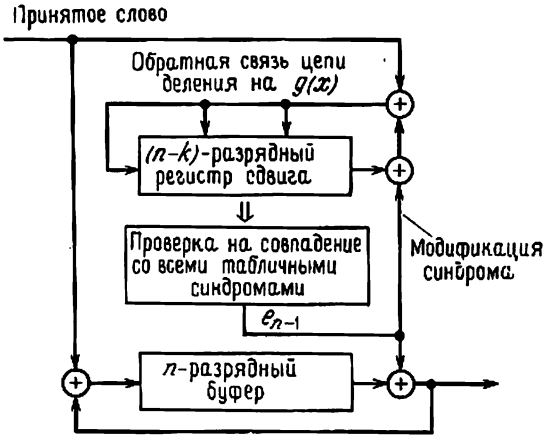


Рис. 6.20. Другой декодер Меггитта.

ления закончен. Следовательно, полный процесс работы декодера занимает 30 тактов.

Обычно декодер Меггитта используют в несколько иной форме, показанной на рис. 6.20. Этот вариант декодера отличается от предыдущих наличием линии, помеченной словами «модификация синдрома». Хотя, как следует из предыдущего описания, эта линия не является необходимой для работы декодера, она необходима для тех реализаций декодера, которые будут описаны ниже. Сейчас операцию модификации синдрома следует рассматривать как своего рода попытку сделать декодер более изящным без существенного его усложнения.

Чтобы понять использование модификации синдрома, предположим, что при наличии ошибки мы хотим ее исправить и полностью ликвидировать ее влияние так, как будто ее не было вообще. Согласно теореме Меггитта, необходимо позаботиться только о случае, когда ошибка произошла в старшем символе. После этого остальные компоненты будут учтены автоматически.

Пусть $e'(x) = e_{n-1}x^{n-1}$. Тогда вклад в синдром от этой единственной компоненты вектора ошибок равен

$$s'(x) = R_{g(x)}[e_{n-1}x^{n-1}].$$

После исправления e_{n-1} этот вклад надо вычесть из истинного синдрома, заменив содержимое $s(x)$ синдромного регистра на $s(x) - s'(x)$. Но многочлен $s'(x)$ может иметь много ненулевых коэффициентов, так что такое вычитание затрагивает многие ячейки синдромного регистра и создает дополнительные неудоб-

ства. Чтобы избежать этих неудобств, изменим определение синдрома, задав его равенством

$$s(x) = R_{g(x)}[x^{n-k}v(x)].$$

Это определение отличается от первоначального, но не хуже его, и с ним можно работать точно так же, как и с первоначальным. Его преимущество состоит в том, что

$$s'(x) = R_{g(x)}[x^{n-k}e_{n-1}x^{n-1}] = R_{g(x)}[R_{x^n-1}[e_{n-1}x^{2n-k-1}]] =$$
$$= R_{g(x)}[e_{n-1}x^{n-k-1}] = e_{n-1}x^{n-k-1},$$

так как степень многочлена $g(x)$ равна $n-k$. Но теперь отличен от нуля только один коэффициент многочлена $s'(x)$, а именно старший. Соответственно после исправления e_{n-1} для замены $s(x)$ на $s(x) - s'(x)$ достаточно только вычесть e_{n-1} из старшего коэффициента синдрома. При этом полностью ликвидируется вклад, вносимый в синдром исправляемым символом.

Предварительное умножение $e(x)$ на x^{n-k} выполняется сдвигом $v(x)$ в новую позицию синдромного регистра.

На рис. 6.21 изображен модифицированный декодер Меггитта для (15, 11)-кода Хэмминга, декодер которого уже рассматривался выше (рис. 6.19). Принятый многочлен $v(x)$ умножается на x^4 и делится на $g(x)$, так что синдром равен

$$s(x) = R_{g(x)}[x^4e(x)] = R_{g(x)}[R_{x^n-1}[x^4e(x)]].$$

Преимущество такой модификации состоит в том, что соответствующий ошибке $e(x) = x^{14}$ синдром равен

$$s(x) = R_{x^4+x+1}[x^{18}] = R_{x^4+x+1}[x^3] = x^3,$$

а это дает возможность исправления с помощью сигнала обратной связи.

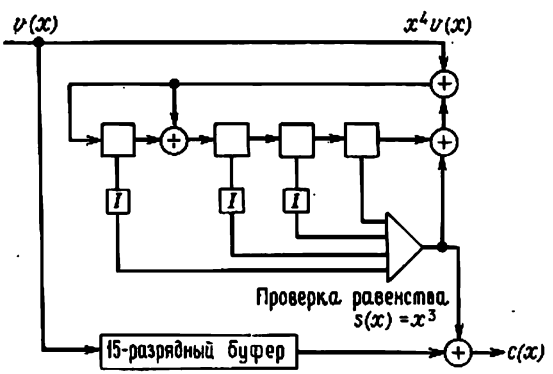


Рис. 6.21. Другой декодер Меггитта для (15, 11)-кода Хэмминга.

Следующим примером является декодер для (15, 7)-кода БЧХ, исправляющего две ошибки (этот код будет определен в гл. 7). Сейчас мы только укажем без доказательства, что этот код цикличен, порождается многочленом $g(x) = x^8 + x^7 + x^6 + x^4 + 1$ и позволяет исправлять две ошибки. Среди них содержатся 15 конфигураций ошибок, содержащих ненулевой старший разряд: одна одиночная ошибка и 14 двойных.

Принятое слово умножается на x^8 и делится на $g(x)$. Следовательно, если $e(x) = x^{14}$, то

$$s(x) = R_{g(x)}[x^8 \cdot x^{14}] = x^7.$$

Аналогично если $e(x) = x^{14} + x^{13}$, то

$$s(x) = R_{g(x)}[x^8 \cdot (x^{14} + x^{13})] = x^7 + x^6.$$

Продолжая таким образом, можно вычислить все подлежащие запоминанию синдромы. Для нахождения ошибок декодер сравнивает вычисленный по принятому слову синдром со всеми этими 15 синдромами. Такой декодер Меггитта показан на рис. 6.22. Отметим, что для нахождения ошибки в старшем разряде содержимое 8-битового синдромного регистра сравнивается с каждым из 15 табличных 8-битовых синдромов. Такое сравнение производится после каждого из 15 сдвигов, ибо каждая позиция в свою очередь проверяется на наличие ошибки в соответствующем бите. Всего декодирование занимает 30 тактов: 15 для вычисления синдрома и 15 для локализации ошибки.

Можно упростить этот декодер и далее и получить схему, изображенную на рис. 6.23. Заметим, что при циклическом сдвиге принятого слова многие из 15 исправляемых конфигураций ошибок появляются дважды. Например, вектор (000000100000001) после восьми циклических сдвигов переходит в вектор (00000001$\bar{0}$000001), где черта под единицей использвана в качестве маркера сдвигаемой единицы. Каждой из этих ошибок соответствует свой табличный синдром. Если удалить один из этих синдромов, то при соответствующем циклическом сдвиге ошибка все-таки будет найдена. Следовательно, для каждой такой пары достаточно запоминать только один синдром. В данном случае имеется только восемь синдромов, которые нужно запомнить. На рис. 6.22 они выписаны в первом столбце таблицы синдромов. Однако в тот момент, когда первая ошибка достигнет конца буфера, она может быть не исправлена, хотя в течение 15 сдвигов будет исправлена по меньшей мере одна ошибка. Для исправления обеих позиций необходимо произвести два полных цикла сдвигов, так что декодирование займет 45 тактов. Теперь необходимость модернизации синдрома очевидна, так как в противном случае ошибка не будет

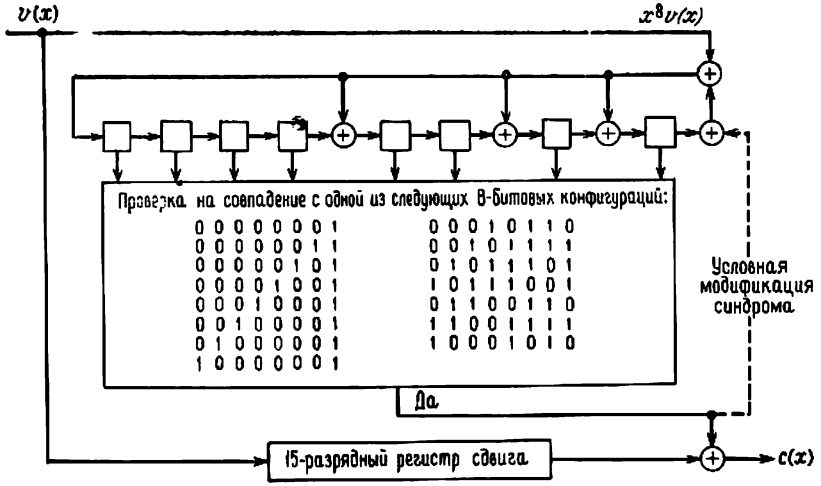


Рис. 6.22. Декодер Меггитта для (15, 7)-кода БЧХ.

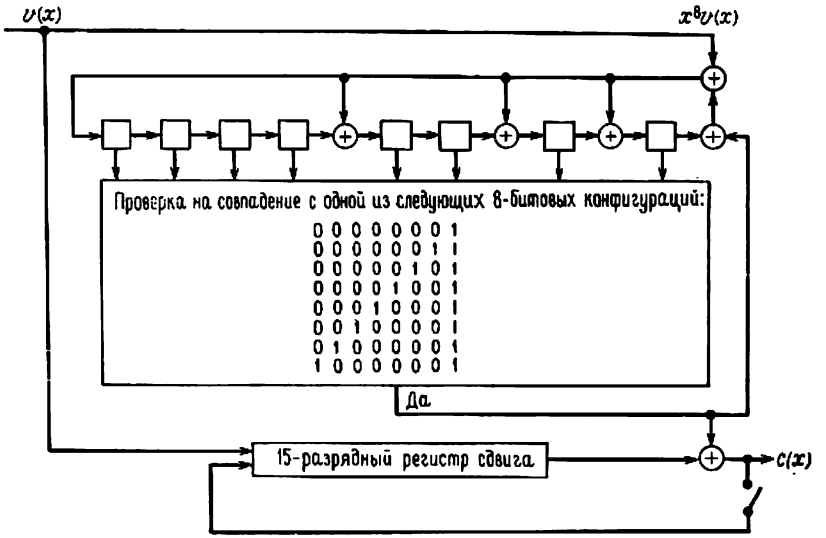


Рис. 6.23. Другой декодер Меггитта для (15, 7)-кода БЧХ.

распознана и при втором ее поступлении в конец буферного регистра.

Показанный на рис. 6.23 декодер для (15, 7)-кода БЧХ почти идентичен декодеру с вылавливанием ошибок, который будет описан в следующем параграфе.

6.5. ВЫЛАВЛИВАНИЕ ОШИБОК

Описанный в конце предыдущего параграфа вариант декодера содержит таблицу синдромов, которая, как сразу видно, содержит только слова веса 1 и 2. На самом деле эти синдромы очень похожи на исправляемые конфигурации ошибок, левая часть которых содержит только нулевые символы и опущена. Такой декодер известен под названием *декодера с вылавливанием ошибок*. Декодер с вылавливанием ошибок является модификацией декодера Мегитта, применимой к некоторым циклическим кодам.

Предположим, что все произошедшие ошибки расположены близко друг от друга. Тогда соответствующим образом сдвинутый синдром будет точной копией ошибки. Определим длину конфигурации ошибок как наименьшее число последовательных разрядов регистра, которые надо просмотреть для того, чтобы при некотором циклическом сдвиге конфигурации ошибок все ошибочные позиции оказались внутри этих разрядов. Предположим, что наибольшая длина исправляемой конфигурации не превосходит длины синдрома. Тогда при некотором циклическом сдвиге конфигурации ошибок синдром будет равен ошибке. Для таких кодов синдром можно циклически сдвигать до тех пор, пока не получится исправляемая конфигурация. Для завершения исправления ошибок надо теперь вычесть содержимое синдрома из соответствующего циклического сдвига принятого вектора.

На рис. 6.24 приведен вылавливающий ошибки декодер для (15, 7)-кода БЧХ. Для полного исправления двойных ошибок требуется 45 тактов работы декодера: 15 для генерирования синдрома, 15 для исправления первой ошибочной позиции и 15 для исправления второй ошибочной позиции. На рис. 6.25 представлен конвейерный вариант декодера, в котором использованы три генератора синдромов. Он позволяет декодировать непрерывно поступающие 15-битовые блоки с тактовой линейной скоростью. На рис. 6.26 расписаны 45 тактов работы декодера с вылавливанием ошибок для типичной конфигурации ошибок (ошибки вылавливаются на тактах номер 28 и 33).

В качестве второго примера декодера с вылавливанием ошибок рассмотрим декодер для (7, 3)-кода Рида—Соломона над $GF(8)$ с порождающим многочленом

$$g(x) = x^4 + (z+1)x^3 + x^2 + zx + (z+1),$$

где элементы поля записаны в виде многочленов от z. В другом виде порождающий многочлен записывается как

$$g(x) = x^4 + \alpha^3 x^3 + x^2 + \alpha x + \alpha^3,$$

где элементы поля заданы в виде степеней примитивного элемента $\alpha = z$. В гл. 7, где рассматриваются коды Рида—Соломона,

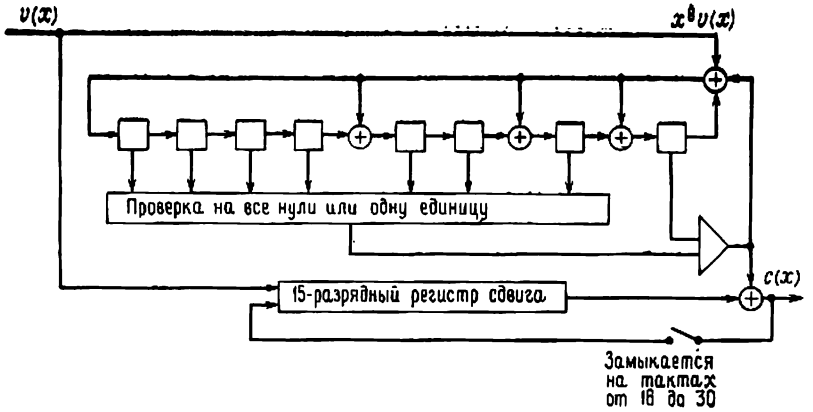


Рис. 6.24. Декодер с вылавливанием ошибок для (15, 7)-кода БЧХ.

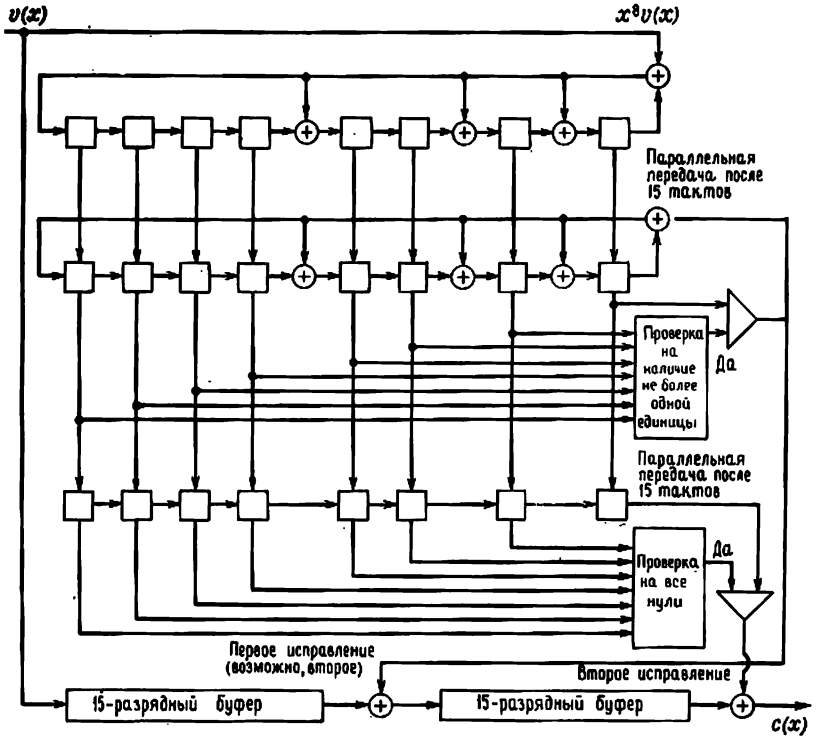
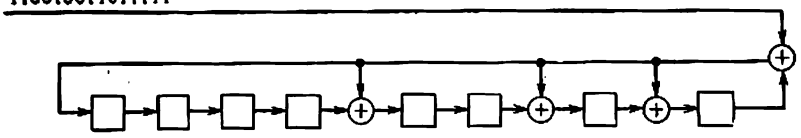


Рис. 6.25. Конвейерный вариант декодера с вылавливанием ошибок для (15, 7)-кода БЧХ.

$$i(x) = x^6 + 1$$
$$g(x) = x^8 + x^7 + x^6 + x^4 + 1$$
$$c(x) = x^{14} + x^{13} + x^{12} + x^{10} + x^8 + x^7 + x^4 + 1$$
$$e(x) = x^{11} + x$$
$$v(x) = x^{14} + x^{13} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^4 + x + 1$$

1100100110111111



Такт	Синдромный регистр	Такт	Средний регистр	Такт	Нижний регистр
0	0 0 0 0 0 0 0 0				
1	1 0 0 0 1 0 1 1	16	0 1 1 0 0 0 1 1	31	0 0 0 0 0 1 0 0
2	0 1 0 0 0 1 0 1	17	1 0 1 1 1 0 1 0	32	0 0 0 0 0 0 1 0
3	0 0 1 0 0 0 1 0	18	0 1 0 1 1 1 0 1	33	0 0 0 0 0 0 0 1
4	1 0 0 1 1 0 1 0	19	1 0 1 0 0 1 0 1	34	0 0 0 0 0 0 0 0
5	1 1 0 0 0 1 1 0	20	1 1 0 1 1 0 0 1	35	·
6	0 1 1 0 0 0 1 1	21	1 1 1 0 0 1 1 1	36	·
7	0 0 1 1 0 0 0 1	22	1 1 1 1 1 0 0 0	37	·
8	0 0 0 1 1 0 0 0	23	0 1 1 1 1 1 0 0	38	Везде только нули
9	0 0 0 0 1 1 0 0	24	0 0 1 1 1 1 1 0	39	·
10	0 0 0 0 0 1 1 0	25	0 0 0 1 1 1 1 1	40	·
11	1 0 0 0 1 0 0 0	26	1 0 0 0 0 1 0 0	41	·
12	0 1 0 0 0 1 0 0	27	0 1 0 0 0 0 1 0	42	·
13	0 0 1 0 0 0 1 0	28	0 0 1 0 0 0 0 1	43	·
14	1 0 0 1 1 0 1 0	29	0 0 0 1 0 0 0 0	44	·
15	1 1 0 0 0 1 1 0	30	0 0 0 0 1 0 0 0	45	·

Рис. 6.26. Пример вылавливания ошибок для (15, 7)-кода БЧХ.

будет показано, что этот код исправляет две ошибки. Сейчас мы примем это утверждение без доказательства.

Исправляемые конфигурации ошибок содержат не более двух ненулевых символов и задаются многочленом степени не выше 6. Их всегда можно циклически сдвинуть в многочлен степени не выше 3. Так как степень синдромного многочлена не превосходит 3, то можно применить декодирование с вылавливанием ошибок.

Такой декодер показан на рис. 6.27. В двоичной реализации восьмеричные разряды регистра сдвига представляют собой параллельные регистры с тремя двоичными разрядами. Все линии пе-

редачи данных в каждый момент времени передают три бита. Элементы умножения на $z+1$ (т. е. на α^3) и на z (т. е. на α) в цепи обратной связи являются просто определяемыми устройствами двоичной логики с тремя входами и тремя выходами. Для исправления всех ошибок такому декодеру необходим всего 21 такт. Первые семь тактов уходят на вычисление синдрома; следующие семь тактов требуются на исправление хотя бы одной ошибки, а иногда и обеих ошибок. Последние семь тактов уходят на исправление второй ошибки (если она еще не исправлена).

На рис. 6.28 отдельные операции алгоритма декодирования отслеживаются в деталях. Это очень ценный пример, так как он демонстрирует технику вылавливания ошибок и одновременно представляет собой конкретный пример взаимосвязи абстрактных понятий теории полей Галуа и практического построения логических цепей, использующих регистры сдвига.

Содержимое синдромного и информационного регистров для каждого сдвига выписаны в таблице. После семи сдвигов синдромный регистр содержит синдром. Начиная с восьмого сдвига декодер исправляет ошибки, если они образуют вылавливаемую кон-

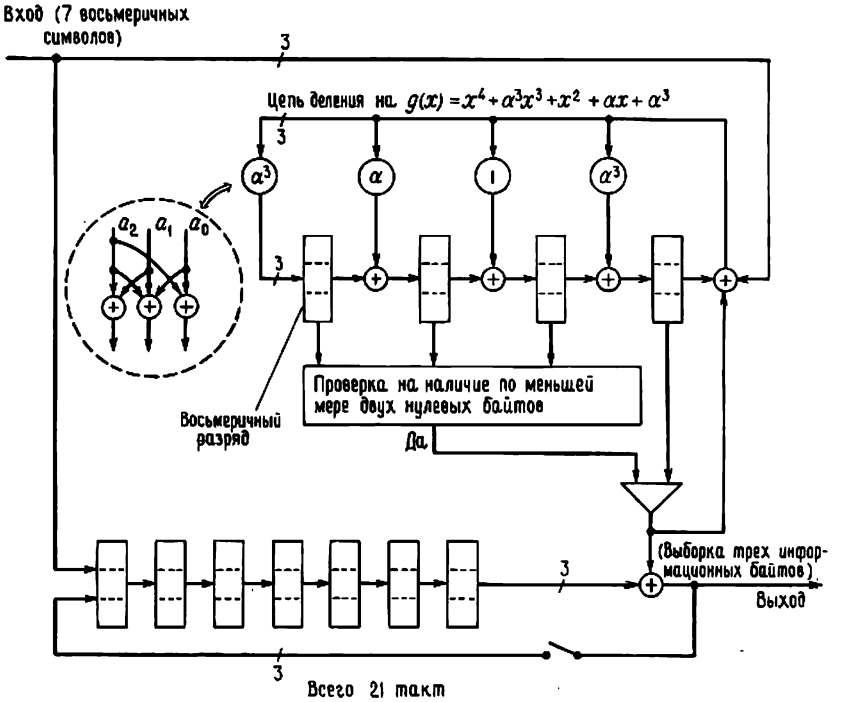


Рис. 6.27. Вылавливание ошибок для (7, 3)-кода Рида—Соломона.

$\alpha^0 = 1$
$\alpha^1 = z$
$\alpha^2 = z^2$
$\alpha^3 = z + 1$
$\alpha^4 = z^2 + z$
$\alpha^5 = z^2 + z +$
$\alpha^6 = z^2 + 1$

$g(x) = x^4 + \alpha^3 x^3 + x^2 + \alpha x + \alpha^3$
$c(x) = \alpha^4 x^6 + \alpha^4 x^4 + \alpha^3 x^3 + \alpha^6 x + \alpha^6$
$e(x) = \alpha^4 x^4 + \alpha^3$
$v(x) = \alpha^4 x^6 + \alpha^3 x^3 + \alpha^6 x + \alpha^4$

Такт	Синдромный регистр		Информационный регистр
0	0 0 0 0		0 0 0 0 0 0 0
1	1 α^5 α^4 1		α^4 0 0 0 0 0 0
2	α^3 α^3 α^4 α^6		0 α^4 0 0 0 0
3	$\alpha^2\alpha$ $\alpha^4\alpha$		0 0 α^4 0 0 0
4	α^3 α^4 α^3 α^6		α^3 0 0 α^4 0 0
5	$\alpha^2\alpha$ $\alpha^3\alpha^5$		0 α^3 0 0 α^4 0
6	α^4 0 0 α^6		α^6 0 α^3 0 0 α^4
7	α^6 0 α^3 α^6	Синдром	$\alpha^4\alpha^6$ 0 α^3 0 0 α^4
8	$\alpha^2\alpha^2\alpha^6\alpha^5$		$\alpha^4\alpha^4\alpha^6$ 0 α^3 0 0
9	α 1 $\alpha^3\alpha^5$		0 $\alpha^4\alpha^4\alpha^6$ 0 α^3 0
10	α $\alpha^5\alpha^4$ 1		0 0 $\alpha^4\alpha^4\alpha^6$ 0 α^3
11	α^3 0 $\alpha^4\alpha^6$		α^3 0 0 $\alpha^4\alpha^4\alpha^6$ 0
12	$\alpha^2\alpha$ $\alpha^6\alpha$		0 α^3 0 0 $\alpha^4\alpha^4\alpha^6$
13	[α^4 0 0 α^3]	← Выловленная ошибка	α^6 0 α^3 0 0 $\alpha^4\alpha^4$
14	0 α^4 0 0		$\alpha^6\alpha^6$ 0 α^3 0 0 α^4
15	0 0 α^4 0		$\alpha^6\alpha^6$ 0 α^3 0 0 | α^4
16	[0 0 0 α^4]	← Выловленная ошибка	$\alpha^5\alpha^6$ 0 α^3 0 | 0 α^4
17	0 0 0 0		$\alpha^6\alpha^6$ 0 α^3 | α^4 0 α^4
18	0 0 0 0		$\alpha^6\alpha^6$ 0 | $\alpha^3\alpha^4$ 0
19	0 0 0 0		$\alpha^6\alpha^6$ | 0 $\alpha^3\alpha^4$
20	0 0 0 0		α^6 | α^6 0 α^3
21	0 0 0 0		| $\alpha^6\alpha^6$ 0

Рис. 6.28. Пример вылавливания ошибок.

фигурацию. К ним принадлежат ошибки, содержащие не более двух ненулевых символов, один из которых расположен в крайней правой позиции. Если произошло не более двух ошибок, то это случится хотя бы однажды в течение следующих семи тактов работы (в примере это случилось на 13-м такте). Следовательно, ошибка выловлена. Подчеркнем, что символы синдрома упорядочены в порядке возрастания слева направо. В момент вылавливания ошибки в синдромном регистре записан 4-символьный сегмент (e_4, e_5, e_6, e_0) сдвинутого многочлена ошибки. Поскольку

ошибка поймана, она исправляется в информационном регистре, а в синдромном регистре e_0 устанавливается равным нулю (или вычитается сам из себя). После 14 тактов синдромный регистр содержит синдром оставшейся конфигурации ошибки. Процесс повторяется в течение последних семи тактов, после чего исправление ошибок закончено полностью.

Если в начале синдром оказался отличным от нуля, а в течение вторых семи тактов работы декодера не была поймана ни одна ошибка, то в принятом слове произошло более двух ошибок (но обратное утверждение неверно). Это позволяет легко включить в декодер дополнительную логическую цепь для обнаружения неисправляемых конфигураций ошибок.

В качестве последнего примера этого параграфа рассмотрим декодер с вылавливанием ошибок для кодов, исправляющих пакеты ошибок. В отличие от ситуации для кодов, исправляющих независимые ошибки, исправляющие пакеты ошибок коды всегда допускают декодирование с вылавливанием ошибок. Мы опишем декодер для (14, 6)-кода, исправляющего пакеты ошибок длины 4. Этот код получается перемежением двух копий (7, 3)-кода из табл. 5.1. Порождающий многочлен кода-перемежения равен

$$g(x) = x^8 + x^6 + x^4 + 1.$$

Вылавливание пакета ошибок длины не более 4 сводится к проверке равенства нулю четырех самых левых символов синдрома. Схему, однако, можно немного улучшить. Заметим, что разряды с четными номерами и разряды с нечетными номерами не взаимодействуют. Поэтому цепь вычисления синдрома можно рассматривать как две независимые цепи вычисления синдромов для каж-

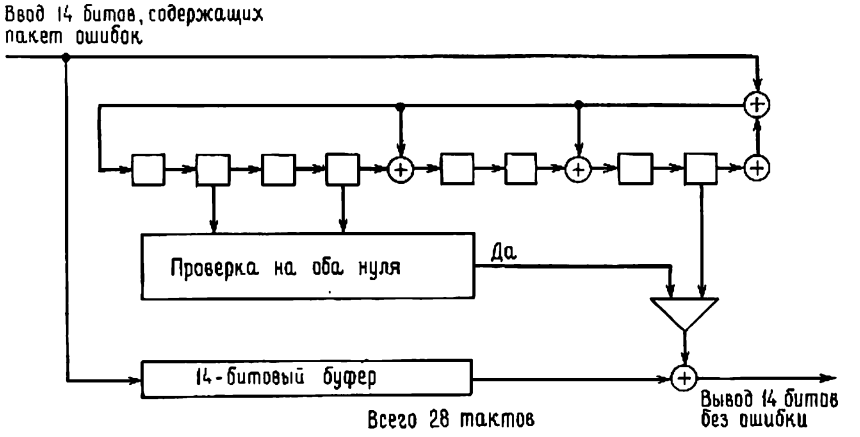


Рис. 6.29. Декодер с вылавливанием ошибок для кода-перемежения, исправляющего пакеты ошибок.

дого из двух перемежаемых кодов, но с последующим перемежением позиций синдромов. Такая модификация декодера с вылавливанием ошибок показана на рис. 6.29. Этот декодер позволяет исправлять все пакеты ошибок длины 4, а также некоторые дополнительные конфигурации ошибок. Исправляется любая конфигурация, состоящая из пакета длины 2 в позициях с нечетными номерами и пакета длины 2 в позициях с четными номерами.

6.6. УКОРОЧЕННЫЕ ЦИКЛИЧЕСКИЕ КОДЫ

Любой систематический циклический код можно укоротить, а именно от (n, k)-кода можно перейти к $(n-b, k-b)$-коду выбрасыванием b информационных позиций в каждом кодовом слове. Будем полагать, что b меньше k и что выбрасываются b старших разрядов. Последнее объясняется тем, что выбрасываемые символы при укорочении полагаются равными нулю и поэтому не передаются, но при декодировании декодер их восстанавливает, так что декодирование осуществляется на полной длине кода. Если минимальное расстояние исходного кода равно d^*, то минимальное расстояние укороченного кода не меньше d^*. Аналогично если исходный код позволяет исправлять пакеты ошибок длины t, то укороченный код позволяет исправлять пакеты длины t (или больше). С помощью укорочения и перемежения из кодов табл. 5.1 можно построить большой набор хороших кодов, исправляющих пакеты ошибок.

Укороченный циклический код уже не является циклическим, так как многочлен $R_{x^{n'-1}}[xc(x)]$ в общем случае уже не является кодовым словом для произвольного кодового слова $c(x)$. Однако укороченный циклический код все-таки обладает алгебраической структурой подмножества соответствующего кольца. В то время как циклический код является идеалом кольца многочленов по модулю $x^n - 1$, укороченный циклический код является идеалом кольца многочленов по модулю некоторого многочлена $f(x)$ степени $n' = n - b$. Точный результат дается следующей теоремой.

Теорема 6.6.1. *Если $\mathscr{C}$ — укороченный циклический код, то существует многочлен $f(x)$, такой, что если $c(x)$ — кодовое слово, а $a(x)$ — произвольный многочлен, то $R_{f(x)}[a(x)\,c(x)]$ также является кодовым словом.*

Доказательство. Пусть $g(x)$ — порождающий многочлен укорачиваемого циклического кода длины n, и пусть n' — длина укороченного циклического кода. Тогда, согласно алгоритму деления,

$$x^{n'} = g(x)\,Q(x) + s(x).$$

Так как $n' = n - b > n - k = \deg g(x)$, то степень остатка $s(x)$ меньше n'. Пусть $f(x) = x^{n'} - s(x)$; тогда степень много-

члена $f(x)$ равна n' и $g(x)$ делит $f(x)$. Если $c(x)$ кратен $g(x)$ и $a(x)$ — произвольный многочлен, то, согласно алгоритму деления, имеем

$$a(x)\,c(x) = f(x)\,Q'(x) + r(x).$$

Так как $g(x)$ делит и $c(x)$, и $f(x)$, то $r(x)$ кратен $g(x)$; таким образом, в кольце $GF(q)[x]/(f(x))$ многочлен $a(x)\,c(x) - r(x)$ кратен многочлену $g(x)$, что и требовалось доказать. □

Укороченный циклический код длины $n' = n - b$ можно декодировать с помощью декодера Меггитта, сконструированного для исходного (n, k)-кода. Отсчет времени в таком декодере основан на группах по n тактов, хотя входное кодовое слово содержит n' символов. Это временно́е рассогласование иногда удается устранить конструкцией устройства, так что оно становится несущественным; в других случаях целесообразнее сбалансировать время. Мы рассмотрим такую модификацию принятого слова, которая позволяет перестроить работу декодера Меггитта на цикл из n' тактов и соответственно ускорить декодирование укороченных циклических кодов.

Переопределим синдром так, чтобы можно было обойти тактовое время работы регистра, соответствующее выбрасываемым символам. Вместо вычисления остатка от деления многочлена $x^{n-k}\,v(x)$ на $g(x)$ определим синдром при b выбрасываемых символах равенством

$$s(x) = R_{g(x)}[x^{n-k+l}v(x)].$$

Чтобы обосновать такой выбор, предположим, что старший символ укороченного циклического кода ошибочен, т. е. что

$$e(x) = e_{n'-1}x^{(n-b)-1}.$$

Тогда

$$s(x) = R_{g(x)}[e_{n'-1}x^{2n-k-1}] =$$
$$= R_{g(x)}[R_{x^n-1}[e_{n'-1}x^{2n-k-1}]] = e_{n'-1}x^{n-k-1}.$$

Следовательно, единственным ненулевым коэффициентом многочлена $s(x)$ является старший коэффициент.

Пусть

$$a(x) = R_{g(x)}[x^{n-k+b}].$$

Тогда, согласно правилам вычислений по модулю, многочлен $s(x)$ можно записать в виде

$$s(x) = R_{g(x)}[a(x)\,v(x)].$$

Все, что нужно сделать, сводится к предварительному (перед делением на $g(x)$) умножению $v(x)$ на фиксированный многочлен $a(x)$. Такое умножение можно реализовать с помощью устрой-

ства деления на $g(x)$ путем выбора точки ввода $v(x)$ в цепь деления. Покажем на примерах, как это делается.

Сначала приведем два примера декодеров для циклических кодов. Первым рассмотрим укороченный циклический код, исправляющий пакеты ошибок. Предположим, что в некотором приложении потребовалось укоротить (511, 499)-код из табл. 5.1 до (272, 260)-кода. Этот код исправляет все пакеты ошибок длины не более 4. В данном случае $g(x) = x^{12} + x^8 + x^5 + x^3 + 1$, $x^{n-k+b} = x^{251}$ и необходимо вычислить

$$a(x) = R_{g(x)}[x^{251}].$$

Одним из способов этого вычисления является представление x^{251} в виде

$$x^{251} = (x^{12})^{16}(x^{12})^4(x^{11})$$

с тем, чтобы воспользоваться равенством $x^{12} = x^8 + x^5 + x^3 + 1$. Повторяя возведение в квадрат величины x^{12} и проводя редукцию по модулю $g(x)$, быстро вычисляем $(x^{12})^4$ и $(x^{12})^{16}$, а, следовательно, и x^{251}, так что

$$a(x) = x^{11} + x^9 + x^7 + x^3 + x^2 + 1,$$

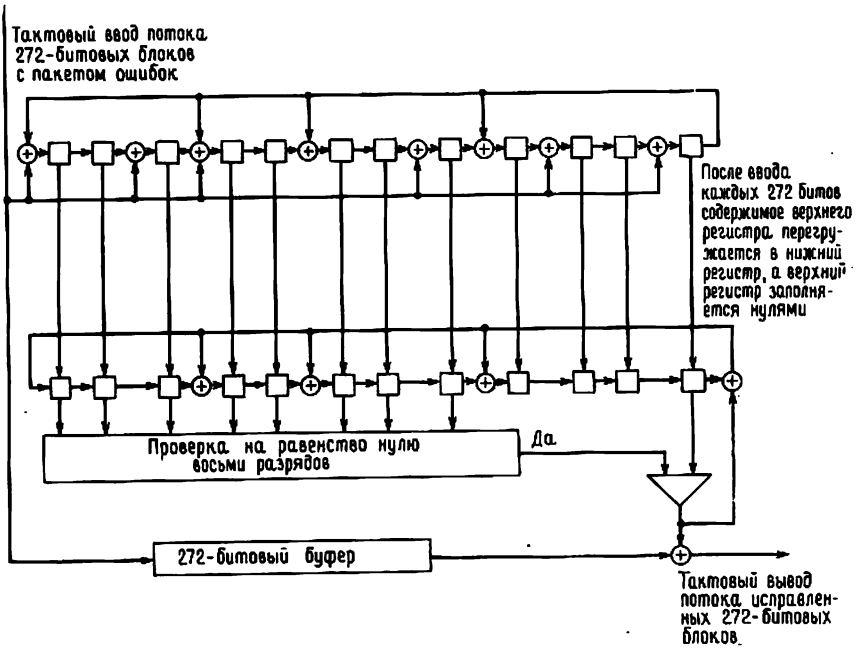


Рис. 6.30. Декодер с вылавливанием ошибок для (272, 260)-кода, исправляющего пакеты ошибок.

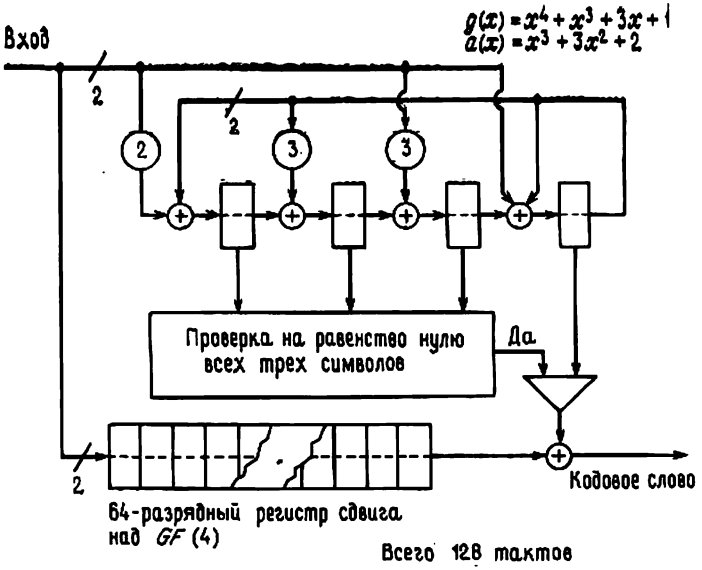


Рис. 6.31. Декодер Меггитта для (64, 60)-кода Хэмминга над *GF* (4).

и, наконец,

$$s(x) = R_{x^{12}+x^9+x^6+x^3+1}[(x^{11} + x^9 + x^7 + x^3 + x^2 + 1)\,v(x)].$$

Вычисление такого синдрома можно реализовать на одной цепи деления на $g(x)$, если при каждой итерации входящий коэффициент многочлена $v(x)$ подавать в соответствующую точку текущего остатка. Декодер Меггитта для рассматриваемого укороченного кода изображен на рис. 6.30. Этот декодер является конвейерным, так что он может обрабатывать непрерывный поток входящих битов.

В качестве второго примера рассмотрим (64, 60)-код над *GF* (4), являющийся укорочением (85, 81)-кода Хэмминга. Как уже обсуждалось в § 5.5, (85, 81)-код Хэмминга является циклическим с порождающим многочленом $g(x) = x^4 + x^3 + 3x + 1$. В данном случае $x^{n-k+b} = x^{25}$. Выполняя деление «уголком», получаем

$$a(x) = R_{x^4+x^3+3x+1}[x^{25}] = x^3 + 3x^2 + 2,$$

и модифицированный синдром задается равенством

$$s(x) = R_{x^4+x^3+3x+1}[(x^3 + 3x^2 + 2)\,v(x)].$$

Показанное на рис. 6.31 устройство является декодером Меггитта для этого укороченного кода, время декодирования для которого равно 128 тактам. Для вычисления модифицированного синдрома

декодер умножает принятое слово на $x^3 + 3x^2 + 2$ и затем делит на $x^4 + x^3 + 3x + 1$. Полное время декодирования, равное 128 тактам, состоит из 64 тактов, необходимых для вычисления модифицированного синдрома, и 64 тактов, необходимых для исправления ошибок. При желании можно сделать декодер конвейерным или последовательно соединить два декодера так, чтобы декодер работал одновременно с вводом данных.

6.7. ДЕКОДЕР МЕГГИТТА ДЛЯ КОДА ГОЛЕЯ

В тех случаях, когда данный код не позволяет непосредственно использовать декодер с вылавливанием ошибок, иногда удается так добавить несколько дополнительных цепей, чтобы аннулировать влияние некоторых мешающих применению декодера конфигураций ошибок. В общем случае для получения удовлетворительного решения требуется некоторая изобретательность. Мы опишем такой декодер для (23, 12)-кода Голея.

Длина вектора ошибок равна 23, а вес не превосходит 3. Длина синдромного регистра равна 11. Если данная конфигурация ошибок не вылавливается, то она не может быть циклически сдвинута так, чтобы все три ошибки появились в 11 младших разрядах. Можно убедиться (возможно, после нескольких прикидок), что в этом случае по одну сторону от одной из трех ошибочных позиций стоит по меньшей мере пять, а по другую сторону — по меньшей мере шесть нулей. Следовательно, каждая исправляемая конфигурация ошибок может быть с помощью циклических сдвигов приведена к одному из трех следующих видов (позиции нумеруются числами от 0 до 22):

1) все ошибки (не более трех) расположены в 11 старших разрядах;

2) одна ошибка занимает пятую позицию, а остальные расположены в 11 старших разрядах;

3) одна ошибка занимает шестую позицию, а остальные расположены в 11 старших разрядах.

Таким образом, в декодере надо заранее вычислить величины $s_5(x) = R_{g(x)}[x^{n-k}x^5]$ и $s_6(x) = R_{g(x)}[x^{n-k}x^6]$. Тогда ошибка вылавливается, если вес $s(x)$ не превышает 3 или если вес $s(x) - s_5(x)$ либо $s(x) - s_6(x)$ не превышает 2. В декодере можно либо исправлять все три ошибки, если эти условия выполнены, либо исправлять только две ошибки в младших 11 битах, дожидаясь удаления из регистра третьего ошибочного бита.

Разделив x^{16} и x^{17} на порождающий многочлен

$$g(x) = x^{11} + x^{10} + x^6 + x^5 + x^4 + x^2 + 1,$$

имеем

$$s_5(x) = x^9 + x^8 + x^6 + x^5 + x^2 + x,$$
$$s_6(x) = x^{10} + x^9 + x^7 + x^6 + x^3 + x^2.$$

Следовательно, если ошибка содержится в пятой или шестой позициях, то синдром соответственно равен (01100110110) или (00110011011). Наличие двух дополнительных ошибок в 11 старших разрядах приводит к тому, что в соответствующих позициях два из этих битов заменяются на противоположные.

Декодер отслеживает синдром, отличающийся от нулевого синдрома не более чем в трех позициях, а также синдром, отличающийся от выписанных двух синдромов не более чем в двух позициях.

ЗАДАЧИ

6.1.а. Построить поле GF (8) с помощью примитивного многочлена $p(x) = x^3 + x + 1$ (α обозначает примитивный элемент $\alpha = x$).

б. Для заданного произвольного многочлена $g(x)$ над GF (2) построить простое логическое устройство, вычисляющее $g(\alpha^3)$.

в. Сконструировать устройство, реализующее умножение двух произвольных элементов поля GF (8).

г. Сконструировать устройство, реализующее умножение произвольного элемента поля GF (8) на скаляр α^4.

д. Указать те элементы поля GF (8), квадраты которых также принадлежат этому полю; затабулировать такие элементы.

е. Сконструировать устройство, вычисляющее квадратный корень из тех элементов поля GF (8), для которых он существует.

6.2. Пусть поле GF (16) задано многочленом $x^4 + x + 1$. Определим два произвольных элемента этого поля:

$$\beta = \beta_3 x^3 + \beta_2 x^2 + \beta_1 x + \beta_0,$$
$$\gamma = \gamma_3 x^3 + \gamma_2 x^2 + \gamma_1 x + \gamma_0,$$

где коэффициенты принадлежат полю GF (2). Построить параллельное логическое устройство, выполняющее умножение двух произвольных элементов из GF (16).

6.3. Исходя из равенств $\beta^{-1} = \left(\beta^{2^{m-1}-1}\right)^2$ и $2^m - 1 = \sum_{k=0}^{m-1} 2^k$, и используя устройство возведения в квадрат и умножитель, сконструировать итеративное логическое устройство вычисления элемента β^{-1} по заданному β за m тактов работы.

6.4. Циклический (15, 9, 5)-код над GF (4) порождается многочленом

$$g(x) = x^6 + 3x^5 + x^4 + x^3 + 2x^2 + 2x + 1.$$

а. Построить кодирующее устройство для этого кода, основанное на регистре сдвига.

б. Построить декодер Меггитта для этого кода.

6.5. Порождаемый многочленом $g(x) = x^4 + x^3 + x^2 + 1$ двоичный циклический код позволяет исправлять пакеты ошибок длины 2.

а. Чему равны длина и скорость этого кода?

б. Найти минимальное расстояние этого кода.

в. Сконструировать систематический кодер для этого кода.

г. Сконструировать декодер с вылавливанием ошибок, позволяющий исправлять все пакеты длины 2.

6.6. Исправляющий три ошибки (23, 12)-код является циклическим с порождающим многочленом

$$g(x) = x^{11} + x^{10} + x^6 + x^5 + x^4 + x^2 + 1$$

или

$$g(x) = x^{11} + x^9 + x^7 + x^6 + x^5 + x + 1.$$

а. Найти проверочный многочлен $h(x)$ этого кода.

б. Сконструировать кодер, использующий регистр сдвига.

в. Можно ли методом вылавливания ошибок исправлять все тройные ошибки? Почему?

6.7. Используя регистры сдвига с 4-битовыми регистрами, сконструировать кодер и декодер для рассмотренного в § 3.4 (17, 15)-кода Хэмминга над полем из 16 элементов. Показать все логические цепи.

6.8. В процессе считывания и записи в запоминающем устройстве ЭВМ, состоящем из 256 8-битовых слов, возможны искажения одного 8-битового слова. Для борьбы с этим явлением решено воспользоваться укороченным кодом Хэмминга над $GF(2^8)$, исправляющим одно искаженное слово.

Описать (256, 252)-код и сконструировать кодер и декодер с вылавливанием ошибки. (Заметим, что задачей схемы является отыскание и исправление в памяти одного искаженного слова.)

6.9. Сконструировать кодер и декодер с вылавливанием ошибок для двоичного (19537, 19408)-кода Файра с порождающим многочленом $g(x) = (x^{19} - 1) \times p(x)$, где $p(x)$ — примитивный многочлен степени 10.

6.10. Задаваемый одним корнем в точке α^{15} поля GF (256) двоичный циклический код длины 17 представляет собой (17, 9, 5)-код, принадлежащий к классу квадратично-вычетных кодов. Найти порождающий многочлен кода и сконструировать модифицированный декодер с вылавливанием ошибок, позволяющий исправлять все двойные ошибки. Сконструировать также декодер, основанный на просмотре таблицы синдромов. Сравнить сложность и скорость этих двух декодеров.

6.11. (7, 5)-код Рида—Соломона над GF (8) порождается многочленом $g(x) = x^2 + \alpha^4 x + \alpha^3$. Построить циклический код, исправляющий пакеты ошибок длины 2 над GF (8), выполняя следующую последовательность шагов.

а. Выписать порождающий многочлен исправляющего восьмеричные пакеты длины 2 кода, получающегося перемежением двух одинаковых исходных кодов Рида—Соломона.

б. Чему равны длина и скорость так построенного кода?

в. Составить для этого кода схему кодера с использованием регистра сдвига.

г. Сконструировать декодер с вылавливанием ошибок, позволяющий исправлять все пакеты длины 2.

6.12. Построить (1072, 1024)-код для исправления пакетов ошибок длины 16 путем укорочения и перемежения подходящего кода из табл. 5.1. Построить (1080, 1024)-код для исправления пакетов длины 20 путем укорочения и перемежения кода Файра.

6.13. Исправляющий две ошибки двоичный (31, 21)-код порождается многочленом

$$g(x) = x^{10} + x^9 + x^8 + x^6 + x^5 + x^3 + 1.$$

а. Показать, что декодер с вылавливанием ошибок не позволяет исправлять всех ошибок в пределах конструктивного расстояния.

б. Показать, что простая модификация декодера с вылавливанием ошибок позволяет организовать декодирование в пределах конструктивного расстояния.

в. Построить другую конструкцию декодера с вылавливанием ошибок, позволяющего исправлять все двойные ошибки, используя вместо циклического сдвига подстановку

$$v(x) \leftarrow v(x^2) \pmod{x^n - 1}.$$

ЗАМЕЧАНИЯ

Многие идеи данной главы уже ясно просматриваются как идеи теории дискретных фильтров, но с заменой поля вещественных или комплексных чисел на поле Галуа. Это смутно ощущалось еще в самом начале и становилось все более очевидным по мере развития предмета. Содержащие регистры сдвига устройства были сразу использованы большинством исследователей и вошли в литературу без всяких фанфар. Использование регистров сдвига в кодерах и декодерах можно найти в работах Питерсона [1960] и Ченя [1964]. В виде учебного материала эти идеи появились уже в книге Питерсона [1961]. Меггитт опубликовал описание устройств своих декодеров в 1960 и 1961 гг. Не совсем ясно, кому принадлежит идея декодера с вылавливанием ошибок, но обычно ее приписывают Прейнджу.

Способы модификации декодера с вылавливанием ошибок, позволяющие корректировать исправляемые, но не вылавливаемые ошибки, рассматривал Касами [1964]. На методах Касами основан описанный в § 6.7 декодер для кода Голея. Использование отличных от циклических перестановок исследовала Мак-Вильямс [1964]. Другие ранние результаты приводятся в статьях Митчелла [1962] и Рудолфа и Митчелла [1964].

ГЛАВА 7

КОДЫ БОУЗА-ЧОУДХУРИ-ХОКВИНГЕМА

Коды Боуза—Чоудхури—Хоквингема (БЧХ) представляют собой обширный класс кодов, способных исправлять несколько ошибок и занимающих заметное место в теории и практике кодирования. Интерес к кодам БЧХ определяется по меньшей мере следующими четырьмя обстоятельствами: 1) среди кодов БЧХ при небольших длинах существуют хорошие (но, как правило, не лучшие из известных) коды; 2) известны относительно простые и конструктивные методы их кодирования и декодирования (хотя если единственным критерием является простота, то предпочтение следует отдать другим кодам); 3) коды Рида—Соломона, являющиеся широко известным подклассом недвоичных кодов, обладают определенными оптимальными свойствами и прозрачной весовой структурой; 4) полное понимание кодов БЧХ, по всей видимости, является наилучшей отправной точкой для изучения многих других классов кодов.

В этой главе мы будем говорить о кодах БЧХ во временно́м представлении. Это будет отражать исторически первый подход к изучению кодов БЧХ. В гл. 8 мы изложим те же самые идеи при помощи частотной интерпретации полей Галуа.

Мы сначала определим исправляющие t ошибок коды БЧХ над $GF(q)$ длины $q^m - 1$, задавая порождающий многочлен $g(x)$. Коды такой длины называют *примитивными кодами БЧХ*. Мы докажем, что эти коды исправляют t ошибок, построив в явном виде алгоритм декодирования. Затем мы обобщим наши рассуждения на случай кодов БЧХ, длина которых является делителем $q^m - 1$.

7.1. ОПРЕДЕЛЕНИЕ КОДОВ БЧХ

Порождающий многочлен циклического кода можно представить в виде

$$g(x) = \text{НОК}\,[f_1(x), f_2(x), \ldots, f_r(x)],$$

где $f_1(x), \ldots, f_r(x)$ — минимальные многочлены корней $g(x)$.

Используя этот подход, мы будем строить коды по порождающему многочлену, который будет задаваться своими корнями.

Пусть $c(x)$ — кодовый многочлен, а $e(x)$ — многочлен ошибок. Принятый многочлен с коэффициентами из $GF(q)$ запишется в виде

$$v(x) = c(x) + e(x).$$

Мы можем вычислить значение этого многочлена на элементах из $GF(q^m)$; в частности, нас будут интересовать значения многочлена в точках, являющихся корнями $g(x)$, скажем в точках $\gamma_1, \ldots, \gamma_r$. Тогда поскольку $c(\gamma_j) = 0$ для любых γ_j, являющихся корнями $g(x)$, то

$$v(\gamma_j) = c(\gamma_j) + e(\gamma_j) = e(\gamma_j).$$

Таким образом,

$$v(\gamma_j) = \sum_{i=0}^{n-1} e_i \gamma_j^i, \quad j = 1, \ldots, r,$$

для всех γ_j, являющихся корнями $g(x)$. В результате мы получаем r уравнений, содержащих только величины, определяемые ошибками и не зависящие от кодового слова. Если эти уравнения можно разрешить относительно e_i, то мы сможем определить многочлен ошибок. Будем выбирать γ_j таким образом, чтобы система r уравнений могла быть решена относительно e_i каждый раз, когда не более t неизвестных отличны от нуля.

Для произвольного циклического кода с порождающим многочленом $g(x)$, имеющим корни $\gamma_1, \ldots, \gamma_r$, определим компоненты синдрома

$$S_j = v(\gamma_j), \quad j = 1, \ldots, r.$$

Эти элементы поля отличны от синдромного многочлена $s(x)$, но содержат эквивалентную информацию. Мы хотим подобрать $\gamma_1, \ldots, \gamma_r$ так, чтобы по $S_1, \ldots, S_r$ можно было найти t ошибок. Вскоре мы докажем, что если α — примитивный элемент поля $GF(q^m)$, то таким множеством является

$$\{\alpha, \alpha^2, \alpha^3, \ldots, \alpha^{2t}\}.$$

Приняв это пока на веру, выберем многочлен $g(x)$ с указанной последовательностью корней. Задав длину $n = q^m - 1$ примитивного кода для некоторого m и число t ошибок, которое необходимо исправить, поступим следующим образом:

1) выберем примитивный многочлен степени m и построим поле $GF(q^m)$;

2) найдем минимальные многочлены $f_j(x)$ для α^j, $j = 1, \ldots, 2t$;

3) положим $g(x) = \text{НОК}\,[f_1(x), \ldots, f_{2t}(x)]$.

В следующем параграфе, подробно описав алгоритм декодирования, мы докажем, что такой циклический код может исправлять t ошибок. Иногда построенные таким образом коды БЧХ могут исправлять более t ошибок. Поэтому величина

$$d = 2t + 1$$

называется *конструктивным расстоянием* кода. Истинное минимальное расстояние кода d^* может быть больше конструктивного.

В табл. 7.1 задано представление поля $GF(16)$ как расширение поля $GF(2)$, построенное по примитивному многочлену $p(z) = z^4 + z + 1$, в нее включены также минимальные многочлены $GF(2)$ для всех элементов из поля $GF(16)$, где $\alpha = z$ — примитивный элемент $GF(16)$. Заметим, что минимальные многочлены для любой четной степени α всегда уже содержатся в одной из предыдущих строк таблицы. Это следствие теоремы 5.3.4, которая утверждает, что элементы β и β^2 для любых β имеют одинаковые минимальные многочлены над полем $GF(2)$. Данный факт немного поможет нам при вычислении $g(x)$.

Порождающий многочлен для исправляющего две ошибки кода БЧХ длины 15 получается следующим образом:

$$g(x) = \text{НОК}\,[f_1(x), f_2(x), f_3(x), f_4(x)] =$$
$$= \text{НОК}\,[x^4 + x + 1,\ x^4 + x + 1,\ x^4 + x^3 + x^2 + x + 1,\ x^4 + x + 1] =$$

Таблица 7.1

Представления поля $GF(2^4)$

В виде степени	В виде многочлена	В двоичном виде	В десятичном виде	Минимальные многочлены
0	0	0000	0	
α^0	1	0001	1	$x+1$
α^1	z	0010	2	x^4+x+1
α^2	z^2	0100	4	x^4+x+1
α^3	z^3	1000	8	$x^4+x^3+x^2+x+1$
α^4	$z+1$	0011	3	x^4+x+1
α^5	z^2+z	0110	6	x^2+x+1
α^6	z^3+z^2	1100	12	$x^4+x^3+x^2+x+1$
α^7	z^3+z+1	1011	11	x^4+x^3+1
α^8	z^2+1	0101	5	x^4+x+1
α^9	z^3+z	1010	10	$x^4+x^3+x^2+x+1$
α^{10}	z^3+z+1	0111	7	x^2+x+1
α^{11}	z^3+z^2+z	1110	14	x^4+x^3+1
α^{12}	z^3+z^2+z+1	1111	15	$x^4+x^3+x^2+x+1$
α^{13}	z^3+z^2+1	1101	13	x^4+x^3+1
α^{14}	z^3+1	1001	9	x^4+x^3+1

$$= (x^4 + x + 1)(x^4 + x^3 + x^2 + x + 1) =$$
$$= x^8 + x^7 + x^6 + x^4 + 1.$$

Поскольку степень $g(x)$ равна 8, $n - k = 8$. Отсюда $k = 7$, и мы получили порождающий многочлен (15, 7)-кода БЧХ, исправляющего 2 ошибки. Отметим, что коды БЧХ строятся по заданным n и t. Значение k не известно, пока не найден $g(x)$.

Тем же способом мы можем построить порождающий многочлен для другого примитивного кода БЧХ длины 15.

Пусть $t = 3$:

$$g(x) = \text{НОК}\,[f_1(x), f_2(x), f_3(x), f_4(x), f_5(x), f_6(x)] =$$
$$= (x^4 + x + 1)(x^4 + x^3 + x^2 + x + 1)(x^2 +$$
$$+ x + 1) = x^{10} + x^8 + x^5 + x^4 + x^2 + x + 1.$$

Получился порождающий многочлен для (15, 5)-кода БЧХ, исправляющего три ошибки.

Пусть $t = 4$:

$$g(x) = \text{НОК}\,[f_1(x), f_2(x), f_3(x), f_4(x), f_5(x), f_6(x),$$
$$f_7(x), f_8(x)] = (x^4 + x + 1)(x^4 + x^3 +$$
$$+ x^2 + x + 1)(x^2 + x + 1)(x^4 + x^3 + 1) =$$
$$= x^{14} + x^{13} + x^{12} + x^{11} + x^{10} + x^9 + x^8 +$$
$$+ x^7 + x^6 + x^5 + x^4 + x^3 + x^2 + x + 1.$$

Получился порождающий многочлен для (15,1)-кода БЧХ. Это простой код с повторением, исправляющий семь ошибок.

Пусть $t = 5, 6, 7$. Каждый из этих случаев приводит к такому же порождающему многочлену, как и при $t = 4$. При $t > 7$ код БЧХ не определен, поскольку ненулевых элементов поля $GF(16)$ всего 15.

В табл. 7.2 приведено представление поля $GF(16)$ как расширение поля $GF(4)$, построенное по примитивному многочлену $p(z) = z^2 + z + 2$. Эта таблица содержит также минимальные многочлены над $GF(4)$ для всех элементов из поля $GF(16)$, где $\alpha = z$ — примитивный элемент.

Порождающий многочлен для исправляющего одиночные ошибки кода БЧХ над $GF(4)$ длины 15 находится следующим образом:

$$g(x) = \text{НОК}\,[f_1(x), f_2(x)] =$$
$$= (x^2 + x + 2)(x^2 + x + 3) = x^4 + x + 1.$$

Получился порождающий многочлен для (15, 11)-кода БЧХ над $GF(4)$, исправляющего одиночные ошибки. Этим кодом последовательность 11 четверичных символов (что эквивалентно 22 би-

Таблица 7.2

Представления поля $GF(4^2)$

$GF(4)$

+	0	1	2	3
0	0	1	2	3
1	1	0	3	2
2	2	3	0	1
3	3	2	1	0

·	0	1	2	3
0	0	0	0	0
1	0	1	2	3
2	0	2	3	1
3	0	3	1	2

В виде степени	В виде многочлена	В четверичном виде	В десятичном виде	Минимальные многочлены
0	0	00	0	
α^0	1	01	1	$x+1$
α^1	z	10	4	x^2+x+2
α^2	$z+2$	12	6	x^2+x+3
α^3	$3z+2$	32	14	x^2+3x+1
α^4	$z+1$	11	5	x^2+x+2
α^5	2	02	2	$x+2$
α^6	$2z$	20	8	x^2+2x+1
α^7	$2z+3$	23	11	x^2+2x+2
α^8	$z+3$	13	7	x^2+x+3
α^9	$2z+2$	22	10	x^2+2x+1
α^{10}	3	03	3	$x+3$
α^{11}	$3z$	30	12	x^2+3x+3
α^{12}	$3z+1$	31	13	x^2+3x+1
α^{13}	$2z+1$	21	9	x^2+2x+2
α^{14}	$3z+3$	33	15	x^2+3x+3

там) кодируется в последовательность 15 четверичных символов. Такой код не является кодом Хэмминга.

Таким же образом мы можем найти порождающие многочлены для других кодов БЧХ над $GF(4)$ длины 15.

Пусть $t = 2$:

$$g(x) = \text{НОК}\,[f_1(x),\ f_2(x),\ f_3(x),\ f_4(x)] =$$
$$= (x^2+x+2)\,(x^2+x+3)\,(x^2+3x+1) =$$
$$= x^6+3x^5+x^4+x^3+2x^2+2x+1.$$

Получился порождающий многочлен для (15, 9)-кода БЧХ над $GF(4)$, исправляющего две ошибки.

Пусть $t = 3$:

$$g(x) = x^9 + 3x^8 + 3x^7 + 2x^6 + x^5 + 2x^4 + x + 2.$$

Это дает (15, 6)-код БЧХ над $GF(4)$, исправляющий три ошибки.

Пусть $t = 4$:

$$g(x) = x^{11} + x^{10} + 2x^8 + 3x^7 + 3x^6 + x^5 + 3x^4 + x^3 + x + 3.$$

Это дает (15, 4)-код БЧХ над $GF(4)$, исправляющий четыре ошибки.

Пусть $t = 5$:

$$g(x) = x^{12} + 2x^{11} + 3x^{10} + 2x^9 + 2x^8 + x^7 + 3x^6 + 3x^4 + 3x^3 + x^2 + 2.$$

Это дает (15, 3)-код БЧХ над $GF(4)$, исправляющий пять ошибок.

Пусть $t = 6$:

$$g(x) = x^{14} + x^{13} + x^{12} + x^{11} + x^{10} + x^9 + x^8 + x^7 + x^6 + x^5 + x^4 + x^3 + x^2 + x + 1.$$

Получается (15, 1)-код БЧХ над $GF(4)$, исправляющий шесть ошибок. Это простой код с повторением, который в действительности исправляет семь ошибок.

Теперь мы дадим формальное определение кода БЧХ. Оно будет более общим, чем данное выше определение примитивного кода БЧХ, так как в качестве корней $g(x)$ будут браться $2t$ последовательных степеней произвольного элемента β поля (не обязательно примитивного элемента). Длина кода будет равна порядку элемента β, т. е. такому наименьшему n, для которого $\beta^n = 1$.

Определение 7.1.1. Пусть заданы q и m, и пусть β — любой элемент $GF(q^m)$ порядка n. Тогда для любого положительного целого числа t и любого целого числа j_0 соответствующий код БЧХ является циклическим кодом длины n с порождающим многочленом

$$g(x) = \text{НОК}[f_{j_0}(x), f_{j_0+1}(x), \ldots, f_{j_0+2t-1}(x)],$$

где $f_j(x)$ — минимальный многочлен элемента β^j.

Часто выбирают $j_0 = 1$, что, как правило, (но не всегда), приводит к многочлену $g(x)$ с наименьшей степенью. Обычно требуется большая длина кода, и тогда выбирается элемент β поля с наибольшим порядком, т. е. примитивный элемент.

7.2. ДЕКОДЕР ПИТЕРСОНА—ГОРЕНСТЕЙНА—ЦИРЛЕРА

Коды БЧХ являются циклическими, и, следовательно, к ним применимы любые методы декодирования циклических кодов. Имеются, однако, существенно лучшие алгоритмы, разработанные специально для декодирования кодов БЧХ. Рассматриваемый в данном параграфе алгоритм впервые был предложен Питерсоном для двоичных кодов (мы опишем его общий случай, разработанный Горенстейном и Цирлером). Доказав, что этот алгоритм позволяет исправлять t ошибок, мы тем самым докажем, что описанное в предыдущем параграфе построение кодов БЧХ приводит к коду, исправляющему t ошибок. Для упрощения уравнений всюду полагается $j_0 = 1$, хотя все выкладки без изменения идей могут быть проделаны для произвольного j_0.

Предположим, что в основе конструкции кода БЧХ лежит элемент α поля, возможно не примитивный. Многочлен ошибок равен

$$e(x) = e_{n-1}x^{n-1} + e_{n-2}x^{n-2} + \cdots + e_1x + e_0,$$

где не более t коэффициентов отличны от нуля. Предположим, что на самом деле произошло ν ошибок, $0 \leqslant \nu \leqslant t$, и что этим ошибкам соответствуют неизвестные позиции $i_1, i_2, \ldots, i_\nu$. Многочлен ошибок можно записать в виде

$$e(x) = e_{i_1}x^{i_1} + e_{i_2}x^{i_2} + \cdots + e_{i_\nu}x^{i_\nu},$$

где e_{i_l} — величина l-й ошибки (в двоичном случае $e_{i_l} = 1$). Мы не знаем ни $i_1, \ldots, i_\nu$, ни $e_{i_1}, \ldots, e_{i_\nu}$; в действительности мы даже не знаем числа ν. Для исправления ошибок нужно вычислить все эти числа. Чтобы получить компоненту синдрома S_1, надо найти значения полученного многочлена в точке α:

$$S_1 = v(\alpha) = c(\alpha) + e(\alpha) = e(\alpha) =$$

$$= e_{i_1}\alpha^{i_1} + e_{i_2}\alpha^{i_2} + \cdots + e_{i_\nu}\alpha^{i_\nu}.$$

Принятые здесь обозначения слишком громоздки. Для их упрощения определим для всех $l = 1, \ldots, \nu$ величины ошибок $Y_l = e_{i_l}$ и локаторы ошибок $X_l = \alpha^{i_l}$, где i_l — истинное положение l-й ошибки, а X_l — элемент поля, ассоциированный с этим положением. Заметим, что так как порядок элемента α равен n, то все локаторы рассматриваемой конфигурации ошибок различны.

В этих обозначениях S_1 запишется в виде

$$S_1 = Y_1X_1 + Y_2X_2 + \ldots + Y_\nu X_\nu.$$

Аналогично можно вычислить значения принятого многочлена при всех степенях α, входящих в определение $g(x)$. Для $j = 1, \ldots, 2t$ определим синдромы равенствами

$$S_j = v(\alpha^j) = c(\alpha^j) + e(\alpha^j) = e(\alpha^j).$$

Тогда получим следующую систему из $2t$ уравнений относительно ν неизвестных локаторов $X_1, \ldots, X_\nu$ и ν неизвестных величин ошибок $Y_1, \ldots, Y_\nu$:

$$\begin{aligned} S_1 &= Y_1X_1 + Y_2X_2 + \ldots + Y_\nu X_\nu, \\ S_2 &= Y_1X_1^2 + Y_2X_2^2 + \cdots + Y_\nu X_\nu^2, \\ &\vdots \qquad\qquad\qquad\qquad \vdots \\ S_{2t} &= Y_1X_1^{2t} + Y_2X_2^{2t} + \cdots + Y_\nu X_\nu^{2t}. \end{aligned}$$

В силу определения синдрома эта система уравнений должна иметь хотя бы одно решение. Мы увидим, что это решение единственно. Наша задача состоит в вычислении неизвестных по заданным компонентам синдрома, т. е. в решении системы нелинейных уравнений. Описываемый метод решения таких систем подходит для произвольного поля.

Эту систему нелинейных уравнений трудно решать непосредственно. Воспользуемся искусственным приемом, определив некоторые промежуточные переменные, которые могут быть вычислены по компонентам синдрома и по которым можно вычислить затем локаторы ошибок.

Рассмотрим многочлен от x:

$$\Lambda(x) = \Lambda_\nu x^\nu + \Lambda_{\nu-1}x^{\nu-1} + \cdots + \Lambda_1 x + 1,$$

известный под названием многочлена локаторов ошибок и определяемый как многочлен, корнями которого являются обратные к локаторам ошибок величины X_l^{-1} для $l = 1, \ldots, \nu$. Итак,

$$\Lambda(x) = (1 - xX_1)(1 - xX_2)\ldots(1 - xX_\nu).$$

Если коэффициенты этого многочлена известны, то для вычисления локаторов ошибок нужно найти его корни. Поэтому попытаемся сначала вычислить по заданным компонентам синдрома коэффициенты $\Lambda_1, \ldots, \Lambda_\nu$. Если нам это удастся, то задача будет близка к решению.

Умножим обе части равенства, определяющего этот многочлен, на $Y_lX_l^{j+\nu}$ и положим $x = X_l^{-1}$. Тогда левая часть обратится в нуль, и мы получим

$$0 = Y_lX_l^{j+\nu}(1 + \Lambda_1X_l^{-1} + \Lambda_2X_l^{-2} + \cdots + \Lambda_{\nu-1}X_l^{-(\nu-1)} + \Lambda_\nu X_l^{-\nu}),$$

или

$$Y_l(X_l^{j+\nu} + \Lambda_1 Y_l^{j+\nu-1} + \cdots + \Lambda_\nu X_l^j) = 0.$$

Это равенство выполняется при каждом l и при каждом j. Просуммируем эти равенства по l от 1 до ν. Для каждого j это дает

$$\sum_{l=1}^{\nu} Y_l(X_l^{j+\nu} + \Lambda_1 X_l^{j+\nu-1} + \cdots + \Lambda_\nu X_l^j) = 0,$$

или

$$\sum_{l=1}^{\nu} Y_l X_l^{j+\nu} + \Lambda_1 \sum_{l=1}^{\nu} Y_l X_l^{j+\nu-1} + \cdots + \Lambda_\nu \sum_{l=1}^{\nu} Y_l X_l^j = 0.$$

Каждая сумма в левой части последнего равенства является компонентой синдрома, так что уравнение приводится к виду

$$S_{j+\nu} + \Lambda_1 S_{j+\nu-1} + \Lambda_2 S_{j+\nu-2} + \cdots + \Lambda_\nu S_j = 0.$$

Так как $\nu \leqslant t$, то для j в интервале $1 \leqslant j \leqslant \nu$ все индексы задают известные компоненты синдрома. Таким образом, получаем систему уравнений

$$\Lambda_1 S_{j-\nu-1} + \Lambda_2 S_{j+\nu-2} + \cdots + \Lambda_\nu S_j = -S_{j+\nu}, \quad j = 1, \ldots, \nu,$$

т. е. систему линейных уравнений, связывающих компоненты синдрома с коэффициентами многочлена $\Lambda(x)$. Эту систему можно записать в матричном виде

$$\begin{bmatrix} S_1 & S_2 & S_3 & \ldots & S_{\nu-1} & S_\nu \\ S_2 & S_3 & S_4 & \ldots & S_\nu & S_{\nu+1} \\ S_3 & S_4 & S_5 & \ldots & S_{\nu+1} & S_{\nu+2} \\ \cdot & \cdot & \cdot & & \cdot & \cdot \\ \cdot & \cdot & \cdot & & \cdot & \cdot \\ \cdot & \cdot & \cdot & & \cdot & \cdot \\ S_\nu & S_{\nu+1} & S_{\nu+2} & \ldots & S_{2\nu-2} & S_{2\nu-1} \end{bmatrix} \begin{bmatrix} \Lambda_\nu \\ \Lambda_{\nu-1} \\ \Lambda_{\nu-2} \\ \cdot \\ \cdot \\ \cdot \\ \Lambda_1 \end{bmatrix} = \begin{bmatrix} -S_{\nu+1} \\ -S_{\nu+2} \\ -S_{\nu+3} \\ \cdot \\ \cdot \\ \cdot \\ -S_{2\nu} \end{bmatrix}.$$

Если матрица невырождена, то эту систему можно решить путем обращения матрицы. Докажем, что если произошло ν ошибок, то матрица невырождена. Первым шагом этого доказательства служит доказательство невырожденности матрицы Вандермонда.

Теорема 7.2.1. *Матрица Вандермонда, определяемая как произвольная матрица вида*

$$\mathbf{A} = \begin{bmatrix} 1 & 1 & \ldots & 1 \\ X_1 & X_2 & \ldots & X_\mu \\ X_1^2 & X_2^2 & \ldots & X_\mu^2 \\ \cdot & \cdot & & \cdot \\ \cdot & \cdot & & \cdot \\ \cdot & \cdot & & \cdot \\ X_1^{\mu-1} & X_2^{\mu-1} & & X_\mu^{\mu-1} \end{bmatrix},$$

имеет отличный от нуля определитель тогда и только тогда, когда $X_i \neq X_j$ *для всех* $i, j = 1, \ldots, \mu$; $i \neq j$.

Доказательство. Обратное утверждение очевидно, поскольку если любые два X_j равны, то в матрице оказываются два одинаковых столбца и, следовательно, определитель равен нулю.

Для доказательства прямого утверждения воспользуемся индукцией. Утверждение верно при $\mu = 1$. Мы сейчас покажем, что если оно верно для матриц Вандермонда размера $(\mu - 1) \times (\mu - 1)$, то оно верно и для матриц Вандермонда размера $\mu \times \mu$. Заменим X_1 в матрице на переменную x. Тогда определитель будет функцией от x и запишется в виде

$$D(x) = \det \begin{bmatrix} 1 & x & x^2 & \ldots & x^{\mu-1} \\ 1 & X_2 & X_2^2 & \ldots & X_2^{\mu-1} \\ \cdot & & & & \\ \cdot & & & & \\ \cdot & & & & \\ 1 & X_\mu & X_\mu^2 & \ldots & X_\mu^{\mu-1} \end{bmatrix}.$$

Определитель можно разложить в сумму элементов первой строки, умноженных на соответствующие этим элементам алгебраические дополнения. Это даст многочлен от x степени $\mu - 1$, который может быть записан в виде

$$D(x) = d_{\mu-1}x^{\mu-1} + \ldots + d_1 x + d_0$$

и, таким образом, имеет не более $\mu - 1$ корней. Коэффициент $d_{\mu-1}$ сам по себе равен определителю матрицы Вандермонда и по предположению индукции не равен нулю. Если для любого i, $2 \leqslant i \leqslant \mu$, мы положим $x = X_i$, то два столбца в матрице будут равны и $D(X_i) = 0$. Таким образом, каждое X_i $(i \neq 1)$ есть корень $D(x)$, и, поскольку все они различны и их число равно $\mu - 1$, легко получить разложение многочлена

$$D(x) = d_{\mu-1} \prod_{i=2}^{\mu} (x - X_i).$$

Отсюда следует, что определитель исходной матрицы Вандермонда равен

$$D(X_1) = d_{\mu-1} \prod_{i=2}^{\mu} (X_1 - X_i).$$

Последнее выражение отлично от нуля, поскольку $d_{\mu-1}$ отлично от нуля, а X_1 отличен от остальных X_i. Поэтому определитель матрицы Вандермонда размера $\mu \times \mu$ не равен нулю, и по индукции получаем справедливость теоремы для любого μ. □

Теперь у нас все готово для доказательства центральной теоремы алгоритма декодирования. Эта теорема дает условие, по которому можно определить число ν произошедших в действительности ошибок.

Теорема 7.2.2. *Матрица компонент синдрома*

$$\mathbf{M} = \begin{bmatrix} S_1 & S_2 & \dots & S_\mu \\ S_2 & S_3 & \dots & S_{\mu+1} \\ \vdots & & & \\ S_\mu & S_{\mu+1} & \dots & S_{2\mu-1} \end{bmatrix}$$

невырождена, если μ равно числу ν произошедших в действительности ошибок. Матрица является вырожденной, если μ больше ν.

Доказательство. Пусть $X_\mu = 0$ для $\mu > \nu$, и пусть $\mathbf{A}$ — матрица Вандермонда

$$\mathbf{A} = \begin{bmatrix} 1 & 1 & \dots & 1 \\ X_1 & X_2 & \dots & X_\mu \\ \vdots & & & \vdots \\ X_1^{\mu-1} & X_2^{\mu-1} & \dots & X_\mu^{\mu-1} \end{bmatrix}$$

с элементами $A_{ij} = X_j^{i-1}$, а $\mathbf{B}$ — диагональная матрица

$$\mathbf{B} = \begin{bmatrix} Y_1X_1 & 0 & \dots & 0 \\ 0 & Y_2X_2 & \dots & 0 \\ \vdots & & & \\ 0 & 0 & \dots & Y_\mu X_\mu \end{bmatrix}$$

с элементами $B_{ij} = Y_iX_i\delta_{ij}$, где $\delta_{ij} = 1$, если $i = j$, и $\delta_{ij} = 0$ в противном случае. Тогда произведение матриц $\mathbf{ABA}^\tau$ будет иметь элементы

$$(\mathbf{ABA}^\tau)_{ij} = \sum_{l=1}^{\mu} X_l^{i-1} \sum_{k=1}^{\mu} Y_lX_l\delta_{lk}X_k^{j-1} =$$

$$= \sum_{l=1}^{\mu} X_l^{i-1}Y_lX_lX_l^{j-1} = \sum_{l=1}^{\mu} Y_lX_l^{i+j-1},$$

которые совпадают с элементами матрицы $\mathbf{M}$. Таким образом, $\mathbf{M} = \mathbf{ABA}^\tau$ и, следовательно, определитель матрицы $\mathbf{M}$ удовлетворяет соотношению

$$\det(\mathbf{M}) = \det(\mathbf{A}) \det(\mathbf{B}) \det(\mathbf{A}).$$

Если μ больше ν, то $\det(\mathbf{B}) = 0$. Тогда $\det(\mathbf{M}) = 0$ и $\mathbf{M}$ вырождена. Если μ равно ν, то $\det(\mathbf{B}) \neq 0$. Кроме того, определитель матрицы Вандермонда $\mathbf{A}$ не равен нулю, если все столбцы матрицы различны, что выполняется, если μ равно ν. Следовательно, $\det(\mathbf{M}) \neq 0$, и теорема доказана. □

Эта теорема является основной при построении алгоритма декодирования. Прежде всего найдем правильное значение ν, поступая следующим образом. В качестве пробного значения возьмем $\nu = t$ и вычислим определитель матрицы $\mathbf{M}$. Если он не равен нулю, то мы получили правильное значение для ν; если же определитель равен нулю, то уменьшим значение ν на 1 и повторим процедуру. Поступать таким образом необходимо до тех пор, пока не будет получен определитель, отличный от нуля, и тогда мы узнаем число ν ошибок, которые произошли на самом деле. Затем обратим матрицу $\mathbf{M}$ и вычислим коэффициенты $\Lambda(x)$. Найдя корни $\Lambda(x)$, найдем локаторы ошибок. Если код двоичный, то ошибки известны. В противном случае вернемся к уравнениям, определяющим компоненты синдрома:

$$
\begin{aligned}
S_1 &= Y_1X_1 + Y_2X_2 + \ldots + Y_\nu X_\nu,\\
S_2 &= Y_1X_1^2 + Y_2X_2^2 + \ldots + Y_\nu X_\nu^2,\\
&\vdots\\
S_{2t} &= Y_1X_1^{2t} + Y_2X_2^{2t} + \ldots + Y_\nu X_\nu^{2t}.
\end{aligned}
$$

Поскольку локаторы ошибок известны, мы имеем систему $2t$ линейных уравнений с ν неизвестными значениями ошибок. Первые ν уравнений могут быть решены относительно значений ошибок, если определитель матрицы коэффициентов этих уравнений не равен нулю. Но

$$
\det\begin{bmatrix} X_1 & X_2 & \ldots & X_\nu \\ X_1^2 & X_2^2 & \ldots & X_\nu^2 \\ \vdots & & & \\ X_1^\nu & X_2^\nu & \ldots & X_\nu^\nu \end{bmatrix} = (X_1X_2\ldots X_\nu)\det\begin{bmatrix} 1 & 1 & \ldots & 1 \\ X_1 & X_2 & \ldots & X_\nu \\ \vdots & & & \\ X_1^{\nu-1} & X_2^{\nu-1} & \ldots & X_\nu^{\nu-1} \end{bmatrix}.
$$

По теореме 7.2.1 определитель последней матрицы не равен нулю, если произошло ν ошибок, поскольку $X_1, X_2, \ldots, X_\nu$ различны и не равны нулю.

Алгоритм декодирования представлен на рис. 7.1. Здесь мы предположили, что j_0 произвольно, хотя все рассуждения проводились для частного случая $j_0 = 1$. Рассуждения остаются теми же для произвольного j_0.

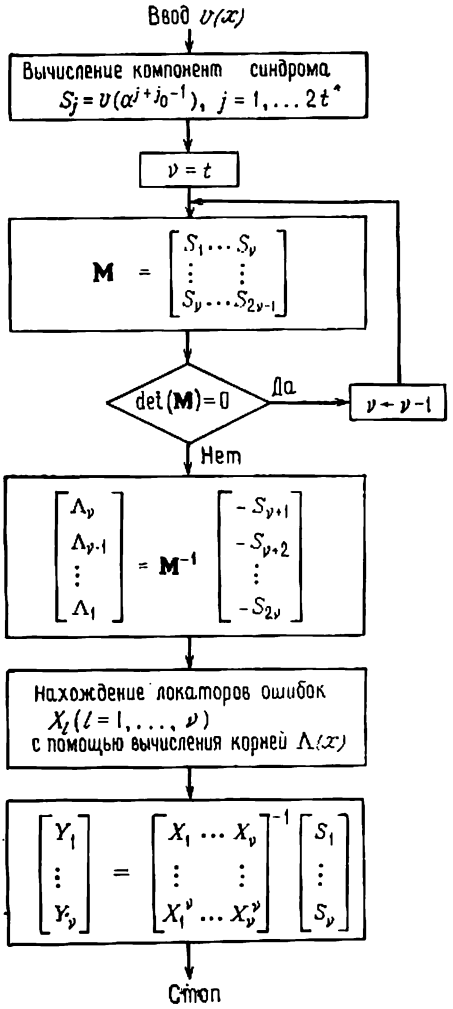


Рис. 7.1. Декодер Питерсона—Горенстейна—Цирлера.

Поскольку число элементов поля конечно, обычно простейшим путем нахождения корней многочлена $\Lambda(x)$ является метод проб и ошибок, известный как *процедура Ченя*. Эта процедура состоит просто в последовательном вычислении $\Lambda(\alpha^j)$ для каждого j и проверки полученных значений на нуль. Наиболее простым способом вычисления значения $\Lambda(x)$ в точке β является схема Горнера:

$$\Lambda(\beta) = (\ldots(((\Lambda_\nu\beta + \Lambda_{\nu-1})\beta + \Lambda_{\nu-2})\beta + \Lambda_{\nu-3})\beta + \ldots + \Lambda_0).$$

Для вычисления $\Lambda(\beta)$ по схеме Горнера требуется только ν умножений и ν сложений.

В качестве примера процедуры декодирования рассмотрим декодирование (15,5)-кода БЧХ, исправляющего три ошибки и имеющего порождающий многочлен

$$g(x) = x^{10} + x^8 + x^5 + x^4 + x^2 + x + 1.$$

Для примера будем считать принятый многочлен равным $v(x) = x^7 + x^2$. Ясно, что если бы произошло не более трех ошибок, то кодовое слово должно было бы быть нулевым и $v(x) = e(x)$, но декодер не может сделать такого заключения. Выполним все шаги алгоритма декодирования. Сначала вычислим компоненты синдрома, используя арифметику в поле $GF(16)$:

$$\begin{aligned}
S_1 &= \alpha^7 + \alpha^2 = \alpha^{12},\\
S_2 &= \alpha^{14} + \alpha^4 = \alpha^9,\\
S_3 &= \alpha^{21} + \alpha^6 = 0,\\
S_4 &= \alpha^{28} + \alpha^8 = \alpha^3,\\
S_5 &= \alpha^{35} + \alpha^{10} = \alpha^0,\\
S_6 &= \alpha^{42} + \alpha^{12} = 0.
\end{aligned}$$

Пусть $\nu = 3$, тогда

$$\mathbf{M} = \begin{bmatrix} S_1 & S_2 & S_3 \\ S_2 & S_3 & S_4 \\ S_3 & S_4 & S_5 \end{bmatrix} = \begin{bmatrix} \alpha^{12} & \alpha^9 & 0 \\ \alpha^9 & 0 & \alpha^3 \\ 0 & \alpha^3 & 1 \end{bmatrix}.$$

Определитель $\mathbf{M}$ равен нулю; следовательно, полагаем $\nu = 2$. Тогда

$$\mathbf{M} = \begin{bmatrix} S_1 & S_2 \\ S_2 & S_3 \end{bmatrix} = \begin{bmatrix} \alpha^{12} & \alpha^9 \\ \alpha^9 & 0 \end{bmatrix}.$$

Определитель не равен нулю; следовательно, произошло две ошибки. Далее,

$$\mathbf{M}^{-1} = \begin{bmatrix} 0 & \alpha^6 \\ \alpha^6 & \alpha^9 \end{bmatrix}$$

и

$$\begin{bmatrix} \Lambda_2 \\ \Lambda_1 \end{bmatrix} = \begin{bmatrix} 0 & \alpha^6 \\ \alpha^6 & \alpha^9 \end{bmatrix} \begin{bmatrix} 0 \\ \alpha^3 \end{bmatrix} = \begin{bmatrix} \alpha^9 \\ \alpha^{12} \end{bmatrix};$$

следовательно,

$$\Lambda(x) = \alpha^9 x^2 + \alpha^{12} x + 1.$$

Используя процедуру Ченя, получаем разложение

$$\Lambda(x) = (\alpha^7 x + 1)(\alpha^2 x + 1) = \alpha^9 (x - \alpha^8)(x - \alpha^{13}).$$

Многочлен локаторов ошибок $\Lambda(x)$ имеет корни α^8 и α^{13}, а локаторы ошибок равны элементам, обратным корням. Таким образом, ошибки произошли во второй и седьмой позициях. Поскольку код является двоичным, значения ошибок равны 1 и $e(x) = x^7 + x^2$.

7.3. КОДЫ РИДА—СОЛОМОНА

Важным и широко используемым подмножеством кодов БЧХ являются коды Рида—Соломона. Это такие коды БЧХ, у которых мультипликативный порядок алфавита символов кодового слова делится на длину кода. Таким образом, $m = 1$ и поле символов $GF(q)$ совпадает с полем локаторов ошибок $GF(q^m)$. Обычно мы будем выбирать α примитивным; тогда

$$n = q^m - 1 = q - 1.$$

Минимальный многочлен над $GF(q)$ элемента β, взятого из того же поля, равен

$$f_\beta(x) = x - \beta.$$

Поскольку поле символов и поле локаторов ошибок совпадают, все минимальные многочлены линейны. В коде Рида—Соломона, исправляющем t ошибок, обычно полагается $j_0 = 1$, и тогда порождающий многочлен записывается в виде

$$g(x) = (x - \alpha)(x - \alpha_2) \ldots (x - \alpha^{2t}).$$

Степень этого многочлена всегда равна $2t$, откуда следует, что параметры кода Рида—Соломона связаны соотношением

$$n - k = 2t.$$

В коде Рида—Соломона можно выбрать также любое другое значение j_0, причем с помощью разумного выбора j_0 иногда удается упростить кодер. Таким образом,

$$g(x) = (x - \alpha^{j_0})(x - \alpha^{j_0+1}) \ldots (x - \alpha^{j_0+2t-1}).$$

В качестве примера найдем $g(x)$ для (15,11)-кода Рида—Соломона с $t = 2$ над $GF(16)$. Может быть выбрано любое j_0; мы выберем $j_0 = 1$. Тогда

$$\begin{aligned} g(x) &= (x - \alpha)(x - \alpha^2)(x - \alpha^3)(x - \alpha^4) = \\ &= x^4 + (z^3 + z^2 + 1)x^3 + (z^3 + z^2)x^2 + z^3 x + (z^2 + z + 1) = \\ &= x^4 + \alpha^{13}x^3 + \alpha^6 x^2 + \alpha^3 x + \alpha^{10}. \end{aligned}$$

(Здесь использовано приведенное в табл. 7.1 представление элементов поля $GF(16)$ в виде многочленов от z.) Поскольку сте-

пень $g(x)$ равна 4, $n - k = 4$ и $k = 11$. Информационный многочлен представляет собой последовательность одиннадцати символов из $GF(16)$, что эквивалентно 44 битам.

В качестве второго примера найдем $g(x)$ для (7, 3)-кода Рида—Соломона с $t = 2$ над $GF(8)$. Может быть выбрано любое j_0; мы выберем $j_0 = 4$. Тогда

$$g(x) = (x - \alpha^4)(x - \alpha^5)(x - \alpha^6)(x - \alpha^0) =$$
$$= x^4 + (z^2 + 1)x^3 + (z^2 + 1)x^2 + (z + 1)x + z$$

(здесь использовано представление элементов поля $GF(8)$ в виде многочленов от z). Информационный многочлен представляет собой последовательность трех восьмеричных символов (что эквивалентно девяти битам). Предположим, что

$$i(x) = (z^2 + z)x^2 + x + (z + 1).$$

Кодовое слово несистематического кода запишется в виде

$$c(x) = i(x)g(x) =$$
$$= (\alpha^4 x^2 + x + \alpha^3)(x^4 + \alpha^6 x^3 + \alpha^6 x^2 + \alpha^3 x + \alpha) =$$
$$= \alpha^4 x^6 + \alpha x^5 + \alpha^6 x^4 + 0x^3 + 0x^2 + \alpha^5 x + \alpha^4,$$

что представляет собой последовательность семи восьмеричных символов.

Коды Рида—Соломона являются оптимальными в смысле границы Синглтона.

Теорема 7.3.1. *Код Рида—Соломона имеет минимальное расстояние* $n - k + 1$ *и является кодом с максимальным расстоянием.*

Доказательство. Пусть $d = 2t + 1$ — конструктивное расстояние кода. Минимальное расстояние d^* удовлетворяет неравенству

$$d^* \geqslant d = 2t + 1 = n - k + 1,$$

поскольку для кодов Рида—Соломона $2t = n - k$. Но для любого линейного кода имеет место граница Синглтона

$$d^* \leqslant n - k + 1.$$

Следовательно, $d^* = n - k + 1$ и $d^* = d$. □

Доказанная теорема утверждает, что при фиксированных n и k не существует кода, у которого минимальное расстояние больше, чем у кода Рида—Соломона. Этот факт часто является веским основанием для использования кода Рида—Соломона. Не стоит, однако, понимать это утверждение буквально. Часто предпочтение отдается кодам с такими параметрами (n', k'),

при которых не существует кода Рида—Соломона. В то же время коды Рида—Соломона всегда оказываются короче всех других циклических кодов над тем же алфавитом.

7.4. СИНТЕЗ АВТОРЕГРЕССИОННЫХ ФИЛЬТРОВ

При использовании описанного в § 7.2 алгоритма декодирования кодов БЧХ основной объем вычислений приходится на решение системы уравнений

$$\begin{bmatrix} S_1 & S_2 & S_3 & \dots & S_\nu \\ S_2 & S_3 & S_4 & \dots & S_{\nu+1} \\ S_3 & S_4 & S_5 & \dots & S_{\nu+2} \\ \cdot & & & & \\ \cdot & & & & \\ \cdot & & & & \\ S_\nu & S_{\nu+1} & S_{\nu+2} & \dots & S_{2\nu-1} \end{bmatrix} \begin{bmatrix} \Lambda_\nu \\ \Lambda_{\nu-1} \\ \Lambda_{\nu-2} \\ \\ \\ \\ \Lambda_1 \end{bmatrix} = \begin{bmatrix} -S_{\nu+1} \\ -S_{\nu+2} \\ -S_{\nu+3} \\ \\ \\ \\ -S_{2\nu} \end{bmatrix}.$$

При небольших значениях ν эту систему можно решить с помощью обращения матрицы. Для обращения матрицы размера $\nu \times \nu$ необходимо произвести порядка ν^3 действий. Однако на практике часто требуется использовать коды, исправляющие большое число ошибок, и в таких случаях желателен более эффективный метод решения данной системы. Такой метод был разработан Берлекэмпом. В основе этого метода лежит тот факт, что выписанное выше матричное уравнение не произвольно: матрица обладает специальной структурой. Это позволяет существенно упростить связанные с нахождением вектора Λ вычисления, хотя сама процедура вычислений становится более сложной для понимания, чем простое обращение матрицы.

Остановимся на предложенном Месси варианте алгоритма, который понял, что наилучший способ реализации алгоритма состоит в переформулировке задачи в виде задачи построения схемы с использованием регистров сдвига с линейной обратной связью. Предположим, что нам известен вектор Λ. Тогда первая строка выписанной выше системы определяет значение $S_{\nu+1}$ через значения $S_1, \dots, S_\nu$, вторая строка определяет $S_{\nu+2}$ через $S_2, \dots, S_{\nu+1}$ и т. д. Этот последовательный процесс описывается уравнением

$$S_j = -\sum_{i=1}^{\nu} \Lambda_i S_{j-i}, \quad j = \nu+1, \dots, 2\nu.$$

Для фиксированного Λ это уравнение определяет авторегрессионный фильтр. Он может быть реализован как регистр сдвига с линейной обратной связью, множители в отводах которого задаются вектором Λ.

Переформулированная таким образом задача сводится к построению изображенного на рис. 7.2 регистра сдвига с линейной обратной связью, генерирующего известную последовательность компонент синдрома. Задача состоит в том, чтобы среди большого числа таких регистров найти регистр сдвига с наименьшей длиной. Это позволит определить вектор ошибок минимального веса в принятом слове, или, что то же самое, определить многочлен Λ наименьшей степени. Многочлен минимальной степени имеет степень ν и определяется единственным образом, поскольку матрица размера $\nu \times \nu$ исходной системы уравнений обратима.

Процедура построения авторегрессионного фильтра является также методом решения матричного уравнения относительно вектора Λ. Ниже будет приведена процедура построения такого регистра сдвига. Описанный метод окажется применимым для вычисления в любых полях и не будет предполагать каких-либо особых свойств последовательности $S_1, S_2, \ldots, S_{2t}$. Эта последовательность не обязательно должна состоять из компонент синдрома, но если ее элементы все же являются компонентами синдрома для исправляемой конфигурации ошибок, то процедура всегда приведет к регистру сдвига с ненулевым весовым множителем в самом правом отводе. Для произвольного регистра сдвига с линейной обратной связью, описываемой многочленом $\Lambda(x)$, длина регистра может быть больше, чем степень $\Lambda(x)$, поскольку несколько самых правых ячеек регистра может быть не охвачено обратной связью.

Для построения требуемого регистра сдвига необходимо найти две величины: длину регистра сдвига L и многочлен обратной связи $\Lambda(x)$:

$$\Lambda(x) = \Lambda_\nu x^\nu + \Lambda_{\nu-1} x^{\nu-1} + \ldots + \Lambda_1 x + 1,$$

где $\deg \Lambda(x) \leqslant L$. Обозначим эту пару через $(L, \Lambda(x))$. Необходимо найти регистр сдвига с обратной связью наименьшей длины, который генерирует последовательность $S_1, \ldots, S_{2t}$, при соответствующем начальном состоянии.

Процедура построения регистра является индуктивной. Для каждого r, начиная с $r = 1$, построим регистр сдвига, генериру-

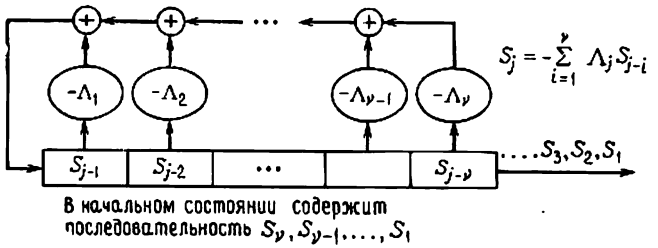


Рис. 7.2. Многочлен локаторов ошибок в цепи регистра сдвига.

ющий первые r компонент синдрома. Регистр сдвига $(L_r, \Lambda^{(r)}(x))$ является регистром сдвига минимальной длины, генерирующим $S_1, \ldots, S_r$. Он не обязательно должен быть единственным: может существовать несколько таких регистров сдвига, но все они имеют одинаковую длину. К началу r-й итерации уже будет задан список регистров сдвига

$$\begin{aligned}&(L_1, \quad \Lambda^{(1)}(x)),\\ &(L_2, \quad \Lambda^{(2)}(x)),\\ &\vdots\\ &(L_{r-1}, \ \Lambda^{(r-1)}(x)).\end{aligned}$$

Основная идея алгоритма Берлекэмпа—Месси состоит в нахождении способа построения нового регистра минимальной длины $(L_r, \Lambda^{(r)}(x))$, генерирующего последовательность $S_1, \ldots, S_{r-1}, S_r$. Это будет сделано с использованием предыдущего регистра сдвига, в котором при необходимости будут надлежащим образом изменены длина и весовые множители.

При r-й итерации вычислим следующий выход $(r-1)$-го регистра сдвига:

$$\hat{S}_r = -\sum_{j=1}^{n-1} \Lambda_j^{(r-1)} S_{r-j}.$$

Поскольку $n-1$ больше, чем степень $\Lambda^{(r-1)}$, многие слагаемые в сумме будут равны нулю, и, таким образом, ее можно записать как сумму от 1 до $\deg \Lambda^{(r-1)}(x)$. Однако обозначения будут менее громоздкими, если сумма будет записываться так, как указано выше.

Вычитая $\hat{S}_r$ из требуемого выхода S_r, получаем величину Δ_r, называемую *r-й невязкой:*

$$\Delta_r = S_r - \hat{S}_r = S_r + \sum_{j=1}^{n-1} \Lambda_j^{(r-1)} S_{r-j},$$

или, что эквивалентно,

$$\Delta_r = \sum_{j=0}^{n-1} \Lambda_j^{(r-1)} S_{r-j}.$$

Если Δ_r равняется нулю, то положим $(L_r, \Lambda^{(r)}(x)) = (L_{r-1}, \Lambda^{(r-1)}(x))$, и r-я итерация будет закончена. В противном случае следующим образом изменим весовые множители в цепи обратной связи регистра сдвига:

$$\Lambda^{(r)}(x) = \Lambda^{(r-1)}(x) + Ax^l \Lambda^{(m-1)}(x),$$

где A — элемент поля, l — целое число, а $\Lambda^{(m-1)}(x)$ — один из многочленов регистра сдвига, встречавшихся в одной из

предыдущих итераций. Теперь, используя новый многочлен, снова вычислим невязку (обозначим ее Δ_r')

$$\Delta_r' = \sum_{j=0}^{n-1} \Lambda_j^{(r)} S_{r-j} = \sum_{j=0}^{n-1} \Lambda_j^{(r-1)} S_{r-j} + A \sum_{j=0}^{n-1} \Lambda_j^{(m-1)} S_{r-j-1}.$$

Теперь все готово для определения величин m, l и A. Выберем m меньше r и такое, что $\Delta_m \neq 0$. Выберем также $l = r - m$ и $A = -\Delta_m^{-1}\Delta_r$. Тогда

$$\Delta_r' = \Delta_r - \frac{\Delta_r}{\Delta_m}\Delta_m = 0,$$

и, следовательно, новый регистр сдвига будет генерировать последовательность $S_1, \ldots, S_{r-1}, S_r$. Однако нам нужен не любой такой регистр сдвига, а регистр сдвига с минимальной длиной. У нас остался произвол в выборе m, для которого $\Delta_m \neq 0$. Выбрав в качестве m номер ближайшей итерации, в которой выполнялось условие $L_m > L_{m-1}$, получим в каждой итерации регистр минимальной длины, однако такое усовершенствование требует для своего выполнения дополнительного времени.

Практическая реализация того, что было изложено до настоящего момента, представлена на рис. 7.3. Два регистра сдвига при итерациях m и r изображены на рис. 7.3, *а*. В качестве m-й итерации выбирается та, при которой регистр сдвига $(L_{m-1}, \Lambda^{m-1}(x))$ не способен генерировать компоненту синдрома S_m, и минимальная длина регистра сдвига, генерирующего требуемую компоненту синдрома при m-й итерации, больше L_{m-1}. Будем также считать, что регистр сдвига $(L_{r-1}, \Lambda^{(r-1)}(x))$ не способен генерировать S_r; в противном случае он остается без изменения.

На рис. 7.3, *б* регистр сдвига $(L_{m-1}, \Lambda^{(m-1)}(x))$ превращен во вспомогательный регистр путем увеличения его длины, изменения его связей и такой его перестройки, которая позволяет скомпенсировать неспособность регистра $(L_{r-1}, \Lambda^{(r-1)}(x))$ генерировать S_r. Отметим, что вспомогательный регистр имеет дополнительный отвод с весовым множителем единица, соответствующим коэффициенту $\Lambda_0^{(m-1)}$. На протяжении первых $r - 1$ итераций в остальных отводах цепи обратной связи формируется отрицательное значение этой соответствующей дополнительному отводу величины, и поэтому на выходе вспомогательного регистра формируется последовательность нулей, не оказывающая влияния на генерируемые компоненты синдрома. При r-й итерации эти величины взаимно не уничтожаются, и выход вспомогательного регистра оказывается ненулевым. Коэффициент A выбирается таким образом, чтобы его добавление к r-му выходному сигналу обратной связи главного регистра приводило бы к получению нужной компоненты синдрома S_r.

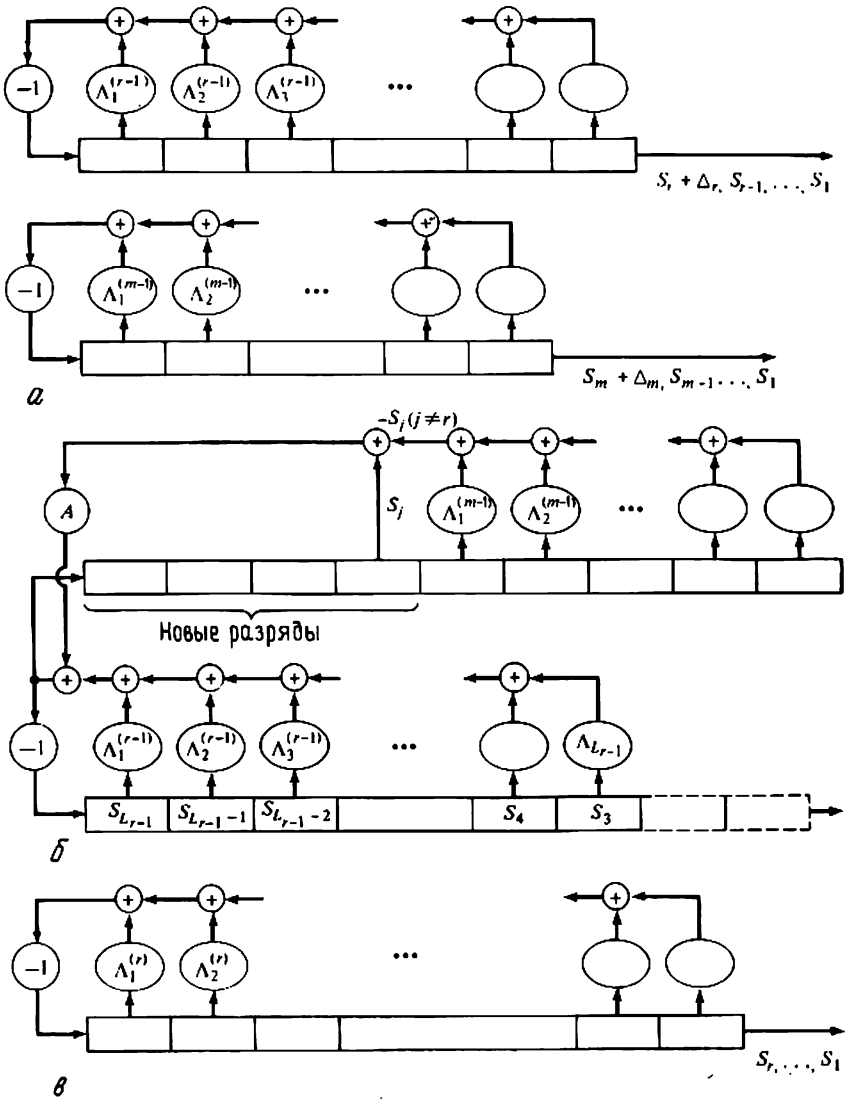


Рис. 7.3. Конструкция Берлекэмпа—Месси.

На рис. 7.3, *в* показано, как два регистра сдвига, изображенные на рис. 7.3, *б*, сливаются в один регистр, что в силу принципа суперпозиции не меняет общего поведения схемы. Это дает регистр $(L_r, \Lambda^{(r)}(x))$. Иногда $L_r = L_{r-1}$, иногда $L_r > L_{r-1}$. В последнем случае при проведении дальнейших итераций m заменяется на r.

Точное описание процедуры дается следующей теоремой, которая утверждает, что эта процедура приводит к наименьшему регистру сдвига с требуемым свойством. Довольно длинное доказательство теоремы займет всю оставшуюся часть этого параграфа.

Теорема 7.4.1 (алгоритм Берлекэмпа—Месси). *Пусть заданы* $S_1, \ldots, S_{2t}$ *из некоторого поля, и пусть при начальных условиях* $\Lambda^{(0)}(x) = 1$, $B^{(0)}(x) = 1$ *и* $L_0 = 0$ *выполняются следующие рекуррентные равенства, используемые для вычисления* $\Lambda^{(2t)}(x)$:

$$\Delta_r = \sum_{j=0}^{n-1} \Lambda_j^{(r-1)} S_{r-j},$$

$$L_r = \delta_r (r - L_{r-1}) + (1 - \delta_r) L_{r-1},$$

$$\begin{bmatrix} \Lambda^{(r)}(x) \\ B^{(r)}(x) \end{bmatrix} = \begin{bmatrix} 1 & -\Delta_r x \\ \Delta_r^{-1}\delta_r & (1 - \delta_r)x \end{bmatrix} \begin{bmatrix} \Lambda^{(r-1)}(x) \\ B^{(r-1)}(x) \end{bmatrix},$$

$r = 1, \ldots, 2t$, *где* $\delta_r = 1$, *если одновременно* $\Delta_r \neq 0$ *и* $2L_{r-1} \leqslant r - 1$, *и* $\delta_r = 0$ *в противном случае. Тогда* $\Lambda^{(2t)}(x)$ *является многочленом наименьшей степени, коэффициенты которого удовлетворяют равенствам* $\Lambda_0^{(2t)} = 1$ *и*

$$S_r + \sum_{j=1}^{n-1} \Lambda_j^{(2t)} S_{r-j} = 0, \quad r = L_{2t} + 1, \ldots, 2t.$$

В этой теореме Δ_r может равняться нулю, но только в том случае, когда $\delta_r = 0$. Положим тогда по определению $\Delta_r^{-1}\delta_r = 0$.

В каждой итерации для умножения матриц требуется не более $2t$ умножений, а для вычисления Δ_r — не более t умножений. Всего производится $2t$ итераций, так что делается не более $6t^2$ умножений. Следовательно, использование этого алгоритма обычно эффективнее обращения матрицы, в котором требуется порядка t^3 операций.

Доказательство теоремы 7.4.1 сводится к двум приведенным ниже леммам. Сначала в лемме 7.4.2 находится неравенство, связывающее L_r и L_{r-1}. Затем в лемме 7.4.3 используется указанный в теореме 7.4.1 алгоритм для построения по регистру сдвига минимальной длины, генерирующему последовательность $S_1, \ldots, S_{r-1}$, регистра сдвига, генерирующего последовательность $S_1, \ldots, S_r$. Из леммы 7.4.3 будет следовать, что вытекающее из теоремы 7.4.1 построение приводит к регистру сдвига минимальной длины, поскольку его параметры будут удовлетворять границе из леммы 7.4.2.

Лемма 7.4.2. *Пусть* $(L_{r-1}, \Lambda^{(r-1)}(x))$ *— регистр сдвига с линейной обратной связью минимальной длины, генерирующий последовательность* $S_1, \ldots, S_{r-1}$, *а* $(L_r, \Lambda^{(r)}(x))$ *— регистр сдвига*

с линейной связью минимальной длины, генерирующий $S_1, \ldots, S_{r-1}, S_r$, *и* $\Lambda^{(r)}(x) \neq \Lambda^{(r-1)}(x)$. *Тогда*

$$L_r \geqslant \max[L_{r-1}, r - L_{r-1}].$$

Доказательство. Неравенство, которое требуется доказать, разбивается на два неравенства:

$$L_r \geqslant L_{r-1} \quad \text{и} \quad L_r \geqslant r - L_{r-1}.$$

Первое неравенство очевидно, поскольку если регистр сдвига с линейной обратной связью генерирует некоторую последовательность, то он генерирует и любую меньшую последовательность, являющуюся началом исходной. Если $L_{r-1} \geqslant r$, то второе неравенство также становится очевидным. Поэтому предположим, что $L_{r-1} < r$ и что второе неравенство не выполняется, и попытаемся прийти к противоречию. Из предположения следует, что $L_r \leqslant r - 1 - L_{r-1}$. Для сокращения записи введем следующие обозначения: $c(x) = \Lambda^{(r-1)}(x)$, $b(x) = \Lambda^{(r)}(x)$, $L = L_{r-1}$ и $L' = L_r$. В новых обозначениях исходные предположения примут вид $r \geqslant L + L' + 1$ и $L < r$. Кроме того, из условий леммы следует, что

$$S_r \neq -\sum_{i=1}^{L} c_i S_{r-i},$$

$$S_j = -\sum_{i=1}^{L} c_i S_{j-i}, \quad j = L+1, \ldots, r-1,$$

и

$$S_j = -\sum_{k=1}^{L'} b_k S_{j-k}, \quad j = L'+1, \ldots, r.$$

Теперь установим противоречие. С одной стороны,

$$S_r = -\sum_{k=1}^{L'} b_k S_{r-k} = \sum_{k=1}^{L'} b_k \sum_{i=1}^{L} c_i S_{r-k-i},$$

где справедливость разложения S_{r-k} следует из того, что $r-k$ пробегает содержащиеся среди чисел $L+1, \ldots, r-1$ целые значения от $r-1$ до $r-L'$, поскольку по предположению $r \geqslant L + L' + 1$. С другой стороны,

$$S_r \neq -\sum_{i=1}^{L} c_i S_{r-i} = \sum_{i=1}^{L} c_i \sum_{k=1}^{L'} b_k S_{r-i-k},$$

где справедливость разложения S_{r-i} следует из того, что $r-i$ пробегает содержащиеся среди чисел $L'+1, \ldots, r-1$ целые значения от $r-1$ до $r-L$, а это опять следует из предположения $r \geqslant L + L' + 1$. Изменение порядка суммирования в пра-

вой части последнего равенства приводит к выражению для S_r, полученному в правой части предыдущего равенства. Таким образом, получаем доказывающее лемму противоречие $S_r \neq S_r$. □

Если удастся построить регистр сдвига с длиной, удовлетворяющей соотношению из леммы 7.4.2 с заменой знака нестрогого неравенства знаком равенства, то тем самым будет построен регистр минимальной длины. Следующая лемма показывает, что такое построение дается теоремой 7.4.1.

Лемма 7.4.3. *Предположим, что* $(L_i, \Lambda^i(x))$, $i = 1, \ldots, r$, *является последовательностью регистров сдвига минимальной длины с линейной обратной связью, таких, что* $\Lambda^{(i)}(x)$ *генерирует* $S_1, \ldots, S_i$. *Если* $\Lambda^{(r)}(x) \neq \Lambda^{(r-1)}(x)$, *то*

$$L_r = \max [L_{r-1},\ r - L_{r-1}]$$

и любой регистр сдвига, генерирующий $S_1, \ldots, S_r$ *и имеющий длину, совпадающую с величиной правой части последнего равенства, является регистром сдвига минимальной длины. Такой регистр сдвига определяется теоремой* 7.4.1.

Доказательство. Согласно лемме 7.4.2, L_r не может быть меньше величины в правой части последнего равенства. Если удастся построить какой-либо регистр сдвига, который генерирует требуемую последовательность и длина которого совпадает с указанной величиной, то он будет регистром сдвига минимальной длины. Доказательство будет проводиться по индукции. Мы построим регистр сдвига, удовлетворяющий теореме, в предположении, что такие регистры последовательно построены для всех $k \leqslant r - 1$. Для каждого k, $k = 1, \ldots, r - 1$, будем обозначать через $(L_k, \Lambda^k(x))$ регистр сдвига минимальной длины, генерирующий $S_1, \ldots, S_k$. Примем за предположение индукции, что

$$L_k = \max [L_{k-1},\ k - L_{k-1}], \quad k = 1, \ldots, n-1,$$

каждый раз, когда $\Lambda^{(k)}(x) \neq \Lambda^{(k-1)}(x)$. Это верно для $k = 0$, поскольку $L_0 = 0$ и $L_1 = 1$. Значение k в последней итерации, приведшей к изменению длины, будем обозначать через m. Последнее означает, что при завершении $(r - 1)$-й итерации m является целым числом, таким, что

$$L_{r-1} = L_m > L_{m-1}.$$

Теперь имеет место следующее равенство:

$$S_j + \sum_{i=1}^{L_{r-1}} \Lambda_i^{(r-1)} S_{j-i} = \sum_{i=0}^{L_{r-1}} \Lambda_i^{(r-1)} S_{j-i} = \begin{cases} 0, & j = L_{r-1}, \ldots, r-1, \\ \Delta_r, & j = r. \end{cases}$$

Если $\Delta_r = 0$, то регистр $(L_{r-1}, \Lambda^{(r-1)}(x))$ также генерирует первые r символов, и, таким образом,

$$L_r = L_{r-1} \quad \text{и} \quad \Lambda^{(r)}(x) = \Lambda^{(r-1)}(x).$$

Если $\Delta_r \neq 0$, то необходимо построить новый регистр сдвига. Напомним, что изменение длины регистра произошло при $k = m$. Следовательно,

$$S_j + \sum_{j=1}^{L_{m-1}} \Lambda_i^{(m-1)} S_{j-1} = \begin{cases} 0, & j = L_{m-1}, \ldots, m-1, \\ \Delta_m \neq 0, & j = m, \end{cases}$$

и по предположению индукции

$$L_{r-1} = L_m = \max [L_{m-1}, \quad m - L_{m-1}] = m - L_{m-1},$$

поскольку $L_m > L_{m-1}$. Теперь выберем новый многочлен

$$\Lambda^{(r)}(x) = \Lambda^{(r-1)}(x) \quad \Lambda_r \Delta_m^{-1} x^{r-m} \Lambda^{(m-1)}(x)$$

и положим $L_r = \deg \Lambda^{(r)}(x)$. В этом случае поскольку $\deg \Lambda^{(r-1)}(x) \leqslant L_{r-1}$ и $\deg [x^{r-m} \Lambda^{(m-1)}(x)] \leqslant r - m + L_{m-1}$, то

$$L_r \leqslant \max [L_{r-1}, \quad r - m + L_{m-1}] \leqslant \max [L_{r-1}, \ r - L_{r-1}].$$

Из последнего неравенства и леммы 7.4.2 при условии, что $\Lambda^{(r)}(x)$ генерирует $S_1, \ldots, S_r$, следует, что $L_r = \max [L_{r-1}, \ r - L_{r-1}]$. Осталось доказать, что регистр сдвига $(L_r, \Lambda^{(r)}(x))$ генерирует требуемую последовательность. Докажем это непосредственным вычислением разности между S_j и сигналом на выходе обратной связи регистра сдвига:

$$S_j - \left(- \sum_{i=1}^{L_r} \Lambda_i^{(r)} S_{j-i} \right) = S_j + \sum_{i=1}^{L_{r-1}} \Lambda_i^{(r-1)} S_{j-i} -$$

$$- \Delta_r \Delta_m^{-1} \left[S_{j-r+m} + \sum_{i=1}^{L_{m-1}} \Lambda_i^{(m-1)} S_{j-r+m-i} \right] =$$

$$= \begin{cases} 0, & j = L_r, L_r + 1, \ldots, r-1, \\ \Delta_r - \Delta_r \Delta_m^{-1} \Delta_m = 0, & i = r. \end{cases}$$

Итак, регистр $(L_r, \Lambda^{(r)}(x))$ генерирует $S_1, \ldots, S_r$; в частности, $(L_{2t}, \Lambda^{(2t)}(x))$ генерирует $S_1, \ldots, S_{2t}$. Лемма доказана. □

7.5. БЫСТРОЕ ДЕКОДИРОВАНИЕ КОДОВ БЧХ

Уяснение работы декодера Питерсона—Горенстейна—Цирлера — лучший путь к пониманию процесса декодирования кодов БЧХ. Но при построении декодера приходится жертвовать концептуальной ясностью во имя вычислительной эффективности. Опи-

санный в § 7.2 декодер Питерсона—Горенстейна—Цирлера предполагает обращение двух матриц размера $t \times t$. Хотя обращение матрицы в конечном поле не приводит к ошибкам округления, вычислительная работа, особенно для больших значений t, может оказаться чрезмерно большой. В то же время обращения обеих матриц можно избежать. Обращение первой матрицы, необходимое для вычисления многочлена локаторов ошибок, можно обойти, используя алгоритм Берлекэмпа—Месси. Обращение второй матрицы, необходимое для вычисления значений ошибок, можно обойти, воспользовавшись процедурой, носящей название алгоритма Форни. Этот параграф начнется с описания алгоритма Форни, а затем будет продолжено рассмотрение алгоритма Берлекэмпа—Месси.

Для описания алгоритма Форни нам понадобится многочлен локаторов ошибок

$$\Lambda(x) = \Lambda_v x^v + \Lambda_{v-1} x^{v-1} + \ldots + \Lambda_1 x + 1,$$

имеющий корни X_l^{-1}, $l = 1, \ldots, v$:

$$\Lambda(x) = \prod_{l=1}^{v} (1 - xX_l).$$

Определим синдромный многочлен

$$S(x) = \sum_{j=1}^{2t} S_j x^j = \sum_{j=1}^{2t} \sum_{l=1}^{v} Y_l X_l^j x^j$$

и многочлен значений ошибок $\Omega(x)$:

$$\Omega(x) = S(x)\Lambda(x) \pmod{x^{2t}}.$$

Многочлен значений ошибок будет иногда использоваться в дальнейших рассуждениях. Его связь с локаторами и значениями ошибок устанавливается следующей теоремой.

Теорема 7.5.1. *Многочлен значений ошибок можно записать в следующем виде:*

$$\Omega(x) = \sum_{i=1}^{v} Y_i X_i \prod_{l \neq i} (1 - X_l x).$$

Доказательство. Из определения сомножителей, входящих в равенство для $\Omega(x)$, получаем

$$\Omega(x) = \left[\sum_{j=1}^{2t} \sum_{l=1}^{v} Y_l X_l^j x^{j-1}\right] \left[\prod_{i=1}^{v} (1 - X_i x)\right] \pmod{x^{2t}} =$$

$$= \sum_{i=1}^{v} Y_i X_i \left[(1 - X_i x) \sum_{j=1}^{2t} (X_i x)^{j-1}\right] \prod_{l \neq i} (1 - X_l x) \pmod{x^{2t}}.$$

Выражение в квадратных скобках является разложением для $(1 - X_i^{2t}x^{2t})$. Таким образом,

$$\Omega(x) = \sum_{i=1}^{v} Y_i X_i (1 - X_i^{2t}x^{2t}) \prod_{l \neq i} (1 - X_l x) \pmod{x^{2t}}.$$

Последнее выражение, взятое по модулю x^{2t}, совпадает с выражением, которое требуется доказать. Тем самым теорема доказана. □

Теперь все готово для того, чтобы выписать выражение для значений ошибок, которое намного проще аналогичного выражения, использующего обращение матрицы.

Теорема 7.5.2 (алгоритм Форни). *Значения ошибок получаются из равенства*

$$Y_l = \frac{X_l^{-1}\,\Omega(X_l^{-1})}{\prod_{j \neq l}(1 - X_j X_l^{-1})} = -\frac{\Omega(X_l^{-1})}{\Lambda'(X_l^{-1})}.$$

Доказательство. Используя равенство из утверждения теоремы 7.5.1 при $x = X_l^{-1}$, получим

$$\Omega(X_l^{-1}) = Y_l X_l \prod_{j \neq l}(1 - X_j X_l^{-1}),$$

откуда вытекает первая часть теоремы.

С другой стороны, производная многочлена $\Lambda(x)$ равна

$$\Lambda'(x) = -\sum_{i=1}^{v} X_i \prod_{j \neq i}(1 - xX_j),$$

и, таким образом,

$$\Lambda'(X_l^{-1}) = -X_l \prod_{j \neq l}(1 - X_j X_l^{-1}),$$

откуда сразу следует утверждение теоремы. □

Алгоритм Форни обладает существенным преимуществом перед обращением матрицы, но использует деление. В гл. 9 будет предложено другое решение, не использующее деление.

Теперь вернемся к вычислению многочлена локаторов ошибок, использующему алгоритм Берлекэмпа—Месси теоремы 7.4.1. Предложенный Месси вариант решения этой задачи показан на рис. 7.4, *а*. По заданным S_j, $j = 1, \ldots, 2t$, находится вектор Λ минимальной длины, удовлетворяющий t уравнениям

$$S_j + \sum_{k=1}^{t} S_{j-k}\Lambda_k = 0, \quad j = t+1, \ldots, 2t.$$

Это означает, что требуется по заданным $2t$ компонентам вектора $\mathbf{S}$ из свертки вычислить вектор Λ, если априори известно,

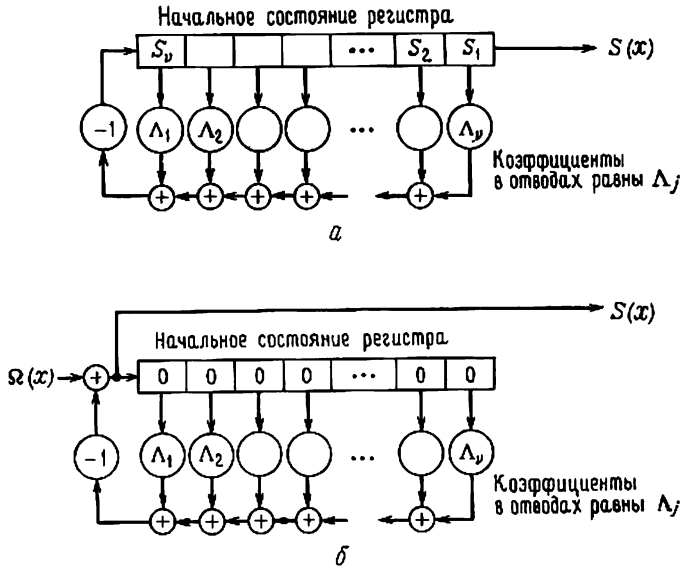


Рис. 7.4. Генерация спектра ошибок. *а* — вариант Месси; *б* — вариант Берлекэмпа.

что $\Lambda_j = 0$ при $j > t$. Удовлетворяющий этому равенству вектор Λ определяет коэффициенты многочлена локаторов ошибок

$$\Lambda(x) = \prod_{l=1}^{\nu} (1 - xX_l),$$

где X_l, $l = 1, \ldots, \nu$, являются локаторами ошибок. Многочлен значений ошибок не вычисляется с помощью этого алгоритма, но получается позже из $\Lambda(x)$ и $S(x)$ согласно определению $\Omega(x) = S(x)\Lambda(x) \pmod{x^{2t}}$.

Вариант Берлекэмпа решения этой задачи показан на рис. 7.4,*б*, из которого видно, что многочлену значений ошибок отводится центральная роль. Такой вариант предполагает поиск векторов Λ и Ω, компоненты которых равны нулю при $t < k \leqslant n$ и удовлетворяют $2t$ уравнениям

$$S_j + \sum_{k=1}^{t} S_{j-k}\Lambda_k = \Omega_j, \quad j = 1, \ldots, 2t,$$

где $S_j = 0$ при $j \leqslant 0$. Заметим, что j пробегает $2t$ значений. Решение задачи дается двумя многочленами: многочленом локаторов ошибок и многочленом значений ошибок.

Два описанных выше варианта эквивалентны. В дальнейшем будет рассмотрен вариант, предложенный Месси. При необходи-

мости алгоритм Берлекэмпа—Месси можно преобразовать в форму Берлекэмпа, производя итерации непосредственно для многочленов $\Lambda^{(r)}(x)$ и $\Omega^{(r)}(x)$.

На рис. 7.5 приведена блок-схема алгоритма Берлекэмпа—Месси. Как было показано, этот алгоритм позволяет вычислять многочлен локаторов ошибок исходя из заданных $2t$ компонент

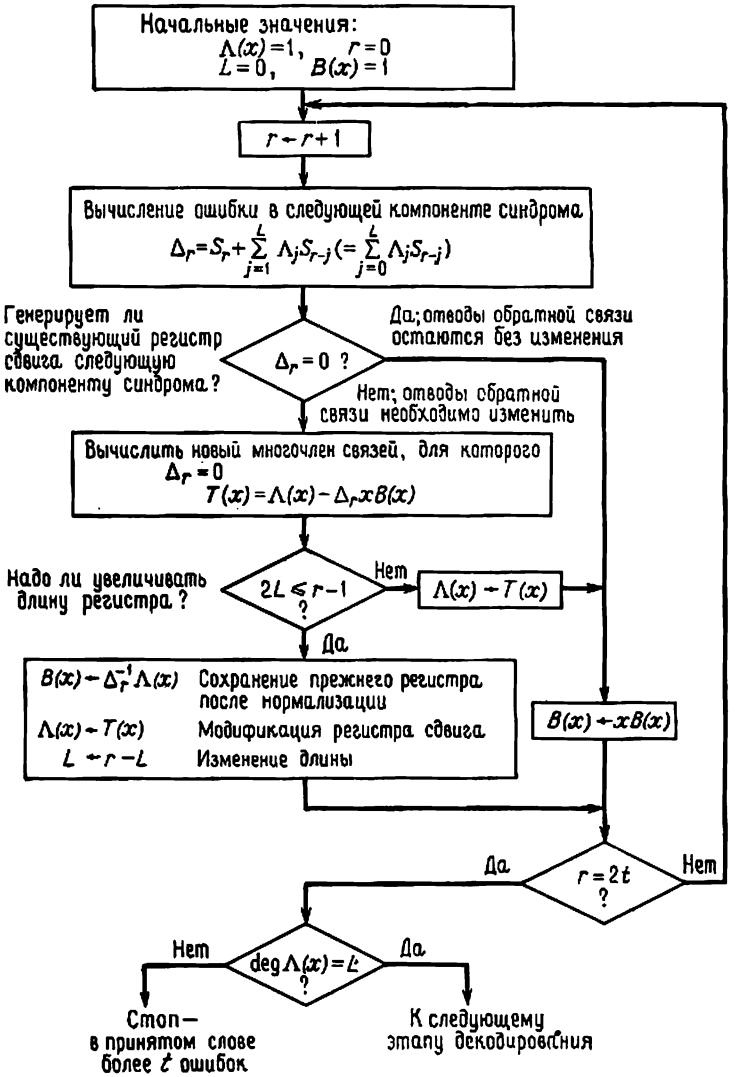


Рис. 7.5. Алгоритм Берлекэмпа—Месси.

синдрома $S_1, \ldots, S_{2t}$. Если j_0 в коде отлично от 1, то определим $S_j = V_{j+j_0-1}$, $j = 1, \ldots, 2t$. Эти компоненты синдрома участвуют в вычислениях точно так же, как и прежние компоненты. Сам алгоритм при этом не претерпевает никаких изменений, и дело сводится лишь к соответствующему изменению индексов внутренних переменных в алгоритме.

Этот алгоритм и его обоснование будут понятнее, если во всех деталях проследить работу алгоритма по схеме на рис. 7.5 для конкретных примеров. В табл. 7.3 приведены необходимые вычисления для (15,9)-кода Рида—Соломона, исправляющего тройные ошибки. В достоверности приведенных вычислений можно убедиться, проделав шесть итераций по изображенной на рис. 7.5 схеме. Табл. 7.4 содержит аналогичные вычисления для исправляющего тройные ошибки (15,5)-кода БЧХ.

Внутренняя логика работы алгоритма Берлекэмпа—Месси может показаться несколько загадочной. Возможно, ответы на

Таблица 7.3

Алгоритм Берлекэмпа—Месси для (15, 9)-кода Рида—Соломона, исправляющего тройные ошибки

$$g(x) = (x+\alpha)(x+\alpha^2)(x+\alpha^3)(x+\alpha^4)(x+\alpha^5)(x+\alpha^6)$$
$$= x^6 + \alpha^{10}x^5 + \alpha^{14}x^4 + \alpha^4 x^3 + \alpha^6 x^2 + \alpha^9 x + \alpha^6$$

$$i(x) = 0$$

$$c(x) = \alpha x^7 + \alpha^5 x^5 + \alpha^{11}x^2 = e(x)$$

$$S_1 = \alpha\alpha^7 + \alpha^5\alpha^5 + \alpha^{11}\alpha^2 = \alpha^{12}$$
$$S_2 = \alpha\alpha^{14} + \alpha^5\alpha^{10} + \alpha^{11}\alpha^4 = 1$$
$$S_3 = \alpha\alpha^{21} + \alpha^5\alpha^{15} + \alpha^{11}\alpha^6 = \alpha^{14}$$
$$S_4 = \alpha\alpha^{28} + \alpha^5\alpha^{20} + \alpha^{11}\alpha^8 = \alpha^{13}$$
$$S_5 = \alpha\alpha^{35} + \alpha^5\alpha^{25} + \alpha^{11}\alpha^{10} = 1$$
$$S_6 = \alpha\alpha^{42} + \alpha^5\alpha^{30} + \alpha^{11}\alpha^{12} = \alpha^{11}$$

r	Δ_r	$T(x)$	$B(x)$	$\Lambda(x)$	L
0			1	1	0
1	α^{12}	$1+\alpha^{12}x$	α^3	$1+\alpha^{12}x$	1
2	α^7	$1+\alpha^3 x$	$\alpha^3 x$	$1+\alpha^3 x$	1
3	1	$1+\alpha^3 x+\alpha^3 x^2$	$1+\alpha^3 x$	$1+\alpha^3 x+\alpha^3 x^2$	2
4	1	$1+\alpha^{14}x$	$x+\alpha^5 x^2$	$1+\alpha^{14}x$	2
5	α^{11}	$1+\alpha^{14}x+\alpha^{11}x^2+\alpha^{14}x^3$	$\alpha^4+\alpha^3 x$	$1+\alpha^{14}x+\alpha^{11}x^2+\alpha^{14}x^3$	3
6	0	$1+\alpha^{14}x+\alpha^{11}x^2+\alpha^{14}x^3$	$\alpha^4 x+\alpha^3 x^2$	$1+\alpha^{14}x+\alpha^{11}x^2+\alpha^{14}x^3$	3

$$\Lambda(x) = 1+\alpha^{14}x+\alpha^{11}x^2+\alpha^{14}x^3 = (1+\alpha^7 x)(1+\alpha^5 x)(1+\alpha^2 x)$$

Таблица 7.4

Алгоритм Берлекэмпа—Месси для (15, 5)-кода БЧХ, исправляющего тройные ошибки

$g(x) = x^{10} + x^8 + x^5 + x^4 + x^2 + x + 1$

$i(x) = 0$

$v(x) = x^7 + x^5 + x^2 = e(x)$

$S_1 = \alpha^7 + \alpha^5 + \alpha^2 = \alpha^{14}$

$S_2 = \alpha^{14} + \alpha^{10} + \alpha^4 = \alpha^{13}$

$S_3 = \alpha^{21} + \alpha^{15} + \alpha^6 = 1$

$S_4 = \alpha^{28} + \alpha^{20} + \alpha^8 = \alpha^{11}$

$S_5 = \alpha^{35} + \alpha^{25} + \alpha^{10} = \alpha^5$

$S_6 = \alpha^{42} + \alpha^{30} + \alpha^{12} = 1$

r	Δ_r	$T(x)$	$B(x)$	$\Lambda(x)$	L
0		1	1	1	0
1	α^{14}	$1+\alpha^{14}x$	α	$1+\alpha^{14}x$	1
2	0	$1+\alpha^{14}x$	αx	$1+\alpha^{14}x$	1
3	α^{11}	$1+\alpha^{14}x+\alpha^{12}x^2$	$\alpha^4+\alpha^3x$	$1+\alpha^{14}x+\alpha^{12}x^2$	2
4	0	$1+\alpha^{14}x+\alpha^{12}x^2$	$\alpha^4x+\alpha^3x^2$	$1+\alpha^{14}x+\alpha^{12}x^2$	2
5	α^{11}	$1+\alpha^{14}x+\alpha^{11}x^2+\alpha^{14}x^3$	$\alpha^4+\alpha^3x+\alpha x^2$	$1+\alpha^{14}x+\alpha^{11}x^2+\alpha^{14}x^3$	3
6	0	$1+\alpha^{14}x+\alpha^{11}x^2+\alpha^{14}x^3$	$\alpha^4x+\alpha^3x^2+\alpha x^3$	$1+\alpha^{14}x+\alpha^{11}x^2+\alpha^{14}x^3$	3

$\Lambda(x) = 1 + \alpha^{14}x + \alpha^{11}x^2 + \alpha^{14}x^3 = (1 + \alpha^7 x)(1 + \alpha^5 x)(1 + \alpha^2 x)$

возникшие вопросы помогут найти следующие соображения. На некоторых итерациях, скажем на r-й итерации, наравне с выбранным по алгоритму Берлекэмпа—Месси регистром с линейной обратной связью могут существовать другие регистры минимальной длины с линейной обратной связью, генерирующие требуемые символы. Все эти регистры будут генерировать одну и ту же требуемую последовательность r компонент синдрома; в то же время последующие компоненты, не входящие в r-ю итерацию, у них будут различны. Каждый раз, когда это возможно, на следующей итерации выбирается регистр сдвига с линейной обратной связью с прежней длиной, генерирующий следующую требуемую компоненту синдрома; если такого регистра не оказывается, то длину регистра приходится увеличивать. Однако при решении вопроса о необходимости увеличения длины регистра никак не учитывается значение следующей компоненты синдрома (проверяется лишь выполнение соотношения $\Delta_{r+1} \neq 0$).

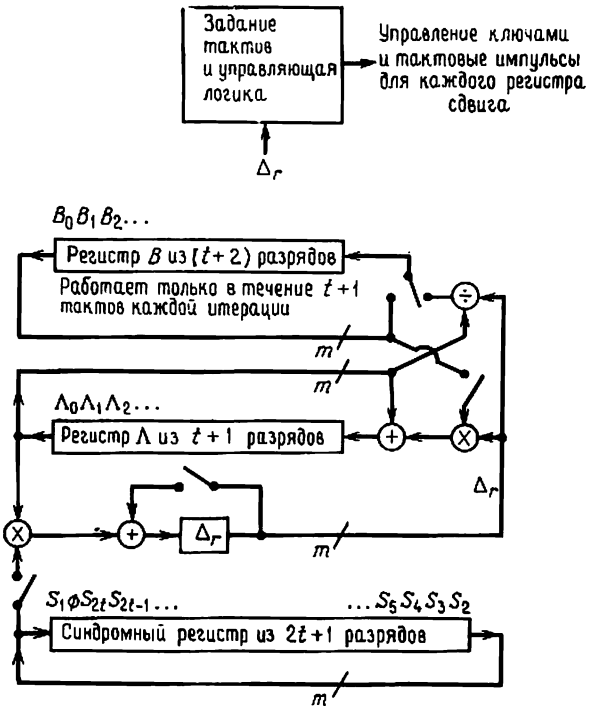


Рис. 7.6. Схема аппаратурной реализации алгоритма Берлекэмпа—Месси. *Замечания.* Требуется $2t$ итераций из $2(t+1)$ тактов каждая. Все данные передаются m параллельными битами.

Таким образом, на $(r+1)$-шаге имеется по меньшей мере столько же возможных вариантов регистра сдвига, сколько значений может принимать S_{r+1}.

Описанный алгоритм может быть реализован в виде программы для универсальных или специализированных ЭВМ, предназначенных для вычислений в полях Галуа. Если требуется очень большая скорость декодирования, то можно воспользоваться специальным жестким исполнением декодера, возможно включающим регистры сдвига.

Схема аппаратурной реализации построенного на регистрах сдвига устройства приведена на рис. 7.6. Три регистра, изображенные на рисунке, используются для хранения значений коэффициентов многочленов $S(x)$, $\Lambda(x)$ и $B(x)$; длина каждого регистра должна быть не меньше той, которая необходима для хранения соответствующего многочлена наибольшей степени, а иногда выбирается немного большей. Если регистры содержат многочлены со степенями меньше максимальных, то неиспользо-

ванные разряды заполняются нулями. Регистры $S(x)$ и $B(x)$ имеют на один разряд больше, чем необходимо для хранения многочленов максимальной степени; отметим наличие дополнительного разряда в регистре $S(x)$.

Такой выбор длин регистров вместе с выбором числа тактов работы каждого из регистров в течение одной итерации приводит к тому, что в результате каждой итерации коэффициенты многочленов сдвигаются от своего первоначального положения на одну позицию. Это обеспечивает умножение $B(x)$ на x и такую переиндексацию компонент S_j, чтобы получились компоненты S_{r-j}, входящие в выражение для Δ_r. Чтобы понять это, снова обратимся к рис. 7.6, на котором регистр $S(x)$ показан в своем исходном состоянии. В результате каждой итерации содержимое этого регистра сдвигается на одну позицию вправо, что позволяет в следующей итерации правильно произвести умножение соответствующих коэффициентов многочленов $S(x)$ и $\Lambda(x)$. Регистр Λ в течение одной итерации сдвигается на полную длину дважды: когда вычисляется Δ_r и когда в этот регистр вводится новое значение Λ.

На рис. 7.7 представлена последовательность вычислений, производимых декодером, использующим алгоритм Берлекэмпа—Месси и алгоритм Форни. В случае когда произошло не более t

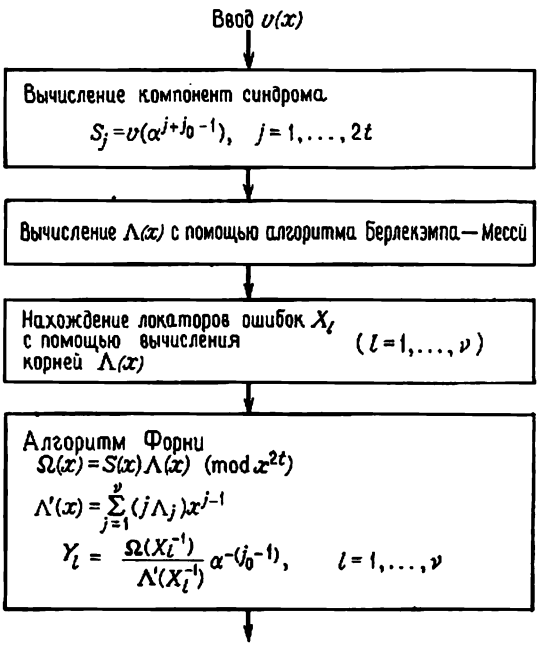


Рис. 7.7. Быстрое декодирование кодов БЧХ.

ошибок, такой декодер выдаст правильное выражение для многочлена локаторов ошибок. В случае когда произойдет более t ошибок, декодирование будет неудачным и получится либо многочлен, не удовлетворяющий условиям, которым должен удовлетворять многочлен локаторов ошибок, либо допустимый, но неправильный многочлен. В первом случае декодер может обнаружить ошибку и пометить сообщение как не поддающееся декодированию. Решение об обнаружении неисправляемой конфигурации ошибок принимается тогда, когда выполняется хотя бы одно из двух условий:

1) число различных лежащих в $GF(q^m)$ корней $\Lambda(x)$ отлично от L;

2) значения ошибок не лежат в поле символов кода.

При декодировании сначала осуществляется проверка многочлена локаторов ошибок. Если он удовлетворяет требуемым условиям, то декодер приступает к вычислению многочлена значений ошибок. Может случиться, что значения ошибок не будут лежать в поле символов кодовых слов, если последнее меньше поля локаторов ошибок. Такой символ не может быть внесен каналом, и, следовательно, его наличие указывает на то, что вектор ошибок содержит более t искаженных позиций.

В § 9.6 будут рассмотрены способы декодирования кодов БЧХ с исправлением некоторых конфигураций более чем t ошибок; здесь же мы обсудим противоположный случай, когда число исправляемых ошибок ограничивается конструктивным расстоянием. На практике такое декодирование используется для значительного уменьшения вероятности ошибки декодирования, платой за которое является увеличение вероятности отказа от декодирования.

Для уменьшения вероятности неправильного декодирования число исправляемых декодером ошибок можно ограничить числом t, таким, что $2t + 1$ будет строго меньше d^* (при четном d^* это выполняется всегда). В этом случае декодер всегда будет обнаруживать неисправляемую конфигурацию ошибок при условии, что число v действительно случившихся ошибок удовлетворяет неравенству

$$t + v < d^*,$$

поскольку для перехода кодового слова в неправильную сферу декодирования должно произойти не менее $d^* - t$ ошибок.

В таком декодере хорошо применять алгоритм Берлекэмпа—Месси. Для вычисления $\Lambda(x)$ необходимо сделать $2t$ итераций по алгоритму Берлекэмпа—Месси. Остальные $\tau - t$ итераций, $\tau = d^* - 1 - t$, требуются для проверки равенства нулю невязок Δ_r. Если хотя бы при одной из этих дополнительных итера-

ций Δ_r не равняется нулю, то принятое слово объявляется словом, в котором произошло более t ошибок.

Правомерность описанной процедуры доказывается следующей теоремой.

Теорема 7.5.3. *Пусть произошло не более* τ *ошибок и заданы* S_j *при* $j = 1, \ldots, t + \tau$ *и* $\tau > t$. *Пусть, далее,* $(L_{2t}, \Lambda^{2t}(x))$ — *регистр сдвига с линейной обратной связью, генерирующий последовательность* S_j, $j = 1, \ldots, 2t$, *и*

$$\Delta_r = \sum_{j=0}^{n-1} \Lambda_j^{(2t)} S_{r-j},$$

причем Δ_r *равняются нулю при* $r = 2t + 1, \ldots, t + \tau$ *и* $L_{2t} \leqslant t$. *Тогда произошло не более* t *ошибок и* $\Lambda^{(2t)}(x)$ *будет являться правильным многочленом локаторов ошибок.*

Доказательство. В условии теоремы заданы компоненты синдрома S_j для $j = 1, \ldots, t + \tau$. Если бы имелись еще $\tau - t$ компонент синдрома S_j, $j = t + \tau + 1, \ldots, 2\tau$, то можно было бы исправлять τ ошибок, т. е. любую конфигурацию предполагаемых ошибок. Вообразим на минуту, что эти дополнительные компоненты сообщены нам добрым волшебником или переданы по неведомому добавочному каналу. Тогда, используя алгоритм Берлекэмпа—Месси, можно найти многочлен локаторов ошибок, степень которого равна числу ошибок. Но при $2t$-й итерации $L_{2t} \leqslant t$, и по предположению Δ_r равняется нулю при $r = 2t + 1, \ldots, t + \tau$. Поэтому значение L не будет меняться до итерации с номером $t + \tau + 1$, и, следовательно, по правилу изменения L

$$L_{2\tau} \geqslant (t + \tau + 1) - L_{2t} \geqslant (t + \tau + 1) - t = \tau + 1,$$

а это противоречит предположению о том, что произошло не более τ ошибок. □

7.6. ДЕКОДИРОВАНИЕ ДВОИЧНЫХ КОДОВ БЧХ

Все, что было сказано в этой главе до настоящего момента, справедливо в любом конечном поле. В случае поля $GF(2)$ проведенные рассуждения можно дополнительно упростить. Очевидное упрощение состоит в том, что при декодировании необходимо вычислять лишь позиции ошибок, а значения ошибок всегда равны единице (хотя декодер может вычислять значения ошибок в качестве дополнительной проверки).

Дальнейшее упрощение декодирования менее очевидно; намек на его возможность содержится в примере, приведенном в табл. 7.4.

Нетрудно заметить, что значение Δ_r всегда обращается в нуль при четных итерациях. Если это происходит всегда при декодировании двоичных кодов, то итерации с четными номерами можно пропускать.

В этом параграфе будет доказано, что дело обстоит действительно так. Доказательство основывается на следующем факте: в поле $GF(2)$ синдромы с четными номерами определяются синдромами с нечетными номерами по формуле

$$S_{2j} = \sum_{j=1}^{n} v(\alpha^{2j}) = \left[\sum_{j=1}^{n} v(\alpha^{j})\right]^2 = S_j^2,$$

что следует из теоремы 5.3.3. Получим для нескольких начальных значений r алгебраические выражения для коэффициентов $\Lambda^{(r)}(x)$. Следуя алгоритму, приведенному на рис. 7.5, и используя справедливые для всех двоичных кодов равенства $S_4 = S_2^2 = S_1^4$, получаем

$$\begin{aligned}
&\Delta_1 = S_1,\\
&\Delta_2 = S_2 + S_1^2 = 0,\\
&\Delta_3 = S_3 + S_1S_2,\\
&\Delta_4 = S_4 + S_1S_3 + S_1^{-1}S_2S_3 + S_2^2 = 0,\\
&\Lambda^{(1)}(x) = S_1x + 1,\\
&\Lambda^{(2)}(x) = S_1x + 1,\\
&\Lambda^{(3)}(x) = (S_1^{-1}S_3 + S_2)x^2 + S_1x + 1.
\end{aligned}$$

Отсюда следует, что для любых двоичных кодов БЧХ Δ_2 и Δ_4 всегда равняются нулю. Бесконечное продолжение этого прямого пути для вычисления других значений синдромов с четными номерами невозможно. Вместо этого сформулируем общий результат, показывающий, что $\Delta_r = 0$ для всех четных r.

Теорема 7.6.1. *Пусть в расширении поля $GF(2)$ задана произвольная последовательность $S_1, S_2, \ldots, S_{2\nu-1}$, удовлетворяющая условию $S_{2j} = S_j^2$ для $2j \leqslant 2\nu - 1$; пусть также задан регистр сдвига с линейной обратной связью $\Lambda(x)$ и выполняются равенства*

$$S_j = \sum_{i=1}^{n-1} \Lambda_i S_{j-i}, \quad j = \nu, \ldots, 2\nu - 1.$$

Если следующий член последовательности задается равенством

$$S_{2\nu} = \sum_{i=1}^{n-1} \Lambda_i S_{2\nu-i},$$

то

$$S_{2\nu} = S_\nu^2.$$

Доказательство. Покажем, что при сделанных предположениях выражения для S_ν^2 и $S_{2\nu}$ совпадают. С одной стороны,

$$S_\nu^2 = \left(\sum_{i=1}^{n-1} \Lambda_i S_{\nu-i}\right)^2 = \sum_{i=1}^{n-1} \Lambda_i^2 S_{\nu-i}^2 = \sum_{i=1}^{n-1} \Lambda_i^2 S_{2\nu-2i}.$$

С другой стороны,

$$S_{2\nu} = \sum_{k=1}^{n-1} \Lambda_k S_{2\nu-k} = \sum_{k=1}^{n-1} \sum_{i=1}^{n-1} \Lambda_k \Lambda_i S_{2\nu-k-i}.$$

Из симметрии последнего выражения следует, что каждое слагаемое с $i \neq k$ входит в сумму дважды. Поскольку вычисления производятся в расширении поля $GF(2)$, сумма этих слагаемых будет равняться нулю. Поэтому вклад в последнюю сумму дадут только диагональные члены, для которых $i = k$:

$$S_{2\nu} = \sum_{i=1}^{n-1} \Lambda_i^2 S_{2\nu-2i}.$$

Это выражение совпадает с полученным в начале доказательства выражением для S_ν^2, что доказывает теорему. □

Таким образом, по индукции получаем, что Δ_r равно нулю для четных r; поэтому можно рассматривать только нечетные итерации, формально объединяя две последовательные итерации:

$$\Lambda^{(r)}(x) = \Lambda^{(r-2)}(x) - \Delta_r x^2 B^{(r-2)}(x),$$

$$B^{(r)}(x) = \delta_r \Delta_r^{-1} \Lambda^{(r-2)}(x) + (1 - \delta_r) x^2 B^{(r-2)}(x).$$

Используя эти формулы, можно пропустить итерации с четными номерами, и в результате декодирование двоичных кодов ускоряется. Заметим, что поскольку при доказательстве теоремы были использованы лишь соотношения сопряженности между компонентами синдрома и ничего не говорилось о двоичных конфигурациях ошибок, то описанное упрощение возможно даже для конфигураций более чем t ошибок. Поэтому те же виды проверки применяются при числе ошибок, большем t.

7.7. ДЕКОДИРОВАНИЕ С ПОМОЩЬЮ АЛГОРИТМА ЕВКЛИДА

Декодирование с помощью алгоритма Евклида представляет собой один из способов декодирования, отличный от рассмотренных ранее. Принцип работы такого декодера несколько проще понять, однако считается, что он менее эффективен в практическом использовании; впрочем, справедливость такого мнения, возможно, в значительной мере зависит от конкретного применения.

В гл. 4 алгоритм Евликда был описан как рекуррентная процедура нахождения наибольшего общего делителя двух многочленов. Небольшое расширение этого алгоритма позволяет дополнительно вычислять многочлены $a(x)$ и $b(x)$, такие, что

$$\text{НОД}\ [s(x),\ t(x)] = a(x)\,s(x) + b(x)\,t(x).$$

Для двух произвольных многочленов $s(x)$ и $t(x)$ повторим алгоритм Евклида в более удобной для нас матричной форме, используя для алгоритма деления запись $s(x) = [s(x)/t(x)]\,t(x) + r(x)$.

Теорема 7.7.1 (алгоритм Евклида для многочленов). *Пусть заданы два многочлена* $s(x)$ и $t(x)$, *такие, что* $\deg s(x) \geqslant \deg t(x)$, *пусть* $s^{(0)}(x) = s(x)$, $t^{(0)}(x) = t(x)$, *и пусть* $\mathbf{A}^{(0)}(x) = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$. *Тогда решением рекуррентных уравнений*

$$Q^{(r)}(x) = \left\lfloor \frac{s^{(r)}(x)}{t^{(r)}(x)} \right\rfloor,$$

$$\mathbf{A}^{(r+1)}(x) = \begin{bmatrix} 0 & 1 \\ 1 & -Q^{(r)}(x) \end{bmatrix} \mathbf{A}^{(r)}(x),$$

$$\begin{bmatrix} s^{(r+1)}(x) \\ t^{(r+1)}(x) \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 1 & -Q^{(r)}(x) \end{bmatrix} \begin{bmatrix} s^{(r)}(x) \\ t^{(r)}(x) \end{bmatrix} = \mathbf{A}^{(r+1)}(x) \begin{bmatrix} s(x) \\ t(x) \end{bmatrix}$$

будет многочлен

$$s^{(R)}(x) = \gamma\ \text{НОД}\ [s(x),\ t(x)],$$

причем найдется такой скаляр γ, *что*

$$\gamma\,\text{НОД}\,[s(x),\ t(x)] = A_{11}^{(R)}(x)\,s(x) + A_{12}^{(R)}(x)\,t(x),$$

где R *таково, что* $t^{(R)}(x) = 0$.

Доказательство. Поскольку $\deg t^{(r+1)}(x) < \deg t^{(r)}(x)$, то в конце концов $t^{(R)}(x) = 0$ при некотором R, и процесс обязательно закончится. Таким образом,

$$\begin{bmatrix} s^{(R)}(x) \\ 0 \end{bmatrix} = \left\{ \prod_{r=R-1}^{0} \begin{bmatrix} 0 & 1 \\ 1 & -Q^{(r)}(x) \end{bmatrix} \right\} \begin{bmatrix} s(x) \\ t(x) \end{bmatrix},$$

откуда следует, что любой делитель многочленов $s(x)$ и $t(x)$ делит также и $s^{(R)}(x)$.

Легко проверить, что

$$\begin{bmatrix} 0 & 1 \\ 1 & -Q^{(r)}(x) \end{bmatrix}^{-1} = \begin{bmatrix} Q^{(r)}(x) & 1 \\ 1 & 0 \end{bmatrix},$$

и поэтому

$$\begin{bmatrix} s(x) \\ t(x) \end{bmatrix} = \left\{ \prod_{r=0}^{R-1} \begin{bmatrix} Q^{(r)}(x) & 1 \\ 1 & 0 \end{bmatrix} \right\} \begin{bmatrix} s^{(R)}(x) \\ 0 \end{bmatrix},$$

так что многочлен $s^{(R)}(x)$ должен делить $s(x)$ и $t(x)$, а следовательно, и НОД $[s(x), t(x)]$. Отсюда получаем, что НОД $[s(x), t(x)]$ делит $s^{(R)}(x)$ и сам делится на $s^{(R)}(x)$; таким образом,

$$s^{(R)}(x) = \gamma \text{ НОД } [s(x), t(x)].$$

Далее

$$\begin{bmatrix} s^{(R)}(x) \\ 0 \end{bmatrix} = \mathbf{A}^{(R)}(x) \begin{bmatrix} s(x) \\ t(x) \end{bmatrix},$$

и, следовательно,

$$s^{(R)}(x) = A_{11}^{(R)}(x)\, s(x) + A_{12}^{(R)}(x)\, t(x).$$

Это доказывает последнее утверждение теоремы. □

В теореме 7.7.1 найден смысл матричных элементов $A_{11}^{(R)}(x)$ и $A_{12}^{(R)}(x)$. Можно интерпретировать и два остальных элемента,) для чего нам потребуется обратить матрицу $\mathbf{A}^{(r)}(x)$. Напомним, что

$$\mathbf{A}^{(r)}(x) = \prod_{l=r-1}^{0} \begin{bmatrix} 0 & 1 \\ 1 & -Q^{(l)}(x) \end{bmatrix}.$$

Из этого равенства видно, что определитель матрицы $\mathbf{A}^{(r)}(x)$ равен $(-1)^r$. Обратная матрица запишется в следующем виде:

$$\begin{bmatrix} A_{11}^{(r)}(x) & A_{12}^{(r)}(x) \\ A_{21}^{(r)}(x) & A_{22}^{(r)}(x) \end{bmatrix}^{-1} = (-1)^r \begin{bmatrix} A_{22}^{(r)}(x) & -A_{12}^{(r)}(x) \\ -A_{21}^{(r)}(x) & A_{11}^{(r)}(x) \end{bmatrix}.$$

Следствие 7.7.2. *Многочлены $A_{21}^{(r)}(x)$ и $A_{22}^{(r)}(x)$, полученные с помощью алгоритма Евклида, удовлетворяют равенствам*

$$s(x) = (-1)^R A_{22}^{(R)}(x)\, \gamma \text{ НОД } [s(x), t(x)],$$
$$t(x) = -(-1)^R A_{21}^{(R)}(x)\, \gamma \text{ НОД } [s(x), t(x)].$$

Доказательство. Используя выписанное выше выражение для обратной к $\mathbf{A}^{(R)}(x)$ матрицы, получаем

$$\begin{bmatrix} s(x) \\ t(x) \end{bmatrix} = (-1)^R \begin{bmatrix} A_{22}^{(R)}(x) & -A_{12}^{(R)}(x) \\ -A_{21}^{(R)}(x) & A_{11}^{(R)}(x) \end{bmatrix} \begin{bmatrix} s^{(R)}(x) \\ 0 \end{bmatrix},$$

откуда и вытекает утверждение следствия. □

Опишем два совершенно различных способа использования алгоритма Евклида при декодировании; один из них будет приведен ниже, а другой — в § 9.1.

Напомним, что синдромный многочлен, многочлен локаторов ошибок и многочлен значений ошибок связаны соотношением

$$\Omega(x) = S(x)\,\Lambda(x) \pmod{x^{2t}}$$

и условиями $\deg \Lambda(x) \leqslant t$ и $\deg \Omega(x) \leqslant t-1$. Постараемся теперь использовать доказательство алгоритма Евклида для решения этого уравнения относительно $\Lambda(x)$ и $\Omega(x)$. Из этого доказательства легко установить, что

$$\begin{bmatrix} s^{(r)}(x) \\ t^{(r)}(x) \end{bmatrix} = \begin{bmatrix} A_{11}^{(r)}(x) & A_{12}^{(r)}(x) \\ A_{21}^{(r)}(x) & A_{22}^{(r)}(x) \end{bmatrix} \begin{bmatrix} s(x) \\ t(x) \end{bmatrix},$$

и поэтому

$$t^{(r)}(x) = t(x)\,A_{22}^{(r)}(x) \pmod{s(x)}.$$

Если положить $t(x) = S(x)$ и $s(x) = x^{2t}$, то последнее равенство можно рассматривать как уравнение, которое нам необходимо решить. Это уравнение выполняется для каждого r. Чтобы решить поставленную задачу, необходимо найти такое значение r (если оно существует), при котором $\deg A_{22}^{(r)}(x) \leqslant t$ и $\deg t^{(r)}(x) \leqslant t-1$. Удовлетворить последним условиям можно, выбрав r' равным такому значению r, при котором

$$\deg t^{(r-1)}(x) \geqslant t \quad \text{и} \quad \deg t^{(r)}(x) \leqslant t-1.$$

Поскольку $\deg t^{(0)}(x) = 2t$ и степень $t^{(r)}(x)$ строго убывает с возрастанием r, эти неравенства определяют единственное значение r'.

По определению r' получаем

$$\deg t^{(r')}(x) \leqslant t-1.$$

Степень $A_{22}^{(r)}(x)$ возрастает с возрастанием r. Остается лишь показать, что

$$\deg A_{22}^{(r')}(x) \leqslant t.$$

Это неравенство доказывается с помощью обращения матрицы $\mathbf{A}(x)$. Сначала напомним, что

$$\mathbf{A}^{(r')}(x) = \prod_{r=r'-1}^{0} \begin{bmatrix} 0 & 1 \\ 1 & -Q^{(r)}(x) \end{bmatrix},$$

откуда следует что $\deg A_{22}^{(r)}(x) \geqslant \deg A_{12}^{(r)}(x)$. Напомним также, что $\deg s^{(r)}(x) > \deg t^{(r)}(x)$. Из этих неравенств и матричного равенства

$$\begin{bmatrix} s(x) \\ t(x) \end{bmatrix} = (-1)^{r'} \begin{bmatrix} A_{22}^{(r)}(x) & -A_{12}^{(r)}(x) \\ -A_{21}^{(r)}(x) & A_{11}^{(r)}(x) \end{bmatrix} \begin{bmatrix} s^{(r)}(x) \\ t^{(r)}(x) \end{bmatrix}$$

вытекает, что $\deg s(x) = \deg A_{22}^{(r')}(x) + \deg s^{(r')}(x)$, и, поскольку $s^{(r')}(x) = t^{(r'-1)}(x)$, получаем

$$\deg A_{22}^{(r')}(x) = \deg s(x) - \deg t^{(r'-1)}(x) \leqslant 2t - t = t,$$

где неравенство следует из определения r'.

Теперь мы почти полностью доказали следующую теорему.

Теорема 7.7.3. *Пусть заданы* $s^0(x) = x^{2t}$, $t^{(0)}(x) = S(x)$, *где* $S(x)$ *— синдромный многочлен кода БЧХ, исправляющего* t *ошибок, и пусть* $\mathbf{A}^{(0)}(x) = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$. *Будем решать следующие рекуррентные уравнения до тех пор, пока не будет выполнено неравенство* $\deg t^{(r')} \leqslant t - 1$:

$$Q^{(r)}(x) = \left[\frac{s^{(r)}(x)}{t^{(r)}(x)} \right],$$

$$\mathbf{A}^{(r+1)}(x) = \begin{bmatrix} 0 & 1 \\ 1 & -Q^{(r)}(x) \end{bmatrix} \mathbf{A}^{(r)}(x),$$

$$\begin{bmatrix} s^{(r+1)}(x) \\ t^{(r+1)}(x) \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 1 & -Q^{(r)}(x) \end{bmatrix} \begin{bmatrix} s^{(r)}(x) \\ t^{(r)}(x) \end{bmatrix}.$$

Тогда многочлены

$$\Omega(x) = \Delta^{-1} t^{(r')}(x),$$

$$\Lambda(x) = \Delta^{-1} A_{22}^{(r')}(x),$$

где $\Delta = A_{22}^{(r')}(0)$, *являются единственным решением уравнения*

$$\Omega(x) = S(x)\Lambda(x) \pmod{x^{2t}},$$

удовлетворяющим условиям $\deg \Lambda(x) \leqslant t$, $\Lambda_0 = 1$ и $\deg \Omega(x) \leqslant t - 1$.

Доказательство. Деление на Δ обеспечивает равенство $\Lambda_0 = 1$. С другой стороны, из предыдущих рассуждений следует, что последнее уравнение и все условия удовлетворяются. Единственность полученного решения вытекает из того факта, что для синдрома кода БЧХ, исправляющего t ошибок, существует только одно такое решение. □

После того как $\Lambda(x)$ и $\Omega(x)$ найдены, декодирование можно завершить любым из методов, предложенных для алгоритма Берлекэмпа—Месси. Блок-схема соответствующего декодера представлена на рис. 7.8, где декодер завершается алгоритмом Форни (хотя его можно завершить и иначе).

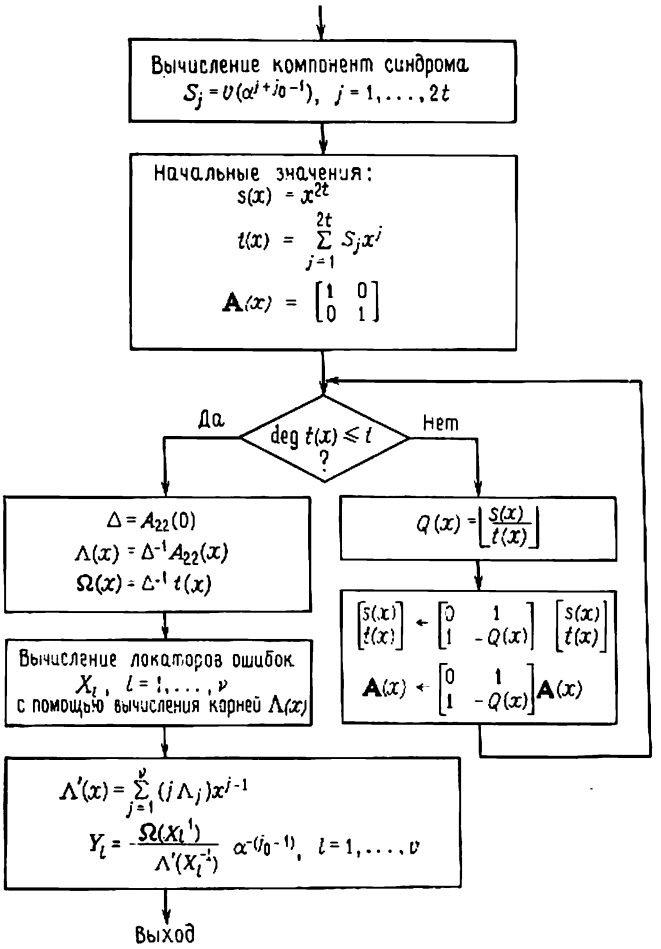


Рис. 7.8. Декодирование кодов БЧХ при помощи алгоритма Евклида.

7.8. КАСКАДНЫЕ (ГНЕЗДОВЫЕ [1]) КОДЫ

Одним из путей построения блоковых кодов с большими длинами является каскадирование кодов. Этот прием состоит в сочетании кода с символами из малого алфавита с кодом с символами из

[1]) Гнездовые (nested) коды называются также каскадными (concatenated) кодами, поскольку кодеры и декодеры для них группируются в каскады. Мы предпочитаем резервировать термин «каскадный» за каскадными кодовыми словами, которые выписываются одно за другим, образуя единую цепочку. [Поскольку в русскоязычной литературе по теории корректирующих кодов термин «гнездовые коды» вообще не применяется, а термин «каскадные коды» является общепринятым, мы будем везде пользоваться последним. — *Перев.*]

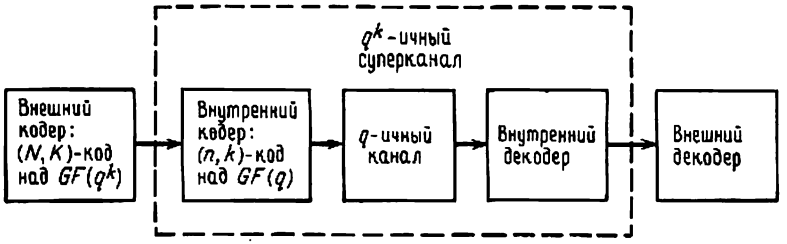


Рис. 7.9. Каскадное кодирование.

алфавита с бóльшим объемом. Представим себе последовательность q-ичных символов длины kK. Эту последовательность можно разбить на K блоков по k символов и рассматривать каждый такой блок как элемент q^k-ичного алфавита. Последовательность из K таких элементов кодируется (N, K)-кодом, заданным над полем $GF(q^k)$. Затем каждый из N получившихся q-^{k}ичных символов представляется в виде последовательности из k q-ичных символов и кодируется q-ичным (n, k)-кодом. Таким образом, каскадный код имеет два различных уровня кодирования.

Итак, суть каскадных кодов состоит в следующем. Пусть при передаче по q-ичному каналу в качестве внутреннего используется (n, k)-код. Этот код, заданный над $GF(q)$, содержит q^k слов. На вход внутреннего кодера подается последовательность k символов, которые могут рассматриваться как один элемент поля $GF(q^k)$. Этот q^k-ичный символ поступает на внутренний кодер, а затем выходит из внутреннего декодера (возможно, с искажениями). Таким образом, систему из внутреннего кодера, канала и внутреннего декодера можно рассматривать как некий суперканал с большим входным/выходным алфавитом из поля $GF(q^k)$. Для передачи сообщений по такому суперканалу могут быть построены суперкоды. На рис. 7.9 для передачи по суперканалу сообщения кодируются (N, K)-кодом над полем $GF(q^k)$.

Примеры. Пусть внутренним кодом является исправляющий 2 ошибки (7, 3)-код Рида—Соломона над $GF(8)$, а внешним — исправляющий 3 ошибки (511, 505)-код Рида—Соломона над $GF(8^3)$.

Полученный из этих кодов каскадный код над $GF(8)$ имеет параметры (3577, 1515) и исправляет любую конфигурацию 11 ошибок, а также очень большое число конфигураций с бóльшим числом ошибок. На рис. 7.10 наглядно представлено строение кодового слова. Заметим, что в действительности кодовое слово является вектором, состоящим из 3577 восьмеричных символов. На рисунке это слово изображено в виде двумерного множества только для того, чтобы продемонстрировать способ построения слова каскадного кода.

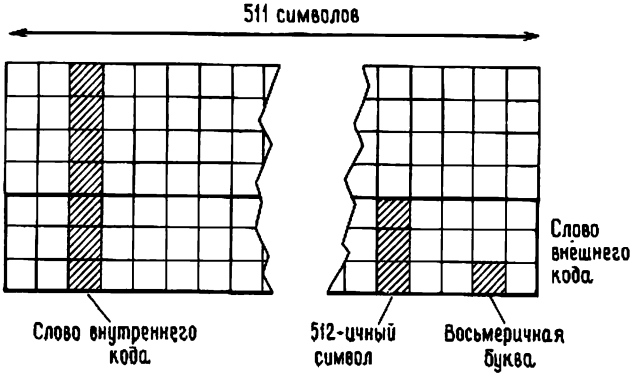


Рис. 7.10. Каскадный код, построенный из (511, 505)- и (7, 3)-кодов Рида—Соломона.

Другой пример кодового слова каскадного кода представлен на рис. 7.11. Этот код используется при передаче по восьмеричному каналу. Он получается из (7, 4)-кода Рида—Соломона над GF (8) и укороченного (22, 18)-кода Рида—Соломона над GF (4096). Кодовое слово содержит 72 восьмеричных информационных символа и имеет длину, равную 154. Отметим, что все множество кодовых слов после внутреннего кодирования составляет одно кодовое слово каскадного кода.

Принятое сообщение, представленное на рис. 7.11, в результате прохождения по каналу искажено ошибками и стираниями. Приемник объявляет символ стертым, если он не удовлетворяет каким-либо внутренним проверкам на соответствие этого символа правильному. Нестертые символы могут оказаться либо правильными, либо ошибочными. На рис. 7.11 позиции стираний отмечены прочерками, а ошибочные символы подчеркнуты. Ясно, что декодер знает позиции с прочерками, но не знает, какие символы подчеркнуты. Декодирование с исправлением стираний будет рассмотрено в § 9.2.

Этот пример демонстрирует большие возможности каскадного кода, поскольку с его помощью удается успешно декодировать даже столь искаженное сообщение. Заметим, что длинные последовательности стираний могут быть вызваны случайными сбоями

Исходные данные — 72 восьмеричных символа

0 1 2 3 4 5 6 7 6 5 4 3 2 1 0 0 1 2
3 4 5 6 7 6 5 4 3 2 1 0 0 1 2 3 4 5
6 7 6 5 4 3 2 1 0 0 1 2 3 4 5 6 7 6
5 4 3 2 1 0 0 1 2 3 4 5 6 7 6 5 4 3

Кодовое слово внешнего (22,18, 5)-кода Рида—Соломона над *GF* (4096) (каждый столбец представляет один символ, проверочные символы расположены в первых четырех столбцах)

```
0 4 6 5 | 0 1 2 3 4 5 6 7 6 5 4 3 2 1 0 0 1 2
2 4 6 5 | 3 4 5 6 7 6 5 4 3 2 1 0 0 1 2 3 4 5
4 0 0 2 | 6 7 6 5 4 3 2 1 0 0 1 2 3 4 5 6 7 6
2 1 7 2 | 5 4 3 2 1 0 0 1 2 3 4 5 6 7 6 5 4 3
```

Кодовое слово внутреннего кода — 22 кодовых слова (7, 4, 4)-кода Рида—Соломона над *GF* (8) (каждый столбец представляет одно кодовое слово, проверочные символы расположены в первых трех строках)

```
7 7 6 7   4 1 4 5 4 1 0 4 2 5 4 3 6 2 6 4 1 4
4 4 5 6   0 4 7 2 6 5 2 4 2 1 5 4 3 4 6 0 4 7
2 2 5 2   4 6 1 7 1 1 2 5 3 0 7 3 3 2 2 4 6 1
---------+-----------------------------------
0 4 6 5 | 0 1 2 3 4 5 6 7 6 5 4 3 2 1 0 0 1 2
2 4 6 5 | 3 4 5 6 7 6 5 4 3 2 1 0 0 1 2 3 4 5
4 0 0 2 | 6 7 6 5 4 3 2 1 0 0 1 2 3 4 5 6 7 6
2 1 7 2 | 5 4 3 2 1 0 0 1 2 3 4 5 6 7 6 5 4 3
```

Принятое сообщение с ошибками и стираниями

```
4 7 – 7 4 – 4 5 4 1 – 4 2 1 4 – 6 2 6 4 – 4
4 3 – – 0 2 7 2 – – 2 4 2 4 5 4 3 4 5 0 4 7
2 2 – 2 4 4 1 7 1 1 2 5 3 5 7 3 3 – 2 4 6 1
0 4 – – 1 6 0 – – – – – – – – – – – – – –
– – – – – – – – – – – – – – 1 0 0 1 2 2 4 5
4 0 – 2 6 1 6 5 4 3 2 – – – 4 – 0 – 5 6 – 6
2 1 5 2 5 2 3 – 1 0 0 1 2 6 4 5 6 7 6 5 4 1
```

Сообщение после кодирования внутренним декодером

```
0 4 – 5 0 – 2 3 4 5 6 7 6 7 4 3 2 1 0 0 1 2
2 4 – 5 3 – 5 6 7 6 5 4 3 3 1 0 0 1 2 3 4 5
4 0 – 2 6 – 6 5 4 3 2 1 0 0 1 2 3 4 5 6 7 6
2 1 – 2 5 – 3 2 1 0 0 1 2 6 4 5 6 7 6 5 4 3
```

Сообщение после кодирования внешним декодером

```
0 1 2 3 4 5 6 7 6 5 4 3 2 1 0 0 1 2
3 4 5 6 7 6 5 4 3 2 1 0 0 1 2 3 4 5
6 7 6 5 4 3 2 1 0 0 1 2 3 4 5 6 7 6
5 4 3 2 1 0 0 1 2 3 4 5 6 7 6 5 4 3
```

Рис. 7.11. Восьмеричный каскадный код.

в приемнике или случайной интерференцией при построчной передаче символов. Однако поскольку внутренний кодер формирует столбцы, такой способ передачи требует использования так называемой *перемежающей памяти* (corner-turning memory) между внутренним кодером и каналом. В такой памяти слова внутреннего кода записываются в столбцы до тех пор, пока не будет построено все кодовое слово каскадного кода, которое затем построчно считывается из памяти в канал.

Используя каскадный принцип построения кодов, можно даже сочетать блоковые и сверточные коды. В гл. 12 будет описан простой сверточный код с декодированием по Витерби, хорошо исправляющий отдельные ошибки в длинных последовательностях двоичных символов. В то же время большое число близко расположенных ошибок при декодировании по Витерби приведет к пакету ошибок. Для исправления этих пакетов может использоваться внешний код Рида—Соломона. Известно, что этот метод каскадирования кода Рида—Соломона и слабого, но простого сверточного кода является мощным орудием конструирования декодеров для каналов с гауссовским шумом.

7.9. КОДЫ ЮСТЕСЕНА

Любой код над полем $GF(q^m)$ можно превратить в код над полем $GF(q)$, «развернув» каждый q^m-ичный символ исходного кода в последовательность m q-ичных символов. В результате из линейного (N, K)-кода над $GF(q^m)$ получится линейный (mN, mK)-код над $GF(q)$. В кодовом слове минимального веса d^* ненулевых символов запишутся в виде последовательности из md^* символов, из которых только часть будут ненулевыми. Скорость полученного кода будет такая же, как и у исходного кода, а минимальное расстояние будет составлять много меньшую долю длины.

Это «развертывание» представляет собой простой способ построения кода, исправляющего многократные пакеты ошибок. Если исходный код имел минимальное расстояние, равное 5, то в результате получится код, исправляющий одиночные пакеты q-ичных ошибок длины $m + 1$ и многие конфигурации из двух пакетов меньшей длины. Например, (n, k)-код Рида—Соломона над $GF(2^m)$ развертывается в двоичный (mn, mk)-код, исправляющий многократные пакеты ошибок. Однако, как правило, такой код плохо исправляет случайные ошибки.

Юстесен обнаружил, что если разумным образом дополнить эту конструкцию, то можно получить хорошие коды. В результате получился класс кодов, называемых кодами Юстесена, содержащий, как будет показано ниже, некоторые очень хорошие коды. К сожалению, коды Юстесена хороши только при больших длинах и поэтому не представляют особого практического инте-

реса. Эти коды могут служить примером построения хороших кодов с большой длиной. Алгоритмы декодирования кодов Юстесена еще не достаточно разработаны.

Кодовое слово $(2mN, mK)$-кода Юстесена представляет собой $[2m \times (q^m - 1)]$-таблицу элементов поля $GF(q)$. Построение кода Юстесена начинается с выбора фиксированного (N, K)-кода Рида—Соломона, содержащего столько слов, сколько должно содержаться в конструируемом коде Юстесена. Прежде всего построим начинающуюся с кодового слова $\mathbf{c} = (c_0, c_1, \ldots, c_{N-1})$ кода Рида—Соломона $(2 \times N)$-таблицу над $GF(q^m)$:

$$\begin{bmatrix} c_0 & c_1 & c_2 & \ldots & c_{N-1} \\ \alpha^0 c_0 & \alpha^1 c_1 & \alpha^2 c_2 & \ldots & \alpha^{N-1} c_{N-1} \end{bmatrix}.$$

Каждый элемент этой таблицы лежит в поле $GF(q^m)$. Заменим теперь каждый символ в таблице соответствующим m-мерным вектором-столбцом над $GF(q)$. Получившаяся таблица элементов из $GF(q)$ размера $2m \times N$ будет образовывать одно кодовое слово кода Юстесена. Эту таблицу можно любым удобным образом преобразовать в вектор. После повторения такой процедуры для каждого слова кода Рида—Соломона получится код Юстесена.

Прежде всего заметим, что указанная процедура приводит к линейному коду. Это важно, поскольку в этом случае для определения минимального расстояния кода достаточно найти значение минимального веса в коде. Остановимся теперь на той особенности конструкции Юстесена, благодаря которой она относится к классу хороших кодов. Поскольку слова кода Рида—Соломона имеют вес не менее $N - K + 1$, каждая представляющая кодовое слово таблица содержит не менее $N - K + 1$ ненулевых столбцов. Далее, если в двух столбцах таблицы совпадают m верхних элементов, то в этих столбцах m нижних элементов обязательно будут различными. Таким образом, в таблице нет двух одинаковых ненулевых столбцов. Чтобы оценить минимальный вес кодового слова, допустим, что все $N - K + 1$ ненулевых столбцов различны и имеют минимальный возможный вес. Искомая оценка получится суммированием весов $N - K + 1$ $2m$-мерных векторов минимального веса.

Теорема 7.9.1. *Для минимального расстояния $(2mN, 2mK)$-кода Юстесена, построенного из (N, K)-кода Рида—Соломона, при каждом l, удовлетворяющем неравенству*

$$\sum_{i=1}^{l} (q-1)^i \binom{2m}{i} \leqslant N - K + 1,$$

имеет место оценка $d^* \geqslant \sum_{i=1}^{l} i (q-1)^i \binom{2m}{i}$.

Доказательство. Кодовое слово минимального веса содержит $N-K+1$ различных ненулевых столбцов. Таким образом, вес этого слова не меньше веса слова, которое получится при записи в этих $N-K+1$ столбцах $N-K+1$ различных последовательностей длины $2m$, имеющих минимальный вес. Существует $\binom{2m}{i}$ способов выбора i ненулевых элементов в последовательности длины $2m$, а число таких ненулевых элементов равняется $q-1$. Следовательно, при любом l, удовлетворяющем неравенству

$$\sum_{i=1}^{l}(q-1)^i\binom{2m}{i}\leqslant N-K+1,$$

существует столбец веса l. Минимальное расстояние кода будет не меньше суммы весов

$$d^*\geqslant\sum_{i=1}^{l}i(q-1)^i\binom{2m}{i}. \qquad \square$$

Для того чтобы наилучшим образом использовать результат теоремы 7.9.1, требуется помощь ЭВМ. Но для больших значений N вычисление биномиальных коэффициентов даже на ЭВМ оказывается весьма непростой задачей. Сейчас будет предложен другой подход к исследованию поведения кода при больших длинах. (Последующий текст не связан непосредственно с теорией кодирования и необходим лишь для описания асимптотического поведения кода при больших длинах.)

Лемма 7.9.2. *Для всех p, удовлетворяющих условию $0<p<1/2$, причем pn — целое число, имеет место неравенство*

$$\sum_{k=0}^{pn}\binom{n}{k}\leqslant 2^{nH(p)},$$

где $H(p)=-p\log_2(p)-(1-p)\log_2(1-p)$.

Доказательство. Пусть λ — произвольное положительное число. Справедлива следующая цепочка неравенств:

$$2^{\lambda(1-p)n}\sum_{k=0}^{pn}\binom{n}{k}=2^{\lambda(1-p)n}\sum_{k=(1-p)n}^{n}\binom{n}{k}\leqslant\sum_{k=(1-p)n}^{n}2^{\lambda k}\binom{n}{k}\leqslant$$

$$\leqslant\sum_{k=0}^{n}2^{\lambda k}\binom{n}{k}=(1+2^{\lambda})^n.$$

Отсюда получаем

$$\sum_{k=0}^{pn}\binom{n}{k}\leqslant(2^{-\lambda(1-p)}+2^{\lambda p})^n.$$

Выберем теперь $\lambda = \log_2 [(1-p)/p]$; тогда

$$\sum_{k=0}^{pn} \binom{n}{k} \leqslant [2^{-(1-p)\log_2[(1-p)/p]} + 2^{p\log_2[(1-p)/p]}]^n =$$

$$= [2^{-(1-p)\log_2(1-p) - p\log_2 p}(2^{\log_2 p} + 2^{\log_2(1-p)})]^n =$$

$$= 2^{nH(p)}(p + 1 - p)^n,$$

что доказывает лемму. □

Лемма 7.9.3. *Для любого множества M различных ненулевых двоичных слов длины $2m$ сумма W весов слов удовлетворяет неравенству*

$$W \geqslant 2m\lambda(M - 2^{2mH(\lambda)}),$$

где λ — любое число в интервале $(0, 1/2)$.

Доказательство. В силу леммы 7.9.2 в рассматриваемом множестве для любого λ из интервала $(0, 1/2)$ число последовательностей длины $2m$, имеющих вес не более $2m\lambda$, удовлетворяет неравенству

$$\sum_{i=1}^{2m\lambda} \binom{2m}{i} < 2^{2mH(\lambda)}.$$

Таким образом, для каждого λ существует не менее $M - 2^{2mH(\lambda)}$ слов веса более $2m\lambda$. Следовательно, сумма весов M слов множества удовлетворяет неравенству

$$W \geqslant 2m\lambda(M - 2^{2mH(\lambda)}).$$

Лемма доказана. □

Теорема 7.9.4. *Пусть для каждого фиксированного R $(0 < R < 1/2)$ $K = \lceil 2NR \rceil$. Тогда при каждом целом m (n, k)-код Юстесена $(n = 2mN,\ k = 2mK)$ имеет скорость не менее R и минимальное расстояние, удовлетворяющее неравенству*

$$d^*/n \geqslant (1 - 2R)(H^{-1}(1/2) - o(1)),$$

где $o(1)$ — функция от n, стремящаяся к нулю при $r \to \infty$.

Доказательство. Поскольку $K = \lceil 2NR \rceil$, то $K \geqslant 2NR$ и, следовательно, скорость кода $K/(2N)$ превышает R. Кодовое слово с содержит не менее $N - K + 1$ различных ненулевых столбцов, т. е. не менее $N - K + 1$ различных ненулевых двоичных последовательностей длины $2m$. Далее,

$$N - K + 1 \geqslant N(1 - 2R).$$

Выберем $N(1-2R)$ этих последовательностей длины $2m$. Тогда условия леммы 7.9.3 удовлетворяются, и из этой леммы следует оценка

$$d^* \geqslant W \geqslant 2m\lambda\,[N(1-2R) - 2^{2mH(\lambda)}],$$

или

$$\frac{d^*}{n} \geqslant \lambda(1-2R)\left(1 - \frac{2^{mH(\lambda)}}{1-2R}\right).$$

Полагая

$$\lambda = H^{-1}\left(\frac{1}{2} - \frac{1}{\log_2 2m}\right),$$

приходим к утверждению теоремы. □

Из теоремы 7.9.4 следует важность кодов Юстесена. Эта теорема гласит, что для достаточно больших n выполняется неравенство

$$d^*/n \geqslant 0{,}11\,(1-2R).$$

При фиксированном $R < 1/2$ отношение d^*/n отделено от нуля при $n \to \infty$. Коды Юстесена являются единственным известным классом кодов с заданной в явном виде конструкцией, для которых установлено это свойство [1]).

Коды Юстесена имеют скорость, меньшую 1/2. Коды с большей скоростью можно получить, выкалывая компоненты исходного кода. Получаемые таким образом коды также представляют только теоретический интерес как пример кодов, имеющих хорошие характеристики при очень больших длинах.

ЗАДАЧИ

7.1. Найти порождающий многочлен $g(x)$ для исправляющего двойные ошибки двоичного кода длины $n=31$. Использовать примитивный элемент α и примитивный многочлен $p(x) = x^5 + x^2 + 1$.

7.2. Найти кодовое слово, соответствующее информационному многочлену $\alpha^5 x + \alpha^3$ для кода Рида—Соломона длины 15, исправляющего одиночные ошибки и построенного по примитивному элементу α поля $GF(2^4)$.

7.3.а. Найти порождающий многочлен $g(x)$ для исправляющего двойные ошибки кода Рида—Соломона, основываясь на примитивном элементе α поля $GF(2^4)$ и полагая $j_0 = 1$.

---

[1]) Коды Юстесена отнюдь не являются единственным подклассом обширного класса каскадных кодов, обладающих тем свойством, что при увеличении n оба параметра кода — и скорость, и отношение расстояние/длина — отделены от нуля. Подчеркиваемая автором «явность» конструкции Юстесена в равной мере может быть отнесена и к конструкции Зяблова линейных каскадных кодов, если допустить несущественный перебор внутренних кодов, лежащих на границе Варшамова—Гилберта (см. Зяблов В. В. Оценка сложности построения двоичных линейных каскадных кодов. — Проблемы передачи информации, 1971, Вып. 1, с. 5—13). — *Прим. ред.*

б. Используя метод декодирования Питерсона—Горенстейна—Цирлера (ПГЦ), найти многочлены локаторов ошибок, локаторы и значения ошибок, если $S_1 = \alpha^4$, $S_2 = 0$, $S_3 = \alpha^8$ и $S_4 = \alpha^2$.

в. Решить задачу п. б, используя алгоритм Берлекэмпа—Месси.

7.4. В § 7.1 был получен порождающий многочлен (15, 6, 7)-кода БЧХ над полем GF (4). Найти порождающий многочлен (15, 7, 7)-кода БЧХ над тем же полем.

7.5. Исправляющий двойные ошибки (15, 7)-код БЧХ имеет порождающий многочлен $g(x) = x^8 + x^7 + x^6 + x^4 + 1$. Комбинируя идеи декодера Меггитта и алгоритма ПГЦ, построить декодер, работающий по принципу декодера Меггитта, но использующий последовательность компонент синдрома S_1, S_2, S_3, S_4 в расширении поля, а не сам синдромный многочлен $s(x)$.

7.6. Найти порождающий многочлен (63, 55, 5)-кода над GF (8).

7.7. Предположим, что задан код Рида—Соломона с минимальным расстоянием $2t + 1$ и что ν — целое число, меньшее t. При декодировании требуется исправлять все ошибки кратности до ν включительно и обнаруживать ошибки с кратностями в пределах от $\nu + 1$ до $2t - \nu$. Как это сделать, используя алгоритм Берлекэмпа—Месси? Решить ту же задачу в случае кода БЧХ.

7.8. Показать, что каждый код БЧХ является подкодом над подполем некоторого кода Рида—Соломона с тем же конструктивным расстоянием. В каком случае скорость подкода над подполем будет такой же, как и у исходного кода Рида—Соломона? Найти в несистематическом (7, 5)-коде Рида—Соломона 16 информационных последовательностей, которые порождают кодовые слова (7, 4)-кода Хэмминга.

7.9. Найти порождающий многочлен для исправляющего двойные ошибки (23, 12)-кода БЧХ над GF (2). (Заметим, что найденный код представляет собой код Голея; он является примером кода БЧХ, у которого минимальное расстояние больше конструктивного.)

7.10. По троичному каналу в каждый момент времени передается один из трех символов: синусоидальный импульс с нулевой фазой, синусоидальный импульс с фазой 120° и синусоидальный импульс с фазой 240°. Обозначим множество передаваемых символов через {0, 1, 2} и будем считать, что ошибки в канале происходят случайно и с равными вероятностями.

Построить исправляющий тройные ошибки код длины 80 для передачи по такому каналу, учитывая, что примитивный многочлен степени 4 над GF (3) записывается в виде

$$p(x) = x^4 + x + 2.$$

Какова скорость полученного кода? Как использовать этот код для передачи блоков двоичных данных?

7.11. Память на магнитной ленте хранит байты из восьми битов как один 256-ичный символ. Построить в этом алфавите исправляющий одиночные ошибки (15, 13)-код над GF (256), а также схему декодера.

7.12. Многочлен $y^{20} + y^3 + 1$ является примитивным многочленом над GF (2). Примитивный элемент $\alpha = y$ с минимальным многочленом $y^{20} + y^3 + 1$ используется для построения исправляющего двойные ошибки кода БЧХ длины $2^{20} = 1\,048\,576$.

а. По принятому слову вычислены компоненты синдрома

$$S_1 = y^4, \quad S_3 = y^{12} + y^9 + y^6.$$

Найти остальные компоненты синдрома.

б. Найти многочлен локаторов ошибок.

в. Найти позиции ошибок или ошибки.

7.13.а. Найти порождающий многочлен $g(x)$ двоичного кода БЧХ длины $n = 15$ с $d = 7$. При построении использовать примитивный элемент α, $j_0 = 1$ и неприводимый многочлен $p(x) = x^4 + x + 1$.

6. Пусть при использовании кода из п.а по принятой последовательности v вычислены компоненты синдрома

$$S_1 = v(\alpha) = \alpha + \alpha^2, \quad S_3 = v(\alpha^3) = \alpha^3, \quad S_5 = v(\alpha^5) = 1.$$

(i) Найти S_2, S_4 и S_6.
(ii) Найти число ошибок ν, если известно, что $\nu \leqslant 3$.
(iii) Найти многочлен локаторов ошибок.
(iv) Найти вектор ошибок.

ЗАМЕЧАНИЯ

Коды БЧХ независимо открыли Хоквингем [1959] и Боуз и Рой-Чоудхури [1960]. Вскоре Горенстейн и Цирлер [1961] установили, что открытые ранее Ридом и Соломоном [1960] коды являются частным случаем недвоичных кодов БЧХ.

Поведение минимального расстояния и скорости кодов БЧХ большой длины исследовалось многими авторами на протяжении большого промежутка времени. Коды БЧХ длины $q^m - 1$ становятся неудовлетворительными, когда эта длина оказывается много больше q. Для умеренных длин кодов так происходит потому, что в случае двоичных кодов на каждую исправляемую ошибку приходится порядка m проверочных символов, а в случае кода с $q \neq 2$ — порядка $2m$ проверочных символов. Асимптотическое поведение таких кодов несколько лучше, хотя все еще остается неудовлетворительным. Точное изложение этих вопросов потребовало бы значительных усилий.

Касами и Токура [1969] открыли бесконечный подкласс примитивных кодов БЧХ, для которых граница БЧХ строго меньше истинного минимального расстояния; первым примером такого кода был (127,43)-код БЧХ с $d = 29$ и $d^* = 31$. Чинь [1970] придумал примеры циклических кодов, которые лучше кодов БЧХ; одним из таких примеров является циклический (63, 28, 15)-код.

Впервые процедура исправления ошибок для двоичных кодов БЧХ была предложена Питерсоном [1960], а для кодов БЧХ с произвольным алфавитом — Горенстейном и Цирлером [1961]. Чень [1964] и Форни [1965] упростили эту процедуру. Итеративный алгоритм нахождения многочлена локаторов ошибок был предложен Берлекэмпом [1968]. Месси [1969] представил этот алгоритм в виде процедуры построения авторегрессионных фильтров. Бартон [1971] показал, как осуществить алгоритм Берлекэмпа—Месси, не используя деления, хотя для вычисления значений ошибок деление все равно необходимо.

Сугияма, Касахара, Хирасава и Намекава [1975] впервые предложили использовать при декодировании алгоритм Евклида. Велч и Шольц [1979] построили использующий разложение в непрерывную дробь декодер, совершенно эквивалентный декодеру, использующему алгоритм Евклида. Другой способ использования алгоритма Евклида при декодировании был предложен Мандельбаумом [1977] и будет рассмотрен в § 9.1.

ГЛАВА 8

КОДЫ, ОСНОВАННЫЕ НА СПЕКТРАЛЬНЫХ МЕТОДАХ

Обработка дискретных сигналов основывается на применении преобразования Фурье. Исследование сигналов с непрерывным временем, принимающих вещественные и комплексные значения, существенно связано с преобразованием Фурье; в случае сигналов с дискретным временем аналогичную роль играет дискретное преобразование Фурье. Для многих значений n существуют также преобразования Фурье на векторном пространстве последовательностей длины n над полем Галуа $GF(q)$. Преобразования Фурье в поле Галуа могут играть важную роль в исследовании и обработке $GF(q)$-значных сигналов, т. е. кодовых слов. Основываясь на преобразовании Фурье, можно построить теорию кодирования существенно отличным от использовавшегося ранее способом. Циклические коды можно определить как коды, в которых некоторые спектральные компоненты слов равны нулю. Декодирование кодов БЧХ и кодов Рида—Соломона также может быть описано на спектральном языке.

В данной и в последующих главах коды и алгоритмы изучаются с частотной точки зрения. Пересматривая многие положения с частотной точки зрения, мы можем углубить наше понимание предмета и найти альтернативные методы кодирования и декодирования. Частотный подход мы используем также для введения некоторых дополнительных классов кодов.

8.1. ПРЕОБРАЗОВАНИЯ ФУРЬЕ В ПОЛЕ ГАЛУА

В поле комплексных чисел дискретное преобразование Фурье вектора $\mathbf{p} = \{p_i,\ i = 0, \ldots, N-1\}$ с комплексными компонентами определяется как вектор $\mathbf{P} = \{P_k,\ k = 0, \ldots, N-1\}$, задаваемый равенствами

$$P_k = \sum_{i=0}^{N-1} e^{-2\pi j N^{-1} ik} p_i, \quad k = 0, \ldots, N-1,$$

где $j = \sqrt{-1}$. Ядро преобразования Фурье $\exp(-2\pi jN^{-1})$ равно корню степени N из единицы в поле комплексных чисел. В конечном поле $GF(q^m)$ элемент α порядка n равен корню степени n из единицы. Проводя аналогию между $\exp(-2\pi jN^{-1})$ и α, получаем следующее определение.

Определение 8.1.1. Пусть $\mathbf{v} = \{v_i,\ i = 0, \ldots, n-1\}$ — вектор над $GF(q)$, где n делит $q^m - 1$ при некотором m, и пусть α — элемент порядка n в поле $GF(q^m)$. Преобразование Фурье в поле Галуа вектора $\mathbf{v}$ определяется как вектор $\mathbf{V} = \{V_j,\ j = 0, \ldots, n-1\}$, задаваемый равенствами

$$V_j = \sum_{i=0}^{n-1} \alpha^{ij} v_i, \quad j = 0, \ldots, n-1.$$

Дискретный индекс i естественно назвать временем, а $\mathbf{v}$ — *временно́й функцией* или *сигналом*. Аналогично индекс j можно назвать *частотой*, а $\mathbf{V}$ — *частотной функцией* или *спектром*.

В качестве длины преобразования Фурье можно выбрать произвольный делитель числа $q^m - 1$, но наиболее важную роль играют примитивные длины $n = q^m - 1$. В последнем случае α является примитивным элементом поля $GF(q^m)$. В отличие от поля комплексных чисел в поле Галуа преобразование Фурье существует не для любой длины n, так как не для любого n в поле существует элемент этого порядка. Для многих целей, однако, таких элементов оказывается достаточно. Если m — наименьшее целое, такое, что n делит $q^m - 1$, то над полем $GF(q)$ существует преобразование Фурье длины n и компоненты этого преобразования лежат в поле $GF(q^m)$. К сожалению, для некоторых значений n преобразования, хотя они и существуют, лежат в столь большом расширении поля, что неприемлемы для данного практического применения.

В случае дискретного преобразования Фурье преобразование $\mathbf{P}$ вещественнозначной временно́й функции $\mathbf{p}$ является комплексным. Аналогично если в случае преобразования в поле Галуа временна́я функция $\mathbf{v}$ принимает значения в поле $GF(q)$, то ее спектр $\mathbf{V}$ лежит в расширении поля $GF(q^m)$. В применениях, связанных с контролем ошибок, все связанные с декодированием действия осуществляются на самом деле в большом поле $GF(q^m)$, однако начинать мы должны с вектора на входе канала, т. е. в малом поле $GF(q)$.

Теорема 8.1.2. *Над полем $GF(q)$ характеристики p вектор и его спектр связаны соотношениями*

$$V_j = \sum_{i=0}^{n-1} \alpha^{ij} v_i, \quad v_i = (1/n) \sum_{j=0}^{n-1} \alpha^{-ij} V_j,$$

где n интерпретируется как число поля, т. е. по модулю p.

Доказательство. В любом поле

$$x^n - 1 = (x - 1)(x^{n-1} + x^{n-2} + \cdots + x + 1).$$

По определению элемента α элемент α^r при всех r является корнем многочлена в левой части. Следовательно, для всех $r \not\equiv 0$ по модулю n элемент α^r является корнем последнего многочлена. Но это эквивалентно равенству

$$\sum_{j=0}^{n-1} \alpha^{rj} = 0, \quad r \not\equiv 0 \pmod{n};$$

если же $r = 0$ по модулю n, то

$$\sum_{j=0}^{n-1} \alpha^{rj} = n \pmod{p},$$

что всегда отлично от нуля, если n не кратно характеристике p. Комбинируя эти равенства, получаем

$$\sum_{j=0}^{n-1} \alpha^{-ij} \sum_{k=0}^{n-1} \alpha^{kj} v_k = \sum_{k=0}^{n-1} v_k \sum_{j=0}^{n-1} \alpha^{(k-i)j} = (n \bmod p)\, v_i.$$

Наконец, $q^m - 1 = p^M - 1$ кратно n и поэтому n не кратно p. Следовательно, $n \not\equiv 0 \pmod{p}$. Это доказывает теорему. □

Преобразование Фурье обладает многими сильными свойствами, которые переносятся на случай конечных полей. Примером является свойство свертки, доказываемое в приведенной ниже теореме. (Можно доказать и обратную теорему, поменяв местами временну́ю и частотную области).

Теорема 8.1.3 (теорема о свертке). *Предположим, что*

$$e_i = f_i g_i, \quad i = 0, \ldots, n - 1.$$

Тогда

$$E_j = (1/n) \sum_{k=0}^{n-1} F_{((j-k))} G_k, \quad j = 0, \ldots, n - 1,$$

где двойные скобки означают, что индексы вычисляются в арифметике по модулю n.

Доказательство. Найдем преобразование Фурье вектора с компонентами $e_i = f_i g_i$:

$$E_j = \sum_{i=0}^{n-1} \alpha^{ij} f_i \left((1/n) \sum_{k=0}^{n-1} \alpha^{-ik} G_k \right) =$$

$$= (1/n) \sum_{k=0}^{n-1} G_k \left(\sum_{i=0}^{n-1} \alpha^{i((j-k))} f_i \right) = (1/n) \sum_{k=0}^{n-1} G_k F_{((j-k))}. \qquad \square$$

Заметим также, что выбор $j = 0$ в формуле свертки

$$E_j = \sum_{i=0}^{n-1} \alpha^{ij} f_i g_i = (1/n) \sum_{k=0}^{n-1} F_{((j-k))} G_k$$

приводит к формуле типа равенства Парсеваля:

$$\sum_{i=0}^{n-1} f_i g_i = (1/n) \sum_{k=0}^{n-1} F_{((n-k))} G_k.$$

Теорема 8.1.4 (свойство сдвига). *Если* $\{v_i\} \leftrightarrow \{V_j\}$ *является парой преобразования Фурье, то парами преобразований Фурье являются также*

$$\{\alpha^i v_i\} \leftrightarrow \{V_{((j+1))}\} \quad \text{и} \quad \{v_{((i-1))}\} \leftrightarrow \{\alpha^j V_j\}.$$

Доказательство получается непосредственной подстановкой. □

Иногда вектор $\mathbf{v}$ задается многочленом $v(x)$. С помощью преобразования Фурье в поле Галуа многочлен

$$v(x) = v_{n-1}x^{n-1} + \cdots + v_1 x + v_0$$

может быть преобразован в многочлен

$$V(x) = V_{n-1}x^{n-1} + \cdots + V_1 x + V_0,$$

который называется *спектральным многочленом* или *ассоциированным с* $v(x)$ *многочленом*. Как устанавливает следующая теорема, свойства спектра тесно связаны с корнями многочленов.

Теорема 8.1.5.

(i) *Элемент* α^j *является корнем многочлена* $v(x)$ *тогда и только тогда, когда* j*-я частотная компонента* V_j *равна нулю.*

(ii) *Элемент* α^{-i} *является корнем многочлена* $V(x)$ *тогда и только тогда, когда* i*-я временная компонента* v_i *равна нулю.*

Доказательство утверждения (i) очевидно, так как

$$v(\alpha^j) = \sum_{i=0}^{n-1} v_i \alpha^{ij} = V_j.$$

Утверждение (ii) доказывается тем же путем. □

Таким образом, если один говорит о корнях многочлена в конечном поле, а другой — о равных нулю спектральных компонентах, то в действительности они говорят об одном и том же, хотя терминология и точки зрения различны: в первом случае на первый план выдвигается разложение многочленов, во втором — преобразование Фурье.

8.2. ОГРАНИЧЕНИЯ СОПРЯЖЕННОСТИ И ИДЕМПОТЕНТЫ

Преобразование Фурье длины n над $GF(q)$ принимает значения в расширении поля $GF(q^m)$. Если мы начнем с произвольного n-мерного вектора над $GF(q^m)$ и вычислим обратное преобразование Фурье, то в общем случае не получим временно́го вектора над $GF(q)$; возможны компоненты из большего поля. Нам надо найти ограничения на спектр, которые бы гарантировали попадание компонент временно́го вектора в $GF(q)$.

Ограничения такого рода знакомы по полю комплексных чисел. Напомним, что в поле комплексных чисел спектр $P(f)$ имеет вещественное обратное преобразование Фурье тогда и только тогда, когда $P^*(-f) = P(f)$. Следующая теорема описывает множество ограничений, которые называются *ограничениями* (или *условиями*) *сопряженности* и устанавливают аналогичное условие для конечного поля.

Теорема 8.2.1. *Пусть* $\mathbf{V}$ *есть* n*-мерный вектор с компонентами из* $GF(q^m)$, *где* n *делит* $q^m - 1$. *Тогда обратное преобразование Фурье* $\mathbf{v}$ *является вектором с компонентами из* $GF(q)$ *тогда и только тогда, когда выполняются следующие равенства*:

$$V_j^q = V_{((qj))}, \quad j = 0, \ldots, n-1.$$

Доказательство. По определению

$$V_j = \sum_{i=0}^{n-1} \alpha^{ij} v_i, \quad j = 0, \ldots, n-1.$$

В поле характеристики p для любого целого r справедливо равенство $(a+b)^{p^r} = a^{p^r} + b^{p^r}$. Далее, если v_i — элемент поля $GF(q)$, то $v_i^q = v_i$ при всех i. Следовательно,

$$V_j^q = \left(\sum_{i=0}^{n-1} \alpha^{ij} v_i\right)^q = \sum_{i=0}^{n-1} \alpha^{qij} v_i^q = \sum_{i=0}^{n-1} \alpha^{qij} v_i = V_{(qj))}.$$

Обратно, предположим, что $V_j^q = V_{((qj))}$ для всех j. Тогда

$$\sum_{i=0}^{n-1} \alpha^{iqj} v_i^q = \sum_{i=0}^{n-1} \alpha^{iqj} v_i, \quad j = 0, \ldots, n-1.$$

Пусть $k = qj$. Так как q взаимно просто с $n = q^m - 1$, то, когда j принимает все значения от 0 до $n-1$, число k также принимает все значения от 0 до $n-1$. Следовательно,

$$\sum_{i=0}^{n-1} \alpha^{ik} v_i^q = \sum_{i=0}^{n-1} \alpha^{ik} v_i, \quad k = 0, \ldots, n-1,$$

и в силу однозначности преобразования Фурье $v_i^q = v_i$ для всех i. Таким образом, v_i являются корнями многочлена $x^q - x$ при всех i, и этими корнями исчерпывается все поле $GF(q)$. □

Чтобы применить данную теорему, следующим образом разобьем числа по mod n на подмножества, известные под названием *классов сопряженных элементов*:

$$A_j = \{j, jq, \ldots, jq^{m_j-1}\},$$

где m_j — наименьшее целое положительное число, удовлетворяющее равенству $jq^{m_j} = j \pmod n$. В силу конечности поля такое m_j всегда существует. Например, если $q = 2$ и $n = 7$, то классы сопряженных элементов имеют вид

$$A_0 = \{0\}, \quad A_1 = \{1, 2, 4\}, \quad A_2 = \{3, 6, 5\}.$$

Класс сопряженных элементов A_j выделяет в спектре множество частот. Назовем это множество частот *хордой*. Теорема 8.2.1 утверждает, что если временной сигнал принимает значения в поле $GF(q)$, то значение спектра в одной из частот хорды определяет значения спектра при всех частотах этой хорды. В следующем параграфе мы воспользуемся этим, чтобы дать спектральное описание кодов.

На рис. 8.1 приведены классы сопряженных элементов для некоторых малых полей в несколько измененных обозначениях. Для того чтобы подчеркнуть получающиеся симметрии, классы сопряженных элементов выписаны с использованием отрицательных целых чисел. При желании на рис. 8.1 отрицательное целое число j можно заменить положительным целым $n + j$. Классы сопряженных элементов по модулю 21 включены в таблицу для напоминания того, что в качестве модуля можно выбирать любое целое число. Заметим, что если члены классов сопряженных элементов по модулю 21 умножить на 3, то эти классы становятся классами сопряженных элементов по модулю 63.

Определение 8.2.2. *q-ичным следом* элемента β поля $GF(q^m)$ называется сумма

$$\mathrm{tr}(\beta) = \sum_{i=0}^{m-1} \beta^{q^i}.$$

Согласно теореме 4.6.10, q-я степень q-ичного следа элемента β равна q-ичному следу элемента β, и, следовательно, q-ичный след является элементом поля $GF(q)$. Если класс сопряженных элементов, которому принадлежит β, содержит m элементов, то $\mathrm{tr}(\beta)$ равен сумме всех элементов этого класса сопряженных элементов. В противном случае число элементов в классе сопряженных делит m, и кратность, с которой каждый элемент входит в след,

По модулю 7

{−1, −2, 3}
{0}
{1, 2, −3}

По модулю 15

{−1, −2, −4, 7}
{0}
{1, 2, 4, −7}
{3, 6, −3, −6}
{5, −5}

По модулю 31

{−5, −10, 11, −9, 13}
{−3, −6, −12, 7, 14}
{−1, −2, −4, −8, 15}
{0}
{1, 2, 4, 8, −15}
{3, 6, 12, −7, −14}
{5, 10, −11, 9, −13}

По модулю 21

{−3, −6, 9}
{−1, −2, −4, −8, 5, 10}
{0}
{1, 2, 4, 8, −5, −10}
{3, 6, −9}
{7, −7}

По модулю 63

{−11, −22, 19, −25, 13, 26}
{−9, −18, 27}
{−5, −10, −20, 23, −17, 29}
{−3, −6, −12, −24, 15, 30}
{−1, −2, −4, −8, −16, 31}
{0}
{1, 2, 4, 8, 16, −31}
{3, 6, 12, 24, −15, −30}
{5, 10, 20, −23, 17, −29}
{7, 14, 28, −7, −14, −28}
{9, 18, −27}
{11, 22, −19, 25, −13, −26}
{21, −21}

По модулю 127

{−21, −42, 43, −41, 45, −37, 53}
{−19, −38, 51, −25, −50, 27, 54}
{−13, −26, −52, 23, 46, −35, 57}
{−11, −22, −44, 39, −49, 29, 58}
{−9, −18, −36, 55, −17, −34, 59}
{−7, −14, −28, −56, 15, 30, 60}
{−5, −10, −20, −40, 47, −33, 61}
{−3, −6, −12, −24, −48, 31, 62}
{−1, −2, −4, −8, −16, −32, 63}
{0}
{1, 2, 4, 8, 16, 32, −63}
{3, 6, 12, 24, 48, −31, −62}
{5, 10, 20, 40, −47, 33, −61}
{7, 14, 28, 56, −15, −30, −60}
{9, 18, 36, −55, 17, 34, −59}
{11, 22, 44, −39, 49, −29, −58}
{13, 26, 52, −23, −46, 35, −57}
{19, 38, −51, 25, 50, −27, −54}
{21, 42, −43, 41, −45, 37, −53}

Рис. 8.1. Классы сопряженных элементов.

равна получающемуся частному. Из определения следа и теоремы 4.6.10 следует, что

$$\mathrm{tr}\,(\beta + \gamma) = \mathrm{tr}\,(\beta) + \mathrm{tr}\,(\gamma)$$

и что все сопряженные элементы имеют один и тот же след.

Теорема 8.2.3. *Над GF (q^m) q-ичный след принимает в качестве своего значения каждое из чисел поля GF (q) одинаково часто, а именно q^{m-1} раз.*

Доказательство. Пусть γ — элемент поля $GF(q)$, а β — элемент из $GF(q^m)$, след которого равен γ. Тогда β является корнем многочлена

$$x^{q^{m-1}} + x^{q^{m-2}} + \cdots + x^q + x - \gamma.$$

Степень этого многочлена равна q^{m-1}, и, следовательно, он имеет не более q^{m-1} корней. Но всего существует только q таких многочленов, и каждый из элементов поля $GF(q^m)$ является корнем одного из них. Это доказывает теорему. □

Одно из полезных свойств следа устанавливает следующая теорема.

Теорема 8.2.4. *Квадратное уравнение*

$$x^2 + x + a = 0,$$

где a — элемент поля $GF(2^m)$, имеет корни в поле $GF(2^m)$ тогда и только тогда, когда двоичный след элемента a равен нулю.

Доказательство. Пусть β — корень этого квадратного уравнения. Вычисленный в точке β двоичный след соответствующего квадратичного трехчлена равен

$$\mathrm{tr}(\beta^2 + \beta + a) = \mathrm{tr}(0) = 0.$$

По отношению к сложению след дистрибутивен, а следы элементов β и β^2 являются одним и тем же элементом поля $GF(2)$. Следовательно,

$$\mathrm{tr}(a) = 0.$$

Обратно, каждое β является корнем многочлена $x^2 + x + a$ для некоторого a, а именно для a, равного $-(\beta + \beta^2)$. Всего имеется 2^{m-1} таких a, для которых след равен нулю, и этого в точности хватает, чтобы составить 2^{m-1} уравнений с двумя корнями каждое. Доказательство закончено. □

Предположим теперь, что мы выбрали хорду A_k и определили спектр

$$W_j = \begin{cases} 0, & j \in A_k, \\ 1, & j \notin A_k. \end{cases}$$

Согласно теореме 8.2.1, обратное преобразование Фурье для этого спектра является вектором над $GF(q)$, который можно представить многочленом $w(x)$. Так как в частотной области свертка $w^2(x)$ преобразуется в произведение W_j^2, то $W_j^2 = W_j$,

и, следовательно, многочлен $w(x)$ обладает следующим специальным свойством:

$$w^2(x) = w(x) \pmod{x^n - 1}.$$

Произвольный многочлен $w(x)$, удовлетворяющий условию $w^2(x) = w(x) \pmod{x^n - 1}$, называется *идемпотентом*.

Каждый идемпотент может быть получен следующим образом. Выберем несколько хорд и положим $W_j = 0$, если j принадлежит одной из них, и $W_j = 1$ в противном случае. Обратное преобразование Фурье дает во временно́й области многочлен, который является идемпотентом, и каждый идемпотент можно построить таким способом.

Завершим этот параграф применением полученных результатов к циклическим кодам.

Теорема 8.2.5. *Каждый циклический код содержит в качестве кодового слова единственный многочлен $w(x)$, такой, что многочлен $c(x)$ является кодовым словом тогда и только тогда, когда*

$$c(x)\, w(x) = c(x) \pmod{x^n - 1}.$$

Многочлен $w(x)$ является идемпотентом.

Доказательство. Пусть $g(x)$ — порождающий многочлен, и пусть

$$W_j = \begin{cases} 0, & \text{если } g(\alpha^j) = 0, \\ 1, & \text{если } g(\alpha^j) \neq 0. \end{cases}$$

Тогда $w(x)$ является идемпонентом. Его корни совпадают с корнями $g(x)$, и, следовательно, он является кодовым словом. Кроме того, $W_j G_j = G_j$ для всех j, и поэтому $w(x)\, g(x) = g(x)$. Далее, многочлен $c(x)$ является кодовым словом тогда и только тогда, когда для некоторого $a(x)$ выполняется равенство $c(x) = a(x)\, g(x)$, откуда следует, что

$$c(x)\ w(x) = a(x)\ w(x)\, g(x) = a(x)\, g(x) = c(x) \pmod{x^n - 1};$$

это и доказывает теорему. □

8.3. СПЕКТРАЛЬНОЕ ОПИСАНИЕ ЦИКЛИЧЕСКИХ КОДОВ

Каждое слово $c(x)$ циклического кода задается многочленом степени $n - 1$. В несистематическом виде оно может быть записано как $c(x) = g(x)\, d(x)$, где $d(x)$ — информационный многочлен

степени $k-1$. Во временно́й области это дает (циклическую) свертку:

$$c_l = \sum_{k=0}^{n-1} g_{((l-k))} d_k.$$

Следовательно, в частотной области операция кодирования может быть записана в виде произведения

$$C_j = G_j D_j.$$

Любой удовлетворяющий этому равенству спектр задает в частотной области кодовое слово при условии, что во временно́й области все компоненты являются $GF(q)$-значными. В силу произвольности информационного спектра единственная существенная роль компонент G_j состоит в том, чтобы определить частоты, в которых стоят нулевые компоненты C_j спектра кодового слова. Таким образом, можно дать следующее альтернативное определение циклического кода. Циклическим кодом $\mathscr{C}$ называется множество таких слов над $GF(q)$, у которых все спектральные компоненты, принадлежащие заданному множеству так называемых *проверочных частот* $j_1, \ldots, j_{n-k}$, равны нулю.

Хотя каждое слово циклического кода является вектором над $GF(q)$, спектр кодового слова является вектором над $GF(q^m)$. Следовательно, циклический код может быть определен как множество $GF(q)$-значных обратных преобразований Фурье множества всех спектральных векторов, компоненты которых в заданном множестве частот равны нулю. Нельзя выбирать произвольный спектральный вектор, у которого стоят нули в заданном множестве частот; обратные преобразования некоторых из таких векторов могут иметь компоненты, не принадлежащие полю $GF(q)$. Для того чтобы кодовое слово принадлежало полю $GF(q)$, выбирать нужно только спектр, который удовлетворяет условиям сопряженности, приведенным в теореме 8.2.1.

Коды БЧХ являются такими циклическими кодами, в которых проверочные частоты выбираются последовательно. Исправляющий t ошибок код БЧХ длиной $n = q^m - 1$ определяется как множество всех кодовых слов над $GF(q)$, спектр которых равен нулю в заданном блоке из $2t$ последовательных частот.

Доказательство границы БЧХ в частотной области представляется простым и интуитивно очевидным. Следует, пожалуй, привести это второе доказательство, чтобы выявить различие в подходах и методах.

Теорема 8.3.1 (граница БЧХ). *Пусть n делит $q^m - 1$ при некотором m. Единственным вектором из $GF^n(q)$ веса не более $d-1$, имеющим $d-1$ последовательных нулевых компонент спектра, является нулевой вектор.*

Доказательство. Обозначим через $i_1, \ldots, i_\nu$ индексы ν ненулевых компонент вектора $\mathbf{c}$, $\nu \leqslant d-1$. Определим в частотной области вектор, обратное преобразование Фурье которого содержит нулевые компоненты для всех частот i, для которых $c_i \neq 0$. Такой вектор может быть выбран многими способами. Один из возможных выборов основывается на многочлене локаторов $\Lambda(x)$:

$$\Lambda(x) = \prod_{k=1}^{\nu} (1 - x\alpha^{-i_k}) = \Lambda_\nu x^\nu + \Lambda_{\nu-1}x^{\nu-1} + \cdots + \Lambda_1 x + \Lambda_0.$$

Вектор $\boldsymbol{\Lambda}$ представляет собой частотный спектр, разумное определение которого сводится к тому, что его обратное преобразование $\boldsymbol{\lambda} = \{\lambda_i\}$ равно нулю для каждого момента времени i, для которого $c_i \neq 0$. Выписанное выше произведение во временно́й области равно нулю ($\lambda_i c_i = 0$ для $i = 0, \ldots, n-1$); следовательно, в частотной области циклическая свертка равна нулю:

$$\boldsymbol{\Lambda} * \mathbf{C} = \mathbf{0}.$$

Так как $\Lambda_0 = 1$ и $\Lambda_k = 0$ при $k > d-1$, то свертка может быть записана в виде

$$C_j = -\sum_{k=1}^{d-1} \Lambda_k C_{((j-k))}.$$

Но на блоке длины $d-1$ вектор $\mathbf{C}$ равен нулю. Следовательно, по рекурсии вектор $\mathbf{C}$ всюду равен нулю, так что и $\mathbf{c}$ должен быть нулевым вектором. □

Если $n = q-1$ (или n делит $q-1$), то код БЧХ является кодом Рида—Соломона; кодовое слово и его спектр лежат в одном и том же поле. Используя для вычисления спектральных компонент информационные символы, можно осуществлять кодирование непосредственно в частотной области. Каждый спектр, у которого $2t$ последовательных компонент равны нулю, является кодовым словом. Кодирование осуществляется следующим образом. Какие-либо $2t$ последовательных частот (например, первые $2t$) выбираются для обеспечения необходимого ограничения: символы в этих частотах полагаются равными нулю. Остальные $n-2t$ координат спектра заполняются информационными символами из $GF(q)$. Тогда обратное преобразование Фурье дает (несистематическое) кодовое слово, как показано на рис. 8.2. Поскольку имеется $n-2t$ частот, в которых записываются информационные символы, то получается $(n, n-2t)$-код Рида—Соломона.

Для более общего случая кодов БЧХ кодирование является более сложным. Теперь мы имеем два поля: поле символов $GF(q)$ и поле локаторов $GF(q^m)$, используемое для спектра. Опять $2t$ последовательных компонент спектра выбираются для того, чтобы

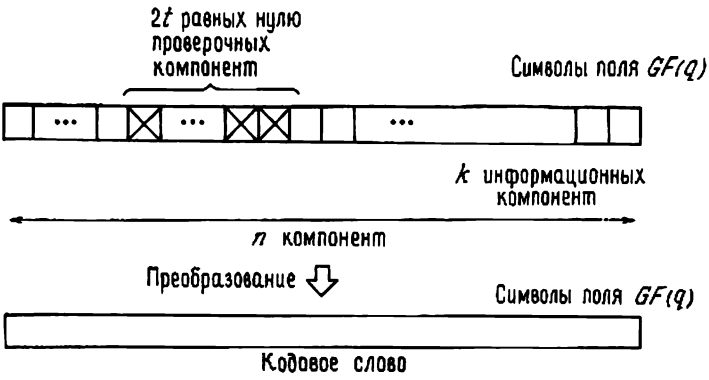


Рис. 8.2. Кодирование кодом Рида—Соломона с помощью преобразования Фурье.

в них записывались нули. Остальные компоненты представляют собой k информационных символов, которые должны выбираться из $GF(q^m)$ такими q^k способами, чтобы обратное преобразование Фурье принимало значение в $GF(q)$. В качестве иллюстрации на рис. 8.3 приводится соответствующая процедура для двоичного кода длины 63. Каждая компонента спектра здесь задается как 6-битовое двоичное число, а вектор спектра представлен как список 63 6-битовых чисел. Кодовое слово также представлено списком 63 6-битовых двоичных чисел, но с тем ограничением, что в каждом 6-битовом числе только бит наименьшего порядка может быть отличен от нуля. Таким образом, в действительности кодовое слово является 63-битовым двоичным словом. Мы хотим задать спектральный вектор так, чтобы сигнальный вектор был двоичным кодовым словом подобного вида. Теорема 8.2.1 дает нам ограничения, необходимые для построения подходящего спектрального вектора.

Хотя построение кода длины 63 можно выполнить сразу, мы предварительно рассмотрим более простой пример, а именно построим в частотной области (7,4)-код Хэмминга, используя теорему 8.2.1. Такое построение показано на рис. 8.4. В качестве проверочных частот выбраны компоненты C_1 и C_2, так что одиночная ошибка может быть исправлена. Информация заключена в частотных компонентах C_0 и C_3. Остальные частоты заполняются согласно ограничениям, даваемым теоремой: $C_1^4 = C_2^2 = C_4 = 0$ и $C_3^4 = C_6^2 = C_5$. Согласно теореме 8.2.1, $C_0^2 = C_0$, и, следовательно, C_0 может иметь только значения 0 или 1. Эквивалентное «битовое содержание» компоненты C_0 равно одному биту, а эквивалентное битовое содержание координаты C_3 равно трем битам. Таким образом, для однозначного задания спектра надо исполь-

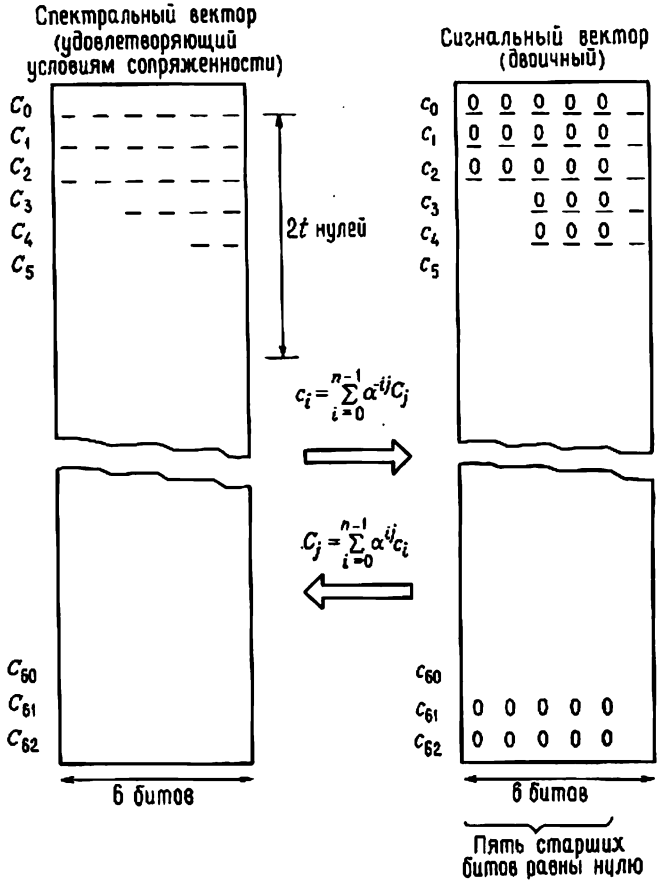


Рис. 8.3. Кодирование кодом БЧХ с помощью преобразования Фурье.

зовать четыре информационных бита кода Хэмминга. Эти информационные биты связаны с частотной, а не с временно́й областью.

В общем случае числа по модулю n разбиваются на классы сопряженных элементов:

$$A_j = \{j, jq, jq^2, \ldots, jq^{m_j-1}\}.$$

Если спектральная компонента C_j задана, то каждая другая спектральная компонента, индекс которой принадлежит классу сопряженных с j элементов, является степенью C_j и, следовательно, не может выбираться произвольно. Далее, если мощность этого класса сопряженных элементов равна r, то

$$C_j^{q^r} = C_j, \quad \text{т. е.} \quad C_j^{q^r-1} = 1.$$

Кодовые слова в частотной области							Кодовые слова во временной области						
C_0	C_1	C_2	C_3	C_4	C_5	C_6	c_0	c_1	c_2	c_3	c_4	c_5	c_6
0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	α^0	0	α^0	α^0	1	1	1	0	1	0	0
0	0	0	α^1	0	α^4	α^2	0	0	1	1	1	0	1
0	0	0	α^2	0	α^1	α^4	0	1	0	0	1	1	1
0	0	0	α^3	0	α^5	α^6	1	1	0	1	0	0	1
0	0	0	α^4	0	α^2	α^1	0	1	1	1	0	1	0
0	0	0	α^5	0	α^6	α^3	1	0	0	1	1	1	0
0	0	0	α^6	0	α^3	α^5	1	0	1	0	0	1	1
1	0	0	0	0	0	0	1	1	1	1	1	1	1
1	0	0	α^0	0	α^0	α^0	0	0	0	1	0	1	1
1	0	0	α^1	0	α^4	α^2	1	1	0	0	0	1	0
1	0	0	α^2	0	α^1	α^4	1	0	1	1	0	0	0
1	0	0	α^3	0	α^5	α^6	0	0	1	0	1	1	0
1	0	0	α^4	0	α^2	α^1	1	0	0	0	1	0	1
1	0	0	α^5	0	α^6	α^3	0	1	1	0	0	0	1
1	0	0	α^6	0	α^3	α^5	0	1	0	1	1	0	0

Рис. 8.4. (7,4)-код Хэмминга.

Следовательно, мы не можем произвольно выбирать элемент из $GF(q^m)$ в качестве возможного значения для C_j: допустимы или только те элементы поля, порядок которых делит $q^r - 1$, или нулевой элемент. Порядок каждого элемента поля $GF(q^m)$ делит $q^m - 1$; следовательно, $q^r - 1$ делит $q^m - 1$, и ясно, что мощность каждого класса сопряженных элементов делит m.

Для описания кодера разобьем множество $q^m - 1$ чисел на классы сопряженных элементов и выберем по одному числу из каждого класса в качестве представителя. Эти представители единственным образом определяют значимые символы. Для формирования кода БЧХ в качестве проверочных частот выбираются $2t$ спектральных компонент, которые полагаются равными нулю. Остальные значимые символы являются информационными и могут принимать произвольные значения с учетом ограничений на порядок. Все остальные символы, индексы которых принадлежат тому же классу сопряженных элементов, не являются свободными; они образуют связанные частоты.

На рис. 8.5 изображена ситуация для поля $GF(64)$. В первый столбец вносятся компоненты свободных частот. Если выбрать C_1, C_2, C_3, C_4, C_5 и C_6 в качестве проверочных частот, то получится

исправляющий три ошибки код БЧХ. Тогда C_0, C_7, C_9, C_{11}, C_{13}, C_{15}, C_{21}, C_{23}, C_{27} и C_{31} являются информационными символами. C_9 и C_{27} должны равняться или нулю, или элементу порядка 7 (так как $C_9^8 = C_9$ и $C_{27}^8 = C_{27}$); эти элементы принадлежат подполю GF (8). C_{21} должно равняться или нулю, или элементу порядка 3 (так как $C_{21}^4 = C_{21}$); эти элементы принадлежат подполю GF (4). C_0 должно быть или нулем, или элементом порядка 1; эти элементы образуют подполе GF (2). Все остальные элементы являются произвольными элементами поля GF (64). Для определения этих символов требуется всего 45 битов информации. Таким образом, получается (63,45)-код БЧХ, исправляющий три ошибки.

После выбора компоненты свободной частоты компоненты связанных частот определяются как ее соответствующие степени. Затем полный спектр преобразуется в кодовое слово. Формируемые таким образом 2^{45} кодовых слов являются в точности теми же 2^{45} кодовыми словами, которые получаются кодированием во временно́й области. Вплоть до момента, пока не понадобится выделение информации, декодеру не приходится заботиться о том, каким способом осуществлялось кодирование. Однако на последнем шаге, когда из исправленного кодового слова извлекается информация, декодеру необходимо знать, как эта информация была закодирована. Если кодирование осуществлялось в частотной области, то информационные символы должны вычисляться в частотной области.

Свободные частоты	Связанные частоты	Битовое содержание
$\{C_0\}$		1
$\{C_1$	$C_2\ C_4\ C_8\ C_{16}\ C_{32}\}$	6
$\{C_3$	$C_6\ C_{12}\ C_{24}\ C_{48}\ C_{33}\}$	6
$\{C_5$	$C_{10}\ C_{20}\ C_{40}\ C_{17}\ C_{34}\}$	6
$\{C_7$	$C_{14}\ C_{28}\ C_{56}\ C_{49}\ C_{35}\}$	6
$\{C_9$	$C_{18}\ C_{36}\}$	3
$\{C_{11}$	$C_{22}\ C_{44}\ C_{25}\ C_{50}\ C_{37}\}$	6
$\{C_{13}$	$C_{26}\ C_{52}\ C_{41}\ C_{19}\ C_{38}\}$	6
$\{C_{15}$	$C_{30}\ C_{60}\ C_{57}\ C_{51}\ C_{39}\}$	6
$\{C_{21}$	$C_{42}\}$	2
$\{C_{23}$	$C_{46}\ C_{29}\ C_{58}\ C_{53}\ C_{43}\}$	6
$\{C_{27}$	$C_{54}\ C_{45}\}$	3
$\{C_{31}$	$C_{62}\ C_{61}\ C_{59}\ C_{55}\ C_{47}\}$	6

Рис. 8.5. Структура спектра над полем GF (64).

8.4. РАСШИРЕННЫЕ КОДЫ РИДА—СОЛОМОНА

К коду Рида—Соломона в общем случае можно добавить две дополнительные компоненты; мы будем всегда помещать одну из них в начале, а другую в конце кодового слова. Коды, получаемые путем добавления одной или обеих дополнительных компонент, называются *расширенными кодами Рида—Соломона*. Каждый из добавленных символов может использоваться и как информационный, и как проверочный, т. е. либо для увеличения скорости передачи, либо для увеличения минимального расстояния кода. Мы используем этот менее конкретный термин — расширенные коды Рида—Соломона, хотя эти же коды можно построить увеличением числа слов в кодах Рида—Соломона с минимальным расстоянием d^* или удлинением кодов Рида—Соломона с минимальным расстоянием $d^* - 2$. В любом случае получится один и тот же расширенный код Рида—Соломона.

Надо определить два новых локатора и соответственно ввести некоторые новые обозначения. Если исходные компоненты нумеруются элементами поля, то для одной новой компоненты можно использовать нулевой элемент поля, так что остается определить еще один дополнительный символ для другой. Обычно используется символ ∞ [1]). Если исходные компоненты нумеруются показателями степени примитивного элемента, то для обозначения нового локатора нельзя использовать нуль и необходимо ввести два новых символа. В качестве этих двух символов мы будем пользоваться знаками — и $+$. Таким образом, слово расширенного кода записывается в виде

$$(c_-, c_0, c_1, c_2, \ldots, c_{q^m-3}, c_{q^m-2}, c_+),$$

и $n = q^m + 1$. Вектор, получающийся исключением символов c_- и c_+, будем называть *внутренним*. Мы будем изучать расширенные коды с помощью свойств преобразований Фурье внутренних векторов, дополняя их свойствами расширенного векторного пространства. Когда мы будем говорить о спектре кодового слова, то будем иметь в виду спектр внутреннего вектора.

Сначала дадим определение расширенного циклического кода, а затем — в качестве частного случая — определение расширенного кода Рида—Соломона.

Определение 8.4.1. Расширенным циклическим (n, k)-кодом над $GF(q)$ называется линейный код длины $n = q^m + 1$, каждое

[1]) Для расширения поля вещественных чисел общепринятым и удобным соглашением является использование символов $+\infty$ и $-\infty$ с доопределением на них арифметических операций. Такое расширение не образует поля. То же самое можно сделать с полем Галуа, добавив символ ∞ и определив операции равенствами $a + \infty = \infty$, $a \cdot \infty = \infty$. Такое множество не является полем.

слово которого удовлетворяет следующим условиям: спектр $(C_0, C_1, \ldots, C_{n-3})$ кодового слова $(c_-, c_0, c_1, \ldots, c_{n-3}, c_+)$ содержит нули в заданном множестве $n-k-2$ позиций с индексами $j_1, \ldots, j_{n-k-2}$, а две оставшиеся спектральные компоненты удовлетворяют равенствам $C_{j_0} = c_-$, $C_{j_{n-k-1}} = c_+$.

Расширенный циклический код в общем случае не является циклическим.

Определение 8.4.2. Расширенным кодом Рида—Соломона называется линейный код длины $n = q + 1$ над $GF(q)$, спектр кодовых слов $(c_-, c_0, c_1, \ldots, c_{n-2}, c_+)$ которого при любых целых j_0 и t удовлетворяет следующим условиям:

1) $C_j = 0$, $j = j_0 + 1, \ldots, j_0 + 2t - 2$;

2) $C_{j_0} = c_-$;

3) $C_{j_0+2t-1} = c_+$.

Число $2t+1$ называется конструктивным расстоянием расширенного кода Рида—Соломона. Определение содержит ограничения, состоящие в том, что $2t-2$ соответствующих спектральных компонент должны равняться нулю, а компоненты, окаймляющие эти $2t-2$ компонент, должны равняться c_- и c_+ соответственно. Эти две специальные частоты называются *граничными частотами*.

По сравнению с кодом Рида—Соломона, который получается удалением c_- и c_+ и приравниванием нулю спектральных компонент C_{j_0} и C_{j_0+2t-1}, расширенный код Рида—Соломона всегда содержит два дополнительных информационных символа при неизменном минимальном расстоянии. Впоследствии мы выясним, что это означает в частотной области, а сначала приведем основанное на свойствах матрицы Вандермонда доказательство этого факта.

Теорема 8.4.3. *Расширенный код Рида—Соломона над $GF(q)$ является $(q+1, k)$-кодом с минимальным расстоянием $2t+1 = n-k+1 = q-k+2$.*

Доказательство. Сначала предположим для простоты, что $j_0 = 0$. Проверочная матрица кода равна

$$\mathbf{H} = \begin{bmatrix} -1 & 1 & 1 & \ldots & 1 & 1 & 0 \\ 0 & \alpha & \alpha^2 & \ldots & \alpha^{q-2} & \alpha^{q-1} & 0 \\ 0 & \alpha^2 & \alpha^4 & \ldots & \alpha^{(q-2)2} & \alpha^{(q-1)2} & 0 \\ \vdots & & & & & & \vdots \\ 0 & \alpha^{q-k+1} & \alpha^{(q-k-1)2} & \ldots & \alpha^{(q-k-1)(q-2)} & \alpha^{(q-k-1)(q-1)} & 0 \\ 0 & \alpha^{q-k} & \alpha^{(q-k)2} & \ldots & \alpha^{(q-k)(q-2)} & \alpha^{(q-k)(q-1)} & -1 \end{bmatrix}.$$

Если любые $d-1$ столбцов этой проверочной матрицы линейно независимы, то минимальное расстояние кода равно по меньшей мере d. После исключения первого и последнего столбцов любое множество из $q-k+1$ столбцов образует матрицу Вандермонда, которая невырождена, и, следовательно, любые $d-1$ внутренних столбцов линейно независимы. В случае же когда выбранное множество из $q-k+1$ столбцов содержит первый и последний столбцы, соответствующий определитель можно вычислить, разлагая его по элементам сначала первого, а потом последнего столбцов. Это приведет к выбрасыванию сначала первой, а потом последней строки, и оставшаяся часть снова приводит к матрице Вандермонда, определитель которой отличен от нуля. Следовательно, любые $q-k+1$ столбцов линейно независимы, и поэтому минимальное расстояние кода равно по меньшей мере $q-k+2$.

Далее, если $j_0 \neq 0$, то изменения в проверочной матрице при неизменных первом и последнем столбцах состоят в том, что все ее элементы умножаются на α^{j_0}. Это, однако, никак не влияет на проведенные выше рассуждения. □

На рис. 8.6 изображен кодер в частотной области для расширенного кода Рида—Соломона. Мы сможем лучше понять эти коды, если будем представлять их кодер как модификацию кодера в частотной области для обычного кода Рида—Соломона с минимальным расстоянием d^*. Последний кодер определяется блоком

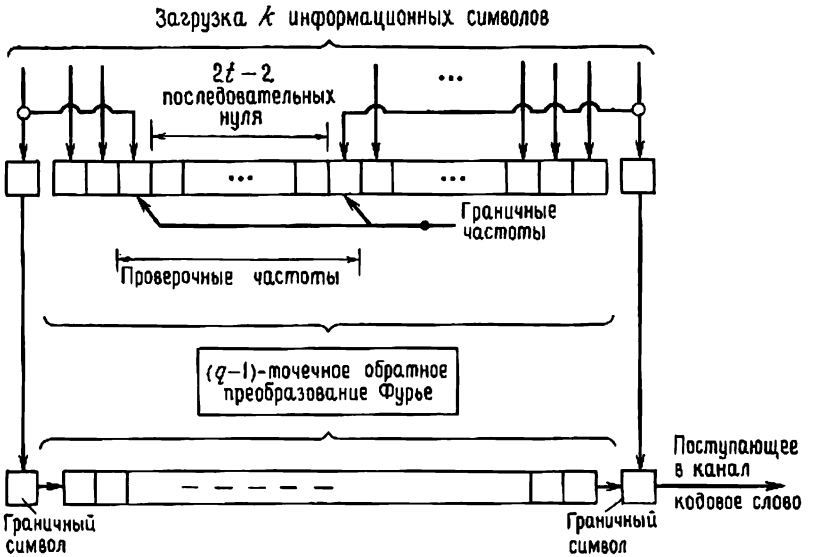


Рис. 8.6. Кодер для расширенного кода Рида—Соломона в частотной области.

из $d^* - 1$ последовательных частот, в которых стоят нулевые компоненты спектра. Остальные компоненты спектра принимают произвольные значения информационных символов из $GF(q)$.

Для расширения этого кода за счет увеличения числа информационных символов используются две граничные частоты спектра в блоке проверочных частот спектра, которым придаются произвольные значения двух информационных символов, причем во временно́й области кодовое слово тоже дополняется этими двумя символами. В результате получается расширенный код Рида — Соломона с тем же минимальным расстоянием d^*, но с двумя дополнительными информационными символами.

Если же мы хотим расширить исходный код так, чтобы увеличить его минимальное расстояние, то два дополнительных символа, присоединяемых к блоку проверочных частот, объявляются новыми проверочными частотами. Значения компонент этих частот не меняются, но во временно́й области те же два символа дописываются к кодовому слову. Это приводит к коду с таким же числом информационных символов, как и у исходного кода, но с минимальным расстоянием $d^* + 2$. Конечно, мы получим тот же самый расширенный код, если будем исходить из кода Рида—Соломона с минимальным расстоянием $d^* + 2$ и увеличивать в нем число информационных символов.

Кодер расширенного (n, k)-кода Рида—Соломона во временно́й области показан на рис. 8.7. В этом случае расширение рассматривается как удлинение $(n - 2, k)$-кода Рида—Соломона с минимальным расстоянием $d^* - 2$ до кода с расстоянием d^*. Для кодирования k информационных символов внутреннего вектора кодового слова используется порождающий многочлен с корнями

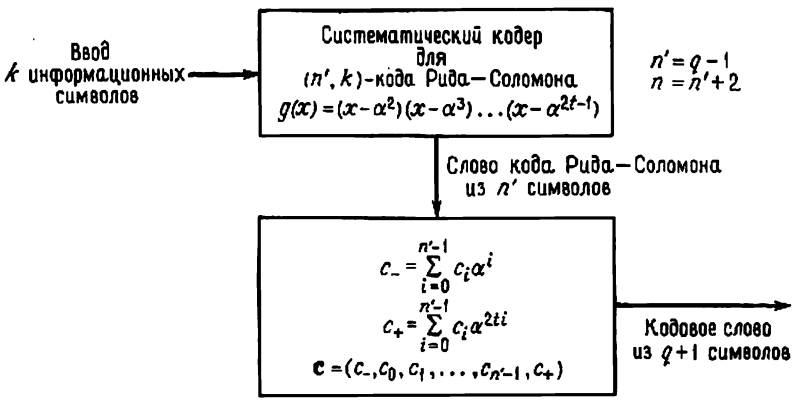


Рис. 8.7. Систематический кодер для расширенного кода Рида—Соломона.

$\alpha^2, \ldots, \alpha^{2t-1}$. Граничные символы определяются при этом соответственно равенствами

$$c_- = \sum_{i=0}^{n'-1} c_i \alpha^i \quad \text{и} \quad c_+ = \sum_{i=0}^{n'-1} c_i \alpha^{2ti}$$

и присоединяются к внутреннему вектору для формирования кодового слова.

8.5. РАСШИРЕННЫЕ КОДЫ БЧХ

Теперь рассмотрим коды с канальным алфавитом $GF(q)$, для построения которых используется поле локаторов $GF(q^m)$. Используя конструкцию подкода над подполем, можно из расширенного кода Рида—Соломона над $GF(q^m)$ построить расширение кода Рида—Соломона над меньшим подполем $GF(q)$. В силу ограничений на подполе $GF(q)$ иногда дополнительные компоненты расширения нельзя использовать в качестве информационных. А именно они, могут содержать только одни и те же значения символов для всех кодовых слов и, следовательно, вообще могут быть выкинуты.

В данном параграфе рассматривается несколько иной подход к построению кода над $GF(q)$ из расширения кода над $GF(q^m)$. Этот подход состоит в том, чтобы выбирать из расширения кода Рида—Соломона только те слова, у которых $q^m - 1$ компонент внутреннего вектора принимают значения из подполя $GF(q)$. Для двух символов расширения допускаются произвольные значения из поля $GF(q^m)$, но в кодовом слове они записываются в виде начального и конечного «хвостов», содержащих m $GF(q)$-ичных символов каждый.

Как мы увидим, такое кодирование не требует особого труда, но допускает декодирование с помощью преобразований и приводит ко многим хорошим кодам. Мы назовем эти коды *расширенными кодами БЧХ*. Для получения из расширенного кода БЧХ исходного кода достаточно выбрать те кодовые слова, хвосты которых равны нулю.

Вектор C размерности $q^m - 1$ над $GF(q^m)$ задает допустимый спектр для внутреннего вектора $GF(q)$-значного кодового слова, если он удовлетворяет условиям сопряженности

$$C_j^q = C_{((qj))}.$$

При построении расширенного кода БЧХ с конструктивным расстоянием d мы поступим так же, как и при построении расширенного кода Рида—Соломона. Выберем в ка-

честве проверочных $d-1$ смежных частот, начиная с j_0, и положим

$$C_{j_0} = c_-,$$
$$C_j = 0, \quad j = j_0 + 1, \ldots, j_0 + d - 3,$$
$$C_{j_0+d-2} = c_+,$$

где c_- и c_+ — произвольные элементы поля $GF(q^m)$, не нарушающие ограничения сопряженности. Остальные спектральные координаты выбираются произвольно, но удовлетворяют этим ограничениям. Тогда внутренний вектор кодового слова получается обратным преобразованием Фурье этого спектра. Внутреннему вектору в кодовом слове предшествует хвост, состоящий не более чем из m $GF(q)$-ичных символов и представляющий c_-, а за внутренним вектором следует аналогичный хвост, представляющий c_+. Ограничения сопряженности на C_{j_0} или C_{j_0+d-2} могут привести к тому, что порядок элементов c_- или c_+ будет меньше, чем $q^m - 1$, так что c_- или c_+ будут записываться числом $GF(q)$-ичных символов, меньшим m. В этом случае длина хвоста будет делителем m.

При рассмотрении кодов БЧХ мы обычно полагали $j_0 = 1$, что, как правило, приводило к лучшим кодам. В данном параграфе мы увидим, что, как правило, выбор $j_0 = 0$ или -1 приводит к хорошим расширенным кодам БЧХ, хотя другой выбор j_0 иногда будет давать некоторое улучшение.

Сначала будем считать $j_0 = 0$ и d нечетным ($d = 2t + 1$). На рис. 8.8 приведены все расширенные коды, получающиеся из примитивного кода БЧХ длины $n \leqslant 255$. Проверочными частотами расширенного кода БЧХ являются $C_0, \ldots, C_{2t-1}$, хотя в приводимых для сравнения примитивных кодах БЧХ проверочными частотами являются $C_1, \ldots, C_{2t}$. В качестве примера построим некоторые коды длины $n' = 63$, используя приведенные на рис. 8.1 классы сопряженных элементов. Так как $C_0^2 = C_0$, то этот элемент должен равняться нулю или единице, так что c_- является двоичным символом. C_{2t-1} (а следовательно, и c_+) является произвольным символом поля $GF(64)$, за исключением случаев $2t - 1 = 9$ и $2t - 1 = 21$, когда $j = 9, 18, 36, 27, 54, 45$ и $j = 42$, а c_+ содержит три и два бита информации соответственно. Далее, $2t - 1$ должно являться наименьшим целым числом в своем классе сопряженных элементов, так как в противном случае соответствующая компонента спектра равна нулю и, следовательно, C_{2t-1} также должна равняться нулю.

Пусть $t = 4$. Тогда $2t - 1 = 7$ и $C_0, C_1, \ldots, C_7$ являются проверочными частотами, причем $C_0 = c_-$ — произвольный элемент из $GF(2)$, а $C_7 = c_+$ — произвольный элемент из $GF(64)$. Далее, C_0 соответствует одному проверочному биту, а каждая из

$d_{БЧХ}$	Примитивный код БЧХ	Расширенный код БЧХ	$n-k$	Примитивный код БЧХ	Расширенный код БЧХ	$n-k$	Примитивный код БЧХ	Расширенный код БЧХ	$n-k$
3	(15, 11)	(20, 15)	5	(31, 26)	(37, 31)	6	(63, 57)	(70, 63)	7
5	(15, 7)	(20, 11)	9	(31, 21)	(37, 26)	11	(63, 51)	(70, 57)	13
7	(15, 5)	(18, 7)	11	(31, 16)	(37, 21)	16	(63, 45)	(70, 51)	19
9					(37, 16)	21	(63, 39)	(70, 45)	25
11				(31, 11)			(63, 36)	(67, 39)	28
13					(37, 11)	26	(63, 30)	(70, 36)	34
15				(31, 6)			(63, 24)	(70, 30)	40
17								(70, 24)	46
19									
21							(63, 18)		
23							(63, 16)	(66, 18)	48
25								(70, 16)	54
27							(63, 10)		
29									

$d_{БЧХ}$	Примитивный код БЧХ	Расширенный код БЧХ	$n-k$	Примитивный код БЧХ	Расширенный код БЧХ	$n-k$
3	(127, 120)	(135, 127)	8	(255, 247)	(264, 255)	9
5	(127, 113)	(135, 120)	15	(255, 239)	(264, 247)	17
7	(127, 106)	(135, 113)	22	(255, 231)	(264, 239)	25
9	(127, 99)	(135, 106)	29	(255, 223)	(264, 231)	33
11	(127, 92)	(135, 99)	36	(255, 215)	(264, 223)	41
13	(127, 85)	(135, 92)	43	(255, 207)	(264, 215)	49
15	(127, 78)	(135, 85)	50	(255, 199)	(264, 207)	57
17		(135, 78)	57	(255, 191)	(264, 199)	65
19	(127, 71)			(255, 187)	(260, 191)	69
21	(127, 64)	(135, 71)	64	(255, 179)	(264, 187)	77
23	(127, 78)	(135, 64)	71	(255, 171)	(264, 179)	85
25		(135, 57)	78	(255, 163)	(264, 171)	93
27	(127, 50)			(255, 155)	(264, 163)	101
29	(127, 43)	(135, 50)	85	(255, 147)	(264, 155)	109

частот C_1, C_3, C_5 и C_7 соответствует шести проверочным битам. Следовательно, $n = 63 + 1 + 6$, и мы получаем (70,45)-код с минимальным расстоянием, равным по меньшей мере 9.

Пусть $t = 8$. Тогда $2t - 1 = 15$ и проверочными частотами являются C_0, C_1, ..., C_{15}, причем $C_0 = c_-$ — произвольный элемент поля GF (2), а $C_{15} = c_+$ - произвольный элемент поля GF (64). Далее, C_0 соответствует одному проверочному биту, каждая из проверочных частот C_1, C_3, C_5, C_7, C_{11}, C_{13} и C_{15} соответствует шести проверочным битам, а C_9 соответствует трем проверочным битам. Следовательно, получаем (70,24)-код, минимальное расстояние которого равно по меньшей мере 17.

Второй пример иллюстрирует случай, когда конструктивное расстояние расширенного кода БЧХ меньше минимального расстояния исходного кода БЧХ. На рис. 8.8 имеется несколько таких случаев, и это происходит всегда, когда код БЧХ с конструктивным расстоянием d совпадает с кодом БЧХ с конструктивным расстоянием $d + 2$.

Положим теперь $j_0 = -1$. При таком выборе j_0 и $c_+ \neq 0$ минимальное расстояние кода четно, так как правая проверочная частота должна иметь нечетный индекс. Чтобы сделать минимальное расстояние нечетным, добавим проверку на четность для битов одного из хвостов. Дополнительный проверочный символ позволит декодеру обнаружить ошибку в таком хвосте. Этого достаточно, чтобы увеличить на единицу минимальное расстояние кода. Мы не будем здесь доказывать этот факт, а отложим его доказательство до § 9.3, где будет описана процедура декодирования $d/2$ ошибок. Существование такого декодера является доказательством возможности увеличения на единицу минимального расстояния.

Все расширенные коды с $j_0 = -1$, получающиеся из примитивных кодов БЧХ длины $n \leqslant 255$, приведены на рис. 8.9. Были рассмотрены также другие значения для j_0, но в диапазоне $n \leqslant 500$ при $d \leqslant 29$ не было найдено других интересных случаев. Единственным исключением является GF (64). При $j_0 = -9$ можно построить (70, 20, 21)-код, который является расширением кода БЧХ и лучше всех известных кодов с такими же n и k.

На рис. 8.10 изображен кодер для расширенного (n, k, d)-кода БЧХ. В этом кодере используется порождающий многочлен $(n', k, d-2)$-кода БЧХ, корни которого расположены в точках $\alpha^{j_0+1}, \ldots, \alpha^{j_0+d-3}$. В хвост кодового слова дописываются значения спектра в граничных частотах. В случае необходимости к одному из хвостов можно дописать его проверку на четность.

$d_{БЧХ}$	Примитивный код БЧХ	Расширенный код БЧХ	$n-k$	Примитивный код БЧХ	Расширенный код БЧХ	$n-k$	Примитивный код БЧХ	Расширенный код БЧХ	$n-k$
3	(15, 11)			(31, 26)			(63, 57)		
5	(15, 7)	(24, 14)	10	(31, 21)	(42, 30)	12	(63, 51)	(76, 62)	14
7	(15, 5)	(24, 10)	14	(31, 16)	(42, 25)	17	(63, 45)	(76, 56)	20
9					(42, 20)	22	(63, 39)	(76, 50)	26
11				(31, 11)	(42, 15)	27	(63, 36)	(76, 44)	32
13							(63, 30)	(73, 38)	35
15				(31, 6)	(42, 10)	32	(63, 24)	(76, 35)	41
17								(76, 29)	47
19								(76, 23)	53
21							(63, 18)		
23							(63, 16)		
25								(72, 17)	55
27							(63, 10)	(76, 15)	61
29									

$d_{БЧХ}$	Примитивный код БЧХ	Расширенный код БЧХ	$n-k$	Примитивный код БЧХ	Расширенный код БЧХ	$n-k$
3	(127, 120)			(255, 247)		
5	(127, 113)	(142, 126)	16	(255, 239)	(272, 254)	18
7	(127, 106)	(142, 119)	23	(255, 231)	(272, 246)	26
9	(127, 99)	(142, 112)	30	(255, 223)	(272, 238)	34
11	(127, 92)	(142, 105)	37	(255, 215)	(272, 230)	42
13	(127, 85)	(142, 98)	44	(255, 207)	(272, 222)	50
15	(127, 78)	(142, 91)	51	(255, 199)	(272, 214)	58
17		(142, 84)	58	(255, 191)	(272, 206)	66
19	(127, 71)	(142, 77)	65	(255, 187)	(272, 198)	74
21	(127, 64)			(255, 179)	(268, 190)	78
23	(127, 57)	(142, 70)	72	(255, 171)	(272, 186)	86
25		(142, 63)	79	(255, 163)	(272, 178)	94
27	(127, 50)	(142, 56)	86	(255, 155)	(272, 170)	102
29	(127, 43)	(142, 49)	93	(255, 147)	(272, 162)	110

Рис. 8.9. Таблица расширенных кодов БЧХ с $j_0 = -1$.

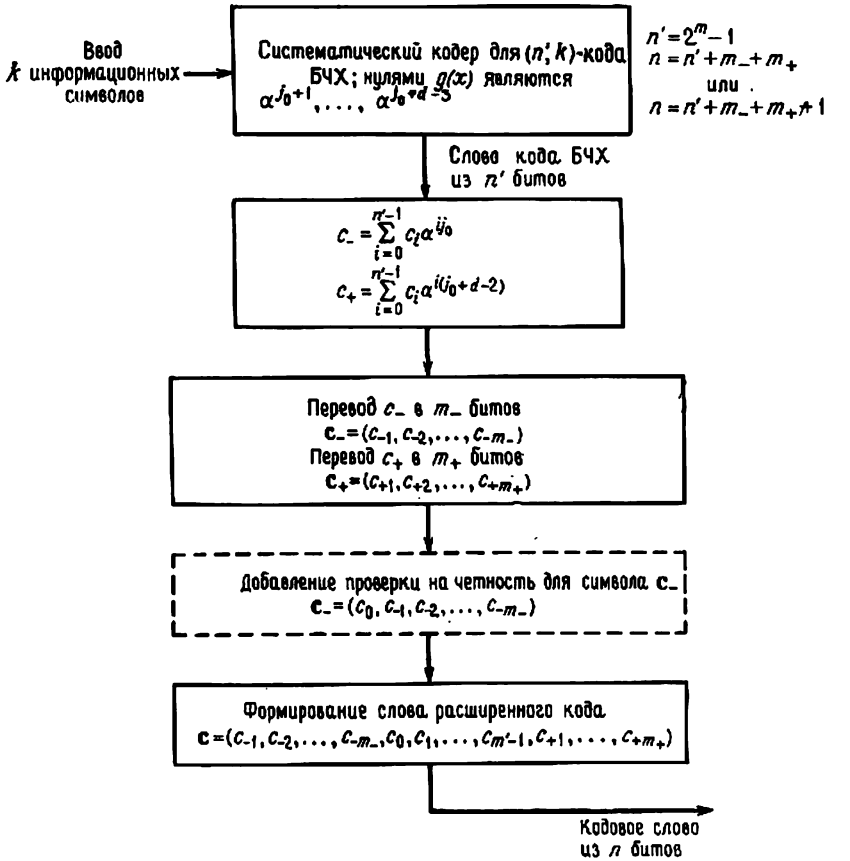


Рис. 8.10. Систематический кодер для расширенного кода БЧХ.

8.6. АЛЬТЕРНАНТНЫЕ КОДЫ

Код БЧХ длины $n = q^m - 1$ над $GF(q)$ представляет собой ограничение на подполе $GF(q)$ кода Рида—Соломона над полем $GF(q^m)$. Иными словами, код БЧХ состоит из всех $GF(q)$-значных слов кода Рида—Соломона. Поэтому минимальное расстояние кода БЧХ по меньшей мере равно минимальному расстоянию исходного кода Рида—Соломона. К сожалению, коды БЧХ большой длины с бо́льшим минимальным расстоянием не содержат нужного нам числа кодовых слов. Точнее говоря, в любой последовательности кодов БЧХ растущей длины с ограниченной скоростью (все коды последовательности удовлетворяют условию $k/n \geqslant R'$ для некоторого фиксированного R') нормированное минимальное расстояние d^*/n стремится к нулю с ростом n. Исходный код

Рида—Соломона содержит достаточно кодовых слов, но его ограничение на подполе либо содержит мало кодовых слов, либо характеризуется плохой дистанционной структурой. В данном параграфе рассматриваются иные пути увеличения минимального расстояния при другом способе ограничения кода Рида—Соломона на подполе.

Альтернантные коды представляют собой класс линейных кодов, которые строятся из кодов БЧХ таким образом, чтобы при фиксированной скорости получить (хотя бы в принципе) большее минимальное расстояние. Пусть $n = q^m - 1$. Выберем и зафиксируем над $GF(q^m)$ n-мерный вектор $\mathbf{h}$ с ненулевыми компонентами и назовем его *шаблоном* (во *временно́й области*). Выберем также код Рида—Соломона над $GF(q^m)$ с конструктивным расстоянием $2t + 1$. Альтернативный код состоит из всех $GF(q)$-значных векторов $\mathbf{c}$, таких, что вектор с компонентами $c_i' = h_i c_i$, $i = 0, \ldots, n - 1$, является словом кода Рида — Соломона.

Чтобы определить этот же код иначе, допустим, что все h_i отличны от нуля, и положим $g_i = h_i^{-1}$. Для каждого кодового слова $\mathbf{c}'$ кода Рида—Соломона образуем вектор с компонентами $c_i = g_i c_i'$, $i = 0, \ldots, n - 1$. Если вектор $\mathbf{c}$ является $GF(q)$-значным, то он принадлежит альтернантному коду. Альтернантный код определяется как множество всех $GF(q)$-значных слов, которые могут быть получены таким способом.

Обычно выбирается шаблон $\mathbf{h}$, все компоненты которого отличны от нуля; но если какие-то компоненты шаблона равны нулю, то и кодовое слово содержит нуль в этих компонентах. Нулевые компоненты кодового слова не содержат никакой информации и просто не передаются. Если алгоритм декодирования основан на полном слове, то в случае необходимости декодер может восстановить эти пропущенные нулевые компоненты.

При надлежащем выборе шаблона альтернантные коды обладают очень большим истинным минимальным расстоянием; при больших длинах они по существу так же хороши, как и любые известные хорошие коды. К сожалению, для больших n не известны правила выбора хороших шаблонов, хотя, как будет показано в следующем параграфе, хороших шаблонов достаточно много.

Определение альтернантных кодов легко переносится в частотную область. Предположим, что все компоненты вектора $\mathbf{h}$ отличны от нуля. Обозначим через $\mathbf{H}$ преобразование вектора $\mathbf{h}$ и назовем его *шаблоном в частотной области*. Поскольку $h_i c_i$ при $i = 0, \ldots, n - 1$ являются компонентами слова кода Рида—Соломона, то циклическая свертка $\mathbf{H} * \mathbf{C}$ дает спектр слова кода Рида—Соломона. Таким образом,

$$\sum_{k=1}^{n-1} H_{((j-k))} C_k = 0, \quad j = j_0, \ldots, j_0 + 2t - 1.$$

Так как все компоненты $\mathbf{h}$ отличны от нуля, то вектор $\mathbf{H}$ обратим, т. е. существует вектор $\mathbf{G}$ (преобразование вектора $\{g_i = h_i^{-1}, i = 0, \ldots, n-1\}$), такой, что $\mathbf{H} * \mathbf{G}$ представляет собой дельта-функцию. (Если $j = 0$, то $(\mathbf{H} * \mathbf{G})_j = 1$; в противном случае $(\mathbf{H} * \mathbf{G})_j = 0$.)

Через многочлены эта свертка записывается так:

$$H(x)\, G(x) = 1 \pmod{x^n - 1}.$$

Если $H(x)$ — многочлен над малым полем $GF(q)$, то и $G(x)$ — многочлен над этим же полем. Для примитивных n это доказывается следующим образом. Многочлен $H(x)$ не имеет корней в поле $GF(q^m)$, так как $H(\alpha^{-i}) = nh_i \neq 0$. Следовательно, $H(x)$ взаимно прост с $x^n - 1 = x^{q^m-1} - 1$, и, согласно алгоритму Евклида, над $GF(q)$ существуют такие $G(x)$ и $F(x)$, что

$$H(x)\, G(x) + (x^n - 1)\, F(x) = 1.$$

Следовательно,

$$H(x)\, G(x) = 1 \pmod{x^n - 1}.$$

Альтернантные коды в частотной области можно определить следующим образом.

Определение 8.6.1. Пусть $\mathbf{H}$ — фиксированный n-мерный вектор в частотной области, и пусть j_0 и t — фиксированные целые числа. Альтернантный код $\mathcal{C}$ (в частотной области) определяется как множество, содержащее каждый вектор $\mathbf{c}$, преобразование $\mathbf{C}$ которого удовлетворяет следующим двум условиям:

1) $\sum_{k=0}^{n-1} H_{((j-k))} C_k = 0, \quad j = j_0, \ldots, j_0 + 2t - 1,$

и

2) $C_k^q = C_{((qk))}$.

Первое из этих условий накладывает ограничение на свертку, которая в обычном определении во временной области задается в виде произведения многочленов; второе условие гарантирует $GF(q)$-значность кодового слова во временной области. Вектор с компонентами:

$$T_j = \sum_{k=0}^{n-1} H_{((j-k))} C_k, \quad j = 0, \ldots, n-1,$$

называется *профильтрованным спектром*.

Из тесной взаимосвязи альтернантных кодов с кодами Рида—Соломона, очевидно, следует, что их минимальное расстояние не меньше конструктивного расстояния $2t + 1$. Следующая теорема показывает, что их размерность также удовлетворяет условию $k \geqslant n - 2tm$.

Теорема 8.6.2. *Пусть $\mathcal{C}$ представляет собой линейный (n, K, D)-код над $GF(q^m)$, а $\mathcal{C}'$ является его ограничением на подполе $GF(q)$ с параметрами (n, k, d). Тогда*

$$D \leqslant d \leqslant n \quad и \quad (n-K) \leqslant (n-k) \leqslant m(n-K).$$

Доказательство. Нетривиальным является только последнее неравенство. Произвольное проверочное уравнение над $GF(q^m)$, которому удовлетворяет код, приводит не более чем к m линейно независимым проверочным уравнениям над $GF(q)$. Отсюда и вытекает последнее неравенство. □

Следствие 8.6.3. *Размерность k альтернантного кода с конструктивным расстоянием $2t+1$ удовлетворяет неравенству $k \geqslant n - 2tm$.*

Доказательство. Использовать теорему 8.6.2 и равенство $n - K = 2t$. □

В следующем параграфе мы покажем, что минимальное расстояние некоторых альтернантных кодов намного больше конструктивного расстояния, но приведенное там доказательство не является конструктивным.

Поучительно вывести границу БЧХ в частотной области для дистанционной структуры альтернантных кодов, наследуемой из кодов Рида—Соломона.

Теорема 8.6.4. *Если вектор* $\mathbf{c}$ *содержит не более $d-1$ ненулевых компонент и если профильтрованный спектр равен нулю в некоторых $d-1$ последовательных компонентах $(T_k = 0,\ k = k_0, \ldots, k_0 + d - 2)$, то $c_i = 0$ для всех i, где $\mathbf{T} = \mathbf{H} * \mathbf{C}$, а $\mathbf{H}$ — обратимый фильтр.*

Доказательство. Многочлен локаторов $\Lambda(x)$ был определен таким образом, что компоненты λ_i его преобразования равны нулю, если $c_i \neq 0$. Тогда $\lambda_i c_i = 0$ и отсюда следует, что $\mathbf{\Lambda} * \mathbf{C} = \mathbf{0}$. Следовательно, $\mathbf{\Lambda} * (\mathbf{H} * \mathbf{C}) = \mathbf{H} * (\mathbf{\Lambda} * \mathbf{C}) = \mathbf{0}$. Но $\mathbf{\Lambda}$ отличен от нуля только на блоке, длина которого не превышает $d-1$, а $\mathbf{H} * \mathbf{C}$ равно нулю на блоке длины $d-1$. Поэтому $\mathbf{H} * \mathbf{C} = 0$, и, таким образом, $\mathbf{C} = \mathbf{0}$. Следовательно, $\mathbf{c}$ — нулевой вектор.

8.7. ХАРАКТЕРИСТИКИ АЛЬТЕРНАНТНЫХ КОДОВ

Привлекательность альтернантных кодов объясняется тем, что в этом классе имеются хорошие коды большой длины. Под этим подразумевается, что существует последовательность кодов возрастающей длины, для которых скорость k/n и нормированное минимальное расстояние d^*/n отделены от нуля при бесконечном

возрастании n. Докажем это. Метод доказательства сводится к демонстрации того, что число слов малого веса над $GF(q)$ невелико и что каждое из них не может принадлежать слишком большому числу альтернантных кодов. А так как альтернантных кодов достаточно много, то какой-то из них не может содержать слов малого веса. Следовательно, этот код должен обладать большим минимальным расстоянием.

В доказательстве теоремы не указываются значения параметров k и d^* для кода, а только даются их нижние границы. Для удобства доказательства данной теоремы под (n, k, d)-кодом понимается код длины n, размерность которого *не меньше* k и минимальное расстояние которого *не меньше* d^*.

Теорема 8.7.1. *Пусть* $n = q^m - 1$ *для некоторых* $GF(q)$ *и* m, *и пусть* d *и* r — *некоторые целые числа, удовлетворяющие неравенству*

$$\sum_{j=1}^{d-1} \binom{n}{j} (q-1)^j < (q^m - 1)^r.$$

Тогда существует альтернантный $(n, n - mr, d)$*-код.*

Доказательство. Идея приводимого ниже доказательства состоит в подсчете числа альтернантных кодов некоторого определенного вида и последующем нахождении доли тех кодов, которые содержат заданный вектор $\mathbf{v}$ веса $j < d$. Оказывается, что векторов такого веса меньше, чем кодов рассматриваемого вида, так что некоторые из этих кодов не содержат векторов $\mathbf{v}$ веса $j < d$.

(i) Пусть $\mathcal{C}$ — код Рида—Соломона над $GF(q^m)$ с конструктивным расстоянием $r + 1$ и $K = n - r$ информационными символами, и пусть $\mathcal{C}(\mathbf{h})$ — альтернантный код над $GF(q)$, порождаемый из $\mathcal{C}$ шаблоном $\mathbf{h}$, где $\mathbf{h}$ — вектор над $GF(q^m)$, все компоненты которого отличны от нуля. Тогда

$$\mathcal{C}(\mathbf{h}) = \{\mathbf{c} \in GF^n(q) : \mathbf{hc} \in \mathcal{C}\},$$

где $\mathbf{hc}$ — вектор с компонентами $\{h_i c_i,\ i = 0, \ldots, n-1\}$. Поскольку $h_i \neq 0$, то всего существует $(q^m - 1)^n$ таких кодов. Каждый из них представляет собой ограничение линейного кода $\{\mathbf{c} \in GF^n(q^m) : \mathbf{hc} \in \mathcal{C}\}$ на подполе $GF(q)$, и, следовательно, согласно теореме 8.6.2, для каждого такого кода $k \geqslant n - mr$.

(ii) Выберем над $GF(q)$ вектор $\mathbf{v}$ веса $j < d$. Всего имеется $\binom{n}{j}(q-1)^j$ векторов веса j и

$$\sum_{j=1}^{d-1} \binom{n}{j} (q-1)^j$$

векторов веса меньше d.

(iii) Так как любые $n - r$ компонент кода Рида—Соломона полностью определяют кодовый вектор, то вектор $\mathbf{v}$ веса j может принадлежать не более чем $(q^m - 1)^{n-r}$ определенным в п. (i) альтернантным кодам. Действительно, при фиксированном $\mathbf{v}$ в векторе $\mathbf{h}$ можно независимо выбрать только $n - r$ компонент так, чтобы вектор $\mathbf{hv}$ принадлежал $\mathcal{C}$.

(iv) Теперь скомбинируем п. (i), (ii) и (iii). Согласно (i), число альтернантных кодов равно $(q^m - 1)^n$. Максимальное число кодов, которые могут содержать вектор веса меньше d, равно $(q^m - 1)^{n-r} \sum_{j=1}^{d-1} \binom{n}{j} (q-1)^j$. Предположим, что

$$(q^m - 1)^n > (q^m - 1)^{n-r} \sum_{j=1}^{d-1} \binom{n}{j} (q-1)^j.$$

Тогда некоторый код с числом информационных символов $n - mr$ имеет минимальное расстояние не меньше d. Это эквивалентно утверждению теоремы. □

Отметим, что, как уже было указано в доказательстве теоремы, альтернантные коды образуют очень большой класс. Для произвольного k и примитивной длины кода число альтернантных (n, k)-кодов над $GF(q)$ равно $(q^m - 1)^n = n^n$. Доказанная теорема только гласит, что некоторые из этих кодов хороши, но не указывает, как их найти.

Пусть q равно 2. Тогда теорема 8.7.1 утверждает существование двоичных альтернантных кодов, удовлетворяющих условию

$$\sum_{j=1}^{d-1} \binom{n}{j} < 2^{n-k}.$$

Граница для левой части этого неравенства, но в противоположную сторону уже была получена в лемме 7.9.2. Ниже (в гл. 14) мы рассмотрим оценки биномиальных коэффициентов $\binom{n}{j}$ по формуле Стирлинга. Применяя оценку из теоремы 14.4.1, имеем

$$\sum_{k=0}^{pn} \binom{n}{k} < \binom{n}{pn} \geqslant 2^{n[H(p)-o(1)]},$$

где член $o(1)$ стремится к нулю с ростом n. Если пренебречь этим членом, то доказанная теорема утверждает, что для достаточно больших n существуют двоичные коды со скоростью R и нормированным минимальным расстоянием d/n, удовлетворяющим условию

$$-(d/n)\log(d/n) - (1 - d/n)\log(1 - d/n) < 1 - R.$$

Очень важно, что это неравенство связывает только величины R и d/n и что оно имеет ненулевые решения. Следовательно, нами

получено сильное утверждение о существовании асимптотически хороших альтернантных кодов.

Полученная форма границы эквивалентна границе, известной под названием *границы Гилберта*, которая была выведена на много лет раньше, чем были построены альтернантные коды. Граница Гилберта утверждает существование хороших (в данном понимании) кодов, и, таким образом, альтернантные коды образуют класс кодов, реализующих это утверждение. Ситуация такова, что в настоящее время нет каких-либо определенных свидетельств существования двоичных кодов, асимптотически лучших, чем предписывает граница Гилберта. Следовательно, очень правдоподобно, что асимптотически оптимальные коды принадлежат классу альтернантных кодов. Однако класс альтернантных кодов весьма обширен, и, чтобы реализовать приведенную в теореме 8.7.1 границу, необходимо разработать конструктивные методы выделения хороших кодов. Хотя, согласно указанной в теореме 8.7.1 границе, альтернантные коды лучше введенных в § 7.9 кодов Юстесена, тем не менее коды Юстесена задаются точной конструкцией, а как точно указать хороший альтернантный код большой длины, пока еще неизвестно.

8.8. КОДЫ ГОППЫ

Коды Гоппы представляют собой частный подкласс альтернантных кодов, открытый раньше, чем класс альтернантных кодов, но все еще заслуживающий отдельного внимания.

Определение 8.8.1. *Кодом Гоппы* с конструктивным расстоянием d называется альтернантный код с конструктивным расстоянием d, у которого обращение частотного шаблона $\mathbf{G}$ имеет ширину d. Иначе говоря, обращение частотного шаблона задается многочленом $G(x)$ степени $d-1$, который называется *многочленом Гоппы*. *Кодом Гоппы в узком смысле* называется код Гоппы с $2t$ проверочными частотами с локаторами α^{n-2t+1}, α^{n-2t+2}, ..., α^0.

Теорема 8.8.2. *Вектор* $\mathbf{c}$ *является кодовым для кода Гоппы с многочленом Гоппы* $G(x)$ *тогда и только тогда, когда*

$$\sum_{i=0}^{n-1} c_i [\alpha^{ij}/G(\alpha^{-i})] = 0, \quad j = 0, \ldots, 2t-1.$$

Доказательство вытекает непосредственно из теоремы о свертке. □

Теорема 8.8.3. *Минимальное расстояние* d^* *и размерность* k *кода Гоппы с многочленом Гоппы степени* $2t$ *удовлетворяют неравенствам* $d^* \geqslant 2t+1$, $k \geqslant n - 2tm$.

Доказательство вытекает непосредственно из следствия 8.6.3. □

Как следует из определения 8.8.1, длинами кодов Гоппы могут быть только те числа, которые являются делителями $q^m - 1$. Однако все укорочения и удлинения кодов Гоппы также принято называть кодами Гоппы. Для построения кодов Гоппы с длинами, равными q^m и $q^m + 1$, можно использовать описанную в § 8.4 процедуру удлинения кода, согласно которой с каждой стороны кодового слова дописывается по одному q-ичному символу. В принципе можно строить и более длинные коды Гоппы, приписывая с каждой стороны кодового слова q^m-ичные представления символов так, как это делалось в § 8.5 при расширении кодов БЧХ.

Будучи подклассом класса альтернантных кодов, класс кодов Гоппы сохраняет то свойство, что он содержит много кодов, минимальное расстояние которых много больше d [1]). Однако так же, как и в общем случае альтернантных кодов, мало что известно о построении хороших кодов Гоппы. Аналогично в случае общих кодов Гоппы не известны ни хорошие алгоритмы кодирования, ни хорошие алгоритмы декодирования, реализующие минимальное расстояние кода. Известные коды Гоппы интересны прежде всего тем, что позволяют дополнить код БЧХ в узком смысле ($j_0 = 1$) еще одним информационным символом, а другие коды расширить даже на большее число символов. Иначе говоря, известные хорошие коды Гоппы хороши не в смысле описываемых теоремой 8.7.1 возможностей, а в некотором ином смысле.

В частотной области коды Гоппы допускают описание с помощью изображенного на рис. 8.11 устройства с регистром сдвига. Вместо стандартного введения информации во временну́ю область информация вводится в частотную область — или в спектр или в профильтрованный спектр, как показано на рис. 8.11, причем в обоих случаях необходимо обеспечить выполнение ограничений сопряженности. Какие-либо общие методы практической реализации таких ограничений не известны, но для кодов умеренной мощности можно составить и решить систему алгебраических уравнений, описывающих эти ограничения. Ниже приводится пример такого решения. Содержащий информацию профильтрованный спектр пропускается через фильтр с конечным импульсным откликом, веса в отводах которого задаются многочленом Гоппы. Фильтр работает циклически: обращение ко входу является пери-

---

[1]) Это утверждение, конечно, не следует из того, что коды Гоппы являются подклассом класса альтернантных кодов, а нуждается в отдельном доказательстве; такое доказательство можно найти, например, в статье: Гоппа В. Д. На неприводимых кодах достигается пропускная способность ДСК. — Проблемы передачи информации, 1974, вып. 1, с. 111—112 и в монографии: Мак-Вильямс Ф. Дж., Слоэн Н. Дж. А. Теория кодов, исправляющих ошибки.— М.: Связь, 1979, с. 111—112. — *Прим. перев.*

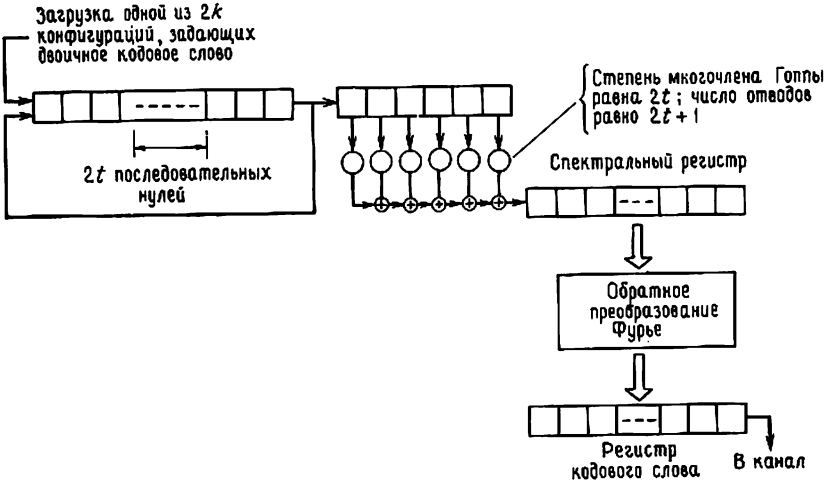


Рис. 8.11. Частотный кодер для кода Гоппы.

одическим. После этого кодовое слово получается обратным преобразованием Фурье спектра.

Другое описание кодов Гоппы дается следующей теоремой.

Теорема 8.8.4. *Код Гоппы в узком смысле над GF (q) длины* $n = q^m - 1$, *задаваемый многочленом Гоппы* $G(x)$, *состоит из всех векторов* $\mathbf{c} = (c_0, \ldots, c_{n-1})$ *над GF (q), удовлетворяющих условию*

$$\sum_{i=0}^{n-1} c_i \prod_{i' \neq i} (x - \alpha^{-i'}) = 0 \,(\mathrm{mod}\, G(x)).$$

Доказательство. Перепишем условие теоремы в виде

$$\sum_{i=0}^{n-1} c_i \prod_{i' \neq i} (x - \alpha^{-i'}) = A(x)\, G(x).$$

Так как степень многочлена $G(x)$ равна $2t$, а степень многочлена в левой части этого равенства не превосходит $n - 1$, то степень многочлена $A(x)$ не превосходит $n - 2t - 1$. Таким образом, $A_j = 0$ при $j = n - 2t, \ldots, n - 1$. Рассмотрим это неравенство в точке $x = \alpha^{-i}$:

$$c_i \prod_{i' \neq i} (\alpha^{-i} \quad \alpha^{-i'}) = A(\alpha^{-i})\, G(\alpha^{-i}),$$

или

$$c_i \left[(\alpha^{-i})^{n-1} \prod_{i' \neq i} (1 - \alpha^{-(i'-i)}) \right] = A(\alpha^{-i})\, G(\alpha^{-i}),$$

или

$$c_i\alpha^i \prod_{k\neq 0} (1-\alpha^k) = A(\alpha^{-i})\,G(\alpha^{-i}).$$

Определим теперь

$$T(x) = \prod_{k\neq 0} (1-\alpha^k)^{-1}\, xA(x) \pmod{x^n-1}$$

так, что

$$T(\alpha^{-1}) = \prod_{k\neq 0} (1-\alpha^k)^{-1}\,[\alpha^{-i}A(\alpha^{-1})]$$

и

$$c_i = T(\alpha^{-1})\,G(\alpha^{-1}).$$

Наконец, заметим, что так как $A_j = 0$ при $j = n-2t, \ldots, n-1$, то $T_j = 0$ по модулю n при $t = n-2t+1, \ldots, n$, и, таким образом, условие теоремы эквивалентно условию, определяющему код Гоппы в узком смысле:

$$C(x) = T(x)\,G(x) \pmod{x^n-1},$$

что и завершает доказательство теоремы. □

Описываемая теоремой 8.8.4 форма задания кода Гоппы допускает удлинение кода на один символ — до кода длины q^m. Надо просто добавить одну компоненту, приписывая ей локатор, равный нулевому элементу поля. Таким образом мы получаем следующее альтернативное определение.

Определение 8.8.5. Код Гоппы над $GF(q)$ длины $n = q^m$, задаваемый многочленом Гоппы $G(x)$, является множеством всех векторов $\mathbf{c} = (c_0, \ldots, c_{n-1})$ над $GF(q)$, удовлетворяющих условию

$$\sum_{i=0}^{n-1} c_i \prod_{i'\neq i} (x-\beta_{i'}) = 0 \pmod{G(x)},$$

где $\beta_{i'}$ пробегает все элементы поля $GF(q^m)$.

Вернемся теперь к частному случаю двоичных кодов Гоппы, ограничиваясь только случаем, когда многочлен Гоппы не имеет кратных корней ни в одном расширении. Такие коды Гоппы называются *сепарабельными*. Как мы увидим, минимальное расстояние сепарабельных двоичных кодов Гоппы равно $2r+1$, где r — степень многочлена Гоппы. Это намного лучше, чем граница для произвольных кодов Гоппы, согласно которой $d^* \geqslant r+1$. Ключом к доказательству этого утверждения является следующая теорема, накладывающая ограничения на многочлен, взаимный

к локаторному многочлену. Многочлен, взаимный к локаторному, определяется равенством

$$\tilde{\Lambda}_c(x) = \sum_{i=1}^{\nu} (x - \beta_i),$$

где β_i — локатор i-й единицы в кодовом слове c, а ν — число единиц в кодовом слове.

Теорема 8.8.6. *Если определяющий двоичный код Гоппы в узком смысле многочлен Гоппы $G(x)$ не имеет корней в поле $GF(2^m)$, то вектор c является кодовым словом тогда и только тогда, когда формальная производная $\tilde{\Lambda}_c'(x)$ многочлена, взаимного к локаторному, делится на $G(x)$.*

Доказательство. Формальная производная многочлена, взаимного к локаторному, равна

$$\tilde{\Lambda}_c'(x) = \sum_{i=1}^{\nu} \prod_{i' \neq i} (x - \beta_{i'}).$$

Поскольку код является двоичным, то c_i равны нулю или единице, так что это выражение для $\tilde{\Lambda}_c'(x)$ следует из формулы, приведенной в теореме 8.8.4. □

Заметим, что в любом расширении поля $GF(2)$ формальная производная произвольного многочлена является полным квадратом, так как нечетные степени в ней пропадают. Предположим, что имеется сепарабельный код Гоппы с многочленом $G(x)$. Тогда не только $G(x)$ делит $\tilde{\Lambda}_c'(x)$, но и $G^2(x)$ должен делить $\tilde{\Lambda}_c'(x)$, так как $\tilde{\Lambda}_c'(x)$ является полным квадратом. В самом деле, мы получим тот же самый код, если в качестве многочлена Гоппы выберем $\overline{G}(x) = G^2(x)$ вместо $G(x)$. Но степень этого многочлена равна $2r$, и, таким образом, $d^* \geqslant 2r + 1$.

Хотя минимальное расстояние сепарабельного кода Гоппы не меньше $2r + 1$, определение 8.8.1 вводит только r, а не $2r$ синдромов, и обычные методы декодирования непосредственно не применимы. Для данного случая можно было бы предложить специальный вариант алгоритма декодирования, использующего только r синдромов. Вместо этого мы модифицируем описание кода так, чтобы были непосредственно пригодны описанные в гл .7 и 9 алгоритмы декодирования. Это можно сделать, не меняя самого кода, а только описывая его другим образом: вместо $G(x)$ в качестве многочлена Гоппы надо выбрать $G^2(x)$. Получится тот же самый код, но границы для характеристик преобразуются к следующему виду:

$$d^* \geqslant \bar{r} + 1 = 2r + 1$$

и

$$k \geqslant n - m\bar{r}/2 = n - mr,$$

где $\bar{r}$ — степень многочлена $G^2(x)$. Теперь для того же кода и тех же характеристик мы имеем $2r$ проверочных частот, и можно непосредственно использовать все известные алгоритмы декодирования.

Наименьшим кодом Гоппы является двоичный (8, 2, 5)-код Гоппы. Выберем $G(x) = x^2 + x + 1$. Корни этого многочлена различны и лежат в $GF(4)$ или в некотором расширении $GF(4)$ и, следовательно, не могут лежать в $GF(8)$. Таким образом, $G(x)$ может быть выбран для построения кода Гоппы длины 8 по меньшей мере с двумя информационными символами и с минимальным расстоянием, равным по меньшей мере 5. Ниже мы увидим, что число информационных символов равно в точности двум. Для описания этого кода сначала воспользуемся определением 8.8.1, а затем дадим его описание в частотной области, основанное на теореме 8.8.2.

Так как $H(x)\,G(x) + x\,(x^7 - 1) = 1$ и $G(x) = x^2 + x + 1$, то $H(x) = x^6 + x^5 + x^3 + x^2 + 1$. Далее, $\overline{G}(x) = x^4 + x^2 + 1$ и $\overline{H}(x) = x^6 + x^5 + x^4 + x^3 + 1$. Многочлены $H(x)$ и $\overline{H}(x)$ можно вычислить по многочленам $G(x)$ и $\overline{G}(x)$ при помощи алгоритма Евклида.

Проверочными частотами являются α^{-3}, α^{-2}, α^{-1} и α^0, и выполняются равенства $T_0 = c_+$, $T_{-1} = 0$, $T_{-2} = 0$ и $T_{-3} = 0$. Вместе с уравнением

$$T_k = \sum_{j=0}^{n-1} \overline{H}_{k-j} C_j$$

это дает

$$\begin{aligned} c_+ &= C_0 + C_1 + C_2 + C_3 + C_4, \\ 0 &= C_6 + C_0 + C_1 + C_2 + C_3, \\ 0 &= C_5 + C_6 + C_0 + C_1 + C_2, \\ 0 &= C_4 + C_5 + C_6 + C_0 + C_1. \end{aligned}$$

Кроме того, имеются ограничения сопряженности $C_j^2 = C_{((2j))}$. Любой спектр, удовлетворяющий этим проверочным уравнениям и ограничениям сопряженности, является спектром кодового слова. Воспользуемся ограничениями сопряженности для того, чтобы исключить C_2, C_4, C_5 и C_6:

$$\begin{aligned} c_+ &= C_0 + (C_1 + C_1^2 + C_1^4) + C_3, \\ 0 &= C_0 + (C_1 + C_1^2) + (C_3 + C_3^2), \\ 0 &= C_0 + (C_1 + C_1^2) + (C_3^2 + C_3^4), \\ 0 &= C_0 + (C_1 + C_1^4) + (C_3^2 + C_3^4). \end{aligned}$$

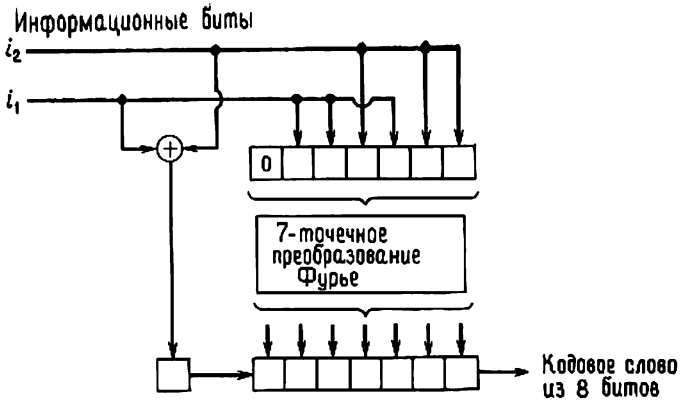


Рис. 8.12. Кодер для (8, 2, 5)-кода Гоппы.

Если положить $C_0 = 0$, в качестве C_1 и C_3 выбрать произвольные элементы поля GF (2), а символ удлинения c_+ взять равным сумме C_1 и C_3, то все уравнения будут удовлетворяться. Таким образом, имеются в точности два информационных символа. Кодер для такого кода в частотной области показан на рис. 8.12.

Согласно теореме 8.8.2, тот же самый код можно получить кодированием во временно́й области. Проверочная матрица **H** кода задается элементами $\alpha^{ij}/G\ (\alpha^{-i})$, так что

$$\mathbf{H} = \begin{bmatrix} 1 & 1 & \alpha^2 & \alpha^4 & \alpha^2 & \alpha & \alpha & \alpha^4 \\ 0 & 1 & \alpha^3 & \alpha^6 & \alpha^5 & \alpha^5 & \alpha^6 & \alpha^3 \end{bmatrix},$$

где первый столбец дописан для построения расширенного (8.2)-кода. Заменяя каждый элемент поля его 3-битовым представлением, получаем

$$\mathbf{H} = \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 \end{bmatrix}.$$

Так как строки этой матрицы линейно независимы то она задает (8,2)-код.

Более сложный пример двоичного кода Гоппы получается при выборе многочлена Гоппы равным многочлену $G\ (x) = x^3 + x + 1$, который имеет три различных корня в GF (8) или в некотором его расширении и, следовательно, не имеет корней в GF (32).

Согласно теореме 8.8.2, выбор в качестве многочлена Гоппы $G(x)$ или $G^2(x)$ приводит к (31,16, 7)-коду Гоппы или расширенному (32, 17, 7)-коду Гоппы. (31, 16, 7)-код Гоппы ничем не лучше (31, 16, 7)-кода БЧХ, за исключением того, что его можно расширить до (32, 17, 7)-кода, тогда как код БЧХ расширить нельзя.

Квадрат многочлена $G(x)$ равен $\overline{G}(x) = x^6 + x^2 + 1$, и обращения многочленов Гоппы соответственно равны

$$H(x) = x^{30} + x^{27} + x^{25} + x^{24} + x^{23} + x^{20} + x^{18} + x^{17} + x^{16} + \\ + x^{13} + x^{11} + x^{10} + x^9 + x^6 + x^4 + x^3 + x^2,$$

$$\overline{H}(x) = x^{29} + x^{26} + x^{23} + x^{22} + x^{20} + x^{19} + x^{18} + x^{17} + x^{15} + \\ + x^{12} + x^9 + x^8 + x^6 + x^5 + x^4 + x^3 + x.$$

В рассматриваемом примере введем информацию непосредственно в спектр. Тогда профильтрованный спектр дается равенствами

$$T_k = \sum_{j=0}^{n-1} \overline{H}_{((k-j))} C_j, \quad k = 0, \ldots, n-1,$$

а проверочными частотами являются частоты с индексами $k = -5, -4, \ldots, 0$. Выписывая эти равенства совместно с граничными синдромами c_+ и c_-, получаем

$$c_+ = C_2 + C_5 + C_8 + C_9 + C_{11} + C_{12} + C_{13} + C_{14} + C_{16} + \\ + C_{19} + C_{22} + C_{23} + C_{25} + C_{26} + C_{27} + C_{28} + C_{30},$$

$$0 = C_1 + C_4 + C_7 + C_8 + C_{10} + C_{11} + C_{12} + C_{13} + C_{15} + \\ + C_{18} + C_{21} + C_{22} + C_{24} + C_{25} + C_{26} + C_{27} + C_{29},$$

$$0 = C_0 + C_3 + C_6 + C_7 + C_9 + C_{10} + C_{11} + C_{12} + C_{14} + \\ + C_{17} + C_{20} + C_{21} + C_{23} + C_{24} + C_{25} + C_{26} + C_{28},$$

$$0 = C_{30} + C_2 + C_5 + C_6 + C_8 + C_9 + C_{10} + C_{11} + C_{13} + \\ + C_{16} + C_{19} + C_{20} + C_{22} + C_{23} + C_{24} + C_{25} + C_{27},$$

$$0 = C_{29} + C_1 + C_4 + C_5 + C_7 + C_8 + C_9 + C_{10} + C_{12} + C_{15} + \\ + C_{18} + C_{19} + C_{21} + C_{22} + C_{23} + C_{24} + C_{26},$$

$$c_- = C_{28} + C_0 + C_3 + C_4 + C_6 + C_7 + C_8 + C_9 + C_{11} + C_{14} + \\ + C_{17} + C_{18} + C_{20} + C_{21} + C_{22} + C_{23} + C_{25}.$$

Любые c_+, c_- и C_j над $GF(32)$, удовлетворяющие условиям сопряженности $C_j^2 = C_{((2j))}$, $j = 0, \ldots, 30$, и выписанным уравнениям,

определяют кодовое слово. Используя условия сопряженности, приведем систему проверочных уравнений к следующему виду:

$$
\begin{aligned}
c_+ &= (C_1^2 + C_1^8 + C_1^{16}) + C_3^4 + (C_5 + C_5^8) + (C_7^2 + C_7^4 + C_7^8 + C_7^{16}) + \\
&\quad + (C_{11} + C_{11}^2 + C_{11}^4 + C_{11}^8) + (C_{15}^2 + C_{15}^8 + C_{15}^{16}), \\
0 &= (C_1 + C_1^4 + C_1^8) + (C_3^4 + C_3^8) + (C_5^2 + C_5^{16}) + (C_7 + C_7^8) + \\
&\quad + (C_{11} + C_{11}^2 + C_{11}^4 + C_{11}^8 + C_{11}^{16}) + (C_{15} + C_{15}^4 + C_{15}^8), \\
0 &= C_0 + (C_3 + C_3^2 + C_3^4 + C_3^8 + C_3^{16}) + (C_5^2 + C_5^4 + C_5^8) + \\
&\quad + (C_7 + C_7^2 + C_7^4 + C_7^8) + (C_{11} + C_{11}^8 + C_{11}^{16}) + C_{15}^{16}, \\
0 &= (C_1^2 + C_1^8 + C_1^{16}) + (C_3^2 + C_3^8) + (C_5 + C_5^2 + C_5^4 + C_5^8) + \\
&\quad + (C_7^8 + C_7^{16}) + (C_{11} + C_{11}^2 + C_{11}^4) + (C_{15}^2 + C_{15}^8 + C_{15}^{16}), \\
0 &= (C_1 + C_1^4 + C_1^8) + (C_3^4 + C_3^8) + (C_5 + C_5^2 + C_5^8 + C_5^{16}) + \\
&\quad + (C_7 + C_7^{16}) + (C_{11}^2 + C_{11}^8 + C_{11}^{16}) + (C_{15} + C_{15}^4 + C_{15}^{16}), \\
c_- &= C_0 + (C_1^4 + C_1^8) + (C_3 + C_3^2 + C_3^{16}) + (C_5^4 + C_5^8 + C_5^{16}) + \\
&\quad + (C_7 + C_7^2 + C_7^4 + C_7^8) + (C_{11} + C_{11}^2 + C_{11}^{16}) + C_{15}^{16}.
\end{aligned}
$$

Рассмотрим решение этой системы во всех подробностях. Сначала возведем в квадрат третье уравнение и разрешим его относительно C_{15}:

$$
\begin{aligned}
C_{15} &= C_0 + (C_3 + C_3^2 + C_3^4 + C_3^8 + C_3^{16}) + (C_5^4 + C_5^8 + C_5^{16}) + \\
&\quad + (C_7^2 + C_7^4 + C_7^8 + C_7^{16}) + (C_{11} + C_{11}^2 + C_{11}^{16}).
\end{aligned}
$$

Воспользуемся этим равенством, чтобы исключить C_{15} из остальных пяти уравнений:

$$
\begin{aligned}
c_+ &= C_0 + (C_1^2 + C_1^8 + C_1^{16}) + (C_3 + C_3^2 + C_3^8 + C_3^{16}) + (C_5 + C_5^8 + \\
&\quad + C_5^{16}) + (C_7 + C_7^2 + C_7^8 + C_7^{16}) + (C_{11} + C_{11}^4 + C_{11}^8), \\
0 &= C_0 + (C_1 + C_1^4 + C_1^8) + (C_3 + C_3^2 + C_3^{16}) + (C_5^2 + C_5^8 + C_5^{16}) + \\
&\quad + (C_7 + C_7^2 + C_7^8 + C_7^{16}) + (C_{11}^2 + C_{11}^4 + C_{11}^8 + C_{11}^{16}), \\
0 &= C_0 + (C_1^2 + C_1^8 + C_1^{16}) + (C_3 + C_3^4 + C_3^{16}) + \\
&+ (C_5 + C_5^2 + C_5^4 + C_5^8 + C_5^{16}) + (C_7 + C_7^4 + C_7^8 + C_7^{16}) + (C_{11} + C_{11}^4), \\
0 &= C_0 + (C_1 + C_1^4 + C_1^8) + (C_3 + C_3^2 + C_3^{16}) + (C_5^2 + C_5^8 + C_5^{16}) + \\
&\quad + (C_7 + C_7^2 + C_7^8 + C_7^{16}) + (C_{11}^2 + C_{11}^4 + C_{11}^8 + C_{11}^{16}), \\
c_- &= (C_1^4 + C_1^8) + (C_3^4 + C_3^8) + (C_5^2 + C_5^{16}) + (C_{11}^2 + C_{11}^8).
\end{aligned}
$$

Четвертое уравнение совпадает со вторым и может быть отброшено. Сложим второе уравнение, третье уравнение и возведенное в квадрат третье уравнение. Это дает

$$C_1 = C_0 + (C_3 + C_3^2 + C_3^8) + (C_5^2 + C_5^8 + C_5^{16}) + \\ + (C_7 + C_7^8) + (C_{11}^2 + C_{11}^{16}).$$

Подставляя это значение для C_1, получаем следующие четыре уравнения:

$$c_+ = C_3^2 + (C_5 + C_5^2 + C_5^4 + C_5^{16}) + (C_7 + C_7^2 + C_7^4 + C_7^{16}) + \\ + (C_{11} + C_{11}^4 + C_{11}^{16}),$$

$$0 = (C_3^4 + C_3^8) + (C_5 + C_5^{16}) + (C_7 + C_7^{16}) + \\ + (C_{11} + C_{11}^4 + C_{11}^8 + C_{11}^{16}),$$

$$0 = (C_3 + C_3^2 + C_3^4 + C_3^8) + (C_5 + C_5^4 + C_5^8 + C_5^{16}) + \\ + (C_7 + C_7^{16}) + (C_{11} + C_{11}^4 + C_{11}^8 + C_{11}^{16}),$$

$$c_- = (C_3 + C_3^2 + C_3^8 + C_3^{16}) + (C_5 + C_5^2 + C_5^4 + C_5^8) + \\ + (C_7 + C_7^2 + C_7^4 + C_7^8) + (C_{11}^4 + C_{11}^{16}).$$

Возведем последнее уравнение в четвертую степень и сравним результат со вторым уравнением. Это показывает, что $c_- = 0$ и что четвертое уравнение можно отбросить. Прибавим восьмую степень третьего уравнения к третьему уравнению. Это дает в точности второе уравнение, так что его можно отбросить. Итак, остались два уравнения:

$$c_+ = C_3^2 + (C_5 + C_5^2 + C_5^4 + C_5^{16}) + (C_7 + C_7^2 + C_7^4 + C_7^{16}) + \\ + (C_{11} + C_{11}^4 + C_{11}^{16}),$$

$$0 = (C_3^4 + C_3^8) + (C_5 + C_5^{16}) + (C_7 + C_7^{16}) + \\ + (C_{11} + C_{11}^4 + C_{11}^8 + C_{11}^{16}).$$

Возводя первое уравнение в шестнадцатую степень, получаем

$$C_3 = c_+^{16} + (C_5 + C_5^2 + C_5^8 + C_5^{16}) + (C_7 + C_7^2 + C_7^8 + C_7^{16}) + \\ + (C_{11}^2 + C_{11}^8 + C_{11}^{16}).$$

Подстановка этого выражения в последнее уравнение дает

$$0 = c_+^2 + c_+^4.$$

Возведение этого равенства в шестнадцатую степень приводит к уравнению

$$0 = c_+ (1 + c_+),$$

из которого следует, что c_+ равно нулю или единице.

Итак, мы пришли к следующему правилу: C_0 и c_+ являются произвольными элементами поля GF (2); C_5, C_7 и C_{11} являются произвольными элементами поля GF (32); C_1, C_3 и C_{15} определяются следующими уравнениями:

$$C_1 = c_+ + C_0 + C_5^8 + (C_7 + C_7^2 + C_7^8 + C_7^{16}) + (C_{11} + C_{11}^2 + C_{11}^8),$$

$$C_3 = c_+ + (C_5 + C_5^2 + C_5^8 + C_5^{16}) + (C_7 + C_7^2 + C_7^8 + C_7^{16}) + (C_{11}^2 + C_{11}^8 + C_{11}^{16}),$$

$$C_{15} = c_+ + C_0 + (C_5^4 + C_5^8 + C_5^{16}) + (C_7^2 + C_7^4 + C_7^8 + C_7^{16}) + (C_{11}^4 + C_{11}^8).$$

Все остальные C_j находятся из условий сопряженности. Обратное преобразование Фурье дает 31-компонентное кодовое слово во временно́й области, к которому в виде хвоста дописывается символ c_+. Построенный код является расширенным (32, 17, 7)-кодом Гоппы. Так как ограничения сопряженности привели к равенству $c_- = 0$, то второго символа при расширении добавить не удается.

8.9. КОДЫ ПРЕПАРАТЫ

Хороших классов нелинейных кодов известно мало, и среди них находится класс кодов Препараты. Начнем изучение этих кодов с построения в частотной области одного примера, а именно (15, 8)-кода Препараты.

Но прежде чем изучать коды Препараты, вернемся к рассмотренной в § 5.6 процедуре декодирования двоичных кодов БЧХ, исправляющих две ошибки; в этом случае взаимный многочлен локаторов ошибок выписывается явно; за исключением случая $S_1 = 0$, он равен

$$\tilde{\Lambda}(x) = x^2 + S_1 x + S_1^{-1}(S_1^3 + S_3) = 0.$$

Частный случай $S_1 = 0$ соответствует случаю отсутствия ошибок и легко проверяется. Если положить $0/0 = 0$, то многочлен локаторов ошибок будет включать и случай отсутствия ошибок, и в этом случае уравнение вырождается в уравнение $x^2 = 0$ и имеет корни $X_1 = 0$ и $X_2 = 0$. В случае одной ошибки уравнение принимает вид

$$x^2 + S_1 x = 0$$

и имеет корни $X_1 = 0$ и $X_2 = S_1$.

Сделаем замену переменных, полагая $x = S_1 z$. Тогда новый многочлен, который мы также обозначим через $\tilde{\Lambda}$, будет иметь вид

$$\tilde{\Lambda}(z) = z^2 + z + (1 + S_1^{-3} S_3).$$

Этот многочлен имеет корни в поле $GF(2^m)$ тогда и только тогда, когда $\mathrm{tr}\,(1 + S_1^{-3} S_3)$ равен нулю. В четных расширениях GF (4) всегда $\mathrm{tr}\,(1) = 0$, и это условие преобразуется так:

$$\mathrm{tr}\,(S_1^{-3} S_3) = 0.$$

Далее, в полях характеристики 2, согласно теореме 8.2.3, ровно половина элементов имеет равный нулю след. При каждом значении S_1 элемент S_3 может принимать только 2^{m-1} значений, которые образуют в $GF(2^m)$ подпространство размерности $m - 1$. Один бит в S_3 «не используется». На первый взгляд может показаться, что на частоте C_3 кодового слова можно ввести дополнительный бит информации. Это невозможно, так как неиспользуемый бит элемента S_3 зависит от S_1. Но коды Препараты позволяют лучше распорядиться этим обстоятельством: если спектр лежит в расширении поля GF (4), то можно переупорядочить другие информационные биты так, чтобы ввести дополнительный информационный бит.

$(2^m - 1, 2^m - 2m, 5)$-код Препараты над GF (2) существует для каждого четного m, большего 2. По сравнению с соответствующим кодом БЧХ, исправляющим две ошибки, код Препараты всегда содержит один дополнительный информационный бит. Мы рассмотрим только (15, 8, 5)-код Препараты.

Определение 8.9.1. Двоичный (15,8)-код Препараты определяется[1]) как множество двоичных слов длины 15, спектр которых удовлетворяет следующим ограничениям: $C_1 = 0$, $C_3 = A$, $C_5 = B$, где либо

$$1)\ A \in \{1, \alpha^3, \alpha^6, \alpha^9, \alpha^{12}\} \text{ и } B = 0,$$

либо

$$2)\ B \in \{1, \alpha^5, \alpha^{10}\} \text{ и } A = 0,$$

и, кроме того, выполняются условия сопряженности.

Отметим, что, согласно условиям сопряженности, $C_5^4 = C_5$, так что C_5 может быть только элементом поля GF (4). Так как C_0 представляет собой произвольный элемент поля GF (2), а C_7 произвольный элемент поля GF (16), то C_0 и C_7 вместе содержат

[1]) При таком определении нулевое слово не является кодовым. В других определениях используется такой сдвиг кода, при котором нулевое слово включается в код.

пять битов информации. Остальные три бита описываются восемью выборами A и B, так что код является (15, 8)-кодом.

Покажем теперь, как исправляются двойные ошибки, и тем самым докажем, что минимальное расстояние этого кода равно по меньшей мере 5. Пусть V_j, $j = 0, \ldots, n-1$, представляют собой спектральные компоненты принятого слова. Определим синдромы

$$S_1 = E_1 = V_1,$$
$$S_2 = E_2 = V_2,$$
$$S_3 = E_3 + A = V_3,$$
$$S_4 = E_4 = V_4,$$
$$S_5 = E_5 + B = V_5.$$

Выбор A и B приемнику не известен, но ограничен условиями определения 8.9.1. Далее,

$$S_1 = X_1 + X_2,$$
$$S_3 = X_1^3 + X_2^3 + A,$$
$$S_5 = X_1^5 + X_2^5 + B.$$

Сначала разрешим эти уравнения относительно A и B; затем вычитание A из S_3 сведет задачу к декодированию исправляющего две ошибки кода БЧХ. Простые вычисления дают

$$S_1^3 + (S_3 + A) = X_1X_2S_1,$$
$$(S_3 + A)^2 + S_1(S_5 + B) = X_1X_2S_1^4.$$

Исключая X_1X_2, получаем квадратное уравнение

$$A^2 + S_1^3A + (S_1^3S_3 + S_3^2 + S_1^6 + S_1S_5 + S_1B) = 0.$$

Оно может быть разрешено относительно A и B следующим образом. Сначала положим $B = 0$ и решим получающееся при этом квадратное уравнение относительно A. Затем положим $A = 0$ и решим получающееся линейное уравнение относительно B. Это даст нам три решения для пары (A, B). Из доказательства приведенной ниже теоремы следует, что только одно из них удовлетворяет ограничениям определения 8.9.1.

Теорема 8.9.2. *Минимальное расстояние* (15, 8)*-кода Препараты равно* 5.

Доказательство. Докажем, что минимальное расстояние кода равно по меньшей мере пяти, построив процедуру декодирования, исправляющую две ошибки. Для этого достаточно дать правило нахождения по принятому слову величин A и B, так как после

этого задача сводится к декодированию исправляющего две ошибки кода БЧХ.

Предположим, что произошло не более двух ошибок. Если $S_1 = 0$, то ошибок не произошло. Мы предполагаем, что S_1 было вычислено и оказалось отличным от нуля.

Сначала положим $B = 0$ и найдем корни квадратного относительно A уравнения

$$A^2 + S_1^3 A + (S_1^3 S_3 + S_3^2 + S_1^6 + S_1 S_5) = 0.$$

Удовлетворяющие нашим ограничениям возможные решения для A исчерпываются ненулевыми кубами. Затем положим $A = 0$ и найдем решение линейного относительно B уравнения

$$(S_1^3 S_3 + S_3^2 + S_1^6 + S_1 S_5) + S_1 B = 0.$$

Шаг 1. Сначала покажем, что рассматриваемое квадратное уравнение не может иметь двух решений, которые оба являются ненулевыми кубами. Если A_1 и A_2 — корни данного квадратного уравнения, то $A_1 + A_2 = S_1^3$, так что сумма двух корней также является ненулевым кубом. Необходимо только доказать, что в поле GF (16) сумма двух ненулевых кубов не может быть равна ненулевому кубу. Выпишем множество ненулевых кубов

$$U_3 = \{1, \alpha^3, \alpha^6, \alpha^9, \alpha^{12}\}.$$

Тогда прямое вычисление всех возможных сумм дает множество

$$U_3 + U_3 = \{0, \alpha, \alpha^2, \alpha^4, \alpha^5, \alpha^7, \alpha^8, \alpha^{10}, \alpha^{11}, \alpha^{13}, \alpha^{14}\},$$

которое не содержит ненулевых кубов. Таким образом, в поле GF (16) сумма двух ненулевых кубов не может быть ненулевым кубом, и поэтому A_1 и A_2 не могут одновременно удовлетворять ограничениям определения 8.9.1.

Шаг 2. Предположим теперь, что $(A, 0)$ и $(0, B)$ — два решения исходного уравнения. Покажем, что оба они не могут удовлетворять накладываемым ограничениям. Разность двух уравнений дает

$$A^2 + S_1^3 A + S_1 B = 0.$$

Пусть $A = S_1^3 A'$. Тогда

$$(A')^2 + A' + S_1^{-3} B = 0.$$

Но как S_1^5, так и B являются ненулевыми элементами поля GF (4). Следовательно, элемент A' должен удовлетворять одному из следующих уравнений:

$$(A')^2 + A' + 1 = 0,$$
$$(A')^2 + A' + \alpha^5 = 0,$$
$$(A')^2 + A' + \alpha^{10} = 0.$$

Перебирая все корни этих уравнений, получаем, что

$$A' \in \{\alpha^5, \alpha^{10}, \alpha, \alpha^2, \alpha^4, \alpha^8\}.$$

Но в этом множестве не содержится ни одного куба. Таким образом, $(A, 0)$ и $(0, B)$ не могут одновременно удовлетворять нашим ограничениям. Это завершает доказательство теоремы. □

ЗАДАЧИ

8.1. Исходя из определения пары преобразования Фурье $\{c_i\} \leftrightarrow \{C_j\}$, доказать следующие стандартные свойства дискретного преобразования Фурье.

а. Линейность:

$$\{ac_i + bc_i'\} \leftrightarrow \{aC_j + bC_j'\}.$$

б. Свойство циклического сдвига:

$$\{c_{((i-1))}\} \leftrightarrow \{\alpha^j C_j\}.$$

в. Модульность:

$$\{\alpha^i c_i\} \leftrightarrow \{C_{((j+1))}\}.$$

8.2. Доказать, что преобразование Фурье сигнального вектора $c_i = \alpha^{ri}$ содержит единственную ненулевую спектральную компоненту. Что происходит, если $r = 0$? Доказать, что спектр вектора с единственной ненулевой компонентой всюду отличен от нуля.

8.3. Доказать, что $E_j = F_j G_j$ тогда и только тогда, когда

$$e_i = (1/n) \sum_{k=0}^{n-1} f_{((i-k))} g_k.$$

8.4. Сколько идемпотентов содержится в кольце многочленов по модулю многочлена $x^{15} - 1$? Перечислить их.

8.5. Основная теорема алгебры гласит, что многочлен степени d имеет не более d корней. Используя эту теорему, доказать границу БЧХ для кода Рида—Соломона, спектр $\{C_j\}$ кодовых слов которого удовлетворяет условию $C_j = 0$ для всех $j > n - k$.

8.6.а. Найти порождающий многочлен для двоичного расширенного (42, 25, 7)-кода БЧХ. Построить кодер.

б. Найти порождающий многочлен расширенного (17, 9, 7)-кода БЧХ над $GF(4)$. Построить кодер.

8.7. Пусть $G(x) = x^2 + x + 1$ — многочлен Гоппы для (32, 22, 5)-кода Гоппы. Разработать правило кодирования и процедуру декодирования.

8.8. Пусть $G(x) = x^2 + x + \alpha^3$ — многочлен Гоппы для (16, 8, 5)-кода Гоппы, где α — примитивный элемент поля $GF(16)$. Найти проверочную матрицу кода.

8.9. Модифицируя несистематический кодер для (15, 7)-кода БЧХ, построить кодер для (15, 8)-кода Препараты.

8.10. Код Препараты можно декодировать с помощью алгоритма Берлекэмпа—Месси для двоичных кодов, используя три итерации ($r = 1, 3, 5$) и выбирая A и B так, чтобы $\Delta_5 = 0$. Выписать уравнения для A и B, которые должны быть разрешены при таком алгоритме декодирования.

8.11. Двоичный (63, 52)-код Препараты может быть определен при помощи своего спектра следующим образом: $C_1 = 0$, $C_3 = A$, $C_9 = B$, где либо

1) $A \in \{1, \alpha^7, \alpha^{14}, \alpha^{21}, \alpha^{28}, \alpha^{35}, \alpha^{42}, \alpha^{49}, \alpha^{56}\}$ и $B = 0$, либо

2) $B \in \{1, \alpha^9, \alpha^{18}, \alpha^{27}, \alpha^{36}, \alpha^{45}, \alpha^{54}\}$ и $A = 0$,

а остальные спектральные компоненты удовлетворяют только условиям сопряженности.

а. Доказать, что этим кодом можно закодировать 52 информационных бита.

б. Исследуя уравнение

$$A^4 + S_1^6 A^2 + S_1^9 A + [S_1^3 S_9 + S_3^4 + S_1^6 S_3^2 + S_1^9 S_3] + S_1^3 B = 0$$

и изучая его корни, доказать, что минимальное расстояние рассматриваемого кода не меньше 5.

ЗАМЕЧАНИЯ

Спектральное описание кодов, контролирующих ошибки, можно найти в очень ранних работах, хотя связь с преобразованием Фурье не была сразу осознана. Спектральный декодер использовался в основополагающей работе Рида и Соломона [1960] для нахождения минимального расстояния предлагаемых кодов, но, так как описанный в этой работе декодер не имел практического значения, спектральные методы не изучались еще долгие годы. Мэттсон и Соломон [1961] ввели спектральный многочлен, сыгравший большую роль в теоретических исследованиях, но смысл спектрального многочлена как преобразования Фурье вновь не был осознан, и тесная связь теории корректирующих кодов с обработкой сигналов опять осталась скрытой. Близкие идеи разрабатывал Мандельбаум [1979], но на языке интерполяции.

Преобразование Фурье над конечными полями рассматривал Поллард [1971], а использовать его в теории кодов, контролирующих ошибки, предложил Гор [1973], работа которого получила дальнейшее развитие в работах Ченя и Чоя [1975] и Лемпеля и Винограда [1977]. Приведенное в данной главе доказательство границы БЧХ частично использует перенесенное в частотную область доказательство Ченя [1972].

Идея расширения кода широко известна. Расширенные коды Рида—Соломона рассматривал Вулф [1969]. Конструкции расширения кодов БЧХ были предложены Андриановым и Сасковцом [1966], Слоэном, Редди и Чинем [1972]; Касахарой, Сугиямой, Хирасавой и Намекавой [1975]. Описание конструкций в виде преобразований дал Блейхут [1980].

Альтернантные коды были введены Хельгертом [1974], который дал им такое название потому, что проверочная матрица этих кодов может быть записана в виде так называемой (в математической литературе) альтернантной матрицы. К альтернантным кодам относятся открытые ранее коды Гоппы [1970]. Дельсарт [1975] предложил определять альтернантные коды как ограничения на подполе модифицированных кодов Рида—Соломона, так что его определение значительно отличается от исходного.

Коды Препараты имеют интересную историю. Препарата [1968] открыл этот класс кодов в процессе исследования свойств наименьшего кода в классе, а именно (15, 8, 5)-кода, известного под названием кода Нордстрома—Робинсона [1967]. Два последних автора построили этот нелинейный (15, 8, 5)-код с помощью ЭВМ как расширение ранее построенного Нейдлером нелинейного (12, 5, 5)-кода [1962] или нелинейного (13, 6, 5)-кода Грина [1966]. В свою очередь коды Препараты стимулировали открытие еще более сложных нелинейных кодов, а именно низкоскоростных кодов Кердока [1972] и исправляющих три ошибки кодов Гетелса [1976].

ГЛАВА 9

АЛГОРИТМЫ, ОСНОВАННЫЕ НА СПЕКТРАЛЬНЫХ МЕТОДАХ

Оценивать код, контролирующий ошибки, надо не только по его скорости и минимальному расстоянию, но и по возможности построения для него экономного декодера. Обычно имеется много способов декодирования заданного кода. Конструктору приходится выбирать между несколькими различными алгоритмами декодирования или вариантами одного алгоритма, и он должен быть знаком со всеми возможностями, чтобы выбрать наилучшую для данного конкретного приложения. Выбор зависит не только от параметров кода, таких, как длина и минимальное расстояние, но и от того, какая часть алгоритма реализована аппаратурно, а какая программно, от требуемой скорости декодирования и даже от стоимости имеющихся блоков схемы.

В данной главе изучаются методы декодирования, позволяющие работать в частотной области. Сюда включены методы декодирования с исправлением ошибок и стираний и методы декодирования за пределами конструктивного расстояния кода [1]).

9.1. СПЕКТРАЛЬНЫЕ МЕТОДЫ ДЕКОДИРОВАНИЯ

Вернемся к задаче декодирования кодов БЧХ и будем интерпретировать с частотной точки зрения уже рассмотренные вопросы. Используя преобразование Фурье, разработаем новые процедуры декодирования.

Принятое слово $\mathbf{v}$ с компонентами $v_i = c_i + e_i$, $i = 0, \ldots, n-1$, представляет собой сумму кодового слова $\mathbf{c}$ и вектора ошибок $\mathbf{e}$. Декодер должен так обработать слово $\mathbf{v}$, чтобы удалить из него вектор ошибок $\mathbf{e}$; после этого из $\mathbf{c}$ выделяется информа-

---

[1]) Имеется в виду исправление векторов ошибок, вес которых превышает половину конструктивного расстояния кода. — *Прим. перев.*

ция. Координаты синдрома для зашумленного слова v кода БЧХ задаются следующими равенствами:

$$S_j = \sum_{i=0}^{n-1} \alpha^{i\,(j+j_0-1)} v_i = v\left(\alpha^{j+j_0-1}\right), \quad j = 1, \ldots, 2t.$$

Очевидно, что компоненты синдрома вычисляются как $2t$ компонент преобразования Фурье принятого вектора. Компоненты преобразования Фурье зашумленного кодового слова $\mathrm{v} = \mathrm{c} + \mathrm{e}$ равны $V_j = C_j + E_j$, $j = 0, \ldots, n-1$, а компоненты синдрома представляют собой $2t$ компонент этого спектра от j_0-й до $(j_0 + 2t - 1)$-й. Но, согласно построению кодов БЧХ, спектральные компоненты, расположенные в проверочных частотах (при $j = j_0, \ldots, j_0 + 2t - 1$), равны нулю: $C_j = 0$ для $j = j_0, \ldots, j_0 + 2t - 1$. Следовательно,

$$S_j = V_{j+j_0-1} = E_{j+j_0-1}, \quad j = 1, \ldots, 2t.$$

Блок компонент синдрома как бы образует окно, через которое можно наблюдать $2t$ из n компонент спектра конфигурации ошибок. Но если вес конфигурации ошибок не превышает t, то, согласно границе БЧХ, этих $2t$ компонент синдрома достаточно, чтобы однозначно восстановить вектор ошибок.

Предположим, что произошло $\nu < t$ ошибок с локаторами α^{i_k} при $k = 1, \ldots, \nu$. Многочлен локаторов ошибок равен

$$\Lambda(x) = \prod_{k=1}^{\nu} (1 - x\alpha^{i_k}).$$

Обратное преобразование Фурье вектора Λ вычисляется как значения $\Lambda(\alpha^{-i})$ многочлена $\Lambda(x)$ в точках α^{-i}. Ясно, что $\Lambda(\alpha^{-i})$ равно нулю тогда и только тогда, когда i представляет собой позицию ошибки. Таким образом, $\Lambda(x)$ определяется как такой многочлен, для которого во временно́й области $\lambda_i = 0$ при всех $e_i \neq 0$. Следовательно, $\lambda_i e_i = 0$ для всех i, и, таким образом, по теореме о свертке свертка в частотной области равна нулю:

$$\sum_{j=0}^{n-1} \Lambda_j E_{k-j} = 0, \quad k = 0, \ldots, n-1.$$

Так как степень многочлена $\Lambda(x)$ не превосходит t, то $\Lambda_j = 0$ для $j > t$. Следовательно,

$$\sum_{j=0}^{t} \Lambda_j E_{k-j} = 0, \quad k = 0, \ldots, n-1.$$

Поскольку $\Lambda_0 = 1$, эти уравнения можно переписать в виде

$$E_k = -\sum_{j=1}^{t} \Lambda_j E_{k-j}, \quad k = 0, \ldots, n-1.$$

Эта система содержит n уравнений, связывающих $n - t$ неизвестных (t коэффициентов многочлена $\Lambda(x)$ и $n - 2t$ компонент вектора $\mathbf{E}$) и $2t$ известных компонент вектора $\mathbf{E}$, заданных компонентами синдрома. Таким образом, t уравнений

$$S_k = -\sum_{j=1}^{t} \Lambda_j S_{k-j}, \quad k = t+1, \ldots, 2t,$$

содержит только известные компоненты синдрома и t неизвестных компонент вектора $\mathbf{E}$. Как мы видели в гл. 7, такую систему всегда можно разрешить относительно Λ, например с помощью алгоритма Берлекэмпа—Месси.

Остальные компоненты спектра $\mathbf{S}$ можно получить рекуррентным продолжением, а именно используя выписанное выше уравнение для свертки, вычислить S_{2t+1} по уже известным компонентам $\mathbf{S}$ и Λ, затем найти S_{2t+2} и т. д. Это вычисление может быть выполнено с помощью регистра сдвига с линейной обратной связью, весовые множители в отводах которого совпадают с компонентами вектора Λ, а начальное состояние задается компонентами синдрома $S_1, \ldots, S_t$. Это позволяет вычислить S_j для всех j; E_j равны S_{j-i_0+1}, а $C_j = V_j - E_j$. Обратное преобразование Фурье завершает декодирование.

На рис. 9.1 приведена блок-схема реализации описанной процедуры декодирования (за исключением преобразований Фурье). Такой декодер может быть использован независимо от того, в какой области реализован кодер: в частотной или во временно́й. Если кодер реализован во временно́й области, то для вычисления кодового слова во временно́й области надо воспользоваться обратным преобразованием Фурье применительно к исправленному спектру, а затем уже выделить из этого кодового слова переданную информацию. Если же кодирование производилось в частотной области, то информационные символы определяются непосредственно по исправленному спектру. В этом случае обратного преобразования в декодере делать не надо.

Вычисления можно организовать многими способами. На рис. 9.2 приведены четыре различные схемы декодеров. Первая схема относится к рассмотренному в гл. 7 варианту декодера. Во второй схеме кодирование осуществляется в частотной области, так что как только вычислен исправленный спектр, декодирование закончено. Третья и четвертая схемы иллюстрируют тот факт, что, хотя в частотной области с помощью рекуррентного продолжения вычислена конфигурация ошибок, окончательное исправление ошибок можно производить как во временно́й, так и в частотной областях.

Четвертая схема похожа на первую, хотя их реализации сильно отличаются. Вычисление синдрома аналогично прямому

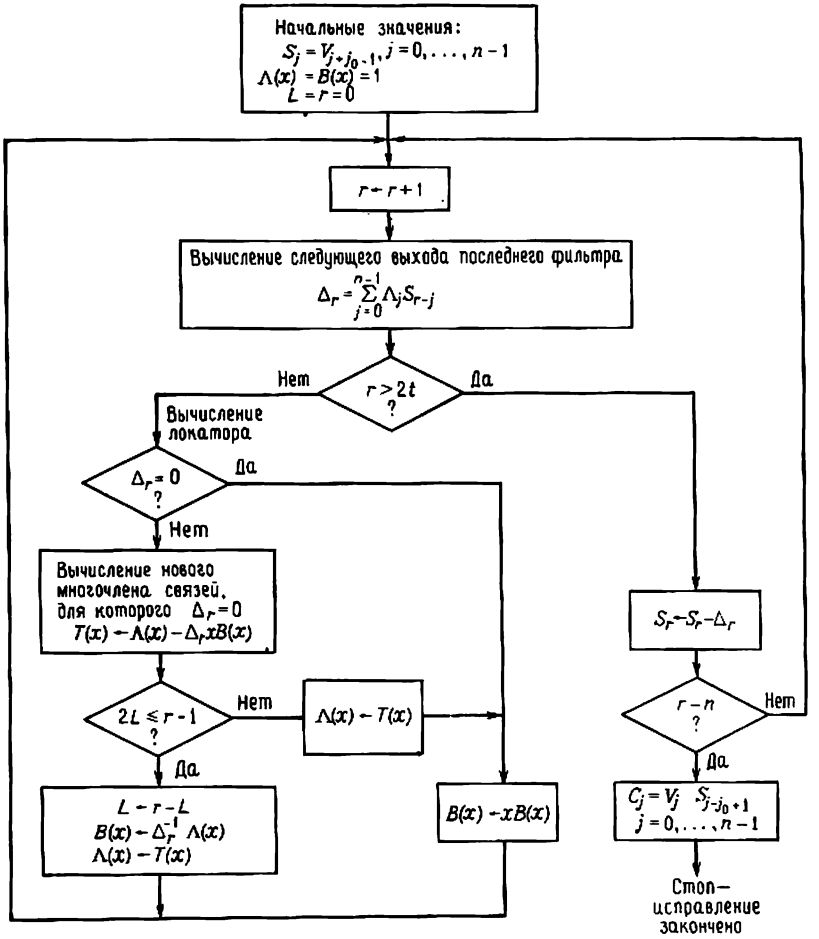


Рис. 9.1. Частотный декодер.

преобразованию Фурье. Процедура Ченя аналогична обратному преобразованию Фурье. Первая является n-точечным преобразованием вектора с ν ненулевыми компонентами и $GF(q^m)$-значным выходным вектором, а вторая представляет собой n-точечное преобразование вектора с n ненулевыми компонентами и $GF(q)$-значным выходом. Алгоритм Форни заменяется рекуррентным продолжением. Четвертая схема представляется более простой из-за простоты реализации отдельных блоков декодера.

Конструктивный выбор одной из этих схем частично определяется возможными методами вычисления преобразования Фурье, и поэтому в последнем параграфе главы рассматриваются различ-

ные алгоритмы вычисления преобразования Фурье. Помимо выбора алгоритма вычисления преобразования Фурье и способа реализации декодера конструктор может так модернизировать декодер, чтобы вычислять только часть спектральных компонент. Например, в четвертом варианте схемы декодера на рис. 9.2 требуется вычислить только $2t$ компонент спектра. Возможно

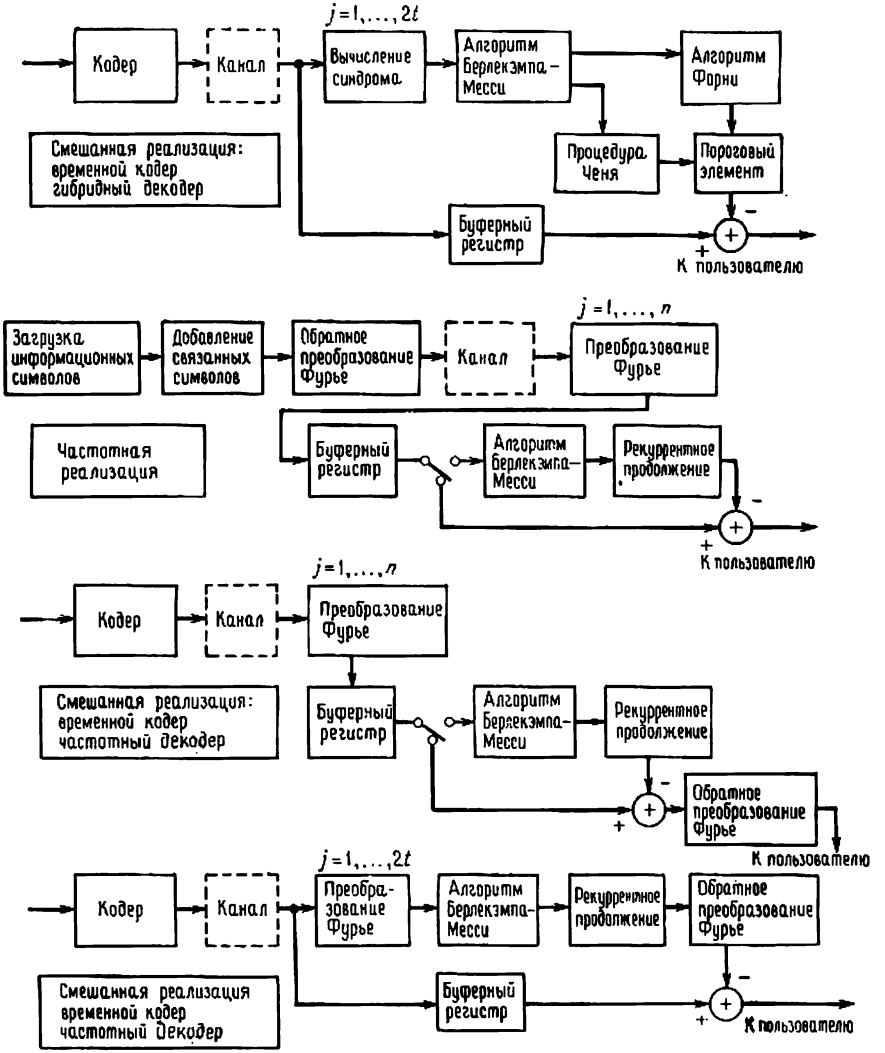


Рис. 9.2. Некоторые схемы кодера/декодера для кодов БЧХ.

также следующее упрощение: запишем принятый многочлен в виде

$$v(x) = Q(x)\, g(x) + s(x),$$

где $g(x)$ — порождающий многочлен кода БЧХ, а $s(x)$ — синдромный многочлен. Синдромный многочлен можно вычислить с помощью цепи деления на $g(x)$. Тогда

$$S_j = v(\alpha^j) = Q(\alpha^j)\, g(\alpha^j) = s(\alpha^j) = s(\alpha^j),$$

$$j = j_0, 1, \ldots, j_0 + 2t,$$

так как для этих значений индекса j выполняется равенство $g(\alpha^j) = 0$. Теперь компоненты синдрома могут быть вычислены как преобразование Фурье многочлена $s(x)$, который имеет не n, а только $n - k$ ненулевых коэффициентов. Этот метод оказывается полезным только тогда, когда легче осуществить деление на $g(x)$, чем вычислять преобразование Фурье в n входных точках. При малых значениях n это иногда имеет место.

Существует и другой алгоритм декодирования, основанный на алгоритме Евклида и совершенно отличный от описанного в § 7.7. Этот метод, ориентированный на частотную область, будет описан в данном параграфе.

В поле $GF(q^m)$ имеет место разложение

$$x^n - 1 = \prod_{i=0}^{n-1} (x - \alpha^i).$$

Определим истинно-локаторный многочлен $\Lambda^0(x) = (x^n - 1)/\Lambda(x)$. Он действительно является многочленом, так как $\Lambda(x)$ представляет собой произведение различных линейных множителей, каждый из которых делит $x^n - 1$. Корни многочлена $\Lambda^0(x)$ определяют правильные компоненты принятого вектора.

Теорема 9.1.1. *Многочлен ошибок $E(x)$ и многочлен локаторов $\Lambda(x)$ удовлетворяют равенству*

$$\text{НОД}[x^n - 1,\ E(x)] = (x^n - 1)/\Lambda(x) = \Lambda^0(x).$$

Доказательство. Многочлены $E(x)$ и $\Lambda(x)$ не имеют общих корней, так как по определению $\Lambda(\alpha^i) = 0$ тогда и только тогда, когда $E(\alpha^i) \neq 0$. Каждый элемент α^i должен быть корнем либо $E(x)$, либо $\Lambda(x)$. Следовательно,

$$\Lambda(x)\, E(x) = P(x)\,(x^n - 1)$$

для некоторого многочлена $P(x)$, взаимно простого с $\Lambda(x)$. Тогда

$$\text{НОД}\,[x^n - 1,\ E(x)] = (x^n - 1)/\Lambda(x) = \Lambda^0(x),$$

что и требовалось доказать. □

Применим описанный в теореме 7.7.1 алгоритм Евклида с $s(x) = x^n - 1$ и $t(x) = E(x)$, временно игнорируя тот факт, что в общем случае известны только $2t$ старших коэффициентов многочлена $E(x)$. Поскольку

$$\Lambda(x)\ \text{НОД}\,[x^n - 1,\ E(x)] = x^n - 1,$$

то, согласно следствию 7.7.2,

$$\Lambda(x) = (-1)^R A_{22}^{(R)}(x)$$

определяет многочлен $A_{22}^{(R)}(x)$, вычисляемый в процессе выполнения алгоритма Евклида.

Если многочлен $E(x)$ известен, то с помощью алгоритма Евклида мы таким образом можем вычислить $\Lambda(x)$. Но нам известны только те коэффициенты многочлена $E(x)$, для которых соответствующие коэффициенты многочлена $C(x)$ равны нулю, и, таким образом, на первый взгляд эта формула для $\Lambda(x)$ кажется бесполезной. На самом деле ее можно использовать и в том случае, когда известны лишь $2t$ коэффициентов многочлена $E(x)$, если только они правильны.

Рассмотрим код БЧХ, корнями порождающего многочлена которого являются элементы $\alpha^{n-2t}, \ldots, \alpha^{n-1}$. (Если использовать в качестве примитивного элемента поля α^{-1}, то это обычные корни с показателями от 1 до $2t$.) Построим алгоритм декодирования для такого кода БЧХ. Нам надо показать, что если произошло не более t ошибок, то итерации алгоритма Евклида зависят только от $2t$ старших коэффициентов многочлена $E(x)$. Остальные коэффициенты могут выбираться произвольно.

Теорема 9.1.2. *Пусть заданы E_j для $j = n - 2t, \ldots, n - 1$. Тогда имеется не более одного способа задания E_j, $j = 0, \ldots, n - 2t - 1$, такого, что уравнение*

$$\Lambda(x)\ \text{НОД}\,[x^n - 1,\ E(x)] = x^n - 1$$

разрешимо относительно $\Lambda(x)$ степени t или меньше.

Доказательство. Доказательство теоремы показывает, что она является одной из форм границы БЧХ. Предположим, что степень многочлена $\Lambda(x)$ не больше t и что он удовлетворяет усло-

виям теоремы. Выпишем последний шаг алгоритма Евклида в форме, приведенной в доказательстве следствия 7.7.2:

$$\begin{bmatrix} x^n - 1 \\ E(x) \end{bmatrix} = (-1)^R \begin{bmatrix} A_{22}^{(R)}(x) & -A_{12}^{(R)}(x) \\ -A_{21}^{(R)}(x) & -A_{11}^{(R)}(x) \end{bmatrix} \begin{bmatrix} \text{НОД}[x^n - 1, E(x)] \\ 0 \end{bmatrix}.$$

Отсюда видно, что $(-1)^R A_{22}^{(R)}(x) = \Lambda(x)$. Вместе с тем в прямой форме последнее равенство имеет вид

$$\begin{bmatrix} \text{НОД}[x^n - 1, E(x)] \\ 0 \end{bmatrix} = \begin{bmatrix} A_{11}^{(R)}(x) & A_{12}^{(R)}(x) \\ A_{21}^{(R)}(x) & A_{22}^{(R)}(x) \end{bmatrix} \begin{bmatrix} x^n - 1 \\ E(x) \end{bmatrix},$$

что приводит к требованию

$$0 = (-1)^R A_{21}^{(R)}(x)(x^n - 1) + \Lambda(x) E(x).$$

Следовательно, многочлен $\Lambda(x) E(x)$ кратен многочлену $x^n - 1$. Таким образом, многочлен $E(x)$ имеет корни в тех точках поля $GF(q^m)$, в которых не имеет корней многочлен $\Lambda(x)$. Поскольку степень многочлена $\Lambda(x)$ не превосходит t, то он имеет не более t нулей. Тогда компоненты e_i обратного преобразования Фурье обращаются в нуль по меньшей мере $n - t$ раз. Согласно границе БЧХ, должен найтись такой вектор $\mathbf{e}$, преобразование Фурье которого совпадает с известными коэффициентами многочлена $E(x)$ в позициях $j = n - 2t, \ldots, n - 1$. Это завершает доказательство теоремы. □

Одним из непосредственных способов применения доказанной теоремы является попытка перебрать всеми возможными способами неизвестные коэффициенты многочлена $E(x)$ и воспользоваться алгоритмом Евклида для вычисления $\Lambda(x)$. Только один многочлен $\Lambda(x)$ будет иметь степень не больше t. Он-то и будет правильным многочленом локаторов ошибок, а соответствующий многочлен $E(x)$ — правильным спектром вектора ошибок. Однако так как $\Lambda(x)$ однозначно определяется по известным компонентам $E(x)$, то более целесообразной представляется возможность избежать перебора неизвестных коэффициентов многочлена $E(x)$. Такой алгоритм представлен на рис. 9.3.

Рассмотрим алгоритм деления

$$s(x) = Q(x) t(x) + r(x),$$

где $\deg t(x) = a$ и $\deg s(x) = b$. В этом алгоритме степень частного $Q(x)$ равна $b - a$ и сам многочлен-частное $Q(x)$ зависит от $t(x)$ только через коэффициенты $t_a, t_{a-1}, \ldots, t_{b-a}$. Для вычисле-

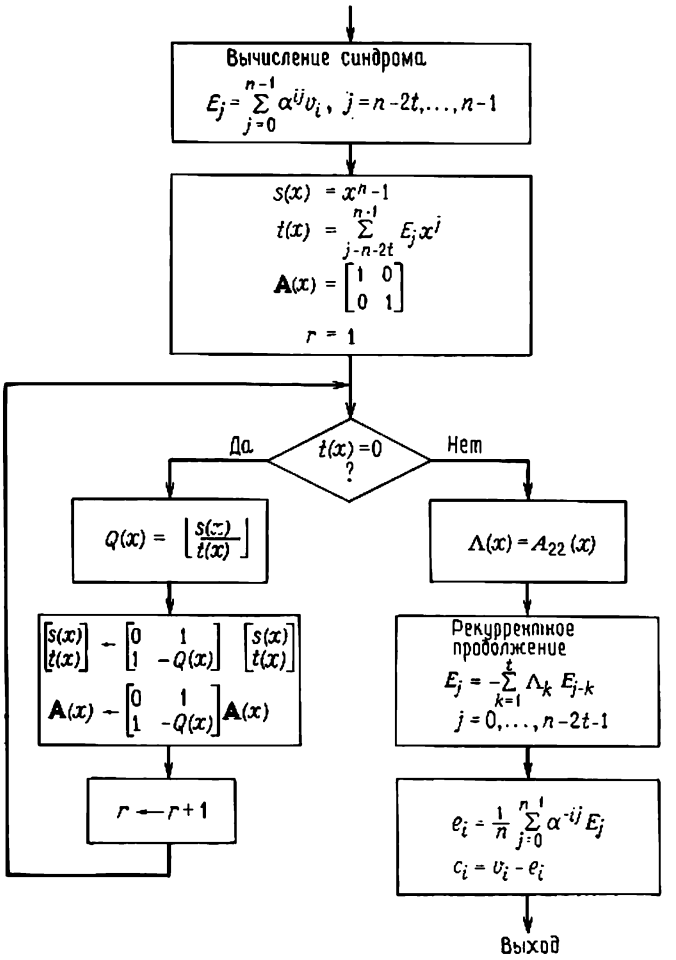


Рис. 9.3. Декодер для кодов БЧХ, основанный на алгоритме Евклида.

ния $Q(x)$ воспользуемся следующей итеративной процедурой при $r = 0, \ldots, b - a$:

$$Q_{b-a-r} = \left[s_{b-r} - \sum_{l=0}^{r-1} Q_{b-a-l} t_{u+l-r} \right] \Big/ t_u.$$

Данный алгоритм является слабой модификацией обычного алгоритма деления многочленов.

Используем эту процедуру вычисления частного на каждом шаге алгоритма Евклида. В r-й итерации теоремы 7.7.1 вычислим только коэффициенты $s_j^{(r+1)}$ и $t_j^{(r+1)}$ для значений j от $n - 2t +$

$+\sum_{l=1}^{r} \deg Q^{(l)}(x)$ до $n-1$. Эти старшие фрагменты многочленов $s^{(r+1)}(x)$ и $t^{(r+1)}(x)$ являются именно теми многочленами, которые необходимы для выполнения итерационных шагов алгоритма Евклида при построении рассматриваемого ряда многочленов-частных. Так как

$$\begin{bmatrix} \text{НОД}[x^n-1,\ E(x)] \\ 0 \end{bmatrix} = \left\{ \prod_{l=R-1}^{0} \begin{bmatrix} 0 & 1 \\ 1 & -Q^{(l)}(x) \end{bmatrix} \right\} \begin{bmatrix} x^n-1 \\ E(x) \end{bmatrix} =$$

$$= \left\{ \prod_{l=R-1}^{r+1} \begin{bmatrix} 0 & 1 \\ 1 & -Q^{(l)}(x) \end{bmatrix} \right\} \begin{bmatrix} s^{(r+1)}(x) \\ t^{(r+1)}(x) \end{bmatrix},$$

то можно записать

$$\left\{ \prod_{l=R-1}^{r+1} \begin{bmatrix} Q^{(l)}(x) & 1 \\ 1 & 0 \end{bmatrix} \right\} \begin{bmatrix} \text{НОД}[x^n-1, E(x)] \\ 0 \end{bmatrix} =$$

$$= \left\{ \prod_{l=r}^{0} \begin{bmatrix} 0 & 1 \\ 1 & -Q^{(l)}(x) \end{bmatrix} \right\} \begin{bmatrix} x^n-1 \\ E(x) \end{bmatrix} = \begin{bmatrix} s^{(r+1)}(x) \\ t^{(r+1)}(x) \end{bmatrix}.$$

Поскольку степень НОД $[x^n-1, E(x)]$ равна по меньшей мере $n-t$, отсюда следует, что степень многочлена $t^{(r+1)}(x)$ равна по меньшей мере $n-t+\sum_{l=r+1}^{R-1} \deg Q^{(l)}(x)$. Таким образом $t_j^{(r+1)}$ известны для всех j от $j=n-2t+\sum_{l=1}^{r} \deg Q^{(l)}(x)$ до $n-t+\sum_{l=r+1}^{R} \deg Q^{(l)}(x)$ при ненулевом верхнем индексе. Этого диапазона достаточно для вычисления $Q^{(l+1)}(x)$ и продолжения итерации.

9.2. ИСПРАВЛЕНИЕ СТИРАНИЙ И ОШИБОК

Контролирующие ошибки коды могут быть использованы в каналах, в которых кроме ошибок происходят и стирания. Принятое по такому каналу слово состоит из входных символов канала (часть из них может быть ошибочными) и пробелов (или эквивалентных символов), обозначающих стирания. Декодер должен исправить ошибки и восстановить стертые символы. Код с минимальным расстоянием d^* позволяет декодировать любую конфигурацию, содержащую ν ошибок ρ стираний, при условии, что

$$d^* \geqslant 2\nu + 1 + \rho.$$

Наибольшее ν, удовлетворяющее этому неравенству, обозначим через t_ρ.

Чтобы декодировать принятое слово с ρ стираниями, необходимо найти кодовое слово, которое в нестертых позициях отличается от принятого слова не более чем в t_ρ символах. Поскольку

по определению параметра t_ρ такое слово единственно, разыскивается любое подходящее слово. Нам нужно только найти какое-либо решение, так как мы знаем, что оно лишь одно.

Один из возможных способов декодирования состоит в угадывании стертых позиций с последующим применением любой процедуры исправления ошибок. Если процедура приводит к исправлению не более t_ρ ошибок в нестертых позициях (и, возможно, некоторых угаданных символах в стертых позициях), то кодовое слово восстанавливается. Если необходимо исправлять слишком много ошибок, то делается новая попытка угадать стертые символы. При этом любая подходящая для данного кода процедура исправления ошибок может быть модифицирована в процедуру исправления ошибок и стираний. Практически она подходит в тех случаях, когда число комбинаций угадываемых стертых символов мало. В противном случае такая процедура проб и ошибок становится практически неприемлемой. В таких случаях восстановление стертых символов должно осуществляться некоторым регулярным способом, входящим в алгоритм декодирования.

Как мы видели, частный класс кодов БЧХ допускает многие сильные алгоритмы декодирования. Все они могут быть обобщены на случай декодирования ошибок и стираний. Для простоты будем полагать $j_0 = 1$.

Пусть v_i, $i = 0, \ldots, n-1$, обозначают символы принятого вектора. Предположим, что стирания произошли в позициях $i_1, \ldots, i_\rho$. Во всех этих позициях принятое слово содержит пробелы, которые мы сначала заполним нулями. Определим n-мерный вектор стираний, положив символы в стертых позициях равными f_{i_l}, $l = 1, \ldots, \rho$, а в остальных позициях равными нулю. Тогда

$$v_i = c_i + e_i + f_i, \quad i = 0, \ldots, n-1.$$

Пусть ψ — произвольный вектор, символы которого равны нулю в каждой позиции стирания и только в них. Тогда

$$\psi_i v_i = \psi_i (c_i + e_i + f_i) = \psi_i c_i + \psi_i e_i.$$

Определим модифицированное принятое слово равенством $v_i' = \psi_i v_i$, модифицированный вектор ошибок равенством $e_i' = \psi_i e_i$ и модифицированное кодовое слово равенством $c_i' = \psi_i c_i$. (Модифицированный вектор ошибок содержит ошибки в тех же позициях, что и исходный вектор ошибок e.) Тогда последнее равенство приводится к виду

$$v_i' = c_i' + e_i'.$$

Теперь задача состоит в декодировании вектора $\mathbf{v}'$ и сводится к задаче вычисления вектора $\mathbf{e}'$, которую мы умеем решать при достаточном числе компонент синдрома.

Следующий шаг построения алгоритма состоит в выборе вектора Ψ, основанном на некоторых проводимых в частотной области рассуждениях. Пусть $U_l = \alpha^{i_l}$ для $l = 1, \ldots, \rho$ обозначают локаторы стираний. Определим многочлен локаторов стираний равенством

$$\Psi(x) = \sum_{k=0}^{n-1} \Psi_k x^k = \prod_{l=1}^{\rho} (1 - xU_l).$$

Этот многочлен специально определен так, чтобы компоненты ψ_i обратного преобразования вектора Ψ были равны нулю, как только $f_i \neq 0$. Следовательно, $\psi_i f_i = 0$. В частотной области это дает

$$\mathbf{V}' = (\Psi * \mathbf{C}) + \mathbf{E}'.$$

Но вектор Ψ отличен от нуля только на блоке длины $\rho + 1$, а по построению кода БЧХ вектор $\mathbf{C}$ имеет нулевые компоненты в блоке длины $2t$. Следовательно, свертка $\Psi * \mathbf{C}$ равна нулю в блоке длины $2t - \rho$. Полагая $S'_j = V'_j$, определим на этом блоке модифицированный синдром. Тогда

$$S'_j = (\Psi * \mathbf{V})_j = E'_j,$$

что задает достаточное число компонент синдрома для исправления модифицированного вектора ошибок.

Итак, точно так же, как в случае наличия только ошибок (при условии, что их не более $(2t - \rho)/2$), по этим $2t - \rho$ известным компонентам вектора $\mathbf{E}'$ можно найти многочлен локаторов ошибок $\Lambda(x)$. Коль скоро многочлен локаторов ошибок известен, можно скомбинировать его с многочленом локаторов стираний и дальнейшее декодирование проводить так же, как и в случае наличия только ошибок. Чтобы проделать это, определим сначала многочлен локаторов ошибок и стираний:

$$\overline{\Lambda}(x) = \Psi(x)\,\Lambda(x).$$

Обратное преобразование Фурье вектора $\overline{\Lambda}$ содержит нуль в позиции каждой ошибки и каждого стирания. Таким образом, $\bar{\lambda}_i = 0$, если $e_i \neq 0$ или $f_i \neq 0$. Следовательно, $\bar{\lambda}_i (e_i + f_i) = 0$,

$$\overline{\Lambda} * (\mathbf{E} + \mathbf{F}) = 0,$$

и многочлен $\overline{\Lambda}$ отличен от нуля в блоке длины, не превышающей $\nu + \rho + 1$. Следовательно, используя известные компоненты вектора $\overline{\Lambda}$ и $2t$ известных компонент вектора $\mathbf{E} + \mathbf{F}$, можно воспользоваться этим содержащим свертку уравнением для того,

чтобы рекуррентно продолжить вектор $\mathbf{E}+\mathbf{F}$ до всех n компонент. Затем вычисляются

$$C_i = V_i - (E_i \mid F_i).$$

Обратное преобразование Фурье завершает декодирование.

Вычисление многочлена локаторов ошибок по модифицированному значению синдрома можно выполнить с помощью алгоритма Берлекэмпа—Месси. Можно, однако, сделать это гораздо лучше, если скомбинировать несколько шагов. Идея алгоритма Берлекэмпа—Месси состоит в рекуррентной процедуре вычисления многочлена $\Lambda(x)$ за $2t$ итераций, начиная с оценок $\Lambda^{(0)}(x) = 1$ и $B^{(0)}(x) = 1$.

В случае наличия стираний в уравнении для невязки Δ_r компоненты синдрома заменяются компонентами модифицированного синдрома:

$$\Delta_r = \sum_{j=0}^{n-1} \Lambda_j^{(r-1)} S'_{r-j}.$$

Многочлен локаторов ошибок вычисляется с помощью n итераций при начальных условиях $\Lambda^{(0)}(x) = B^{(0)}(x) = 1$.

Но что произойдет, если заменить начальные условия на $\Lambda^{(0)}(x) = B^{(0)}(x) = \Psi(x)$? Заметим, что в этом случае

$$\Lambda^{(r)}(x)\Psi(x) = \Lambda^{(r-1)}(x)\Psi(x) - \Delta_r x B^{(r-1)}(x)\Psi(x),$$

$$B^{(r)}(x)\Psi(x) = (1-\delta_r)\, x B^{(r-1)}(x)\Psi(x) +$$

$$+ \delta_r \Delta_r^{-1} \Lambda^{(r-1)}(x)\Psi(x).$$

Если мы временно определим $\overline{\Lambda}^{(r)}(x) = \Lambda^{(r)}(x)\Psi(x)$ и будем вычислять невязку Δ_r по формуле

$$\Delta_r = \sum_{j=0}^{n-1} \overline{\Lambda}_j^{(r-1)} S_{r-j} = \sum_{j=0}^{n-1} \overline{\Lambda}_{r-j}^{(r-1)} S_j,$$

то

$$\Delta_r = \sum_{j=0}^{n-1} \left(\sum_{k=0}^{n-1} \Lambda_k^{(r-1)} \Psi_{r-j-k} \right) S_j = \sum_{k=0}^{n-1} \Lambda_k^{(r-1)} S'_{r-k}.$$

Итак, если в качестве начальной точки в алгоритме Берлекэмпа—Месси вместо 1 выбрать многочлен $\Psi(x)$, то модифицированный синдром вычисляется неявно и нет необходимости вводить его в явном виде. Отбросим теперь обозначение $\overline{\Lambda}(x)$, заменив его на $\Lambda(x)$ и назвав *многочленом локаторов ошибок и стираний*. При начальных условиях, даваемых многочленом $\Psi(x)$, алгоритм Берлекэмпа—Месси рекуррентно порождает многочлен локаторов ошибок и стираний в соответствии с уравнениями

$$\Lambda^{(r)}(x) = \Lambda^{(r-1)}(x) - \Delta_r x B^{(r-1)}(x),$$

$$B^{(r)}(x) = (1 - \delta_r) x B^{(r-1)}(x) + \delta_r \Delta_r^{-1} \Lambda^{(r-1)}(x),$$

$$\Delta_r = \sum_{j=0}^{n-1} \Lambda_j^{(r-1)} S_{r-j}.$$

Дополненный правилами восстановления стираний алгоритм Берлекэмпа—Месси приведен на рис. 9.4. Сопоставим его с алгоритмом на рис. 9.1. Единственное изменение по сравнению с алгоритмом для исправления только ошибок состоит в вычислении многочлена локаторов стираний; сравнительно с остальными вычислениями алгоритма оно является тривиальным. Это вычисление производится с помощью показанной на рис. 9.4 специальной вспомогательной цепи, содержащей первые ρ итераций. Индекс r используется в качестве счетчика для первых ρ итераций, а после вычисления многочлена стираний в качестве счетчика последующих итераций алгоритма Берлекэмпа—Месси до тех пор, пока r не превзойдет $2t$. При каждом стирании длина L регистра сдвига возрастает на единицу, а затем изменяется в соответствии с обычными правилами процедуры алгоритма Берлекэмпа—Месси. Поскольку, однако, этот алгоритм не учитывал стираний, то в конце приходится добавить исправления стираний и заменить r и L на $r - \rho$ и $L - \rho$ соответственно.

Прежде чем закончить данный параграф, заметим, что вместо пробела, обозначающего стирание, можно оставить лучше всего угаданный символ, пометив факт наличия в данной компоненте стирания некоторым специальным образом. Такие стирания назовем *помеченными*. Эти позиции не влияют на алгоритм декодирования, так как стоящие в них символы заменяются нулями. Иногда вне основного декодера удается использовать дополнительные блоки, извлекающие пользу из этой слабой информации. Например, можно отказаться от полученного на выходе декодера слова, если слишком многие из помеченных стираний не совпадают с результатом декодирования.

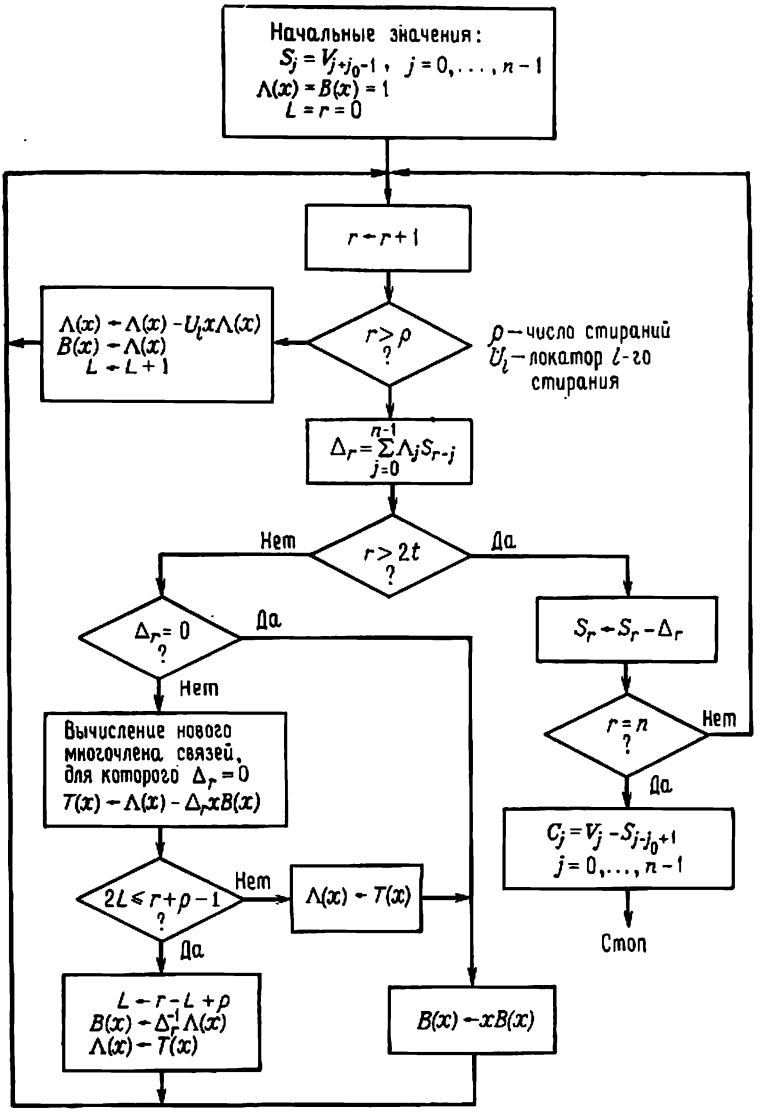


Рис. 9.4. Исправляющий ошибки и стирания декодер для кодов БЧХ.

9.3. ДЕКОДИРОВАНИЕ РАСШИРЕННЫХ КОДОВ РИДА—СОЛОМОНА

Незначительная модификация любого декодера для кода Рида—Соломона позволяет получить декодер для расширенного кода Рида—Соломона. Один из возможных способов состоит просто

в переборе всех q^2 возможных значений ошибок в двух символах расширения и последующих попытках декодирования для каждой пары. Если такая попытка приводит к кодовому слову при t или менее исправлениях, то результат правилен, так как только одно кодовое слово находится на расстоянии t от принятого слова.

Лучше, однако, отдельно рассматривать два случая возможных конфигураций ошибок: в первом случае искажен хотя бы один из символов расширения, а во втором ошибок в символах расширения нет. Сообщение при этом декодируется дважды (для каждого из случаев по отдельности). Пусть [1]) $S_j = V_{j+j_0}$ при $j = 1, \ldots, 2t-2$, и пусть $S_0 = V_{j_0} = v_-$ и $S_{2t-1} = V_{j_0+2t-1} = v_+$, если v_- и v_+ не искажены. В первом случае во внутреннем векторе происходит не более $t-1$ ошибок и поэтому $2t-2$ компонент синдрома $S_1, \ldots, S_{2t-2}$ достаточно для того, чтобы правильно восстановить внутренний спектр конфигурации ошибок. Эти $2t-2$ компонент синдрома можно рекуррентно продолжить до $n-2$ компонент спектра конфигурации ошибок, так как $S_j = E_{j+j_0}$. Во втором случае все ошибки лежат во внутреннем векторе, но их не более t, и поэтому для восстановления спектра конфигурации ошибок компонент достаточно $2t$ компонент $S_0, \ldots, S_{2t-1}$. Вообще говоря, одной попытки декодирования недостаточно и приходится делать обе попытки но так как минимальное расстояние кода равно $2t+1$, может быть найдена только одна конфигурация не более чем из t ошибок.

Чтобы не удваивать сложность декодера дублированием декодеров, опишем вариант разделения вычислений, который дает лишь незначительное увеличение сложности. В этом параграфе мы опишем модификацию алгоритма Берлекэмпа—Месси для декодирования расширенных кодов Рида—Соломона, а в следующем сделаем то же самое для расширенных кодов БЧХ. Ограничимся вычислением многочлена локаторов ошибок, так как остальные шаги алгоритма нам уже известны.

Наиболее важными расширениями кодов Рида—Соломона являются расширения на один символ, так как среди них содержатся коды длины 2^m, а во многих приложениях в качестве длины кода используется именно степень двойки. Мы начнем с этого случая, так как он представляет практический интерес из-за простоты декодера и так как его легко объяснить.

В рассматриваемом коде спектральные компоненты $C_{j_0}, \ldots, C_{j_0+2t-2}$ равны нулю и единственную проверку на четность дает символ $c_+ = C_{j_0+2t-1}$. Равенство нулю $2t-1$ компонент гарантирует, что минимальное расстояние в коде равно $2t$, и поэтому код позволяет исправлять $t-1$ и обнаруживать t ошибок. Блок

[1]) В случае расширения кода на два символа удобно изменить нашу обычную индексацию и обозначить компоненты синдрома через $S_0, \ldots, S_{2t-1}$.

из $2t$ компонент синдрома начинается с компоненты S_1, равной V_{j_0}, и заканчивается компонентой S_{2t}, равной $V_{j_0+2t-1} - v_+$. Предположим, что произошло не более t ошибок, и проведем итерации алгоритма Берлекэмпа—Месси от S_1 до S_{2t-1}. Если $L_{2t-2} \leqslant t-1$ и невязка Δ_{2t-1} равна нулю, то, согласно теореме 7.5.3, во внутреннем векторе произошло не более $t-1$ ошибок и вычисленное значение многочлена локаторов ошибок правильно. В противном случае во внутреннем векторе произошло t ошибок, и, следовательно, символ v_+ правилен. Дополнительная компонента синдрома S_{2t} позволяет продолжить алгоритм Берлекэмпа—Месси еще на один шаг, так что полное число итераций равно $2t$, и поэтому найден многочлен локаторов t ошибок. Декодер для расширенного на одну позицию кода Рида—Соломона показан на рис. 9.5.

Декодер для расширенного на два символа кода Рида—Соломона более сложен и описать его труднее. Предположим, что произошло не более t ошибок. В нашем распоряжении имеются $2t-2$ компонент внутреннего синдрома, начиная с компоненты S_1, равной V_{j_0+1}. Согласно теореме 7.5.3, воспользуемся ими для исправления $2t-2$ ошибок во внутреннем векторе и обнаружения t ошибок. Иначе говоря, начиная с S_1 и выполняя $2t-4$ итераций алгоритма Берлекэмпа—Месси, найдем многочлен локаторов ошибок. Если $L_{2t-4} \leqslant t-2$, то сделаем еще две итерации и остановимся, если хотя бы одна из невязок Δ_{2t-3} или Δ_{2t-2} отлична от нуля. Если обе невязки равны нулю, то во внутреннем векторе произошло не более $2t-2$ ошибок. В этом случае многочлен локаторов ошибок вычислен правильно и может быть использован для дальнейшего исправления ошибок во внутреннем векторе. Так как полезная информация может быть полностью выделена из внутреннего вектора, то декодирование закончено. В противном случае внутренний вектор содержит $t-1$ или t ошибок, и поэтому искажено не более одного граничного символа. Следовательно, хотя бы одна из компонент S_0 и S_{2t-1} синдрома правильна, а если все ошибки произошли во внутреннем векторе, то они правильны обе. Присоединяя сначала одну, а затем вторую из них к остальным компонентам синдрома, получим два блока по $2t-1$ символов, хотя бы один из которых содержит $2t-1$ правильных компонент синдрома.

Теперь дважды применим алгоритм Берлекэмпа—Месси, сначала к одному из этих блоков (от S_0 до S_{2t-2}), а затем к другому (от S_1 до S_{2t-1}). Каждый раз будем исправлять $t-1$ и обнаруживать t ошибок. Если все t ошибок расположены во внутреннем векторе, то обе попытки позволят обнаружить t ошибок во внутреннем векторе, и, следовательно, оба символа расширения являются правильными. Если при одной или при обеих попытках не обнаруживается t ошибок, то во внутреннем векторе произошло

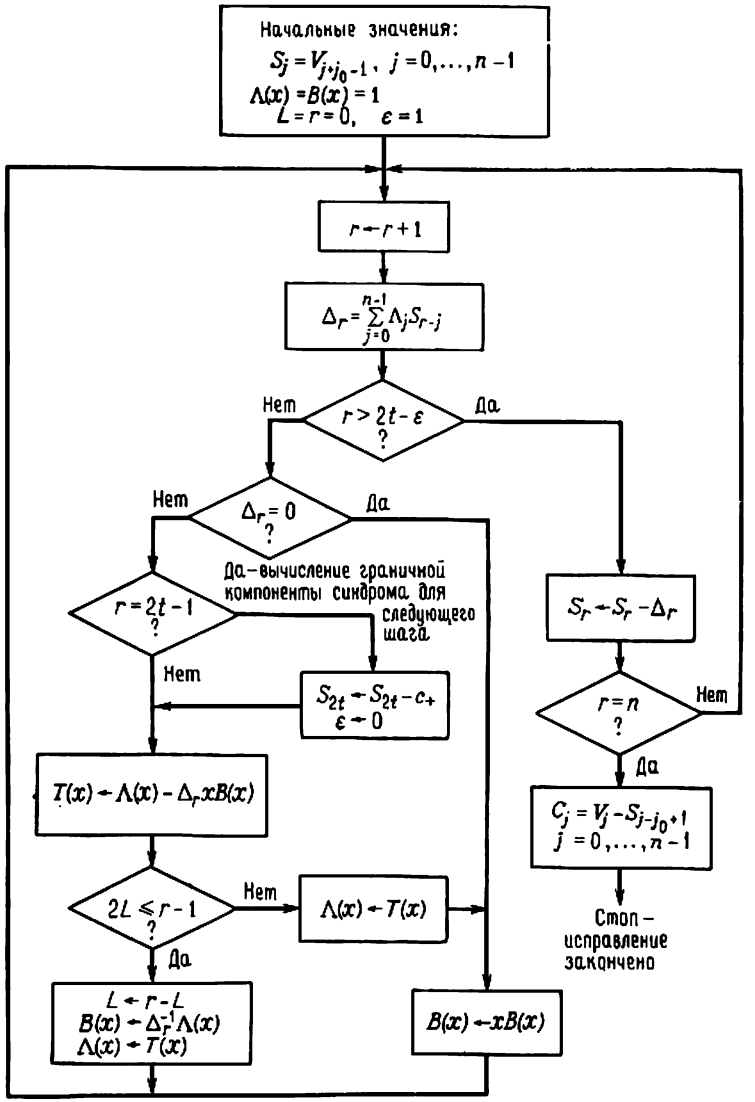


Рис. 9.5. Декодер для расширенного на один символ кода Рида—Соломона.

только t — 1 ошибок, и, следовательно, многочлен локаторов ошибок вычислен правильно и может быть использован для исправления внутреннего вектора. Так как информация может быть полностью извлечена из внутреннего вектора, декодирование закончено.

Наконец, если во внутреннем векторе обнаружено t ошибок, то оба граничных символа правильны. Следовательно, и обе граничные компоненты синдрома также правильны, и в нашем распоряжении имеется $2t$ правильных компонент синдрома, позволяющих исправить t ошибок во внутреннем векторе.

Описанная процедура декодирования все же дважды обращается к алгоритму Берлекэмпа—Месси. Такое описание выбрано из соображений простоты объяснения; процедуру можно сделать более компактной.

9.4. ДЕКОДИРОВАНИЕ РАСШИРЕННЫХ КОДОВ БЧХ

Теперь мы можем описать декодирование расширенных на два символа кодов БЧХ, определенных в § 8.5. Если конструктивное расстояние d кода является нечетным числом, то представленный в расширении поля код со своими хвостами образует подкод кода Рида—Соломона, и можно использовать уже описанный декодер для расширенного на две позиции кода Рида—Соломона. Если конструктивное расстояние кода четно, то к одному из хвостов добавим проверку на четность. Пока еще без доказательства мы утверждаем, что эта процедура приводит к коду, минимальное расстояние которого не меньше $d+1$. Докажем этот факт, выписав соответствующий алгоритм декодирования. Декодер аналогичен декодеру для расширенного на два символа кода Рида—Соломона.

Теорема 9.4.1. *Расширение на два символа двоичного кода БЧХ с четным конструктивным расстоянием d при использовании дополнительной проверки на четность на одном из хвостов приводит к коду, минимальное расстояние которого равно по меньшей мере $d+1$.*

Доказательство. Пусть $t = d/2$. Построим декодер, исправляющий t ошибок. В нашем распоряжении имеются $2t-3$ компонент внутреннего синдрома и две граничные компоненты синдрома $S_0 = V_{j_0} - v_-$ и $S_{2t-2} = V_{j_0+2t-2} - v_+$, которые правильны, если граничные символы v_- и v_+ не содержат ошибки. По предположению конструктивное расстояние d является четным числом, а в силу ограничений сопряженности числа j_0 и j_0+2t-2 должны быть нечетными. Эти требования выполняются только в том случае, когда $j_0 < 0$ и $j_0+2t-2 > 0$. Следовательно, ограничения сопряженности гарантируют также, что $C_{j_0-1} = 0$ и $C_{j_0+2t-1} = 0$, так как частоты $(j_0-1)/2$ и $(j_0+2t-1)/2$ являются четными. Следовательно, мы имеем еще две компоненты синдрома, а именно $S_{-1} = V_{j_0-1}$ и $S_{2t-1} = V_{j_0+2t-1}$.

Так как v_+ дополнительно содержит проверку на четность, то можно определить, четно или нечетно число произошедших

в этом хвосте ошибок. Предположим, что эта проверка на четность указывает на ошибку. Тогда во внутреннем векторе содержится не более $t-1$ ошибок. У нас имеется $2t-3$ внутренних компонент синдрома. Это позволяет исправить $t-2$ и обнаружить $t-1$ ошибок во внутреннем векторе. Если во внутреннем векторе обнаружено $t-1$ ошибок, то символ v_- ошибок не содержит, величина S_0 является правильной и может быть использована для еще одной итерации при вычислении многочлена локаторов $t-1$ ошибок. Это завершает декодирование в случае, когда проверка на четность в символе v_+ указывает ошибку.

В противном случае, когда проверка на четность в символе v_+ не указывает ошибки, в этом хвосте имеется четное число ошибок. Тогда для исправления $t-3$ ошибок во внутреннем векторе выполним, начиная с S_1, $2t-6$ итераций алгоритма Берлекэмпа—Месси (используя тем самым $2t-3$ компонент внутреннего синдрома) и зарезервируем остальные три итерации для обнаружения t или меньше ошибок во внутреннем векторе. Если $t-3$ ошибки не исправились, то внутренний вектор содержит $t-2$, или $t-1$, или t ошибок, и, следовательно, граничные символы содержат не более двух ошибок. Следовательно, либо v_+ правилен, либо v_+ содержит две ошибки, а v_- правилен.

Хотя бы одна из граничных компонент синдрома правильна. Присоединим S_{-1} или S_0 к одному концу внутреннего синдрома и S_{2t-1} или S_{2t} к другому концу. Это даст нам два синдрома с $2t-1$ компонентами, хотя бы один из которых правилен. Для каждого из синдромов воспользуемся алгоритмом Берлекэмпа—Месси для исправления $t-1$ и обнаружения t ошибок во внутреннем векторе. Если во внутреннем векторе содержится t ошибок, то оба синдрома должны указать на это, и в таком случае граничные символы не искажены. Если одна или обе попытки декодирования не обнаруживают во внутреннем векторе t ошибок, то в нем содержится не более $t-1$ ошибок и тогда вычисленный многочлен локаторов ошибок является правильным и позволяет завершить декодирование.

Наконец, если во внутреннем векторе обнаружено t ошибок, то в граничных символах ошибок нет и в нашем распоряжении имеются $2t+1$ компонент синдрома, позволяющих исправить t ошибок.

9.5. ДЕКОДИРОВАНИЕ ВО ВРЕМЕННОЙ ОБЛАСТИ

В § 9.1 было показано, что задача декодирования кодов БЧХ может быть сформулирована как задача спектрального оценивания. Декодер состоит из преобразователя Фурье (вычислителя синдрома), авторегрессионного спектрального анализатора (алго-

ритма Берлекэмпа—Месси) и обратного преобразователя Фурье. Задание декодера такими блоками облегчает его исследование, однако практическая реализация декодера может существенно отличаться от этой схемы.

В настоящем параграфе мы покажем, как преобразовать вычислительный алгоритм, чтобы избежать преобразования потока данных. Это позволит исключить из декодера оба преобразования Фурье. Главная идея такого преобразования основывается на том, что рекурсия в алгоритме Берлекэмпа—Месси линейна по обоим входным многочленам (хотя нелинейна по невязке). Во временнóй области рекуррентные уравнения имеют свои аналоги, которые можно найти аналитически с помощью преобразования Фурье. Это дает систему уравнений декодирования, связывающих непосредственно поступающие данные, и исправление ошибок производится без преобразований Фурье.

Чтобы исключить из вычислений преобразование Фурье, применим это преобразование ко всем величинам, входящим в формулировку теоремы 7.4.1. Преобразование вектора $\mathbf{\Lambda}$, обозначающее вектор локаторов ошибок во временнóй области, обозначим через $\boldsymbol{\lambda} = \{\lambda_i \mid i = 0, \ldots, n-1\}$. Преобразование вектора $\mathbf{B}$ обозначим соответственно через $\mathbf{b} = \{b_i \mid i = 0, \ldots, n-1\}$. Алгоритм Берлекэмпа—Месси превращается во временнóй области в рекуррентный процесс, который описывается следующей теоремой.

Теорема 9.5.1. *Пусть* $\mathbf{v}$ *— принятое зашумленное слово кода БЧХ, и пусть величины* $\lambda_i^{(2t)}$, $i = 0, \ldots, n-1$, *вычисляются с помощью рекуррентных уравнений*

$$\Delta_r = \sum_{i=0}^{n=1} \alpha^{ir} (\lambda_i^{(r-1)} v_i),$$

$$L_r = \delta_r (r - L_{r-1}) + \bar{\delta}_r L_{r-1},$$

$$\begin{bmatrix} \lambda_i^{(r)} \\ b_i^{(r)} \end{bmatrix} = \begin{bmatrix} 1 & -\Delta_r \alpha^i \\ \Delta_r^{-1} \delta_r & \bar{\delta}_r \alpha^i \end{bmatrix} \begin{bmatrix} \lambda_i^{(r-1)} \\ b_i^{(r-1)} \end{bmatrix},$$

где $r = 1, \ldots, 2t$, *начальные условия имеют вид* $\lambda_i^{(0)} = 1$, $b_i^{(0)} = 1$ *для всех* i, *а* $\delta_r = (1 - \bar{\delta}_r) = 1$, *если одновременно* $\Delta_r \neq 0$ *и* $2L_{r-1} \leqslant r - 1$, *и* $\delta_r = 0$ *в противном случае. Тогда* $\lambda_i^{(2t)} = 0$ *тогда и только тогда, когда* $e_i \neq 0$.

Доказательство. Применим преобразование Фурье ко всем входящим в формулировку теоремы векторным величинам. Тогда все рекуррентные уравнения запишутся в виде, указанном в фор-

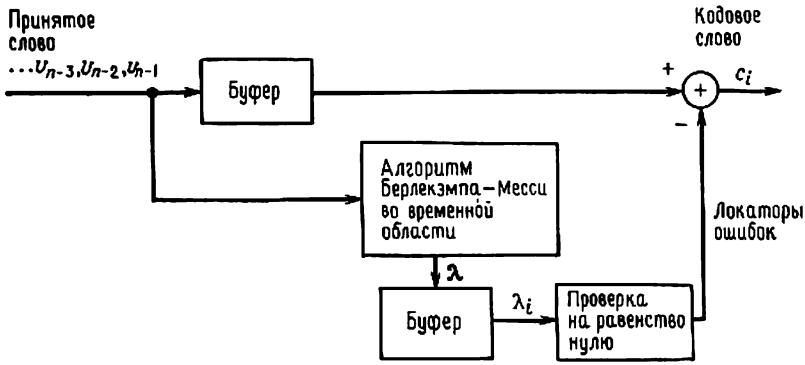


Рис. 9.6. Временной декодер для двоичного кода БЧХ.

мулировке теоремы 7.4.1, за исключением уравнения для невязки

$$\Delta_r = \sum_{j=0}^{n-1} \Lambda_j^{(r-1)} V_{r-j},$$

в котором вместо вектора $\mathbf{S}$ стоит вектор $\mathbf{V}$. Но $\Lambda_j^{(r-1)} = 0$ для $j > 2t$ и $V_j = S_j$ для $j = 1, \ldots, 2t$. Следовательно,

$$\Delta_r = \sum_{j=0}^{n-1} \Lambda_j^{(r-1)} S_{r-j},$$

и алгоритм во временно́й области формулируется аналогично теореме 7.4.1. □

Так как в двоичном случае i-я координата принятого слова содержит ошибку тогда и только тогда, когда $\lambda_i = 0$, то с вычислением вектора $\boldsymbol{\lambda}$ исправление ошибок почти заканчивается. Декодер для двоичного кода с декодированием во временно́й области показан на рис. 9.6.

Для недвоичных кодов во временно́й области нельзя ограничиться вычислением локаторов ошибок; надо вычислить еще величины ошибок. В частотной области величины ошибок можно вычислить с помощью следующих рекуррентных уравнений:

$$E_k = -\sum_{j=1}^{t} \Lambda_j E_{k-j}, \quad k = 2t + 1, \ldots, n - 1.$$

Как только многочлен локаторов ошибок вычислен, можно согласно этой рекурсии по известным $2t$ компонентам вектора $\mathbf{E}$ вычислить все остальные его компоненты, проведя $n - 2t$ итераций.

Следующая теорема описывает соответствующий этому процесс во временно́й области. В этом случае нельзя просто воспользо-

ваться преобразованием Фурье, а нужна некоторая перестройка.

Теорема 9.5.2. *Пусть* $\mathbf{v} = \mathbf{c} + \mathbf{e}$ — *принятое зашумленное слово кода БЧХ. При заданном во временной области векторе локаторов ошибок* $\boldsymbol{\lambda}$ *решением множества рекуррентных уравнений*

$$\Delta_r = \sum_{i=0}^{n-1} \alpha^{ir} v_i^{(r-1)} \lambda_i,$$

$$v_i^{(r)} = v_i^{(r-1)} - \Delta_r \alpha^{-ri}$$

является $v_i^{(n)} = e_i$, $i = 0, \ldots, n-1$.

Доказательство. Разобьем шаг рекурсии в частотной области на два шага, начиная с принятого спектра $\mathbf{V}$ и покомпонентно преобразуя его в $\mathbf{E}$:

$$\Delta_r = V_r - \left(- \sum_{j=1}^{n-1} \Lambda_j V_{r-j} \right) = \sum_{j=0}^{n-1} \Lambda_j V_{r-j},$$

$$V_j^{(r)} = \begin{cases} V_j^{(r-1)}, & j \neq r, \\ V_j^{(r-1)} - \Delta_r, & j = r. \end{cases}$$

Так как $V_j^{(2t)} = E_j$ для $j = 1, \ldots, 2t$ и $\Lambda_j = 0$ для $j > t$, то выписанные выше уравнения эквивалентны следующим:

$$E_r = - \sum_{j=1}^{n-1} \Lambda_j E_{r-j}.$$

Эта эквивалентность и доказывает теорему. □

На рис. 9.7 изображен временной декодер для полей характеристики 2. Левая часть блок-схемы соответствует теореме 9.5.1, а правая — теореме 9.5.2. В процессе проведения итераций в левой части формируется преобразование Фурье многочлена локаторов ошибок. При каждой итерации в правой части принятое слово преобразуется таким образом, что одна компонента спектра принятого слова переходит в компоненту спектра вектора ошибок. Таким образом, после выполнения последней итерации принятое слово преобразуется в вектор ошибок.

Привлекательность декодера во временной области объясняется двумя причинами.

1. Вычисления проводятся в один этап, а процедура Ченя и процедура вычисления синдрома отсутствуют. Это приводит к более экономной конструкции, особенно при аппаратурной реализации отдельных блоков декодера.

2. Такой декодер при умеренных значениях n должен выполнять намного меньше вычислений, чем декодер, использую-

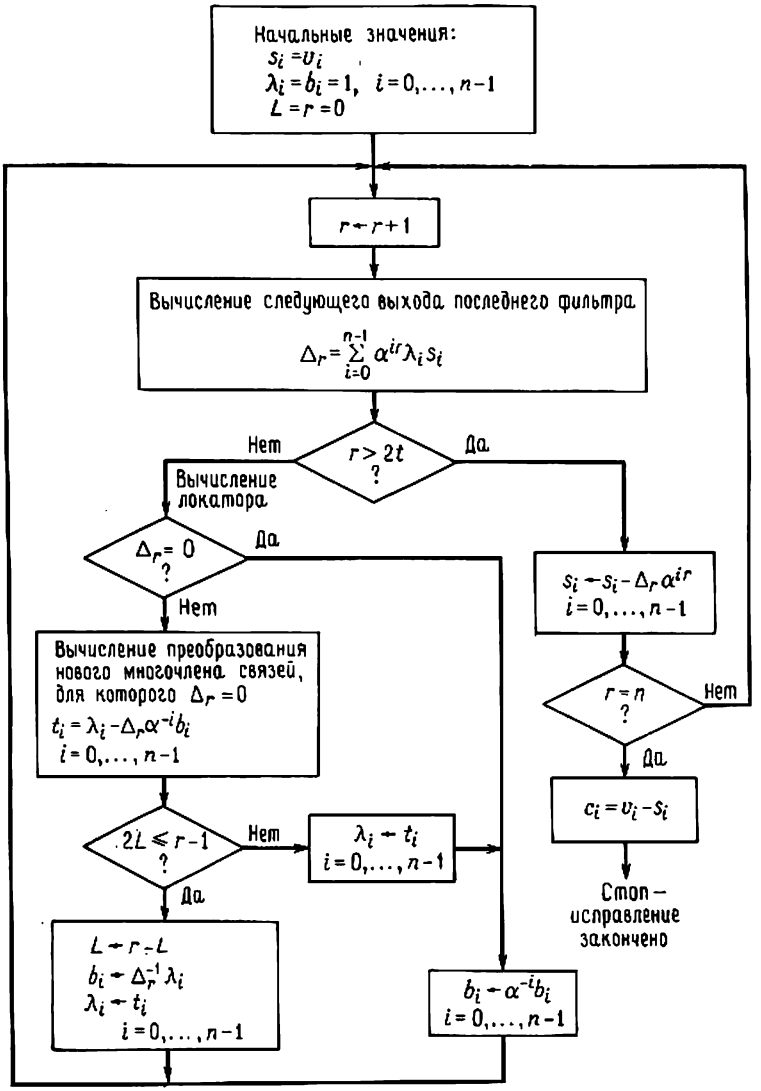


Рис. 9.7. Временной декодер.

щий прямое преобразование Фурье и общепринятый алгоритм Берлекэмпа—Месси.

Независимо от числа исправляемых ошибок работа декодера занимает n тактов. Так как за каждый такт обрабатывается вектор размерности n, то сложность декодера является величиной

$O(n^2)$. При больших n можно построить декодер с меньшим числом вычислений. Однако для малых n, включающих интервал практически используемых длин кода, временной декодер более привлекателен и обладает простой структурой. В эпоху больших интегральных схем структурная простота по меньшей мере столь же важна, сколь и число арифметических операций.

9.6. ДЕКОДИРОВАНИЕ ЗА ГРАНИЦЕЙ БЧХ

Часто случается, что конфигурации ошибок, вес которых мало отличается от радиуса упаковки кода, не попадают ни в одну сферу декодирования. Такие лежащие между сферами декодирования ошибки не совсем обоснованно называют *неисправляемыми конфигурациями ошибок*. Они обычно обнаруживаются, но не исправляются. Иногда удается исправить некоторые из этих ошибок, расширив множество исправляемых конфигураций ошибок. Укажем некоторые методы, позволяющие выполнить такое декодирование.

Распознать неисправляемые кодом БЧХ конфигурации ошибок можно либо по тому, что число корней многочлена локаторов ошибок, лежащих в поле локаторов, не равно его степени, либо по тому, что компоненты вектора ошибок выходят за пределы поля символов $GF(q)$. Чтобы обнаружить такие ситуации, нет необходимости полностью выполнять все вычисления алгоритма декодирования: они могут быть распознаны по свойствам рекуррентно вычисляемого спектра ошибок:

$$E_j = -\sum_{k=1}^{t} \Lambda_k E_{j-k}.$$

Согласно теореме 8.2.1, принадлежность всех компонентов конфигурации ошибок полю $GF(q)$ можно распознать, проверив выполнение условия $E_j^q = E_{((jq))}$ для всех индексов j в спектре этой конфигурации. Если такая проверка указывает на неисправляемую конфигурацию, то вычисления прерываются. Требование, состоящее в том, что степень многочлена $\Lambda(x)$ равна числу корней многочлена $\Lambda(x)$ в поле $GF(q^m)$, можно проверить с помощью условия периодичности последовательности коэффициентов $\{E_j\} : E_{((n+j))} = E_j$ для всех j. Таким образом получается второе правило проверки для обнаружения неисправляемой конфигурации ошибок и прекращения декодирования. Это правило основывается на следующей теореме.

Теорема 9.6.1. *Предположим, что $\Lambda(x)$ является ненулевым многочленом наименьшей степени, удовлетворяющим условию*

$$E_j = -\sum_{k=1}^{\deg \Lambda(x)} \Lambda_k E_{j-k}$$

для $j = 2t + 1, \ldots, n + 2t$, *где* $n = q^m - 1$. *Число различных корней многочлена* $\Lambda(x)$ *в поле* $GF(q^m)$ *равно степени этого многочлена тогда и только тогда, когда* $E_{((n+j))} = E_j$ *для всех* $j = 1, \ldots, 2t$.

Доказательство. Предположим, что число различных корней многочлена $\Lambda(x)$ в поле $GF(q^m)$ равно его степени. Тогда $\Lambda(x)$ делит $x^n - 1$ и существует многочлен $\Lambda^0(x)$, такой, что $\Lambda^0(x)\,\Lambda(x) = x^n - 1$. Следовательно, так как

$$\sum_{k=0}^{t} \Lambda_k E_{j-k} = 0,$$

то выполняется равенство $\Lambda(x)\,E(x) = 0$, и, таким образом, $\Lambda^0(x)\,\Lambda(x)\,E(x) = (x^n - 1)\,E(x) = 0$. Отсюда вытекает, что $E_j = E_{((n+j))}$ для всех j.

Обратно, предположим, что $E_j = E_{((n+j))}$ для всех j. Тогда $(x^n - 1)\,E(x) = 0$. Но по условию $\Lambda(x)$ представляет собой такой ненулевой многочлен наименьшей степени, что $\Lambda(x)\,E(x) = 0 \pmod{x^n - 1}$. Согласно алгоритму деления

$$(x^n - 1) = \Lambda(x)\,Q(x) + r(x),$$

так что

$$(x^n - 1)\,E(x) = \Lambda(x)\,E(x)\,Q(x) + r(x)\,E(x).$$

Следовательно, $r(x)\,E(x) = 0 \pmod{x^n - 1}$. Но степень $r(x)$ меньше степени $\Lambda(x)$, так что многочлен $r(x)$ должен быть равен нулю. Следовательно, многочлен $\Lambda(x)$ делит $x^n - 1$ и все его корни являются корнями многочлена $x^n - 1$. □

Если нарушается хотя бы одно из двух условий, т. е. если $E_j^q \neq E_{((qj))}$ для некоторого j или $E_{((n+j))} \neq E_j$ для некоторого j, то обнаруживается конфигурация более чем из t ошибок.

Иногда желательно декодировать за пределами конструктивного расстояния. Небольшое продвижение за конструктивное расстояние дает продолжение алгоритма Берлекэмпа—Месси после $2t$ итераций. Основная идея состоит во введении дополнительных неизвестных синдромов (или дополнительных невязок) и аналитическом продолжении алгоритма декодирования, при котором многочлен ошибок вычисляется как функция от этих неизвестных. Затем, придавая этим неизвестным различные возможные значения, методом проб и ошибок в поле символов кода разыскивается вектор ошибок минимального веса. К сожалению, сложность этого метода очень быстро растет с отходом от конструктивного расстояния, так что его можно использовать не всегда.

Тем не менее описанный выше метод применим во многих случаях; укажем некоторые из них.

1. Двоичные коды БЧХ, у которых проверочные частоты образуют блок длины $2t$, но истинное минимальное расстояние превосходит $2t+1$. (Одним из примеров такого кода является код Голея. Другой пример дается (127, 43, 31)-кодом БЧХ, конструктивное расстояние которого равно только 29.)
2. Циклический код, проверочные частоты которого не образуют непрерывного блока.
3. Декодирование некоторых ошибок веса $t+1$ в коде Рида—Соломона или коде БЧХ, исправляющем t ошибок.
4. Декодер для альтернантных кодов.

Такое продолжение алгоритма можно строить как во временно́й, так и в частотной области, а также некоторым смешанным способом. Можно строить одновременно и многочлен локаторов ошибок $\Lambda(x)$ (или $\lambda(x)$), и многочлен ошибок $e(x)$. Сложность этого процесса зависит от подхода. Если символы кода лежат в подполе поля локаторов ошибок, то схема, в которой сначала вычисляется многочлен локаторов ошибок, приводит к неопределенностям, разрешить которые можно только вычислением соответствующих многочленов ошибок и проверкой принадлежности их компонент полю символов кода. В этом случае разумно ввести в вычисления дополнительные проверки, позволяющие сразу ликвидировать неопределенные ситуации при построении $\Lambda(x)$.

Начнем с рассмотрения кода Рида—Соломона над $GF(q)$ с конструктивным расстоянием $2t+1$. Произвольный многочлен $\Lambda(x)$ степени $t+\nu$ с $t+\nu$ различными корнями в $GF(q)$ представляет собой допустимый многочлен локаторов ошибок, если

$$\sum_{j=0}^{n-1} \Lambda_j S_{r-j} = 0, \quad r = 1+t+\nu, \ldots, 2t.$$

Такой многочлен наименьшей степени с $\Lambda_0 = 1$ (если он является единственным) соответствует кодовому слову, находящемуся на наименьшем расстоянии от принятого слова. Если степень этого многочлена не выше t, то он будет вычислен алгоритмом Берлекэмпа—Месси. Но даже если произошло больше, чем t ошибок, а многочлен наименьшей степени с указанными свойствами является единственным, то принятое слово может быть однозначно декодировано, коль скоро этот многочлен найден. Для кода Рида—Соломона нет необходимости вычислять величины ошибок, так как ошибки всегда лежат в поле $GF(q)$. Если многочлен наименьшей степени не является единственным, то имеется несколько возможных конфигураций ошибок одного и того же веса, и все они согласуются с принятым словом.

Предположим, что произошло $t+1$ ошибок; тогда, чтобы вычислить $\Lambda(x)$, достаточно найти две неизвестные компоненты синдрома S_{2t+1} и S_{2t+2} или две невязки Δ_{2t+1} и Δ_{2t+2}. Итак, продолжим аналитически алгоритм Берлекэмпа—Месси на две итерации с двумя неизвестными значениями невязки. Имеем

$$\begin{bmatrix} \Lambda^{(2t+2)}(x) \\ B^{(2t+2)}(x) \end{bmatrix} = \begin{bmatrix} 1 & -\Delta_{2t+2}x \\ \Delta_{2t+2}^{-1}\delta_{2t+2} & \bar{\delta}_{2t+2}\,x \end{bmatrix} \begin{bmatrix} 1 & -\Delta_{2t+1}x \\ \Delta_{2t+1}^{-1}\delta_{2t+1} & \bar{\delta}_{2t+1}x \end{bmatrix} \begin{bmatrix} \Lambda^{(2t)}(x) \\ B^{(2t)}(x) \end{bmatrix},$$

где неизвестные удовлетворяют условиям Δ_{2t+1}, $\Delta_{2t+2} \in GF(q)$; δ_{2t+1}, $\delta_{2t+2} \in \{0, 1\}$. Все остальные величины в правой части известны по $2t$ синдромам. Преобразование частотной области во временну́ю дает

$$\begin{bmatrix} \lambda_i^{(2t+2)} \\ b_i^{(2t+2)} \end{bmatrix} = \begin{bmatrix} 1 & -\Delta_{2t+2}\alpha^{-i} \\ \Delta_{2t+2}^{-1}\delta_{2t+2} & \bar{\delta}_{2t+2}\alpha^{-i} \end{bmatrix} \begin{bmatrix} 1 & -\Delta_{2t+1}\alpha^{-i} \\ \Delta_{2t+1}^{-1}\delta_{2t+1} & \delta_{2t+1}\alpha^{-i} \end{bmatrix} \begin{bmatrix} \lambda_i^{(2t)} \\ b_i^{(2t)} \end{bmatrix},$$

где мы воспользовались тем фактом, что если $E_j' = E_{j-1}$, то обратное преобразование удовлетворяет равенству $e_i' = \alpha^{-i}e_i$. Теперь, если возможно, мы должны выбрать неизвестные так, чтобы вектор ошибок содержал не более $t+1$ ненулевых компонент.

Рассмотрим два возможных случая [1]):

(1) $\lambda_i^{(2t+2)} = \lambda_i^{(2t)} - A_1\alpha^{-i}b_i^{(2t)} - A_2\alpha^{-2i}b_i^{(2t)}$,

(2) $\lambda_i^{(2t+2)} = \lambda_i^{(2t)} - A_1\alpha^{-i}b_i^{(2t)} - A_2\alpha^{-2i}\lambda_i^{(2t)}$,

где в первом случае A_1 и A_2 — константы, не равные одновременно нулю, а во втором случае A_1 отлично от нуля. При $\lambda_i^{(2t+2)} = 0$ каждое из этих уравнений надо разрешить относительно A_1 и A_2 для ровно $t+1$ значения индекса i. Это даст локаторы $t+1$ ошибок, и дальнейшее декодирование может быть завершено как декодирование стираний.

Хотя такой поиск представляется весьма трудоемким, он очень прост по структуре и хорошо упорядочен, так что не составляет труда сконструировать для его выполнения блок, содержащий регистр сдвига. Поиск решения можно организовать различным

[1]) Имеются в виду две возможности, задаваемые уравнением для первой компоненты предыдущего векторного равенства при $\delta_{2t+1} = 0$ и $\delta_{2t+1} = 1$ соответственно. — *Прим. перев.*

образом. Одним из возможных подходов является построение гистограммы. Например, для уравнения (2) при каждом значении A_1 построим гистограмму величины $(\lambda_i^{(2t)} - A_1\alpha^{-i}b_i^{(2t)})^{-1} \times$ $\times\, \alpha^{-2i}\lambda_i^{(2t)}$. Любая составляющая гистограммы, принимающая значение $t+1$, соответствует возможной конфигурации из $t+1$ ошибок.

Декодер можно модифицировать далее так, чтобы он исправлял $t+\nu$ ошибок. Хотя уравнения при этом удлиняются, такой подход представляется практически вполне возможным для $t+3$ и $t+4$ в зависимости от длины кода.

Теперь рассмотрим двоичные коды. Они отличаются от кодов Рида—Соломона тем, что для них декодер может быть упрощен в силу теоремы 7.6.1, но одновременно усложняется за счет того, что многие из находимых конфигураций ошибок веса $t+\nu$ не являются двоичными и поэтому должны быть отброшены.

Предположим, что у нас имеется двоичный код БЧХ с конструктивным расстоянием $2t+1$, и мы хотим исправлять все однозначно декодируемые конфигурации веса $t+1$. Единственными величинами, которые может измерять декодер, являются $2t$ компонент синдрома $S_1, \ldots, S_{2t}$. Во всех остальных частотах стоят либо произвольные значения, либо значения, определяемые условиями сопряженности. Итеративный алгоритм в двоичном случае дает

$$\Lambda^{(2t+2)}(x) = \Lambda^{(2t)}(x) - \Delta_{2t+1}xB^{(2t)}(x).$$

Во временно́й области он переписывается в виде

$$\lambda_i^{(2t+2)} = \lambda_i^{(2t)} - \Delta_{2t+1}\alpha^{-i}b_i^{(2t)}$$

и предполагается, что конфигурация ошибок веса t или менее не была найдена алгоритмом.

Построим гистограмму величины $\alpha^i\lambda_i^{(2t)}/b_i^{(2t)}$ по всем ненулевым элементам поля $GF(q)$. Каждая равная $t+1$ компонента этой гистограммы соответствует кандидату на роль конфигурации ошибок. Зная локаторы ошибок, вычислим эту конфигурацию. Если она двоична и единственна, то декодирование завершено успешно.

Рассмотрим далее двоичный код, у которого проверочные частоты не являются последовательными. Например, проверочными частотами двоичного циклического (63, 28, 15)-кода являются C_1, C_3, C_5, C_7, C_9, C_{11} и C_{21}. Как можно проверить на ЭВМ, минимальное расстояние кода равно 15. По сравнению с (63, 24, 15)-кодом БЧХ этот код обладает тем преимуществом, что обеспечивает бо́льшую скорость, но зато для кода БЧХ известен алгоритм декодирования. Однако ценой незначительного усложнения частотный декодер для кода БЧХ удается преобра-

зовать в декодер для (63, 28, 15)-кода. Используя описанную выше процедуру, можно найти все конфигурации ошибок веса не более 7, которые согласуются с 12 последовательными проверочными частотами, а затем вычислить S_{21} для каждой из них Только одна из них будет соответствовать измеренному значению S_{21}.

9.7. ДЕКОДИРОВАНИЕ АЛЬТЕРНАНТНЫХ КОДОВ

Альтернантные коды являются ограничением на подполе простейшей модификации кодов Рида—Соломона. Минимальное расстояние d^* альтернантного кода не меньше конструктивного расстояния d исходного кода Рида—Соломона. Любая процедура декодирования кода Рида—Соломона может быть использована для декодирования альтернантных кодов. Необходимо лишь модифицировать синдром либо с помощью обратного преобразования шаблона, либо путем умножения на него во временно́й области, либо путем вычисления свертки в частотной области; никаких других изменений не требуется.

Таким образом, для декодирования альтернантных кодов в пределах конструктивного расстояния можно применять любой временной или частотный декодер для кодов БЧХ. Привлекательность альтернантных кодов, однако, состоит в том, что их минимальное расстояние много больше их конструктивного расстояния. В то же время использование альтернантных кодов с БЧХ-декодером дает мало преимуществ по сравнению с использованием кодов БЧХ с БЧХ-декодером. Конечно, есть резон в том, чтобы использовать альтернантные коды с БЧХ-декодером при исправлении ошибок в пределах конструктивного расстояния и обнаружении ошибок вплоть до минимального расстояния. Можно также применить описанные в § 9.6 методы декодирования за конструктивным расстоянием, хотя и в ограниченных пределах.

Полные возможности альтернантных кодов не будут реализованы до тех пор, пока не будут найдены конструктивная процедура построения дающих хорошие коды шаблонов и алгоритм декодирования, реализующий их минимальное расстояние.

Профильтрованный спектр принятого слова дается равенством

$$T_j = \sum_{k=0}^{n-1} H_{((j-k))} V_k.$$

Для кода Гоппы спектр в свою очередь вычисляется из $\mathbf{T}$ по формуле

$$V_j = \sum_{k-0}^{n-1} G_{((j-k))} T_k = \sum_{k=0}^{2t} G_k T_{((j-k))},$$

где степень многочлена Гоппы $G(x)$ равна $2t$ и $G(x)H(x) = 1 \pmod{x^n - 1}$. Поскольку

$$\sum_{k=0}^{n-1} H_{((j-k))}C_k = 0, \quad j = n - 2t + 1, \ldots, n,$$

то компоненты синдрома даются равенствами

$$S_{j-(n-2t)} = \sum_{k=0}^{n-1} H_{((j-k))}V_k, \quad j = n - 2t + 1, \ldots, n,$$

или

$$S_j = \sum_{k=0}^{n-1} H_{((j+n-2t-k))}V_k, \quad j = 1, \ldots, 2t.$$

По этим компонентам синдрома вычислим многочлен локаторов ошибок и продолжим его рекуррентно для получения всех компонент S_j, $j = 1, \ldots, n$. Спектр вектора ошибок находится из равенств

$$E_j = \sum_{k=0}^{n-1} G_k S_{((j-k-n+2t))}, \quad j = 0, \ldots, n-1.$$

На рис. 9.8 построен частотный декодер для альтернантных кодов. Синдром модифицируется в частотной области с помощью свертки со спектром шаблона. В принципе если эта операция реализуется до преобразования Фурье с помощью покомпонентного умножения во временной области, то число необходимых умножений будет меньше. Принятое слово, однако, состоит из символов малого алфавита, скажем $GF(2)$. Поэтому могут оказаться предпочтительнее преобразование Фурье в малом поле и модификация синдрома с помощью свертки в частотной области.

За исключением шага, связанного с шаблоном, декодирование альтернантных кодов происходит точно так же, как декодирование кодов Рида—Соломона. Так же, как и коды Рида—Соломона, альтернантные коды можно декодировать за пределами конструктивного расстояния. В пределах минимального расстояния кодовое слово единственно. Декодер для кода Рида—Соломона, применяемый для декодирования альтернантных кодов, может указать и другие слова, но они будут содержать компоненты, не принадлежащие полю символов кода, и такие слова должны быть отброшены.

При декодировании кодов Гоппы в узком смысле такого отбрасывания ложных кодовых слов можно заранее избежать, если добавлять только компоненты синдрома, согласующиеся с ограничениями сопряженности $E_{2j} = E_j^2$. Две дополнительные ком-

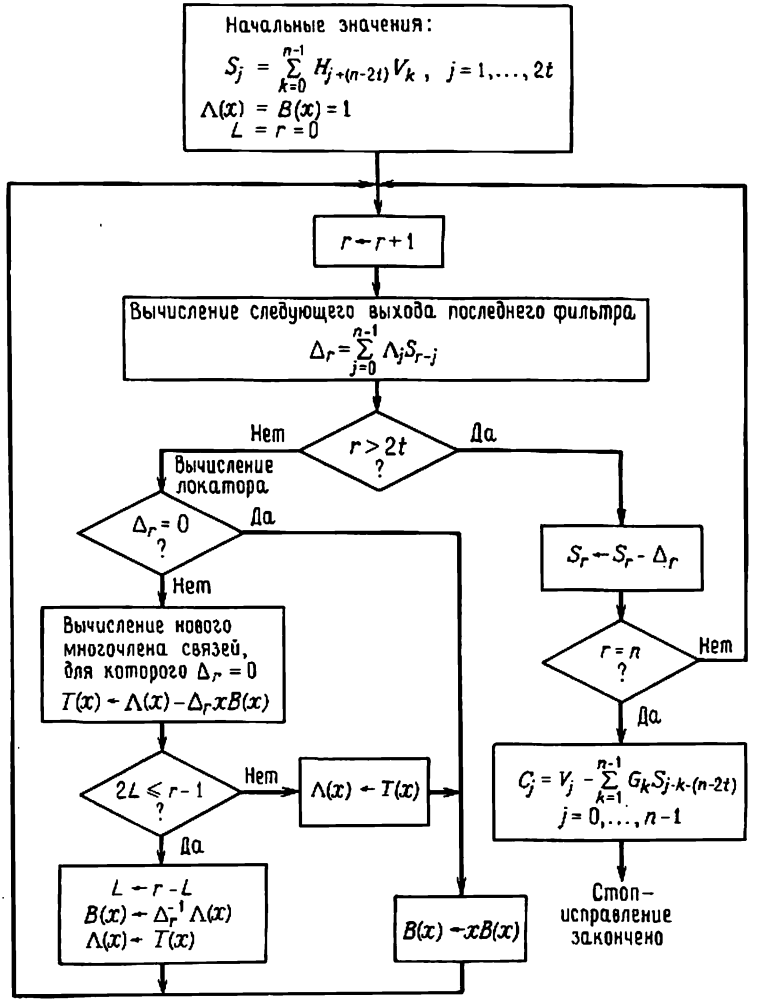


Рис. 9.8. Декодер для альтернантного кода.

поненты синдрома S_{2t+1} и S_{2t+2}, необходимые для исправления $t+1$ ошибок, неизвестны. Но

$$E_j = \sum_{k=0}^{2t} G_k S_{((j-k-n+2t))};$$

следовательно

$$E_1 = G_0 S_{2t+1} + \sum_{k=1}^{2t} G_k S_{((2t+1-k))}, \quad E_2 = G_0 S_{2t+2} + \sum_{k=1}^{2t} G_k S_{((2t+2-k))},$$

и так как $E_2 = E_1^2$, то

$$S_{2t+2} = G_0^{-1}\left[\sum_{k=1}^{2t} G_k S_{2t+2-k} \mid G_0^2 S_{2t+1}^2 + \sum_{k=1}^{2t} G_k^2 S_{2t+1-k}^2\right].$$

Таким образом, в качестве неизвестного надо ввести только S_{2t+1}. В общем случае неизвестными являются компоненты с нечетными индексами, а компоненты синдрома с четными индексами вычисляются. Необходима только половина новых компонент синдрома, а конфигурация ошибок всегда двоична. Если результирующий многочлен локаторов ошибок допустим, то декодирование закончено. К сожалению, для декодирования s ошибок за пределами конструктивного расстояния необходимо просмотреть q^s возможностей для неизвестных компонент синдрома.

9.8. ВЫЧИСЛЕНИЕ ПРЕОБРАЗОВАНИЙ В КОНЕЧНЫХ ПОЛЯХ

Рассмотрим преобразование Фурье

$$V_j = \sum_{i=0}^{n-1} \alpha^{ij} v_i,$$

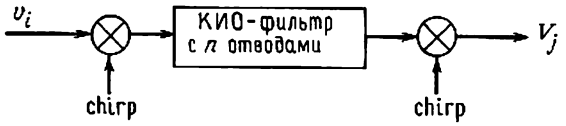
где α — элемент порядка n в поле $GF(q^m)$. Для малых значений n его можно вычислять в том виде, в каком оно записано, но так как вычисление содержит n^2 умножений, то оно становится практически неприемлемым при больших длинах. Необходимы эффективные методы вычисления преобразования Фурье, и мы располагаем некоторыми такими методами. Одним из наиболее мощных методов, пригодных для многих значений длин, является переход от одномерного преобразования к многомерному преобразованию Фурье. Все такие методы известны под общим названием *быстрого преобразования Фурье*. Изучение этих методов мы отложим до гл. 11; в этом же параграфе мы рассмотрим другие методы вычисления преобразования Фурье, а именно chirp-алгоритм Блюстейна, алгоритм Рейдера для простых значений длин преобразования и алгоритм Герцеля.

Представленные на рис. 9.9 алгоритмы Блюстейна и Рейдера являются различными способами перехода от преобразования Фурье к свертке; первый из них переводит n-точечное преобразование Фурье в n-точечную свертку, а второй (который применяется в случае, когда n — простое число) переводит n-точечное преобразование Фурье в $(n-1)$-точечную свертку.

Во всех случаях, когда элемент α имеет квадратный корень, может использоваться chirp-алгоритм Блюстейна. Согласно теореме 4.6.15, каждый элемент поля Галуа имеет квадратный корень. Для полей характеристики 2 $\sqrt{\alpha} = \alpha^{(n+1)/2}$, так как

Chirp-алгоритм Блюстейна:

$$V_j = \sum_{i=0}^{n-1} \alpha^{ij} v_i = \beta^{j^2} \sum_{i=0}^{n-1} \beta^{-(i-j)^2} [\beta^{i^2} v_i]$$



Алгоритм Рейдера:

Длина n должна быть простым числом
В качестве примитивного элемента поля $GF(n)$ используется π.

$$1, 2, 3, \ldots, n-1 = \pi^1, \pi^2, \pi^3, \ldots, \pi^{n-1} (\mathrm{mod}\, n)$$

$$V_j = \sum_{i=0}^{n-1} \alpha^{ij} v_i, \quad i = \pi^{-[n-r(i)]}, \quad j = \pi^{r(j)}$$

$$V_0 = \sum_{i=0}^{n-1} v_i, \quad V_j = v_0 + \sum_{i=1}^{n-1} \alpha^{ij} v_i =$$

$$= V_0 + \sum_{i=1}^{n-1} \left(\alpha^{\pi^{r(i)+r(j)}} - 1\right) v_i,$$

$$j = 0, \ldots, n-1$$

$$V'_l = V_0 + \sum_{k=1}^{n-1} \left(\alpha^{\pi^{l-k}} - 1\right) v'_k,$$

$$l = 0, \ldots, n-1$$

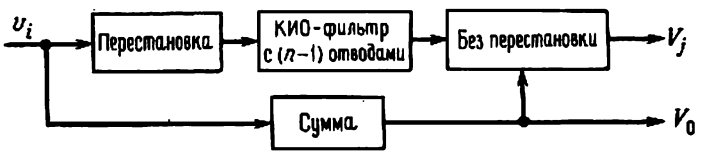


Рис. 9.9. Переход от преобразования Фурье к свертке.

$n+1$ всегда четно. Если характеристика поля отлична от 2, то квадратный корень из α лежит либо в $GF(q^m)$, либо в $GF(q^{2m})$.

Chirp-алгоритм Блюстейна определяется выражением

$$V_j = \beta^{j^2} \sum_{i=0}^{n-1} \beta^{-(j-i)^2} (\beta^{i^2} v_i),$$

где β — квадратный корень из α.

В этом варианте преобразования Фурье производятся следующие вычисления:

$$\beta^{j^2} \sum_{i=0}^{n-1} \beta^{-(j-i)^2} (\beta^{i^2} v_i) = \sum_{i=0}^{n-1} \beta^{2ij} v_i = \sum_{j=0}^{n-1} \alpha^{ij} v_i = V_j.$$

Заметим, что chirp-преобразование состоит из поточечного умножения v_i на β^{i^2} (n умножений) с последующей циклической сверткой (КИО-фильтр с n отводами), за которой снова следует поточечное умножение на β^{j^2} (n умножений). Поэтому полное число операций по-прежнему имеет порядок n^2, так что chirp-алгоритм асимптотически не эффективнее прямого преобразования Фурье. Однако в некоторых приложениях он допускает более простую аппаратурную реализацию. Кроме того, как будет показано в § 11.1, прямое вычисление свертки можно заменить алгоритмом быстрой свертки.

Алгоритм Рейдера можно использовать для вычисления преобразования Фурье в произвольном поле $GF(q)$, если только длина n является простым числом. Так как n — простое число, можно использовать структуру поля $GF(n)$. Это простое поле не следует путать ни с полем $GF(q)$, ни с любым ограничением или расширением этого поля.

Выберем примитивный элемент π простого поля $GF(n)$. Тогда каждое не превосходящее n целое число можно записать как степень элемента π. Перепишем преобразование Фурье в поле $GF(q)$, выделив нулевую частотную и нулевую временную компоненты:

$$V_0 = \sum_{i=0}^{n-1} v_i,$$

$$V_j = v_0 + \sum_{i=1}^{n-1} \alpha^{ij} v_i = V_0 + \sum_{i=1}^{n-1} (\alpha^{ij} - 1) v_i, \quad j = 1, \ldots, n-1.$$

Пусть $r(i)$ обозначает единственное для каждого i от 1 до $n-1$ целое число, такое, что в поле $GF(n)$ выполняется равенство $\pi^{r(i)} = i$. Функция $r(i)$ является перестановкой на множестве

$\{1, 2, \ldots, n-1\}$. Тогда V_j может быть записано в следующем виде:

$$V_{\pi r(j)} = V_0 + \sum_{i=1}^{n-1} \left(\alpha^{\pi r(i)+r(j)} - 1\right) v_{\pi r(i)}.$$

Так как $r(i)$ задает перестановку, можно положить $l = r(j)$ и $k = n - 1 - r(i)$ и выбрать k в качестве индекса суммирования; это дает

$$V_{\pi l} = V_0 + \sum_{k=0}^{n-1} \left(\alpha^{\pi l - k} - 1\right) v_{\pi n-k-1},$$

или

$$V'_l - V_0 = \sum_{k=1}^{n-1} \left(\alpha^{\pi l - k} - 1\right) v'_k,$$

где $V'_l = V_{\pi l}$ и $v'_k = v_{\pi n-k-1}$ — переставленные компоненты входных и выходных последовательностей данных соответственно. Теперь мы получили уравнения для вычисления циклической свертки последовательностей $\{v'_k\}$ и $\{\alpha^{\pi k} - 1\}$. Таким образом, переставляя индексы входных и выходных данных, мы записали преобразование Фурье в виде свертки. Как видно из формул, необходимое для вычисления число операций опять имеет порядок n^2. Однако, так же, как и для алгоритма Блюстейна, свертка может быть вычислена при помощи алгоритма быстрой свертки, который будет описан в § 11.1.

Используя алгоритм Рейдера и равенство $\omega = \alpha^3$, построим двоичное пятиточечное преобразование Фурье; задача состоит в вычислении величин

$$V_j = \sum_{i=0}^{4} \omega^{ij} v_i, \quad j = 0, \ldots, 4.$$

Это преобразование нужно вычислить в поле $GF(16)$, так как наименьшее m, для которого 5 делит $2^m - 1$, равно 4. Элемент 2 является примитивным в поле $GF(5)$, и в этом поле имеем равенства

$$\begin{aligned} 2^0 &= 1, & 2^0 &= 1, \\ 2^1 &= 2, & 2^{-1} &= 3, \\ 2^2 &= 4, & 2^{-2} &= 4, \\ 2^3 &= 3, & 2^{-3} &= 2, \end{aligned}$$

задающие перестановку индексов. Таким образом,

$$V'_l - V_0 = \sum_{k=0}^{3} \left(\omega^{2l-k} - 1\right) v'_k.$$

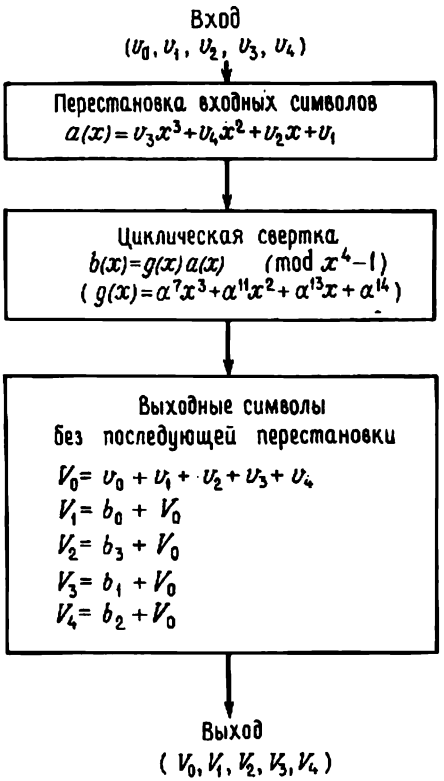


Рис. 9.10. Вычисление пятиточечного преобразования Фурье с помощью алгоритма Рейдера.

Здесь суммирование является четырехточечной циклической сверткой. Полагая коэффициенты g_k равными $\omega^{2^k} - 1$, фильтр Рейдера можно описать многочленом $g(x)$ над $GF(16)$; тогда при $\omega = \alpha^3$

$$g(x) = (\omega^3 - 1)\,x^3 + (\omega^4 - 1)\,x^2 + (\omega^2 - 1)\,x + (\omega - 1) =$$
$$= \alpha^7x^3 + \alpha^{11}x^2 + \alpha^{13}x + \alpha^{14}.$$

Вход и выход фильтра также можно описать многочленами, коэффициенты которых задаются соответственно переставленными компонентами векторов $\mathbf{v}$ и $\mathbf{V}$:

$$a(x) = v_3x^3 + v_4x^2 + v_2x + v_1,$$
$$b(x) = (V_2 - V_0)\,x^3 + (V_4 - V_0)\,x^2 + (V_3 - V_0)\,x + (V_1 - V_0),$$
$$b(x) = g(x)\,a(x) \pmod{x^4 - 1}.$$

Многочлен $g(x)$ фиксирован. Многочлен $a(x)$ получается из многочлена $v(x)$ отбрасыванием первого коэффициента и перестановкой остальных. Коэффициенты многочлена $V(x)$ получаются из коэффициентов многочлена $b(x)$ в естественном порядке. Алгоритм приведен на рис. 9.10. Для выполнения циклической свертки можно выбрать любой удобный метод.

В заключение данного параграфа приведен алгоритм вычисления преобразования Фурье в поле Галуа, основанный на том, что элемент конечного поля содержит в этом поле много сопряженных элементов. Выполнение этого алгоритма требует около $n \log n$ умножений в $GF(q^m)$, но также около n^2 сложений; поэтому он называется *полубыстрым алгоритмом*. Он представляет собой аналог в конечном поле алгоритма вычисления комплексного преобразования Фурье, известного в теории обработки дискретных сигналов под названием *алгоритма Герцеля*. Этот алгоритм требует не более $\log n$ умножений и $n \log n$ сложений символов поля для вычисления одной спектральной компоненты. При вычислении всех компонент спектра это приводит не более чем к $n \log n$ умножениям и $n^2 \log n$ сложениям элементов поля.

Величина $V_j = \sum_i \alpha^{ij} v_i$, j-я компонента преобразования Фурье, представляет собой значение многочлена $v(x)$ в точке α^j. Пусть $f_j(x)$ обозначает минимальный многочлен элемента α^j. Степень m_j этого многочлена не превосходит m. Можно записать

$$v(x) = f_j(x)\, Q(x) + r(x),$$

и, следовательно,

$$v(\alpha^j) = f_j(\alpha^j)\, Q(\alpha^j) + r(\alpha^j) = r(\alpha^j).$$

Таким образом, для вычисления V_j надо разделить $v(x)$ на $f_j(x)$ и вычислить значение полученного остатка $r(x)$ в точке α^j. Поскольку $f_j(x)$ — многочлен степени m_j над полем $GF(q)$, деление на $f_j(x)$ требует $(n - m_j)\, m_j$ умножений элементов поля $GF(q^m)$ на элементы поля $GF(q)$ и $(n - m_j)\, m_j$ сложений в поле $GF(q^m)$. Степень многочлена-остатка $r(x)$ не превосходит $m_j - 1$, так что вычисление величины $r(\alpha^j)$ требует $m_j - 1$ умножений и столько же сложений в $GF(q^m)$. Следовательно, одну компоненту V_j можно вычислить по $v(x)$ с помощью $m_j - 1$ умножений в поле $GF(q^m)$, $(n - m_j)\, m_j$ умножений элементов поля $GF(q^m)$ на элементы поля $GF(q)$ и $(n - m_j + 1)\, m_j - 1$ сложений в поле $GF(q^m)$.

Поскольку преобразованный вектор содержит n компонент, такие вычисления нужно проделать $n = q^m - 1$ раз, что дает не более $m(q^m - 1)$ умножений в поле $GF(q^m)$. Если два элемента сопряжены, то они имеют общий минимальный многочлен и, следовательно, общий остаток $r(x)$. Поэтому нет необходимости каждый раз вычислять промежуточный многочлен $r(x)$, а можно вычислить его один раз для всего класса сопряженных элементов.

В частности, если $q = 2$, то необходимое число умножений в поле $GF(2^m)$ не превосходит $n \log_2 (n + 1)$. В отличие от алгоритма быстрого преобразования Фурье это остается верным даже тогда, когда $2^m - 1$ является простым числом. С другой стороны, число сложений не превосходит величины $(n - 1)\, n + m\,(n - 1) = n^2 + nm - 2n$, так что необходимое число сложений является величиной $O(n^2)$.

Если входной вектор принадлежит малому полю $GF(q)$, то его преобразование (хотя оно является вектором в поле $GF(q^m)$) может быть вычислено без умножений в поле $GF(q^m)$. Это происходит потому, что в таком случае $v(x)$ является многочленом над $GF(q)$ и вычисление величины V_j сводится к умножению элементов из $GF(q)$ на элементы из $GF(q^m)$. Например, если $q = 2$, то это означает сложение элементов из $GF(2^m)$, выбираемых в соответствии с ненулевыми компонентами многочлена $r(x)$. Таким образом, для двоичного кода синдром вычисляется без всяких умножений в поле $GF(2^m)$ не более чем с $n \log n$ сложениями в $GF(2)$ и не более чем с $\log n$ сложениями в $GF(2^m)$.

ЗАДАЧИ

9.1.а. Построить декодер для (15, 8)-кода Препараты, дополняя алгоритм Берлекэмпа—Месси для двоичного случая еще одной итерацией.

б. Используя результат, полученный в п. а, построить декодер для кода Препараты во временно́й области.

9.2. Описать, как исправлять стирания в принятом слове с $2t$ стираниями, дважды используя декодер для исправления только ошибок в двоичном коде. Как использовать такой декодер для одновременного исправления ошибок и стираний?

9.3. Заново решить задачу 7.3.б из гл. 7, используя описанный в § 9.1 алгоритм Евклида.

9.4. Построить блок-схему декодера ошибок и стираний во временно́й области.

9.5. Построить блок-схему декодера во временно́й области для расширенного на одну позицию кода Рида—Соломона.

9.6. Если допустить возможность выполнения обратного преобразования Фурье для $\Omega(x)$ и $\Lambda'(x)$, то во временно́й области можно построить декодер с $2t$ тактами работы, использующий алгоритм Форни. Выписать итеративные уравнения для вычисления многочленов $\Omega(x)$ и $\Lambda'(x)$, аналогичные итеративному алгоритму Берлекэмпа—Месси. Использовать обратное преобразование Фурье этих уравнений для построения декодера с $2t$ тактами работы во временной области.

9.7. Построить алгоритм Форни для произвольного j_0, комбинируя решение задачи 8.1 и утверждение теоремы 7.5.2.

9.8. Выписать полностью уравнения пятиточечного преобразования Фурье в поле $GF(16)$, вычисляемого по алгоритму Рейдера. Построить диаграмму перестановки компонент векторов и блок-схему КИО-фильтра с указанием весовых множителей.

9.9. Использовать алгоритм Герцеля для вычисления в поле $GF(8)$ преобразования Фурье вектора, задаваемого многочленом

$$v(x) = \alpha^3 x^6 + \alpha^5 x^5 + x^4 + \alpha x^3 + \alpha^3 x^2 x + 1.$$

ЗАМЕЧАНИЯ

Предложенный Ридом и Соломоном [1960] метод декодирования аналогичен описанному в данной главе методу декодирования в частотной области, но первоначально он разрабатывался для временно́й области. Для частотной области декодер впервые предложил Мандельбаум [1971], хотя он использовал терминологию китайской теоремы об остатках. Программная реализация такого декодера была сделана Пашбургом [1974].

Первое обсуждение декодирования со спектральной точки зрения можно найти у Гора [1973]. Он заметил, что информация может быть закодирована в частотной области и что спектр вектора ошибок можно вычислять с помощью рекуррентного продолжения. Реализация декодеров с преобразованиями областей рассматривалась также Михельсоном [1976]. Общее обсуждение спектральных методов декодирования можно найти у Блейхута [1979]. Отображение этих декодеров обратно во временну́ю область было проведено в работах Блейхута [1979, 1980]. Основанный на алгоритме Евклида метод декодирования предложен Мандельбаумом [1977].

Замечания о канале со стираниями принадлежат Элайсу [1954]. Ранние работы по обобщению алгоритмов декодирования кодов БЧХ на случай не только ошибок, но и стираний выполнены Блюмом и Вейсом [1960], а также Форни [1965]. Описанный в настоящей главе подход принадлежит Блейхуту [1979].

Декодирование расширенных кодов было впервые описано в работах Вулфа [1969] и Касахары с соавторами [1975]. Описанный в настоящей главе вариант принадлежит Блейхуту [1980]. Декодирование альтернантных кодов и кодов Гоппы разработано Паттерсоном [1975] и Хельгертом [1977]. Дельсарт [1975] указал, что эти коды можно декодировать с помощью декодера для кодов Рида—Соломона.

Декодирование за пределами границы БЧХ предложили Берлекэмп [1968], Хартман [1972] и Блейхут [1979]. Алгоритм полного алгебраического декодирования кодов БЧХ с исправлением двух ошибок описан Берлекэмпом [1968], а алгоритм полного алгебраического декодирования некоторых кодов БЧХ с исправлением трех ошибок — Вандерхорстом и Бергером [1976].

Алгоритмы Блюстейна [1970], Рейдера [1968] и Герцеля [1958] были предложены этими авторами в связи с задачами дискретной обработки комплекснозначных сигналов. Дополнительные сведения об этих алгоритмах можно найти в монографиях Оппенгейма и Шафера [1975] или Рабинера и Голда [1975]. Сервейт [1978] предложил свой вариант полубыстрого алгоритма.

ГЛАВА 10

МНОГОМЕРНЫЕ СПЕКТРАЛЬНЫЕ МЕТОДЫ

Подобно тому как было введено одномерное преобразование Фурье над конечным полем, можно определить многомерное преобразование Фурье над конечным полем. Оно представляет собой преобразование, определяемое на многомерных таблицах данных, и полезно во многих отношениях. Многомерные спектральные методы дают возможность построить новые коды, которые допускают более эффективные схемы декодирования или обладают новыми свойствами. Многомерные спектральные методы являются также полезным инструментом исследования структуры циклических кодов, в частности кодов БЧХ. Многомерные коды тесно связаны с современными методами теории обработки дискретных сигналов, основанными на китайских теоремах об остатках. Наилучшими известными многомерными кодами являются коды-произведения, и наше изложение мы начнем с описания этих кодов.

Корректирующая способность построенных с помощью многомерных методов кодов в общем случае ниже корректирующей способности уже рассмотренных кодов. Мы останавливаемся на изучении этих кодов потому, что они дают более глубокое проникновение в существо методов, потому, что кодовые слова иногда удобнее обрабатывать в виде многомерных таблиц и потому, что таким образом можно построить хорошие коды, одновременно исправляющие и случайные ошибки, и пакеты ошибок.

10.1. КОДЫ-ПРОИЗВЕДЕНИЯ

Если длина линейного кода $n = n_1 n_2$, то компоненты кодового слова можно расположить в виде $(n_1 \times n_2)$-таблицы. В некоторых случаях такая таблица, элементы которой мы обозначим через c_{ij} или c_{ij}, является более удобной, так как позволяет при описании и обработке рассматривать строки и столбцы по отдельности. В этом случае код называется *многомерным кодом*. Размерность кода как векторного пространства, конечно, остается равной числу k ин-

формационных символов; лишь таблица индексов является многомерной. Мы будем рассматривать только двумерный случай, хотя все идеи применимы и при размерности, большей двух.

Начнем с простейшего случая многомерных кодов — с кодов-произведений.

Определение 10.1.1. *Кодом-произведением* $\mathscr{C} = \mathscr{C}_1 \times \mathscr{C}_2$ двух кодов $\mathscr{C}_1$ и $\mathscr{C}_2$ называется код, словами которого являются все двумерные таблицы со строками, являющимися словами кода $\mathscr{C}_1$, и столбцами, являющимися словами кода $\mathscr{C}_2$.

Систематическое представление кода $\mathscr{C}$ получается из систематических представлений кодов $\mathscr{C}_1$ и $\mathscr{C}_2$, соответственно первые k_1 и k_2 символов которых являются информационными символами. Слово такого кода $\mathscr{C}$ приведено на рис. 10.1, где виден и способ кодирования: сначала кодом $\mathscr{C}_1$ кодируется каждая из первых k_2 строк таблицы, а затем кодом $\mathscr{C}_2$ кодируются все столбцы таблицы. Легко проверить, что то же слово можно получить, кодируя сначала первые k_1 столбцов, а потом все строки. Для последовательной передачи по каналу кодовое слово разбивается на части, например считывается по строкам или по столбцам.

Слово кода-произведения может быть записано в виде многочлена от двух переменных. А именно если компоненты кодового слова обозначить $c_{ii'}$, $i = 0, \ldots, n_1 - 1$, $i' = 0, \ldots, n_2 - 1$, то имеем следующее представление:

$$c(x, y) = \sum_{i=0}^{n_1-1} \sum_{i'=0}^{n_2-1} c_{ii'} x^i y^{i'}.$$

Группируя слагаемые, это выражение можно переписать любым из двух способов:

$$c(x, y) = \sum_{i=0}^{n_1-1} \left(\sum_{i'=0}^{n_2-1} c_{ii'} y^{i'} \right) x^i,$$

или

$$c(x, y) = \sum_{i'=0}^{n_2-1} \left(\sum_{i=0}^{n_1-1} c_{ii'} x^i \right) y^{i'}.$$

Согласно определению кода-произведения, в первом выражении заключенная в скобки сумма (для каждого i) является словом кода $\mathscr{C}_2$, а во втором выражении заключенная в скобки сумма (для каждого i') является словом кода $\mathscr{C}_1$.

Коды-сомножители $\mathscr{C}_1$ и $\mathscr{C}_2$ могут быть циклическими, однако это не означает, что код-произведение $\mathscr{C}$ также является циклическим. В § 10.2 будут выписаны достаточные для цикличности $\mathscr{C}$ условия.

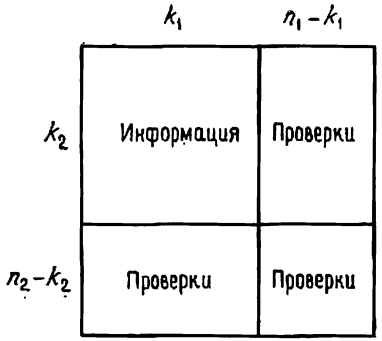


Рис. 10.1. Структура кодового слова систематического (n_1n_2, k_1k_2)-кода.

Теорема 10.1.2. *Если минимальные расстояния кодов* $\mathscr{C}_1$ *и* $\mathscr{C}_2$ *равны* d_1^* *и* d_2^* *соответственно, то минимальное расстояние кода* $\mathscr{C}$ *равно* $d_1^*d_2^*$.

Доказательство. Код является линейным, и кодовое слово минимального веса в каждом ненулевом столбце содержит не менее d_2^* ненулевых символов, а число ненулевых столбцов в таком слове равно по меньшей мере d_1^*, так что минимальный вес равен по меньшей мере $d_1^*d_2^*$. Кроме того, существует хотя бы одно такое слово. □

Из этой теоремы следует, что если составляющие коды могут исправлять t_1 и t_2 ошибок соответственно, то код-произведение может исправлять $2t_1t_2 + t_1 + t_2$ ошибок.

Если на рис. 10.1 положить $n_2 - k_2$ равным нулю и считывать кодовое слово по столбцам, то получим перемежение k_2 слов строчного (n_1, k_1)-кода. Таким образом, код-произведение является обобщением кода-перемежения. Он подобен коду-перемежению, но некоторые его строки содержат только проверки. Отсюда становится ясным, что код-произведение можно использовать как код, исправляющий пакеты ошибок. Он позволяет исправлять пакеты ошибок длины n_2b_1, если строчный код исправляет пакеты длины b_1. При считывании слов того же самого кода-произведения по строкам он исправляет пакеты ошибок длины n_1b_2, если столбцовый код исправляет пакеты длины b_2. При надлежащем выборе правила линейного считывания слов кода-произведения этот код можно использовать для исправления пакетов ошибок длины $\max(n_1b_2, n_2b_1)$. Мы видели, что этот же код может исправлять $(d_1^*d_2^* - 1)/2$ случайных ошибок. Следующая теорема утверждает, что такой код можно использовать для одновременного исправления и случайных ошибок, и их пакетов.

Теорема 10.1.3. *Ни один смежный класс кода, построенного как произведение линейного* (n_1, k_1, d_1^*)*-кода и линейного* $(n_2, k_2,$

d_2^*)-кода, не содержит одновременно слова веса не больше t при $t = (d_1^* d_2^* - 1)/2$ и слова, представляющего собой пакет ошибок длины не больше $\max(n_2 t_1, n_1 t_2)$ *при* $t_1 \leqslant (d_1^* - 1)/2$ *и* $t_2 \leqslant (d_2^* - 1)/2$.

Доказательство. Для определенности предположим, что $n_2 t_1 > n_1 t_2$ и что код считывается по столбцам. Если пакет ошибок длины не более $n_2 t_1$ и отличная от него конфигурация ошибок веса не более t лежат в одном смежном классе, то их разность представляет собой кодовое слово. Вес каждой ненулевой строки такого кода равен не меньше d_1^*. Так как длина пакета не превосходит $n_2 t_1$, то каждая ненулевая строка может содержать не более t_1 ошибок из этого пакета и, следовательно, должна содержать по меньшей мере $d_1^* - t_1$ других случайных ошибок. Таким образом, t случайных ошибок разбросаны самое большое по $\lfloor t/(d_1^* - t_1) \rfloor$ строкам. Следовательно, кодовое слово содержит не более этого количества ненулевых строк, в каждой из которых имеется не более t_1 ошибок, принадлежащих пакету. Полное число ошибок не превосходит $\lfloor t/(d_1^* - t_1) \rfloor t_1 + t$, но, поскольку $d_1^* - t_1$ больше t_1, это число меньше $2t$. Это противоречит тому, что минимальный вес кода равен по меньшей мере $2t + 1$ и поэтому такое кодовое слово не может существовать, и, следовательно, рассматриваемые пакет ошибок и конфигурация случайных ошибок не могут принадлежать одному и тому же смежному классу. □

Код-произведение можно использовать в соответствии с доказанной теоремой. Декодер для такого кода можно строить в виде составного декодера, содержащего декодер для исправления пакетов ошибок и декодер для исправления случайных ошибок, каждый из которых исправляет требуемое число ошибок и обнаруживает неисправляемые конфигурации ошибок. При этом либо оба декодера укажут одну и ту же конфигурацию ошибок, либо успешно декодирует только один из них; в последнем случае второй декодер укажет неисправляемую конфигурацию ошибок. Иногда декодер для исправления случайных ошибок может указать все пакеты ошибок. В § 10.3 будет описан декодер для произведения двух кодов БЧХ, исправляющий все случайные ошибки в пределах конструктивного расстояния.

10.2. КИТАЙСКИЕ ТЕОРЕМЫ ОБ ОСТАТКАХ

Когда можно однозначно определить целое число, если заданы только его вычеты по модулям нескольких целых чисел? Ответ на этот вопрос был известен еще в древнем Китае. Мы сейчас рассмотрим эту теорему, а также ее современные аналоги, относящиеся к множеству полиномиальных модулей. Китайская теорема об остатках будет доказана в два этапа. Сначала мы докажем единственность решения, а затем его существование.

Теорема 10.2.1. *Для заданного множества целых положительных попарно взаимно простых чисел $m_1, m_2, \ldots, m_k$ и множества неотрицательных целых чисел $c_1, c_2, \ldots, c_k$ при $c_i < m_i$ система сравнений*

$$c_i = c \pmod{m_i}, \quad i = 1, \ldots, k,$$

имеет не более одного решения c в интервале $0 \leqslant c < \prod_{i=1}^{k} m_i$.

Доказательство. Предположим, что c и c' являются двумя лежащими в рассматриваемом интервале решениями. Тогда

$$c = Q_i m_i + c_i \quad \text{и} \quad c' = Q_i' m_i + c_i,$$

и, следовательно, $c - c'$ кратно m_i для каждого i, а так как m_i попарно взаимно просты, то $c - c'$ кратно $\prod_{i=1}^{k} m_i$. Но число $c - c'$ лежит между $-(\prod_{i=1}^{k} m_i - 1)$ и $\prod_{i=1}^{k} m_i - 1$. Единственным положительным числом, удовлетворяющим этим условиям, является $c - c' = 0$. Следовательно, $c = c'$. □

Для того чтобы практически найти решение выписанной в теореме 10.2.1 системы сравнений, воспользуемся следствием 4.1.4 из алгоритма Евклида, согласно которому в кольце целых чисел НОД $(r, s) = ar + bs$ для некоторых целых a и b.

Для заданного множества попарно взаимно простых положительных целых чисел $m_1, m_2, \ldots, m_k$ положим $M = \prod_{i=1}^{k} m_i$ и $M_i = M/m_i$. Тогда НОД $(M_i, m_i) = 1$, и, следовательно, существуют такие целые N_i и n_i, что

$$N_i M_i + n_i m_i = 1.$$

Теперь можно доказать следующую теорему.

Теорема 10.2.2. *Пусть $M = \prod_{i=1}^{k} m_i$ — произведение попарно взаимно простых положительных целых чисел, пусть $M_i = M/m_i$, и пусть N_i удовлетворяют равенству $N_i M_i + n_i m_i = 1$. Тогда единственным решением системы сравнений*

$$c_i = c \pmod{m_i}, \quad i = 1, \ldots, k,$$

будет

$$c = \sum_{i=1}^{k} c_i N_i M_i \pmod{M}.$$

Доказательство. Поскольку мы уже знаем, что решение рассматриваемой системы сравнений единственно, надо только доказать, что выписанное выше c действительно является решением. Но

$$c = \sum_{r=1}^{k} c_r N_r M_r \pmod{m_i} = c_i N_i M_i \pmod{m_i},$$

ибо m_l делит M_r при $r \neq i$. Наконец, так как $N_iM_i + n_im_i = 1$, то $N_iM_i = 1 \pmod{m_i}$ и $c = c_i \pmod{m_i}$, что и завершает доказательство. □

Теперь мы перейдем к кольцу многочленов над некоторым полем. В случае любого такого кольца справедлива китайская теорема об остатках для многочленов, которая доказывается точно так же, как и в случае кольца целых чисел.

Теорема 10.2.3. *Для заданного множества попарно взаимно простых многочленов $m_1(x), m_2(x), \ldots, m_k(x)$ и множества многочленов $c_1(x), c_2(x), \ldots, c_k(x)$, таких, что* $\deg c_i(x) < \deg m_i(x)$, *система сравнений*

$$c_i(x) = c(x) \pmod{m_i(x)}, \quad i = 1, \ldots, k,$$

имеет не более одного решения $c(x)$, удовлетворяющего условию

$$\deg c(x) < \sum_{i=1}^{k} \deg m_i(x).$$

Доказательство. По существу доказательство совпадает с доказательством теоремы 10.2.1. Предположим, что имеются два решения, а именно

$$c(x) = Q_i(x)\, m_i(x) + c_i(x)$$

и

$$c'(x) = Q_i'(x)\, m_i(x) + c_i(x),$$

так что разность $c(x) - c'(x)$ кратна многочлену $m_i(x)$ для каждого i. Тогда многочлен $c(x) - c'(x)$ кратен и многочлену $\prod_{i=1}^{k} m_i(x)$, причем

$$\deg [c(x) - c'(x)] < \deg \left[\prod_{i=1}^{k} m_i(x) \right].$$

Следовательно, $c(x) - c'(x) = 0$, и доказательство закончено. □

Так же как и в кольце целых чисел, в кольце многочленов над произвольным полем выполняется равенство

$$\text{НОД } [r(x), s(x)] = a(x)\, r(x) + b(x)\, s(x)$$

для некоторых $a(x)$ и $b(x)$. Для заданного множества попарно взаимно простых $m_i(x)$ положим $M(x) = \prod_{r=1}^{k} m_r(x)$ и $M_i(x) = M(x)/m_i(x)$ и допустим, что $N_i(x)$ и $n_i(x)$ удовлетворяют равенствам $N_i(x)\, M_i(x) + n_i(x)\, m_i(x) = 1$; согласно следствию 4.3.7, такие многочлены $N_i(x)$ и $n_i(x)$ всегда существуют.

Теорема 10.2.4. *Пусть $M(x) = \prod_{r=1}^{k} m_i(x)$ — произведение попарно взаимно простых многочленов, и пусть $M_i(x) = M(x)/m_i(x)$*

и $N_i(x)$ *удовлетворяют равенствам* $N_i(x)\,M_i(x) + n_i(x)\,m_i(x) = 1$. *Тогда единственным решением системы сравнений*

$$c_i(x) = c(x) \pmod{m_i(x)}, \quad i = 1, \ldots, k,$$

является

$$c(x) = \sum_{i=1}^{k} c_i(x)\,N_i(x)\,M_i(x) \pmod{M(x)}.$$

Доказательство. Так как мы уже знаем, что рассматриваемая система сравнений имеет не более одного решения, то нам надо только доказать, что выписанное выше $c(x)$ действительно является решением. Но

$$c(x) = c_i(x)\,N_i(x)\,M_i(x) \pmod{m_i(x)},$$

ибо $m_i(x)$ делит $M_r(x)$, если $r \neq i$. Наконец, так как

$$N_i(x)\,M_i(x) + n_i(x)\,m_i(x) = 1,$$

то $N_i(x)\,M_i(x) = 1 \pmod{m_i(x)}$ и $c(x) = c_i(x) \pmod{m_i(x)}$, что и завершает доказательство. □

Китайская теорема об остатках позволяет установить условия, при которых произведение циклических кодов является циклическим кодом.

Теорема 10.2.5. *Если длины* n_1 *и* n_2 *циклических кодов* $\mathscr{C}_1$ *и* $\mathscr{C}_2$ *взаимно просты, то при соответствующем расположении компонент в кодовом слове код-произведение* $\mathscr{C}$ *также является циклическим.*

Доказательство. Если $0 \leqslant i \leqslant n_1 - 1$ и $0 \leqslant i' \leqslant n_2 - 1$, то, согласно китайской теореме об остатках, имеется единственное целое число $I(i, i')$, лежащее в интервале $0 \leqslant I(i, i') \leqslant n_1 n_2 - 1$ и удовлетворяющее сравнениям

$$I(i, i') = i \pmod{n_1},$$
$$I(i, i') = i' \pmod{n_2}.$$

Перепишем кодовое слово $c(x, y)$ в следующем порядке:

$$c(z) = \sum_{I=0}^{n_1 n_2 - 1} c_{(I \bmod n_1)\,(I \bmod n_2)}\,z^I.$$

Чтобы увидеть, что такое множество слов $c(z)$ образует циклический код, достаточно заметить, что в указанной записи $zc(z) \pmod{z^{n_1 n_2} - 1}$ соответствует многочлену

$$xyc(x, y) \pmod{x^{n_1} - 1} \pmod{y^{n_2} - 1}.$$

Так как этот многочлен является кодовым словом, то и $zc(z) \pmod{z^{n_1 n_2} - 1}$ является кодовым словом. □

Взаимосвязь между $c(z)$ и $c(x, y)$ можно лучше пояснить с помощью следующей двумерной таблицы:

$$c_{ii'} = \begin{bmatrix} c_{00} & c_{01} & c_{02} & \dots & c_{0(n_1-1)} \\ c_{10} & c_{11} & c_{12} & \dots & c_{1(n_1-1)} \\ \cdot & \cdot & \cdot & & \cdot \\ \cdot & \cdot & \cdot & & \cdot \\ \cdot & \cdot & \cdot & & \cdot \\ c_{(n_2-1)0} & c_{(n_2-1)1} & c_{(n_2-1)2} & \dots & c_{(n_2-1)(n_1-1)} \end{bmatrix}.$$

Коэффициенты многочлена $c(z)$ получаются при считывании элементов этой таблицы вдоль диагонали, начиная с c_{00} и возвращаясь после каждого достижения края таблицы, так, как если бы таблица была продолжена периодически в обоих направлениях. Так как n_1 и n_2 взаимно просты, такая расширенная диагональ не пройдет дважды через один и тот же элемент до тех пор, пока все $n_1 n_2$ элементов не будут пройдены.

10.3. ДЕКОДИРОВАНИЕ КОДА-ПРОИЗВЕДЕНИЯ

Если $\nu \leqslant (d_1^* d_2^* - 1)/2$, то код-произведение способен исправлять ν ошибок. Такая теоретическая корректирующая способность кода, однако, мало что дает без практических алгоритмов декодирования. В данном параграфе описывается такой алгоритм декодирования, основанный на исправляющем только ошибки подалгоритме для одного кода-сомножителя и на исправляющем ошибки и стирания подалгоритме для другого кода-сомножителя. Такие декодеры были описаны только для кодов БЧХ, и, следовательно, мы будем говорить только о произведениях кодов БЧХ.

Рассмотрим случай, когда $d_1^* d_2^*$ нечетно, и без ограничения общности предположим, что $d_2^* \geqslant d_1^*$. Задача декодера, исправляющего в произведении кодов БЧХ только случайные ошибки, состоит в вычислении некоторого кодового слова, находящегося от принятого слова не далее радиуса упаковки $t = (d_1^* d_2^* - 1)/2$, так как такое слово единственно. Декодер состоит из внутреннего декодера для кода БЧХ и внешней цепи обработки, содержащей произвольный подходящий декодер для кода БЧХ, исправляющий ошибки и стирания. Сначала опишем упрощенный вариант декодера, потом улучшим его, а затем докажем, что он выполняет нужные исправления.

Сначала декодер должен «исправлять» каждый столбец. Для алгоритма декодирования безразлично, передается ли кодовое слово по строкам или по столбцам, но при обсуждении этого алгоритма удобнее предполагать, что кодовое слово передается по столбцам, и воспринимать столбец как отдельно декодируемый

подблок. Позже, при декодировании строк, неправильные подблоки будут выявлены и исправлены.

Пусть в j-м столбце принятого слова произошло ν_j ошибок; тогда

$$\nu = \sum_{j=0}^{n_1-1} \nu_j.$$

Первый шаг состоит в использовании БЧХ-декодера для исправления каждого столбца. Для каждого j-го исправленного столбца указывается число ω_j сделанных исправлений. Это число представляет собой вес оценки $\hat{e}_i(j)$ вектора-столбца ошибки и равно ν_j, если декодирование произведено правильно. Если обнаружена неисправляемая ошибка, то столбец стирается. Нестертые столбцы с наибольшими значениями чисел ω_j рассматриваются строчным декодером как наименее надежные.

Затем исправляющий ошибки и стирания БЧХ-декодер применяется для исправления каждой строки. Теперь, однако, имеется сторонняя информация, которую можно использовать для правомерного стирания дополнительных столбцов. Сначала, игнорируя эту стороннюю информацию, декодируем, как удастся, все строки, начиная с первой. Строчный декодер может потерпеть неудачу и может внести ненулевую конфигурацию ошибок. Любой из этих случаев означает, что некоторые столбцы были декодированы неправильно. Требуется какое-то правило, указывающее, что нужно делать: принять декодированную строку или стереть наименее надежный столбец и попытаться декодировать строку сначала.

Основную теорему мы докажем сначала в предположении, что столбцовый декодер не производит никаких стираний. Впоследствии эта теорема будет использована и при наличии стираний с помощью простого уменьшения числа d_1^* на число стертых столбцов. Рассматриваемые методы доказательства будут использованы в гл. 15, когда мы будем изучать декодирование с мягким решением.

Что можно сказать о строчном декодере, если дано, что код-произведение может исправлять $t = (d_1^* d_2^* - 1)/2$ ошибок и что столбцовый декодер уже исправил ω_j ошибок в j-м столбце? Предположим, что i-я строка декодирована и для нее вычислена оценка $\{\hat{e}_j(i)\}$ вектора-строки ошибки. Обозначим через U_i множество индексов j, для которых $\hat{e}_j(i)$ отлично от нуля. Если j принадлежит множеству U_i, то j-й столбец продекодирован неправильно; в j-м столбце имеется по меньшей мере d_2^* ошибок, среди которых ω_j могли быть внесены столбцовым декодером, причем ω_j меньше d_2^*. Следовательно, если строчному декодеру удастся правильно найти конфигурацию ошибок, то в исходном принятом слове в j-м

столбце содержалось по меньшей мере $d_2^* - \omega_j$ ошибок для каждого j из множества U_i и, как уже известно, ω_j ошибок для каждого j, не принадлежащего множеству U_i. Таким образом, разумное условие принятия результата декодирования i-й строки дается следующим неравенством:

$$\sum_{j \in U_i} (d_2^* - \omega_j) + \sum_{j \notin U_i} \omega_j \leqslant t.$$

В приведенной ниже теореме доказывается, что это неравенство является необходимым и достаточным условием правильности декодирования i-й строки. Сначала перепишем это неравенство в форме скалярного произведения, удобной для использования в доказательстве теоремы. Для этого представим его в виде

$$2 \sum_{j \in U_i} (1 - \omega_j/d_2^*) + 2 \sum_{j \notin U_i} (\omega_j/d_2^*) < d_1^*.$$

Тогда

$$\sum_{j \in U_i} 1 + \sum_{j \notin U_i} 1 - 2 \sum_{j \in U_i} (1 - \omega_j/d_2^*) - 2 \sum_{j \notin U_i} (\omega_j/d_2^*) > n_1 - d_1^*.$$

Пусть ξ_j равно -1, если j принадлежит U_i, и 1 в противном случае. Тогда

$$\sum_{j=0}^{n_1-1} \xi_j (1 - 2\omega_j/d_2^*) > n_1 - d_2^*.$$

Если, наконец, $a_j = 1 - 2\omega_j/d_2^*$, то неравенство приводится к виду

$$\sum_{j=0}^{n_1-1} \xi_j a_j > n_1 - d_1^*.$$

Теперь все готово для доказательства теоремы.

Теорема 10.3.1. *Пусть ω_j — число позиций, в которых j-й столбец исправленной столбцовым декодером таблицы отличается от принятого слова. Для каждого i существует в точности одно слово строчного кода, удовлетворяющее неравенству*

$$\sum_{j \in U_i} (d_2^* - \omega_j) + \sum_{j \notin U_i} \omega_j \leqslant t.$$

Здесь U_i — множество номеров j позиций, в которых компоненты i-й строки кодового слова отличаются от компонент этой строки в исправленной столбцовым декодером таблице.

Доказательство. На первом шаге мы докажем, что правильное слово строчного кода в i-й строке удовлетворяет выписанному неравенству. На втором шаге мы докажем единственность такого слова.

Шаг 1. Пусть φ — число неправильно декодированных столбцов, и пусть U_φ — множество индексов этих столбцов. Тогда $n_1 - \varphi$ столбцов декодированы правильно. Но

$$\sum_{i \in U_\varphi} \nu_i + \sum_{i \notin U_\varphi} \nu_i \leqslant t,$$

и число ν_j произошедших в j-м столбце принятого слова ошибок удовлетворяет равенству $\nu_j = \omega_j$, если j-й столбец декодирован правильно, и неравенству $\nu_j \geqslant d_2^* - \omega_j > \omega_j$, если j-й столбец декодирован неправильно. Множество U_i декодированных столбцов, содержащих в i-й строке ошибку, удовлетворяет включению $U_i \subset U_\varphi$; следовательно,

$$\sum_{i \in U_i} (d_2^* - \omega_i) + \sum_{i \notin U_i} \omega_i \leqslant t,$$

а это и доказывает, что для каждой строки кодовое слово удовлетворяет данному неравенству.

Шаг 2. Рассмотрим i-ю строку. Предположим, что $\mathbf{c}_r$ и $\mathbf{c}_{r'}$ являются словами строчного кода, и обозначим i-ю строку после «исправления» столбцов через $\boldsymbol{\mu}$. Пусть ξ_j равно 1, если $\mathbf{c}_r$ совпадает с $\boldsymbol{\mu}$ в j-й компоненте, и ξ_j — —1 в противном случае. Пусть ξ'_j обозначает аналогичную величину для кодового слова $\mathbf{c}_{r'}$. Определим множества

$$T = \{j \mid c_{rj} = c_{r'j}\},$$
$$V = \{j \mid c_{rj} \neq c_{r'j} \quad \text{и} \quad c_{rj} = \mu_j\},$$
$$W = \{j \mid c_{rj} \neq c_{r'j} \quad \text{и} \quad c_{rj} \neq \mu_j\}.$$

Каждый индекс j принадлежит одному из этих трех множеств; тогда

$$\sum_{j=0}^{n_1-1} \xi_j a_j = \sum_{j \in T} \xi_j a_j + \sum_{j \in V} a_j - \sum_{j \in W} a_j = A_T + A_V - A_W,$$

где величины A_T, A_V и A_W определены очевидным образом. Число элементов в T не превосходит $n_1 - d_1^*$. Если $\mathbf{c}_r$ удовлетворяет условиям теоремы, то

$$\sum_{j=0}^{n_1-1} \xi_j a_j = A_T + A_V - A_W > n_1 - d_1^*,$$

и, следовательно, $A_V - A_W > 0$.

Теперь рассмотрим кодовое слово $\mathbf{c}'_r$. Мы имеем

$$\sum_{j=0}^{n_1-1} \xi'_j a_j = \sum_{j \in T} \xi'_j a_j + \sum_{j \in V} \xi'_j a_j + \sum_{j \in W} \xi'_j a_j = A_T - A_V + B_W,$$

где величина B_W определена очевидным образом и на множестве V выполняется равенство $c_{r\cdot j} \neq \mu_j$. Но $B_W \leqslant A_W$, и, как мы видели, $A_W - A_V < 0$. Следовательно,

$$\sum_{j=0}^{n_1-1} \xi_j a_j \leqslant A_T - A_V + A_W < A_T \leqslant n_1 - d_1^*,$$

так что $\mathbf{c}_{r\cdot}$ не удовлетворяет неравенству. □

Опишем теперь практический способ вычисления такого кодового слова. Мы уже знаем, как строить декодеры для исправления ошибок и стираний с жестким решением. К любому из таких декодеров присоединим внешнюю цепь обработки. По имеющейся таблице декодированных столбцов для i-й строки построим пробную цепочку векторов, полученных при жестком декодировании ошибок и стираний. Индексы упорядочим следующим образом: $\omega_{j_1} \geqslant \omega_{j_2} \geqslant \ldots \geqslant \omega_{j_{n_1}}$ или $a_{j_1} \leqslant a_{j_2} \leqslant \ldots \leqslant a_{j_{n_1}}$. Для каждого l из интервала от 0 до $d_1^* - 1$ будем стирать l компонент с наибольшими значениями ω_j. Такое построение приведет к вектору $\boldsymbol{\mu}^{(l)}$ со стираниями. Декодируем этот вектор декодером для исправления ошибок и стираний и проверим результат декодирования $\mathbf{c}_r$ на соответствие условию, даваемому теоремой 10.3.1. Если он удовлетворяет этому условию, то $\mathbf{c}_r$ будет кодовым словом для данной строки; в противном случае увеличим l и повторим процедуру. Следующая теорема доказывает, что описанная процедура всегда приводит к кодовому слову.

Теорема 10.3.2. *Если кодовое слово* $\mathbf{c}_r$ *удовлетворяет неравенству*

$$\sum_{j=0}^{n_1-1} \xi_j a_j > n_1 - d_1^*,$$

то хотя бы для одного l *выполняется условие*

$$\sum_{j=0}^{n_1-1} \xi_j b_j^{(l)} > n_1 - d_1^*,$$

где

$$b_{j_1}^{(l)} = b_{j_2}^{(l)} = \ldots = b_{j_l}^{(l)} = 0, \quad b_{j_{l+1}}^{(l)} = b_{j_{l+2}}^{(l)} = \ldots = b_{j_{n_1}}^{(l)} = 1,$$

и, следовательно, $\mathbf{c}_r$ *представляет собой единственное кодовое слово, вычисляемое исправляющим ошибки и стирания декодером.*

Доказательство. Доказательство достаточно провести для всех l от 0 до n_1, так как при $l \geqslant d_1^*$ утверждение теоремы, очевидно, не может выполняться.

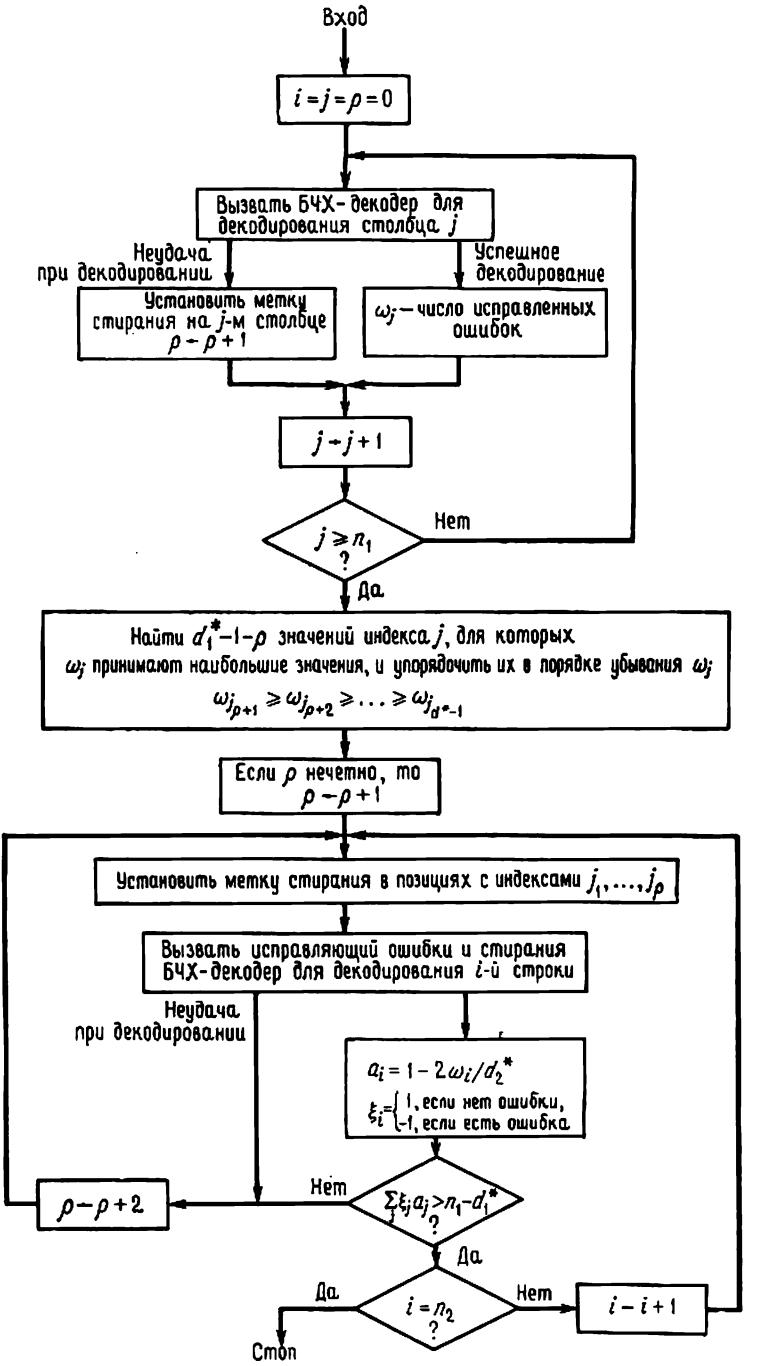


Рис. 10.2. Декодер для кода-произведения кодов БЧХ.

Идея доказательства состоит в том, что μ принадлежит выпуклой оболочке множества векторов $\{\mu^{(l)}\}$. Пусть

$$\lambda_0 = a_{j_1},$$
$$\lambda_l = a_{j_{l+1}} - a_{j_l}, \quad 1 \leqslant l \leqslant n_1 - 1,$$
$$\lambda_{n_1} = 1 - a_{j_{n_1}},$$

откуда следует, что $0 \leqslant \lambda_l \leqslant 1$ и $\sum_{l=0}^{n_1} \lambda_l = 1$. Кроме того,

$$\sum_{l=0}^{l'-1} \lambda_l = a_{j_{l'}} \quad \text{и} \quad a_j = \sum_{l=0}^{n_1} \lambda_l b_j^{(l)}.$$

Предположим теперь, что для всех l

$$\sum_{j=0}^{n_1-1} \xi_j b_j^{(l)} \leqslant n_1 - d_1^*.$$

Тогда

$$\sum_{j=0}^{n_1-1} \xi_j a_j = \sum_{l=0}^{n_1} \lambda_l \sum_{j=0}^{n_1-1} \xi_j b_j^{(l)} \leqslant \sum_{l=0}^{n_1} \lambda_l (n_1 - d_1^*) \leqslant n_1 - d_1^*,$$

что противоречит предположению теоремы. Следовательно, теорема доказана. □

Изображенный на рис. 10.2 декодер основан на двух предыдущих теоремах. Если при декодировании i-й строки некоторые столбцы уже стерты, то используется та же самая процедура декодирования, но без учета стертых столбцов и при уменьшении d_1^* на число стертых столбцов. Далее, если при декодировании i-й строки некоторые столбцы были стерты или отмечены как содержащие ошибку, то при декодировании всех последующих строк они остаются стертыми. В случае когда необходимо стереть только один столбец, декодер всегда стирает два, так что число стертых столбцов всегда четно. Это не уменьшает корректирующих способностей процедуры, но сводит число необходимых итераций не более чем к $n_2 + (d_1^* - 1)/2$.

10.4. МНОГОМЕРНЫЕ СПЕКТРЫ

Двумерные таблицы, размеры которых согласуются с длиной преобразования, можно исследовать с помощью двумерного преобразования Фурье. Пусть $\{v_{ii'}\}$ — двумерная $(n \times n)$-таблица над $GF(q)$, которую назовем *двумерным сигналом*, пусть для некоторого m длина n делит $q^m - 1$, и пусть α — элемент порядка n из поля $GF(q^m)$. Таблицу с элементами

$$V_{jj'} = \sum_{i=0}^{n-1} \sum_{i'=0}^{n-1} \alpha^{ij} \alpha^{i'j'} v_{ii'}$$

назовем *двумерным спектром*, а индексы j и j' — частотными переменными. Используя одномерное обратное преобразование, очевидно, получаем

$$v_{i\,i'} = (1/n)\,(1/n) \sum_{j=0}^{n-1} \sum_{j'=0}^{n-1} \alpha^{-ij} \alpha^{-i'j'} V_{j\,j'}.$$

На рис. 10.3 приведены два двумерных спектра над GF (8). В каждом квадрате сетки записан восьмеричный символ.

Чтобы задать контролирующий ошибки код, выберем множество из $N - K$ компонент в качестве (двумерных) проверочных частот и положим их равными нулю, как на рис. 10.3. Остальные K компонент будут информационными символами, принимающими значения в поле GF (q^m), а в качестве кодового слова, соответствующего данным информационным символам, выберем результат обратного двумерного преобразования Фурье. Построенный так код, очевидно, является линейным, но, вообще говоря, не циклическим.

Если искомый код лежит в подполе поля GF (q^m) (в рассматриваемом примере единственным подполем поля GF (8) является поле GF (2)), то необходимо взять ограничение кода, выбирая только те кодовые слова, все компоненты которых принадлежат этому подполю, и тем самым получая двумерный подкод над подполем. Можно также построить двумерные альтернантные коды, если перед ограничением на подполе умножить кодовое слово на двумерный шаблон.

Двумерные спектры не обязательно должны быть квадратными, но если это так и если $n + 1 = q^m$, то наибольшим рассматриваемым полем является GF $(n + 1)$. В случае когда спектр задается $(n_1 \times n_2)$-таблицей $(n_1 \neq n_2)$, приходится работать в наименьшем поле GF (q^m) при некотором m, таком, что и n_1 и n_2 делят $q^m - 1$. Пусть $\{v_{i\,i'}\}$ — двумерный сигнал размера $n_1 \times n_2$, и

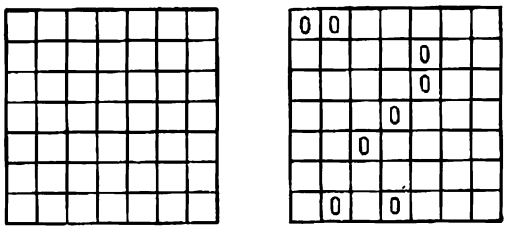


Рис. 10.3. Двумерный спектр над GF (8). *а* — спектр без ограничений; *б* — спектр с ограничениями.

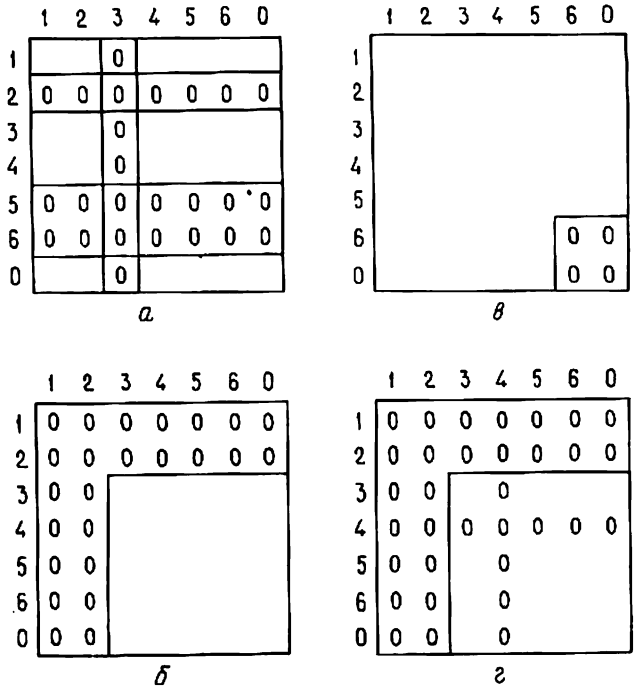


Рис. 10.4. Спектры некоторых кодов над GF (8). *а* — произведение циклических кодов; *б* — произведение кодов Рида—Соломона; *в* — дуальный код-произведение; *г* — произведение (7, 4)-кодов БЧХ.

пусть β и γ — элементы поля $GF(q^m)$ порядков n_1 и n_2 соответственно; тогда

$$V_{jj'} = \sum_{i=0}^{n_1-1} \sum_{i'=0}^{n_2-1} \beta^{ij} \gamma^{i'j'} v_{ii'}.$$

Используя одномерное обратное преобразование, очевидно, снова получаем

$$v_{ii'} = (1/n_1)\,(1/n_2) \sum_{j=0}^{n_1-1} \sum_{j'-0}^{n_2-1} \beta^{-ij} \gamma^{-i'j'} V_{jj'}.$$

Для примера выберем в качестве проверочных частот двумерного кода все элементы некоторых вертикальных и горизонтальных полос, как показано на рис. 10.4, *а*. Кодовыми словами являются все временные функции, в спектре которых в этих частотах стоят нули, т. е. для всех проверочных частот $(j,\ j')$

$$\sum_{i=0}^{n_1-1} \sum_{i'=0}^{n_2-1} \beta^{ij} \gamma^{i'j'} c_{ii'} = 0.$$

Это позволяет по-иному интерпретировать определение двумерного кода, задавая его как множество многочленов от двух переменных

$$c(x, y) = \sum_{i=0}^{n_1-1} \sum_{i'=0}^{n_2-1} c_{ii'} x^i y^{i'},$$

удовлетворяющих равенствам

$$c(\beta^j, \gamma^{i'}) = 0$$

для всех проверочных частот j и j'. Так как проверочные частоты были выбраны только по вертикальным и горизонтальным полосам, то

$$c(\beta^j, y) = 0 \quad \text{и} \quad c(x, \gamma^{i'}) = 0$$

для каждой проверочной частоты j и j'. Это показывает, что $\{(c_{ii'})\}$ является кодом-произведением.

Если полосы проверочных частот идут подряд, то получится произведение двух кодов Рида—Соломона. На рис. 10.4, *б* приведено спектральное задание (49, 25, 9)-кода над *GF* (8). Каждый из 25 информационных символов принимает одно из восьми возможных значений. Для формирования кодового слова надо перевести результирующую таблицу во временну́ю область при помощи преобразования Фурье.

Аналогичную процедуру можно использовать и для построения кода над *GF* (2), выбирая только двоичные кодовые слова. Для этого надо в частотной области определить спектр так, чтобы он задавал только двоичное кодовое слово. Двумерные ограничения сопряженности задаются равенствами

$$C_{jj'}^2 = C_{(2j \bmod n)\,(2j' \bmod n)}.$$

Пример такого построения показан на рис. 10.5. Каждое подмножество в этой таблице является подмножеством сопряженных частот. Любой из членов подмножества можно выбрать произвольно, а остальные символы в частотах подмножества определятся этим однозначно. Символ C_{00} может принимать только значения 0 или 1, так как он является квадратом самого себя. Остальные информационные символы являются восьмеричными. Весь спектр целиком определяется 49 битами, но, конечно, некоторые из них являются проверочными и не несут никакой информации.

Ограничение кода на рис. 10.4, *б* с помощью этой таблицы на двоичное подполе показано на рис. 10.4, *г*. В таблице на этом рисунке имеется только 16 незаполненных частот, которые с учетом ограничений сопряженности можно загрузить 16 битами информации. Это является следствием того, что все проверочные символы, частоты которых расположены в первом столбце и первой

	Битовое содержание		Битовое содержание
$\{C_{11}\ C_{22}\ C_{44}\}$	3	$\{C_{33}\ C_{66}\ C_{55}\}$	3
$\{C_{12}\ C_{24}\ C_{41}\}$	3	$\{C_{35}\ C_{63}\ C_{56}\}$	3
$\{C_{21}\ C_{42}\ C_{14}\}$	3	$\{C_{53}\ C_{36}\ C_{65}\}$	3
$\{C_{13}\ C_{26}\ C_{45}\}$	3	$\{C_{01}\ C_{02}\ C_{04}\}$	3
$\{C_{31}\ C_{62}\ C_{54}\}$	3	$\{C_{10}\ C_{20}\ C_{40}\}$	3
$\{C_{15}\ C_{23}\ C_{46}\}$	3	$\{C_{03}\ C_{06}\ C_{05}\}$	3
$\{C_{51}\ C_{32}\ C_{64}\}$	3	$\{C_{30}\ C_{60}\ C_{50}\}$	3
$\{C_{16}\ C_{25}\ C_{43}\}$	3	$\{C_{00}\}$	1
$\{C_{61}\ C_{52}\ C_{34}\}$	3		

Рис. 10.5. Множества сопряженных элементов для двумерного случая.

строке, попадают в различные подмножества сопряженных элементов. Построенный (49, 16, 9)-код мало привлекателен. Как следует из самого построения, в силу ограничений сопряженности коды-произведения часто обладают плохими характеристиками. Для построения хороших кодов-произведений надо выбирать взаимно простые длины. Но тогда основное поле становится большим, и обычно дело завершается выбором циклического кода, корни порождающего многочлена которого лежат в этом большом поле.

Второй случай, иллюстрируемый 10.4, *в*, называется *дуальным кодом-произведением*. Это код, дуальный к коду-произведению. Дуальный код-произведение мало пригоден для исправления независимых ошибок, так как его минимальное расстояние невелико. Как будет показано в § 10.7, он хорош для исправления кратных низкоплотностных пакетов ошибок. Код задается выбором прямоугольника самых старших проверочных частот высотой a и шириной b. Легко видеть, что минимальное расстояние этого кода удовлетворяет неравенству

$$d \geqslant 1 + \min(a, b).$$

В качестве примера укажем (49, 45, 3)-код над GF (8). Ограничением этого кода на двоичное подполе является (49, 39, $d \geqslant 3$)-код.

10.5. БЫСТРЫЕ КОДЫ БЧХ

Как мы видели в гл. 9, кодер и декодер для кодов БЧХ обычно содержат два преобразования Фурье. В следующей главе будет показано, что в случае составного n использование алгоритмов быстрого преобразования Фурье (БПФ) позволяет существенно уменьшить объем вычислений. Но БПФ-алгоритм Кули—Тьюки

требует введения некоторых регулирующих вычисление членов, а БПФ-алгоритм Гуда—Томаса — некоторой перестановки адресов. Во избежание этого мы сейчас несколько изменим определение кода БЧХ (что приведет к эквивалентному коду) и назовем полученный вариант кода БЧХ *быстрым кодом БЧХ*, так как он уменьшает работу декодера. Он приспособлен к алгоритму Гуда—Томаса (который будет рассматриваться в § 11.2), причем во временной области сохраняется порядок символов, полученный после перестановки.

Пусть $n = n_1 n_2 = q^m - 1$, где n_1 и n_2 взаимно просты, и пусть код состоит из всех двумерных $GF(q)$-значных временных функций $\{c_{ii'},\ i = 0, \ldots, n_1 - 1,\ i' = 0, \ldots, n_2 - 1\}$, таких, что их двумерные преобразования удовлетворяют проверочным соотношениям

$$C_{11} = C_{22} = C_{33} = \ldots = C_{2t,\,2t} = 0,$$

где индексы вычисляются по модулям n_1 и n_2 соответственно. Это определение задает линейный исправляющий t ошибок код, тривиальным образом отличающийся от кода БЧХ. Скорость и минимальное расстояние остаются без изменений. Неизменяемость скорости вытекает из следующей теоремы.

Теорема 10.5.1. *В двумерном случае класс сопряженных элементов для* $j \pmod{n_1}$ *и* $j \pmod{n_2}$ *содержит такое же число элементов, как и класс* $j \pmod{n_1 n_2}$.

Доказательство. Пусть r — наименьшее целое положительное число, такое, что $q^r j = j \pmod{n_1}$ и $q^r j = j \pmod{n_2}$. Пусть s представляет собой наименьшее целое положительное число, такое, что $q^s j = j \pmod{n_1 n_2}$. Тогда

$$q^r j = a n_1 n_2 + j \quad \text{и} \quad q^s j = b n_1 n_2 + j$$

для некоторых a и b. Очевидно, что такие наименьшие r и s равны. □

Доказательство того, что минимальное расстояние не меньше $2t + 1$, дается следующей процедурой декодирования t ошибок. Для принятого слова с двумерным преобразованием $V_{jj'}$ определим компоненты синдрома равенствами

$$S_j = V_{(j \bmod n_1,\ j \bmod n_2)}, \quad j = 1, \ldots, 2t.$$

Если одиночная ошибка величины $e_{i_1 i_1'}$ располагается в строке с номером i_1 и столбце с номером i_1', то

$$S_j = e_{i_1 i_1'} (\beta^{i_1} \gamma^{i_1'})^j = Y_1 X_1^j,$$

где X_1 равно некоторой степени примитивного элемента α, которая однозначно определяется для каждой строки и каждого столбца. При ν ошибках компоненты синдрома имеют вид

$$S_j = \sum_{k=1}^{\nu} Y_k X_k^j$$

и задают единственное решение, если произошло не более t ошибок. Воспользуемся алгоритмом Берлекэмпа—Месси и его рекуррентным продолжением для вычисления всех S_j, $j = 1, \ldots, n$, и положим

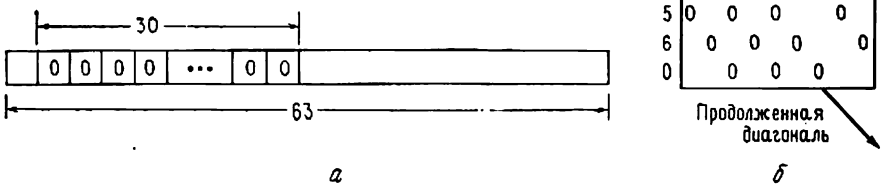
$$E_{(j \bmod n_1,\ j \bmod n_2)} = S_j, \quad j = 1, \ldots, n.$$

Так как n_1 и n_2 взаимно просты, положение каждой компоненты синдрома в двумерной таблице определяется однозначно. Наконец, полагая

$$C_{jj'} = V_{jj'} - E_{jj'}$$

и выполняя двумерное обратное преобразование Фурье, завершаем декодирование.

В графической интерпретации вся эта процедура выполняется вдоль продолженной диагонали $(n_1 \times n_2)$-таблицы, показанной на рис. 10.6. На этой продолженной диагонали определяются $2t$ проверочных частот. Алгоритм Берлекэмпа—Месси работает вдоль продолженной диагонали, и так как n_1 и n_2 взаимно просты то рекуррентное продолжение восстанавливает все элементы таблицы, двумерное преобразование Фурье которой дает таблицу ошибок.



а *б*

Рис. 10.6. Спектр быстрого кода БЧХ. *а* — спектр (63, 33, 31)-кода Рида—Соломона; *б* — двумерный спектр быстрого (63, 33, 31)-кода Рида—Соломона.

10.6. ДЕКОДИРОВАНИЕ МНОГОМЕРНЫХ КОДОВ

Двумерное кодовое слово задается двумерным спектром, у которого некоторые спектральные компоненты равны нулю. Один такой пример был рассмотрен в предыдущем параграфе. Предположим, что код определен так, что для некоторых пар индексов (j_k, j'_k), помеченных индексом k, выполняются условия

$$\sum_{i=0}^{n_1-1} \sum_{i'=0}^{n_2-1} c_{ii'} \beta^{ij_k} \gamma^{i'j'_k} = 0.$$

Тогда соответствующая компонента двумерного синдрома для конфигурации ошибок $\{e_{ii'}\}$ веса t определяется равенством

$$S_{j_k j'_k} = \sum_{i=0}^{n_1-1} \sum_{i'=0}^{n_2-1} e_{ii'} \beta^{ij_k} \gamma^{i'j'_k}$$

и может быть вычислена по принятому слову $\{v_{ii'}\}$.

Опишем теперь конфигурацию ошибок, используя строчные локаторы, столбцовые локаторы и величины ошибок. Локаторы в строке i_l и столбце i'_l определяются соответственно равенствами $X_l = \beta^{i_l}$ и $Y_l = \gamma^{i'_l}$. Величину ошибки обозначим Z_l. Компоненты синдрома при этом принимают вид

$$S_{j_k j'_k} = \sum_{l=1}^{t} Z_l X_l^{j_k} Y_l^{j'_k}.$$

Двумерные коды естественно определять так, чтобы система выписанных выше уравнений была разрешима относительно X_l, Y_l и Z_l при $l = 1, \ldots t$. Например, проверочные частоты можно выбрать так, чтобы получилась следующая система уравнений:

$$S_{11} = Z_1 X_1 Y_1 + Z_2 X_2 Y_2 + \ldots + Z_\nu X_\nu Y_\nu = E_{11},$$

$$S_{12} = Z_1 X_1 Y_1^2 + Z_2 X_2 Y_2^2 + \ldots + Z_\nu X_\nu Y_\nu^2 = E_{12},$$

$$\vdots$$

$$S_{1,\,2t} = Z_1 X_1 Y_1^{2t} + Z_2 X_2 Y_2^{2t} + \ldots + Z_\nu X_\nu Y_\nu^{2t} = E_{1,\,2t},$$

которая уже встречалась в описании алгоритма декодирования кода Рида—Соломона. Имеется только одно отличие: Y_l не обязательно различны, так как в одном и том же столбце может произойти несколько ошибок. Можно, однако, сгруппировать ошибки с одинаковыми Y_l, и это приведет к аналогичной системе с меньшим ν, которое также удовлетворяет условию $\nu \leqslant t$. Тогда опять можно воспользоваться алгоритмом Берлекэмпа—Месси и рекуррентным продолжением для вычисления E_{11}, E_{12}, ..., E_{1n}, хотя

такая процедура не дает однозначного решения для пар (X_l, Y_l) при $l = 1, 2, \ldots, \nu$, так как некоторые ошибки попадают в один и тот же столбец. В общем случае для $n_1 = n_2 = n$ справедлива следующая теорема, которую можно обобщить, заменив n на НОК (n_1, n_2).

Теорема 10.6.1. *Предположим, что произошло $\nu \leqslant t$ ошибок и что для любых целых j_0, j_0' и a, a' известны компоненты синдрома $S_{j_0+ak,\ j_0'+a'k}$ при $k = 1, \ldots, 2t$. Тогда компоненты синдрома $S_{j_0+ak,\ j_0'+a'k}$ при $k = 1, \ldots, n$ определяются однозначно.*

Доказательство. Нам даны $2t$ уравнений для $k = 1, \ldots, 2t$:

$$S_{j_0+ak,\ j_0'+a'k} = \sum_{l=1}^{\nu} Z_l X_l^{j_0+ak} Y_l^{j_0'+a'k} =$$

$$= \sum_{l=1}^{\nu} Z_l \left(X_l^{j_0} Y_l^{j_0'}\right)\left(X_l^{a} Y_l^{a'}\right)^k.$$

Обозначим через ν' число различных величин $X_l^a X_l^{a'}$ при всех l, через $\overline{Y}_{l'}$, $l' = 1, \ldots, \nu'$, различные значения в этом наборе, а через $\overline{X}_{l'}$ сумму всех множителей при фиксированном $\overline{Y}_{l'}$ в данном уравнении (она имеет одно и то же значение для всех k). Тогда

$$S_{j_0+ak,\ j_0'+a'k} = \sum_{l'=1}^{\nu'} \overline{X}_{l'} \overline{Y}_{l'}^k, \quad k = 1, \ldots, 2t,$$

где теперь все $Y_{l'}$ различны и $\nu' \leqslant t$. Алгоритм Берлекэмпа—Месси с последующим рекуррентным продолжением позволяет вычислить остальные компоненты синдрома

$$S_{j_0+ak,\ j_0'+a'k} = \sum_{l'=1}^{\nu'} \overline{X}_{l'} \overline{Y}_{l'}^k, \quad k = 1, \ldots, n. \qquad \square$$

Согласно доказанной теореме, любые $2t$ компонент синдрома, расположенных в таблице на произвольной прямой (горизонтальной, вертикальной или идущей под любым углом) можно продолжать до всех компонент синдрома на этой прямой. Далее, в силу ограничений сопряженности каждая из этих новых компонент синдрома определяет все сопряженные с ней компоненты. Конечно, не обязательно, чтобы вычисленные так компоненты синдрома давали остальные его компоненты. Размеры таблицы не всегда задаются взаимно простыми числами, так что продолжение прямой не обязательно включает все компоненты.

Посмотрим теперь, как граница БЧХ обобщается на случай двух (или более) размерностей. Предположим, что где-то в первой строке имеются $2t$ подряд идущих компонент синдрома. Тогда их продолжение дает все компоненты синдрома в первой строке.

Аналогично расположенные где-то во второй строке подряд идущие $2t$ компонент синдрома могут быть использованы для вычисления всех компонент синдрома во второй строке. Далее, любые расположенные подряд $2t$ компонент синдрома в первых $2t$ строках могут быть продолжены до всех компонент синдрома в каждой из этих $2t$ строк и, следовательно, $2t$ непрерывно расположенных компонент синдрома по столбцам. Последние в свою очередь могут быть продолжены до всех компонент синдрома в своем столбце, а этого уже достаточно для однозначного вычисления конфигурации ошибок. Простейший пример дается $(2t \times 2t)$-таблицей известных компонент синдрома. Общая ситуация описывается следующим образом.

Теорема 10.6.2 (двумерная граница БЧХ). *Если a взаимно просто с n и число ошибок ν не превосходит t, то множество компонент синдрома*

$$S_{a(m_k+k'),\, a'(m_k+k')+k},$$

где a, a' и m_k — некоторые целые числа, $k' = 1, \ldots, 2t$, $k = 1, \ldots, 2t$, однозначно определяет конфигурацию ошибок.

Доказательство. Для каждого k воспользуемся предыдущей теоремой для вычисления компонент синдрома

$$S_{a(m_k+k'),\, a'(m_k+k')+k}, \quad k' = 1, \ldots, n, \quad k = 1, \ldots, 2t.$$

Переопределим индекс k' так, чтобы отбросить в индексах числа m_k и записать известные компоненты синдрома в виде

$$S_{ak',\, a'k'+k}, \quad k' = 1, \ldots, n, \quad k = 1, \ldots, 2t.$$

Вновь воспользуемся предыдущей теоремой для вычисления для каждого k' компонент синдрома

$$S_{ak',\, a'k'+k}, \quad k' = 1, \ldots, n, \quad k = 1, \ldots, n.$$

Поскольку k пробегает все значения от 1 до n, его можно переопределить так, чтобы избавиться от $a'k'$; таким образом, вычисляются все компоненты синдрома $S_{ak',\, k}$, при $k' = 1, \ldots, n$ и $k = 1, \ldots, n$. Поскольку a и n взаимно просты, то индекс ak' пробегает все n значений. Следовательно, синдром вычислен полностью и конфигурация ошибок определена. □

10.7. ДЛИННЫЕ КОДЫ НАД МАЛЫМИ ПОЛЯМИ

Для кодов с очень большой длиной и высокой скоростью сложность декодера может являться более важным фактором, чем скорость кода. С практической точки зрения разница между скоростями 0,98 и 0,99 может быть не столь уж существенной,

если декодеры для этих двух кодов сильно отличаются в цене. В настоящем параграфе рассматриваются коды очень большого объема, декодеры для которых имеют ограниченную сложность. Все рассматриваемые коды являются многомерными; характеристики этих кодов в общем случае лучше, чем у кодов-произведений, а декодирование реализуется с помощью вычислений в поле Галуа, мощность которого намного меньше длины кода. Такие коды могут быть противопоставлены кодам БЧХ.

Для кода БЧХ длины $n = q^m - 1$ декодер производит вычисления в поле $GF(q^m)$. При больших n мощность такого поля Галуа очень велика. Коды, которые будут сейчас описаны, можно декодировать с помощью вычислений в малом поле Галуа. Одним из этих примеров будет двоичный код, слова которого задаются двумерной таблицей с $n = 2^m - 1$ строк и таким же числом столбцов. Следовательно, длина кода равна n^2, но мы надеемся все вычисления провести в поле $GF(2^m)$. Хотя скорость такого кода ниже скорости кода БЧХ с теми же n и d, в некоторых приложениях он может оказаться предпочтительнее из-за малой длины, на которой производит вычисления декодер.

Первым примером является произведение двух кодов БЧХ, а именно (n_1, k_1, d_1)-кода и (n_2, k_2, d_2)-кода, и, следовательно, является кодом с параметрами (n_1n_2, k_1k_2, d_1d_2). Например, произведение расширенного (256, 232, 25)-кода Рида—Соломона на самого себя дает (65 536, 53 824, 625)-код над GF (256), который позволяет исправлять 312 8-битовых байтов, пакеты длины 3072 8-битовых байтов, а также многие конфигурации ошибок более чем из 312 байтов. Вместо этого можно воспользоваться (65 536, 64 289, 625)-кодом БЧХ, который позволяет также исправлять 312 случайных байтов и обеспечивает при этом существенно большую скорость, но его декодирование требует вычислений в 16-битовой арифметике поля $GF(2^{16})$. Код-произведение компенсирует это другим преимуществом, позволяя помимо 312 случайных байтов исправлять многие другие конфигурации ошибок, чего не может код БЧХ. Декодировать код-произведение можно описанным в § 10.3 способом.

Второй пример дается кодом, дуальным к коду-произведению двух кодов Рида Соломона или их подкодов над подполями. Дуальный код исправляет случайные ошибки, а также конфигурации многократных низкоплотностных пакетов, определяемых как конфигурации, состоящие из сегментов ошибок, вес которых больше типичного. Исправляющие многократные низкоплотностные пакеты ошибок коды полезны для тех каналов, в которых происходит ритмичное изменение вероятности ошибки, например для каналов с замиранием.

В частности, дуальный к квадрату (255, 239, 25)-кода Рида—Соломона код представляет собой (65 025, 64 769, 17)-код

над $GF(256)$ и исправляет все случайные конфигурации ошибок из восьми 8-битовых байтов, а также все конфигурации ошибок из 64 8-битовых байтов, образующие низкоплотностные пакеты из восьми искаженных столбцов, каждый из которых содержит восемь искаженных байтов.

Двумерные проверочные частоты этого кода задаются парами (j, j') для $j = 1, \ldots, 16$ и $j' = 1, \ldots, 16$. Основанная на теореме 10.6.1 процедура декодирования сводится к следующему. По данному принятому слову $\{v_{ii'}\}$ вычисляются компоненты синдрома $S_{jj'}$ по формулам

$$S_{jj'} = \sum_{i=0}^{255} \sum_{i'=0}^{255} \alpha^{ij}\alpha^{i'j'}v_{ii'}, \quad j, j' = 1, \ldots, 16.$$

Алгоритм Берлекэмпа—Месси с последующим рекуррентным продолжением используется для вычисления всех компонент синдрома в j-й строке.

После такой обработки в декодере каждой из 16 строк оказываются вычисленными компоненты $S_{jj'}$ синдрома для $j = 1, \ldots, 255$ и $j' = 1, \ldots, 16$. Обратное преобразование Фурье каждой из этих 16 строк содержит ненулевые символы только в тех восьми (или менее) столбцах, которые содержат ошибки. Пометим эти восемь столбцов. Преобразование Фурье этих 16 строк в каждом из помеченных восьми столбцов дает 16 элементов $T_{j'}^{(i)}$ поля $i = 1, \ldots, 255$, $j' = 1, \ldots, 16$. Стоящие в i-м столбце элементы этого поля можно выразить через величины ошибок и строчные локаторы для этого столбца:

$$T_1^{(i)} = Z_1^{(i)}(X_1^{(i)})^1 + Z_2^{(i)}(X_2^{(i)})^1 + \ldots + Z_{\nu(i)}^{(i)}(X_{\nu(i)}^{(i)})^1,$$
$$T_2^{(i)} = Z_1^{(i)}(X_1^{(i)})^2 + Z_2^{(i)}(X_2^{(i)})^2 + \ldots + Z_{\nu(i)}^{(i)}(X_{\nu(i)}^{(i)})^2,$$
$$\vdots$$
$$T_{16}^{(i)} = Z_1^{(i)}(X_1^{(i)})^{16} + Z_2^{(i)}(X_2^{(i)})^{16} + \ldots + Z_{\nu(i)}^{(i)}(X_{\nu(i)}^{(i)})^{16},$$

где $\nu(i)$ — число ошибок в i-м столбце. Если $\nu(i) \leqslant 8$ для каждого i, то эта система уравнений может быть решена с помощью алгоритма Берлекэмпа—Месси, что и завершает декодирование.

Можно также построить код, дуальный к коду-произведению для кодов БЧХ. Но такие коды содержатся в более мощных кодах с тем же минимальным расстоянием и той же способностью исправлять низкоплотностные пакеты ошибок. В общем случае предпочтительнее эти последние коды; опишем их.

Для иллюстрации этой идеи рассмотрим частный случай двоичного кода с $t = 8$, $n = 255$ и длиной $n^2 = 65\,025$. Как показано на рис. 10.7, *а*, выберем проверочные частоты так, чтобы в блоке

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	....
1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0						
2																						
3	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0						
4																						
5	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0						
6																						
7	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0						
8																						
9	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0						
10																						
11	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0						
12																						
13	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0						
14																						
15	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0						
16																						
17																						
18																						
19																						

а

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	...
1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0						
2																						
3	0		0		0	0	0		0	0	0	0	0	0	0							
4																						
5	0		0		0		0		0	0	0		0	0	0							
6	0																					
7	0		0		0		0		0		0		0	0	0							
8																						
9	0		0		0		0		0		0		0		0							
10	0		0																			
11	0		0		0		0		0		0		0		0							
12	0																					
13	0		0		0		0		0		0		0		0							
14	0		0		0																	
15	0		0		0		0		0		0		0		0							
16																						
17																						
18																						
19																						

б

Рис. 10.7. Двумерные проверочные частоты. *а* — расположение проверочных символов по строкам; *б* — другой выбор проверочных символов.

с $j = 1, \ldots, 2t$ и $j' = 1, \ldots, 2t$ можно было бы вычислить все компоненты синдрома. Тогда, согласно теореме 10.4.2, можно будет вычислить и остальные компоненты синдрома.

Для построения кода, исправляющего восемь ошибок, надо соответствующим образом выбрать 128 проверочных частот. Сначала выберем в качестве проверочных частот пары (1, 1), (1, 2), ..., (1, $2t$), так что $C_{11} = C_{12} = \ldots = C_{1,2t} = 0$. Все выбранные так проверочные частоты лежат в разных классах сопряженных элементов, и каждый класс содержит восемь элементов, так что каждая из этих частот эквивалентна восьми проверочным битам. Если произошло не более $t = 8$ ошибок, то вычисленные в этих частотах компоненты синдрома могут быть продолжены до компонент $S_{1j'}$, $j' = 1, \ldots, n$. Далее, используя ограничения сопряженности для этой верхней строки, заполняем строки с номерами 2, 4, 8 и 16. Выберем теперь в качестве проверочных частот пары (3, 1), (3, 2), ..., (3, $2t$). Это дает еще 16×8 проверочных битов. Вычисление компонент синдрома в третьей строке дает также значения компонент синдрома в строках 6 и 12.

Продолжая таким образом, выберем все приведенные на рис. 10.7 проверочные частоты. Это дает возможность вычислить все компоненты синдрома для частот, заполняющих угловой квадрат $2t \times 2t$, а тем самым и все компоненты синдрома (напомним, что произошло не более t ошибок). Каждый проверочный символ эквивалентен восьми проверочным битам, так что всего получаем 1024 проверочных битов. Таким образом построен (65 025, 64 001, 17)-код, характеристики которого лучше, чем у (65 535, 65 407, 17)-кода БЧХ. Достоинство этого кода состоит в том, что, несмотря на его большую длину, для него существует сравнительно простой алгоритм декодирования, позволяющий исправлять большое число пакетов ошибок (такой алгоритм будет сейчас описан).

Декодер для этого кода аналогичен описанному ранее. По принятому двумерному слову вычисляется двумерное преобразование Фурье в 128 проверочных частотах. Это дает спектральные компоненты $E_{jj'}$ вектора ошибок в проверочных частотах. Покажем, как по этим известным компонентам вычислить остальные компоненты спектра ошибок. Результат применения алгоритма Берлекэмпа—Месси к первой строке рекуррентно продолжим на все 255 компонент этой строки и, используя ограничения сопряженности, заполним строки с номерами 2, 4, 8 и 16. То же самое проделаем с третьей строкой, заполняя строки с номерами 3, 6 и 12. Повторим вычисления для пятой строки и т. д. до тех пор, пока не будут заполнены $2t$ строк.

После заполнения $2t$ строк начинается декодирование столбцов. Один из способов состоит в использовании алгоритма Бер-

лекэмпа—Месси для заполнения всех входов таблицы $E_{jj'}$. Обратное двумерное преобразование Фурье завершает декодирование. Более простой процедурой является следующая. Вычислим одномерное обратное преобразование Фурье для $2t$ известных строк символов $E_{jj'}$. Это дает множество из $2t$ компонент синдрома для каждого столбца; ненулевые компоненты могут иметь не более $2t$ столбцов и только к ним необходимо применить алгоритм Берлекэмпа—Месси.

На рис. 10.7, *б* показан другой возможный способ выбора проверочных частот, который приводит к двумерному коду с большей скоростью, но с более сложным декодером. В этом случае алгоритм Берлекэмпа—Месси должен применяться для декодирования попеременно строк и столбцов. При необходимости для вычисления новых компонент $E_{jj'}$ используются ограничения сопряженности.

ЗАДАЧИ

10.1. Множество всех (7×7)-матриц над $GF(2)$ образует векторное пространство. Выписать базис этого пространства. Сколько векторов содержит это пространство? Является ли подпространством множество всех таких матриц с нулевой главной диагональю? Если да, то сколько векторов содержит это подпространство?

10.2. Пусть $p(x) = x^2 + x + 1$ и $q(x) = x^4 + x + 1$ — многочлены над $GF(2)$. Найти «китайские многочлены» $a(x)$ и $b(x)$, удовлетворяющие равенству $a(x)\,p(x) + b(x)\,q(x) = 1$.

10.3. Пусть $\mathscr{C}$ — простой $(n, n-1)$-код с проверкой на четность, и пусть $\mathscr{C}^3$ есть $(n^3, (n-1)^3)$-код, представляющий собой трехмерное произведение трех одномерных копий $\mathscr{C}$.

а. Сколько ошибок может исправить код $\mathscr{C}^3$?

б. Привести пример двух конфигураций ошибок одинакового веса, обнаруживаемых кодом, но имеющих один и тот же синдром и, следовательно, неисправляемых.

10.4.а. С помощью перемежения расширенного кода Рида—Соломона над $GF(64)$ построить (3840, 3360)-код, исправляющий пакеты из 240 6-битовых байтов. Чему равно минимальное расстояние кода-перемежения?

б. Перевести этот код в (4096, 3360)-код-произведение, который также исправляет пакеты из 240 байтов, но, кроме того, исправляет и случайные конфигурации ошибок из 22 байтов.

в. Описать декодеры для кодов из предыдущих пунктов задачи.

10.5. Сколько ошибок может исправлять произведение (7, 4)-кода Хэмминга на самого себя? Используя в качестве готового блока декодер для (7, 4)-кода Хэмминга, построить простой декодер, исправляющий ошибки в пределах радиуса упаковки кода-произведения.

10.6. Пусть a и b удовлетворяют равенству $an_1 + bn_2 = 1$ для взаимно простых n_1 и n_2. При заданном циклическом (n_1, k_1)-коде с порождающим многочленом $g_1(x)$ и заданном циклическом (n_2, k_2)-коде с порождающим многочленом $g_2(x)$ доказать, что порождающий многочлен циклического кода-произведения длины $n = n_1 n_2$ равен

$$g(x) = \text{НОД}\,[g_1(x^{bn_2}),\ g_2(x^{an_1}),\ x^n - 1].$$

ЗАМЕЧАНИЯ

Произведение кодов ввел Элайс [1954], доказавший, что минимальное расстояние кода-произведения равно произведению минимальных расстояний исходных кодов. Бартон и Уэлдон [1965] доказали, что при взаимно простых длинах произведение двух циклических кодов является циклическим кодом. В этой же работе изучалось исправление пакетов ошибок и независимых ошибок кодами-произведениями. Методы декодирования кода-произведения описали Редди и Робинсон [1972], а также Уэлдон [1971]. В § 10.4—10.6 развиваются идеи автора (Блейхут [1979]). Использовать дуальный код-произведение для исправления кратных низкоплотностных пакетов предложили Чень и Нг [1973].

ГЛАВА 11

БЫСТРЫЕ АЛГОРИТМЫ

Хорошие коды с большой длиной и большим объемом алфавита, такие, как коды Рида—Соломона, находят все более широкое применение. В будущем потребуются еще более длинные коды, контролирующие ошибки, и поэтому задача уменьшения сложности алгоритмов декодирования становится важной. Мы уже рассмотрели некоторые эффективные алгоритмы декодирования кодов Рида—Соломона и связанных с ними кодов и знаем, как строить декодеры, содержащие порядка n^2 вычислений.

В данной главе проводится исследование сложности декодеров. Мы начнем с уже рассмотренных декодеров и опишем для них способы более эффективного выполнения вычислений. В качестве меры сложности вычислений выбирается число умножений (а иногда и число сложений) в проводимых вычислениях. Будут строиться методы, уменьшающие сложность вычислений именно в этом отношении. За это приходится платить усложнением структуры: для уменьшения числа сложений и умножений приходится увеличивать структурную сложность, привлекая алгоритмы с более сложными индексацией и разветвлением. Безусловно, во многих приложениях регулярность алгоритма оказывается важнее, чем число умножений и сложений. Такой аспект сложности трудно определить численно, и поэтому мы его не рассматриваем.

11.1. ЛИНЕЙНАЯ СВЕРТКА И ЦИКЛИЧЕСКАЯ СВЕРТКА

Линейная свертка

$$c_i = \sum_{k=0}^{n-1} g_{i-k} d_k$$

может быть также записана в виде произведения многочленов

$$c(x) = g(x)\, d(x).$$

Стандартный способ вычисления произведения двух многочленов требует примерно $(\deg d(x)) \times (\deg g(x))$ умножений и сло-

жений, но, возможно, существуют другие способы его вычисления, для которых это число можно уменьшить. Так как свертка является элементом многих вычислительных операций, то имеет смысл построить эффективные способы ее вычисления. Более того, циклическую свертку

$$c(x) = g(x)\, d(x) \pmod{x^n - 1}$$

можно сначала вычислить как линейную, а затем привести по модулю $x^n - 1$. Следовательно, эффективные способы вычисления линейной свертки дают также эффективные способы вычисления циклической свертки, и, обратно, эффективные способы вычисления циклической свертки можно легко превратить в эффективные способы вычисления линейной свертки.

Циклическая свертка может быть записана в виде

$$c_i = \sum_{k=0}^{n-1} g_{((i-k))} d_k, \quad i = 0, \ldots, n-1,$$

где двойные скобки обозначают вычисление индексов по модулю n. Согласно теореме о свертке, в частотной области имеем

$$C_j = G_j D_j,$$

и, таким образом, циклическую свертку можно вычислить, последовательно применяя преобразование Фурье, покомпонентное умножение и обратное преобразование Фурье. Если n представляется произведением большого числа сомножителей, то быстрое преобразование Фурье (БПФ), которое будет описано в следующем параграфе, позволяет снизить число вычислений. Преобразование Фурье в поле $GF(q^m)$ существует только для целых n, делящих число $q^m - 1$, и некоторые из этих n не допускают алгоритмов БПФ. Но в теории обработки дискретных сигналов хорошо известны методы удлинения векторов $\mathbf{d}$, $\mathbf{g}$ и $\mathbf{c}$ до такой длины n', что компоненты $\mathbf{c}$ с индексами $i = 0, \ldots, n-1$, сохраняют нужные значения, а вектор $\mathbf{c}$ связан с векторами $\mathbf{d}$ и $\mathbf{g}$ соотношением циклической свертки, вычисляемой, однако, на длине n', удобной для применения алгоритмов БПФ.

Возьмем, например, удовлетворяющую условию $n' \geqslant 2n$ длину n', на которой легко вычислить преобразование Фурье, и положим

$$a'_i = \begin{cases} a_i, & i = 0, \ldots, n-1, \\ 0, & i = n, \ldots, n'-1, \end{cases}$$

$$b'_i = \begin{cases} b_i, & i = 0, \ldots n-1, \\ 0, & i = n, \ldots n'-n-1, \\ b_{i+n-n'}, & i = n'-n, \ldots, n'-1, \end{cases}$$

$$c'_i = \sum_{k=0}^{n'-1} a'_k b'_{((i-k))},$$

где двойные скобки в индексе обозначают вычисление по модулю n'. Тогда $c'_i = c_i$, $i = 0, \ldots, n-1$. Остальные значения c'_i не представляют интереса и могут быть отброшены.

Разработаны методы сведения длинной линейной свертки к множеству коротких циклических сверток. Опишем метод, оказавшийся чрезвычайно полезным в обработке дискретных сигналов и известный под названием *метода перекрытия с накоплением*. Он применим также к длинным сверткам в полях Галуа.

Предположим, что имеется устройство, вычисляющее циклическую свертку длины n, и что мы хотим воспользоваться этим устройством для умножения многочлена $a(x)$, степень A которого меньше n, на многочлен $b(x)$, степень B которого больше n, чтобы получить произведение $c(x) = a(x)\,b(x)$. По имеющемуся многочлену $b(x)$ сформируем множество многочленов $\{b^{(1)}(x), b^{(2)}(x), \ldots, b^{(s)}(x)\}$, степень каждого из которых не превосходит $n-1$, по следующему правилу:

$$b_i^{(1)} = \begin{cases} b_i, & i = 0, \ldots, n-1, \\ 0, & \text{в остальных случаях,} \end{cases}$$

$$b_i^{(2)} = \begin{cases} b_{i+(n-A)}, & i = 0, \ldots, n-1, \\ 0, & \text{в остальных случаях,} \end{cases}$$

$$b_i^{(3)} = \begin{cases} b_{i+2(n-A)} & i = 0, \ldots, n-1, \\ 0, & \text{в остальных случаях,} \end{cases}$$

$$\vdots$$

$$b_i^{(s)} = \begin{cases} b_{i+(s-1)(n-A)}, & i = 0, \ldots, n-1, \\ 0, & \text{в остальных случаях,} \end{cases}$$

где s выбрано достаточно большим для того, чтобы были представлены все коэффициенты многочлена $b(x)$. Отметим, что коэффициенты построенных многочленов перекрываются. Пусть

$$c^{(l)}(x) = a(x)\,b^{(l)}(x) \pmod{x^n - 1}, \quad l = 1, \ldots, s.$$

Коэффициенты многочлена $c(x)$, за исключением первых A коэффициентов, содержатся среди коэффициентов многочленов $c^{(l)}(x)$, а именно

$$c_i = c_i^{(1)}, \qquad i = A, \ldots, n-1,$$

$$c_{i+(n-A)} = c_i^{(2)}, \qquad i = A, \ldots, n-1,$$

$$c_{i+2(n-A)} = c_i^{(3)}, \qquad i = A, \ldots, n-1$$

и т. д. A младших коэффициентов многочлена $c(x)$ теряются. Во многих приложениях при вычислении линейной свертки нужны не все коэффициенты выходного многочлена, и тогда описанная

процедура может быть удовлетворительной. Если же нужны все коэффициенты выходного многочлена $c(x)$, то в проделанных выкладках следует просто заменить многочлен $b(x)$ многочленом $x^A b(x)$. Это позволит вычислить все необходимые коэффициенты, но все индексы будут сдвинуты на число A.

Каждая циклическая свертка, за исключением последней, дает $n - A$ коэффициентов линейной свертки и A бесполезных коэффициентов, которые отбрасываются. Так как степень многочлена $b^{(s)}(x)$ может быть меньше, чем $n - 1$, то последняя свертка может давать больше, чем $n - A$, значимых коэффициентов.

11.2. БЫСТРЫЕ АЛГОРИТМЫ СВЕРТКИ

Известно много алгоритмов вычисления циклической свертки, и многие из них более эффективны, чем использование быстрого преобразования Фурье и теоремы о свертке. В данном параграфе будет описан быстрый алгоритм Винограда вычисления свертки. Этот метод пригоден для любого поля, но мы интересуемся только конечными полями $GF(q)$. Он сводится к разбиению свертки на легко вычисляемые короткие свертки, которые формируются в соответствии с китайской теоремой об остатках для многочленов.

Алгоритм Винограда находит вычет

$$c(x) = g(x)\, d(x) \pmod{b(x)}$$

для некоторого фиксированного многочлена $b(x)$ над $GF(q)$. Для построения алгоритма вычисления линейной свертки

$$c(x) = g(x)\, d(x)$$

выберем любое целое число N, большее степени $c(x)$, и любой многочлен $b(x)$ степени N. Тогда имеем тривиальное утверждение

$$c(x) = c(x) \pmod{b(x)},$$

так что алгоритм Винограда применим и для вычисления линейных сверток.

Чтобы разбить вычисление свертки по модулю многочлена на отдельные части, разложим $b(x)$ на взаимно простые множители над некоторым подполем поля $GF(q)$:

$$b(x) = b_1(x)\, b_2(x) \ldots b_s(x).$$

Обычно в качестве поля разложения выбирается простое подполе $GF(p)$, и будем рассматривать этот случай; теория, однако, столь же пригодна и для любого подполя. Процедура минимизирует число умножений в $GF(q)$ и не делает попыток такой минимизации в $GF(p)$. В большинстве практических приложений простое поле является полем $GF(2)$, умножение в котором тривиально.

Алгоритм Винограда для свертки основан на вычетах

$$c_k(x) = R_{b_k(x)}[c(x)], \quad k = 1, \ldots, s.$$

Вычисление циклической свертки разбивается на два шага. Сначала вычисляются вычеты для $k = 1, \ldots, s$:

$$\begin{aligned} c_k(x) &= g(x)\, d(x) \pmod{b_k(x)} \\ &= g_k(x)\, d_k(x) \pmod{b_k(x)}. \end{aligned}$$

Вычисление вычетов $g_k(x)$ и $d_k(x)$ не требует умножений [1]).

Согласно китайской теореме для многочленов, выражение для многочлена $c(x)$ через эти вычеты дается формулой

$$c(x) = a_1(x)\, c_1(x) + \ldots + a_s(x)\, c_s(x) \pmod{b(x)},$$

причем все многочлены $a_1(x), \ldots, a_s(x)$ являются многочленами над $GF(p)$. Так как коэффициенты многочленов $a_k(x)$ принадлежат простому полю $GF(p)$, этот последний шаг также не требует умножений. Только для вычисления коротких сверток, задаваемых произведениями многочленов $g_k(x)\, d_k(x)$, требуется производить умножения в поле $GF(q)$; так как оба многочлена $g_k(x)$ и $d_k(x)$ имеют $\deg b_k(x)$ коэффициентов, то обычный метод умножения многочленов приводит к оценке необходимого числа умножений в виде суммы

$$\sum_{k=1}^{s} [\deg b_k(x)]^2.$$

Это может существенно уменьшить число умножений. Иногда удается дополнительно уменьшить число умножений, разбивая одну или несколько коротких сверток на еще более короткие и повторяя описанную процедуру.

Одна из модификаций алгоритма Винограда вычисления свертки состоит в выборе многочлена $b(x)$ степени несколько меньше нужной. Тогда описанная процедура приводит к неправильному значению свертки, но ее можно подправить несколькими дополнительными вычислениями. Это делается следующим образом. Согласно алгоритму деления,

$$c(x) = Q(x)\, b(x) + R_{b(x)}[c(x)].$$

Если $\deg b(x) > \deg c(x)$, то многочлен-частное $Q(x)$ тождественно равен нулю, и ситуация аналогична рассмотренной выше. Если $\deg b(x) \leqslant \deg c(x)$, то алгоритм Винограда приводит к вычислению только остатка $R_{b(x)}[c(x)]$. Член $Q(x)\, b(x)$

[1]) Это утверждение (и аналогичные ему утверждения ниже) основывается на том, что, как уже было указано, при оценке числа операций алгоритма Винограда не учитываются операции в базовом поле $GF(p)$; если $GF(p) = GF(2)$, то умножений по существу нет. — *Прим. перев.*

можно вычислить с помощью дополнительных операций. В простейшем случае $\deg b(x) = \deg c(x)$ и степень $Q(x)$ должна равняться нулю. Если $b(x)$ представляет собой приведенный многочлен степени N, то, очевидно, коэффициент при x^N в многочлене $c(x)$ равен $Q(x) = c_N$. Следовательно,

$$c(x) = c_N b(x) + R_{b(x)}[c(x)],$$

где c_N легко вычисляется как произведение старших коэффициентов многочленов $g(x)$ и $d(x)$.

Для вычисления циклической свертки длины n можно сначала вычислить линейную свертку, а затем выполнить приведение по модулю $x^n - 1$. Однако, как правило, лучше сразу выбрать $b(x)$ равным $x^n - 1$, и приведение по модулю $x^n - 1$ производить в процессе применения китайской теоремы об остатках. В случае линейной свертки степень многочлена $b(x)$ должна быть больше суммы степеней многочленов $g(x)$ и $d(x)$, в то время как степень многочлена $x^n - 1$ может быть намного меньше. В этом случае либо многочлены, по модулям которых производятся вычисления, имеют меньшие степени, либо число таких многочленов меньше. С другой стороны, простые делители многочлена $x^n - 1$ заранее фиксированы, а делители многочлена $b(x)$, хотя степень его и выше, могут быть выбраны так, как нам удобно; взаимно простые делители многочлена $b(x)$ даже не обязаны быть простыми.

На рис. 11.1 приведены некоторые алгоритмы циклической свертки в полях характеристики 2, построенные с помощью рассмотренных методов. В компактной матричной форме эти алгоритмы записываются равенством

$$\mathbf{c} = \mathbf{B}[(\mathbf{Cg}) \cdot (\mathbf{Ad})],$$

где **Cg** и **Ad** — векторы одинаковой размерности, а $[(\mathbf{Cg}) \cdot (\mathbf{Ad})]$ — их покомпонентное произведение. Матричное представление этих алгоритмов однозначно, но последовательность выполнения сложений не регламентирована. При разумном упорядочении сложений и использовании частичных сумм можно также минимизировать число сложений. Если многочлен $g(x)$ фиксирован, то вектор **Cg** может быть вычислен заранее. Тогда алгоритм вычисления свертки можно записать в виде $\mathbf{c} = \mathbf{BGAd}$, где **G** представляет собой диагональную матрицу, на диагонали которой выписаны компоненты вектора **Cg**. В качестве примера такой формы алгоритма циклической свертки на рис. 11.2 выписан алгоритм четырехточечной циклической свертки.

Данный параграф мы завершаем простым примером построения кодера для (7, 3)-кода Рида—Соломона. Эта простенькая задача не позволяет в полной мере продемонстрировать мощь рассматриваемого метода, но зато дает возможность легко про-

Двухточечная циклическая свертка, 3 умножения

$$\begin{bmatrix} c_0 \\ c_1 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix} \left\{ \begin{bmatrix} 1 & 1 \\ 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} g_0 \\ g_1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} d_0 \\ d_1 \end{bmatrix} \right\}$$

Трехточечная циклическая свертка, 4 умножения

$$\begin{bmatrix} c_0 \\ c_1 \\ c_2 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix} \left\{ \begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix} \begin{bmatrix} g_0 \\ g_1 \\ g_2 \end{bmatrix} \cdot \begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix} \begin{bmatrix} d_0 \\ d_1 \\ d_2 \end{bmatrix} \right\}$$

Четырехточечная циклическая свертка, 9 умножений

$$\begin{bmatrix} c_0 \\ c_1 \\ c_2 \\ c_3 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 0 \end{bmatrix} \left\{ \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} g_0 \\ g_1 \\ g_2 \\ g_3 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} d_0 \\ d_1 \\ d_2 \\ d_3 \end{bmatrix} \right\}$$

Пятиточечная циклическая свертка, 10 умножений

$$\begin{bmatrix} c_0 \\ c_1 \\ c_2 \\ c_3 \\ c_4 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 0 & 1 & 1 & 0 & 1 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \end{bmatrix} \left\{ \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 & 0 \end{bmatrix} \begin{bmatrix} g_0 \\ g_1 \\ g_2 \\ g_3 \\ g_4 \end{bmatrix} \cdot \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 & 0 \end{bmatrix} \begin{bmatrix} d_0 \\ d_1 \\ d_2 \\ d_3 \\ d_4 \end{bmatrix} \right\}$$

Рис. 11.1. Алгоритмы коротких сверток

следить работу алгоритма. Более того, мы не будем пытаться построить наиболее эффективный алгоритм, а опишем такой путь построения алгоритма, который даст возможность проиллюстрировать многие идеи.

Для (7, 3)-кода Рида—Соломона с порождающим многочленом $g(x) = x^4 + \alpha^3 x^3 + x^2 + \alpha x + \alpha^3$ степень многочлена $d(x)$ не превосходит двух. Для непосредственного вычисления $g(x)\,d(x)$

Девятиточечная циклическая свертка, 19 умножений

$$\begin{bmatrix} c_0 \\ c_1 \\ c_2 \\ c_3 \\ c_4 \\ c_5 \\ c_6 \\ c_7 \\ c_8 \end{bmatrix} = \begin{bmatrix} 1&1&1&1&1&1&1&1&1 \\ 1&1&0&1&1&0&1&1&0 \\ 1&0&1&1&0&1&1&0&1 \\ 0&1&1&0&1&1&0&1&1 \\ 0&1&1&0&1&1&0&0&0 \\ 0&1&1&0&0&0&0&1&1 \\ 0&0&0&0&1&1&0&1&1 \\ 1&1&0&0&0&1&1&1&1 \\ 0&0&1&1&1&1&1&1&0 \\ 1&1&1&1&0&0&0&1& \\ 1&1&0&0&1&1&1&0&1 \\ 0&1&1&1&0&1&1&1&0 \\ 1&0&1&1&1&0&0&1&1 \\ 1&1&1&0&0&1&1&1&0 \\ 0&0&1&1&1&0&1&1&1 \\ 1&1&0&1&1&1&0&0&1 \\ 0&1&0&0&0&1&0&1&1 \\ 0&0&1&0&1&1&0&1&0 \\ 0&1&1&0&1&0&0&0&1 \end{bmatrix}^T \left\{ \begin{bmatrix} 1&1&1&1&1&1&1&1&1 \\ 1&0&1&1&0&1&1&0&1 \\ 1&1&0&1&1&0&1&1&0 \\ 0&1&1&0&1&1&0&1&1 \\ 1&0&0&1&0&0&0&0&0 \\ 0&0&0&1&0&0&1&0&0 \\ 1&0&0&0&0&0&1&0&0 \\ 1&1&1&1&0&1&0&1&0 \\ 0&1&0&1&1&1&1&0&1 \\ 1&0&1&0&1&0&1&1&1 \\ 1&1&0&1&0&0&0&1&0 \\ 0&1&0&1&1&0&1&0&0 \\ 1&0&0&0&1&0&1&1&0 \\ 1&0&1&1&0&1&0&0&0 \\ 0&0&0&1&0&1&1&0&1 \\ 1&0&1&0&0&0&1&0&1 \\ 0&0&0&0&1&1&0&1&1 \\ 0&1&1&0&0&0&0&1&1 \\ 0&1&1&0&1&1&0&0&0 \end{bmatrix} \begin{bmatrix} g_0 \\ g_1 \\ g_2 \\ g_3 \\ g_4 \\ g_5 \\ g_6 \\ g_7 \\ g_8 \end{bmatrix} \cdot \begin{bmatrix} 1&1&1&1&1&1&1&1&1 \\ 1&0&1&1&0&1&1&0&1 \\ 1&1&0&1&1&0&1&1&0 \\ 0&1&1&0&1&1&0&1&1 \\ 1&0&0&1&0&0&0&0&0 \\ 0&0&0&1&0&0&1&0&0 \\ 1&0&0&0&0&0&1&0&0 \\ 1&1&1&1&0&1&0&1&0 \\ 0&1&0&1&1&1&1&0&1 \\ 1&0&1&0&1&0&1&1&1 \\ 1&1&0&1&0&0&0&1&0 \\ 0&1&0&1&1&0&1&0&0 \\ 1&0&0&0&1&0&1&1&0 \\ 1&0&1&1&0&1&0&0&0 \\ 0&0&0&1&0&1&1&0&1 \\ 1&0&1&0&0&0&1&0&1 \\ 0&0&0&0&1&1&0&1&1 \\ 0&1&1&0&0&0&0&1&1 \\ 0&1&1&0&1&1&0&0&0 \end{bmatrix} \begin{bmatrix} d_0 \\ d_1 \\ d_2 \\ d_3 \\ d_4 \\ d_5 \\ d_6 \\ d_7 \\ d_8 \end{bmatrix} \right\}$$

Семиточечная циклическая свертка, 13 умножений

$$\begin{bmatrix} c_0 \\ c_1 \\ c_2 \\ c_3 \\ c_4 \\ c_5 \\ c_6 \end{bmatrix} = \begin{bmatrix} 1&1&1&1&1&1&1 \\ 0&1&0&0&1&1&1 \\ 1&0&0&1&1&1&0 \\ 1&1&0&1&0&0&1 \\ 1&1&1&0&1&0&0 \\ 0&0&1&1&1&0&1 \\ 0&1&1&1&0&1&0 \\ 0&1&1&1&0&0&1 \\ 0&1&0&1&1&1&0 \\ 0&0&1&0&1&1&1 \\ 1&1&0&0&1&0&1 \\ 1&0&0&1&0&1&1 \\ 1&1&1&0&0&1&0 \end{bmatrix}^T \left\{ \begin{bmatrix} 1&1&1&1&1&1&1 \\ 0&1&1&1&0&0&1 \\ 1&0&1&1&1&0&0 \\ 1&1&0&0&1&0&1 \\ 1&0&0&1&0&1&1 \\ 0&1&0&1&1&1&0 \\ 0&0&1&0&1&1&1 \\ 0&1&1&1&0&1&0 \\ 1&1&0&1&0&0&1 \\ 1&0&1&0&0&1&1 \\ 1&0&0&1&1&1&0 \\ 0&1&0&0&1&1&1 \\ 0&0&1&1&1&0&1 \end{bmatrix} \begin{bmatrix} g_0 \\ g_1 \\ g_2 \\ g_3 \\ g_4 \\ g_5 \\ g_6 \end{bmatrix} \cdot \begin{bmatrix} 1&1&1&1&1&1&1 \\ 1&0&1&1&1&0&0 \\ 0&1&0&1&1&1&0 \\ 0&0&1&0&1&1&1 \\ 1&1&1&0&0&1&0 \\ 1&1&0&0&1&0&1 \\ 1&0&0&1&0&1&1 \\ 0&1&1&1&0&1&0 \\ 1&0&0&1&1&1&0 \\ 0&0&1&1&1&0&1 \\ 1&1&1&0&1&0&0 \\ 1&1&0&1&0&0&1 \\ 1&0&1&0&0&1&1 \end{bmatrix} \begin{bmatrix} d_0 \\ d_1 \\ d_2 \\ d_3 \\ d_4 \\ d_5 \\ d_6 \end{bmatrix} \right\}$$

для полей характеристики 2.

необходимо 15 умножений. Однако поскольку данный частный многочлен $g(x)$ имеет только два отличных от единицы значения коэффициентов, то тривиально просто так организовать вычисление, что понадобится только шесть умножений. Быстрый алгоритм, который будет приведен ниже, превосходит алгоритмы как с 15, так и с шестью умножениями. Заметим также, что можно воспользоваться приведенным на рис. 11.1 алгоритмом

$$\begin{bmatrix} c_0 \\ c_1 \\ c_2 \\ c_3 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 0 \end{bmatrix} \left\{ \begin{bmatrix} G_0 & & & & & & & 0 \\ & G_1 & & & & & & \\ & & G_2 & & & & & \\ & & & G_3 & & & & \\ & & & & G_4 & & & \\ & & & & & G_5 & & \\ & & & & & & G_6 & \\ & & & & & & & G_7 \\ & 0 & & & & & & & G_8 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} d_0 \\ d_1 \\ d_2 \\ d_3 \end{bmatrix} \right\},$$

где

$$\begin{bmatrix} G_0 \\ G_1 \\ G_2 \\ G_3 \\ G_4 \\ G_5 \\ G_6 \\ G_7 \\ G_8 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} g_0 \\ g_1 \\ g_2 \\ g_3 \end{bmatrix}$$

Рис. 11.2. Алгоритм Винограда для вычисления четырехточечной свертки.

семиточечной свертки, который, однако, содержит 13 умножений.

Согласно изложенной выше теории, можно выбрать любой многочлен $b(x)$, степень которого не меньше 7. Выберем $b(x) = x^7 + 1$; тогда

$$(x^7 + 1) = (x + 1)(x^3 + x + 1)(x^3 + x^2 + 1) = \\ = b_1(x)\, b_2(x)\, b_3(x).$$

Можно было бы выбрать также многочлен

$$b(x) = x^2 (x + 1)^2 (x^3 + x + 1),$$

у которого степени взаимно простых делителей несколько ниже, но наш выбор приводит к лучшим результатам.

Прежде всего найдем все необходимые для алгоритма фиксированные многочлены:

$$g^{(1)}(x) = R_{b_1(x)}[g(x)] = \alpha,$$

$$g^{(2)}(x) = R_{b_2(x)}[g(x)] = 1,$$

$$g^{(3)}(x) = R_{b_3(x)}[g(x)] = \alpha^3 x^2 + \alpha^3 x + 1.$$

Согласно китайской теореме об остатках для многочленов,

$$a^{(1)}(x) = x^6 + x^5 + x^4 + x^3 + x^2 + x + 1,$$
$$a^{(2)}(x) = x^4 + x^2 + x + 1,$$
$$a^{(3)}(x) = x^6 + x^5 + x^3 + 1.$$

Теперь нам надо вычислить три короткие свертки:

$$c^{(1)}(x) = g^{(1)}(x)\, d^{(1)}(x) = \alpha d^{(1)}(x),$$
$$c^{(2)}(x) = g^{(2)}(x)\, d^{(2)}(x) = d^{(2)}(x),$$
$$c^{(3)}(x) = g^{(3)}(x)\, d^{(3)}(x) = (\alpha^3 x^2 + \alpha^3 x + 1)\, d^{(3)}(x).$$

По заданному многочлену $d(x)$ вычисляем вычеты

$$d^{(1)}(x) = d_2 + d_1 + d_0,$$
$$d^{(2)}(x) = d_2 x^2 + d_1 x + d_0,$$
$$d^{(3)}(x) = d_2 x^2 + d_1 x + d_0.$$

Таким образом, для вычисления $c^{(1)}(x)$ требуется одно умножение, для вычисления $c^{(2)}(x)$ умножений делать не надо, а для вычисления $c^{(3)}(x)$ требуется три умножения. Собирая вместе все эти вычисления, получаем следующее выражение для свертки:

$$\begin{aligned} c(x) = {} & (A_1 + A_2 + A_4 + A_6 + A_8) x^6 + (A_1 + A_3 + A_4 + A_7) x^5 + \\ & + (A_1 + A_2 + A_4 + A_5 + A_8) x^4 + (A_1 + A_2 + A_3 + \\ & + A_4 + A_5 + A_6) x^3 + \\ & + (A_1 + A_2 + A_3 + A_4 + A_5 + A_6 + A_7) x^2 + \\ & + (A_1 + A_3 + A_4 + A_6 + A_7 + A_8) x + (A_1 + A_5 + \\ & + A_7 + A_8), \end{aligned}$$

где

$$A_1 = \alpha (d_2 + d_1 + d_0),$$
$$A_2 = d_2,$$
$$A_3 = d_1,$$
$$A_4 = d_0,$$
$$A_5 = \alpha^3 d_2,$$
$$A_6 = \alpha^3 d_2 + \alpha^3 d_1,$$
$$A_7 = \alpha^3 d_1 + \alpha^3 d_0,$$
$$A_8 = \alpha^3 d_0 + d_1.$$

Выполняемое в таком виде вычисление свертки $c(x)$ требует четырех умножений и 35 сложений в поле $GF(8)$. Несомненно,

однако, что промежуточные частичные суммы можно вычислять так, чтобы уменьшить число сложений.

Если продолжать процесс, то можно попытаться уменьшить число умножений при вычислении $c^{(3)}(x)$:

$$c^{(3)}(x) = g^{(3)}(x)\, d^{(3)}(x) = (\alpha^3 x^2 + \alpha^3 x + 1)(d_2 x^2 + d_1 x + d_0).$$

Выберем

$$b'(x) = x(x+1)(x^2 + x + 1);$$

тогда

$$c^{(3)}(x) = (\alpha^3 d_2)\, b'(x) + R_{b'(x)}[c^{(3)}(x)].$$

Для вычисления $\alpha^3 d_2$ необходимо только одно умножение. Вычеты даются равенствами

$$g^{(3)(1)}(x) = 1,$$
$$g^{(3)(2)}(x) = 1,$$
$$g^{(3)(3)}(x) = \alpha$$

и

$$d^{(3)(1)}(x) = d_0,$$
$$d^{(3)(2)}(x) = d_0 + d_1 + d_2,$$
$$d^{(3)(3)}(x) = (d_1 + d_2)\, x + (d_0 + d_2).$$

Тогда

$$c^{(3)(1)}(x) = d^{(3)(1)}(x),$$
$$c^{(3)(2)}(x) = d^{(3)(2)}(x),$$
$$c^{(3)(3)}(x) = \alpha d^{(3)(3)}(x).$$

Для вычисления $c^{(3)(3)}(x)$ необходимо два умножения. Мы уже использовали одно умножение для вычисления $\alpha^3 d_2$, так что всего для вычисления $c^{(3)}(x)$ с помощью выписанной выше процедуры требуются три умножения. Так как непосредственное вычисление $c^{(3)}(x)$ также включает три умножения, то предпочтение надо отдать ему.

11.3. БЫСТРЫЕ ПРЕОБРАЗОВАНИЯ ФУРЬЕ

Дискретное преобразование Фурье (ДПФ) в той форме, как оно задается:

$$V_j = \sum_{i=0}^{n-1} \alpha^{ij} v_i,$$

требует порядка n^2 умножений и n^2 сложений. Для составного n имеется несколько способов превратить такое преобразование

ДПФ

$$V_j = \sum_{i=0}^{n-1} \alpha^{ij} v_i$$

Число умножений $\approx n^2$

БПФ-алгоритм Кули—Тьюки [1965]

$$n = n'n'',$$

$$i = i' + n'i'', \quad i' = 0, \ldots, n' - 1, \quad j'' = 0, \ldots, n'' - 1$$

$$j = n''j' + j'', \quad j' = 0, \ldots, n' - 1, \quad j'' = 0, \ldots, n'' - 1.$$

$$V_{j', j''} = \sum_{i'=0}^{n'-1} \beta^{i'j'} \left[\alpha^{i'j''} \sum_{i''=0}^{n''-1} \gamma^{i''j''} v_{i', i''} \right]$$

Число умножений $\approx n(n' + n'') + n$

БПФ-алгоритм Гуда—Томаса ([1960], [1963])

$n = n'n''$, n' и n'' взаимно просты

Перестановка входных символов

$$\left.\begin{aligned} i' &= i \pmod{n'}, \\ i'' &= i \pmod{n''} \end{aligned}\right\} \leftrightarrow \left\{\begin{aligned} &i = i'N''n'' + i''N'n' \pmod{n}, \\ &\text{причем} \\ &N'n' + N''n'' = 1 \end{aligned}\right.$$

Перестановка выходных символов

$$\left.\begin{aligned} j' &= N''j \pmod{n'} \\ j'' &= N'j \pmod{n''} \end{aligned}\right\} \leftrightarrow j = n''j' + n'j'' \pmod{n}$$

$$V_{j', j''} = \sum_{i'=0}^{n'-1} \beta^{i'j'} \left[\sum_{i''=0}^{n''-1} \gamma^{i''j''} v_{i', i''} \right]$$

Число умножений $\approx n(n' + n'')$

Рис. 11.3. Некоторые алгоритмы БПФ.

Фурье в двумерное преобразование или в нечто ему подобное. С вычислительной точки зрения это приводит к существенно более эффективной форме, хотя понять ее структуру труднее. Алгоритмы подобного рода известны под общим названием *быстрого преобразования Фурье* (*БПФ*). Некоторые рассматриваемые в этом параграфе алгоритмы, а именно алгоритм Кули—Тьюки и алгоритм Гуда—Томаса, приведены на рис. 11.3.

Начнем с БПФ-алгоритма Кули—Тьюки. Предположим, что $n = n'n''$. В выражении для преобразования Фурье сделаем следующую перестановку входных и выходных символов:

$$\begin{aligned} i &= i' + n'i'', & i' &= 0, \ldots, n' - 1, \\ & & i'' &= 0, \ldots, n'' - 1, \\ j &= n''j' + j'', & j' &= 0, \ldots, n' - 1, \\ & & j'' &= 0, \ldots, n'' - 1. \end{aligned}$$

Тогда

$$V_{n''j'+j''} = \sum_{i''=0}^{n''-1} \sum_{i'=0}^{n'-1} \alpha^{(i'+n'i'')(n''j'+j'')} v_{i'+n'i''}.$$

Раскроем скобки в показателе степени и положим $\alpha^{n'} = \gamma$ и $\alpha^{n''} = \beta$. Так как порядок элемента α равен $n'n''$, то член $\alpha^{n'n''i''j'} = 1$ можно опустить. Определим теперь двумерные переменные, которые также обозначим через v и V, задавая их равенствами

$$\begin{aligned} v_{i', i''} &= v_{i'+n'i''}, & i' &= 0, \ldots, n' - 1, \\ & & i'' &= 0, \ldots, n'' - 1. \\ V_{j', j''} &= V_{n''j'+j''}, & j' &= 0, \ldots, n' - 1, \\ & & j'' &= 0, \ldots, n'' - 1. \end{aligned}$$

Это задает отображение входного и выходного векторов данных в двумерные таблицы, причем

$$V_{j', j''} = \sum_{i'=0}^{n'-1} \beta^{i'j'} \left[\alpha^{i'j''} \sum_{i''=0}^{n''-1} \gamma^{i''j''} v_{i', i''} \right].$$

Хотя эта формула сложнее исходной, число сложений и умножений при вычислении по ней гораздо меньше, а именно требуется не более $n(n' + n'' + 1)$ умножений по сравнению с n^2, необходимых в исходном варианте. Заметим, что при каждом значении i' внутренняя сумма представляет собой n''-точечное преобразование Фурье и при каждом значении i'' внешняя сумма представляет собой n'-точечное преобразование Фурье. Если соответствующая длина является составным числом, то каждое из этих преобразований в свою очередь можно упростить с помощью быстрого преобразования Фурье. Преобразование длины n с делителями n_i таким образом представляется в форме, требующей около $n \sum_i n_i$ умножений.

БПФ-алгоритм Кули—Тьюки можно наглядно представить в виде отображения двумерной таблицы в двумерную таблицу, пример которого для $n = 15$ и $n = 21$ приведен на рис. 11.4. Преобразование Фурье сначала применяется к каждой строке, а затем к каждому столбцу. Перед применением преобразования Фурье к столбцам производится поэлементное умножение. Отметим, что компоненты спектра упорядочены иначе, чем компоненты

сигнала. Такое упорядочение называется перестановкой адресов. Поля Галуа, в которых преобразование Фурье вычисляется наиболее просто, имеют вид $GF(2^m + 1)$, где число $2^m + 1$ является простым. Известно, что если m не является степенью двойки, то $2^m + 1$ не является простым числом. Обратное утверждение

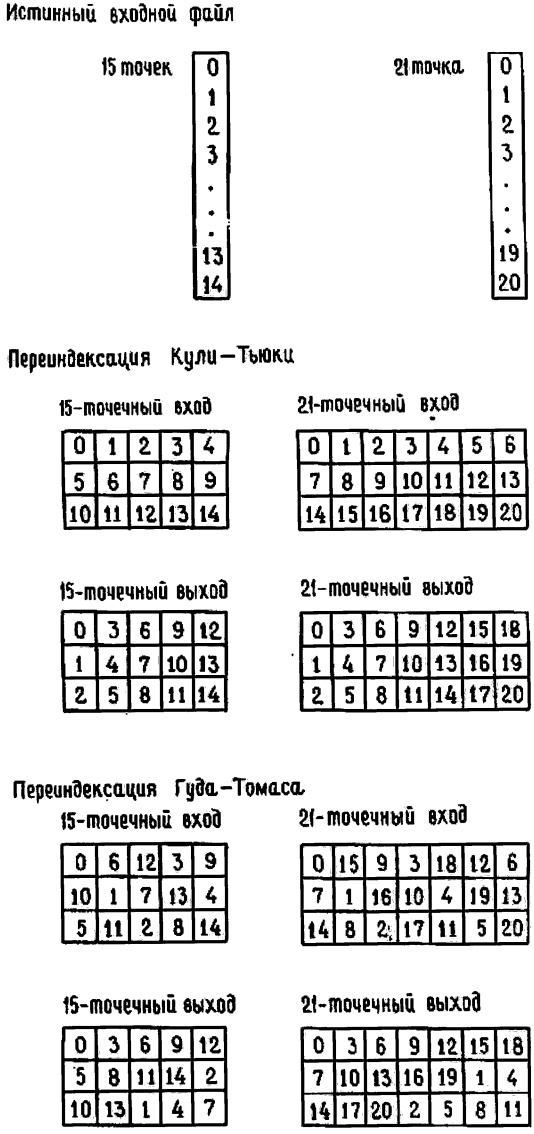


Рис. 11.4. Примеры переиндексации.

неверно, так как известно, что число $2^{32}+1$ является составным. Но для $m=2$, 4, 8 и 16 числа $2^m+1=5$, 17, 257 и 65 537 являются простыми и известны под названием *простых чисел Ферма*. В поле Галуа $GF(q)$, где q — простое число Ферма, все делители числа $q-1$, включая и само это число, являются степенью двойки. Преобразование Фурье

$$V_j=\sum_{i=0}^{n-1}\alpha^{ij}v_i$$

существует для всех n, делящих 2^m, и всех элементов α, порядок которых равен n. Таким образом, в поле $GF(2^{16}+1)$ существуют преобразования Фурье длин, равных 2^{16}, 2^{15}, 2^{14}; ..., 2^2, 2. Построенный в таком поле систематический код Рида—Соломона может быть превращен в двоичный код, исправляющий пакеты ошибок. Этот код можно использовать для систематического кодирования r-битовых байтов; одно из 2^r значений запрещается использовать в качестве информационных символов. Проверочные символы принимают 2^r+1 значений, так что для их двоичного представления требуется $r+1$ битов.

Так как в поле $GF(2^{16}+1)$ порядок элемента 2 равен 32, то преобразование Фурье длины не более 32 является чрезвычайно простым. Чтобы показать это, заметим, что в поле $GF(2^{16}+1)$ справедливо равенство $2^{16}+1=0$, так что $2^{16}=-1$ и $2^{32}=1$. Преобразование Фурье имеет вид

$$V_j=\sum_{i=0}^{31}2^{ij}v_i,$$

так что умножение происходит только на степени двойки и операция умножения по существу представляет собой циклический сдвиг в двоичной арифметической системе. Такое преобразование Фурье очень легко вычисляется с помощью двоичного БПФ-алгоритма Кули—Тьюки, но длина преобразования, равная 32, слишком мала, чтобы иметь много приложений. Более длинные преобразования Фурье должны включать нетривиальные умножения. Однако использование 32-ичного БПФ-алгоритма Кули—Тьюки позволяет уменьшить число таких умножений. Например, рассмотрим 1024-точечное преобразование в поле $GF(2^{16}+1)$:

$$V_j=\sum_{i=0}^{1023}\alpha^{ij}v_i,$$

где α — элемент порядка 1024. БПФ-алгоритм Кули—Тьюки приводит эту формулу к виду

$$V_{n''j'+j''}=\sum_{i'=0}^{31}2^{i'j'}\left[\alpha^{i'j''}\sum_{i''=0}^{31}2^{i''j''}v_{i'+n'i''}\right].$$

Внутренняя сумма при каждом i' является 32-точечным преобразованием Фурье, а внешняя сумма является 32-точечным преобразованием Фурье при каждом i''. Только со степенью элемента α связаны нетривиальные умножения, но их число равно 1024. В общем случае для вычисления преобразования Фурье в поле $GF(2^{16}+1)$ требуется около $n(\lceil \log_{32} n \rceil - 1)$ умножений в этом поле, $(1/2)\, n \log_2 n$ сложений и $(1/2)\, n \log_2 n$ сдвигов.

Другим алгоритмом БПФ является алгоритм Гуда—Томаса для взаимно простых делителей. По сравнению с алгоритмом Кули—Тьюки он немного сложнее в принципе, но и немного проще в вычислительном отношении. В алгоритме Гуда—Томаса форма переиндексации линейного массива в двумерную $(n' \times n'')$-таблицу, позволяющая вычислять одномерное преобразование Фурье с помощью двумерного преобразования Фурье, отличается от формы переиндексации в алгоритме Кули—Тьюки. Алгоритм Гуда—Томаса по идее существенно отличается от алгоритма Кули—Тьюки. В нем n' и n'' предполагаются взаимно простыми, и в основе отображения лежит китайская теорема об остатках. Способ упорядочения данных иллюстрируется рис. 11.4. Упорядочение начинается с левого верхнего угла таблицы, и компоненты записываются вдоль продолжения диагонали. Так как число строк и число столбцов взаимно просты, то продолженная диагональ пройдет через все элементы таблицы. После выполнения двумерного преобразования Фурье компоненты спектра выписываются в двумерной таблице в другом порядке.

БПФ-алгоритм Гуда—Томаса основан на китайской теореме об остатках для целых чисел. Входные индексы определяются своими вычетами по правилу

$$i' = i \pmod{n'}, \qquad i'' = i \pmod{n''},$$

задающему отображение входного индекса i в пару индексов (i', i''), лежащих на продолженной диагонали двумерной таблицы. Согласно китайской теореме об остатках, существуют целые числа N' и N'', такие, что

$$N'n' + N''n'' = 1.$$

Это позволяет следующим образом однозначно вычислить исходный индекс i по паре (i', i''):

$$i = i'N''n'' + i''N'n' \pmod{n}.$$

Выходные индексы упорядочиваются несколько иным способом. Определим

$$j' = N''j \pmod{n'}, \qquad j'' = N'j \pmod{n''}.$$

В эквивалентном виде это можно записать так:

$$j'' = ((N' \bmod n'')\, j \bmod n''),$$

$$j' = ((N'' \bmod n')\, j \bmod n').$$

Тогда выходной индекс j можно вычислить следующим образом:

$$j = n''j' + n'j'' \pmod{n}.$$

Чтобы проверить это, запишем

$$\begin{aligned} j &= n''\,(N''j + Q_1 n') + n'\,(N'j + Q_2 n'') \pmod{n'n''} \\ &= j\,(n''N'' + n'N') \pmod{n} \\ &= j. \end{aligned}$$

При такой переиндексации формула $V_j = \sum_{i=0}^{n-1} \alpha^{ij} v_i$ принимает вид

$$V_{n''j'+n'j''} = \sum_{i''=0}^{n''-1} \sum_{i'=0}^{n'-1} \alpha^{(i'N''n''+i''N'n')\ (n''j'+n'j'')} v_{i'N''n''+i''N'n'}.$$

Выполним умножение в показателе степени. Так как порядок элемента α равен $n'n''$, то члены, содержащие $n'n''$, можно отбросить. Используя выписанные соответствия для индексов, будем рассматривать входной и выходной векторы как двумерные таблицы. Тогда

$$\begin{aligned} V_{j', j''} &= \sum_{i'=0}^{n'-1} \sum_{i''=0}^{n''-1} \alpha^{N''(n'')^2 i'j'} \alpha^{N'(n')^2 i''j''} v_{i', i''} = \\ &= \sum_{i'=0}^{n'-1} \sum_{i''=0}^{n''-1} \beta^{i'j'} \gamma^{i''j''} v_{i', i''}, \end{aligned}$$

где $\beta = \alpha^{N''(n'')^2}$ — элемент порядка n', а $\gamma = \alpha^{N'(n')^2}$ — элемент порядка n''. Полученная формула представляет собой двумерное $(n' \times n'')$-точечное преобразование Фурье, для вычисления которого требуется примерно $n(n' + n'')$ умножений и примерно столько же сложений. Если длина преобразования по столбцам или по строкам является составным числом, то можно провести дальнейшее упрощение, снова применяя быстрое преобразование Фурье. Поступая таким образом, можно преобразование длины n, распадающейся на взаимно простые множители n_l, привести к форме, для вычисления которой требуется порядка $n \sum_l n_l$ умножений и сложений.

Для выполнения преобразования Фурье можно выбрать как алгоритм Кули—Тьюки, так и алгоритм Гуда—Томаса. Иногда возможно даже использование обоих алгоритмов одновременно. Например, используя БПФ-алгоритм Гуда—Томаса, можно раз-

бить 63-точечное преобразование на 7-точечное и 9-точечное, а затем, используя БПФ-алгоритм Кули—Тьюки, свести 9-точечное преобразование к двум 3-точечным. Организованное так вычисление будет аналогично $(3 \times 3 \times 7)$-точечному трехмерному преобразованию Фурье.

11.4. АЛГОРИТМЫ АГАРВАЛА—КУЛИ ВЫЧИСЛЕНИЯ СВЕРТОК

Ту же схему индексации, которая была использована в алгоритме Гуда—Томаса для превращения одномерного преобразования Фурье в многомерное преобразование Фурье, можно использовать для превращения одномерной свертки в многомерную. Этот метод разбиения свертки известен под названием алгоритма Агарвала—Кули вычисления свертки. Он не дает такого же уменьшения числа умножений, как алгоритм Винограда вычисления свертки, но зато и не имеет такой, как у алгоритма Винограда, тенденции роста числа сложений для больших n. Далее этот метод более структурирован и, следовательно, для больших n более обозрим, так как допускает разбиения на подалгебры. Используя алгоритм Агарвала—Кули разбиения длинной свертки на короткие и алгоритм Винограда вычисления коротких сверток, можно получить хорошие алгоритмы для вычисления свертки.

В отличие от алгоритма Винограда для свертки, использующего китайскую теорему об остатках для многочленов, алгоритм Агарвала—Кули основан на китайской теореме об остатках для целых чисел и может быть использован только в тех случаях, когда длина n имеет взаимно простые делители. Для заданных векторов с компонентами g_i и d_i при $i = 0, \ldots, n-1$ требуется вычислить циклическую свертку

$$c_l = \sum_{k=0}^{n-1} g_{((l-k))} d_k, \quad l = 0, \ldots, n-1,$$

где двойные скобки у индексов обозначают вычисления по модулю n.

Превратим эту одномерную свертку в двумерную. В предыдущем параграфе мы уже видели, как с помощью китайской теоремы об остатках отобразить одномерные входные данные в двумерную таблицу и как двумерную таблицу выходных данных отобразить обратно в одномерный вектор. Заменим индексы i и k парами индексов (i', i'') и (k', k'') так, что

$$\begin{aligned} i &= n''i' + n'i'', & i' &= 0, \ldots, n'-1, \\ & & i'' &= 0, \ldots, n''-1, \\ k &= n''k' + n'k'', & k' &= 0, \ldots, n'-1, \\ & & k'' &= 0, \ldots, n''-1. \end{aligned}$$

При рассмотрении алгоритма Гуда—Томаса мы уже видели, как определить новые индексы, чтобы они выполняли свое предназначение:

$$i' = N''i \pmod{n'}, \qquad k' = N''k \pmod{n'},$$
$$i'' = N'i \pmod{n''}, \qquad k'' = N'k \pmod{n''},$$

где N' и N'' представляют собой целые числа, удовлетворяющие условию

$$N'n' + N''n'' = 1.$$

Тогда свертка записывается равенством

$$c_{n''i'+n'i''} = \sum_{k'=0}^{n'-1} \sum_{k''=0}^{n''-1} d_{n''k'+n'k''} g_{n''(i'-k')+n'(i''-k'')}.$$

Двойное суммирование по паре (k', k'') эквивалентно суммированию по k, так как перебираются те же самые члены суммы. Определим теперь двумерные переменные g, d и c:

$$g_{k',k''} = g_{n''k'+n'k''}, \quad k' = 0, \ldots, n' - 1, \quad k'' = 0, \ldots, n'' - 1,$$
$$d_{k',k''} = d_{n''k'+n'k''}, \quad k' = 0, \ldots, n' - 1, \quad k'' = 0, \ldots, n'' - 1,$$
$$c_{k',k''} = c_{n''k'+n'k''} \quad k' = 0, \ldots, n' - 1, \quad k'' = 0, \ldots, n'' - 1,$$

так что свертка задается выражением

$$c_{i',i''} = \sum_{k'=0}^{n'-1} \sum_{k''=0}^{n''-1} d_{k',k''} g_{i'-k',i''-k''},$$

где первые и вторые индексы берутся соответственно по модулю n' и n''. Мы получили настоящую двумерную циклическую свертку.

Для того чтобы описанный алгоритм был полезен, надо располагать эффективным способом вычисления двумерной циклической свертки. Применим вдоль каждого из двух измерений алгоритм Винограда вычисления свертки. Этот метод легче проследить, представив двумерную таблицу в виде одномерной таблицы многочленов. Определим множество n'' многочленов равенствами

$$d_{k''}(x) = \sum_{k'=0}^{n'-1} d_{k',k''} x^{k'}, \quad k'' = 0, \ldots, n'' - 1.$$

Аналогично определим

$$g_{k''}(x) = \sum_{k'=0}^{n'-1} g_{k',k''} x^{k'}, \quad k'' = 0, \ldots, n'' - 1,$$

и

$$c_{k''}(x) = \sum_{k'=0}^{n'-1} c_{k',k''} x^{k'}, \quad k'' = 0, \ldots, n'' - 1.$$

При этом двумерная свертка превращается в одномерную свертку многочленов:

$$c_{i''}(x) = \sum_{k''=0}^{n''-1} g_{((k'-k''))}(x)\, d_{k''}(x) \pmod{x^{n'} - 1}, \quad i'' = 0, \ldots, n'' - 1.$$

Для каждого i'' в этой сумме производится сложение n'' произведений многочленов. Всего имеется $(n'')^2$ произведений многочленов. Каждое из произведений многочленов является сверткой и может быть вычислено с помощью подходящего алгоритма. Одним из таких алгоритмов является алгоритм Винограда вычисления свертки.

Для полного вычисления свертки алгоритмом Агарвала—Кули надо применить также алгоритм Винограда вычисления свертки по второму измерению. Но сначала остановимся и оценим сложность этих промежуточных вычислений. Пусть $M(l)$ и $A(l)$ обозначают соответственно число сложений и число умножений, необходимых для вычисления свертки с помощью имеющихся в некоторой доступной библиотеке алгоритмов вычисления циклической свертки. Тогда для вычисления двумерной свертки необходимо $(n'')^2 M(n')$ умножений и $(n'')^2 A(n') + (n'' - 1) n'' n'$ сложений. Эти вычисления можно выполнить даже еще лучше, воспользовавшись алгоритмом Винограда вычисления свертки и вдоль второй оси.

Так как алгоритм Винограда вычисления свертки представляет собой тождество, содержащее сложения и умножения, то можно заменить в нем арифметические элементы многочленами. Следовательно, $c_{i''}(x)$ можно вычислить за $M(n'')$ умножений многочленов и $A(n'')$ сложений многочленов, а для выполнения каждого умножения двух многочленов потребуется $M(n')$ арифметических умножений и $A(n')$ арифметических сложений, так что всего получится $M(n'')\,M(n')$ арифметических умножений и $M(n'')\,A(n') + A(n'')\,M(n')$ арифметических сложений. Число сложений не является симметричной функцией относительно параметров n' и n'', чем можно воспользоваться для минимизации числа сложений.

Аналогично если длина n циклической свертки является произведением трех или более взаимно простых множителей,

Длина преобразования	Число умножений
2	3
3	4
4	9
5	10
6	12
7	13
9	19
10	30
14	39
15	40
21	76
35	130
45	190
63	247
105	520
315	2 470
630	7 410
1 260	22 230

Рис. 11.5. Характеристики некоторых алгоритмов Агарвала—Кули вычисления циклических сверток. (Используются алгоритмы коротких сверток, приведенные на рис. 11.1.)

то алгоритм Агарвала—Кули позволяет преобразовать эту свертку в трехмерную или многомерную циклическую свертку и вычислять по каждому отдельному измерению циклическую свертку с помощью алгоритма Винограда.

Структуру многомерной циклической свертки можно описать через одномерные циклические свертки также с помощью кронекеровского произведения (кронекеровское произведение будет описано в § 11.5). На рис. 11.5 приведена таблица мультипликативной сложности некоторых алгоритмов Агарвала—Кули вычисления циклической свертки в полях характеристики 2.

11.5. АЛГОРИТМ ВИНОГРАДА БЫСТРОГО ПРЕОБРАЗОВАНИЯ ФУРЬЕ

Отличный от рассмотренного ранее метод вычисления быстрого преобразования Фурье дается БПФ-алгоритмом Винограда. Этот алгоритм основывается на четырех отдельных идеях: алгоритме Рейдера, описанном в § 11.2 алгоритме Винограда вычисления коротких сверток, схеме индексации алгоритма Гуда—Томаса для простых множителей, гнездовом алгоритме Винограда (см.

ниже). БПФ-алгоритм Винограда лучше БПФ-алгоритмов Кули—Тьюки и Гуда—Томаса по числу умножений, но сложнее по структуре. За уменьшение числа умножений приходится платить усложнением индексации и использованием операций отображения.

БПФ-алгоритм Винограда подразделяется на малый БПФ-алгоритм Винограда и большой БПФ-алгоритм Винограда. К малому алгоритму относятся преобразования, длина которых равна малому простому числу или степени малого простого числа. Большой БПФ-алгоритм Винограда объединяет малые БПФ-алгоритмы Винограда, чтобы получить преобразование на большой длине [1]).

Первым шагом является построение малого БПФ-алгоритма Винограда. Если n — малое простое число, то воспользуемся алгоритмом Рейдера, сводя вычисление преобразования Фурье к вычислению циклической свертки, которую в свою очередь вычислим с помощью алгоритма Винограда вычисления коротких сверток. В общем случае уравнения выписываются непосредственно, так что выбирать слишком большие n непрактично. В алгоритме Рейдера сведения дискретного преобразования Фурье к свертке используется только переиндексация; на этом шаге не требуется никаких сложений и умножений. Структура алгоритма вычисления свертки включает сначала сложения, потом умножения и затем опять сложения. Если n представляет собой степень малого простого числа, то можно использовать ту же самую процедуру, заменив алгоритм Рейдера некоторым более общим алгоритмом, который мы рассматривать не будем.

Построим алгоритм Винограда вычисления пятиточечного БПФ в поле $GF(16)$, находящий величины

$$V_j = \sum_{i=0}^{4} \omega^{ij} v_i, \quad j = 0, \ldots, 4, \ \omega = \alpha^3.$$

Воспользуемся рассмотренным ранее (см. § 9.8) алгоритмом Рейдера, сводящим пятиточечное преобразование Фурье к свертке

$$b(x) = g(x)\, a(x) \pmod{x^4 - 1},$$

где

$$g(x) = \alpha^7 x^3 + \alpha^{11} x^2 + \alpha^{13} x + \alpha^{14},$$

$$a(x) = v_3 x^3 + v_4 x^2 + v_2 x + v_1,$$

$$b(x) = (V_3 - V_0)\, x^3 + (V_4 - V_0)\, x^2 + (V_2 - V_0)\, x + (V_1 - V_0).$$

---

[1]) Такое сведение преобразования Фурье на большой длине к совокупности преобразований Фурье на меньших длинах получило название гнездового алгоритма Винограда. — *Прим. перев.*

Здесь многочлен $g(x)$ фиксирован, многочлен $a(x)$ получается из многочлена $v(x)$ перестановкой коэффициентов с отбрасыванием, а многочлен $V(x)$ получается из многочлена $b(x)$ без отбрасывания коэффициента.

Малый пятиточечный алгоритм Винограда вычисления БПФ получится, если для вычисления произведения $g(x)\,a(x)$ воспользоваться алгоритмом Винограда вычисления короткой свертки. Обращаясь к таблице на рис. 11.2, находим алгоритм Винограда вычисления четырехточечной циклической свертки, содержащий девять умножений. Воспользуемся этим алгоритмом для построения алгоритма преобразования Фурье. Соединим операции переиндексации алгоритма Рейдера с матричными операциями алгоритма свертки, выполняя соответствующие перестановки строк и столбцов. Далее, так как коэффициенты многочлена $g(x)$ являются фиксированными константами поля $GF(16)$, то произведение g на соответствующую матрицу можно вычислить заранее. Внеся все эти изменения в алгоритм четырехточечной свертки и присоединяя к нему компоненты V_0 и v_0, получаем малый пятиточечный алгоритм Винограда БПФ. На рис. 11.6 приведена запись этого алгоритма в матричном виде:

$$\mathbf{V} = \mathbf{BDAv}.$$

Отметим, что матрица **A** предварительных сложений и матрица **B** последующих сложений не являются квадратными. Пятиточечный входной вектор перед выполнением покомпонентного умножения вытягивается в десятиточечный, задаваемый матрицей **D**. Первая строка матричного равенства связана с вычислением компоненты V_0 и не содержит умножений. Остальные девять строк определяются алгоритмом четырехточечной циклической свертки. Одна из констант, на которую происходит умножение, равна еди-

$$\begin{bmatrix} V_0 \\ V_1 \\ V_2 \\ V_3 \\ V_4 \end{bmatrix} = \begin{bmatrix} 1&0&0&0&0&0&0&0&0&0 \\ 1&1&0&1&1&0&0&0&0&1 \\ 1&1&0&1&0&1&0&0&1&0 \\ 1&1&1&0&0&1&1&0&0&0 \\ 1&1&1&0&1&0&0&1&0&0 \end{bmatrix} \left\{ \begin{bmatrix} 1 & & & & & & & & & 0 \\ & 1 & & & & & & & & \\ & & \alpha^2 & & & & & & & \\ & & & \alpha^8 & & & & & & \\ & & & & \alpha^5 & & & & & \\ & & & & & \alpha^{10} & & & & \\ & & & & & & \alpha^{14} & & & \\ & & & & & & & \alpha^{13} & & \\ & & & & & & & & \alpha^{11} & \\ 0 & & & & & & & & & \alpha^7 \end{bmatrix} \begin{bmatrix} 1&1&1&1&1 \\ 0&1&0&0&0 \\ 0&1&0&0&1 \\ 0&1&0&0&1 \\ 0&1&1&0&0 \\ 0&1&0&1&0 \\ 0&1&1&1&1 \\ 0&1&1&1&1 \\ 0&1&1&1&1 \\ 0&1&1&1&1 \end{bmatrix} \begin{bmatrix} v_0 \\ v_1 \\ v_2 \\ v_3 \\ v_4 \end{bmatrix} \right\}$$

Рис. 11.6. Малый пятиточечный алгоритм Винограда вычисления БПФ.

нице, так что на самом деле приходится выполнять только восемь умножений [1]).

Рассмотренный способ реализации малого БПФ-алгоритма Винограда можно использовать в случае, когда n является простым числом. Можно также построить малый БПФ-алгоритм Винограда, когда n является степенью малого простого числа. Вообще говоря, пользоваться малыми БПФ-алгоритмами Вино-

---

[1]) Заметим, что для построения 5-точечного ДПФ с девятью умножениями достаточно воспользоваться только тем, что ядро α ДПФ длины 5 удовлетворяет тождеству $1 + \alpha + \alpha^2 + \alpha^3 + \alpha^4 = 0$. Это позволяет организовать вычисление величины $V_j = \sum_{i=0}^{4} \alpha^{ij} v_i$ в полях характеристики 2 (последнее, впрочем, несущественно) следующим образом:

$$V_0 = v_0 + v_1 + v_2 + v_3 + v_4,$$
$$V_2 = (v_0 + v_1) + (v_1 + v_4)\,\alpha^2 + (v_2 + v_4)\,\alpha^4 + (v_3 + v_4)\,\alpha,$$
$$V_1 = (v_0 + v_4) + (v_1 + v_4)\,\alpha + (v_2 + v_4)\,\alpha^2 + (v_3 + v_4)\,\alpha^3,$$
$$V_3 = (v_0 + v_2) + (v_1 + v_4)\,\alpha^3 + (v_2 + v_4)\,\alpha + (v_3 + v_4)\,\alpha^4,$$
$$V_4 = v_0 + V_0 + V_1 + V_2 + V_3.$$

На самом же деле использование алгоритма Рейдера сведения ДПФ простой длины к циклической свертке позволяет построить алгоритм 5-точечного БПФ, содержащий всего пять умножений. А именно для произвольных многочленов $a(x) = a_0 + a_1x + a_2x^2 + a_3x^3$ и $b(x) = b_0 + b_1x + b_2x^2 + b_3x^3$ алгоритм 4-точечной циклической свертки $c(x) = c_0 + c_1x + c_2x^2 + c_3x^3 = a(x)\,b(x) \pmod{(x^4 - 1)}$ перепишем в виде

$$c_0 = (a_0 + a_1)(b_0 + b_2) + b_1 \sum_{i=0}^{3} a_i + (a_0 + a_2)(b_1 + b_2) + a_1 \sum_{i=0}^{3} b_i,$$

$$c_0 + c_2 = (b_0 + b_2) \sum_{i=0}^{3} a_i + (a_1 + a_3) \sum_{i=0}^{3} b_i,$$

$$c_0 + c_1 + c_2 = (b_0 + b_2)(a_0 + a_1) + b_1 \sum_{i=0}^{3} a_i + (a_1 + a_3)(b_2 + b_3) + a_2 \sum_{i=0}^{3} b_i,$$

$$c_0 + c_1 + c_2 + c_3 = \left(\sum_{i=0}^{3} a_i\right)\left(\sum_{i=0}^{3} b_i\right).$$

В рассматриваемом случае $b(x) = \alpha + \alpha^3 x + \alpha^4 x^2 + \alpha^2 x^3$, и, следовательно, в силу тождества для ядра, $\sum_{i=0}^{3} b_i = 1$, так что по существу эти умножения отсутствуют, и приходится выполнять только пять умножений. — *Прим. перев.*

града можно только при малых длинах. При больших длинах преобразований предпочтительнее алгоритмы с менее четкой структурой, хотя бы и за счет некоторого увеличения числа умножений. Этим требованиям удовлетворяет большой БПФ-алгоритм Винограда.

В общем случае БПФ-алгоритм Винограда строится для длины n, равной произведению простых чисел или степеней простых чисел. Рассмотрим случай двух множителей, когда $n = n'n''$. Воспользуемся алгоритмом Гуда—Томаса для преобразования n-точечного преобразования Фурье в двумерное $(n' \times n'')$-точечное преобразование Фурье. Отдельные компоненты этого двумерного преобразования Фурье могут быть вычислены с помощью соответственно n'-точечного и n''-точечного алгоритмов Винограда. Сначала выполняется n'-точечное преобразование Фурье каждой строки, а затем n''-точечное преобразование Фурье каждого столбца.

Перед БПФ-алгоритмом Винограда выполняется, однако, еще один шаг. Так как не существенно, применяется ли преобразование Фурье сначала к строкам или сначала к столбцам двумерного блока, то представляется возможным как-то их совместить. Именно это и делает БПФ-алгоритм Винограда. Он совмещает вычисления преобразований строк и столбцов, уменьшая при этом число умножений. Этот метод использует кронекеровское произведение матриц. Сделаем перерыв в изложении, чтобы ввести определение кронекеровского произведения матриц и доказать одну важную теорему. После этого мы сможем завершить построение БПФ-алгоритма Винограда.

Определение 11.5.1. Пусть $\mathbf{A} = (a_{ik})$ и $\mathbf{B} = (b_{jl})$ — матрицы соответственно размеров $I \times K$ и $J \times L$. Тогда *кронекеровским произведением* матриц $\mathbf{A}$ и $\mathbf{B}$, обозначаемым $\mathbf{A} \times \mathbf{B}$, называется матрица, содержащая IJ строк и KL столбцов, у которой на пересечении строки с номером $(i-1)J + j$ и столбца с номером $(k-1)L + l$ стоит элемент

$$c_{ij,\,kl} = a_{ik}b_{jl}.$$

Чтобы проще понять структуру кронекеровского произведения матриц, следует представить матрицу $\mathbf{A} \times \mathbf{B}$ в виде $(I \times K)$-таблицы, состоящей из $(J \times L)$-блоков, (i, k)-й из которых равен $a_{ik}\mathbf{B}$. Непосредственно из определения вытекает, что кронекеровское произведение матриц некоммутативно, но ассоциативно:

$$\mathbf{A} \times \mathbf{B} \neq \mathbf{B} \times \mathbf{A}, \quad (\mathbf{A} \times \mathbf{B}) \times \mathbf{C} = \mathbf{A}\,(\mathbf{B} \times \mathbf{C}).$$

Элементы матриц $\mathbf{A} \times \mathbf{B}$ и $\mathbf{B} \times \mathbf{A}$ одни и те же, но упорядочены по-разному. Очевидно также, что кронекеровское произведение дистрибутивно относительно обычного сложения матриц.

Следующая теорема утверждает, что кронекеровское произведение двух произведений матриц равно матричному произведению кронекеровских произведений соответствующих матриц.

Теорема 11.5.2. *Если все матричные произведения определены, то кронекеровское произведение удовлетворяет равенству*

$$(\mathbf{A} \times \mathbf{B})(\mathbf{C} \times \mathbf{D}) = (\mathbf{AC}) \times (\mathbf{BD}).$$

Доказательство. Пусть матрицы **A**, **B**, **C** и **D** имеют соответственно размеры $I \times K$, $J \times L$, $K \times M$ и $L \times N$. Так как матрица $\mathbf{A} \times \mathbf{B}$ содержит KL столбцов, а матрица $\mathbf{C} \times \mathbf{D}$ содержит KL строк, то матричное произведение $(\mathbf{A} \times \mathbf{B})(\mathbf{C} \times \mathbf{D})$ определено. Оно содержит IJ строк, которые мы занумеруем парами (i, j), и MN столбцов, которые мы занумеруем парами (m, n). Элемент, стоящий на пересечении строки (i, j) и столбца (m, n), равен $\sum_{kl} a_{ik} b_{jl} c_{km} d_{ln}$. Так как матрица **AC** содержит I строк и M столбцов, а матрица **BD** содержит J строк и L столбцов, то матрица $(\mathbf{AC}) \times (\mathbf{BD})$ также является $(IJ \times KL)$-матрицей. Стоящий на пересечении (i, j)-строки и (m, n)-столбца элемент этой матрицы равен

$$\sum_k a_{ik} c_{km} \sum_l b_{jl} d_{ln} = \sum_{k,\,l} a_{ik} b_{jl} c_{km} d_{ln},$$

что и завершает доказательство. □

Кронекеровское произведение матриц применяется в теории преобразования Фурье. Пусть $\mathbf{W}'$ и $\mathbf{W}''$ — матрицы преобразования Фурье соответственно длин n' и n''. Тогда

$$\mathbf{V}' = \mathbf{W}'\mathbf{v}' \quad \text{и} \quad \mathbf{V}'' = \mathbf{W}''\mathbf{v}''$$

являются соответственно матричными записями преобразований Фурье

$$V'_j = \sum_{i=0}^{n'-1} \beta^{ij} v'_i \quad \text{и} \quad V''_j = \sum_{i=0}^{n''-1} \gamma^{ij} v''_i.$$

Двумерное $(n' \times n'')$-преобразование Фурье двумерного сигнала $\{v_{i'i''}\}$ получается применением преобразования $\mathbf{W}'$ к каждой строке с последующим применением преобразования $\mathbf{W}''$ к каждому столбцу. Считывая двумерный $(n' \times n'')$-сигнал по строкам, можно преобразовать его в одномерный. (Возможно, что двумерный сигнал был предварительно сформирован из одномерного с помощью китайской теоремы об остатках. Считывание такого сигнала по строкам дает новое одномерное представление этого сигнала, которое получается из исходного одномерного сигнала перестановкой его компонент.)

Если предполагать, что одномерные $n'n''$-точечные векторы $\{v_{i'i''}\}$ и $\{V_{i'i''}\}$ упорядочены описанным выше способом, то

двумерное преобразование записывается через кронекеровское произведение в виде

$$\mathbf{V} = (\mathbf{W}' \times \mathbf{W}'')\,\mathbf{v}.$$

Для алгоритма Винограда преобразований длин n' и n' имеет место матричное разложение

$$\mathbf{W}' = \mathbf{B}'\mathbf{D}'\mathbf{A}' \quad \text{и} \quad \mathbf{W}'' = \mathbf{B}''\mathbf{D}''\mathbf{A}'',$$

где $\mathbf{A}'$, $\mathbf{A}''$, $\mathbf{B}'$ и $\mathbf{B}''$ — матрицы целых чисел рассматриваемого поля, а $\mathbf{D}'$ и $\mathbf{D}''$ — диагональные матрицы над $GF(q)$. Умножения на $\mathbf{D}'$ и $\mathbf{D}''$ описывают все умножения, которые приходится выполнять в алгоритме Винограда. Положим $\mathbf{W} = \mathbf{W}' \times \mathbf{W}''$ и дважды применим теорему 11.7.2:

$$\mathbf{W} = (\mathbf{B}'\mathbf{D}'\mathbf{A}') \times (\mathbf{B}''\mathbf{D}''\mathbf{A}'') =$$

$$= (\mathbf{B}' \times \mathbf{B}'')(\mathbf{D}' \times \mathbf{D}'')(\mathbf{A}' \times \mathbf{A}'') = \mathbf{BDA},$$

где кронекеровские произведения $\mathbf{B} = \mathbf{B}' \times \mathbf{B}''$ и $\mathbf{A} = \mathbf{A}' \times \mathbf{A}''$ содержат только элементы из поля $GF(p)$, а кронекеровское произведение $\mathbf{D} = \mathbf{D}' \times \mathbf{D}''$ опять является диагональной матрицей над $GF(q)$. Следовательно, мы опять получили $n'n''$-точечное преобразование Фурье в форме алгоритма Винограда БПФ. Это дает способ построения большого алгоритма Винограда БПФ из малых.

Из приведенного описания матрицы $\mathbf{W}$ видно, что такое преобразование предполагает предварительную запись (с помощью алгоритма Гуда—Томаса для взаимно простых множителей) $n'n''$-точечного одномерного преобразования в виде двумерного преобразования с последующим считыванием преобразуемого двумерного вектора по строкам в виде одномерного $n'n''$-точечного вектора $\mathbf{V}$. Следовательно, описанный способ вычисления $n'n''$-точечного преобразования $\mathbf{V} = (\mathbf{BDA})\,\mathbf{v}$ предполагает предварительную перестановку компонент вектора $\mathbf{v}$ и вычисление компонент вектора $\mathbf{V}$ также в переставленном порядке. Но поскольку вид матриц $\mathbf{A}$ и $\mathbf{B}$ известен, то тривиальная перестановка столбцов матрицы $\mathbf{A}$ и строк матрицы $\mathbf{B}$ позволяет выписывать компоненты векторов $\mathbf{v}$ и $\mathbf{V}$ в их естественном порядке.

Пусть $M(n')$ и $M(n'')$ означают соответственное число умножений, необходимых для выполнения n'- и n''-точечного БПФ по алгоритму Винограда. Тогда число $M(n)$ умножений, необходимых для вычисления $n'n''$-точечного преобразования по алгоритму Винограда, равно примерно $M(n')\,M(n'')$. Более точно,

$$\dim \mathbf{D} = (\dim \mathbf{D}')(\dim \mathbf{D}''),$$

и величины $M(n')$, $M(n'')$ и $M(n)$ не превосходят размеров матриц $\mathbf{D}'$, $\mathbf{D}''$ и $\mathbf{D}$ соответственно, так как один или более элементов диагональных матриц могут оказаться равными единице. Если учитывать все умножения диагональных матриц, включая и умножения на единицу, то получаем простую формулу

$$M(n) = M(n')\,M(n'').$$

11.6. УСКОРЕННЫЙ АЛГОРИТМ БЕРЛЕКЭМПА—МЕССИ

В описанном в § 7.4 алгоритме Берлекэмпа—Месси требуемое число умножений на r-й итерации примерно равно удвоенной степени многочлена $\Lambda^{(r)}(x)$. Так как степень многочлена $\Lambda^{(r)}(x)$ имеет порядок r и в типичном случае равна $r/2$, а алгоритм содержит $2t$ итераций, то для его выполнения необходимо $2t^2$ умножений и примерно столько же сложений. Это квадратичная функция t. Кратко говорят, что алгоритм Берлекэмпа—Месси содержит число умножений порядка t^2, или, формально, $O(t^2)$ умножений. Для очень длинных кодов и большого t число умножений в алгоритме может стать обременительным. В настоящем параграфе рассматривается метод уменьшения вычислительной сложности алгоритма для длинных кодов при большом числе t исправляемых ошибок.

Ускоренный алгоритм обосновывается следующим образом. На начальных итерациях алгоритм Берлекэмпа—Месси в частотной области содержит умеренное число умножений порядка степени многочлена $\Lambda(x)$. Но степень многочлена $\Lambda(x)$ возрастает до существенной доли длины n, и, таким образом, среднее число умножений на итерацию имеет порядок n. Преимущества ускоренного алгоритма связаны попросту с тем, что на ранних шагах несколько итераций в частотной области выполняются одновременно. После такого пакетного выполнения нескольких итераций полученный промежуточный результат используется для модификации синдрома путем ввода в него вычисленных в этом пакете многочленов $\Lambda(x)$ и $B(x)$. Следующий пакет итераций начинает выполнение алгоритма Берлекэмпа—Месси, сначала используя в качестве синдрома вычисленный модифицированный синдром и полагая в начальный момент $\Lambda(x)$ равным 1. Ускоренный декодер эффективен для декодирования длинных, но все еще практически применяемых кодов. В следующем параграфе будут рассмотрены другие, даже более эффективные декодеры.

Построение начнем с более компактной организации алгоритма Берлекэмпа—Месси. Заменим многочлены $\Lambda^{(r)}(x)$ и $B^{(r)}(x)$ (2×2)-матрицей из многочленов

$$\Lambda^{(r)}(x) = \begin{bmatrix} \Lambda_{11}^{(r)}(x) & \Lambda_{12}^{(r)}(x) \\ \Lambda_{21}^{(r)}(x) & \Lambda_{22}^{(r)}(x) \end{bmatrix}.$$

Для обозначения элементов матрицы $\Lambda^{(r)}(x)$ нам понадобится несколько индексов; символ $\Lambda_{ab,j}^{(r)}$ будет означать j-й коэффициент многочлена, стоящего на пересечении строки a и столбца b в r-й итерации.

Напомним, что вычисления алгоритма Берлекэмпа—Месси основываются на двух равенствах:

$$\Delta_r = \sum_{j=0}^{n-1} \Lambda_j^{(r-1)} S_{r-j}$$

и

$$\begin{bmatrix} \Lambda^{(r)}(x) \\ B^{(r)}(x) \end{bmatrix} = \begin{bmatrix} 1 & -\Delta_r x \\ \Delta_r^{-1}\delta_r & (1-\delta_r)x \end{bmatrix} \begin{bmatrix} \Lambda^{(r-1)}(x) \\ B^{(r-1)}(x) \end{bmatrix}.$$

Второе равенство можно записать в виде

$$\begin{bmatrix} \Lambda^{(r)}(x) \\ B^{(r)}(x) \end{bmatrix} = \left\{ \prod_{l=1}^{r} \begin{bmatrix} 1 & -\Delta_l x \\ \Delta_l^{-1}\delta_l & (1-\delta_l)x \end{bmatrix} \right\} \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

Определим матрицу $\Lambda^{(r)}(x)$ равенством

$$\Lambda^{(r)}(x) = \prod_{l=1}^{r} \begin{bmatrix} 1 & -\Delta_l x \\ \Delta_l^{-1}\delta_l & (1-\delta_l)x \end{bmatrix}.$$

Через эту матрицу можно выразить многочлены $\Lambda^{(r)}(x)$ и $B^{(r)}(x)$:

$$\begin{bmatrix} \Lambda^{(r)}(x) \\ B^{(r)}(x) \end{bmatrix} = \Lambda^{(r)}(x) \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} \Lambda_{11}^{(r)}(x) + \Lambda_{12}^{(r)}(x) \\ \Lambda_{21}^{(r)}(x) + \Lambda_{22}^{(r)}(x) \end{bmatrix}.$$

Полученное равенство может быть использовано как для вычисления $\Lambda^{(r)}(x)$, так и для вычисления многочленов $\Lambda^{(r)}(x)$ и $B^{(r)}(x)$. Непосредственное вычисление матрицы $\Lambda^{(r)}(x)$ содержит вдвое больше умножений, так как вместо двух элементов в вычислениях участвуют четыре. Поэтому замена итераций многочленов $\Lambda^{(r)}(x)$ и $B^{(r)}(x)$ итерациями матрицы $\Lambda^{(r)}(x)$ удваивает необходимое число умножений. Это ухудшение будет затем устранено реорганизацией вычислений.

В перестроенном виде алгоритм Берлекэмпа—Месси основывается на двух уравнениях, а именно на уравнениях

$$\Delta_r = \sum_{j=0}^{n-1} \Lambda_{11,j}^{(r-1)} S_{r-j} + \sum_{j=0}^{n-1} \Lambda_{12,j}^{(r-1)} S_{r-j}$$

и

$$\Lambda^{(r)}(x) = \begin{bmatrix} 1 & -\Delta_r x \\ \Delta_r^{-1}\delta_r & (1-\delta_r)\,x \end{bmatrix} \Lambda^{(r-1)}(x).$$

Теперь мы хотим уменьшить число вычислений, группируя итерации в пакеты. Определим матрицу

$$\mathbf{M}^{(r)}(x) = \begin{bmatrix} 1 & -\Delta_r x \\ \Delta_r^{-1}\delta_r & (1-\delta_r)\,x \end{bmatrix},$$

так что

$$\Lambda^{(r)}(x) = \mathbf{M}^{(r)}(x)\,\mathbf{M}^{(r-1)}(x)\ldots\mathbf{M}^{(1)}(x).$$

Это произведение должно вычисляться последовательно по одному члену, начиная справа, так как $\mathbf{M}^{(r)}(x)$ нельзя вычислить до тех пор, пока не вычислена $\Lambda^{(r-1)}(x)$.

Определим следующие частичные произведения рассматриваемых матриц:

$$\Lambda^{(r,\,r')}(x) = \mathbf{M}^{(r)}(x)\,\mathbf{M}^{(r-1)}(x)\ldots\mathbf{M}^{(r'+1)}(x);$$

тогда

$$\Lambda^{(r)}(x) = \Lambda^{(r,\,0)}(x) \quad \text{и} \quad \Lambda^{(r)}(x) = \Lambda^{(r,\,r')}(x)\,\Lambda^{(r')}(x).$$

Определим также вектор модифицированных синдромных многочленов $\mathbf{S}^{(r)}(x)$:

$$\mathbf{S}^{(r)}(x) = \Lambda^{(r)}(x)\begin{bmatrix} S(x) \\ S(x) \end{bmatrix} = \Lambda^{(r,\,r')}(x)\,\mathbf{S}^{(r')}(x).$$

Невязка

$$\Delta_r = \sum_{j=0}^{n-1} \Lambda_{11,\,j}^{(r-1)} S_{r-j} + \sum_{j=0}^{n-1} \Lambda_{12,\,j}^{(r-1)} S_{r-j}$$

становится равной r-му члену свертки и может быть записана через многочлены:

$$\begin{aligned}\Delta(x) &= \Lambda_{11}^{(r-1)}(x)\,S(x) + \Lambda_{12}^{(r-1)}(x)\,S(x) = \\ &= \left[\Lambda_{11}^{(r-1,\,r')}(x)\,\Lambda_{11}^{(r')}(x) + \Lambda_{12}^{(r-1,\,r')}(x)\,\Lambda_{21}^{(r')}(x)\right] S(x) + \\ &\quad + \left[\Lambda_{11}^{(r-1,\,r')}(x)\,\Lambda_{12}^{(r')}(x) + \Lambda_{12}^{(r-1,\,r')}(x)\,\Lambda_{22}^{(r')}(x)\right] S(x).\end{aligned}$$

Группируя члены, получаем

$$\begin{aligned}\Delta(x) &= \Lambda_{11}^{(r-1,\,r')}(x)\left[\Lambda_{11}^{(r')}(x)\,S(x) + \Lambda_{12}^{(r')}(x)\,S(x)\right] + \\ &\quad + \Lambda_{12}^{(r-1,\,r')}(x)\left[\Lambda_{21}^{(r')}(x)\,S(x) + \Lambda_{22}^{(r')}(x)\,S(x)\right].\end{aligned}$$

Стоящие в квадратных скобках многочлены являются компонентами вектора $S^{(r')}(x)$ модифицированных синдромных многочленов. Следовательно, выражение для невязки можно переписать в виде

$$\Delta_r = \sum_{j=0}^{r-r'} \Lambda_{11,\,j}^{(r-1,\,r')} S_{1,\,r-j}^{(r')} + \sum_{j=0}^{r-r'} \Lambda_{12,\,j}^{(r-1,\,r')} S_{2,\,r-j}^{(r')},$$

где $S_{1,j}^{(r)}$ и $S_{2,j}^{(r)}$ обозначают соответственно j-е коэффициенты многочленов в первой и второй компонентах вектора $S^{(r)}(x)$. Для ускорения алгоритма Берлекэмпа—Месси сформируем пакеты по τ итераций в каждом. Если τ делит $2t$, то обращение к каждому из $2t/\tau$ пакетов одинаково. В противном случае последний пакет содержит меньше τ итераций, но мы полагаем его таким же. Индекс r теперь пробегает значения

$$r = 1, 2, \ldots, \tau, \quad \tau + 1, \ldots, 2\tau, \quad 2\tau + 1, \ldots, 3\tau,$$
$$3\tau + 1, \ldots .$$

На r-й итерации k-го пакета модифицированный алгоритм Берлекэмпа—Месси вычисляет $\Lambda^{(r,\,r')}(x)$ согласно равенству

$$\Lambda^{(r,\,r')}(x) = \begin{bmatrix} 1 & -\Delta_r x \\ \Delta_r^{-1}\delta_r & (1-\delta_r)\,x \end{bmatrix} \Lambda^{(r-1,\,r')}(x),$$

где $r' = k\tau$, а Δ_r вычисляется по формуле

$$\Delta_r = \sum_{j=0}^{r-r'} \Lambda_{11,\,j}^{(r-1,\,r')} S_{1,\,r-j}^{(r')} + \sum_{j=0}^{r-r'} \Lambda_{12,\,j}^{(r-1,\,r')} S_{2,\,r-j}^{(r')}.$$

В конце k-го пакета, когда r равно $(k+1)\tau$, матрица локаторов ошибок и вектор модифицированных синдромных многочленов даются соответственно равенствами

$$\Lambda^{(r)}(x) = \Lambda^{(r,\,r')}(x)\,\Lambda^{(r')}(x) \quad \text{и} \quad S^{(r)}(x) = \Lambda^{(r,\,r')}(x)\,S^{(r')}(x),$$

где $r' = k\tau$ и $r = (k+1)\tau$. Теперь ускоренный алгоритм готов начать $(k+1)$-й пакет.

На рис. 11.7 приведена блок-схема ускоренного алгоритма Берлекэмпа—Месси с некоторыми очевидными сокращениями. В левой части рисунка приведен стандартный алгоритм, расписанный для τ итераций, а в правой показаны блоки, осуществляющие связку пакетов по τ итераций каждый. Если правая часть алгоритма выполняется так, как она выписана, то мы не получаем никакой экономии вычислений. Однако умножения многочленов в блоках правой части эквивалентны сверткам, которые могут быть эффективно вычислены с помощью описанных в § 11.1 и 11.3 методов или с помощью быстрого преобразования Фурье и теоремы о свертке. Некоторые возможности иллюстрируются следующим примером.

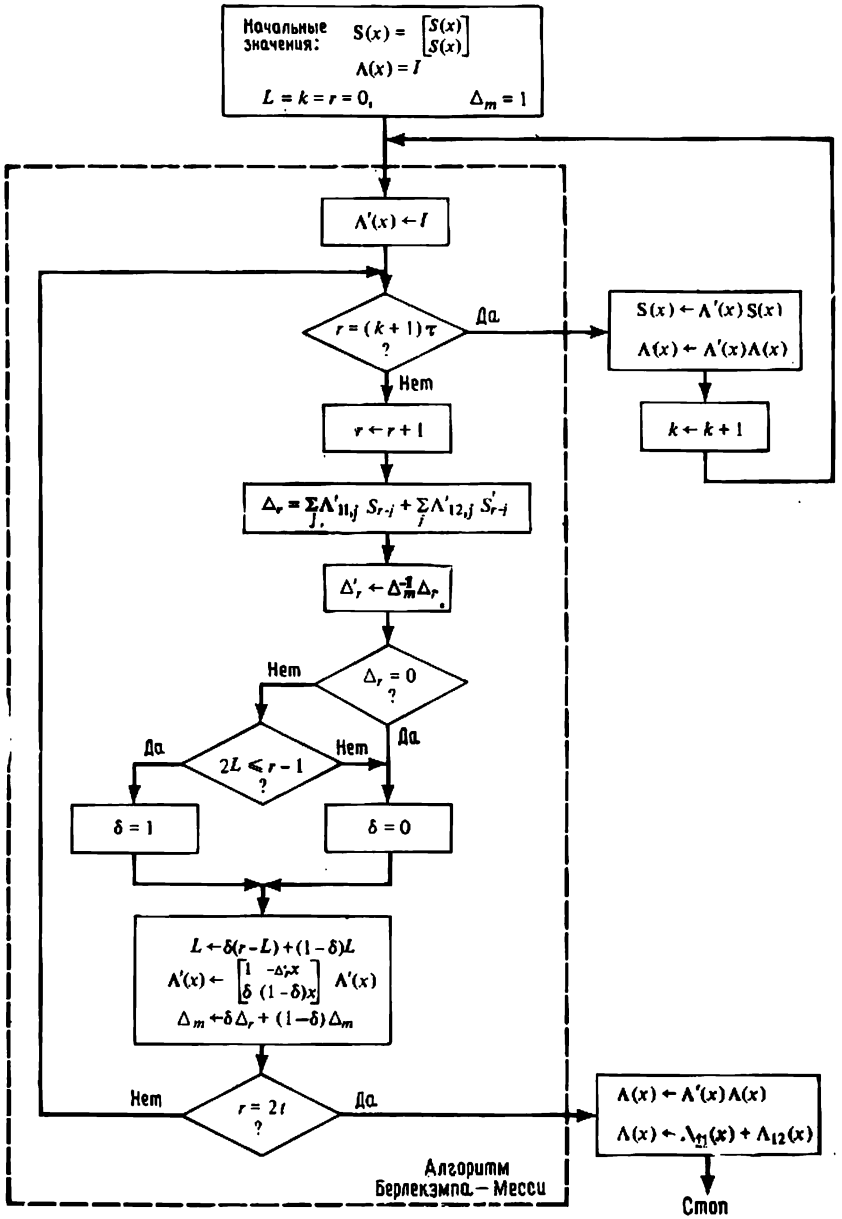


Рис. 11.7. Ускоренный алгоритм Берлекэмпа—Мессн.

Расширенный (4096, 2048)-код Рида—Соломона над $GF(2^{12})$ имеет скорость, равную 1/2, и позволяет исправлять все конфигурации ошибок из 1024 12-битовых байтов. В типичных случаях алгоритм Берлекэмпа—Месси содержит $2t^2$ (или 2 097 152) умножений 12-битовых чисел поля $GF(2^{12})$, а для наихудших конфигураций ошибок это число может возрасти в 1,5 раза. Ускоренный алгоритм позволяет провести вычисления лучше.

Подсчитаем число операций для несколько видоизмененного варианта ускоренного алгоритма Берлекэмпа—Месси. Вместо того чтобы формировать пакеты одинакового объема, делая разветвления алгоритма через каждые τ итераций, будем строить разветвления в те моменты, когда степень многочлена $\Lambda(x)$ достигнет предписанного предела. А именно будем формировать новую ветвь алгоритма тогда, когда степень одного из многочленов, входящих в матрицу $\mathbf{\Lambda}(x)$, станет равной 128, и закончим процесс формирования ветвей после 2048 итераций. В случае когда происходит точно 1024 ошибки, алгоритм будет содержать восемь разветвлений, каждое из которых будет выполнять 128 итераций. Именно этот случай мы сейчас проанализируем.

В течение каждой итерации требующееся для выполнения вычислений, показанных слева на рис. 11.7, число умножений примерно вчетверо больше степени многочлена $\Lambda'(x)$, вычисляемого при этой итерации. Это вдвое больше числа умножений, необходимых в исходном варианте алгоритма Берлекэмпа—Месси. Следовательно, в каждом ответвлении алгоритма имеется примерно τ^2 итераций, выполняющих вычисления, указанные на левой части рис. 11.7. Всего имеется восемь ветвей, и τ равно 128; таким образом, выполнение всех ветвей алгоритма содержит 524 228 умножений. К этому надо прибавить число умножений, необходимых для вычисления указанных в правой части рис. 11.7 сверток, объединяющих вычисленные пакеты. Для вычисления этих линейных сверток воспользуемся алгоритмом 315-точечной циклической свертки, содержащим 2470 умножений. Этот алгоритм представляет собой алгоритм Агарвала—Кули вычисления свертки, построенной из трех алгоритмов вычисления коротких сверток, длины которых соответственно равны пяти, семи и девяти.

После каждого ответвления надо вычислять член вида $\Lambda(x)S(x)$, где степень многочлена $S(x)$ равна $2048\ k\tau$, а степень многочлена $\Lambda(x)$ равна 128. Воспользуемся методом перекрытия с накомплением и алгоритмом 315-точечной циклической свертки. Это даст нам $315 - \deg \Lambda(x) = 187$ правильных выходных точек линейной свертки $\Lambda(x)\,S(x)$. Для вычисления 2048—256 точек алгоритм циклической свертки придется применить 10 раз. Так как $\mathbf{\Lambda}'(x)\,\mathbf{S}(x)$ содержит четыре линейные свертки, всего потребуется 40 обращений к алгоритму циклической свертки.

Продолжая таким образом, всего получим 168 обращений к алгоритму циклической свертки, что в итоге дает 480 480 умножений.

После выполнения каждого пакета из 128 итераций необходимо также вычислить матричное произведение $\Lambda'(x)\,\Lambda(x)$. Разбивая в матричном произведении длинные свертки на короткие и проводя вычисления, аналогичные проделанным, приходим к 216-кратному обращению к алгоритму 315-точечной циклической свертки.

Полное число умножений, необходимых для рассматриваемого декодирования (4096, 2048)-кода Рида—Соломона, получается сложением этих трех величин. Всего получаем 1 462 768 умножений против 2 097 152 необходимых при непосредственном использовании алгоритма Берлекэмпа—Месси.

11.7. РЕКУРРЕНТНЫЙ АЛГОРИТМ БЕРЛЕКЭМПА—МЕССИ

Можно построить декодер, еще более эффективный, чем рассмотренный в предыдущем параграфе. Такой улучшенный декодер представляет не только практический,но и теоретический интерес, так как позволяет доказать, что асимптотическая сложность декодирования кодов Рида—Соломона не превосходит величины $O(n \log^2 n)$ (и очень близка к величине $O(n \log n)$). Этот алгоритм не только обладает хорошими асимптотическими свойствами, но и практически пригоден для кодов с умеренной длиной.

Для простоты в основном будем рассматривать случай, когда $2t$ равно степени двух. Пусть длина $2t = 2^p$. Рекуррентный алгоритм сводит задачу длины 2^p к двум задачам длины 2^{p-1}, которые сочленяются с помощью алгоритма 2^{p-1}-точечной свертки. Каждая из этих двух подзадач в свою очередь расщепляется на две подзадачи. Это продолжается до тех пор, пока не получается задача, состоящая из одной итерации. Если каждая подзадача аналогична исходной задаче, то алгоритм допускает рекуррентную реализацию.

В случае, когда $2t$ не является степенью двух, имеется несколько способов модификации процедуры. Один из них состоит в представлении числа $2t$ в виде произведения простых множителей. Если p_l представляет собой l-й множитель, то на l-м уровне задача подразделяется на p_l подзадач вместо двух. В остальном алгоритм не меняется.

Второй способ состоит в замене числа итераций наименьшей степенью двух, превосходящей число $2t$, и разбиении задачи на каждом уровне пополам. В каждый момент обращения к процедуре проверяется, превысил ли счетчик общего числа операций величину $2t$. Если превысил, то итерация не выполняется и алго-

ритм декодирования переходит к своему следующему шагу. В остальном алгоритм не изменяется.

Формулировка шага рекуррентной процедуры становится очевидной, если заметить, что описанные в предыдущем параграфе вычисления величин Δ_r и $\Lambda^{(r,\,k\tau)}$ по величине $S^{(k\tau)}(x)$ в точности совпадают с процедурой теоремы 7.4.1 с заменой $2t$ на τ, $\Lambda^{(r)}(x)$ на $\Lambda^{(r,\,k\tau)}(x)$ и вектора синдромных многочленов $S(x)$ на $S^{(k\tau)}(x)$. Следовательно, для вычисления $\Lambda^{(r,\,k\tau)}(x)$ можно воспользоваться тем же алгоритмом, что и для вычисления $\Lambda^{(2t)}(x)$.

Этот рекуррентный алгоритм, названный *процедурой БерМес*, приведен на рис. 11.8. Процедура использует самое себя. Она содержит стековую память [1]), в которой при переходе с одного уровня на другой временно запоминаются данные одного уровня. Поведение алгоритма на следующем уровне ничем не отличается от его поведения на предыдущем уровне, но в стеке записываются другие данные. В конце концов задача сводится только к одной итерации, и ее уже нельзя разделить пополам. Тогда процедура БерМес состоит просто в выполнении этой итерации и алгоритм возвращается в надлежащую точку.

Каждый раз при обращении процедуры БерМес к самой себе необходимо запоминание локальных переменных в стеке. Если в начале алгоритма решение задачи требует 2^ρ итераций, то глубина рекурсии равна ρ: имеется ρ уровней рекуррентного алгоритма. Объем стека должен быть достаточно велик для того, чтобы содержать все временные множества переменных.

Алгоритм проходит l-й уровень 2^ρ раз: нулевой уровень — один раз, первый уровень — два раза и т. д.; последний уровень проходится 2^ρ раз. Выполняемое на l-м уровне вычисление $S(x) \leftarrow \Lambda'(x)\,S(x)$ содержит четыре умножения многочленов. Степень многочленов $\Lambda'(x)$ равна примерно $2^{\rho-l-1}-1$, и они умножаются на многочлены степени $2^{\rho-l+1}-1$. Необходимо правильно вычислить только $2^{\rho-l}$ коэффициентов произведения многочленов, а именно коэффициенты, стоящие при степенях переменной от $x^{2^{\rho-l}}$ до $x^{2^{\rho-l+1}-1}$, так как только эти коэффициенты входят в вектор $S(x)$, используемый при втором обращении к процедуре БерМес. На l-м уровне вычисляется также матричное произведение $\Lambda'(x)\,\Lambda(x)$, которое содержит произведения многочленов степени $2^{\rho-l-1}$ каждый.

Число умножений в процедуре БерМес почти полностью определяется свертками, так как итерации алгоритма Берлекэмпа —

---

[1]) Стековая память (push-down stack) — устройство памяти, считывание данных из которой начинается с вершины и производится в порядке, обратном порядку записи («пришел последним — обслуживается первым»). — *Прим. перев.*

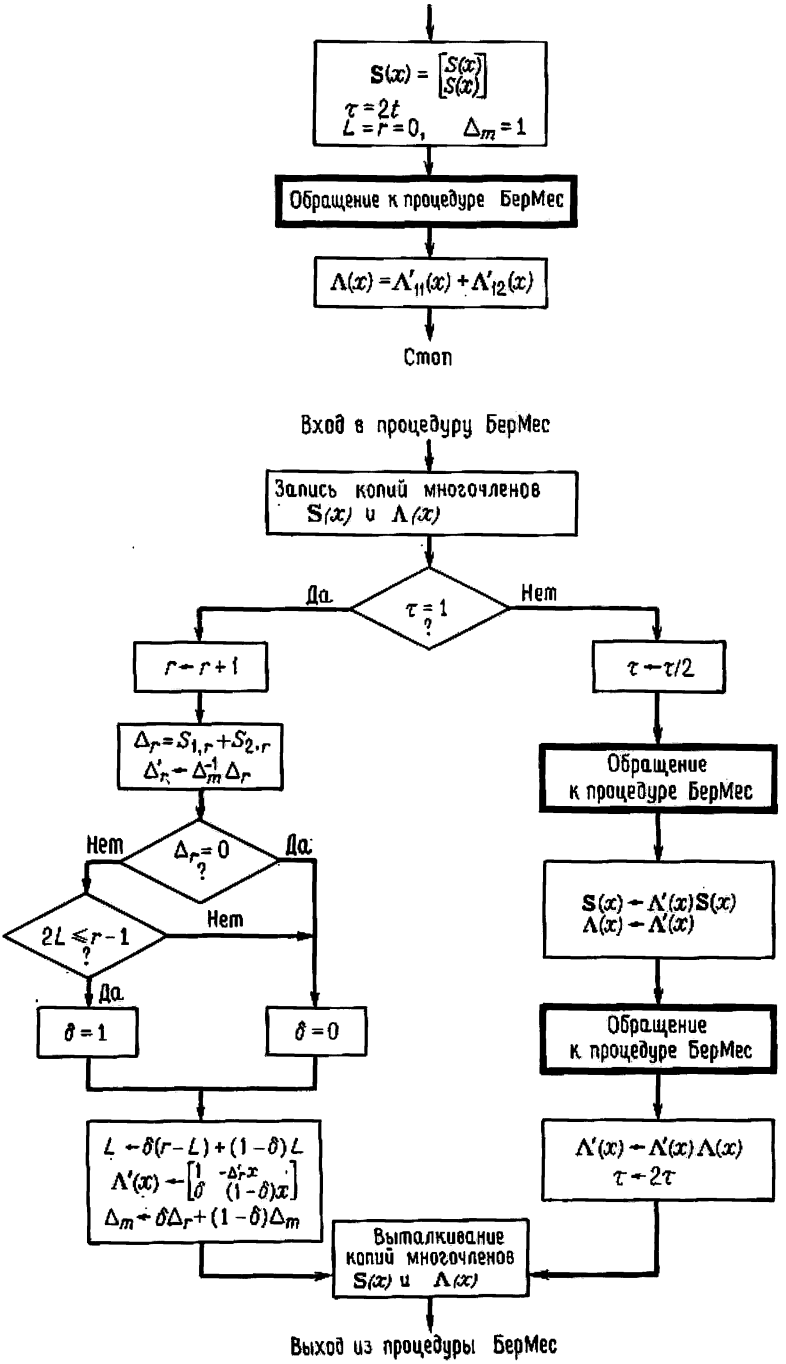


Рис. 11.8. Процедура БерМес.

Уровень	Число обращений	$\deg \Lambda(x)$	$\deg S(x)$ $\ln$	Длина свертки	Число вычислений свертки $\Lambda'(x)S(x)$	Число вычислений свертки $\Lambda'(x)\Lambda(x)$	Число умножений
1	1	511	2041	1260	8	8	355 680
2	2	255	1023	630	2×8	2×8	237 120
3	4	127	511	315	4×8	4×8	158 080
4	8	63	255	140	8×8	8×8	149 760
5	16	31	127	70	16×8	16×8	99 840
6	32	15	63	35	32×8	32×8	66 560
7	64	7	31	21	64×8	64×8	77 824
8	128	3	15	9	128×8	128×8	38 912
9	256	1	7	4	256×8	256×8	36 864
10	512	0	3	2		512×8	12 288
11	1024	0	1	–		–	–
							1 232 928

Рис. 11.9. Подсчет числа умножений в процедуре БерМес.

Месси фактически выполняются только на конечном шаге, на котором все подалгоритмы алгоритма содержат по одной итерации и только по одному умножению (на Δ_m^{-1}), которым при подсчете общего числа умножений можно пренебречь. Вычислительная сложность процедуры БерМес определяется непосредственно вычислительной сложностью алгоритмов свертки. Успех ее использования предполагает наличие хорошего набора алгоритмов свертки.

На рис. 11.9 приведены результаты подсчета числа умножений в рекуррентном алгоритме декодирования применительно к тому же (4096, 2048)-коду Рида—Соломона, который был рассмотрен в предыдущем параграфе. Для вычисления циклических сверток в обеих половинах использовались метод перекрытия с накоплением и одна из коротких сверток, приведенных на рис. 11.5. Выписанные на рис. 11.9 детали расчетов являются убедительным свидетельством сложности структуры рассматриваемого алгоритма. Эта структура, однако, позволяет разумно использовать вычислительные ресурсы при программной и аппаратурной реализации отдельных фрагментов. По мере необходимости она вызывает алгоритмы свертки из библиотеки таких алгоритмов.

В общем случае алгоритм Берлекэмпа—Месси, содержащий $2t$ итераций, имеет измеряемую числом умножений вычислительную сложность порядка $(\log_2 2t)\, C(t)$, где $C(t)$ — число умножений, необходимое для вычисления свертки двух многочленов степени t. Это объясняется тем, что к l-му уровню алгоритм обращается 2^l раз, причем длина выполняемой на l-м уровне свертки равна $2^{-l}t$. Сложность вычисления 2^l сверток длины $2^{-l}t$ каждая

принимает наибольшее значение при l, равном нулю для всех $\log_2 2t$ значений.

Граница асимптотической сложности алгоритма зависит от асимптотической сложности вычисления свертки в полях Галуа. Напомним, что в алгоритме Винограда вычисления свертки требуется по возможности наличие простых делителей над полем $GF(p)$. Для минимизации числа умножений показатели степени этих делителей надо выбирать малыми. Но если n велико, то может не оказаться достаточного числа многочленов малой степени и будет необходимо воспользоваться многочленами степени порядка $\log n$. Это приведет к замене свертки длины n некоторым количеством сверток, наибольшая длина которых имеет порядок $\log n$. Этот же процесс приведет к замене свертки длины $\log n$ некоторым количеством сверток меньшей длины, самая длинная из которых имеет порядок длины, равный $\log(2 \log n)$. Формализуя это рассуждение, можно сказать, что асимптотическая сложность вычисления свертки в поле Галуа равна величине $O(n2^{\log^* n})$, где $\log^* n$ — число, показывающее, сколько раз надо последовательно проинтегрировать логарифм по основанию 2, начиная с n, для того, чтобы получить число, не превосходящее единицы. Таким образом

$$\log_2(\log_2(\ldots(\log_2 n))) \leqslant 1,$$

и число шагов в этом итеративном процессе равно $\log^* n$. Величина $2^{\log^* n}$ стремится к бесконечности медленнее, чем итерированный любое число раз логарифм.

Лучшая известная граница числа $C(n)$ имеет вид $O(n2^{\log^* n})$, и, следовательно, сложность рекуррентного алгоритма Берлекэмпа—Месси оценивается величиной $O(n2^{\log^* n} \log n)$. Таким образом, сложность рекуррентного алгоритма Берлекэмпа—Месси больше, чем $O(n \log n)$, но очень близка к этой границе.

11.8. УСКОРЕННОЕ ДЕКОДИРОВАНИЕ КОДОВ БЧХ

Сейчас мы рассмотрим применение разработанных в предыдущих параграфах методов для построения ускоренного декодера. Достаточно рассмотреть только примитивные [1]) коды Рида—Соломона. Коды БЧХ и непримитивные коды Рида—Соломона декодируются таким же образом. Исправляющий t ошибок примитивный код Рида—Соломона длины $n = q - 1$ представляет собой

[1]) Напомним, что автор называет примитивными кодами Рида—Соломона коды над $GF(q^m)$ длины $n = q^m - 1$, а непримитивными — коды, длина которых делит это n. — *Прим. перев.*

множество всех кодовых слов длины n над $GF(q)$, для которых $2t$ последовательных компонент спектра равны нулю.

Работа декодера основывается на преобразовании Фурье принятого слова $\mathbf{v}$. В типичных случаях сложность преобразования Фурье имеет порядок $n \log_2 n$.

Равенство нулю $2t$ последовательных компонент спектра $\mathbf{C}$ определяет $2t$ компонент синдрома $S_j = E_j = V_j$, $j = 1, \ldots, 2t$. Многочлен локаторов ошибок

$$\Lambda(x) = \prod_{k=1}^{\nu} (1 - x\alpha^{i_k}),$$

где $\nu \leqslant t$ — число действительно произошедших ошибок, удовлетворяет равенствам

$$\sum_{j=0}^{t} \Lambda_j E_{k-j} = 0.$$

Эта свертка задает систему из n уравнений относительно $n - t$ неизвестных, из которых t составляют коэффициенты многочлена $\Lambda(x)$, а $n - 2t$ являются компонентами вектора $\mathbf{E}$. Из этих n уравнений t уравнений связывают только известные компоненты вектора $\mathbf{E}$ с неизвестными коэффициентами многочлена $\Lambda(x)$. Используя алгоритмы свертки сложности $n \log_2 n$ и описанный в предыдущем параграфе рекуррентный алгоритм Берлекэмпа—Месси, можно разрешить эти уравнения относительно неизвестных компонент $\boldsymbol{\Lambda}$ со сложностью, не превосходящей $O(n \log_2^2 n)$. Остальные компоненты вектора $\mathbf{E}$ можно рекуррентно вычислить по полученному $\boldsymbol{\Lambda}$ и известным компонентам вектора $\mathbf{E}$ в соответствии с равенством

$$E_j = -\sum_{k=1}^{t} \Lambda_k E_{i-k}.$$

Вычислив таким образом E_j для всех j, находим компоненты C_j спектра кодового слова согласно равенствам $C_j = V_j - E_j$. Обратное преобразование Фурье завершает декодирование.

Для вычисления E_j, $j = 2t + 1, \ldots, n$, требуется $n(n - 2t)$ умножений, так что сложность этих вычислений имеет порядок n^2. В оставшейся части этого параграфа мы покажем, как свести вычислительную сложность этого шага к величине, не превосходящей $O(n \log_2 n)$.

В варианте Месси алгоритма Берлекэмпа—Месси спектр $\mathbf{E}$ вектора ошибок рассматривается не только на интервале длины n, но и периодически продолжается вне этого интервала. Соотношение между многочленами $E(x)$ и $\Lambda(x)$ имеет вид

$$E(x)\,\Lambda(x) = 0 \pmod{x^n - 1}.$$

Преобразование Фурье этой циклической свертки приводит к равенствам

$$e_i\lambda_i = 0, \quad i = 0, \ldots, n-1.$$

Во временно́й области эти равенства приводят к неопределенностям, так как решения записываются в виде $e_i = 0/\lambda_i$ и $\lambda_i = 0$, когда $e_i \neq 0$. В таком виде эти равенства решать нельзя.

Берлекэмп подходит к алгоритму декодирования несколько иначе. С точки зрения Берлекэмпа, выход регистра сдвига является периодическим только после начального перехода. В начальный момент времени регистр пуст и в него вводится известный многочлен $\Omega(x)$. Уравнения записываются в виде

$$E_j = -\sum_{k=1}^{t} \Lambda_k E_{j-k} + \Omega_j,$$

и возмущения Ω_j, равные нулю при $j > t$, переводят регистр сдвига из начального состояния в описанное выше периодическое состояние.

Спектральный многочлен $E(x)$ для вектора ошибок удовлетворяет уравнению

$$E(x)\,\Lambda(x) = \Omega(x).$$

Многочлен $\Omega(x)$ можно вычислить с помощью рекуррентного алгоритма без увеличения асимптотической сложности, после чего $E(x)$ можно вычислить с помощью деления многочленов. Но сложность последнего шага также растет как n^2. Так как выписанное выражение не является ни конечным, ни периодическим, то непосредственное применение метода преобразований невозможно. Можно, однако, воспользоваться для решения задачи алгоритмом Форни. По имеющемуся $\Lambda(x)$ вычислим

$$\Omega(x) = E(x)\,\Lambda(x) \pmod{x^{2t}}$$

и формальную производную $\Lambda'(x)$ многочлена $\Lambda(x)$. Затем, применив к многочленам $\Lambda(x)$, $\Omega(x)$ и $\Lambda'(x)$ обратное преобразование Фурье, вычислим векторы с компонентами λ_i, ω_i и λ'_i. Тогда, если λ_i равны нулю, решение дается (см. задачу 11.5) равенствами

$$e_i = -\omega_i/\lambda'_i.$$

Так как алгоритм Форни можно реализовать с помощью трех алгоритмов преобразования Фурье и алгоритма свертки, то получаем сложность $O(n \log_2 n)$.

11.9. СВЕРТКА В СУРРОГАТНЫХ ПОЛЯХ

Вычисления в некотором поле можно производить, используя другое поле. Бывают случаи, когда необходимые при обработке сигналов задачи существенно отличаются от имеющегося набора стандартных вычислительных модулей и появляется необходимость подгонки данного вида вычислений к структуре другого вида. Например, может возникнуть необходимость проведения вычислений в поле Галуа с помощью поля комплексных чисел в качестве суррогатного поля.

В простом поле $GF(p)$ циклическую свертку

$$c_i = \sum_{k=0}^{n-1} a_k b_{((i-k))}, \quad i = 0, \ldots, n-1,$$

можно заменить сверткой в кольце целых чисел; необходимо только позаботиться о выполнении вычислений по модулю p. Вид уравнения свертки при этом сохраняется, но арифметические операции выполняются в обычной целочисленной арифметике. Затем для получения свертки в поле $GF(p)$ найденные при вычислении целочисленной свертки компоненты c_i надо привести по модулю p; это сводит задачу к вычислению только целочисленных сверток. Кольцо целых чисел можно далее вложить в подходящее поле, например комплексное. После этого целочисленная свертка может быть вычислена с помощью любого алгоритма свертки в поле комплексных чисел, например с помощью дискретного преобразования Фурье и теоремы о свертке. Практическое вычисление преобразования Фурье в поле комплексных чисел реализуется на словах конечной длины, что вносит в вычисления ошибку округления. Однако нетрудно подобрать такую длину преобразования, чтобы ошибка округления в компонентах c_i всегда была меньше 0,5. Тогда округление каждой вычисленной компоненты в ближайшее целое число даст правильное значение целочисленной свертки. Переход в каждом целом числе к вычету по модулю p даст затем правильное значение свертки в $GF(p)$.

Может оказаться невозможным оправдать выбор поля комплексных чисел в качестве суррогатного поля простым подсчетом числа умножений. Однако для поля комплексных чисел имеется доступный набор устройств вычисления свертки, и выбор поля комплексных чисел в качестве суррогатного поля можно оправдать возможностью выполнения данной вычислительной задачи на имеющемся устройстве.

Предположим, далее, что надо вычислять свертку сигналов над полем $GF(q)$, где q уже не является простым числом. Тогда каждое отдельное произведение элементов поля можно рассматривать как свертку многочленов по модулю некоторого неприводимого многочлена $p(x)$. Вычисление остатка по модулю этого не-

приводимого многочлена производится тогда, когда все свертки вычислены. Исходная свертка при этом превращается в двумерную свертку.

Пусть **a** и **b** — векторы над $GF(q)$. Компоненты этих векторов задаются многочленами

$$a_i = \sum_{l=0}^{m-1} a_{il} z^l \quad \text{и} \quad b_i = \sum_{l=0}^{m-1} b_{il} z^l$$

над простым полем $GF(p)$, где $q = p^m$, а a_{il} и b_{il} представляют собой неотрицательные целые числа, меньшие p. Линейная свертка векторов **a** и **b** записывается в виде

$$c_i = \sum_{k=0}^{n-1} a_k b_{i-k} = \sum_{k=0}^{n-1} \sum_{l=0}^{m-1} \sum_{l'=0}^{m-1} a_{kl} b_{(i-k)\,l'} z^{l+l'} \pmod{p} \pmod{p(x)},$$

где p — характеристика поля, а $p(x)$ — некоторый простой многочлен степени m. Введем величины

$$c_{ii'} = \sum_{k=0}^{n-1} \sum_{k'=0}^{m-1} a_{kk'} b_{(i-k)\,(i'-k')}, \qquad \begin{matrix} i = 0, \ldots, n-1, \\ i' = 0, \ldots, 2m-1, \end{matrix}$$

и будем рассматривать их как двумерные свертки целых чисел; каждое число в этой двумерной таблице лежит в интервале $[0, p-1]$. Эта свертка может быть вычислена в любом подходящем суррогатном поле, например в поле вещественных чисел или в поле комплексных чисел; тогда

$$c_i(z) = \sum_{i'=0}^{2m-1} c_{ii'} z^{i'} \pmod{p} \pmod{p(x)}.$$

Вычисление этого остатка производится в последнюю очередь; сначала находится вычет каждого целого числа по модулю p, а затем вычет многочлена $c_i(z)$ по модулю $p(x)$. Сложность вычисления вычетов пренебрежимо мала по сравнению со сложностью вычисления двумерной свертки.

Вместо поля комплексных чисел можно воспользоваться в качестве суррогатного поля любым конечным полем, даже с характеристикой, отличной от характеристики исходного поля. Для вычисления свертки двоичных последовательностей, длина n которых не превосходит половины простого числа Ферма $2^m + 1$ при $m = 2, 4, 8$ или 16, можно использовать поле $GF(2^m + 1)$. Последовательность над $GF(2)$ можно рассматривать как последовательность целых чисел, принимающих значения, равные только нулю и единице. Длина линейной свертки двух целочисленных последовательностей длины n может быть больше n, но всегда меньше $2^m + 1$. Следовательно, линейная свертка в $GF(2)$ может быть вычислена как циклическая свертка в поле $GF(2^m +$

$+ 1$) с последующим приведением по модулю 2. Циклическая свертка в $GF(2^m + 1)$ может быть вычислена с помощью любого алгоритма быстрой свертки; один из методов состоит в применении быстрого преобразования Фурье и теоремы о свертке.

Например, для непосредственного вычисления свертки в поле $GF(2)$ двух последовательностей длины 2^{15} требуется примерно 2^{31} битовых операций. Вместо этого можно воспользоваться сверткой в поле $GF(2^{16} + 1)$. В этом поле для вычисления преобразования Фурье надо примерно 2^{18} умножений и 2^{20} сложений; для каждого умножения необходимо примерно $(17)^2$ битовых операций, а для каждого сложения — примерно 17 битовых операций. Таким образом, свертка может быть вычислена примерно за $3 \times (17^2 \times 2^{18}) \approx 3 \times 2^{26}$ битовых операций.

Теперь предположим, что нам надо свернуть две последовательности в $GF(2^m)$. Как и ранее, можно представить элементы поля многочленами над $GF(2)$. Если отложить операцию вычисления остатка по модулю $p(x)$ до завершения всех других вычислений, то исходная свертка превратится в двумерную свертку. Эта задача может быть вложена в суррогатное поле $GF(2^{m'} + 1)$, если только длина свертки целых чисел не превышает $2^{m'} + 1$, т. е. если $m \cdot n < 2^{m'} + 1$.

Например, пусть надо вычислить линейную свертку двух последовательностей длины 8192 над $GF(2^8)$. Так как 8 (8192) меньше, чем $2^{16} + 1$, то в качестве суррогатного поля можно воспользоваться полем $GF(2^{16} + 1)$. Двумерную (8×8192)-точечную линейную свертку можно вычислить через двумерную $(16 \times 16\,384)$-точечную циклическую свертку без потери членов. Эта $(16 \times 16\,384)$-точечная циклическая свертка может быть вычислена с помощью методов быстрого преобразования Фурье. 16-точечное преобразование вообще не содержит умножений, а содержит только циклические сдвиги. 16 384-точечное преобразование, как уже обсуждалось выше, может быть разбито с помощью алгоритма Кули—Тьюки БПФ на более короткие свертки. Для того чтобы воспользоваться теоремой о свертке, необходимо выполнить три преобразования, каждое из которых содержит 2048 умножений. В частотной области также необходимо выполнить $16 \times 16\,384$ умножений. Таким образом, для вычисления 8192-точечной свертки над $GF(2^8)$ необходимо выполнить 268 288 умножений в поле $GF(2^{16} + 1)$.

ЗАДАЧИ

11.1. Построить алгоритм быстрой свертки для кодирования (7, 3)-кода Рида—Соломона, корнями порождающего многочлена которого служат α^0, α^1, α^2 и α^3.

11.2. Построить алгоритм быстрой свертки для кодирования (15, 11)-кода Рида—Соломона с порождающим многочленом $g(x) = x^4 + \alpha^{13}x^3 + \alpha^6x^2 + \alpha^3 x + \alpha^{10}$.

11.3.а. Построить алгоритм Винограда трехточечного БПФ в поле GF (16).

б. Используя гнездовой метод Винограда, построить 15-точечный БПФ-алгоритм Винограда над GF (16) на основе трехточечного и пятиточечного БПФ-алгоритмов Винограда.

11.4. Выписать уравнения переиндексации для 35-точечного БПФ-алгоритма Кули—Тьюки.

11.5. Алгоритм Форни, согласно которому $e_i = -\omega_i/\lambda_i'$), аналогичен равенству $E(x)\Lambda(x) = \Omega(x)$. Однако последнее выражение не является периодическим, и, следовательно, метод преобразования Фурье нельзя непосредственно использовать для вывода из него алгоритма Форни. Для вывода алгоритма Форни на основе преобразования Фурье надо рассуждать следующим образом. Выбрать такое n', чтобы в некотором расширении поля содержался элемент порядка $n'n$. Показать, что выражение $E(x)\Lambda(x) = -(x^n - 1)\Omega(x)$ можно рассматривать как периодическое с периодом $n'n$. Использовать преобразование Фурье длины $n'n$ для завершения доказательства.

11.6. Для полей характеристики 2 описать процедуру построения 8-точечной циклической свертки, содержащей не более 28 умножений. Описание должно быть полным, но не содержать непосредственных вычислений.

11.7. Пусть требуется написать программу вычисления 31-точечной циклической свертки в поле GF (32) или в некотором его расширении.

а. Оценить сложность программы, использующей 31-точечное ДПФ и теорему о свертке.

б. Написать программу, использующую 75-точечное быстрое преобразование Фурье в поле $GF(2^{20})$. Построить БПФ из трехточечного и пятиточечного алгоритмов Винограда быстрого преобразования Фурье. Оценить сложность подпрограммы, вычисляющей свертку.

11.8. Для систематического кодирования 8-битовых байтов используется $(256, 256-2l)$-код Рида—Соломона над $GF(2^8 + 1)$. Когда проверочный символ принимает значение 2^8, он не может быть представлен восемью битами и заменяется на нуль, так что в момент кодирования кодовое слово уже содержит ошибку, которая может быть данным кодом исправлена. Чему равна скорость кода в двоичном представлении? Описать ухудшение кодовых характеристик при использовании в проверочных символах только восьми битов.

ЗАМЕЧАНИЯ

Материал данной главы основан на методах, разработанных в теории построения эффективных алгоритмов. В большинстве случаев эти методы развивались вне связи с теорией кодов, контролирующих ошибки. При этом рассматривались только вычисления в полях вещественных и комплексных чисел, но не представляет труда перенести эти же идеи в поле Галуа, что и было сделано в данной главе.

Алгоритмы быстрого преобразования Фурье широко используются в обработке дискретных сигналов, начиная с известной работы Кули и Тьюки [1965]. Однако другой подход к БПФ был предложен в более ранних статьях Гуда [1960] и Томаса [1963]. Взаимосвязь между БПФ и сложностью декодирования обсуждалась Юстесеном [1976] и Сервейтом [1977] [1]).

---

[1]) Аналогичную задачу рассматривал В. Б. Афанасьев; см. Афанасьев В. Б. Быстрое кодирование и обнаружение ошибок кодом Рида—Соломона. — В кн.: Третий международный симпозиум по теории информации. Тезисы докладов. Часть II. — М. Таллин: URSI АН СССР и АН ЭССР, 1973, с. 13—17. — *Прим. перев.*

Основанные на прямых и грубых методах быстрые алгоритмы свертки впервые были построены Агарвалом и Кули [1977]. Виноград [1978] предложил описанный в данной главе общий метод построения, а также доказал, что в полях вещественных и комплексных чисел не существует лучших алгоритмов свертки. Он почти не обращался к полям Галуа, так как в этих полях умножения легко маскируются под сложения. Использование китайской теоремы об остатках для целых чисел для разбиения длинных сверток на короткие предложили Агарвал и Кули [1977]. Алгоритм Винограда БПФ был опубликован в 1978 г. Наше описание не следует оригиналу и связывает его с другими методами обработки сигналов. Алгоритм БПФ Винограда и другие эффективные алгоритмы обсуждал также Нуссбаумер [1981].

Применение БПФ-алгоритма Винограда для декодирования кодов Рида—Соломона рассматривали Миллер, Труонг и Рид [1980].

Ускоренный и рекуррентный алгоритмы Берлекэмпа—Месси публикуются впервые; они были анонсированы в работе Блейхута [1981]. Использование в алгоритмах свертки суррогатных полей предложили Препарата и Сервейт [1977].

ГЛАВА 12
СВЕРТОЧНЫЕ КОДЫ

В современных системах связи довольно часто приходится предусматривать передачу данных с очень большими скоростями — иногда много миллионов битов в секунду. Для защиты таких систем от ошибок нередко используются блоковые коды. Поток данных делится на блоки по k информационных символов, и каждый блок кодируется n символами кодового слова. Кодовые слова для последовательных k-символьных блоков никоим образом не связываются кодером.

При другой схеме кодирования поток данных разбивается на гораздо меньшие блоки длины k_0, которые мы будем называть кадрами информационных символов. Эти кадры информационных символов обычно включают лишь несколько символов. Кадры информационных символов кодируются кадрами кодового слова длины n_0 каждый. Однако вместо того, чтобы независимо кодировать отдельные кадры информационных символов в отдельные кадры кодового слова, кодирование каждого кадра информационных символов в отдельный кадр кодового слова производится с учетом предыдущих m кадров информационных символов. Поэтому процедура кодирования связывает между собой последовательные кадры кодовых слов.

Коды, получаемые таким образом, называются *древовидными кодами*. Наиболее важными древовидными кодами являются коды, известные под названием *сверточных кодов*. Сверточными кодами являются древовидные коды, которые обладают дополнительными свойствами линейности и постоянства во времени. Вначале мы рассмотрим общий класс древовидных кодов, в основном концентрируя свое внимание на изучении сверточных кодов как частного случая древовидных кодов.

12.1. ДРЕВОВИДНЫЕ И РЕШЕТЧАТЫЕ КОДЫ

Изучение древовидных кодов начнем с рассмотрения кодера, представленного на рис. 12.1 в виде регистра сдвига. Многие основные определения могут быть введены с использованием этой

схемы. Информационная последовательность вводится в кодер, начиная с нулевого момента времени и до бесконечности. Поток входящих информационных символов разбивается на сегменты, которые содержат по k_0 символов и называются *кадрами информационных символов*. Кадр информационных символов может, в частности, состоять из единственного символа, что нередко имеет место на практике. В кодере может храниться m кадров. В течение каждого временно́го кадра в регистр сдвига вводится новый кадр информационных символов, а кадр информационных символов, дольше остальных хранившийся в нем, выводится из него и сбрасывается. В конце каждого временно́го кадра в кодере хранятся последние m из поступивших в него кадров (всего mk_0 информационных символов). В начале каждого временно́го кадра кодер по введенному кадру информационных символов и m хранящимся в нем кадрам вычисляет один кадр кодового слова, имеющий длину n_0 символов. Этот кадр кодового слова выводится из кодера, как только следующий кадр информационных символов выводится в него. Следовательно, каждым k_0 информационным символам соответствует передача по каналу n_0 кодовых символов.

Бесконечное множество всех бесконечно длинных кодовых слов, получаемых при поступлении в этот кодер всех возможных входных последовательностей, называется *древовидным* (n_0, k_0)*-кодом*. Скорость R этого древовидного кода определяется отношением

$$R = k_0/n_0.$$

Важной характеристикой сверточного кода является величина $\nu = mk_0$. Она называется *длиной кодового ограничения*. Это нестрогое определение длины кодового ограничения достаточно для

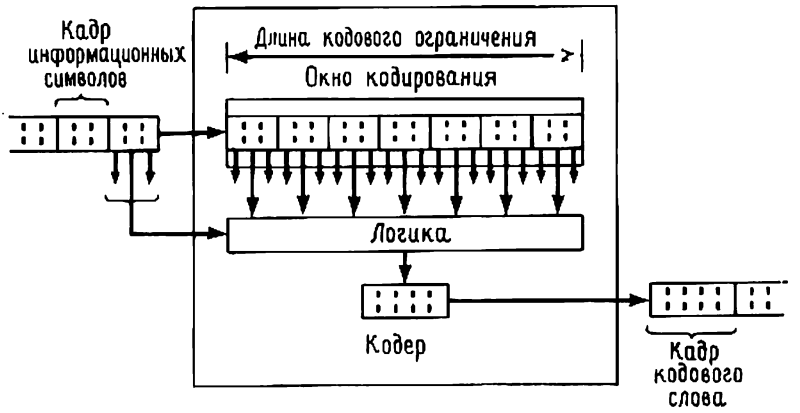


Рис. 12.1. Кодер в виде регистра сдвига.

многих целей (хотя в некоторых случаях необходимо более строгое определение). Формальное определение будет дано в следующем параграфе. На рис. 12.1 изображен кодер, у которого $k_0 = 3$, $n_0 = 5$ и $\nu = 21$.

В древовидном коде используются и несколько других мер длины. Положим $k = (m + 1)\, k_0$. Это k непосредственно связано с длиной кодового ограничения. Назовем ее *информационной длиной слова* сверточного кода. Соответствующая ей мера кодовых последовательностей называется *кодовой длиной блока* n. Она дается формулой

$$n = (m + 1)\, n_0 = kn_0/k_0.$$

Кодовая длина блока кодера на рис. 12.1 равна 40. Кодовая длина блока — это длина кодового слова, на которой сохраняется влияние одного кадра информационных символов. Из соображений удобства реализации на практике значения n_0 и k_0 для древовидных кодов выбираются равными небольшим целым числам; в типичном случае k_0 равно единице. Это означает, что выбор скорости кода ограничен. Невозможно построить практический древовидный код со скоростью, очень близкой к единице, как это обычно делается для блоковых кодов (таких, как коды Рида—Соломона).

Дадим формальное определение древовидного кода.

Определение 12.1.1. *Древовидный* (n_0, k_0)*-код* — это отображение на себя множества полубесконечных последовательностей элементов из $GF(q)$, такое, что если для любого M первые Mk_0 компонент двух полубесконечных последовательностей совпадают, то первые Mn_0 компонент отображений этих последовательностей тоже совпадают.

Древовидный код лучше всего можно представить себе, обратившись к кодеру, изображенному на рис. 12.1. Древовидный код характеризуется длиной кодового ограничения (быть может, бесконечной) и скоростью.

Частные случаи древовидных кодов получаются различными комбинациями следующих четырех свойств. Эти свойства полезно проследить на показанном на рис. 12.1 кодере.

Конечность длины кодового ограничения. Длина кодового ограничения может быть конечной или бесконечной. Практически древовидные коды всегда имеют конечную длину кодового ограничения. Однако в теоретических исследованиях иногда полезны коды с бесконечной длиной кодового ограничения. Древовидный (n_0, k_0)-код с конечной длиной кодового ограничения ν, длиной слова $k = \nu + k_0$ и кодовой длиной блока n называется также *решетчатым* (n, k)*-кодом*.

Постоянство во времени. Если две различные входные последовательности совпадают во всем, но с временным сдвигом на целое число кадров, то соответствующие им кодовые последовательности также совпадают во всем, но с временным сдвигом на то же самое целое число кадров.

Линейность. Кодовая последовательность любой линейной комбинации двух информационных последовательностей совпадает с такой же линейной комбинацией кодовых последовательностей этих двух информационных последовательностей. Иначе говоря, если d_1 и d_2 являются двумя информационными последовательностями с кодовыми последовательностями $G(d_1)$ и $G(d_2)$, то $ad_1 + bd_2$ соответствует кодовая последовательность

$$G(ad_1 + bd_2) = aG(d_1) + bG(d_2).$$

Систематичность. Систематическим древовидным кодом называется код, в котором каждый кадр информационных символов составляет первые k_0 символов первого из тех кадров кодовой последовательности, на которые влияет данный кадр информационных символов.

Определение 12.1.2. Линейный постоянный во времени древовидный (n_0, k_0)-код, имеющий конечную длину слова $k = (m+1)k_0$, называется *сверточным* (n, k)-*кодом*. Сверточный (n, k)-код, удовлетворяющий условию систематичности, называется *систематическим сверточным* (n, k)-*кодом*.

Заметим, что мы можем называть один и тот же код древовидным (n_0, k_0)-кодом или сверточным (n, k)-кодом. На практике k значительно больше k_0, и поэтому недоразумений не возникает.

Определение 12.1.3. Постоянный во времени древовидный (n_0, k_0)-код, имеющий конечную информационную длину слова k, называется *скользящим блоковым* (n, k)-*кодом*. Следовательно, линейный скользящий блоковый код является сверточным кодом.

На рис. 12.2 графически иллюстрируются связи между различными классами древовидных кодов. Существуют и другие возможности, но наиболее интересными являются эти случаи. Примеры кодеров для двух различных сверточных кодов показаны на рис. 12.3; в обоих случаях $n_0 = 2$, $k_0 = 1$. Первый из них служит кодером для систематического двоичного сверточного (12,6)-кода с длиной кодового ограничения, равной 5. Он обладает всеми указанными выше свойствами. Второй является кодером для несистематического двоичного сверточного (6,3)-кода с длиной кодового ограничения 2. В обоих случаях входные символы преобразуются двумя фильтрами, один из которых образуется верхними отводами, а другой — нижними. Символы с выходов этих двух фильтров попеременно во времени считываются и подаются

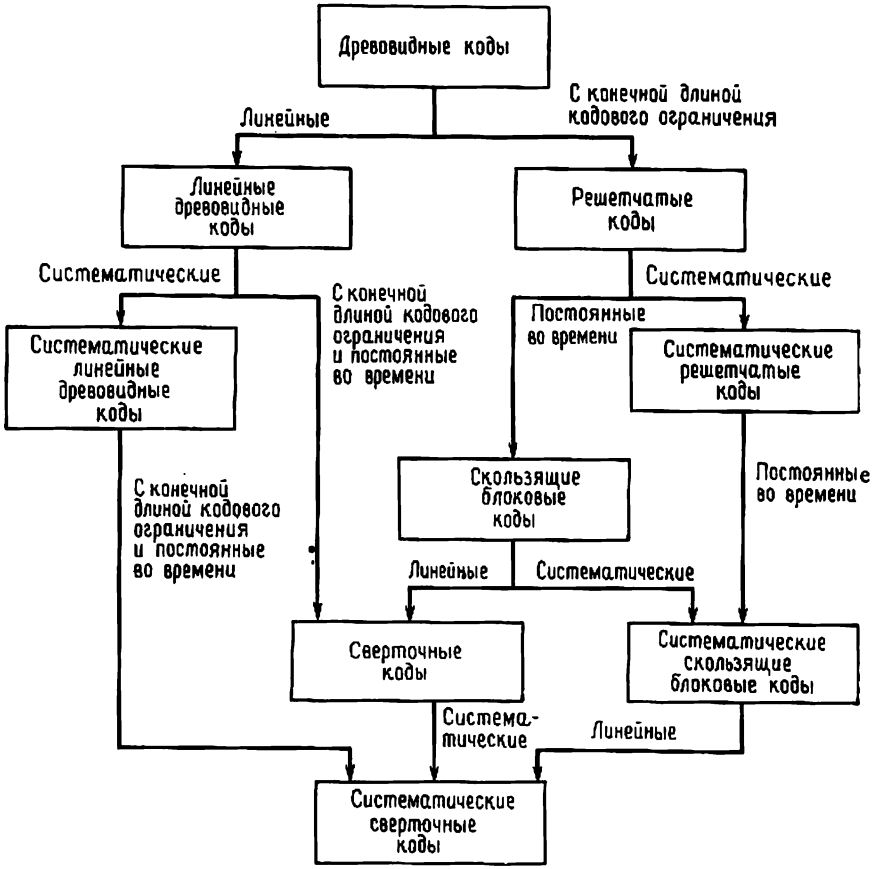


Рис. 12.2. Классификация древовидных кодов.

в буфер по одному символу в единицу времени с каждого выхода; их считывание из буфера производится вдвое чаще, чем поступают входные символы.

Сверточные и другие решетчатые коды удобно описывать специальным графом, называемым *решеткой*; отсюда название — решетчатые коды. Решеткой называется граф, узлы которого находятся в прямоугольной координатной сетке, полубесконечной справа; число узлов в каждом столбце конечно. Конфигурация ребер, соединяющих узлы каждого столбца с узлами столбца справа, одинакова для всех столбцов. Узлы, которые не могут быть достигнуты при движении вправо из верхнего левого узла, обычно не указываются.

Типичная решетка для двоичного кодового алфавита представлена на рис. 12.4. Эту решетку можно использовать для описания второго кодера, изображенного на рис. 12.3. Ее маркировка на рис. 12.5 соответствует этому кодеру. Узлы в каждом столбце решетки представляют q^{ν} состояний, в которых может находиться регистр сдвига. Каждый последующий столбец представляет собой набор состояний в следующий момент времени.

Поступление на вход нового кадра приводит к изменению состояния регистра сдвига, соответствующему ребру, которое ведет к следующему узлу. В нашем примере каждое ребро помечено двумя двоичными символами, передаваемыми в канал при переходе в следующее состояние регистра сдвига. В том простом примере, который мы рассматриваем, ведущая из произвольного узла верхняя прямая соответствует нулевому входному двоичному символу, а нижняя — единичному. В общем же случае каждое ребро помечается k_0 входными символами, которые ему соответствуют.

Маркированная решетка описывает сверточный код в том смысле, что все пути слева направо по решетке обозначают кодовые слова. Маркировка ребер одинакова для каждого сегмента и линейна в том смысле, что линейная комбинация маркировок любого множества ребер является маркировкой некоторого ребра.

Решетка может быть маркирована и при более слабых ограничениях. Если маркировка не обладает свойством линейности, то, как мы уже указывали, код называется скользящим блоковым

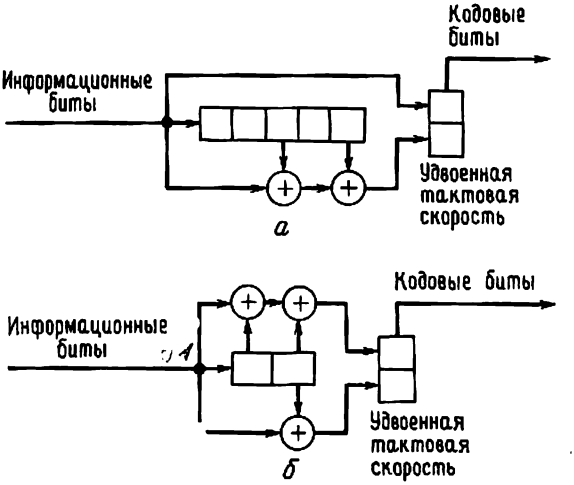


Рис. 12.3. Примеры сверточных кодеров. *а* — кодер для двоичного сверточного (12, 6)-кода; *б* — кодер для двоичного сверточного (6. 3)-кода.

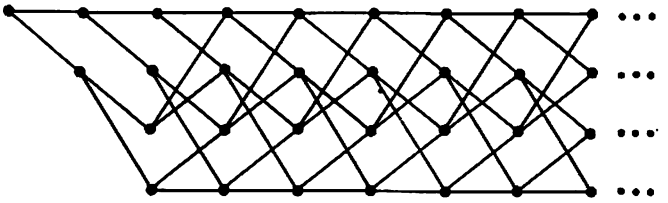


Рис. 12.4. Решетка сверточного кода

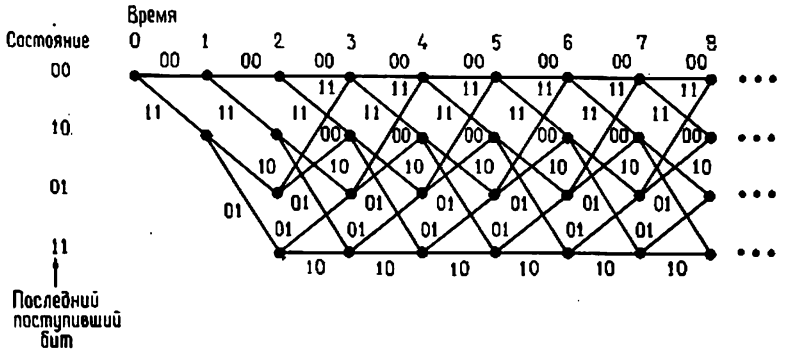


Рис. 12.5. Решетчатая диаграмма сверточного (6, 3)-кода.

кодом. Если маркировка меняется от кадра к кадру, то такой код известен под общим названием решетчатого кода. Наконец, если число состояний в следующих друг за другом временны́х кадрах продолжает неограниченно расти, то такой код называется общим древовидным кодом.

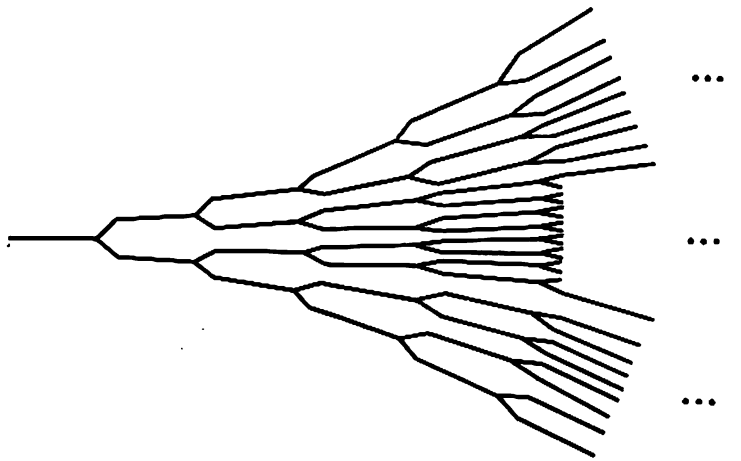


Рис. 12.6. Дерево для двоичного древовидного кода со скоростью 1/2.

Решетчатый код может быть также представлен особым графом, показанным на рис. 12.6 и называемым *деревом*. В этом графе число узлов и ветвей неограниченно увеличивается при росте дерева вправо; этим и объясняется название древовидных кодов. Каждый узел дерева — это состояние, соответствующее всем поступившим в него информационным символам, начиная с нулевого момента времени. Для кодов с бесконечной длиной кодового ограничения или даже умеренно большой длиной кодового ограничения дерево — это именно тот граф, который их описывает. Кодовые слова соответствуют путям по дереву. Для кодов с малой длиной кодового ограничения, однако, удобнее использовать решетку. Как мы уже видели, код полностью описывается последними ν поступившими в кодер символами, и их достаточно для определения состояния.

12.2. ОПИСАНИЕ СВЕРТОЧНЫХ КОДОВ С ПОМОЩЬЮ МНОГОЧЛЕНОВ

Сверточный $((m+1)\,n_0, (m+1)\,k_0)$-код над $GF(q)$ с длиной кодового ограничения $\nu = mk_0$ можно генерировать с помощью наборов фильтров с конечным импульсным откликом (КИО-фильтров); каждый набор состоит из k_0 КИО-фильтров над $GF(q)$. В кодер поступает поток символов со скоростью k_0 символов в единицу времени, а из него выходит в канал поток символов со скоростью n_0 символов в единицу времени. На рис. 12.7 показан кодер для двоичного сверточного кода с $n_0 = 5$ и $k_0 = 1$. Такой кодер состоит из серии фильтров и выходного регулирующего буфера, который необходим для согласования выходной скорости со скоростью фильтров. На рис. 12.8 приведен аналогичный кодер для двоичного сверточного кода с $n_0 = 5$, $k_0 = 3$. Для согласования входной скорости со скоростью фильтров добавлен входной буфер.

Каждый КИО-фильтр может быть представлен многочленом степени не выше m. Если входной поток записать как многочлен (быть может, бесконечной длины), то работа фильтра может быть описана как умножение многочленов. В этом случае кодер сверточного кода может быть представлен множеством многочленов; поэтому и сам код может быть представлен посредством того же множества многочленов. Иначе говоря, код является множеством кодовых слов, которое порождается данным множеством многочленов. Эти многочлены называются *порождающими многочленами* кода. Наибольшая степень порождающих многочленов равна m.

В отличие от блоковых кодов, которые описываются единственным порождающим многочленом, сверточный код требует для своего описания нескольких порождающих многочленов — в общем случае $k_0 n_0$ многочленов. Пусть $g_{ij}(x)$, $i = 1, \ldots, k_0$, $j = 1, \ldots, n_0$ — множество порождающих многочленов. Они могут

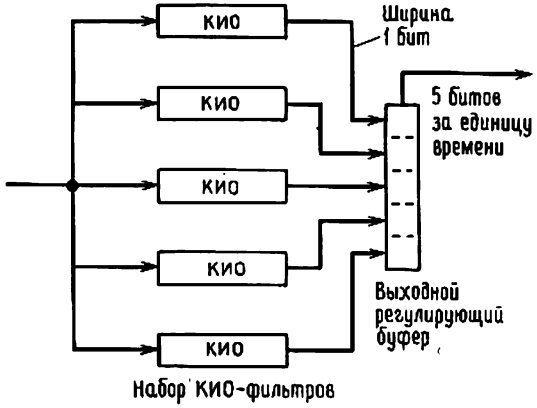


Рис. 12.7. Сверточный кодер.

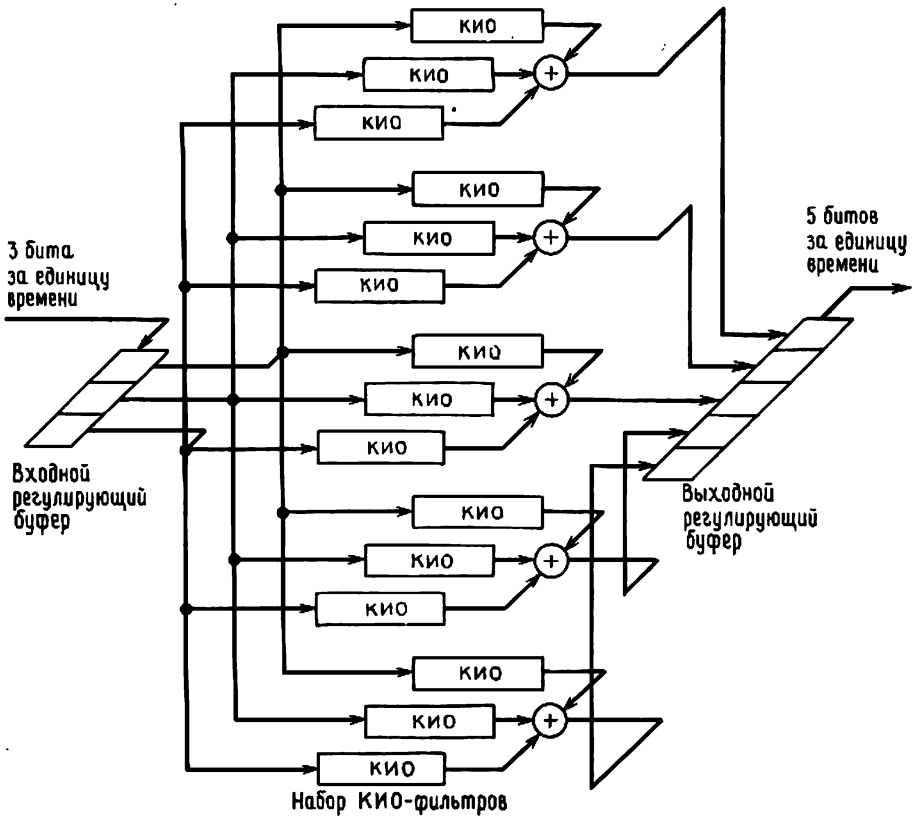


Рис. 12.8. Сверточный кодер ($n_0 = 5$, $k_0 = 3$).

быть объединены в матрицу размера $k_0 \times n_0$, называемую *порождающей матрицей из многочленов* [1]):

$$\mathbf{G}(x) = [g_{ij}(x)].$$

При k_0, большем единицы, некоторые порождающие многочлены могут равняться нулю.

Например, порождающие матрицы из многочленов кодеров на рис. 12.3 записываются в виде

$$\mathbf{G}(x) = [1 \quad x^5 + x^3 + 1]$$

и

$$\mathbf{G}(x) = [x^2 + x + 1 \quad x^2 + 1].$$

Дадим формальное определение длины кодового ограничения сверточного кода, основываясь на порождающей матрице из многочленов $\mathbf{G}(x)$.

Определение 12.2.1. *Длиной кодового ограничения* сверточного кода, задаваемого порождающей матрицей из многочленов $[g_{ij}(x)]$, называется величина [2])

$$\nu = \sum_{i=1}^{k_0} \max_j [\deg g_{ij}(x)].$$

Информационной длиной кодового слова называется

$$k = k_0 \max_{i,\,j} [\deg g_{ij}(x) + 1],$$

а *кодовой длиной блока* называется

$$n = n_0 \max_{i,\,j} [\deg g_{ij}(x) + 1].$$

Например, у сверточных кодов, кодеры которых показаны на рис. 12.9, $\nu = 3$, $k = 6$, $n = 9$ и $\nu = 4$, $k = 6$, $n = 9$ соответственно.

Будем рассматривать входной кадр как k_0 параллельно поступающих символов, а последовательность входных кадров как k_0 параллельных последовательностей символов. Они могут быть

---

[1]) В дальнейшем, следуя автору, мы иногда будем для краткости называть эту матрицу порождающей матрицей. — *Прим. ред.*

[2]) На практике используется несколько различных определений термина *длина кодового ограничения*. Неопровержимый довод в пользу нашего выбора состоит в следующем: код с меньшей решеткой имеет меньшую длину кодового ограничения, решетка всегда имеет q^{ν} узлов, и любой минимальный кодер — будь он кодером с обратной или прямой связью — всегда имеет ν элементов памяти. Следовательно, ν измеряет сложность сверточного кода. Иногда полезны также данные нами определения k и n; в литературе они также называются длиной кодового ограничения.

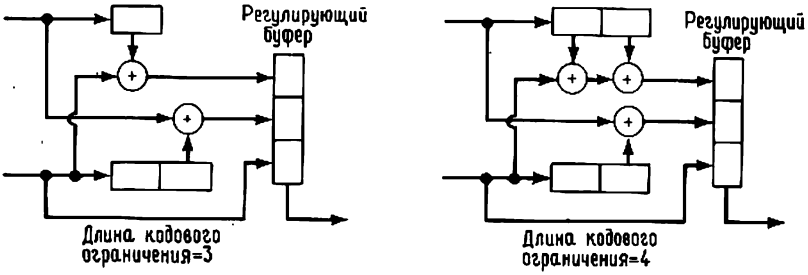


Рис. 12.9. Кодеры для двух сверточных кодов со скоростью 2/3.

представлены k_0 информационными многочленами $d_i(x)$, $i = 1, \ldots, k_0$, или вектором-строкой таких многочленов:

$$\mathbf{d}(x) = [d_1(x), d_2(x), \ldots, d_{k_0}(x)].$$

Аналогично выходное кодовое слово может быть представлено n_0 многочленами кодового слова $c_j(x)$, $j = 1, \ldots, n_0$, или вектором этих многочленов

$$\mathbf{c}(x) = [c_j(x)] = [c_1(x), c_2(x), \ldots, c_{n_0}(x)].$$

Коэффициенты многочленов кодового слова перемежаются в порядке их прохождения по каналу.

Теперь операцию кодирования можно компактно описать с помощью векторно-матричного произведения

$$\mathbf{c}(x) = \mathbf{d}(x)\,\mathbf{G}(x),$$

или, что то же самое,

$$c_j(x) = \sum_{i=1}^{k_0} d_i(x)\, g_{ij}(x).$$

Проверочная матрица $\mathbf{H}(x)$ *из многочленов* является $[(n_0 - k_0) \times n_0]$-матрицей, элементами которой являются многочлены и которая удовлетворяет условию

$$\mathbf{G}(x)\,\mathbf{H}^{\mathrm{T}}(x) = \mathbf{0}.$$

Вектор синдромных многочленов дается уравнением

$$\mathbf{s}(x) = \mathbf{v}(x)\,\mathbf{H}^{\mathrm{T}}(x).$$

Он является $(n_0 - k_0)$-мерным вектором-строкой из многочленов.

Систематический кодер для сверточного кода имеет порождающую матрицу из многочленов вида

$$\mathbf{G}(x) = [\mathbf{I} : \mathbf{P}(x)],$$

где $\mathbf{I}$ — единичная матрица размера $k_0 \times k_0$, а $\mathbf{P}(x)$ — матрица многочленов размера $k_0 \times (n_0 - k_0)$. Для систематических свер-

точных кодов проверочную матрицу из многочленов можно сразу записать в виде

$$\mathbf{H}(x) = [-\mathbf{P}^{\mathrm{T}}(x) : \mathbf{I}],$$

где $\mathbf{I}$ — единичная матрица размера $(n_0 - k_0) \times (n_0 - k_0)$. Можно непосредственно проверить, что

$$\mathbf{G}(x)\,\mathbf{H}^{\mathrm{T}}(x) = \mathbf{0}.$$

Как и в случае блоковых кодов, более корректно было бы говорить о систематических кодерах сверточных кодов, а не о систематических сверточных кодах. Систематические кодеры сверточных кодов, по-видимому, предпочтительнее, так как при их использовании информация не преобразуется и при отсутствии ошибок может считываться непосредственно. Однако в отличие от блоковых кодов не каждый сверточный код эквивалентен (в более узком смысле) систематическому сверточному коду, и некоторые несистематические сверточные коды имеют лучшие дистанционные свойства, чем систематические.

Так как в кодовых словах, не принадлежащих систематическому коду, информация непосредственно не содержится, они должны строиться так, чтобы при отсутствии ошибок ее можно было бы легко восстановить. Упомянутое выше сужение понятия эквивалентности состоит в том, что все кодеры должны конструироваться на базе КИО-фильтров без обратной связи. Если же использовать обратную связь в цепях, выполняющих деление многочленов, то можно построить систематический кодер для любого сверточного кода. На рис. 12.10 дается систематический кодер с обратной связью, соответствующий решетке, изображенной на рис. 12.5.

Перейдем к обсуждению важного частного случая $k_0 = 1$. Упростим обозначения, положив

$$\mathbf{G}(x) = [g_1(x)\ g_2(x) \ldots g_{n_0}(x)]$$

и

$$c_j(x) = d(x)\,g_j(x), \quad j = 1, \ldots, n_0.$$

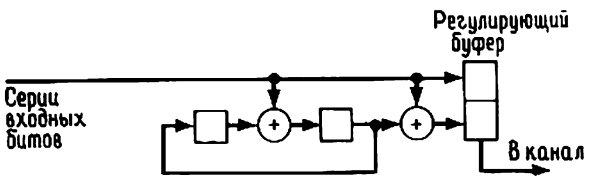


Рис. 12.10. Систематический кодер с обратной связью для сверточного (6, 3)-кода.

Для систематического кода $g_1(x) = 1$.

Определение 12.2.2. Сверточный код, порождающие многочлены $g_1(x), \ldots, g_{n_0}(x)$ которого удовлетворяют условию

$$\text{НОД}\ [g_1(x), \ldots, g_{n_0}(x)] = x^a$$

при некотором a, называется *некатастрофическим сверточным кодом*. В противном случае он называется *катастрофическим сверточным кодом*.

Без ограничения общности мы можем считать $x^a = 1$, так как невыполнение этого равенства просто эквивалентно введению во все фильтры задержки, что, очевидно, бессмысленно.

Некатастрофический сверточный код при отсутствии ошибок можно декодировать, используя алгоритм Евклида для многочленов, согласно которому существуют многочлены $a_1(x), \ldots, a_{n_0}(x)$, такие, что

$$a_1(x)\, g_1(x) + \ldots + a_{n_0} g_{n_0}(x) = 1.$$

Поэтому если многочлен поступающих данных $d(x)$ кодируется по формуле

$$c_j(x) = d(x)\, g_j(x), \quad j = 1, \ldots, n_0,$$

то $d(x)$ можно восстановить, используя соотношение

$$d(x) = a_1(x)\, c_1(x) + \ldots + a_{n_0}(x)\, c_{n_0}(x),$$

что легко проверить подстановкой.

На рис. 12.11 приведен пример. Многочлен поступающих данных вводится слева, а выводится в неизменном виде справа. Комбинированная посимвольная скорость в точках a и b равна удвоенной входной скорости, однако можно так скомбинировать символы, проходящие эти две точки, чтобы они образовывали передаваемое по каналу кодовое слово сверточного кода со скоростью 1/2. Вводимая избыточность используется для нахождения и исправления ошибок; чем больше ошибок, которые могут быть исправлены, тем лучше код.

Задача нахождения хорошего сверточного кода является задачей поиска хорошего множества взаимно простых порождающих многочленов. Найти произвольное множество взаимно простых многочленов нетрудно, а вот найти множество, которое хорошо исправляет ошибки, действительно трудно.

В общем случае k_0 больше единицы. Некатастрофический код определяется через $\binom{n_0}{k_0}$ различных $(k_0 \times n_0)$-подматриц матрицы $\mathbf{G}(x)$. Пусть для нумерации подматриц этого множества используется индекс l, и пусть $\Delta_l(x)$ — определитель l-й $(k_0 \times n_0)$-подматрицы.

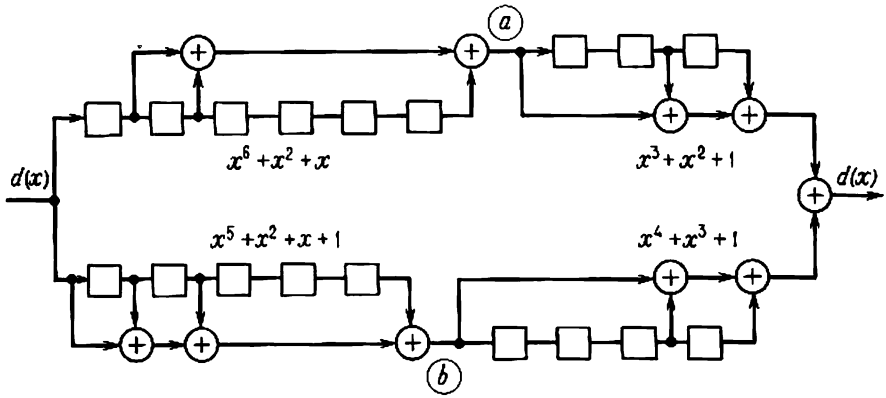


Рис. 12.11. Подробная схема регистра сдвига.

Определение 12.2.3. Сверточный код с матрицей порождающих многочленов $\mathbf{G}(x)$, определители $\Delta_l(x)$, $l = 1, \ldots, \binom{n_0}{k_0}$, подматриц которой удовлетворяют условию

$$\text{НОД}\left[\Delta_l(x),\ l = 1, \ldots, \binom{n_0}{k_0}\right] = x^a$$

при некотором a, называется *некатастрофическим сверточным кодом*. В противном случае он называется *катастрофическим сверточным кодом*.

Как и в предыдущем случае, некатастрофический код может быть обращен. Иначе говоря, существует $(n_0 \times k_0)$-матрица многочленов $\mathbf{G}^*(x)$, такая, что

$$\mathbf{G}^*(x)\,\mathbf{G}(x) = x^a\mathbf{I},$$

где $\mathbf{I}$ — единичная матрица размера $k_0 \times k_0$, а x^a определяет некоторую фиксированную задержку. Найти $\mathbf{G}^*(x)$ в общем случае затруднительно, и мы не будем этим заниматься. В случае систематических кодов сформулированные в определении 12.2.3 условия всегда выполняются. Систематические коды всегда являются некатастрофическими.

12.3. ИСПРАВЛЕНИЕ ОШИБОК И ПОНЯТИЯ РАССТОЯНИЯ

Когда кодовое слово сверточного кода передается по каналу, время от времени в его символах возникают ошибки. Декодер, принимая кодовое слово, должен исправить эти ошибки. Однако длина кодового слова сверточного кода столь велика, что в фиксированный момент времени декодер может хранить в памяти только

его часть. Хотя длина кодового слова бесконечна, все решения принимаются декодером на сегментах кодовых слов конечной длины. Безотносительно к шагу, на котором обрывается слово, принятое декодером для обработки, в силу структуры кода существует некоторая зависимость этого отрезка с теми частями слова, которые декодер еще не наблюдал. Таким образом, возможно, существует полезная информация, которую декодер не использует.

Изучение процедур декодирования сверточных кодов в большинстве случаев ограничивается вопросом исправления ошибок в первом кадре. Если этот кадр может быть исправлен и декодирован, то первый информационный кадр известен. Влияние информационных символов этого кадра на последующие кадры кодовых слов может быть учтено и исключено из них. Следовательно, задача декодирования второго кадра кодового слова аналогична задаче декодирования первого кадра кодового слова.

Продолжая эти рассуждения, приходим к выводу, что при успешном исправлении первых j кадров проблема декодирования $(j+1)$-го кадра аналогична проблеме декодирования первого кадра. Известно много подобных процедур декодирования. Процедура, использующая информационные символы исправленного кадра для явного исключения их влияния на последующие кадры, называется *процедурой с обратной связью*. Другие декодеры оперируют таким образом, чтобы соответствующим образом декодированные предшествующие кадры не оказывали никакого влияния на текущий кадр.

В любом декодере может случиться так, что в связи со слишком большим количеством ошибок первый кадр кодового слова не будет исправлен должным образом. В некоторых декодерах это приводит к введению ошибок в последующие кадры, вызывая их неправильное кодирование. Если ошибка в декодировании одного кадра приводит к появлению в кодовом слове бесконечного числа дополнительных ошибок, то говорят, что в декодере происходит *распространение ошибок*. Если распространение ошибок может быть устранено выбором алгоритма декодирования, это явление называют *обычным распространением ошибок*; если же распространение ошибок вызывается выбором катастрофического порождающего многочлена сверточного кода, то говорят о *катастрофическом распространении ошибок*. Выбор надлежащей конструкции системы позволяет избежать обеих этих возможностей.

Число символов, которые декодер может хранить в памяти, называется *шириной окна декодирования*. Если ставить своей целью обнаружить как можно больше конфигураций ошибок, то в общем случае увеличение ширины окна декодирования всегда приводит к улучшению характеристик, однако в конце концов происходит насыщение. Ширина окна декодирования должна

быть не меньше длины блока n и зачастую в несколько раз превышает последнюю.

Сверточный код характеризуется многими минимальными расстояниями, определяемыми длинами начальных сегментов кодовых слов, между которыми берется минимальное расстояние. Мера расстояния определяется таким образом, что если при декодировании двух кодовых слов получается один и тот же первый информационный кадр, то эти слова считаются эквивалентными.

Определение 12.3.1. Минимальное расстояние Хэмминга для любых начальных сегментов длины l кадров всех пар кодовых слов, отличающихся начальным кадром, называется *l-м минимальным расстоянием* сверточного кода и обозначается через d_l^*. Если l равно $m + 1$, то оно называется просто *минимальным расстоянием* и обозначается через d^*. Последовательность d_1^*, d_2^*, d_3^*, ... называется *дистанционным профилем* сверточного кода.

Так как сверточный код линеен, одно из двух кодовых слов может целиком состоять из нулей. В этом случае l-е минимальное расстояние равно минимальному из всех весов сегментов длины l кадров кодовых слов с ненулевым первым кадром. Оно может быть вычислено по маркированной решетке. Из рис. 12.5 следует, что для сверточного (6, 3)-кода $d_1^* = 2$, $d_2^* = 3$ и $d_i^* = 5$ при всех $i \geqslant 3$.

Предположим, что сверточный код имеет l-е минимальное расстояние d_l^*. Если в первых l кадрах произошло не более t ошибок, причем t удовлетворяет неравенству

$$2t + 1 \leqslant d_l^*,$$

то ошибки, которые появились в первом кадре кодового слова, могут быть исправлены. В частности, выберем $l = m + 1$; минимальное расстояние кода d^* равно d_{m+1}^*. Тогда при t, удовлетворяющем неравенству

$$2t + 1 \leqslant d^*,$$

код исправит первый кадр кодового слова, если на длине первого блока появилось не более t ошибок. Такой код называется *сверточным кодом, исправляющим t* ошибок. Более точное, хотя и более громоздкое название: сверточный код, исправляющий t ошибок на длине блока.

Определение 12.3.2. *Свободным расстоянием* сверточного кода $\mathcal{C}$ называется $d_\infty = \max d_l$.

Очевидно, что

$$d_{m+1} \leqslant d_{m+2} \leqslant \ldots \leqslant d_\infty.$$

Определение 12.3.3. *Свободной длиной* n_∞ сверточного кода называется длина имеющего наименьший вес ненулевого началь-

ного сегмента кодовой последовательности сверточного кода. Таким образом, $d_l = d_\infty$, если $l = n_\infty$, и $d_l < d_\infty$, если $l < n_\infty$.

В примере, приведенном на рис. 12.5, свободное расстояние равно 5, а свободная длина 6. В этом случае свободная длина равна длине блока сверточного кода. В общем случае она может быть больше длины блока.

12.4. МАТРИЧНОЕ ОПИСАНИЕ СВЕРТОЧНЫХ КОДОВ

Сверточный код состоит из бесконечного числа кодовых слов бесконечной длины. Он линеен и может быть описан бесконечной порождающей матрицей. Для описания каждого кода может быть использовано огромное количество порождающих матриц, но удобно оперировать только некоторыми из них. Даже в лучшем случае порождающая матрица сверточного кода более громоздка, чем порождающая матрица блокового кода.

Порождающие многочлены, имеющие индексы i и j, могут быть записаны в виде

$$g_{ij}(x) = \sum_l g_{ijl} x^l.$$

Для построения порождающей матрицы коэффициенты g_{ijl} упорядочиваются в виде матрицы. Пусть при каждом l $\mathbf{G}_l$ является $(k_0 \times n_0)$-матрицей:

$$\mathbf{G}_l = [g_{ijl}].$$

Тогда порождающая матрица сверточного кода, усеченного до блокового кода длины n, записывается в виде

$$\mathbf{G}^{(n)} = \begin{bmatrix} \mathbf{G}_0 & \mathbf{G}_1 & \mathbf{G}_2 & \dots & \mathbf{G}_m \\ \mathbf{0} & \mathbf{G}_0 & \mathbf{G}_1 & \dots & \mathbf{G}_{m-1} \\ \mathbf{0} & \mathbf{0} & \mathbf{G}_0 & \dots & \mathbf{G}_{m-2} \\ \cdot & & & & \cdot \\ \cdot & & & & \cdot \\ \cdot & & & & \cdot \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & & \mathbf{G}_0 \end{bmatrix},$$

где символом 0 обозначена $(k_0 \times n_0)$-матрица, целиком состоящая из нулей. Порождающей матрицей сверточного кода является матрица

$$\mathbf{G} = \begin{bmatrix} \mathbf{G}_0 & \mathbf{G}_1 & \mathbf{G}_2 & \dots & \mathbf{G}_m & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \dots \\ \mathbf{0} & \mathbf{G}_0 & \mathbf{G}_1 & \dots & \mathbf{G}_{m-1} & \mathbf{G}_m & \mathbf{0} & \mathbf{0} & \mathbf{0} & \dots \\ \mathbf{0} & \mathbf{0} & \mathbf{G}_0 & \dots & \mathbf{G}_{m-2} & \mathbf{G}_{m-1} & \mathbf{G}_m & \mathbf{0} & \mathbf{0} & \dots \\ \cdot & & & & & & & & & \\ \cdot & & & & & & & & & \\ \cdot & & & & & & & & & \end{bmatrix},$$

бесконечно продолжающаяся вниз и вправо. За исключением диагональной полосы, состоящей из m ненулевых подматриц, все ее подматрицы являются нулевыми.

В случае систематического сверточного кода эти две матрицы могут также быть записаны в виде

$$\mathbf{G}^{(n)}=\begin{bmatrix} \mathbf{I}\ \mathbf{P}_0 & \mathbf{0}\ \mathbf{P}_1 & \mathbf{0}\ \mathbf{P}_2 & \cdots & \mathbf{0}\ \mathbf{P}_m \\ \mathbf{0}\ \mathbf{0} & \mathbf{I}\ \mathbf{P}_0 & \mathbf{0}\ \mathbf{P}_1 & \cdots & \mathbf{0}\ \mathbf{P}_{m-1} \\ \mathbf{0}\ \mathbf{0} & \mathbf{0}\ \mathbf{0} & \mathbf{I}\ \mathbf{P}_0 & \cdots & \mathbf{0}\ \mathbf{P}_{m-2} \\ \vdots & & & & \vdots \\ \mathbf{0}\ \mathbf{0} & \mathbf{0}\ \mathbf{0} & \mathbf{0}\ \mathbf{0} & \cdots & \mathbf{I}\ \mathbf{P}_0 \end{bmatrix}$$

и

$$\mathbf{G}=\begin{bmatrix} \mathbf{I}\ \mathbf{P}_0 & \mathbf{0}\ \mathbf{P}_1 & \mathbf{0}\ \mathbf{P}_2 & \cdots & \mathbf{0}\ \mathbf{P}_m & \mathbf{0}\ \mathbf{0} & \mathbf{0}\ \mathbf{0} & \cdots \\ \mathbf{0}\ \mathbf{0} & \mathbf{I}\ \mathbf{P}_0 & \mathbf{0}\ \mathbf{P}_1 & \cdots & \mathbf{0}\ \mathbf{P}_{m-1} & \mathbf{0}\ \mathbf{P}_m & \mathbf{0}\ \mathbf{0} & \cdots \\ \mathbf{0}\ \mathbf{0} & \mathbf{0}\ \mathbf{0} & \mathbf{I}\ \mathbf{P}_0 & \cdots & \mathbf{0}\ \mathbf{P}_{m-2} & \mathbf{0}\ \mathbf{P}_{m-1} & \mathbf{0}\ \mathbf{P}_m & \cdots \\ \vdots & & & & \vdots & \mathbf{0}\ \mathbf{P}_{m-2} & \mathbf{0}\ \mathbf{P}_{m-1} & \cdots \\ & & & & & \vdots & \mathbf{0}\ \mathbf{P}_{m-2} & \cdots \\ & & & & & & \vdots & \end{bmatrix}$$

где каждая строка получается сдвигом предыдущей строки вправо, а необозначенные матричные элементы справа и слева от нее равны нулю. Через $\mathbf{I}$ обозначена единичная $(k_0 \times k_0)$-матрица, через $\mathbf{0}$ — матрица того же размера, состоящая из одних нулей, а через $\mathbf{P}_0, \ldots, \mathbf{P}_m$ — матрицы размера $k_0 \times (n_0 - k_0)$. Первая строка описывает кодирование первого информационного кадра в первые m кадров кодового слова. Структура этого матричного выражения соответствует работе регистра сдвига кодера.

Первый информационный кадр кодируется в первый кадр кодового слова элементом, находящимся в левом верхнем углу матрицы $\mathbf{G}$, а именно элементом

$$\mathbf{G}^{(n_0)} = [\mathbf{I}\ \mathbf{P}_0].$$

Аналогично два первых информационных кадра кодируются в два первых кадра кодового слова по правилу

$$\mathbf{G}^{(2n_0)} = \begin{bmatrix} \mathbf{I} & \mathbf{P}_0 & \mathbf{0} & \mathbf{P}_1 \\ \mathbf{0} & \mathbf{0} & \mathbf{I} & \mathbf{P}_0 \end{bmatrix}.$$

Проверочной матрицей является любая матрица $\mathbf{H}$, удовлетворяющая условиям

$$\mathbf{G}^{(ln_0)}(\mathbf{H}^{(ln_0)})^{\mathrm{T}} = 0, \quad l = 0, 1, 2, \ldots,$$

где $\mathbf{G}^{(ln_0)}$ и $\mathbf{H}^{(ln_0)}$ — стоящие в левых верхних углах матриц $\mathbf{G}$ и $\mathbf{H}$ подматрицы, соответствующие l кадрам. Бесконечномерная проверочная матрица $\mathbf{H}$ может быть построена по $\mathbf{G}$. В качестве проверочной выберем матрицу

$$\mathbf{H} = \begin{bmatrix} \mathbf{P}_0^T & -\mathbf{I} & & & & & & \dots \\ \mathbf{P}_1^T & 0 & \mathbf{P}_0^T & -\mathbf{I} & & & & \\ \mathbf{P}_2^T & 0 & \mathbf{P}_1^T & 0 & \mathbf{P}_0^T & -\mathbf{I} & & \\ \vdots & & & & & & & \\ \mathbf{P}_m^T & 0 & \mathbf{P}_{m-1}^T & 0 & \mathbf{P}_{m-2}^T & 0 \dots & \mathbf{P}_0^T & -\mathbf{I} \dots \\ & & \mathbf{P}_m^T & 0 & \mathbf{P}_{m-1}^T & 0 \dots & & \\ & & & & \mathbf{P}_m^T & 0 \dots & & \end{bmatrix},$$

где, как и ранее, все необозначенные элементы являются нулевыми.

Пример. Систематический двоичный сверточный (4,2)-код с $k_0 = 1$ и $m = 1$ описывается матрицами $\mathbf{P}_0 = 1$ и $\mathbf{P}_1 = 1$ размера 1×1. Следовательно,

$$\mathbf{G} = \begin{bmatrix} 1 & 1 & 0 & 1 & & & & \\ & & 1 & 1 & 0 & 1 & & \\ & & & & 1 & 1 & 0 & 1 \\ & & & & & & \cdot & \cdot \cdot \\ & & & & & & & \cdot \cdot \cdot \end{bmatrix}$$

и

$$\mathbf{H} = \begin{bmatrix} 1 & 1 & & & & \\ 1 & 0 & 1 & 1 & & \\ & & 1 & 0 & 1 & 1 \\ & & & & \cdot & \cdot \cdot \\ & & & & & \cdot \cdot \cdot \end{bmatrix}.$$

Пример. Систематический двоичный сверточный (12,4)-код с $k_0 = 1$ и $m = 3$ задается матрицами $\mathbf{P}_0 = [1\ 1]$, $\mathbf{P}_1 = [0\ 1]$, $\mathbf{P}_2 = [1\ 0]$ и $\mathbf{P}_3 = [1\ 1]$. Следовательно,

$$\mathbf{G} = \begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 1 & & & & & & \\ & & & 1 & 1 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 1 & & & \\ & & & & & & 1 & 1 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 1 \\ & & & & & & & & & \cdot & & & & & & & & \\ & & & & & & & & & \cdot & & & & & & & & \\ & & & & & & & & & \cdot & & & & & & & & \end{bmatrix}$$

и

$$\mathbf{H} = \begin{bmatrix} 1 & 1 & 0 & & & & & & & & & & \\ 1 & 0 & 1 & & & & & & & & & & \\ 0 & 0 & 0 & 1 & 1 & 0 & & & & & & & \\ 1 & 0 & 0 & 1 & 0 & 1 & & & & & & & \\ 1 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & & & & \\ 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & & & & \\ 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & \\ 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & \\ & & & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 \\ & & & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ & & & \vdots & & & & & & & & & & \\ & & & \vdots & & & & & & & & & & \\ & & & \cdot & & & & & & & & & & \end{bmatrix}.$$

12.5. НЕКОТОРЫЕ ПРОСТЫЕ СВЕРТОЧНЫЕ КОДЫ

Известно всего несколько конструктивных классов сверточных кодов. В настоящее время не известен ни один такой класс с алгебраической структурой, сравнимой со структурой кодов БЧХ, исправляющих l ошибок; таким образом, не существует конструктивных методов поиска сверточных кодов с большой длиной кодового ограничения. Большинство известных и используемых в настоящее время лучших сверточных кодов было найдено поиском на ЭВМ.

В этом параграфе будет описан общий класс сверточных кодов, исправляющих одну ошибку. Параграф завершается таблицей некоторых хороших сверточных кодов, известных в текущей литературе.

Класс двоичных сверточных кодов, исправляющих одну ошибку и называемых *кодами Вайнера—Эша*, аналогичен классу кодов Хэмминга. Для каждого положительного целого m существует $((m+1)2^m, (m+1)(2^m-1))$-код Вайнера—Эша. Такой код определяется проверочной матрицей $\mathbf{H}'$ $(2^m-1, 2^m-1-m)$-кода Хэмминга. Это проверочная $[m \times (2^m-1)]$-матрица, в которой все 2^m-1 столбцов различны и ненулевые. Выберем такую матрицу, используем ее строки для определения множества $[1 \times (2^m-1)]$-матриц $\mathbf{P}_1^{\mathrm{T}}, \ldots, \mathbf{P}_m^{\mathrm{T}}$ и обозначим через $\mathbf{P}_0^{\mathrm{T}}$ вектор-

строку, все $2^m - 1$ элементов которой равны единице. Тогда проверочная матрица кода Вайнера—Эша запишется в виде

$$\mathbf{H} = \begin{bmatrix} \mathbf{P}_0^{\mathrm{T}} & 1 & 0 & 0 & 0 & 0 & 0 & \dots \\ \mathbf{P}_1^{\mathrm{T}} & 0 & \mathbf{P}_0^{\mathrm{T}} & 1 & 0 & 0 & 0 & \\ \mathbf{P}_2^{\mathrm{T}} & 0 & \mathbf{P}_1^{\mathrm{T}} & 0 & \mathbf{P}_0^{\mathrm{T}} & 1 & 0 & \\ \vdots & & \vdots & & \mathbf{P}_1^{\mathrm{T}} & & & \\ \vdots & & \vdots & & & & & \\ \mathbf{P}_m^{\mathrm{T}} & 0 & & & & & & \\ 0 & 0 & \mathbf{P}_m^{\mathrm{T}} & 0 & & & & \\ \vdots & \vdots & & & & & & \\ \vdots & \vdots & & & & & & \\ \vdots & \vdots & & & & & & \end{bmatrix}$$

и

$$\mathbf{H}^{((m+1)\,2m)} = \begin{bmatrix} \mathbf{P}_0^{\mathrm{T}} & 1 & 0 & 0 & 0 & 0 & & \\ \mathbf{P}_1^{\mathrm{T}} & 0 & \mathbf{P}_0^{\mathrm{T}} & 1 & 0 & 0 & & \\ \mathbf{P}_2^{\mathrm{T}} & 0 & \mathbf{P}_1^{\mathrm{T}} & 0 & \mathbf{P}_0^{\mathrm{T}} & 1 & & \\ \vdots & & & & & & & \\ \vdots & & & & & & & \\ \vdots & & & & & & & \\ \mathbf{P}_m^{\mathrm{T}} & 0 & \mathbf{P}_{m-1}^{\mathrm{T}} & 0 & \mathbf{P}_{m-2}^{\mathrm{T}} & 0 & \dots \mathbf{P}_0^{\mathrm{T}} & 1 \end{bmatrix},$$

где 1 — матрица размера 1×1, состоящая из одной единицы, и 0 — матрица размера 1×1, состоящая из одного нуля.

Теорема 12.5.1. *Минимальное расстояние d^* кода Вайнера—Эша равно* 3; *таким образом, он является сверточным кодом, исправляющим одну ошибку.*

Доказательство. Для вычисления минимального расстояния выбираются два кодовых слова, отличающихся в первом кадре. Так как код линеен, в качестве одного слова можно взять нулевое кодовое слово, а в качестве другого — кодовое слово с единицей в первом кадре. Поскольку для кодовых слов $\mathbf{c}\mathbf{H}^{\mathrm{T}} = 0$, сумма тех столбцов $\mathbf{H}$, в которых $\mathbf{c}$ отлично от нуля, равна нулю. Верхняя строка $\mathbf{H}$ не равна нулю только в первом кадре, поэтому первый кадр содержит по меньшей мере две единицы. Наконец, так как любые два столбца $\mathbf{H}$ в первом кадре линейно независимы, кодовое слово должно содержать еще одну единицу, поэтому минимальное расстояние равно по меньшей мере 3.

Для того чтобы показать, что оно равно точно 3, достаточно показать, что существуют три столбца, сумма которых равна нулю. Возьмем столбец, где $\mathbf{P}_0^{\mathrm{T}}, \dots, \mathbf{P}_m^{\mathrm{T}}$ равны единице. Сумма

этого столбца в первом кадре с тем же столбцом во втором кадре равна $(1, 0, 0, \ldots, 0)^r$, и этот столбец содержится в первом кадре. Следовательно, существуют три столбца, сумма которых равна нулю. □

Например, (12,9)-код Вайнера—Эша соответствует $m = 2$. Его проверочная матрица равна

$$\mathbf{H} = \begin{bmatrix} 1 & 1 & 1 & 1 & & & & & & & & & & & & \\ 1 & 1 & 0 & 0 & 1 & 1 & 1 & 1 & & & & & & & & \\ 1 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 1 & 1 & & & & \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 1 & 1 & 0 & 0 \ldots \\ & \vdots & & & & \vdots & & & & \vdots & & & & \vdots & & \\ & \vdots & & & & \vdots & & & & \vdots & & & & \vdots & & \end{bmatrix}.$$

При таком усечении кода, чтобы его проверочная матрица соответствовала длине блока 12, получается следующая проверочная матрица:

$$\mathbf{H}^{(12)} = \begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 1 & 1 \end{bmatrix}.$$

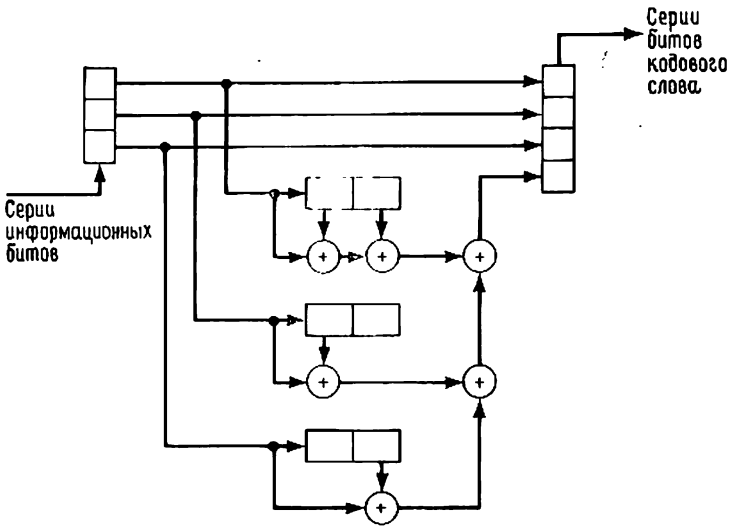


Рис. 12.12. Кодер для (12, 9)-кода Вайнера—Эша.

Порождающая матрица равна

$$\mathbf{G}=\begin{bmatrix}
1\,0\,0\;1\;\;0\,0\,0\;1\;0\,0\,0\;1 & & & & & & & & & & & \\
0\,1\,0\;1\;\;0\,0\,0\;1\;0\,0\,0\;0 & & & & & & & & & & & \\
0\,0\,1\;1\;\;0\,0\,0\;0\;0\,0\,0\;1 & & & & & & & & & & & \\
 & & 1\,0\,0\;1\;0\,0\,0\;1\;0\,0\,0\;1 & & & & & & & & & \\
 & & 0\,1\,0\;1\;0\,0\,0\;1\;0\,0\,0\;0 & & & & & & & & & \\
 & & 0\,0\,1\;1\;0\,0\,0\;0\;0\,0\,0\;1 & & & & & & & & & \\
 & & & & 1\,0\,0\;1\;0\,0\,0\;1\;0\,0\,0\;1 & & & & & & & \\
 & & & & 0\,1\,0\;1\;0\,0\,0\;1\;0\,0\,0\;0 & \ldots & & & & & & \\
 & & & & 0\,0\,1\;1\;0\,0\,0\;0\;0\,0\,0\;1 & & & & & & & \\
 & & & & & & & & & & & \\
 & & & & & & & & & & &
\end{bmatrix},$$

а порождающая матрица кода, усеченного до длины блока 12, представляется в виде

$$\mathbf{G}^{(12)}=\begin{bmatrix}
1\,0\,0\;1\;0\,0\,0\;1\;0\,0\,0\;1 \\
0\,1\,0\;1\;0\,0\,0\;1\;0\,0\,0\;0 \\
0\,0\,1\;1\;0\,0\,0\;0\;0\,0\,0\;1
\end{bmatrix}.$$

Непосредственным рассмотрением $\mathbf{G}^{(12)}$ убеждаемся, что в пределах блока длины 12 каждое ненулевое слово имеет вес не менее трех. Следовательно, в блоке длины 12 код может исправлять одну ошибку.

Кодер для (12,9)-кода Вайнера—Эша представлен на рис. 12.12. Каждому порождающему многочлену на схеме соответствует отдельный КИО-фильтр.

Большинство известных наиболее употребительных сверточных кодов было получено поиском на ЭВМ. На рис. 12.13 приведена таблица таких кодов.

(n, k, d_∞)	Порождающая матрица из многочленов (выражена через коэффициенты многочлена)			
(6, 3, 5)	(x^2+1)	101	(x^2+x+1)	111
(8, 4, 6)	(x^3+x+1)	1011	(x^3+x^2+x+1)	1111
(10, 5, 7)		11001		10111
(12, 6, 8)		110101		101111
(14, 7, 10)		1101101		1001111
(16, 8, 10)		11100101		10011111
(18, 9, 12)		100011101		110101111
(20, 10, 12)		1110111001		1010011011
(22, 11, 14)		10111011001		10001101111
(24, 12, 15)		101110110001		100010111101
(26, 13, 16)		110110101001		100011011111
(28, 14, 16)		101111011100		110010100111
(9, 3, 8)		101	111	111
(12, 4, 10)		1101	1011	1111
(15, 5, 12)		10101	11011	11111
(18, 6, 13)		111001	110101	101111
(21, 7, 15)		1101101	1010011	1011111
(24, 8, 16)		10101001	10011011	11101111
(27, 9, 18)		111101101	110011011	100100111
(30, 10, 20)		1111001001	1010111101	1101100111
(33, 11, 22)		11010111001	10011101101	10111110011
(36, 12, 24)		111011111001	110010111101	101011010011
(39, 13, 24)		110110101000	101111010001	100011011111
(42, 14, 26)		101001011100	100011011101	110110100111
(12, 3, 10)	101		111	111
(16, 4, 13)	1101		1011	1111
(20, 5, 16)	10101		10111	11111
(24, 6, 18)	110101		100111	101111
(28, 7, 20)	1011101		1110011	1100111
(32, 8, 22)	10111001		11010011	11110111
(36, 9, 24)	110011001		101110101	101001111
(40, 10, 27)	1111001001		1010111101	1101010111
(44, 11, 29)	11101011001		11010111001	10011101101
(48, 12, 32)	111011111001		110010111101	101011010011
(52, 13, 33)	1010011001001		1111110010101	1101111011011
(56, 14, 36)	11010010010001		10111110011001	11101010110111

Рис. 12.13. Некатастрофические двоичные сверточные коды с максимальным свободным расстоянием.

12.6. АЛГОРИТМЫ СИНДРОМНОГО ДЕКОДИРОВАНИЯ

Предположим, что принята бесконечная последовательность $\mathbf{v}$, состоящая из слова сверточного кода и шума:

$$\mathbf{v} = \mathbf{c} + \mathbf{e}.$$

Точно так же, как и в случае блоковых кодов, можно вычислить синдром:

$$\mathbf{s} = \mathbf{v}\mathbf{H}^{\mathrm{T}} = \mathbf{e}\mathbf{H}^{\mathrm{T}},$$

однако в этом случае синдром имеет бесконечную длину. Декодер не просматривает весь синдром однократно. Он работает с конца, вычисляя компоненты $\mathbf{s}$ по мере их поступления, исправляя ошибки, и сбрасывая те компоненты $\mathbf{s}$, которые вычислены давно. Декодер содержит таблицу сегментов синдромов и сегментов конфигураций шума, которые приводят к данным сегментам синдрома. Когда декодер находит в таблице полученный сегмент синдрома, он исправляет начальный сегмент кодового слова.

Приведем примеры различных синдромных декодеров. Начнем с декодера для (12,9)-кода Вайнера—Эша. Он показан на рис. 12.14. Входящий поток битов поступает на n_0 параллельных линий; синдром вычисляется путем нахождения проверочного бита по принятым информационным битам и его сравнения с принятым проверочным битом. Необходимо вычислить лишь столько синдромных битов, сколько их требуется для исправления одиночной ошибки в первом кадре. Возможные конфигурации одиночных ошибок в первых трех кадрах и соответствующие первые три бита синдрома приведены в виде таблицы на рис. 12.15. Заметим, что правый бит равен единице тогда и только тогда, когда в первом кадре имеется ошибка. В этом случае два других бита синдрома точно определяют местонахождение ошибки в первом кадре.

Элементы, изображенные на рис. 12.14 штриховыми линиями, могут быть включены в декодер для восстановления кодовых слов, содержащих проверочные биты. Если такой необходимости нет, то эти элементы могут быть исключены.

Необходимо рассмотреть еще одну задачу. После исправления первого кадра нужно изменить синдром таким образом, чтобы он не вызывал ошибочного исправления в следующем кадре. Это можно сделать несколькими способами, и хотя все они пригодны для данного кода, в случае более мощного кода они ведут себя совершенно по-разному. Возможны два варианта:

1) после каждого исправления ошибки устанавливается нулевое состояние регистра синдрома;

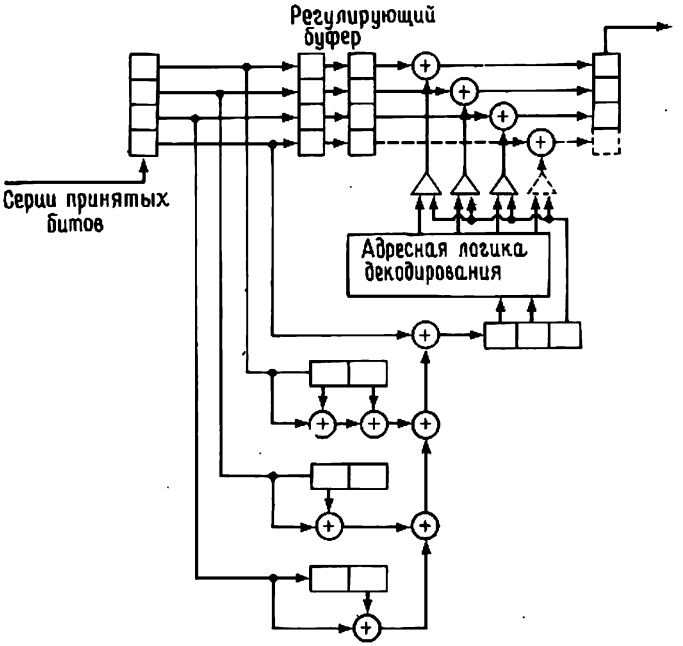


Рис. 12.14. Декодер для (12, 9)-кода Вайнера—Эша.

Конфигурация ошибок				Синдром
Четвертый кадр	Третий кадр	Второй кадр	Первый кадр	
.....	0 0 0 0	0 0 0 0	0 0 0 1	1 1 1
	0 0 0 0	0 0 0 0	0 0 1 0	0 1 1
	0 0 0 0	0 0 0 0	0 1 0 0	1 0 1
	0 0 0 0	0 0 0 0	1 0 0 0	0 0 1
	0 0 0 0	0 0 0 1	0 0 0 0	1 1 0
	0 0 0 0	0 0 1 0	0 0 0 0	1 1 0
	0 0 0 0	0 1 0 0	0 0 0 0	0 1 0
	0 0 0 0	1 0 0 0	0 0 0 0	0 1 0
	0 0 0 1	0 0 0 0	0 0 0 0	1 0 0
	0 0 1 0	0 0 0 0	0 0 0 0	1 0 0
	0 1 0 0	0 0 0 0	0 0 0 0	1 0 0
	1 0 0 0	0 0 0 0	0 0 0 0	1 0 0

Рис. 12.15. Таблица декодирования для (12, 9)-кода Вайнера—Эша.

2) из регистра синдрома вычитается корректирующий синдром. Первый вариант характерен для кодов, исправляющих одну ошибку. Изучим второй вариант на более сложном примере.

На рис. 12.3 был представлен кодер для (6,3)-кода, исправляющего двойные ошибки. Соответствующий декодер изображен на рис. 12.16. Он использует таблицу синдромов, приведенную на рис. 12.17. Кодер исправляет две ошибки в шести первых битах. С помощью обратной связи вклад каждой исправленной ошибки удаляется из регистра синдрома; так как код несистематический, информационные символы восстанавливаются по исправленному

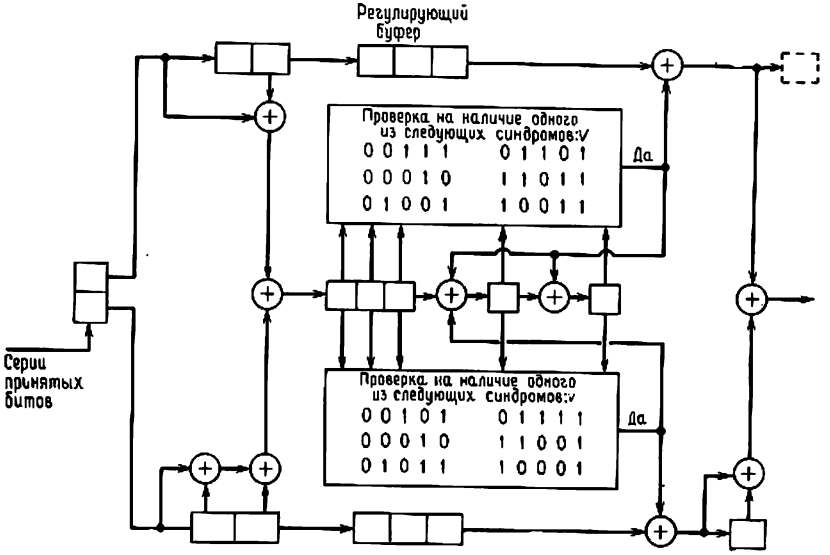


Рис. 12.16. Декодер для сверточного (6, 3)-кода.

Конфигурация ошибок			Синдром
Четвертый кадр	Третий кадр	Второй кадр	Первый кадр	
.....	0 0	0 0	0 1	... 0 0 1 1 1
	0 0	0 0	1 0	0 0 1 0 1
	0 0	0 0	1 1	0 0 0 1 0
	0 0	0 1	0 1	0 1 0 0 1
	0 0	1 0	0 1	0 1 1 0 1
	0 1	0 0	0 1	1 1 0 1 1
	1 0	0 0	0 1	1 0 0 1 1
	0 0	0 1	1 0	0 1 0 1 1
	0 0	1 0	1 0	0 1 1 1 1
	0 1	0 0	1 0	1 1 0 0 1
	1 0	0 0	1 0	1 0 0 0 1

Рис. 12.17. Таблица декодирования для сверточного (6, 3)-кода.

Конфигурация ошибок	Синдром
... 0 0 0 0 0 0 0 0 0 0 0 1	... 1 1 1 0 0 1
0 0 0 0 0 0 0 0 0 0 1 1	1 1 1 0 0 0
0 0 0 0 0 0 0 0 0 1 0 1	0 0 1 0 1 1
0 0 0 0 0 0 0 0 1 0 0 1	1 1 1 0 1 1
0 0 0 0 0 0 0 1 0 0 0 1	0 1 1 1 0 1
0 0 0 0 0 0 1 0 0 0 0 1	1 1 1 1 0 1
0 0 0 0 0 1 0 0 0 0 0 1	1 1 0 0 0 1
0 0 0 0 1 0 0 0 0 0 0 1	1 1 0 0 0 1
0 0 0 1 0 0 0 0 0 0 0 1	1 0 1 0 0 1
0 0 1 0 0 0 0 0 0 0 0 1	1 1 0 0 0 1
0 1 0 0 0 0 0 0 0 0 0 1	0 1 1 0 0 1
1 0 0 0 0 0 0 0 0 0 0 1	0 1 1 0 0 1
0 0 0 0 0 0 0 0 0 0 1 0	0 0 0 0 0 1
0 0 0 0 0 0 0 0 0 1 1 0	1 1 0 0 1 1
0 0 0 0 0 0 0 0 1 0 1 0	0 0 0 0 1 1
0 0 0 0 0 0 0 1 0 0 1 0	1 0 0 1 0 1
0 0 0 0 0 0 1 0 0 0 1 0	0 0 0 1 0 1
0 0 0 0 0 1 0 0 0 0 1 0	0 0 1 0 0 1
0 0 0 0 1 0 0 0 0 0 1 0	0 0 1 0 0 1
0 0 0 1 0 0 0 0 0 0 1 0	0 1 0 0 0 1
0 0 1 0 0 0 0 0 0 0 1 0	0 1 0 0 0 1
0 1 0 0 0 0 0 0 0 0 1 0	1 0 0 0 0 1
1 0 0 0 0 0 0 0 0 0 1 0	1 0 0 0 0 1

Рис. 12.18. Таблица декодирования для сверточного (12, 6)-кода.

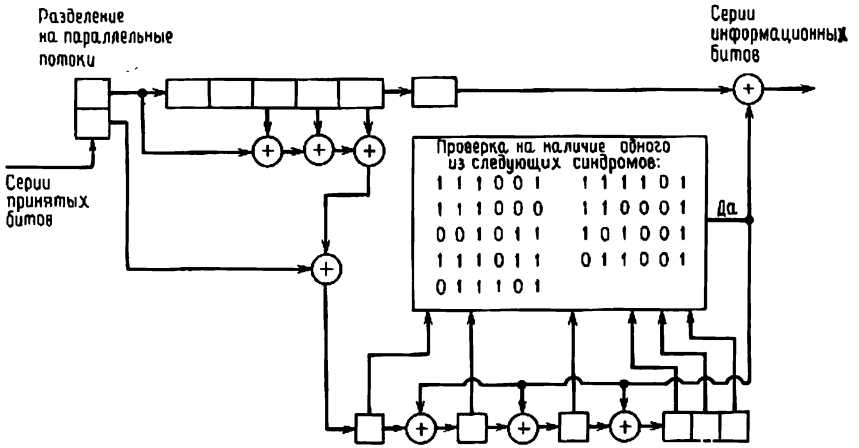


Рис. 12.19. Декодер для сверточного (12, 6)-кода.

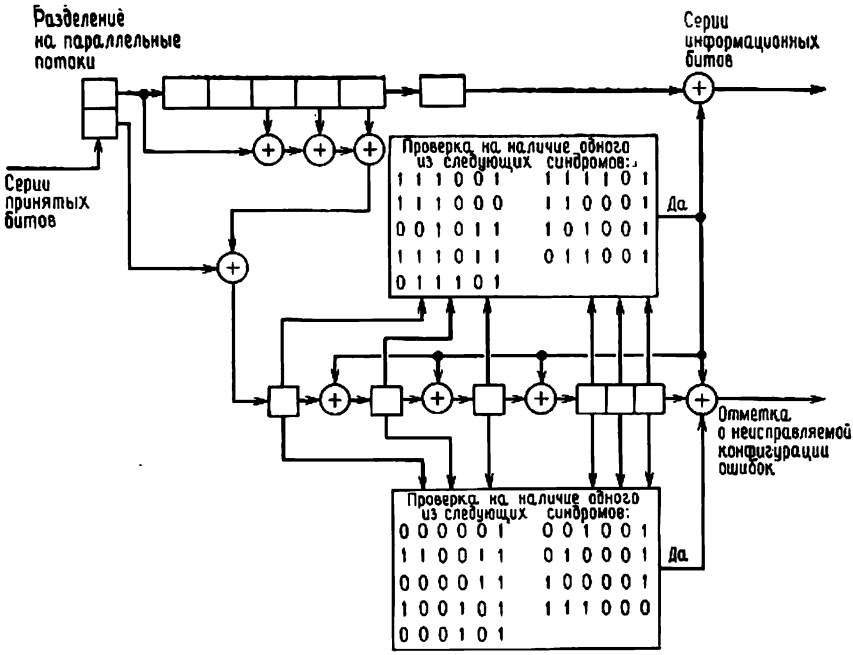


Рис. 12.20. Другой декодер для сверточного (12, 6)-кода.

кодовому слову. Для восстановления информации используется следующее соотношение:

$$1 = \text{НОД}\ [x^2 + x + 1,\ x^2 + 1] =$$
$$= x(x^2 + x + 1) + (x + 1)(x^2 + 1).$$

Декодер на рис. 12.16 не является полным, так как существуют синдромы, не вошедшие в таблицу декодирования. В принципе можно исправлять некоторые другие конфигурации ошибок, включив в таблицу дополнительные синдромы и увеличив длину синдромной памяти. Практически, однако, лучшим способом улучшения характеристик системы является, по-видимому, выбор кода с большей длиной блока. Прежде чем пытаться преобразовать синдромный декодер в полный, следует познакомиться с декодером Витерби, который будет описан в § 12.8.

В заключение параграфа рассмотрим пример систематического сверточного (12,6)-кода. Этот код может исправлять две ошибки в блоке длины 12. В § 13.4 мы вернемся к изучению этого кода, рассматривая его как мажоритарно декодируемый код. Однако в настоящем параграфе используется синдромное его декодирование, таблица декодирования приведена на рис. 12.18. Синдромы,

расположенные выше пробела, соответствуют конфигурациям ошибок, содержащим ошибку в правом информационном бите, а синдромы, расположенные ниже пробела, соответствуют конфигурациям ошибок, содержащим ошибку в правом проверочном бите, но не содержащем ошибок в правом информационном бите. Декодер, представленный на рис. 12.19, проверяет лишь синдромы, расположенные выше пробела, и поэтому позволяет исправлять информационные биты лишь при условии, что конфигурация ошибок лежит в пределах конструктивных возможностей декодера. Однако такой декодер не находит неисправляемых, но обнаруживаемых конфигураций ошибок. Чтобы найти такие конфигурации, в конструкции декодера надо предусмотреть проверку синдромов, расположенных ниже пробела. Такой декодер, обеспечивающий обнаружение ошибок, представлен на рис. 12.20.

12.7. СВЕРТОЧНЫЕ КОДЫ ДЛЯ ИСПРАВЛЕНИЯ ПАКЕТОВ ОШИБОК

Пакетом ошибок длины t называется любая последовательность t символов, первый и последний символы которой не равны нулю. В бесконечно длинном слове, принятом декодером сверточного кода, может появиться много ошибок, и мы можем считать, что эти ошибки собраны в пакеты различной длины. Если отдельные пакеты ошибок приходят не часто, то декодер в каждый момент времени содержит лишь один пакет ошибок. Сверточный код, для которого декодер может исправлять любой изолированный пакет ошибок длины t (при условии, что другие пакеты ошибок находятся достаточно далеко), называется сверточным кодом, способным исправлять пакеты ошибок длины t.

Очевидно, что любой сверточный (n, k)-код, исправляющий t ошибок, будет исправлять любой пакет ошибок длины t. Сверточный код для исправления более длинных пакетов может быть получен перемежением. Чтобы получить сверточный (jn, jk)-код из (n, k)-кода, возьмем j одинаковых (n, k)-кодеров и построим кодовые слова, чередуя их выходные символы. Если исходный код может исправлять любой пакет ошибок длины t, то, очевидно, код-перемежение может исправлять пакет ошибок длины jt. Например, систематический сверточный (14,7)-код с длиной кодового ограничения 6 и порождающими многочленами

$$g_1(x) = 1, \quad g_2(x) = x^6 + x^5 + x^2 + 1$$

может исправлять два любых ошибочных символа в любом интервале длины 14. Взяв четыре одинаковых (14,7)-кода и перемежая символы, можно получить (56,28)-код, который будет исправлять все пакеты ошибок длины 8.

Методом перемежения можно получить сверточный код из сверточного кода. Если $g(x)$ — порождающий многочлен исходного кода, то $g(x^l)$ — порождающий многочлен кода-перемежения. В приведенном выше примере порождающие многочлены после перемежения принимают вид

$$g_1(x) = 1, \quad g_2(x) = x^{24} + x^{20} + x^8 + 1.$$

Мы уже видели, что коды-перемежения для циклических блоковых кодов ведут себя аналогично.

Перемежая приведенные на рис. 12.13 короткие сверточные коды, исправляющие случайные ошибки, можно построить большое число сверточных кодов, исправляющих пакеты ошибок. Эти коды-перемежения будут исправлять не только пакеты ошибок, но и многие конфигурации случайных ошибок. Однако если требуется исправлять только пакеты ошибок, то можно получить лучшие характеристики, используя коды, предложенные специально для исправления пакетов ошибок. Классом таких кодов, способных исправлять любые пакеты ошибок длины не более λn_0, где λ — конструктивный параметр, являются коды Ивадаре.

Определение 12.7.1. Пусть λ и n_0 — любые положительные целые числа. *Кодом Ивадаре* называется исправляющий пакеты ошибок двоичный систематический сверточный код со следующей порождающей $((n_0 - 1) \times n_0)$-матрицей из многочленов:

$$\mathbf{G}(x) = \begin{bmatrix} 1 & & & g_1(x) \\ & 1 & & g_2(x) \\ & & \ddots & \vdots \\ & & 1 & g_{(n_0-1)}(x) \end{bmatrix}$$

где для матричных элементов $g_{in_0}(x)$ использовалось сокращенное обозначение $g_i(x)$ и

$$g_i(x) = x^{(\lambda+1)(2n_0-i)+i-3} + x^{(\lambda+1)(n_0-i)-1}, \quad i = 1, \ldots, n_0 - 1.$$

Наибольшую степень, равную $(\lambda + 1)(2n_0 - 1) - 2$, имеет порождающий многочлен $g_1(x)$. Поэтому коды Ивадаре являются сверточными $((m + 1)n_0, (m + 1)(n_0 - 1))$-кодами с числом кадров m, равным

$$m = (\lambda + 1)(2n_0 - 1) - 2.$$

Как мы увидим, этот код исправляет любые пакеты ошибок длины не более λn_0.

Проверочная матрица из многочленов такого кода равна

$$\mathbf{H}(x) = [g_1(x) \; g_2(x) \ldots g_{(n_0-1)}(x) \; 1].$$

Так как $n_0 - k_0 = 1$, имеется лишь один синдромный многочлен

$$s(x) = \sum_{i=1}^{n_0-1} g_i(x)\, e_i(x) + e_{n_0}(x) =$$

$$= e_{n_0}(x) + \sum_{i=0}^{n_0-1} [x^{(\lambda+1)(n_0-i)-1} + x^{(\lambda+1)(2n_0-i)+i-3}]\, e_i(x).$$

Декодер использует этот многочлен для обнаружения пакетов ошибок. Предположим, что пакет начинается в первом кадре и содержит λn_0 битов. Если он начинается не в первом бите первого кадра, то он может захватывать $(\lambda + 1)$-й кадр. Чтобы доказать, что этот код может исправлять такой пакет ошибок, необходимо показать, что декодер может восстановить по синдрому пакет ошибок, а также что пакет ошибок, начинающийся в более позднем кадре, не вызовет сбоя в работе декодера.

Декодирование кодов Ивадаре лучше всего пояснить на примере, поскольку структура всех этих кодов одинакова. В качестве примера рассмотрим (72,48)-код Ивадаре с $n_0 = 3$, $\lambda = 4$ и, следовательно, с $m + 1 = 24$. Кодер представлен на рис. 12.21.

Обозначим через $c_3(x)$ многочлен проверочных битов, а через $c_1(x)$ и $c_2(x)$ многочлены информационных битов. Соответствующими многочленами ошибок будут $e_1(x)$, $e_2(x)$ и $e_3(x)$. Синдромный многочлен равен

$$s(x) \quad e_3(x) + (x^4 + x^{19})\, e_2(x) + (x^9 + x^{23})\, e_1(x),$$

где для пакета ошибок, начинающегося в первом кадре,

$$e_3(x) = e_{30} + e_{31}x + e_{32}x^2 + e_{33}x^3,$$
$$e_2(x) = e_{20} + e_{21}x + e_{22}x^2 + e_{23}x^3 + e_{24}x^4,$$
$$e_1(x) = e_{10} + e_{11}x + e_{12}x^2 + e_{13}x^3 + e_{14}x^4.$$

Заметим, что если пакет начинается в первом кадре, то e_{34} равняется нулю. Если, далее, e_{24} отличен от нуля, то e_{20} и e_{10} равны нулю; если e_{14} отличен от нуля, то e_{10} равен нулю. Это следует из того, что длина пакета не превосходит 12. На рис. 12.22 представлены коэффициенты $s(x)$ для трех случаев, в которых пакет ошибок начинается в e_{10}, e_{20} или e_{30} соответственно. Порождающие многочлены были выбраны так, чтобы на каждый бит синдрома влиял лишь один ошибочный бит. Кроме того, заметим, что, хотя в начале многочлены перемежены и располагаются при формировании синдрома в порядке $e_1(x)$, $e_2(x)$, $e_3(x)$, над их образами затем производится обратное перемежение и они появляются в обратном порядке, т. е. в порядке $e_3(x)$, $e_2(x)$, $e_1(x)$.

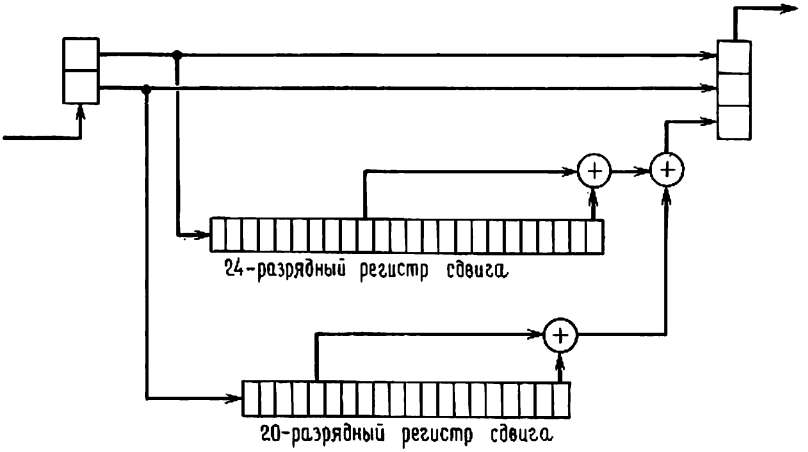


Рис. 12.21. Кодер для (72, 48)-кода Ивадаре.

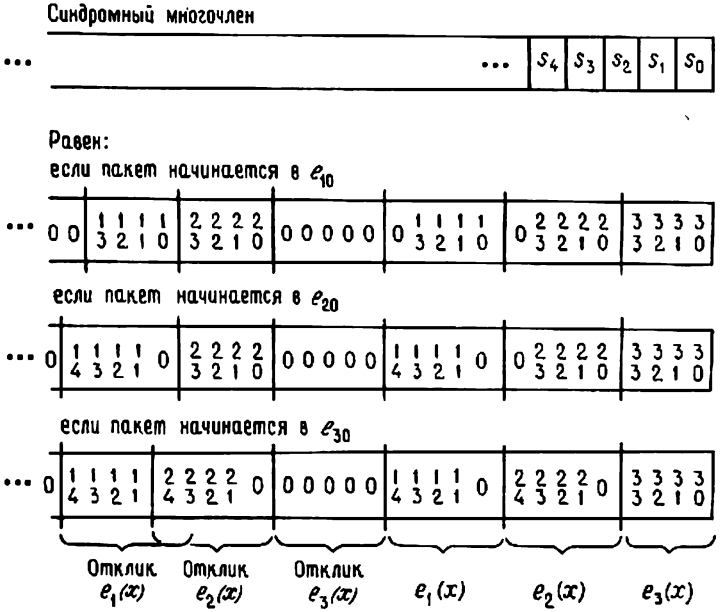


Рис. 12.22. Структура синдрома для (72, 48)-кода Ивадаре.

Каждый бит многочлена $e_3(x)$ воздействует на синдром дважды: за его первым появлением следует отклик с задержкой на 15 битов. Аналогично каждый бит многочлена $e_1(x)$ воздействует на синдром дважды: за его первым появлением следует отклик с за-

держкой на 14 битов. Так как e_{24} и e_{10} не могут одновременно быть ненулевыми, эти пары ошибок никогда не перекрываются. Каждый бит многочлена $e_3(x)$ воздействует на синдром один раз; за его первым появлением следуют только нули — один с задержкой на 14 битов, а другой с задержкой на 15 битов. Эти нули определяют $e_3(x)$.

Если в действительности пакет ошибок начинается в l-м кадре, то синдром сдвигается вправо на l битов и перед ним вставляется l нулей. Процедура декодирования построена таким образом, что эта конфигурация не будет восприниматься как пакет ошибок, начинающийся в первом кадре.

Перейдем к описанию декодера, изображенного на рис. 12.23. После того как в него поступили первые 20 кадров, в регистре синдрома хранятся первые 20 битов синдрома, причем s_0 располагается справа. Аналогично первый принятый бит многочлена $v_2(x)$ находится в крайнем правом разряде $v_2(x)$-регистра. В течение еще четырех последующих тактов первый принятый бит $v_1(x)$ не достигнет крайнего правого разряда $v_1(x)$-регистра. Описание работы декодера начнем с этого момента времени. Первые четыре бита синдрома зависят лишь от ошибок в принятых проверочных битах и нас не интересуют. Следовательно, та часть рис. 12.23,

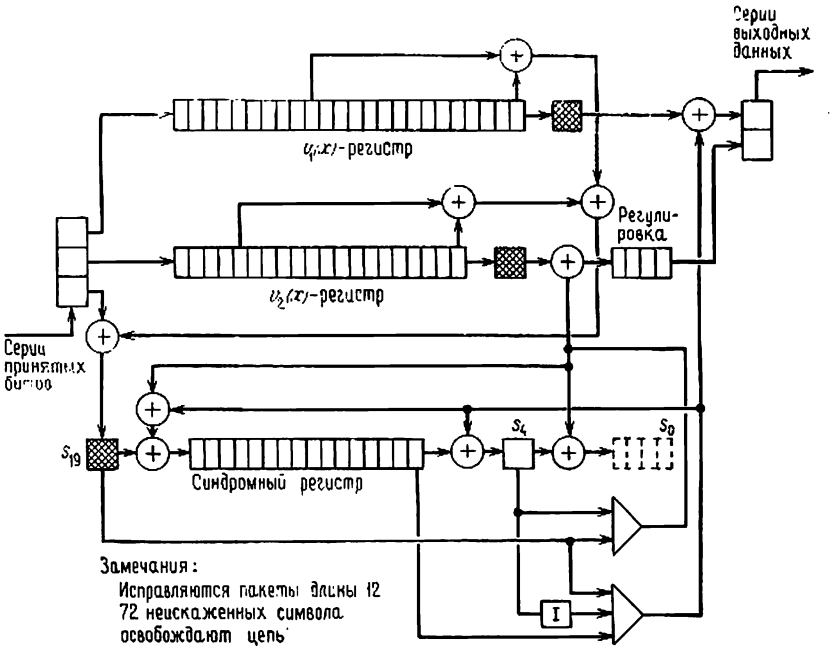


Рис. 12.23. Декодер для (72, 48)-кода Ивадаре.

которая нанесена штриховыми линиями, может быть опущена. Заштрихованные разряды регистра сдвига также могут быть опущены, а s_{19} может вводиться непосредственно из предшествующего сумматора по модулю 2.

Сначала декодер исправляет второй бит каждого кадра пакета ошибок, а затем возвращается к началу пакета и исправляет первый бит каждого кадра. Поэтому декодер имеет четырехразрядный регистр сдвига, в котором он хранит $c_4(x)$ после завершения исправления. Третий бит каждого кадра является проверочным; он не исправляется.

Для исправления ошибки в $v_2(x)$ декодер проверяет компоненты s_4 и s_{19} синдрома. Если обе компоненты равны единице, то правый бит $v_2(x)$ является ошибочным и в потоке данных, а также в регистре синдрома делаются соответствующие исправления. Пакет ошибок $e_3(x)$, начинающийся в более позднем кадре, не может вызвать сбоя, так как отклик $e_3(x)$ равен нулю. Параллельно с этим для нахождения ошибки в $v_1(x)$ декодер четырьмя кадрами позднее проверяет компоненты s_5 и s_{19} синдрома. Если они обе равны единице, а s_4 равна нулю, то правый бит $v_1(x)$-регистра ошибочен. Такая проверка $v_1(x)$ не приводит к нахождению ошибок в течение первых четырех тактов, но затем начинает исправлять ошибки в $v_1(x)$. Пакет $e_3(x)$, начинающийся в более позднем кадре, не может вызвать сбоя, так как отклик $e_3(x)$ равен нулю. В силу выбора способа проверки еще не исправленный бит $e_2(x)$ также не может вызвать сбоя, и поэтому s_4 равна нулю.

После появления пакета ошибок длины 12 принятое слово не должно содержать больше ошибок, чтобы синдрому ничто не мешало во время исправления этого пакета. Для этого необходимо, чтобы до 29-го кадра не происходило ошибок. Следовательно, между пакетами ошибок должно быть свободно от ошибок не менее 24 кадров (72 хороших бита). При выполнении этого правила декодер будет успешно исправлять следующие друг за другом пакеты ошибок длины не больше 12 каждый.

12.8. АЛГОРИТМ ДЕКОДИРОВАНИЯ ВИТЕРБИ

Алгоритм декодирования Витерби является полным алгоритмом декодирования сверточных кодов. Так как он является полным, вероятность отказа от декодирования равна нулю; однако при фиксированном коде вероятность ошибки декодирования будет больше, чем у неполного декодера. Этот алгоритм практически употребляется для двоичных кодов с малой длиной кодового ограничения — в настоящее время пределом являются длины кодового ограничения от 7 до 10. Недвоичные коды декодировать декодером Виберби труднее, и это возможно лишь при очень малых длинах кодового ограничения.

Ниже будет описана процедура, которая называется *декодированием по минимуму расстояния* и приводит к неплохому (в принципе) декодеру для сверточных кодов. Этот декодер является декодером по максимуму правдоподобия в случае двоичного шума без памяти. Выберем окно декодирования ширины b, превышающей длину блока n, вычислим все кодовые слова длины b, сравним принятое слово с каждым из них и выберем кодовое слово, ближайшее к принятому слову. Первый информационный кадр выбранного кодового слова берем в качестве первого информационного кадра декодированного кодового слова. Декодированный первый информационный кадр затем кодируется вновь и вычитается из принятого слова. После этого в декодер вводится n_0 новых символов, а первые n_0 символов сбрасываются, и процесс повторяется для нахождения следующего информационного кадра.

Конечно, для сверточных кодов даже весьма умеренной сложности эта процедура совершенно непрактична, однако знакомство с процедурой декодирования по минимуму расстояния полезно с теоретической точки зрения. В дальнейшем мы опишем практическую реализацию декодера по минимуму расстояния для коротких сверточных кодов.

Непосредственно реализовать декодер по минимуму расстояния трудно. Однако существует эффективный метод реализации такого декодера (в какой-то мере аналогичный использованию БПФ-алгоритма Кули — Тьюки для оценки дискретного преобразования Фурье и использованию алгоритма Берлекэмпа—Месси для синтеза авторегрессионного фильтра). Он называется алгоритмом Витерби и основывается на методах теории динамического программирования. Как и в других случаях, практически полезными оказываются трудные для понимания, но эффективные в вычислительном смысле алгоритмы. Поэтому теоретически лучше изучать декодер по минимуму расстояния, а в реальном декодере, если это возможно, использовать алгоритм Витерби.

Декодер Витерби итеративно обрабатывает кадр за кадром, двигаясь по решетке, аналогичной используемой кодером, и пытаясь повторить путь кодера. В любой момент времени декодер не знает, в каком узле находится кодер и поэтому не пытается декодировать этот узел. Вместо этого декодер по принятой последовательности определяет наиболее правдоподобный путь к каждому узлу и определяет расстояние между каждым таким путем и принятой последовательностью. Это расстояние называется *мерой расходимости* пути. Если все пути в множестве наиболее правдоподобных путей начинаются одинаково, то декодер, как правило, знает начало пути, пройденного кодером.

В следующем кадре декодер определяет наиболее правдоподобный путь к каждому из новых узлов этого кадра. Но путь в каждый новый узел должен пройти через один из старых узлов.

Возможные пути к новому узлу можно получить, продолжая к этому узлу те старые пути, которые можно к нему продолжить. Наиболее правдоподобный путь находится прибавлением приращения меры расходимости на продолжениях старых путей к мере расходимости путей, ведущих в старый узел. В каждый новый узел ведет q^v таких путей, и путь с наименьшей мерой расходимости является наиболее правдоподобным путем к новому узлу. Этот процесс повторяется для каждого из новых узлов. В конце итерации декодер знает наиболее правдоподобный путь к каждому из узлов в новом кадре.

Рассмотрим множество выживших путей, ведущих в множество узлов в r-й временно́й кадр. Эти пути проходят через один или более узлов в первом временно́м кадре. Если все пути проходят через один и тот же узел в первом временно́м кадре, то вне зависимости от того, в каком узле кодер находится в r-й временно́й кадр, мы знаем наиболее правдоподобный среди посещенных им в первый временно́й кадр узлов. Иначе говоря, мы знаем первый информационный кадр, хотя еще не приняли решение об r-м кадре.

Для построения декодера Витерби требуется выбрать ширину окна декодирования b, которая обычно в несколько раз превосходит длину блока, и оценить работу декодера путем моделирования на ЭВМ. В n-й временно́й кадр декодер проверяет, все ли выжившие пути имеют одинаковое первое ребро. Если первое ребро одинаково, то оно определяет первый декодированный информационный кадр, который подается на выход декодера.

Затем декодер отбрасывает первое ребро и использует новый кадр принятого слова для следующей итерации. Если вновь все выжившие пути проходят через тот же узел самого старого среди оставшихся кадров, этот информационный кадр декодируется. Такой процесс декодирования кадров продолжается бесконечно.

Если b выбрано достаточно большим, то во временно́м кадре почти всегда может быть принято однозначное решение. Если для данного канала код построен надлежащим образом, то это решение с большой вероятностью будет правильным. Этому, однако, может помешать несколько обстоятельств. Не все выжившие пути могут проходить через один и тот же узел. В этом случае процесс декодирования нарушается. Декодер может разрешить неопределенность, используя любое произвольное правило. Другая возможность состоит в том, что декодер не принимает решения, а отмечает этот участок как сегмент кодового слова, который невозможно исправить. В этом случае декодер становится неполным декодером. Лучше всего это понять на примере, который будет приведен ниже.

Иногда декодер принимает однозначное, но ошибочное решение. Оно обязательно сопровождается дополнительными ошибоч-

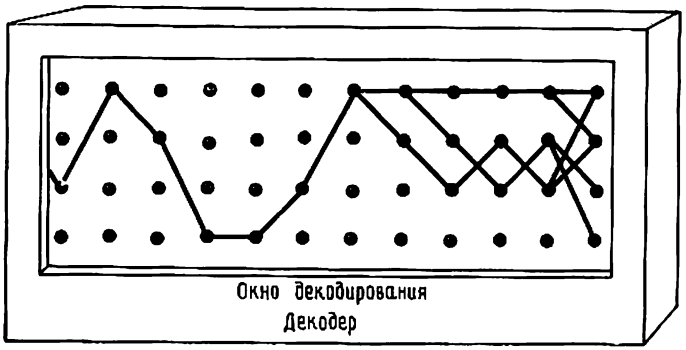


Рис. 12.24. Наглядное представление работы декодера Витерби.

ными решениями декодера, но в случае некатастрофического кода декодер через короткое время обнаружит это.

Полезно представлять себе декодер Витерби как окно, через которое можно наблюдать часть решетки. Это иллюстрируется рис. 12.24. На рисунке можно видеть только часть решетки конечной длины, на которой отмечены выжившие пути. С течением времени решетка скользит влево — на практике довольно быстро. При появлении справа новых узлов некоторые пути продолжаются до них, а другие исчезают; самый старый столбец выходит из-под наблюдения налево. Со временем левый столбец узлов исчезает, и (при нормальной работе) лишь для одного из узлов этого столбца будет существовать проходящий через него путь.

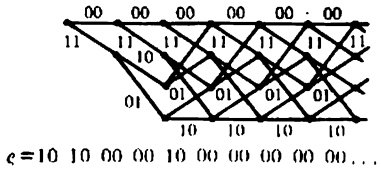
На рис. 12.25 приведен пример работы алгоритма Витерби в случае сверточного (6,3)-кода с порождающими многочленами $g_1(x) = x^2 + x + 1$ и $g_2(x) = x^2 + 1$. Кодер для этого кода показан на рис. 12.3. Для простоты выберем в качестве информационной последовательности нулевую и (тоже для простоты) декодер с окном декодирования b, равным 15. Предположим, что

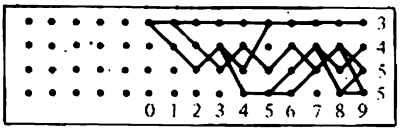
$$\mathbf{v} = 10100000100000000000\ldots;$$

это соответствует передаче нулевого кодового слова с тремя ошибками.

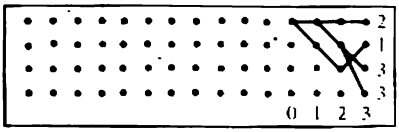
Диаграмма состояний декодера представлена на рис. 12.25. На третьей итерации декодер находит кратчайший путь к каждому узлу третьего кадра. Затем на r-й итерации декодер находит кратчайший путь к каждому узлу r-го кадра, продолжая пути, ведущие к узлам $(r - 1)$-го кадра и сохраняя кратчайший путь к каждому узлу.

Иногда может возникать неопределенность. В нашем примере имеются две неопределенности в пятой итерации. Декодер может либо разрешить неопределенность случайным образом, либо

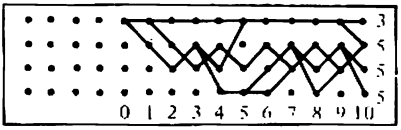




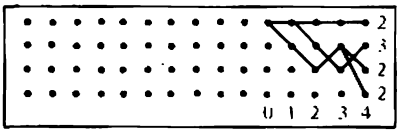
Итерация 9



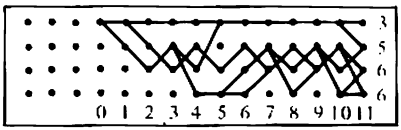
Итерация 3



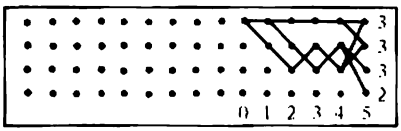
Итерация 10



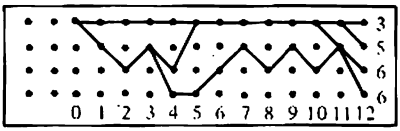
Итерация 4



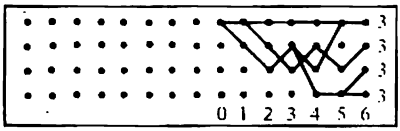
Итерация 11



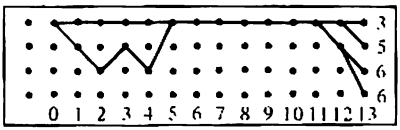
Итерация 5



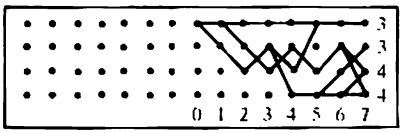
Итерация 12



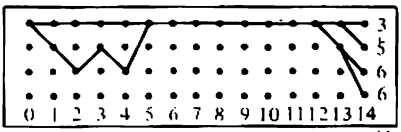
Итерация 6



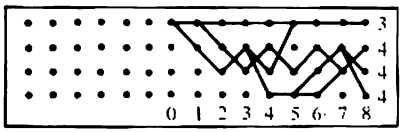
Итерация 13



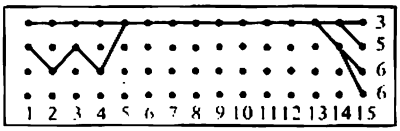
Итерация 7



Итерация 14



Итерация 8



Итерация 15

Рис. 12.25. Пример работы алгоритма Витерби.

сохранять оба пути, относительно которых существует неопределенность. В нашем примере неопределенности сохраняются до тех пор, пока не выявляется более правдоподобный путь или сомнительный участок не исчезает из поля зрения декодера.

По мере продвижения декодера к последующим кадрам из его памяти выводятся более ранние кадры. Если в самом старом кадре существует лишь один узел, через который проходит путь, то декодирование является полным. Если таких путей несколько, то принимается решение об обнаружении неисправляемой конфигурации ошибок. Она либо отмечается в качестве таковой, либо производится случайный выбор.

В нашем примере информационная последовательность может быть декодирована либо как

$$\mathbf{i} = 0\,0\,0\,0\,0\,0\,0\ldots,$$

либо как

$$\mathbf{i} = 1\,0\,1\,0\,0\,0\,0\ldots.$$

Если сделать ширину окна декодирования более 15, то ошибка по-прежнему останется неисправляемой. Для исправления этой конфигурации ошибок требуется код с бо́льшим свободным расстоянием.

Декодер, символически изображенный на рис. 12.25, при практической реализации выглядит совершенно иначе. Например, выжившие пути через решетку представляются таблицей 15-битовых чисел. На каждой итерации некоторые 15-битовые числа сдвигаются влево, причем левый бит сбрасывается, а справа добавляется новый бит. Другие 15-битовые числа не наращиваются, а сбрасываются из таблицы.

По мере продвижения декодера накопленная мера расходимости увеличивается. Для того чтобы избежать проблемы переполнения, необходимо время от времени уменьшать эту меру. Простая процедура состоит в периодическом вычитании из всех мер расходимости наименьшей такой меры. Очевидно, что это не повлияет на выбор максимального расхождения.

В случае когда длина кодового ограничения мала, декодеры Витерби могут работать очень быстро. Например, практически возможно построить декодер со 128 узлами на каждом уровне, который производит 10 миллионов итераций в секунду.

12.9. АЛГОРИТМЫ ПОИСКА ПО РЕШЕТКЕ

Можно улучшить характеристики сверточного кода, увеличив длину кодового ограничения. Декодер Витерби, однако, быстро становится непрактичным: при длине кодового ограничения 10 декодер двоичного кода уже должен помнить 1024 выживших путей.

Для того чтобы ослабить влияние больших длин кодового ограничения, была разработана стратегия, игнорирующая маловероятные пути по решетке как только они становятся маловероятными. При такой стратегии не принимается решений о том, чтобы окончательно оставить данный путь. Время от времени декодер решает вернуться назад и продолжить оставленный ранее путь.

Все такие стратегии поиска наиболее вероятного пути на решетке известны под общим названием *последовательного декодирования*. Последовательное декодирование носит довольно общий характер и может использоваться как в каналах, в которых принимаются жесткие решения (мы рассмотрим их в этом параграфе), так и в каналах, допускающих стирание или даже мягкие решения (мы изучим их в гл. 15).

Представим себе решетку сверточного кода с большой длиной кодового ограничения, скажем равной 40. Для того чтобы найти решение, эту решетку нужно исследовать на нескольких сотнях уровней. Еще больше будет число узлов на каждом уровне. Всего таких узлов будет 2^{40}, или приблизительно 10^{12}. Декодер должен найти по этой решетке путь, наиболее близкий к принятому слову в смысле расстояния Хэмминга - - по крайней мере он должен сделать это с очень высокой вероятностью, хотя время от времени ему дозволено отказываться от декодирования. Такой отказ является дополнительным видом ошибки, не существовавшим в декодере Витерби; декодер может отказаться от декодирования даже в том случае, когда декодирование по минимуму расстояния было бы верным.

В отличие от оптимальной процедуры алгоритма Витерби декодер, описание которого мы начинаем, просматривает только первый кадр, принимает решение и переходит в узел решетки на первом уровне. Затем он продолжает эту процедуру. На каждом уровне он находится в одном узле; он смотрит на следующий кадр, выбирая ребро, ближайшее к принятому кадру, и переходит в узел на следующем уровне.

Если ошибок нет, то эта процедура работает превосходно, однако при наличии ошибок декодер может случайно выбрать неправильную ветвь. Если декодер продолжает следовать по ложному пути, он внезапно обнаружит, что происходит слишком много ошибок. Но это ошибка декодера, а не канала. Последовательный декодер вернется назад на несколько кадров и начнет исследовать альтернативные пути до тех пор, пока не найдется правдоподобный путь; затем он будет следовать вдоль этого альтернативного пути. Ниже будут кратко описаны правила, которыми он руководствуется при этом поиске. Характеристики декодера зависят от ширины b окна декодирования, измеряемой числом кадров кодового слова. Если декодер нашел путь, проходящий через b кадров решетки, он принимает окончательное решение

относительно самого старого кадра, выводит соответствующие биты из окна и вдвигает в окно новый кадр.

Мы можем считать, что последовательный декодер движется вдоль слова сверточного кода с переменной скоростью. Обычно он двигается равномерно, подавая на выход исправленные выходные биты. Иногда возникает небольшое затруднение, и движение на некоторое время замедляется. Реже возникают серьезные трудности, и продвижение декодера прекращается до тех пор, пока он не преодолеет этот сложный участок. Разработаны подробные алгоритмы реализации такой процедуры. Наиболее популярным является алгоритм Фано, к описанию которого мы переходим.

В этом алгоритме требуется знать вероятность p появления ошибочного символа в канале или хотя бы верхнюю границу для p. Пока декодер следует по правильному пути, вероятное число ошибок в первых l кадрах приблизительно равно pn_0l. Декодер допускает несколько большее число ошибок, но если оно намного больше, то декодер сделает вывод, что он находится на ложном пути. Выберем (быть может, моделированием) некоторый параметр p', больший p, но меньший 1/2, и определим перекошенное расстояние формулой [1])

$$t(l) = p'n_0\, l - d\,(l),$$

где $d\,(l)$ — расстояние Хэмминга между принятым словом и текущим путем по решетке. Для правильного пути $d\,(l)$ приблизительно равно pn_0l, а $t(l)$ положительно и возрастает. До тех пор пока $t(l)$ возрастает, декодер продолжает следовать по пути на решетке. Если вдруг $t(l)$ станет уменьшаться, то декодер заключает, что в некотором узле он выбрал ошибочное ребро, и возвращается по решетке, проверяя другие пути. Он может найти лучший путь и проследовать по нему или может возвратиться к тому же самому узлу, но уже более уверенно и продолжить путь через него. Для того чтобы решить, когда $t(l)$ начинает уменьшаться, декодер пользуется текущим порогом T, который всегда кратен приращению Δ порога. Пока декодер движется вперед, порог, оставаясь меньшим $t(l)$ и кратным приращению Δ, принимает максимально возможное при этих ограничениях значение. Благодаря тому что T квантуется, допускается небольшое убывание $t(l)$ без пересечения порога.

В алгоритме Фано требуется, чтобы q^{k_0} ребер, выходящих из каждого узла, были перенумерованы согласно некоторому правилу упорядочения. Это правило ставит в соответствие каждому ребру индекс j, $j = 0, \ldots, q^{k_0} - 1$. Нет необходимости запомнить эти индексы в каждом ребре, достаточно знать правило,

[1]) В этом выражении знак выбран так, чтобы правильной работе декодера соответствовало положительное значение расстояния.

по которому при возвращении декодера в узел по ребру с известным индексом j он мог переупорядочить ребра, найти ребро j и, зная его, найти ребро $j+1$. Наиболее общим правилом является правило минимума расстояния. Ребра упорядочиваются в соответствии с их расстояниями Хэмминга до соответствующего кадра принятого слова, а неопределенность разрешается по любому удобному подправилу. Алгоритм Фано будет нормально функционировать и в том случае, когда ребрам приписан любой фиксированный порядок; например, ребра могут быть упорядочены лексикографически в соответствии с их информационными символами. Это последнее правило упорядочения приводит к более длительному поиску назад — вперед, но освобождает от необходимости вычисления и перевычисления порядка при достижении каждого узла. Какое правило упорядочения более просто в реализации, зависит от конструкции декодера. Для простоты понимания структуры алгоритма правило упорядочения лучше оставить несколько неопределенным; предположим лишь, что в каждом узле ребро, ближайшее к принятому ребру по расстоянию Хэмминга, считается первым. Декодер будет искать ребра из каждого узла в соответствии с этим правилом.

Реализация декодера в виде регистра сдвига, основанная на алгоритме Фано, приведена на рис. 12.26. Основой декодера является копия кодера с несколькими вспомогательными запоминающими регистрами. Декодер пытается подать символы в копию кодера таким образом, чтобы на его выходе появилось кодовое слово, достаточно близкое к принятому слову. После каждой итерации он может обратиться к последнему кадру, поданному в копию кодера. Он может изменить информацию в этом кадре, вернуться к более раннему кадру или ввести новый кадр. Декодер решает, что делать, основываясь на сравнении величины перекошенного расстояния $t(l)$ и текущего значения порога T.

Алгоритм Фано, который контролирует декодер, упрощенно представлен на рис. 12.27. Практические детали, связанные с конечностью размеров буфера, временно игнорируются.

Если шум в канале мал и ошибки редки, то декодер будет циркулировать по правой петле рис. 12.27, каждый раз сдвигая все регистры рис. 12.26 на один кадр вправо. До тех пор пока $t(l)$ остается выше порога, декодер продолжает сдвигаться вправо и повышать порог так, чтобы тот оставался близким к $t(l)$. Если $t(l)$ опускается ниже порога, то декодер проверяет альтернативные ребра этого кадра, пытаясь найти то ребро, которое находится выше порога. Если он не может сделать этого, то возвращается назад.

Как мы увидим в дальнейшем, если декодер начал возвращаться, то алгоритм заставит его двигаться назад до тех пор, пока он не найдет альтернативный путь, который находится над теку-

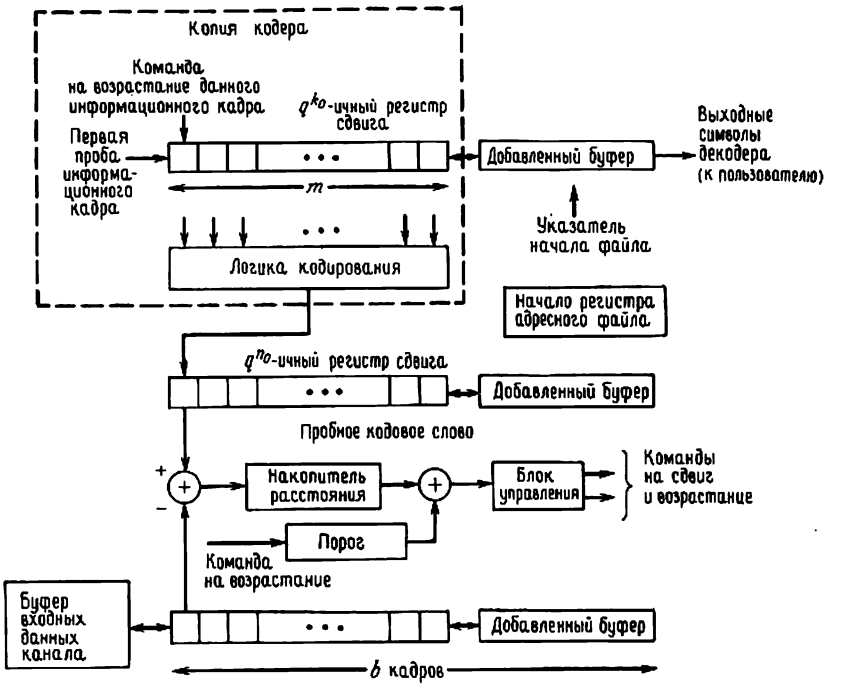


Рис. 12.26. Реализация алгоритма Фано в виде регистра сдвига.

щим значением порога, или узел, в котором был установлен текущий порог. Затем декодер снова двигается вперед уже с пониженным порогом; но теперь, как мы убедимся позднее, порог не повышается до тех пор, пока декодер не придет в новый узел, ранее не исследованный.

Каждый раз, когда декодер, двигаясь вперед, посещает ранее исследованный узел, он имеет меньший порог. Декодер никогда не посетит один и тот же узел дважды с одинаковыми значениями порога. Следовательно, он может посещать любой узел только конечное число раз. Это поведение гарантирует нас от зацикливания декодера. Декодер продолжает обработку данных, проводя правильное или неправильное декодирование.

Теперь необходимо доказать два сделанных ранее утверждения: 1) если декодер не может найти альтернативный путь, то он двигается назад к узлу, в котором было установлено текущее значение порога, и понижает его; 2) декодер не будет повышать порог до тех пор, пока не достигнет ранее не исследованного узла. Что касается первого утверждения, очевидно, что если декодер не может найти новое ребро, от которого он должен двигаться

вперед, то он должен в конце концов возвратиться к указанному узлу. Но если в предшествующем некоторому узлу кадре перекошенное расстояние $t(l-1)$ меньше текущего значения порога T, то порог был увеличен в l-м кадре. В блок-схему на рис. 12.27 включен этот тест на нахождение узла, в котором был установлен текущий порог; теперь в этом узле порог уменьшается.

Для того чтобы доказать второе утверждение, заметим, что после того, как порог понижен на Δ, декодер ищет следующие ребра в том же порядке, что и до этого, до тех пор, пока не найдет то место, в котором перекошенное расстояние, ранее бывшее меньше порога, становится больше него. До этого момента порог T не

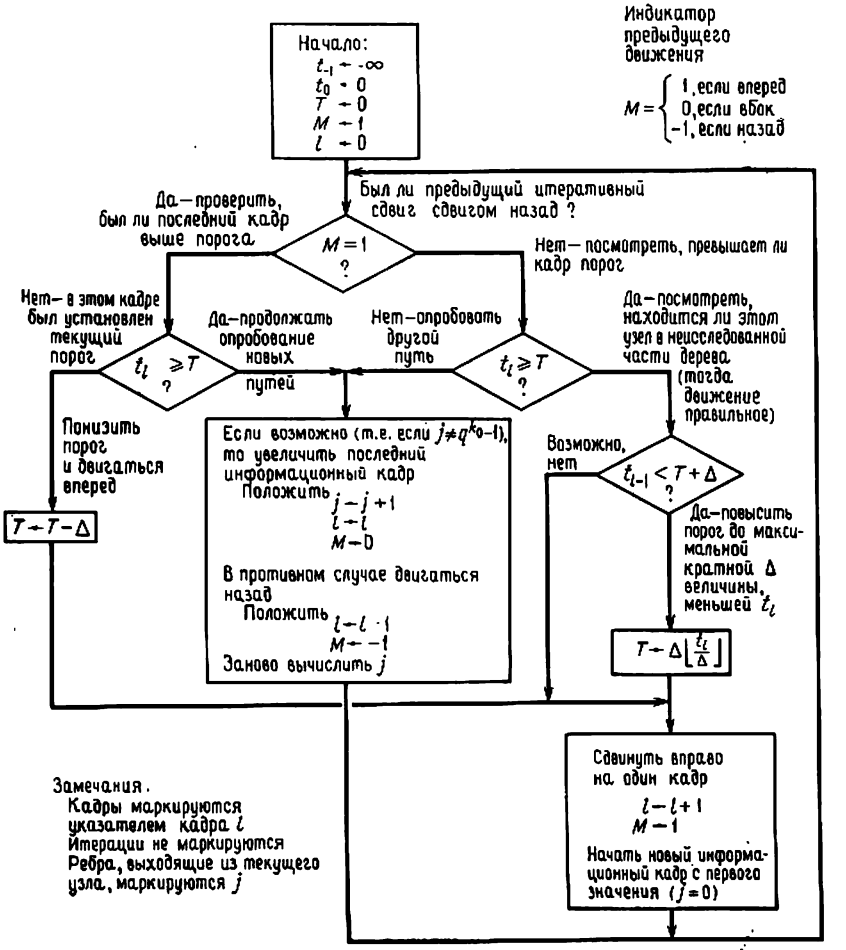


Рис. 12.27. Развернутый алгоритм Фано.

может изменяться. Это происходит из-за того, что после понижения порога на Δ перекошенное расстояние в тех узлах, где оно превышало первоначальный порог, никогда не будет меньше $T + \Delta$. Когда декодер продвигается в новую часть дерева, он в конце концов достигает состояния, в котором $t(l-1) < T + \Delta$ и $t(l) \geqslant T$. В этой точке порог повышается. Это и является тестом для определения того, новый ли узел был посещен, и нет необходимости помнить все посещенные ранее узлы. Такой тест включен в схему на рис. 12.27.

Алгоритм Фано зависит от двух параметров p' и Δ, которые могут быть выбраны при помощи моделирования на ЭВМ. На практике необходимо время от времени уменьшать $t(l)$ и T так, чтобы эти числа не становились слишком большими. Вычитание из обеих функций некоторого числа, кратного Δ, не влияет на последующие вычисления.

При практическом применении следует так же выбирать ширину b окна декодирования в несколько раз больше длины блока. На рис. 12.28 приведена более полная блок-схема алгоритма Фано, отражающая важную роль b. Каждый раз, когда самый старый информационный кадр достигает конца буфера, что отмечается указателем кадра, он выходит из декодера, а указатель кадра уменьшается таким образом, чтобы всегда указывать на самый старый имеющийся кадр. В противном случае декодер может попытаться возвратиться к кадру, уже вышедшему из декодера.

Иногда последовательный декодер делает так много вычислений, что величина входного буфера становится недостаточной. Это явление называется *переполнением буфера* и является существенным ограничением для применения алгоритма Фано. Вероятность переполнения буфера с ростом размера буфера уменьшается очень медленно, и поэтому вне зависимости от того, насколько большим выбран буфер, эта проблема полностью не снимается.

Существуют два способа управления переполнением буфера. Наиболее надежным является периодическая подача в кодер известной последовательности информационных символов (обычно последовательности нулей) с длиной, равной длине кодового ограничения сверточного кода. Если буфер переполнится, то декодер считает декодирование неудавшимся, ждет известного ему момента начала следующей последовательности и снова начинает декодирование. Все данные, поступившие между моментом переполнения буфера и следующим включением, теряются. Такой подход несколько уменьшает скорость кода и ставит при конструировании декодеров задачу синхронизации по времени.

В случае когда длина кодового ограничения не слишком велика, альтернативный путь состоит в движении указателя вперед. Если декодеру удастся найти правильный узел, то он может

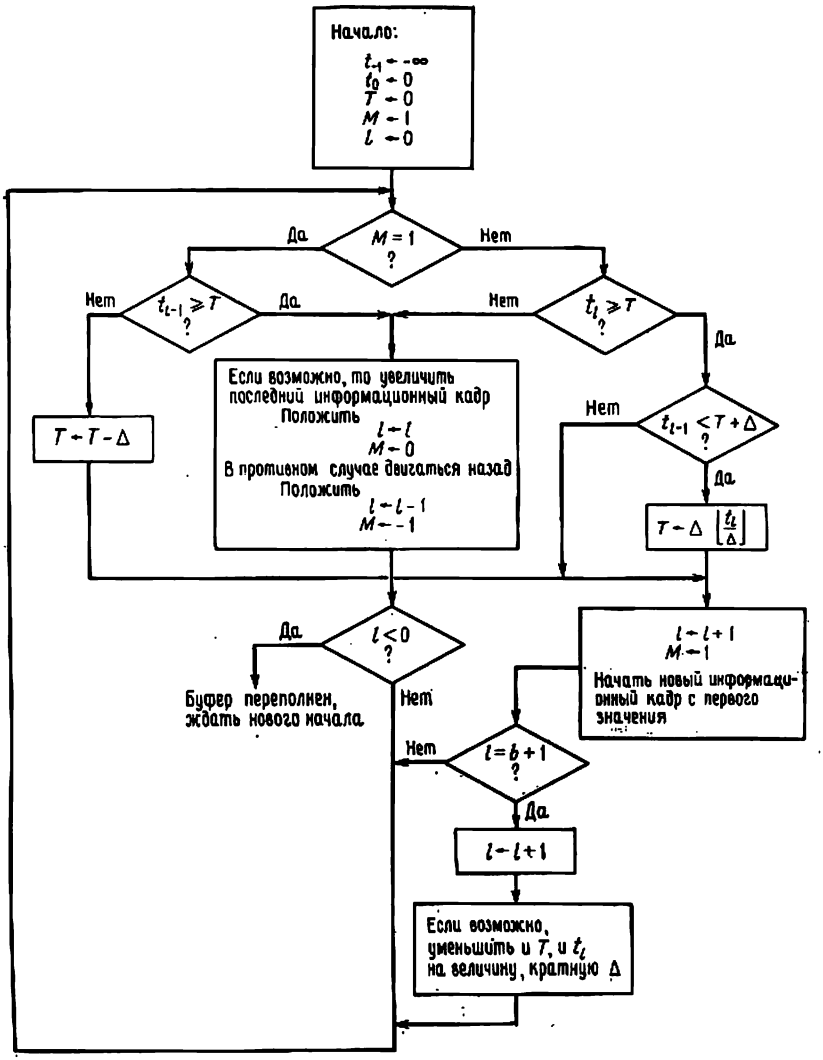


Рис. 12.28. Алгоритм Фано.

перенастроиться. Если вероятность того, что на длине блока не произойдет ошибок, не слишком мала, то декодер, предполагая, что ошибок нет, преобразует кодовое слово в информационное слово. Если код систематический, то это делается непосредственно; в противном случае необходимо применить китайскую теорему об остатках для многочленов. Если длина свободного от ошибок сегмента принятого слова равна длине кодового ограничения, то

декодер находит правильный узел и продолжает декодирование. Если в противоречии с нашим предположением появилась ошибка, то узел неправилен, и буфер скоро снова переполнится. Этот процесс продвижения декодера вперед продолжается до тех пор, пока не будет получено правильное декодирование.

ЗАДАЧИ

12.1.а. Построить кодер для систематического некатастрофического двоичного сверточного (16, 4)-кода с минимальным расстоянием $d^* = 9$ и длиной кодового ограничения $\nu = 3$.

б. Чему равно свободное расстояние этого кода?

12.2. Каждый код Хэмминга над $GF(q)$ может быть использован для построения сверточного кода над $GF(q)$, исправляющего одну ошибку.

а. Построить проверочную матрицу систематического восьмеричного (9, 7)-кода Хэмминга.

б. Основываясь на этом коде Хэмминга, найти проверочную матрицу (30, 27)-кода Вайнера—Эша над $GF(8)$.

в. Построить кодер и синдромный декодер.

г. Чему равна скорость этого кода? Чему равна скорость шестнадцатеричного кода Вайнера—Эша, основанного на (17, 15)-коде Хэмминга над $GF(16)$?

12.3. Построить кодер (12, 9)-кода Вайнера—Эша, использующий один двоичный регистр сдвига длины 3.

12.4. Построить кодер и синдромный декодер для (32, 28)-кода Вайнера—Эша.

12.5. Снова рассмотреть пример, приведенный на рис. 12.25, используя в качестве принятого слова

$$\mathbf{v} = 10001000001000000000\ldots .$$

12.6. Определить, какие из перечисленных ниже сверточных кодов со скоростью 1/2 являются катастрофическими:

а. $g_1(x) = x^2$, $\quad g_2(x) = x^3 + x + 1$.

б. $g_1(x) = x^4 + x^2 + 1$, $\quad g_2(x) = x^4 + x^3 + x + 1$.

в. $g_1(x) = x^4 + x^2 + x + 1$, $\quad g_2(x) = x^4 + x^3 + 1$.

г. $g_1(x) = x^6 + x^5 + x^4 + 1$, $\quad g_2(x) = x^5 + x^3 + x + 1$.

12.7. Сверточный код со скоростью 1/3 и длиной кодового ограничения 2 имеет порождающие многочлены $g_1(x) = x^2 + x + 1$, $g_2(x) = x^2 + x + 1$, $g_3(x) = x^2 + 1$ и d_∞, равное 8.

а. Чему равно минимальное расстояние этого кода?

б. Чему должна быть равна ширина окна декодирования для исправления всех тройных ошибок?

в. Построить исправляющий все двойные ошибки декодер для этого кода с шириной окна декодирования, равной 9.

12.8. Сверточный код над $GF(4)$ со скоростью 1/2 имеет порождающие многочлены $g_1(x) = 2x^3 + 2x^2 + 1$ и $g_2(x) = x^3 + x + 1$.

а. Показать, что этот код не является катастрофическим.

б. Показать, что его минимальное расстояние равно 5.

в. Сколько конфигураций из двух ошибок располагаются на длине ограничения декодирования, равной 8?

г. Построить синдромный декодер для исправления всех двойных ошибок.

12.9. Найти систематический кодер с регистром сдвига и с обратной связью для двоичного сверточного кода с порождающей матрицей

$$G(x) = \begin{bmatrix} x & x^2 & 1 \\ 0 & 1 & x \end{bmatrix}.$$

ЗАМЕЧАНИЯ

Понятие сверточного кода было введено Элайсом [1954] и развито Возенкрафтом [1957]. Вайнер и Эш [1963] предложили для семейства сверточных кодов, исправляющих одну ошибку, общую конструкцию, родственную конструкции кодов Хэмминга. Класс сверточных кодов, исправляющих многократные ошибки, нашел Месси [1963]. Коды Месси просты в декодировании, но в то же время имеют не слишком хорошие характеристики. Общего правила построения семейства быстродействующих высококачественных сверточных кодов, исправляющих многократные ошибки, не известно. Костелло [1969] предложил некоторые пути нахождения хороших сверточных кодов с умеренной длиной блока. Основные используемые в настоящее время коды были найдены поиском на ЭВМ Буссангом [1965], Оденвальдером [1970], Болом и Джелинеком [1971], Ларсеном [1973], Пааске [1974] и Йоханнессоном [1975]. Эти коды приведены на рис. 12.13.

Общее исследование алгебраической структуры сверточных кодов было проведено Месси и Сайном [1968], а также Форни [1970], который впервые использовал решетки. Наше описание таких кодов с помощью многочленов основывается на этих статьях. Костелло [1969] показал, что если в кодере введена обратная связь, то каждый сверточный код можно свести к систематическому. Дополнительное обсуждение абстрактных структур может быть найдено у Линднера и Штайгера [1977].

Создание декодеров для сверточных кодов начали со сложных декодеров с хорошими характеристиками, а затем постепенно перешли к простым декодерам с посредственными характеристиками, которые так популярны теперь. Алгоритм поиска на решетке принадлежит Фано [1963]. Наша формулировка алгоритма Фано следует Галлагеру [1968] и сильно отличается от первоначальной. Витерби [1967] предложил свой алгоритм скорее в учебных целях, чем в качестве серьезного алгоритма. Вскоре после этого Хеллер [1968] указал, что алгоритм Витерби при не слишком больших длинах кодового ограничения вполне практичен.

Общее обсуждение сверточных кодов можно найти в книге Витерби и Омуры [1970], а также в обзорных статьях Форни [1974] и Месси [1975].

ГЛАВА 13

КОДЫ И АЛГОРИТМЫ ДЛЯ ДЕКОДИРОВАНИЯ МАЖОРИТАРНЫМ МЕТОДОМ

Мажоритарное декодирование — это метод декодирования, который сравнительно прост в реализации. Если желательно иметь чрезвычайно быстрые декодеры, то нужно обратиться к мажоритарным декодерам. К сожалению, они могут декодировать чрезвычайно малый класс кодов, и эти коды, как правило, слабее других. Следовательно, для практики мажоритарное декодирование имеет второстепенное значение. Несмотря на это, некоторые практические требования могут быть удовлетворены только в его рамках. Кроме того, эти коды интересны с теоретической точки зрения и открывают новые возможности теории кодов, контролирующих ошибки.

Большинство известных кодов, которые могут быть декодированы мажоритарным методом, — это циклические коды или расширенные циклические коды. Для этих кодов мажоритарные декодеры всегда могут быть реализованы как декодеры Меггитта и отличаются особенно простой древовидной логикой для проверки синдрома. Поэтому с прагматической точки зрения мажоритарно декодируемые коды можно определить как те циклические коды, для которых декодер Меггитта может рассматриваться как стандартный. Однако путь нахождения этих кодов весьма сложен и запутан.

13.1. ДЕКОДИРОВАНИЕ МАЖОРИТАРНЫМ МЕТОДОМ

В кодах Рида—Маллера, изучавшихся в § 3.6, декодирование каждого информационного символа производилось путем нахождения большинства голосов в множестве проверочных равенств. Существуют и другие коды, которые могут быть декодированы мажоритарно. В ретроспективе история таких кодов обычн прослеживается до кодов Рида—Маллера.

Напомним, что любой линейный (n, k)-код над $GF(q)$ имеет проверочную матрицу $\mathbf{H}$, а кодовые слова удовлетворяют равен-

ству $\mathbf{c}\mathbf{H}^{\mathrm{T}} = 0$. Если ограничиться рассмотрением j-й строки матрицы $\mathbf{H}$, то получается проверочное равенство

$$\sum_{i=0}^{n-1} H_{ij}c_i = 0.$$

Взяв любую линейную комбинацию строк $\mathbf{H}$, можно образовать новое проверочное равенство. Всего таким путем можно образовать q^{n-k} проверочных равенств. Искусство мажоритарного декодирования и состоит в выборе хорошего подмножества этих проверок (при условии, что оно существует).

В этом и следующем параграфе мы займемся построением мажоритарных декодеров. В дальнейших параграфах мы изучим построение кодов, допускающих мажоритарное декодирование.

Определение 13.1.1. Множество проверочных равенств называется *согласующимся* [1] в k-й координате, если компонента c_k входит в каждое проверочное равенство этого множества и каждая компонента c_j ($j \neq k$) входит не более чем в одно проверочное равенство этого множества.

Как будет показано в приведенной ниже теореме, мажоритарный декодер оценивает e_k по большинству голосов в проверочных равенствах. Априори не известно, произошла или нет ошибка в k-м принятом бите, и равная нулю величина ошибки — это один из кандидатов при таком голосовании. Если число J проверочных равенств четно, то при одинаковом числе голосов предпочтение отдается решению об отсутствии ошибок. Если J нечетно и число ошибок меньше $(J-1)/2$, то правильное решение всегда принимается большинством голосов.

Теорема 13.1.2. *Если множество J проверочных равенств согласуется в k-й координате и если в принятом слове произошло не более $J/2$ ошибок, то компонента c_k оценивается правильно.*

Доказательство состоит в описании процедуры исправления. Так как компонента c_k входит в каждое проверочное равенство этой совокупности, H_{kj} не равно нулю. Положим

$$s_j = H_{kj}^{-1} \sum_{i=0}^{n-1} H_{ij}v_i = H_{kj}^{-1} \sum_{i=0}^{n-1} H_{ij}e_i.$$

Деление на H_{kj} гарантирует, что коэффициент при e_k в сумме равен единице. Поскольку существует J равенств, то $s_j = e_k$ по меньшей мере для половины значений j; не более $J/2$ других e_i отличны от нуля, и каждое из этих e_i входит в множество проверочных равенств не более одного раза. Следовательно, не менее $J/2$

[1] Здесь часто используется термин *ортогональный*. Мы избегаем этого термина, так как он широко используется в теории векторных пространств.

всех s_j равны e_k, если e_k равно нулю, и более $J/2$ всех s_j равны e_k, если e_k отлично от нуля. Поэтому e_k находится мажоритарным решением по s_j. □

Следствие 13.1.3. *Если для каждой координаты существует J проверочных равенств, которые согласуются в этой координате, то код может исправлять $\lfloor J/2 \rfloor$ ошибок.*

Доказательство следует непосредственно. □

Если J четно, то $J/2$ является целым числом и код может исправлять $J/2$ ошибок. Отсюда следует, что минимальное расстояние кода не меньше $J + 1$. Прямое доказательство этого факта, включая случай нечетных J, дается в приведенной ниже теореме. Естественно назвать $J + 1$ *расстоянием, реализуемым при мажоритарном декодировании*; оно иногда обозначается через $d_{\text{мд}}$. Истинное минимальное расстояние может быть больше.

Теорема 13.1.4. *Если для каждой координаты линейного кода существует J проверочных равенств, которые согласуются в этой координате, то минимальное расстояние кода не меньше $J + 1$.*

Доказательство. Выберем любое ненулевое кодовое слово и любую координату k, в которой находится ненулевой символ. В координате k согласуются J проверочных равенств. Каждое из них включает ненулевую компоненту c_k и поэтому должно включать хотя бы одну другую ненулевую компоненту, поскольку в правых частях всех проверочных равенств стоят нули. Следовательно, существует не менее J других ненулевых компонент. □

Следствие 13.1.5. *Если для некоторой координаты циклического кода существует J проверочных равенств, которые согласуются в этой координате, то минимальное расстояние кода не меньше $J + 1$.*

Доказательство. Каждый циклический сдвиг кодового слова в циклическом коде образует другое кодовое слово того же кода; поэтому множество J проверочных равенств для одной координаты можно использовать для того, чтобы выписать множество проверочных равенств для любой другой координаты. □

Теорема 13.1.6. *Пусть $\tilde{d}$ — минимальное расстояние кода, дуального коду $\mathscr{C}$ над GF (q). Тогда мажоритарный декодер для $\mathscr{C}$ может исправить не более $(n - 1)\,(\tilde{d} - 1)^{-1}/2$ ошибок.*

Доказательство. Линейные комбинации строк матрицы $\mathbf{H}$ образуют кодовые слова в дуальном коде $\mathscr{C}^{\perp}$. Рассмотрим множество J проверочных равенств, согласующихся в первой координате. Каждое равенство содержит не менее $\tilde{d} - 1$ ненулевых компонент среди оставшихся $n - 1$ координат и каждая из оставшихся $n - 1$ координат не равна нулю по крайней мере в одном из J равенств.

Следовательно, $J(\bar{d}-1) \leqslant n-1$, и мажоритарный декодер может исправлять $J/2$ ошибок. □

Итак, при мажоритарном декодировании нельзя декодировать на радиусе упаковки кода, если не выполняется неравенство

$$d^* \leqslant (n-1)/(\bar{d}-1)+1.$$

Это условие необходимо, но не достаточно. Для большинства представляющих практический интерес кодов это условие не выполняется.

Мажоритарный декодер довольно прост, но, вообще говоря, применим лишь к кодам с плохими характеристиками. Поэтому мы введем нечто более сложное, а именно L-шаговый мажоритарный декодер.

Двухшаговый мажоритарный декодер использует мажоритарное решение для локализации ошибки в множестве компонент, а не в конкретной компоненте. Затем для нахождения ошибки уже в этом множестве вновь используется мажоритарное решение. L-шаговый мажоритарный декодер использует мажоритарную логику на L уровнях. На каждом уровне декодер исходит из того, что ошибка уже локализована в некотором множестве компонент, и, используя мажоритарное решение, локализует ошибку в подмножестве этих компонент.

Определение 13.1.7. Множество проверочных соотношений называется согласующимся в множестве координат $i_1, i_2, \ldots, i_r$, если существует множество коэффициентов $A_1, A_2, \ldots, A_r$, такое, что сумма $A_1c_{i_1}+A_2c_{i_2}+\cdots+A_rc_{i_r}$ входит в каждое проверочное равенство данного множества и каждая компонента c_i при $i \neq i_1, i_2, \ldots, i_r$ входит не более чем в одно проверочное равенство множества.

Естественно, L-шаговый мажоритарный декодер может исправлять больше ошибок, чем одношаговый мажоритарный декодер, но, вообще говоря, все еще не позволяет достичь радиуса упаковки кода, как показывает следующая теорема.

Теорема 13.1.8. *Пусть $\bar{d}$ — минимальное расстояние кода, дуального коду $\mathscr{C}$ над GF (q). Тогда L-шаговый мажоритарный декодер для $\mathscr{C}$ может исправлять не более $n/\bar{d}-1/2$ ошибок.*

Доказательство. Чтобы исправлять t ошибок, прежде всего необходимо составить $J=2t$ проверочных равенств, согласующихся в некотором множестве B, содержащем b координат. Каждое такое равенство соответствует линейной комбинации строк матрицы $\mathbf{H}$, и эти линейные комбинации представляют собой кодовые слова дуального кода $\mathscr{C}^{\perp}$. Пусть число ненулевых компонент в l-м проверочном равенстве, не считая компонент, коорди-

наты которых принадлежат B, равно a_l, $l = 1, \ldots, J$. Эти равенства соответствуют кодовым словам дуального кода, и поэтому

$$b + a_l \geqslant d.$$

Суммируя эти J соотношений, получаем

$$Jb + \sum_{l=1}^{J} a_l \geqslant Jd.$$

Так как каждая компонента, координата которой не принадлежит множеству B, может быть ненулевой не более чем в одном проверочном равенстве, то

$$\sum_{l=1}^{J} a_l \leqslant n - b.$$

Исключая b, получаем

$$Jn + \sum_{l=1}^{J} a_l \geqslant J\left(d + \sum_{l=1}^{J} a_l\right).$$

Теперь необходимо вывести второе условие. Разность двух кодовых слов дуального кода также представляет собой кодовое слово, у которого в b позициях стоят нули. Отсюда следует, что при $l \neq l'$

$$a_l + a_{l'} \geqslant d.$$

Существует $J(J-1)$ таких равенств, и каждое a_l входит в $2(J-1)$ из них. Складывая все такие равенства, имеем

$$2(J-1)\sum_{l=1}^{J} a_l \geqslant J(J-1)\,d.$$

Наконец, исключим $\sum_{l=1}^{J} a_l$, используя это равенство и полученное ранее. Это дает

$$2J(n-d) \geqslant J(J-1)\,d.$$

Так как мажоритарный декодер может исправлять $J/2$ ошибок, отсюда следует утверждение теоремы. □

Из этой теоремы, в частности, следует, что (23,12)-код Голея не может быть декодирован мажоритарным методом.

13.2. СХЕМЫ МАЖОРИТАРНОГО ДЕКОДИРОВАНИЯ

Интерес к мажоритарно декодируемым кодам объясняется тем, что декодеры для них просты и быстры. Поэтому важной частью изложения должно быть описание декодеров. Изучим работу декодеров на примерах.

В качестве первого примера рассмотрим (7,3)-код, дуальный (7,4)-коду Хэмминга. Этот код, известный под названием *симплексного*, имеет минимальное расстояние 4 и может исправлять одну ошибку. Так как $1 \leqslant (7-1)\,(3-1)^{-1}/2 = 3/2$, теорема 13.1.6 не исключает возможности мажоритарного декодирования. Один из способов нахождения проверочной матрицы для этого кода основан на том, что в циклическом коде α^0 и α^3 являются корнями порождающего многочлена, лежащими в $GF(8)$. Следовательно,

$$\mathbf{H} = \begin{bmatrix} \alpha^0 & \alpha^0 & \alpha^0 & \alpha^0 & \alpha^0 & \alpha^0 & \alpha^0 \\ \alpha^4 & \alpha & \alpha^5 & \alpha^2 & \alpha^6 & \alpha^3 & \alpha^0 \end{bmatrix}.$$

Над $GF(2)$ эта матрица принимает вид

$$\mathbf{H} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 \end{bmatrix}.$$

В тех проверочных равенствах, которые включают c_0, правая компонента равна единице. Существует восемь таких равенств, коэффициенты которых определяются следующим образом:

$$\begin{array}{c} 1\ 1\ 1\ 1\ 1\ 1\ 1 \\ 0\ 0\ 1\ 0\ 1\ 1\ 1 \\ 0\ 1\ 0\ 0\ 0\ 1\ 1 \\ 0\ 0\ 0\ 1\ 1\ 0\ 1 \\ 1\ 0\ 0\ 1\ 0\ 1\ 1 \\ 1\ 1\ 0\ 0\ 1\ 0\ 1 \\ 1\ 0\ 1\ 0\ 0\ 0\ 1 \\ 0\ 1\ 1\ 1\ 0\ 0\ 1 \end{array}$$

Выбрав третью, четвертую и седьмую строки, получим равенства

$$\begin{aligned} c_0 + c_1 + c_5 &= 0, \\ c_0 + c_2 + c_3 &= 0, \\ c_0 + c_4 + c_6 &= 0, \end{aligned}$$

образующие искомое множество трех равенств, которые согласуются в первой координате, содержащей c_0. Основываясь на них, получим декодер, изображенный на рис. 13.1. Если первоначально регистр сдвига содержит принятый вектор, то в случае, когда произошла одна ошибка, после семи сдвигов он будет содержать исправленное кодовое слово.

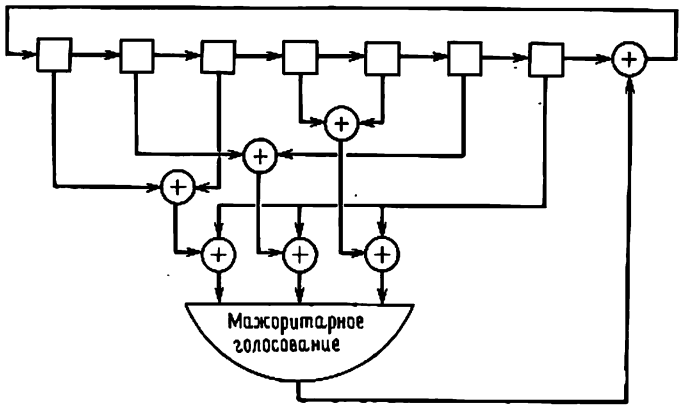


Рис. 13.1. Мажоритарный декодер для симплексного (7, 3)-кода.

Конечно, для исправления одной ошибки необходимо использовать лишь два из приведенных выше проверочных равенств. Добавление третьего равенства приводит к декодеру, основанному на правиле решения «два из трех», а не на правиле «два из двух». До тех пор пока происходят одиночные ошибки, оба декодера ведут себя одинаково, но, если произошло две ошибки, поведение декодеров становится различным. Декодер «два из трех» исправляет несколько бо́льшую долю конфигураций из двух ошибок, хотя оба декодера не исправляют или неправильно исправляют большинство конфигураций из двух ошибок. При желании можно потребовать, чтобы удовлетворялись все три проверочных равенства. Тогда декодер будет исправлять все одиночные и обнаруживать все двойные ошибки.

Можно превратить мажоритарный декодер в декодер Меггитта. Напомним, что синдромный многочлен определялся или как

$$s(x) = R_{g(x)}[v(x)],$$

или как

$$s(x) = R_{g(x)}[x^{n-k} v(x)].$$

Через проверочную матрицу синдромный многочлен линейно связан с принятым многочленом $v(x)$. Запишем каждый коэффициент в виде

$$s_j = \sum_{i=0}^{n-1} H_{ij} v_i, \quad j = 0, \ldots, n-k-1.$$

Любое другое проверочное равенство получается как некоторая линейная комбинация строк **H**, т. е. как линейная комбинация компонент синдрома. Поэтому все проверочные соотношения,

используемые мажоритарным декодером, могут быть получены как линейная комбинация коэффициентов синдрома декодера Меггитта. Таким образом, любой мажоритарный декодер для циклического кода может быть реализован как декодер Меггитта.

Симплексный (7,3)-код является циклическим, и его порождающий многочлен равен $g(x) = x^4 + x^2 + x + 1$. Пусть

$$s(x) = R_{g(x)}[x^4 v(x)] =$$

$$= v_6 x^3 + v_5 x^2 + v_4 x + v_3 + v_2 (x^3 + x + 1) + v_1 (x^3 + x^2 + x) +$$
$$+ v_0 (x^2 + x + 1).$$

Это соотношение можно переписать в виде

$$[s_0\ s_1\ s_2\ s_3] = [v_0\ v_1\ v_2\ v_3\ v_4\ v_5\ v_6] \begin{bmatrix} 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Для мажоритарного декодирования будем использовать следующие проверочные равенства:

$$c_0 + c_1 + c_5 = 0,$$
$$c_0 + c_2 + c_3 = 0,$$
$$c_0 + c_4 + c_6 = 0.$$

Выражение в левой части первого из них совпадает с s_2, второе — с s_0, а третье — с $s_1 + s_3$. На рис. 13.2 показан мажоритарный декодер, реализованный как декодер Меггитта.

Мажоритарные декодеры даже без каких-либо дополнительных модификаций декодируют много ошибочных конфигураций и вне радиуса упаковки кода. Это может показаться заманчивым преимуществом мажоритарных декодеров. Однако, как правило, мажоритарное декодирование применяется для кодов, у которых радиус упаковки мал по сравнению с другими кодами с сопоставимыми скоростью и длиной. Способность декодера декодировать вне радиуса упаковки можно рассматривать лишь как частичную компенсацию за выбор кода с худшими характеристиками.

Для того чтобы мажоритарный декодер, реализованный как декодер Меггитта, исправлял большое число конфигураций ошибок вне радиуса упаковки, необходимо добавить цепь обратной

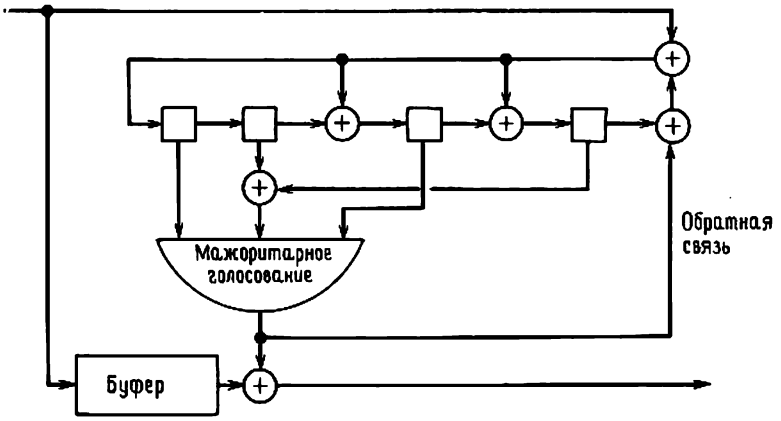


Рис. 13.2. Декодер Меггитта для симплексного (7, 3)-кода.

связи; такая цепь показана на рис. 13.2. Если декодирование вне радиуса упаковки не играет особой роли, то цепь обратной связи можно опустить.

13.3. АФФИННЫЕ ПЕРЕСТАНОВКИ ДЛЯ ЦИКЛИЧЕСКИХ КОДОВ

Циклический сдвиг любого кодового слова в циклическом коде дает другое кодовое слово. Следовательно, весь код инвариантен относительно циклического сдвига. Циклический сдвиг является простым примером перестановки. Циклический код может быть инвариантен и относительно других перестановок. В этом параграфе мы покажем, что многие циклические коды, будучи расширенными, становятся инвариантными относительно большой группы перестановок, называемых аффинными. Аффинные перестановки могут использоваться для построения кодов, для их исследования или для построения декодеров, в частности мажоритарных.

Пусть $\mathscr{C}$ — циклический код длины $n = q^m - 1$, порождаемый многочленом $g(x)$, и пусть $\overline{\mathscr{C}}$ — код длины q^m, получаемый из $\mathscr{C}$ добавлением к каждому кодовому слову $\mathscr{C}$ символа проверки на четность. Иначе говоря, если $(c_{n-1}, \ldots, c_1, c_0)$ — кодовое слово в $\mathscr{C}$, то $(c_{n-1}, \ldots, c_1, c_0, c_\infty)$ — кодовое слово в $\overline{\mathscr{C}}$; здесь c_∞ — символ проверки на четность, определяемый как $c_\infty = -(c_{n-1} + \ldots + c_1 + c_0)$.

Чтобы перенумеровать компоненты кодового слова, будем использовать $GF(q^m)$ как множество локаторов. Ненулевые элементы $GF(q^m)$ суть α^i, где α — примитивный элемент. Нулевой

элемент 0 в $GF(q^m)$ будем обозначать как α^∞. Перенумеруем следующим образом компоненты вектора $(c_{n-1}, \ldots, c_1, c_0, c_\infty)$ из $\bar{\mathscr{C}}$ элементами $GF(q^m)$: компонента c_∞ нумеруется локатором α^∞, а компоненты c_i при $0 \leqslant i < q^m - 1$ — локаторами α^i.

Группа перестановок называется *транзитивной*, если для любой пары локаторов (X, Y) в кодовом слове существует перестановка в группе, меняющая их местами; при этом возможно также изменение порядка других локаторов. Группа перестановок называется *дважды транзитивной*, если для любых двух пар локаторов $((X_1, Y_1), (X_2, Y_2))$, где $X_1 \neq X_2$, $Y_1 \neq Y_2$, существует такая перестановка в группе, которая меняет местами локаторы как в первой, так и во второй паре. При этом также возможно изменение порядка других локаторов.

Аффинная перестановка — это перестановка, которая переводит компоненту с локатором X в компоненту с локатором $aX + b$, где a и b — любые фиксированные элементы из $GF(q^m)$ и $a \neq 0$. Множество всех аффинных перестановок образует группу относительно композиции, так как: 1) если локатор X переходит в локатор $Y = aX + b$ и локатор Y переходит в локатор $Z = a'Y + b'$, то X переходит в $Z = a'aX + a'b + b'$ и 2) перестановка $a^{-1}X - a^{-1}b$ обратна перестановке $aX + b$. Группа аффинных перестановок дважды транзитивна, так как при заданных парах (X_1, Y_1) и (X_2, Y_2) система уравнений

$$Y_1 = aX_1 + b,$$
$$Y_2 = aX_2 + b$$

решается относительно a и b единственным образом.

Теорема 13.3.1. *Любой код длины $n = q^m$, инвариантный относительно группы аффинных перестановок, можно преобразовать в циклический код, отбросив позицию с локатором α^∞.*

Доказательство. Пусть α — примитивный элемент, используемый для задания локаторов. Перестановка $Y = \alpha X$ аффинна. Но она представляет собой циклический сдвиг всех локаторов, кроме α^∞. Следовательно, код является циклическим. □

Несколько труднее сформулировать обратные условия, т. е. условия, при которых можно расширить циклический код и получить код, инвариантный относительно группы аффинных перестановок. Такие циклические коды, о которых пойдет речь в теореме 13.3.4, называются циклическими кодами, обладающими свойством дважды транзитивной инвариантности. Этой теореме будет предшествовать небольшое общематематическое отступление.

Определение 13.3.2. Пусть j — целое число в q-ичном разложении:

$$j = j_0 + j_1 q + j_2 q^2 + \cdots + j_{m-1} q^{m-1}, \quad 0 \leqslant j_i < q.$$

Отличное от j целое число k в q-ичном разложении:

$$k = k_0 + k_1 q + k_2 q^2 + \cdots + k_{m-1} q^{m-1}, \quad 0 \leqslant k_i < q,$$

называется *q-ичным потомком числа j*, если

$$k_l \leqslant j_l, \quad l = 0, \ldots, m-1.$$

Выяснить, является ли данное целое k q-ичным потомком целого j, может оказаться затруднительным. В случае когда q равно простому числу p, для нахождения простого эквивалентного условия, записанного через сочетания $\binom{j}{k}$, будет использоваться приведенная ниже теорема (напомним, что при $m > n$ принято считать $\binom{n}{m} = 0$).

Теорема 13.3.3 (теорема Лукаса). *Пусть p — простое число, и пусть*

$$j = j_0 + j_1 p + \cdots + j_{m-1} p^{m-1}$$

и

$$k = k_0 + k_1 p + \cdots + k_{m-1} p^{m-1}$$

— p-ичные разложения двух произвольных целых чисел j и k. Тогда справедливо следующее соотношение:

$$\binom{j}{k} = \prod_{i=0}^{m-1} \binom{j_i}{k_i} \pmod{p}.$$

Кроме того, $\binom{j}{k}$ равно нулю по модулю p тогда и только тогда, когда k не является p-ичным потомком числа j.

Доказательство. Второе утверждение следует из первого, так как $\binom{j_i}{k_i}$ при всех i не равно нулю по модулю p тогда и только тогда, когда k является p-ичным потомком j. Необходимо доказать лишь первое утверждение теоремы.

Доказательство состоит в разложении многочлена $(1 + x)^j$ двумя различными способами и последующем приравнивании коэффициентов при одинаковых степенях x. Используя теорему 4.6.10, можно записать следующее тождество над $GF(p)$:

$$(1 + x)^{p^i} = 1 + x^{p^i}.$$

Тогда для произвольного j имеем

$$(1 + x)^j = (1 + x)^{j_0 + j_1 p + j_2 p^2 + \cdots + j_{m-1} p^{m-1}} =$$

$$= (1 + x)^{j_0} (1 + x^p)^{j_1} (1 + x^{p^2})^{j_2} \ldots (1 + x^{p^{m-1}})^{j_{m-1}}.$$

Используя биномиальное разложение обеих частей этого равенства, перепишем его в следующем виде:

$$\sum_{k=0}^{j} \binom{j}{k} x^k =$$

$$= \left[\sum_{k_0=0}^{j_0} \binom{j_0}{k_0} x^{k_0} \right] \left[\sum_{k_1=0}^{j_1} \binom{j_1}{k_1} x^{k_1 p} \right] \cdots \left[\sum_{k_{m-1}=0}^{j_{m-1}} \binom{j_{m-1}}{k_{m-1}} x^{k_{m-1} p^{m-1}} \right],$$

где каждое k_i не больше j_i и поэтому меньше p. Далее приравняем коэффициенты при x^k и получим

$$\binom{j}{k} = \sum \binom{j_0}{k_0} \binom{j_1}{k_1} \cdots \binom{j_{m-1}}{k_{m-1}} \pmod{p},$$

где суммирование ведется по всем m-совокупностям $(k_0, k_1, \ldots, k_{m-1})$, в которых каждая компонента меньше p и

$$k = k_0 + k_1 p + k_2 p^2 + \cdots + k_{m-1} p^{m-1}.$$

Выражение в правой части этого равенства представляет собой p-ичное разложение k, являющееся единственным. Поэтому сумма состоит лишь из одного члена и вырождается в равенство

$$\binom{j}{k} = \binom{j_0}{k_0} \binom{j_1}{k_1} \cdots \binom{j_{m-1}}{k_{m-1}} \pmod{p}.$$

Это завершает доказательство теоремы. □

Теперь мы можем охарактеризовать те циклические коды, которые инвариантны относительно группы аффинных перестановок. В формулировку приведенной ниже теоремы входят условия, которым должны удовлетворять корни порождающего многочлена. Порождающие многочлены $g(x)$ с корнем α^0 из рассмотрения исключаются, так как в противном случае выражения становятся неопределенными.

Теорема 13.3.4. *Пусть α — примитивный элемент поля $GF(q^m)$ характеристики p, пусть $\mathscr{C}$ — циклический код длины $q^m - 1$, порождаемый многочленом $g(x)$, у которого α^0 не является корнем, и пусть $\overline{\mathscr{C}}$ — расширенный код, получаемый из него добавлением символа проверки на четность. Расширенный код $\overline{\mathscr{C}}$ инвариантен относительно группы аффинных перестановок тогда и только тогда, когда из того, что α^k — корень $g(x)$, а k' — ненулевой p-ичный потомок k, следует, что $\alpha^{k'}$ также является корнем $g(x)$.*

Доказательство. Пусть $X' = aX + b$ означает аффинную перестановку. Пусть $X_1, X_2, \ldots, X_\nu$ — локаторы ненулевых компонент кодового слова c, и пусть значения этих компонент обозначены через $Y_1, Y_2, \ldots, Y_\nu$. Пусть, далее, $X_1', X_2', \ldots, X_\nu'$ озна-

чают локаторы ненулевых компонент кодового слова при аффинной перестановке, т. е. $X_l' = aX_l + b$.

Вначале предположим, что если α^k корень $g(x)$, то при любом k', являющемся ненулевым p-ичным потомком k, $\alpha^{k'}$ будет корнем $g(x)$. Необходимо доказать, что перестановка кодового слова порождает другое кодовое слово.

Кодовый многочлен удовлетворяет соотношению $c(x) = g(x)\,d(x)$, а добавленный символ равен $c_\infty = -(c_{n-1} + \cdots + c_0)$. Пусть $C_j = c(\alpha^j) = \sum_{l=0}^{n-1} \alpha^{jl} c_l$. Тогда

$$C_j = \sum_{i=0}^{n-1} c_i \alpha^{jl} = \sum_{l=0}^{\nu} Y_l X_l^j = 0$$

при всех j, для которых α^j является корнем $g(x)$. Заметим, что в локаторе α^∞ может быть ненулевой символ. Но α^∞ есть представление нулевого элемента; поэтому соответствующий X_l является нулем. Даже в тех случаях, когда слагаемое $Y_l X_l^j$ при равном нулю X_l не входит в определение C_j, можно считать его входящим, так как оно равно нулю.

Переставленное слово c' является кодовым, если выполняется соотношение $c'_\infty + c'_{n-1} + \cdots + c'_0 = 0$, остающееся, очевидно, справедливым и после перестановки, и если, кроме того,

$$C_j' = \sum_{l=1}^{\nu} Y_l X_l'^j = 0$$

при всех j, для которых α^j является корнем $g(x)$. Рассмотрим такое j:

$$C_j' = \sum_{l=1}^{\nu} Y_l (aX_l + b)^j =$$

$$= \sum_{l=1}^{\nu} Y_l \sum_{k=0}^{j} \binom{j}{k} a^k X_l^k b^{j-k} = \sum_{k=0}^{j} \binom{j}{k} b^{j-k} a^k C_k.$$

По теореме 13.3.3 $\binom{j}{k}$ равно нулю по модулю p, если k не является p-ичным потомком j; для тех же k, которые являются p-ичными потомками j, по предположению C_k равно нулю. Следовательно, в каждом слагаемом суммы или $\binom{j}{k}$ равно нулю, или C_k равно нулю. Поэтому C_j' равно нулю и переставленное кодовое слово снова является кодовым словом.

Теперь докажем обратное. Предположим, что расширенный код инвариантен относительно группы аффинных перестановок. Тогда каждое кодовое слово удовлетворяет равенству

$$C_j' = \sum_{l=1}^{\nu} Y_l (aX_l + b)^j = 0$$

для всех a и b и всех j, для которых α^j является корнем $g(x)$. Как и ранее, имеем

$$C'_j = \sum_{k=0}^{l} \binom{l}{k} b^{j-k} a^k C_k = 0.$$

Пусть K — число p-ичных потомков j; перенумеруем их числами k_l при $l = 1, \ldots, K$. Запишем сумму по ним в виде

$$C'_j = \sum_{l=0}^{K} \binom{l}{k_l} b^{j-k_l} a^{k_l} C_{k_l} = 0.$$

Пусть теперь a и b произвольны. Полагая b равным 1, а a поочередно равным первым K последовательным степеням α, получаем

$$\begin{bmatrix} (\alpha^{k_1})^0 & (\alpha^{k_2})^0 & \ldots & (\alpha^{k_K})^0 \\ (\alpha^{k_1})^1 & (\alpha^{k_2})^1 & \ldots & (\alpha^{k_K})^1 \\ \cdot & \cdot & & \cdot \\ \cdot & \cdot & & \cdot \\ \cdot & \cdot & & \cdot \\ (\alpha^{k_1})^{K-1} & (\alpha^{k_2})^{K-1} & \ldots & (\alpha^{k_K})^{K-1} \end{bmatrix} \begin{bmatrix} \binom{j}{k_1} C_{k_1} \\ \binom{j}{k_2} C_{k_2} \\ \vdots \\ \binom{j}{k_K} C_{k_K} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \cdot \\ \cdot \\ \cdot \\ 0 \end{bmatrix}.$$

Эта матрица — матрица Вандермонда. Она обратима, так как все ее столбцы различны. Поэтому $C_{k_l} = 0$ при $l = 1, \ldots, K$. Следовательно, α^{k_l} — корень $g(x)$ при всех k_l, являющихся потомками j в p-ичном представлении. Это завершает доказательство теоремы. □

13.4. ЦИКЛИЧЕСКИЕ КОДЫ, ОСНОВАННЫЕ НА ПЕРЕСТАНОВКАХ

Теперь опишем метод построения некоторых допускающих одношаговое мажоритарное декодирование кодов, основанный на группе аффинных перестановок (см. § 13.3). Этот метод может применяться для нахождения циклических кодов над $GF(q)$ длины n при условии, что n — составное число, а именно что $n = L \cdot J$. Кроме того, необходимо предположить, что L не сравнимо с нулем по модулю p, где p — характеристика $GF(q)$. Это условие всегда удовлетворяется для тех n, которые являются примитивной длиной или ее делителем.

Выберем длину кода n примитивной и составной; $n = q^m - 1$ и $n = L \cdot J$. Тогда

$$x^n - 1 = (x^J)^L - 1 = \\ = (x^J - 1)(x^{J(L-1)} + x^{J(L-2)} + \ldots + x^J + 1)^L.$$

Обозначим второй член этого разложения через $a(x)$, так что

$$x^n - 1 = (x^J - 1)\, a(x).$$

Ненулевые элементы $GF(q^m)$ являются корнями или $x^J - 1$, или $a(x)$. Если α — примитивный элемент $GF(q^m)$, то $\alpha^{LJ} = 1$ и $\alpha^{L\cdot(J-1)}$, $\alpha^{L(J-2)}$, α, ..., α^L, 1 являются J корнями $x^J - 1$. Поэтому при каждом j, не кратном L, α^j является корнем $a(x)$.

Определим многочлен $\hat{h}(x)$, такой, что взаимный ему многочлен является проверочным многочленом рассматриваемого кода. При всех j, которые не кратны L и q-ичный потомок которых не кратен L, α^j является корнем $\hat{h}(x)$. Так как каждый корень $\hat{h}(x)$ является корнем $a(x)$, многочлен $a(x)$ кратен $\hat{h}(x)$.

Пусть $h(x)$ — многочлен, взаимный к $\hat{h}(x)$, и пусть $g(x)$ определяется соотношением

$$g(x)\, h(x) = x^n - 1.$$

Мы получили два циклических кода, являющихся дуальными. Пусть $\mathscr{C}$ — циклический код с порождающим многочленом $g(x)$, а $\mathscr{C}^\perp$ — циклический код с порождающим многочленом $\hat{h}(x)$. По определению $\hat{h}(x)$ и по теореме 13.3.4 $\mathscr{C}^\perp$ — циклический код, обладающий свойством дважды транзитивной инвариантности. Поэтому можно расширить $\mathscr{C}^\perp$ до кода $\overline{\mathscr{C}^\perp}$, который является инвариантным относительно группы аффинных перестановок.

Составим J проверочных равенств для циклического кода $\mathscr{C}$, согласующихся в одной координате. Так как код циклический, он мажоритарно декодируем и его минимальное расстояние не меньше $J + 1$. Найдем указанные проверочные равенства для кода $\mathscr{C}$, оперируя с дуальным кодом $\mathscr{C}^\perp$.

Как мы видели ранее, α^j является корнем многочлена

$$a(x) = x^{J(L-1)} + x^{J(L-2)} + \cdots + x^J + 1$$

тогда и только тогда, когда j не кратно L. Следовательно, $a(x)$ кратно $\hat{h}(x)$ и является кодовым словом дуального кода. Поэтому все элементы множества многочленов

$$\{a(x),\ xa(x),\ x^2a(x),\ \ldots,\ x^{J-1}\, a(x)\}$$

являются кодовыми словами $\mathscr{C}^\perp$. Вес Хэмминга каждого из указанных кодовых слов равен L, и из рассмотрения $a(x)$ следует, что никакие два из них не имеют общей ненулевой компоненты.

Теперь найдем другое множество кодовых слов из $\mathscr{C}^\perp$, которое может использоваться для задания проверок кода $\mathscr{C}$. Временно припишем в $\mathscr{C}^\perp$ добавочный символ. Впоследствии этот символ будет служить своеобразной точкой опоры при выписывании J согласующихся проверочных равенств. Чтобы этот добавленный символ можно было передвигать внутрь кодового слова, допустим

возможность выбрасывания компонент кода, включая и вновь добавленный символ.

К каждому из кодовых слов этого множества добавим символ проверки на четность. Это дает J кодовых слов в расширенном коде $\overline{\mathscr{C}^{\perp}}$ длины $\bar{n} = n + 1$. Если L не сравнимо с нулем по модулю p, то добавленный символ отличен от нуля и одинаков для каждого из J построенных кодовых слов. Разделим кодовые слова на этот добавленный символ и получим новое множество кодовых слов. Таким образом мы установили существование в $\overline{\mathscr{C}^{\perp}}$ множества J кодовых слов со следующими свойствами:

1) каждое кодовое слово из множества J кодовых слов имеет единицу в локаторе α^{∞};

2) одно и только одно кодовое слово из множества J кодовых слов имеет ненулевую компоненту в локаторе α^j при $j = 0, 1, \ldots, n-1$.

Теперь мы можем использовать теорему 13.3.4. В самом деле, предыдущие определения были выбраны таким образом, чтобы можно было применить эту теорему. Так как код $\overline{\mathscr{C}^{\perp}}$ обладает свойством дважды транзитивной инвариантности, он инвариантен относительно любой аффинной перестановки. В частности, выберем

$$Y = \alpha X + \alpha^{n-1}.$$

Эта перестановка переводит множество кодовых слов $\overline{\mathscr{C}^{\perp}}$ в другое множество кодовых слов $\overline{\mathscr{C}^{\perp}}$, обладающее следующими свойствами:

1) каждое кодовое слово из вновь построенного множества J кодовых слов имеет единицу в локаторе α^{n-1};

2) одно и только одно кодовое слово из вновь построенного множества J кодовых слов имеет ненулевую компоненту в локаторе α^j при $j = \infty, 0, 1, \ldots, n-2$.

Теперь можно опустить локатор α^{∞} и получить множество кодовых слов в $\mathscr{C}^{\perp}$, которое согласуется в координате $n-1$. В самом деле, эти J кодовых слов ортогональны кодовым словам кода $\mathscr{C}$ и поэтому образуют проверочные равенства. Полученные J проверочные равенства согласуются в координате $n-1$, и код является циклическим. Теперь мы можем доказать следующую теорему.

Теорема 13.4.1. *Пусть $n = q^m - 1$ может быть представлено в виде $n = J \cdot L$. Предположим, что порождающий многочлен циклического кода $\mathscr{C}$ над GF (q) строится по следующему правилу. Для каждого целого числа j элемент α^j является корнем многочлена $g(x)$ тогда и только тогда, когда j или любой его q-ичный потомок кратны числу L. Тогда $\mathscr{C}$ является мажоритарно*

декодируемым кодом над GF (q) с минимальным расстоянием не меньше J.

Доказательство. По определению $\hat{h}(x)$ α^{-j} не является корнем $g(x)$ тогда и только тогда, когда α^j является корнем $\hat{h}(x)$. Поэтому $\hat{h}(x)$ имеет корень α^j тогда и только тогда, когда ни j, ни любой его q-ичный потомок не являются ненулевыми кратными L. Следовательно, как мы видели при рассмотрении дуального кода, для каждой координаты имеется J согласующихся в ней проверочных равенств.

Код является кодом над $GF(q)$, если каждый сопряженный с корнем многочлена $g(x)$ элемент также является корнем этого многочлена. Если α^{-j} является корнем $g(x)$, то, поскольку j кратно L, qj кратно $q \cdot L$ (по модулю LJ), и, следовательно, α^{-jq} также корень $g(x)$. Если же α^{-j} является корнем $g(x)$ вследствие того, что некоторый q-ичный потомок j' числа j кратен L, то q-ичный потомок qj' (mod LJ) числа qj (mod LJ) кратен qL (mod LJ). □

Проиллюстрируем теорему простым примером над $GF(2)$. Так как

$$x^{15} - 1 = (x^3 - 1)(x^{12} + x^9 + x^6 + x^3 + 1) =$$

$$= (x^5 - 1)(x^{10} + x^5 + 1),$$

мы можем выбрать или $J = 3$ и $L = 5$, или $J = 5$ и $L = 3$, соответственно получив код, исправляющий одиночные ошибки, или код, исправляющий двойные ошибки. Построим код, исправляющий одиночные ошибки.

Ненулевые кратные 5 равны 5 и 10; число 5 является двоичным потомком чисел 7 и 13; число 10 является двоичным потомком чисел 11 и 14. Следовательно, корнями $\hat{h}(x)$ являются α, α^2, α^3, α^4, α^6, α^8, α^9 и α^{12}, где α — примитивный элемент поля $GF(16)$, а корнями $g(x)$ являются α, α^2, α^4, α^5, α^8, α^{10} и α^0. Следовательно,

$$g(x) = (x - 1)(x - \alpha)(x - \alpha^2)(x - \alpha^4)(x - \alpha^5)(x - \alpha^8) \times$$

$$\times (x - \alpha^{10}) = x^7 + x^3 + x + 1,$$

и код имеет восемь информационных битов.

В табл. 13.1 приведены параметры некоторых из рассмотренных выше кодов, а также соответствующих кодов БЧХ. В каждом случае приведено минимальное расстояние, полученное из соответствующих границ. Действительное минимальное расстояние может быть больше.

Из таблицы видно, что в нескольких случаях выбор мажоритарно декодируемого кода не приводит к ощутимому уменьшению k, хотя в большинстве случаев оно значительно. Важно, однако, понять, что мажоритарный декодер прост и быстр и, как правило,

Таблица 13.1

Сравнение параметров некоторых двоичных кодов, декодируемых в один шаг, с параметрами кодов БЧХ

	Мажоритарно декодируемые		БЧХ	
n	k	$d_{\text{мд}}$	k	$d_{\text{БЧХ}}$
15	8	3	11	3
	6	5	7	5
63	48	3	57	3
	36	9	39	9
	12	21	18	21
255	224	3	247	3
	206	5	239	5
	174	17	191	17
	36	51	91	51
	20	85	47	85
511	342	7	484	7
	138	73	241	73
1023	960	3	1013	3
	832	11	973	11
	780	33	863	33
	150	93	598	93
	30	341	123	341

может исправлять большое число конфигураций более чем из J ошибок. Поэтому перед окончательным выбором применяемого кода полезно провести моделирование.

13.5. СВЕРТОЧНЫЕ КОДЫ С МАЖОРИТАРНЫМ ДЕКОДИРОВАНИЕМ

Некоторые сверточные коды могут декодироваться мажоритарным декодером. Точно так же, как и в случае блоковых кодов, эти декодеры просты и чрезвычайно быстры, но применяемые коды не так мощны, как другие. Предпочтительнее использовать коды с лучшими характеристиками.

Мажоритарно декодируемые коды декодируются путем нахождения большинства голосов в некоторой совокупности синдромов или в некоторой совокупности линейных комбинаций синдромов. Несколько мажоритарно декодируемых сверточных кодов найдено путем поиска на ЭВМ. Некоторые из них затабулированы ниже.

Сверточный (12,6)-код, порождаемый кодером, представленным на рис. 12.3, и декодируемый декодером, представленным на рис. 12.20, может декодироваться и мажоритарно. Чтобы продемонстрировать природу мажоритарных декодеров, мы опишем два

различных мажоритарных декодера для этого кода. На рис. 13.3 приведены синдромы для всех конфигураций одиночных ошибок. Таблица содержит модифицированные синдромы, вычисленные по двум различным правилам. Заметим, что для обеих правил модифицированные синдромы согласуются в первой позиции.

Конфигурации одиночных ошибок	Синдромы $s_5\ s_4\ s_3\ s_2\ s_1\ s_0$	Модифицированные синдромы Правило 1 $s'_3\ s'_2\ s'_1\ s'_0$	Модифицированные синдромы Правило 2 $s''_3\ s''_2\ s''_1\ s''_0$
0 0 0 0 0 0 0 0 0 0 0 1	1 1 1 0 0 1	1 1 1 1	1 1 1 1
0 0 0 0 0 0 0 0 0 0 1 0	0 0 0 0 0 1	0 0 0 1	0 0 0 1
0 0 0 0 0 0 0 0 0 1 0 0	1 1 0 0 1 0	0 1 0 0	1 0 0 0
0 0 0 0 0 0 0 0 1 0 0 0	0 0 0 0 1 0	0 0 1 0	0 1 0 0
0 0 0 0 0 0 0 1 0 0 0 0	1 0 0 1 0 0	0 0 0 0	1 0 0 0
0 0 0 0 0 0 1 0 0 0 0 0	0 0 0 1 0 0	0 1 0 0	0 0 0 0
0 0 0 0 0 1 0 0 0 0 0 0	0 0 1 0 0 0	1 0 0 0	0 0 1 0
0 0 0 0 1 0 0 0 0 0 0 0	0 0 1 0 0 0	1 0 0 0	0 0 1 0
0 0 0 1 0 0 0 0 0 0 0 0	0 1 0 0 0 0	0 0 1 0	0 1 0 0
0 0 1 0 0 0 0 0 0 0 0 0	0 1 0 0 0 0	0 0 1 0	0 1 0 0
0 1 0 0 0 0 0 0 0 0 0 0	1 0 0 0 0 0	0 1 0 0	1 0 0 0
1 0 0 0 0 0 0 0 0 0 0 0	1 0 0 0 0 0	0 1 0 0	1 0 0 0

Правило 1	Правило 2
$s'_0 = s_0$	$s''_0 = s_0$
$s'_1 = s_1 + s_4$	$s''_1 = s_3$
$s'_2 = s_2 + s_5$	$s''_2 = s_1 + s_4$
$s'_3 = s_3$	$s''_3 = s_5$

Рис. 13.3. Модифицированные синдромы для сверточного (12, 6)-кода.

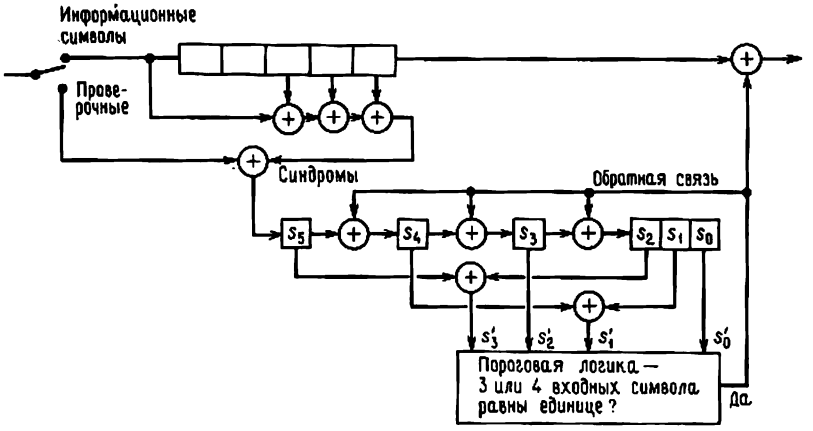


Рис. 13.4. Мажоритарный декодер для сверточного (12, 6)-кода.

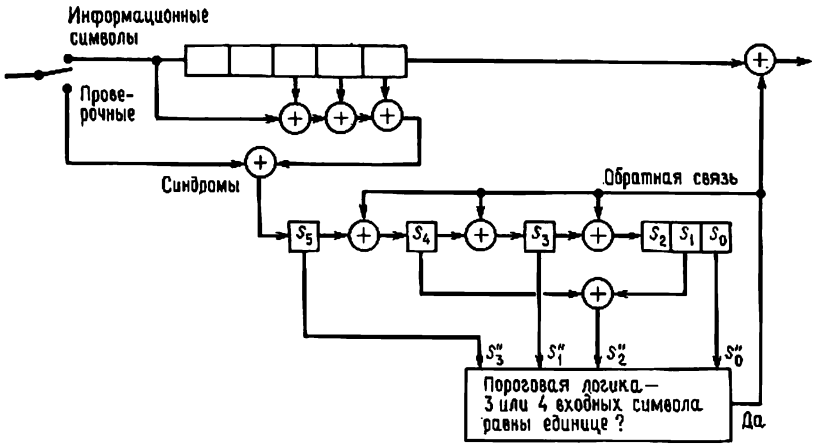


Рис. 13.5. Другой мажоритарный декодер.

Так как в обоих случаях используются четыре модифицированных синдрома, ошибка в первой позиции может быть исправлена даже в том случае, когда произошла вторая ошибка. На рис. 13.4 изображен мажоритарный декодер, основанный на первой совокупности модифицированных синдромов. Этот декодер исправляет все конфигурации из двух ошибок и проще декодера на рис. 12.20. Однако он обладает существенным недостатком, присущим многим мажоритарным декодерам сверточных кодов и состоящим в следующем. Некоторые конфигурации более чем из двух ошибок будут удовлетворять проверке по большинству и приводить к неправильному исправлению. Для части из них сигнал обратной связи будет изменять синдром таким образом, что даже при отсутствии дальнейших ошибок он будет воспроизводить некоторый синдром. Например, при отсутствии последующих ошибок синдром 011010 не приводит к исправлению ошибок, следующий за ним синдром 001101 приводит к исправлению одной ошибки, а следующий за ним синдром 011010 возобновляет цикл. Такое явление называется *обычным распространением ошибок*. Оно присуще мажоритарному декодеру и может наступать даже для некатастрофического кода. В нормальном синдромном декодере обычное распространение ошибок не может иметь места.

На рис. 13.5 изображен другой мажоритарный декодер того же кода, но уже основанный на второй из представленных на рис. 13.3 совокупностей модифицированных синдромов. В таком декодере не происходит обычного распространения ошибок. Этот пример показывает, что в некоторых мажоритарных декодерах сверточных кодов происходит обычное распространение ошибок, которого, впрочем, можно избежать путем тщательного выбора

(n_0, k_0)	(n, k)	$t_{\text{МД}}$	$G(x)$
(2, 1)	(6, 3)	1	$[1 \quad x+1]^t$
(2, 1)	(14, 7)	2	$[1 \quad x^6+x^5+x^2+1]^t$
(2, 1)	(36, 18)	3	$[1 \quad x^{17}+x^{16}+x^{13}+x^7+x^2+1]^t$
(2, 1)	(72, 36)	4	$[1 \quad x^{35}+x^{31}+x^{30}+x^{18}+x^{16}+x^{10}+x^7+1]^t$
(2, 1)	(112, 56)	5	$[1 \quad x^{55}+x^{54}+x^{49}+x^{45}+x^{32}+x^{29}+x^{21}+x^{14}+x^2+1]^t$
(3, 2)	(9, 6)	1	$[1 \quad x+1 \quad x^2+1]^t$
(3, 2)	(42, 28)	2	$[1 \quad x^{12}+x^9+x^8+1 \quad x^{13}+x^{11}+x^6+1]^t$
(3, 2)	(123, 82)	3	$[1 \quad x^{30}+x^{29}+x^{24}+x^6+x^2+1 \quad x^{36}+x^{35}+x^{28}+x^{15}+x^3+1]^t$
(4, 3)	(16, 12)	1	$[1 \quad x+1 \quad x^2+1 \quad x^3+1]^t$
(4, 3)	(80, 60)	2	$[1 \quad x^{19}+x^{15}+x^3+1 \quad x^{18}+x^{17}+x^8+1 \quad x^{13}+x^{11}+x^6+1]^t$

Рис. 13.6. Краткий список мажоритарно декодируемых сверточных кодов.

модифицированных синдромов. Если для выбранного кода это невозможно, то следует использовать нормальный синдромный декодер.

Краткий список сверточных кодов, которые могут быть декодированы мажоритарно, приведен на рис. 13.6. Эти коды получены поиском на ЭВМ.

13.6. ОБОБЩЕННЫЕ КОДЫ РИДА—МАЛЛЕРА

Коды Рида — Маллера первоначально были введены нами как двоичные; теперь же мы перейдем к изучению кодов Рида—Маллера над произвольным полем Галуа $GF(q)$. Хотя класс обобщенных кодов Рида—Маллера (ОРМ) содержит подклассы мажоритарно декодируемых кодов, он также содержит большое число кодов, которые не имеют практического интереса. Все они описываются одной и той же общей теорией.

ОРМ-коды, включая двоичные, будут введены расширением циклического кода, называемого циклическим ОРМ-кодом. В настоящем параграфе мы ограничимся обсуждением циклических ОРМ-кодов с примитивной длиной $q^m - 1$ и ОРМ-кодами, получаемыми из них добавлением символа проверки на четность.

Корни кодовых слов определяются довольно сложным и даже несколько загадочным образом. Определим вес целого числа в q-ичном разложении.

Определение 13.6.1. Пусть j — целое число, q-ичное разложение которого имеет вид

$$j = j_0 + j_1 q + j_2 q^2 + \cdots + j_{m-1} q^{m-1}.$$

Весом j в q-ичном разложении называется сумма (в смысле суммы целых чисел)

$$w_q(j) = j_0 + j_1 + j_2 + \cdots + j_{m-1}.$$

Определение 13.6.2. *Циклическим ОРМ-кодом* [1]) порядка r и длины $n = q^m - 1$ над полем $GF(q)$ называется циклический код, порождающий многочлен $g(x)$ которого имеет корни α^j при всех $j = 1, \ldots, q^m - 1$, таких, что

$$0 < w_q(j) \leqslant (q-1)m - r - 1.$$

Расширение этого циклического кода до кода длины $n = q^m$ путем добавления символа простой проверки на четность называется *ОРМ-кодом* порядка r.

---

[1]) Альтернативное определение заключается в выборе $g(x)$ с корнями α^j, где j удовлетворяют соотношению $0 \leqslant w_q(j) \leqslant (q-1)m - r - 1$; по сравнению с принятым нами определением первое строгое неравенство заменено нестрогим. Тогда ОРМ-код получается удлинением циклического ОРМ-кода путем добавления вектора, состоящего из одних единиц, к $\mathbf{G}$, а не к $\mathbf{H}$. При нашем определении коды Хэмминга являются циклическими ОРМ-кодами.

Из определения 13.6.1 следует, что j и jq (по модулю $q^m - 1$) имеют одинаковый вес в q-ичном разложении. Поэтому если β является корнем $g(x)$, то и все сопряженные с β элементы являются корнями $g(x)$, и определение 13.6.2 в самом деле приводит к коду над $GF(q)$.

При $q = 2$ ОРМ-код сводится к коду, эквивалентному коду Рида—Маллера; это тот же код, но с перестановкой компонент (поэтому он и называется обобщенным кодом Рида—Маллера). Этот код при мажоритарном декодировании может исправлять $2^{m-r-1} - 1$ ошибок. Циклические коды Рида—Маллера порядка $m - 2$ являются кодами Хэмминга, так как в них все j, для которых $w_2(j) = 1$, являются корнями $g(x)$; это j, равное 1 или числам, сопряженным с 1. Коды Хэмминга служат простейшим примером ОРМ-кодов.

Поучительно построить некоторые двоичные ОРМ-коды. Выберем $m = 5$ и $r = 2$. Длина этого кода равна 31, и при мажоритарном декодировании он может исправлять три ошибки. Проверочные частоты кода маркированы всеми отличными от нуля j, для которых

$$w_2(j) \leqslant 2,$$

т. е. теми j, двоичное представление которых содержит не более двух единиц. Это двоичные числа

0 0 0 0 1

0 0 0 1 1

0 0 1 0 1

и все их циклические сдвиги. Поэтому проверочные частоты появляются при $j = 1$, 3 и 5 и при всех сопряженных с ними числах. Этот циклический ОРМ-код второго порядка идентичен (31,16,7)-коду БЧХ над $GF(2)$. Расширение этого циклического кода представляет собой (32,16,8)-код ОРМ.

Далее выберем $m = 5$ и $r = 1$. Этот код может исправлять мажоритарным способом семь ошибок. Индексы проверочных частот теперь удовлетворяют соотношению

$$w_2(j) \leqslant 3.$$

Эти индексы записываются двоичными числами

0 0 0 0 1

0 0 0 1 1

0 0 1 0 1

0 0 1 1 1

0 1 0 1 1

и всеми их циклическими сдвигами. Поэтому проверочные частоты появляются при $j = 1, 3, 5, 7$ и 11 и при всех сопряженных с ними числах. Получаемый циклический ОРМ-код первого порядка является (31,6,15)-кодом, а его расширение представляет собой (32,6,16)-код ОРМ. Так как 9 принадлежит тому же классу сопряженных элементов, что и 5, а 13 сопряжено с 11, циклический код идентичен (31,6,15)-коду БЧХ.

Отсюда следует, что как к (31,16)-коду БЧХ, так и к (31,6)-коду БЧХ применимо мажоритарное декодирование. Иная ситуация возникает при $m = 6$ и $r = 2$. Этот код Рида—Маллера при мажоритарном декодировании может исправлять семь ошибок. Индексы проверочных частот удовлетворяют соотношению

$$w_2(j) \leqslant 3.$$

Эти индексы записываются двоичными числами

0 0 0 0 0 1
0 0 0 0 1 1
0 0 0 1 0 1
0 0 0 1 1 1
0 0 1 0 0 1
0 0 1 0 1 1
0 0 1 1 0 1
0 1 0 1 0 1

и всеми их циклическими сдвигами. Поэтому проверочные частоты появляются при $j = 1, 3, 5, 7, 9, 11, 13$ и 21 и при всех сопряженных с ними числах. Этот циклический ОРМ-код второго порядка является (63,22,15)-кодом. Существует также (63,24)-код БЧХ, исправляющий семь ошибок. Указанный двоичный ОРМ-код имеет меньшую скорость, но зато может быть декодирован мажоритарно. Расширением этого циклического ОРМ-кода является (64, 22, 16)-код ОРМ.

Определим циклический ОРМ-код через спектр кодовых слов. Циклический ОРМ-код порядка r и длины $n = q^m - 1$ над полем $GF(q)$ представляет собой множество слов, спектральные компоненты C_j которых равны нулю при всех j, удовлетворяющих неравенству

$$0 < w_q(j) \leqslant (q-1)\,m - r - 1.$$

Кодирование в частотной области производится следующим образом: спектр во всех указанных частотах полагается равным нулю, оставшиеся частоты заполняются информацией в соответствии с ограничениями сопряженности, а затем производится обратное

преобразование Фурье. Однако конструкция кодера во временно́й области обычно проще.

Минимальное расстояние циклических ОРМ-кодов должно удовлетворять границе БЧХ. Эту границу устанавливает следующая теорема.

Теорема 13.6.3. *Циклический ОРМ-код над GF (2) порядка r и длины $n = q^m - 1$ представляет собой подкод кода БЧХ с конструктивным расстоянием $d = 2^{m-r} - 1$; его минимальное расстояние не меньше указанного конструктивного.*

Доказательство. В двоичном представлении $2^{m-r} - 1$ записывается $(m - r)$-разрядным двоичным числом, состоящим из одних единиц. Числа, меньшие $2^{m-r} - 1$, имеют менее $m - r$ единиц. Поэтому $w_2(j) \leqslant m - r - 1$ при $j = 1, 2, \ldots, 2^{m-r} - 2$ и α^j являются корнями $g(x)$ при $j = 1, 2, \ldots, 2^{m-r} - 2$. Следовательно, этот код является подкодом кода БЧХ с конструктивным расстоянием $d = 2^{m-r} - 1$. □

Недвоичный циклический ОРМ-код порядка r также представляет собой подкод кода БЧХ, конструктивное расстояние которого равно $q^{m-r} - 1$, если r меньше $q - 1$. Если последнее условие не выполняется, то выражение для конструктивного расстояния несколько сложнее, что отражено в следующей теореме.

Теорема 13.6.4. *Пусть $r = (q - 1)Q + R$, где $0 \leqslant R < q - 1$. Циклический ОРМ-код над GF (q) порядка r и длины $n = q^m - 1$ представляет собой подкод кода БЧХ над GF (q) с конструктивным расстоянием*

$$d = (q - R)\, q^{m-Q-1} - 1;$$

его минимальное расстояние не меньше указанного конструктивного.

Доказательство. Рассмотрим q-ичное разложение числа $(q - R)q^{m-Q-1} - 1$. Максимальный значащий символ этого числа равен $q - R - 1$; остальные $m - Q - 1$ символов равны $q - 1$. Следовательно,

$$w_q((q - R)\, q^{m-Q-1} - 1) = (q - 1)(m - Q - 1) + q - R - 1 = (q - 1)\, m - r.$$

Но тогда q-ичные веса всех j, меньших $(q - R)\, q^{m-Q-1} - 1$, меньше $(q - 1)\, m - r$. Следовательно, при всех j, удовлетворяющих неравенству $0 < j \leqslant (q - R)q^{m-Q-1} - 2$, α^j являются корнями порождающего многочлена, а циклический ОРМ-код является подкодом кода БЧХ с теми же корнями порождающего многочлена. Это завершает доказательство. □

$q = 2$				$q = 4$				$q = 8$			
m	r	(n, k)	$d_{\text{БЧХ}}$	m	r	(n, k)	$d_{\text{БЧХ}}$	m	r	(n, k)	$d_{\text{БЧХ}}$
4	1	(15, 5)	7	2	1	(15, 3)	11	2	1	(63, 3)	55
4	2	(15, 11)	3	2	2	(15, 6)	7	2	2	(63, 6)	47
5	1	(31, 6)	15	2	3	(15, 10)	3	2	3	(63, 10)	39
5	2	(31, 16)	7	2	4	(15, 13)	2	2	4	(63, 15)	31
5	3	(31, 26)	3	3	1	(63, 4)	47	2	5	(63, 21)	23
6	1	(63, 7)	31	3	2	(63, 10)	31	2	6	(63, 28)	15
6	2	(63, 22)	15	3	3	(63, 20)	15	2	7	(63, 36)	7
6	3	(63, 42)	7	3	4	(63, 32)	11	2	8	(63, 43)	6
6	4	(63, 57)	3	3	5	(63, 44)	7	2	9	(63, 49)	5
7	1	(127, 8)	63	3	6	(63, 54)	3	2	10	(63, 54)	4
7	2	(127, 29)	31	3	7	(63, 60)	2	2	11	(63, 58)	3
7	3	(127, 64)	15	4	1	(255, 5)	191	2	12	(63, 61)	2
7	4	(127, 99)	7	4	2	(255, 15)	127	3	1	(511, 4)	447
7	5	(127, 120)	3	4	3	(255, 35)	63	3	2	(511, 10)	383
8	1	(255, 9)	127	4	4	(255, 66)	47	3	3	(511, 20)	319
8	2	(255, 37)	63	4	5	(255, 106)	31	3	4	(511, 35)	255
8	3	(255, 93)	31	4	6	(255, 150)	15	3	5	(511, 56)	191
8	4	(255, 163)	15	4	7	(255, 190)	11	3	6	(511, 84)	127
8	5	(255, 219)	7	4	8	(255, 221)	7	3	7	(511, 120)	63
8	6	(255, 247)	3	4	9	(255, 241)	3	3	8	(511, 162)	55
9	1	(511, 10)	255	4	10	(255, 251)	2	3	9	(511, 208)	47
9	2	(511, 46)	127					3	10	(511, 256)	39
9	3	(511, 130)	63					3	11	(511, 304)	31
9	4	(511, 256)	31					3	12	(511, 350)	23
9	5	(511, 382)	15					3	13	(511, 392)	15
9	6	(511, 466)	7					3	14	(511, 428)	7
9	7	(511, 502)	3					3	15	(511, 456)	6
								3	16	(511, 477)	5
								3	17	(511, 492)	4
								3	18	(511, 502)	3
								3	19	(511, 508)	2

Рис. 13.7. Параметры некоторых обобщенных кодов Рида—Маллера.

Характеристики некоторых обобщенных кодов Рида—Маллера представлены на рис. 13.7. При $q = 2$ коды эквивалентны кодам Рида—Маллера. Приведенные на рисунке минимальные расстояния в действительности являются определяемыми теоремой 13.6.4 нижними границами минимальных расстояний. Число информационных символов получается подсчетом: так как код определяется через корни $g(x)$, число проверочных символов находится подсчетом положительных целых чисел, меньших n, для которых $w_q(j) \leqslant (q-1)m - r - 1$.

Теорема 13.6.5. *Код, дуальный ОРМ-коду над GF (q) порядка r и длины $n = q^m$, эквивалентен ОРМ-коду порядка $(q-1)m - r - 1$ и длины $n = q^m$.*

Доказательство. Пусть $\overline{\mathscr{C}}$ — ОРМ-код, и пусть $\mathscr{C}$ — циклический ОРМ-код, получаемый укорочением $\overline{\mathscr{C}}$. Доказательство теоремы распадается на три шага. На шаге 1 находится порождающий многочлен $\bar{h}(x)$ кода $\mathscr{C}^\perp$, дуального циклическому ОРМ-коду $\mathscr{C}$. На шаге 2 показывается, что проверочные многочлены как $\mathscr{C}$, так и $\mathscr{C}^\perp$ имеют множителем $x - 1$. На шаге 3 производится расширение кодов $\mathscr{C}$ и $\mathscr{C}^\perp$ до ОРМ-кодов $\overline{\mathscr{C}}$ и $\overline{\mathscr{C}^\perp}$ и показывается, что расширенные коды дуальны.

Шаг 1. $\mathscr{C}$ является множеством слов с проверочными частотами, которые при всех j удовлетворяют неравенствам

$$0 < j \leqslant q^m - 1$$

и

$$w_q(j) \leqslant (q-1)m - r - 1.$$

При всех указанных j величины α^j являются корнями порождающего многочлена $g(x)$. Для удобства мы заменили 0 на $q^m - 1$ в интервале изменения j (см. примечание на с. 469), так что $w_q(j) > 0$ при всех j. Из теории циклических кодов следует, что для определения проверочных частот дуального кода нужно заменить j на $q^m - 1 - j$ и обратить нестрогое неравенство для q-ичного веса, заменив его строгим неравенством. Поэтому индексами проверочных частот служат те j, для которых

$$w_q(q^m - 1 - j) > (q-1)m - r - 1.$$

Но если

$$j = j_0 + j_1 q + j_2 q^2 + \cdots + j_{m-1} q^{m-1},$$

то

$$q^m - 1 - j = (q - 1 - j_0) + (q - 1 - j_1)\, q + \cdots + (q - 1 - j_{m-1})\, q^{m-1},$$

что может быть проверено сложением равенств. Следовательно, $w_q(q^m - 1 - j) = (q-1)m - w_q(j)$. Поэтому дуальный код имеет проверочные частоты при j, удовлетворяющих неравенству $0 < j \leqslant q^{m-1}$, и

$$w_q(j) = (q-1)m - w_q(q^m - 1 - j) < r + 1.$$

Соответствующие этим j числа α^j являются корнями порождающего многочлена $\bar{h}(x)$ дуального кода $\mathscr{C}^\perp$.

Шаг 2. Число α^0 не является корнем ни $g(x)$, ни $\bar{h}(x)$. Следовательно, проверочные многочлены как $\mathscr{C}$, так и $\mathscr{C}^\perp$ содержат множители $x - 1$.

Шаг 3. Теперь расширим $\mathscr{C}$ и $\mathscr{C}^{\perp}$ и получим два ОРМ-кода. Проверочные матрицы этих расширенных кодов равны

$$\begin{bmatrix} 1 & 1 & \dots & 1 \\ 0 & & & \\ \vdots & & & \\ & & \mathbf{H} & \\ 0 & & & \end{bmatrix} \quad \text{и} \quad \begin{bmatrix} 1 & 1 & \dots & 1 \\ 0 & & & \\ \vdots & & & \\ & & \mathbf{H}' & \\ 0 & & & \end{bmatrix},$$

где $\mathbf{H}$ и $\mathbf{H}'$ — проверочные матрицы кодов $\mathscr{C}$ и $\mathscr{C}^{\perp}$. Если временно исключить из рассмотрения первые строки каждой из матриц, получим, что все строки левой матрицы ортогональны всем строкам правой матрицы; это вытекает из справедливости этого утверждения для $\mathbf{H}$ и $\mathbf{H}'$. Далее, строка, целиком состоящая из единиц, ортогональна самой себе, поскольку длина кода кратна q. И наконец, строка, целиком состоящая из единиц, ортогональна любой другой строке обеих матриц, так как $x - 1$ является делителем обоих проверочных многочленов. Поэтому левая и правая матрицы ортогональны, а размерности расширенных циклических кодов в сумме равны q^m. Доказательство закончено. □

Непримитивные ОРМ-коды могут быть также определены очевидным образом. Этим определением мы завершаем параграф.

Определение 13.6.6. Пусть b делит $q^m - 1$. Циклическим непримитивным ОРМ-кодом порядка r и длины $n = (q^m - 1)/b$ над полем $GF(q)$ является циклический код, порождающий многочлен которого имеет корни α^{bj} при всех $j = 1, \dots, (q^m - 1)/b$, таких, что $0 < w_q(bj) \leqslant (q-1)m - r - 1$.

13.7. ЕВКЛИДОВО-ГЕОМЕТРИЧЕСКИЕ КОДЫ

Конечная геометрия — это конечное множество, в котором определены некоторые подмножества, называемые линиями, плоскостями или аффинными подпространствами и удовлетворяющие некоторому набору аксиом. Конечная геометрия заимствовала свою терминологию у обычной элементарной геометрии. Последняя оперирует с множеством, содержащим бесконечное число точек, и с подмножествами, известными как линии и плоскости. Каждая конечная геометрия определяется собственным набором аксиом, и в ее рамках можно построить систему теорем. Наиболее важными конечными геометриями являются евклидовы геометрии и проективные геометрии. Эти конечные геометрии могут использоваться для построения кодов.

Несколько классов кодов, описываемых на языке конечных геометрий, являются мажоритарно декодируемыми — это евклидово-геометрические коды и проективно-геометрические коды. Мы введем эти классы кодов, используя терминологию кодов Рида—Маллера. Первоначально они были введены иначе, а именно на основе теории конечных геометрий; этим и объясняются их названия. В этом параграфе мы рассмотрим евклидово-геометрические коды, а в следующем — проективно-геометрические коды. Ограничимся изучением лишь кодов над простым полем $GF(p)$. Существенную роль играют три поля Галуа: $GF(p)$ — поле символов евклидово-геометрического кода, $GF(q)$ — поле символов ОРМ-кода, представляющее собой расширение $GF(p)$ для $q = p^s$, и поле локаторов $GF(q^m)$.

Определение 13.7.1. Пусть r, s и m — любые положительные целые числа, и пусть $q = p^s$, где p — простое число. *Евклидово-геометрическим кодом* над $GF(p)$ порядка r и длины $n = q^m$ называется код, дуальный подкоду над подполем ОРМ-кода над $GF(q)$ порядка $(q-1)(m-r-1)$ и длины q^m.

Эквивалентное определение: евклидово-геометрическим кодом называется расширение циклического кода над $GF(p)$ (также называемого евклидово-геометрическим кодом) длины $q^m - 1$; он определяется следующей теоремой.

Теорема 13.7.2. *Пусть α — примитивный элемент $GF(q^{sm})$. Евклидово-геометрический код над $GF(p)$ с параметрами r и s и длиной q^m — это расширенный циклический код, порождаемый многочленом, корнями которого являются α^j при всех j, удовлетворяющих неравенствам $0 < j \leqslant q^m - 1$ и*

$$0 < \max_{0 \leqslant i < s} w_q(jp^i) \leqslant (q-1)(m-r-1).$$

Доказательство. Подкод над подполем $GF(p)$ циклического ОРМ кода порядка $(q-1)(m-r-1)$ обладает порождающим многочленом с корнями α^j при $0 < j \leqslant q^m - 1$, если j удовлетворяет неравенству

$$w_q(j) \leqslant (q-1)m - (q-1)(m-r-1) - 1 = (q-1)(r+1) - 1$$

или если этому неравенству удовлетворяет любое p-сопряженное с j число. Обратно, α^j является корнем проверочного многочлена $h(x)$, если

$$w_q(j') > (q-1)(r+1) - 1$$

для всех j', которые являются p-сопряженными с j числами. Но, согласно теории циклических кодов, порождающий многочлен дуального кода является взаимным к многочлену $h(x)$. Поэтому $\alpha^{j'}$

t	Евклидово-геометрический (n, k)-код	(n, k)-код БЧХ
4	(63, 37)	(63, 39)
10	(63, 13)	(63, 18)
2	(255, 231)	(255, 239)
8	(255, 175)	(255, 191)
10	(255, 127)	(255, 179)
42	(255, 19)	(255, 47)

Рис. 13.8. Параметры некоторых евклидово-геометрических кодов и некоторых кодов БЧХ.

является корнем этого взаимного многочлена, если $\alpha^{n-j'}$ является корнем $h(x)$, т. е. если

$$w_q(n - j') > (q - 1)(r + 1) - 1$$

при всех j', которые являются p-сопряженными с j числами. Но, как было установлено при доказательстве теоремы 13.6.5, $w_q(n - j') = (q - 1)m - w_q(j')$. Следовательно,

$$w_q(j') \leqslant (q - 1)(m - r - 1)$$

при всех j', которые являются p-сопряженными с j числами. Отсюда следует утверждение теоремы. □

Простейшим примером евклидово-геометрического кода служит код с $s = 1$ и $p = 2$. В этом случае

$$w_2(j) \leqslant m - r - 1,$$

если α^j является корнем $g(x)$. Это как раз код Рида—Маллера порядка r. Евклидово-геометрические коды представляют собой обобщение кодов Рида—Маллера. Параметры других евклидово-геометрических кодов приведены на рис. 13.8.

Наша цель — доказать, что евклидово-геометрические коды могут декодироваться мажоритарно в $r + 1$ шагов и что $d_{\text{МД}} = (q^{m-r} - 1)/(q - 1)$. Мы опишем декодер на геометрическом языке, языке евклидовой геометрии, который мы сейчас введем.

Евклидова геометрия размерности m над полем $GF(q)$ (она обозначается через EG (m, q)) состоит из q^m точек (векторное пространство $GF^m(q)$) вместе с некоторыми подмножествами, называемыми *плоскостями* (или *аффинными подпространствами* [1]), рекуррентное определение которых следует ниже. 0-плоскостью (нульмерным аффинным подпространством) называются точки EG (m, q). Ими являются q^m векторов m-мерного векторного про-

[1]) Векторное подпространство должно содержать начало координат векторного пространства. Аффинное подпространство получается сдвигом векторного подпространства (т. е. является его смежным классом).

странства. Из t-плоскости получается $(t+1)$-плоскость как минимальное векторное подпространство, каждый сдвиг которого содержит t-плоскость. Иначе говоря, если E_t является t-плоскостью, то множество $\{\gamma_t \mathbf{u} \mid \gamma_t \in GF(q), \mathbf{u} \in E_t\}$ — это минимальное векторное подпространство, содержащее E_t. Если $\mathbf{v}_{t+1}$ — любая конкретная точка в $GF^m(q)$, не принадлежащая E_t, то $(t+1)$-плоскость E_{t+1} определяется как множество

$$E_{t+1} = \{\mathbf{v}_{t+1} + \gamma_t \mathbf{u} \mid \gamma_t \in GF(q),\ \mathbf{u} \in E_t\}.$$

Приведем более формальное определение.

Определение 13.7.3. *Евклидовой геометрией* EG (m, q) называется векторное пространство $GF^m(q)$ вместе со всеми подпространствами и сдвигами подпространств $GF^m(q)$.

Из этого определения непосредственно вытекает следующая теорема.

Теорема 13.7.4. *t-плоскость содержит ровно q^t точек и сама имеет структуру евклидовой геометрии* EG (t, q). *Можно представить t-плоскость как множество q^t точек*

$$\mathbf{v}_t + \gamma_{i_{t-1}}\mathbf{v}_{t-1} + \gamma_{i_{t-2}}\mathbf{v}_{t-2} + \cdots + \gamma_{i_0}\mathbf{v}_0,$$

где γ_t пробегает множество всех элементов $GF(q)$, включая нулевой, а векторы $\mathbf{v}_0, \mathbf{v}_1, \ldots, \mathbf{v}_t$ над $GF^m(q)$ образуют фиксированное множество линейно независимых элементов t-плоскости.

Можно называть 1-плоскость и 2-плоскость *линиями и плоскостями*. Эти названия связаны с тем, что определение евклидовой геометрии по идее соответствует обычному евклидову пространству с той разницей, что оно строится над конечным, а не над вещественным полем. Можно также считать, что EG (m, q), $GF(q^m)$ и $GF^m(q)$ состоят из одних и тех же элементов. Однако когда мы обозначаем поле через $GF(q^m)$, то имеем в виду его алгебраическую структуру, правила умножения и т. д. Когда мы обозначаем его через EG (m, q), то интересуемся его геометрической структурой, введенной определением 13.7.3. Если мы обозначаем его через $GF^m(q)$, то мы интересуемся его структурой как структурой векторного пространства.

Евклидова геометрия EG (3, 2) представлена на рис. 13.9. Этот пример показывает, что число t-плоскостей в евклидовой геометрии может быть довольно большим. Подсчитаем число t-плоскостей в EG (m, q). Подсчет опирается на величины, известные под названием q-ичных гауссовских коэффициентов и определяемые соотношениями

$$\begin{bmatrix} m \\ i \end{bmatrix} = \prod_{j=0}^{i-1} (q^m - q^j)/(q^i - q^j)$$

3-плоскость	1-плоскость	2-плоскость
000	000,001	000,001,010,011
001	000,010	000,001,100,101
010	000,011	000,001,110,111
011	000,100	000,010,100,110
100	000,101	000,010,101,111
101	000,110	000,011,100,111
110	000,111	000,011,101,110
111	001,010	001,011,101,111
	001,011	001,011,100,110
	001,100	001,010,101,110
	001,101	001,010,100,111
	001,110	010,011,110,111
	001,111	010,011,100,101
	010,011	100,101,110,111
	010,100	
	010,101	
	010,110	
	010,111	
	011,100	
	011,101	
	011,110	
	011,111	
	100,101	
	100,110	
	100,111	
	101,110	
	101,111	
	110,111	

Рис. 13.9. Евклидова геометрия EG(3, 2).

при $i = 1, 2, \ldots, m$ и $\begin{bmatrix} m \\ 0 \end{bmatrix} = 1$.

Теорема 13.7.5.

(i) EG (m, q) *содержит* $q^{m-t} \begin{bmatrix} m \\ t \end{bmatrix}$ *различных* t*-плоскостей, где* $t = 0, 1, \ldots, m$.

(ii) *При любых* s *и* t, *таких, что* $0 \leqslant s \leqslant t \leqslant m$, *каждая* s*-плоскость содержит точно* $\begin{bmatrix} m-s \\ t-s \end{bmatrix}$ *различных* t*-плоскостей из* EG (m, q).

Доказательство. (i) Можно построить t-мерное подпространство, выбирая упорядоченное множество t линейно независимых точек в EG (m, q). Это можно сделать

$$(q^m - 1)(q^m - q) \ldots (q^m - q^{t-1})$$

различными способами. Однако многие из этих множеств независимых точек приводят к одному и тому же t-мерному подпростран-

ству. В самом деле, число множеств, приводящих к одному и тому же t-мерному подпространству, равно

$$(q^t - 1)(q^t - q) \ldots (q^t - q^{t-1}),$$

т. е. равно числу способов, которыми можно выбрать упорядоченную последовательность независимых точек в t-мерном подпространстве. Следовательно, существует

$$\frac{(q^m - 1)(q^m - q) \ldots (q^m - q^{t-1})}{(q^t - 1)(q^t - q) \ldots (q^t - q^{t-1})} = \begin{bmatrix} m \\ t \end{bmatrix}$$

различных t-мерных подпространств. Каждое подпространство имеет q^{m-t} смежных классов, и поэтому существует $q^{m-t} \begin{bmatrix} m \\ t \end{bmatrix}$ t-плоскостей.

(ii) Заданная s-плоскость может быть расширена до t-плоскости выбором последовательности $t - s$ независимых точек, не принадлежащих s-плоскости. Это можно сделать

$$(q^m - q^s)(q^m - q^{s+1}) \ldots (q^m - q^{t-1})$$

способами. Однако многие из этих последовательностей независимых точек расширяют s-плоскость до той же самой t-плоскости. В самом деле, число последовательностей, приводящих к одной и той же t-плоскости, равно

$$(q - q^s)(q^t - q^{s+1}) \ldots (q^t - q^{t-1}),$$

т. е. равно числу способов, которыми можно выбрать упорядоченную последовательность независимых точек на t-плоскости, не используя точки из s-плоскости. Отношение этих двух произведений равно числу различных t-плоскостей, в которых содержится s-плоскость. Это эквивалентно утверждению, которое требовалось доказать. □

Определение 13.7.6. *Вектором инцидентности* подмножества множества q^m элементов, маркированных индексом i, называется вектор размерности q^m, i-я компонента которого равна единице, если элемент, маркированный индексом i, принадлежит этому подмножеству, и нулю в противном случае.

В нашем определении ОРМ-кода элементы $GF(q^m)$ используются для нумерации компонент векторного пространства размерности $n = q^m$ (которое само содержит q^{q^m} векторов). Иначе говоря, элементы векторного пространства $GF^m(q)$ используются для индексации n компонент векторного пространства $GF^{q^m}(q)$. Вектор инцидентности подмножества $GF^m(q)$ — это вектор в $GF^{q^m}(q)$. Если $GF^m(q)$ определяет структуру евклидовой геометрии EG (m, q), то вектор инцидентности плоскости в EG (m, q) является вектором в $GF^{q^m}(q)$. Как мы увидим, вектор инцидент-

ности плоскости в EG (m, q) является кодовым словом ОРМ-кода, содержащимся в $GF^{q^m}(q)$.

Теперь мы можем предложить альтернативное определение евклидово-геометрических кодов.

Теорема 13.7.7. *Евклидово-геометрическим кодом r-го порядка и длины $n = q^m$ над $GF(p)$ является наибольший линейный код над $GF(p)$, содержащий в своем нуль-пространстве векторы инцидентности всех $(r+1)$-плоскостей в* EG (m, q).

Доказательство. Достаточно доказать, что ОРМ-код, который содержит код, дуальный евклидово-геометрическому коду, является наименьшим линейным кодом над $GF(q)$, содержащим все векторы инцидентности. Это следует из того, что компоненты вектора инцидентности могут принимать лишь значения нуль или единица, и поэтому все его компоненты всегда лежат в подполе $GF(p)$. Следовательно, вектор инцидентности принадлежит коду, дуальному евклидово-геометрическому коду, если он принадлежит ОРМ-коду, содержащему этот дуальный код.

Вектор инцидентности принадлежит ОРМ-коду, если он принадлежит циклическому ОРМ-коду и его символ расширения правилен. Но вектор инцидентности $(r+1)$-плоскости содержит q^{r+1} ненулевых компонент, которые прибавляются к нулю по модулю p; поэтому его символ расширения всегда правилен. Необходимо лишь доказать, что вектор инцидентности, у которого последняя компонента опущена, принадлежит циклическому ОРМ-коду. Иначе говоря, нужно доказать, что преобразование Фурье каждого вектора инцидентности $\mathbf{f}$ имеет компоненту F_j, равную нулю, если

$$w_q(j) \leqslant (q-1)\,m - (q-1)\,(m-r-1) - 1,$$

или, что то же самое,

$$w_q(j) < (q-1)\,(r+1).$$

Для доказательства достаточно вычислить F_j и показать, что при всех таких j оно равно нулю, хотя при других j может быть отличным от нуля.

Шаг 1. Используя введенное в теореме 13.7.4 определение, представим $(r+1)$-плоскость как множество $\{\mathbf{v}_{r+1} + \gamma_{i_r}\mathbf{v}_r + \ldots + \gamma_{i_0}\mathbf{v}_0\}$, где индексы

$$\begin{aligned} i_0 &= 0, 1, \ldots, q-1, \\ i_1 &= 0, 1, \ldots, q-1, \\ &\vdots \\ i_r &= 0, 1, \ldots, q-1 \end{aligned}$$

маркируют q элементов поля $GF(q)$, а $\mathbf{v}_0, \ldots, \mathbf{v}_{r+1}$ — фиксированное множество независимых точек на $(r+1)$-плоскости. Если элемент поля α^i принадлежит указанному множеству, то i-я компонента вектора инцидентности $\mathbf{f}$ равна единице; в противном случае она равна нулю. Поэтому спектральная компонента

$$F_j = \sum_{i=0}^{n-1} \alpha^{ij} f_i$$

может быть записана как сумма тех слагаемых, у которых f_i равно единице; слагаемые, в которых f_i равно нулю, можно опустить:

$$F_j = \sum_{i_0=0}^{q-1} \sum_{i_1=0}^{q-1} \cdots \sum_{i_r=0}^{q-1} (v_{r+1} + \gamma_{i_r} v_r + \gamma_{i_{r-1}} v_{r-1} + \cdots + \gamma_{i_0} v_0)^j;$$

здесь $v_0, \ldots, v_{r+1}$ можно считать элементами $GF(q^{r+1})$. Необходимо определить те значения j, при которых F_j равны нулю. Используя полиномиальное разложение, получим

$$F_j = \sum_{i_0=0}^{q-1} \cdots \sum_{i_r=0}^{q-1} \sum_{\mathbf{h}} \frac{j!}{h_0!\, h_1! \ldots h_{r+1}!} v_{r+1}^{h_{r+1}} (\gamma_{i_r} v_r)^{h_r} \ldots (\gamma_{i_0} v_0)^{h_0},$$

где под суммированием по $\mathbf{h}$ понимается суммирование по всем наборам $(h_0, h_1, \ldots, h_{r+1})$, таким, что $h_0 + h_1 + \cdots + h_{r+1} = j$. Затем переменим порядок суммирования и рассмотрим слагаемое вида $\sum_{i=0}^{q-1} (\gamma_i v)^h$, где h фиксировано.

Шаг 2. Суммирование $\sum_{i=0}^{q-1} \gamma_i^h$ проводится по всем элементам поля $GF(q)$, и поэтому сумма может быть выражена через примитивный элемент α. При h, отличных от нуля,

$$\sum_{i=0}^{q-1} \gamma_i^h = 0 + \sum_{k=0}^{q-2} \alpha^{kh}.$$

Величина в правой части равна h-й компоненте преобразования Фурье вектора, состоящего только из единиц. Эта компонента равна нулю во всех случаях, кроме случая, когда h кратно $q-1$. В этом последнем случае она равна $q-1 \pmod p$, или -1. Для h, равного нулю,

$$\sum_{i=0}^{q-1} \gamma_i^0 = \sum_{i=0}^{q-1} 1 = 0 \pmod p;$$

по определению считалось, что $\gamma^0 = 1$ при всех γ, принадлежащих $GF(q)$.

Ненулевые слагаемые в сумме, определяющей F_j, — это слагаемые, соответствующие тем h_l, $l = 0, \ldots, r$, которые являются не-

нулевыми кратными $q-1$; остальные слагаемые равны нулю и могут быть опущены. Сумма принимает вид

$$F_j = (-1)^r \sum_{h_0} \sum_{h_1} \cdots \sum_{h_{r+1}} \frac{j!}{h_0!\, h_1! \ldots h_{r+1}!}\, v_0^{h_0} v_1^{h_1} \ldots v_{r+1}^{h_{r+1}},$$

где суммирование производится по $(h_0, \ldots, h_{r+1})$, таким, что h_l, $l = 0, \ldots, r$, суть ненулевые кратные $q-1$, $h_{r+1} \geqslant 0$ и

$$\sum_{l=0}^{r+1} h_l = j.$$

Шаг 3. По теореме Лукаса (теорема 13.3.3) в равенстве для F_j полиномиальные коэффициенты равны нулю в тех слагаемых, в которых h_l, $l = 0, \ldots, r+1$, не является q-ичным потомком j; поэтому если h_l входит в сумму, то h_l, $l = 0, \ldots, r+1$, является p-ичным потомком j и, следовательно, q-ичным потомком j. Это получится из теоремы Лукаса, если записать

$$\frac{j!}{h_0!\, h_1! \ldots h_{r+1}!} = \frac{j!}{h_0!\,(j-h_0)!} \frac{(j-h_0)!}{h_1! \ldots h_{r+1}!} =$$
$$= \frac{j!}{h_0!\,(j-h_0)!} \frac{(j-h_0)!}{h_1!\,(j-h_0-h_1)!} \frac{(j-h_0-h_1)!}{h_2! \ldots h_{r+1}!} = \ldots .$$

Кроме того, любая сумма h_l, входящих в сумму, равна p-ичному потомку j и, следовательно, q-ичному потомку j.

Резюмируем условия, которым удовлетворяют слагаемые, вносящие вклад в величину F_j:

(i) $\sum_{l=0}^{r+1} h_l = j$;

(ii) h_l, $l = 0, \ldots, r$, должны быть ненулевыми кратными $q-1$ и $h_{r+1} \geqslant 0$;

(iii) каждая позиция в q-ичном разложении j равна сумме соответствующих позиций q-ичных разложений h_l.

Для завершения доказательства необходимо показать, что при j, удовлетворяющем неравенству

$$w_q(j) < (q-1)(r+1),$$

таких слагаемых нет.

Шаг 4. Рассмотрим q-ичное разложение некоторого целого k:

$$k = k_0 + k_1 q + k_2 q^2 + \cdots + k_{m-1} q^{m-1}.$$

Тогда $k \pmod{q-1}$ можно вычислить следующим образом:

$$k = k_0 + k_1 q \dotplus k_{m-1} q^{m-1},$$
$$k = k_0 + k_1 + \cdots \dotplus k_{m-1} = (\mathrm{mod}\ q-1)$$
$$= w_q(k),$$

так как по модулю $q-1$ число q можно заменить единицей. Следовательно,

$$k = w_q(k) \pmod{d-1}.$$

Если k равно ненулевому кратному $q-1$, то и $w_q(k)$ равно ненулевому кратному $q-1$.

Рассмотрим q-ичные разложения j и h_l. Каждая позиция в q-ичном разложении j равна сумме соответствующих позиций в q-ичных разложениях h_l. Поэтому

$$w_q(j) = \sum_{l=0}^{r+1} w_q(h_l),$$

причем при $l = 0, \ldots, r$ $w_q(h_l)$ являются ненулевыми кратными $q-1$. Следовательно, если F_j отлично от нуля, то

$$w_q(j) \geqslant (q-1)(r+1).$$

Теорема доказана. □

Доказательство следующей теоремы включает алгоритм мажоритарного декодирования.

Теорема 13.7.8. *Пусть $q = p^s$. Евклидово-геометрический код r-го порядка и длины $n = q^m$ над $GF(p)$ может быть декодирован мажоритарно в $r+1$ шагов при числе ошибок, не превышающем $(q^{m-r}-1)/[2(q-1)]$.*

Доказательство. Доказательство проводится рекуррентно; показывается, что из множества проверочных равенств, основанных на векторах инцидентности t-плоскостей, можно получить проверочные равенства, основанные на векторах инцидентности $(t-1)$-плоскостей. Отсюда следует, что проверочные равенства, основанные на векторах инцидентности $(r+1)$-плоскостей, как установлено теоремой 13.7.7, в $r+1$ шагов сводятся к проверочным равенствам для отдельных символов. При этом надо рассуждать следующим образом.

Из определения t-плоскости в конечной геометрии следует, что векторы инцидентности всех t-плоскостей, которые содержат данную $(t-1)$-плоскость, определяют множество $r = \left[\begin{matrix} m-(t-1) \\ t-(t-1) \end{matrix}\right]$ проверок, которые согласуются в сумме ошибочных символов, связанных с точками этой $(t-1)$-плоскости. Через любую точку, не принадлежащую заданной $(t-1)$-плоскости E, проходит ровно одна t-плоскость, содержащая E. Следовательно, используя мажоритарное решение, можно получить новую проверку, которая соответствует вектору инцидентности $(t-1)$-плоскости E. Это же может быть сделано для всех $(t-1)$-плоскостей, содержащих заданную $(t-2)$-плоскость, которая в свою очередь определяет множество проверок, согласующихся в данной $(t-2)$-плоскости.

Таким образом, после t шагов по индукции мы получим множество проверочных равенств, согласующихся в 0-плоскости, т. е. в одном ошибочном символе. Число согласующихся проверок, которые могут быть использованы на i-м шаге, равно $\begin{bmatrix} m-t+i \\ 1 \end{bmatrix}$. Следовательно, на каждом шаге проводится не менее $\begin{bmatrix} m-t+1 \\ 1 \end{bmatrix}$ проверок, и корректирующая способность алгоритма равна

$$\frac{1}{2}\begin{bmatrix} m-t+1 \\ 1 \end{bmatrix} = \frac{q^{m-t+1}-1}{2(q-1)}.$$

Этим доказательство теоремы завершается. □

13.8. ПРОЕКТИВНО-ГЕОМЕТРИЧЕСКИЕ КОДЫ

Проективно-геометрические коды образуют класс кодов, похожий на класс евклидово-геометрических кодов. Отличие состоит лишь в том, что они строятся из непримитивных циклических ОРМ-кодов длины $(q^m - 1)/(q - 1)$, а не из ОРМ-кодов длины q^m. Построение проводится так же, как и ранее. Существенную роль играют три поля: $GF(p)$ — поле символов кода с простым p, $GF(q)$ с $q = p^s$ — поле символов непримитивного ОРМ-кода и $GF(q^m)$ — поле локаторов.

Определение 13.8.1. Пусть r, s и m — любые положительные целые числа, и пусть $q = p^s$, где p — простое число. *Проективно-геометрическим кодом* над $GF(p)$ порядка r и длины $n = (q^m - 1)/(q - 1)$ называется код, дуальный подкоду над подполем непримитивного циклического ОРМ-кода над $GF(q)$ порядка $(q - 1) \times (m - r - 1)$ и той же длины.

Этому определению эквивалентно определение проективно-геометрического кода как непримитивного циклического кода, порождающий многочлен которого описывается следующей теоремой.

Теорема 13.8.2. *Проективно-геометрический код над $GF(p)$ с параметрами r и s и длиной $(q^m - 1)/(q - 1)$ является циклическим кодом, порождаемым многочленом, корни которого расположены в точках β^j, причем j таковы, что $0 < j \leqslant (q^m - 1)(q - 1)$ и*

$$0 < \max_{0 \leqslant t < s} w_q(j(q-1)p^t) \leqslant (q-1)(m-r-1),$$

где $q = p^s$, $\beta = \alpha^{q-1}$ и α — примитивный элемент поля $GF(q)$.

Доказательство аналогично доказательству теоремы 13.7.2. □

На рис. 13.10 приведены параметры некоторых проективно-геометрических кодов. Проективно-геометрические коды мажори-

t	Проективно-геометрический (n,k)-код	Укороченный примитивный (n,k)-код БЧХ
2	(21, 11)	(21, 11)
4	(73, 45)	(73, 45)
2	(85, 68)	(85, 71)
10	(85, 24)	(85, 22)
8	(273, 191)	(273, 201)
2	(341, 315)	(341, 323)
10	(341, 195)	(341, 251)
42	(341, 45)	(341, 32)
4	(585, 520)	(585, 545)
36	(585, 184)	(585, 250)
16	(1057, 813)	
2	(1365, 1328)	
10	(1365, 1063)	
42	(1365, 483)	
170	(1365, 78)	

Рис. 13.10. Параметры некоторых проективно-геометрических кодов и некоторых кодов БЧХ.

тарно декодируемы в r шагов. Осталось построить мажоритарный декодер и доказать, что $d_{\text{МД}} = 1 + (q^{m-r+1} - 1)/(q - 1)$.

Процедуру декодирования можно описать на геометрическом языке проективных геометрий. Проективная геометрия близко связана с евклидовой геометрией. Коротко говоря, проективная геометрия — это евклидова геометрия, дополненная некоторыми новыми точками, которые можно назвать бесконечно удаленными точками, и некоторыми новыми плоскостями, проходящими через эти точки. Формальное определение — дело техники.

Проективная геометрия PG (m, q) содержит $(q^{m+1} - 1)/(q - 1)$ точек и определяется ненулевыми точками из $GF^{m+1}(q)$. Таких ненулевых точек $q^{m+1} - 1$, и они разбиваются на $(q^{m+1} - 1)/(q - 1)$ множеств, каждое из которых включает одну точку PG (m, q). В каждую точку PG (m, q) проектируется $q - 1$ точек $GF^{m+1}(q)$ (этим и объясняется название проективной геометрии).

Правило образования этих множеств следующее: для любого ненулевого вектора $\mathbf{v}$ в $GF^{m+1}(q)$ и любого ненулевого элемента λ поля $GF(q)$ $\mathbf{v}$ и $\lambda\mathbf{v}$ принадлежат одному множеству V; иначе говоря, они отображаются в одну и ту же точку PG (m, q). Число таких ненулевых λ равно $q - 1$, и поэтому в множестве содержится $q - 1$ векторов. Следовательно, точки проективной геометрии отождествляются с $(q^{m+1} - 1)/(q - 1)$ различными одномерными подпространствами из $GF^{m+1}(q)$.

Проективная геометрия PG (m, q) размерности m над полем $GF(q)$ — это множество таких $(q^{m+1} - 1)/(q - 1)$ точек вместе

с наборами подмножеств, называемых t-плоскостями ($t = 0, 1, \ldots, m$).

Содержащая V_0 и V_1 1-плоскость определяется следующим образом. Пусть $\mathbf{v}_0 \in V_0$ и $\mathbf{v}_1 \in V_1$; не имеет значения, какие именно элементы выбираются (см. задачу 13.9). Тогда 1-плоскость (или линия), содержащая V_0 и V_1, образуется точками PG (m, q), являющимися образами точек $\beta_0\mathbf{v}_0 + \beta_1\mathbf{v}_1$ в $GF^{m+1}(q)$, где β_0 и β_1 — произвольные элементы поля, не равные одновременно нулю. Выбрать β_0 и β_1 можно $q^2 - 1$ способами, поэтому отображение состоит из $(q^2 - 1)/(q - 1)$ точек PG (m, q). Отсюда следует, что число точек 1-плоскости в PG (m, q) равно $q + 1$.

Аналогично t-плоскость, содержащая V_i, $i = 0, 1, \ldots, t$, определяется следующим образом. Пусть $\mathbf{v}_i \in V_i$ при $i = 0, \ldots, t$. Точки множества $\mathbf{v}_i$ должны быть линейно независимыми над $GF^{m+1}(q)$, так как множество V_i образуется различными точками из PG (m, q). Тогда t-плоскость, содержащая $V_0, \ldots, V_t$, — это множество точек PG (m, q), которые являются образами точек

$$\beta_0\mathbf{v}_0 + \beta_1\mathbf{v}_1 + \cdots + \beta_t\mathbf{v}_t$$

в $GF^{m+1}(q)$, где $\beta_0, \ldots, \beta_t$ — произвольные элементы поля, не равные одновременно нулю. Величины $\beta_0, \ldots, \beta_t$ можно выбрать $q^{t+1} - 1$ способами, поэтому образ состоит из $(q^{t+1} - 1)/(q - 1)$ точек из PG (m, q). Отсюда следует, что $(t + 1)$-плоскость содержит $q^t + q^{t-1} + \cdots + q + 1$ точек.

Теорема 13.8.3. *t-плоскость в* PG (m, q) *содержит* $\begin{bmatrix} t-1 \\ t \end{bmatrix}$ *точек и сама имеет структуру проективной геометрии* PG (t, q).

Доказательство. Утверждение о числе точек немедленно следует из равенства $\begin{bmatrix} t-1 \\ 1 \end{bmatrix} = (q^{t+1} - 1)/(q - 1)$. Утверждение о структуре t-плоскости вытекает из описания структуры PG (t, q). □

Теорема 13.8.4.

(i) PG (m, q) *содержит* $\begin{bmatrix} m+1 \\ t+1 \end{bmatrix}$ *различных t-плоскостей при* $t = 0, 1, \ldots, m$.

(ii) *При любых s и t, таких, что* $0 \leqslant s \leqslant t \leqslant m$, *каждая s-плоскость содержится точно в* $\begin{bmatrix} m-s \\ t-s \end{bmatrix}$ *различных t-плоскостях в* PG (m, q).

Доказательство по существу совпадает с доказательством теоремы 13.7.5. □

В приведенной ниже теореме проективно-геометрические коды определяются альтернативным способом. Эта теорема аналогична теореме 13.7.7 и доказывается точно так же. Она будет использована при доказательстве того, что проективно-геометрические коды мажоритарно декодируемы, точно так же как теорема 13.7.7

использовалась при доказательстве того, что евклидово-геометрические коды мажоритарно декодируемы.

Теорема 13.8.5. *Проективно-геометрический код порядка r и длины $n = (q^m - 1)/(q - 1)$ над $GF(p)$ — это наибольший линейный код над $GF(p)$, содержащий в своем нуль-пространстве векторы инцидентности всех r-плоскостей в* PG (m, q).

Доказательство. Достаточно доказать, что непримитивный циклический ОРМ-код, который содержит код, дуальный проективно-геометрическому коду, является наименьшим линейным кодом над $GF(q)$, который содержит все векторы инцидентности r-плоскостей. Это вытекает из того, что компоненты вектора инцидентности могут принимать лишь нулевые и единичные значения и поэтому всегда принадлежат подполю $GF(p)$. Следовательно, вектор инцидентности принадлежит коду, дуальному проективно-геометрическому коду, если он принадлежит ОРМ-коду, содержащему этот дуальный код.

Вектор инцидентности $\mathbf{f}$ принадлежит непримитивному циклическому ОРМ-коду, если компонента F_j преобразования Фурье равна нулю при

$$w_q((q-1)\,j) \leqslant (q-1)\,m - (q-1)\,(m-r-1) - 1,$$

или

$$w_q((q-1)\,j) < (q-1)\,(r+1).$$

Для доказательства достаточно вычислить F_j и показать, что при таких j оно равно нулю.

Шаг 1. Представим r-плоскость как образ множества

$$\{\gamma_{i_r}\mathbf{v}_r + \gamma_{i_{r-1}}\mathbf{v}_{r-1} + \cdots + \gamma_{i_0}\mathbf{v}_0\},$$

где индексы $i_0, i_1, \ldots, i_r$ маркируют q элементов поля $GF(q)$, а $\mathbf{v}_0, \ldots, \mathbf{v}_r$ — фиксированное множество независимых точек в r-плоскости. Точку, в которой все коэффициенты γ равны нулю, исключать не обязательно, так как она не дает вклада в F_j. Вектор инцидентности $\mathbf{f}$ содержит в i-й компоненте единицу, если элемент поля α^i принадлежит этому множеству, и нуль в противном случае. Положим $j' = (q-1)\,j$ при $j = 0, \ldots, (q^m - 1)/(q-1) - 1$ и вычислим спектральные компоненты $F_{j'}$:

$$F_{j'} = \sum_{i=0}^{n-1} \alpha^{ij'} f_i =$$

$$= \sum_{i_0=0}^{q-1} \sum_{i_1=0}^{q-1} \cdots \sum_{i_r=0}^{q-1} (\gamma_{i_r}v_r + \gamma_{i_{r-1}}v_{r-1} + \cdots + \gamma_{i_0}v_0)^{j'},$$

где $v_0, \ldots, v_{r-1}$ считаются элементами $GF(q^p)$. Нужно определить те значения j', при которых эти компоненты равны нулю. Полиномиальное разложение дает

$$F_{j'} = \sum_{i_0=0}^{q-1} \cdots \sum_{i_r=0}^{q-1} \sum_{\mathbf{h}} \frac{j'!}{h_0!\, h_1! \ldots h_r!} (\gamma_{i_r} v_r)^{h_r} \ldots (\gamma_{i_0} v_0)^{h_0},$$

где суммирование производится по всем $\mathbf{h}$, удовлетворяющим разложению $h_0 + h_1 + \ldots + h_r = j'$. Переменим порядок суммирования и рассмотрим сумму $\sum_{l=0}^{q-1} (\gamma_l v)^h$.

Шаг 2. В $\sum_{l=0}^{q-1} \gamma_l^h$ суммирование проводится по всем элементам поля, и поэтому сумма может быть выражена через примитивный элемент α. Отсюда так же, как и при доказательстве теоремы 13.7.7, заключаем, что вклад в $F_{j'}$ вносят лишь те слагаемые, в которых h_l является ненулевым кратным $q - 1$. Поэтому

$$F_{j'} = (-1)^p \sum_{h_0} \sum_{h_1} \cdots \sum_{h_{r+1}} \frac{'j'!}{h_0!\, h_1! \ldots h_r!} v_0^{h_0} v_1^{h_1} \ldots v_r^{h_r},$$

где суммирование проводится по $(h_0, \ldots, h_r)$, таким, что h_l, $l = 0, \ldots, r$, являются ненулевыми кратными $q - 1$ и

$$\sum_{l=0}^{r+1} h_l = j'.$$

Шаги 3 и 4. Как и при доказательстве теоремы 13.7.7, из теоремы Лукаса вытекает, что при всех l величина h_l является q-ичным потомком j'; следовательно,

$$w_q(j') = \sum_{l=0}^{r} w_q(h_l).$$

Но, как показано при доказательстве теоремы 13.7.7, $w_q(h_l)$ являются ненулевыми кратными $q - 1$ тогда и только тогда, когда h_l есть ненулевое кратное $q - 1$. Следовательно, если $F_{j'}$ отлично от нуля, то

$$[w_q(j') \geqslant (q - 1)(r + 1).$$

Это доказывает теорему. □

Доказательство следующей теоремы включает в себя алгоритм мажоритарного декодирования.

Теорема 13.8.6. *Пусть* $q = p^s$. *Проективно-геометрический код* r*-го порядка длины* $n = (q^m - 1)/(q - 1)$ *над* $GF(p)$ *может быть мажоритарно декодирован в* r *шагов при числе ошибок, не превышающем* $(q^{m-r+1} - 1)/[2(q - 1)]$.

Доказательство. Доказательство проводится рекуррентно; показывается, что из множества проверочных равенств, основанных на векторах инцидентности t-плоскостей, можно получить проверочные равенства, основанные на векторах инцидентности $(t-1)$-плоскостей. Отсюда следует, что проверочные равенства, основанные на векторах инцидентности r-плоскостей, в r шагов могут быть сведены к проверочным соотношениям для отдельных символов. Рассуждения аналогичны доказательству теоремы 13.7.8. □

ЗАДАЧИ

13.1. Показать, что двоичный (15,7)-код БЧХ, исправляющий двойные ошибки, мажоритарно декодируем при использовании декодера Меггитта в качестве мажоритарного декодера.

13.2. Построить двухшаговый мажоритарный декодер для двоичного (7, 4)-кода Хэмминга.

13.3. Построить мажоритарный декодер для двоичного проективно-геометрического (21,12)-кода, исправляющего две ошибки.

13.4. Найти порождающий многочлен для (15,6)-кода над GF (4), исправляющего тройные ошибки и являющегося мажоритарно декодируемым. Построить декодер. Найти порождающий многочлен для кода БЧХ над GF (4), исправляющего три ошибки.

13.5. Если $GF(q)$ является полем характеристики 2, то некоторые мажоритарно декодируемые коды над $GF(q)$ с посредственными характеристиками могут быть получены методом перестановок при выборе $J = (q^m - 1)/(q-1)$. Проверить существование следующих кодов:

(255, 156), $t = 8$, над GF (16);
(4095, 2800), $t = 32$, над GF (64);
(65 535, 47 040), $t = 128$, над GF (256).

При проверке вручную для облегчения работы нужно использовать своего рода четырехмерную таблицу, описывающую q-ичные разложения степеней.

13.6. Доказать, что ни один код Рида—Соломона не может быть декодирован мажоритарно в пределах его радиуса упаковки.

13.7. Построить мажоритарный декодер для представленного на рис. 13.6 сверточного (42, 28)-кода.

13.8. Показать, что если r-плоскость из EG (m, q) содержит начало координат, то как векторное пространство она является линейным подпространством в EG (m, q). Для заданной r-плоскости в EG (m, q) указать число r-плоскостей, не пересекающихся с ней.

13.9. Показать, что при определении 1-плоскости в проективной геометрии не имеет значения, как выбирать в GF^m (q) представляющие V_0 и V_1 точки.

13.10. Доказать, что в проективной геометрии не существует двух параллельных линий, т. е. что любая пара 1-плоскостей имеет ненулевое пересечение.

13.11. Построить высокоскоростной декодер для двоичного (63, 24, 15)-кода БЧХ, одновременно используя четыре мажоритарных декодера для ОРМ (63, 22, 15)-кода.

ЗАМЕЧАНИЯ

Четкая формулировка того, что класс мажоритарно декодируемых кодов образует специальный подкласс кодов, исправляющих ошибки, принадлежит Месси [1963]. Он установил общие принципы обращения с такими кодами; до этого

мажоритарное декодирование использовалось лишь в некоторых частных случаях. Первым, кто использовал мажоритарное декодирование, был Рид [1954], применивший его для декодирования кодов Рида—Маллера. В своей работе Месси специально интересовался сверточными кодами и определил класс мажоритарно декодируемых сверточных кодов. Дальнейшее развитие теории сверточных кодов принадлежит Робинсону и Бернстайну [1967], а также Ву [1976].

Нахождение конструктивных классов мажоритарно декодируемых блоковых кодов оказалось более трудным, и здесь работа продвигалась медленно. Решающим вкладом явилось применение конечных геометрий при построении кодов, первоначально предложенное Рудолфом [1964, 1967] [1]) и реализованное Колесником и Мирончиковым [1968], а также Касами, Лином и Питерсоном [1968], установившими, что коды Рида—Маллера являются циклическими (если исключить проверочный символ) и могут быть обобщены на произвольные алфавиты. Обобщенные коды Рида—Маллера были введены в работе Касами, Лина и Питерсона [1968] и разрабатывались Уэлдоном [1967, 1968], Касами, Лином и Питерсоном в их второй статье [1968], Геталсом и Дельсартом [1968] и Дельсартом, Геталсом и Мак-Вильямс [1970]. Простой метод построения, описанный в § 13.4, принадлежит Лину и Марковскому [1980]. Конечно-геометрические коды подробно обсуждались в обзорной статье Геталса [1975].

---

[1]) Независимо от Рудолфа это было сделано В. Д. Колесником и Е. Т. Мирончиковым; см. Колесников В. Д., Мирончиков Е. Т. Некоторые циклические коды и схема декодирования по большинству проверок. — Проблемы передачи информации, 1965, т. 1, вып. 2, с. 3—17. — *Прим. ред.*

ГЛАВА 14

КОМПОЗИЦИЯ И ХАРАКТЕРИСТИКИ КОНТРОЛИРУЮЩИХ ОШИБКИ КОДОВ

Любая инженерная дисциплина начинается обзором методов решения определенного класса задач. После того как эти методы изучены, переходят к вопросу об оптимальности. Являются ли эти методы наилучшими? Если нет, то в чем и насколько они уступают наилучшим методам?

Чтобы ответить на такие вопросы, необходимо знать, насколько хороши известные коды и насколько хороши наилучшие возможные коды. Вообще говоря, мы не можем ответить удовлетворительно ни на один из этих вопросов. Хотя наилучшие возможные контролирующие ошибки коды для конкретных скоростей и длин неизвестны, известны некоторые границы, а именно границы, за которыми ни одного кода не существует, и границы, в пределах которых они заведомо существуют. Знание этих границ позволяет глубже понять предмет.

В этой главе мы изучим композиционную структуру блоковых кодов, а также вероятности ошибочного декодирования и неудачного декодирования. Затем перейдем к границам для наилучших возможных кодов.

14.1. РАСПРЕДЕЛЕНИЯ ВЕСОВ

Мы знаем, что если линейный блоковый код имеет минимальное расстояние d^*, то существует хотя бы одно кодовое слово веса d^*. Иногда нас не удовлетворяет столь малая информация; мы хотим знать, сколько кодовых слов имеют вес d^* и каковы веса других кодовых слов. Например, в табл. 5.3 были приведены веса слов двоичного (23,12)-кода Голея.

Если код мал, то такую таблицу весов можно построить простым перебором. Для кода большой мощности это сделать невозможно, и необходимо разработать, если удастся, аналитические методы. Так как для многих кодов неизвестно даже минимальное расстояние, то очевидно, что такие методы, вообще говоря, найти трудно.

Пусть A_l — число кодовых слов веса l линейного (n, k)-кода. *Распределением весов* кода называется $(n+1)$-мерный вектор с компонентами A_l, $l = 0, \ldots, n$. Очевидно, что если минимальное расстояние кода равно d^*, то $A_0 = 1$, $A_1, \ldots, A_{d^*-1}$ равны нулю, а A_{d^*} отлично от нуля. Более глубокая характеристика кода требует дополнительного анализа.

Аналитическое описание распределения весов кода — трудная задача, но для важного случая кодов Рида — Соломона (или любого кода с максимальным расстоянием) аналитическое решение известно. Настоящий параграф служит введением в эту задачу. В нем получена формула числа кодовых слов каждого веса для кода с максимальным расстоянием. В случае произвольного линейного кода такую формулу мы дать не сможем, но некоторые общие соображения приведем. Будет выведено тождество Мак-Вильямс, связывающее распределение весов кода с распределением весов дуального кода. Тождество Мак-Вильямс полезно в тех случаях, когда дуальный код достаточно мал, так что его распределение весов можно найти на ЭВМ.

Начнем с точного решения задачи для кодов с максимальным расстоянием.

Теорема 14.1.1. *В коде с максимальным расстоянием любое множество k позиций может быть выбрано как множество информационных позиций; значения символов в них выбираются произвольным образом в $GF(q)$. В оставшихся $n-k$ позициях выбор символов определяется правилом кодирования.*

Доказательство. Код с минимальным расстоянием d^* может восстанавливать любые d^*-1 стертых символов. Так как для кодов с максимальным расстоянием $d^* = n-k+1$, то теорема доказана. □

Из доказательства теоремы ясно, что если код не является кодом с максимальным расстоянием, то утверждение о том, что любое множество k позиций может использоваться как множество информационных позиций, неверно. Это обратное утверждение справедливо для всех двоичных кодов, так как ни один двоичный код не является кодом с максимальным расстоянием.

Для кодов с максимальным расстоянием легко вычислить число кодовых слов веса d^*. Разобьем множество целых чисел от 0 до $n-1$ на два непересекающихся подмножества T_{d^*} и $T^c_{d^*}$, где T_{d^*} состоит из d^* целых чисел. Существует $\binom{n}{d^*}$ способов такого разбиения. Рассмотрим все кодовые слова, имеющие нули в тех позициях, которые принадлежат $T^c_{d^*}$. Любое множество $k = n-d^*+1$ позиций в слове кода с максимальным расстоянием однозначно определяет это кодовое слово. Выберем в качестве $n-d^*+1$ информационных позиций $n-d^*$ позиций, принад-

лежащих $T^c_{d^*}$, плюс одну дополнительную позицию. В этой дополнительной позиции символ может принимать одно из q значений, и тогда символы в оставшихся $d^* - 1$ позициях определены. Следовательно, существует точно q кодовых слов, у которых в заданных $n - d^*$ позициях стоят нули. Одно из них — нулевое кодовое слово, а $q - 1$ слов имеют вес, равный d^*. Так как $n - d^*$ нулевых позиций, принадлежащих $T^c_{d^*}$, могут быть выбраны $\binom{n}{d^*}$ способами, то получим

$$A_{d^*} = \binom{n}{d^*}(q-1).$$

Чтобы найти A_l при $l > d^*$, используем сходные, но гораздо более сложные рассуждения. Это будет сделано по ходу доказательства следующей теоремы.

Теорема 14.1.2. *Распределение весов (n, k)-кода над $GF(q)$ с максимальным расстоянием определяется формулами $A_0 = 1$, $A_l = 0$ при $l = 1, \ldots, d^* - 1$ и*

$$A_l = \binom{n}{l}(q-1)\sum_{j=0}^{l-d^*}(-1)^j\binom{l-1}{j}q^{l-d^*-j}$$

при $l \geqslant d^*$.

Доказательство. При $l < d^*$ теорема очевидна. Дальнейшее доказательство распадается на три шага.

Шаг 1. Разобъем совокупность целых чисел от 0 до $n - 1$ на два непересекающихся подмножества T_l и T^c_l, где T_l состоит из l целых чисел, и будем рассматривать лишь те кодовые слова, которые содержат нулевые компоненты в позициях с номерами из T^c_l и ненулевые компоненты во всех других позициях. Пусть M_l — число таких кодовых слов веса l. Для произвольного кода

$$A_l = \binom{n}{l}M_l;$$

поэтому необходимо лишь доказать, что

$$M_l = (q-1)\sum_{j=0}^{l-d^*}(-1)^j\binom{l-1}{j}q^{l-d^*-j}.$$

Чтобы доказать это равенство, установим одно неявное соотношение, связывающее M_l и $M_{l'}$ при l', меньших l, и l, бо́льших d^*.

Выберем совокупность $n - d^* + 1$ информационных компонент следующим образом: будем считать информационными все $n - l$ компонент, расположенных в принадлежащих T_l^c позициях, а также любые $l - d^* + 1$ компонент, расположенных в принадлежащих T_l позициях. Напомним, что компоненты, расположенные в позициях, принадлежащих T_l^c, равны нулю. Произвольным образом приписывая значения оставшимся $l - d^* + 1$ компонентам, получим $q^{l-d^*+1} - 1$ ненулевых кодовых слов, вес которых не превышает l.

В множестве l позиций, принадлежащих T_l, можно выбрать любое подмножество l' позиций. Число кодовых слов веса l' с ненулевыми компонентами в этих l' позициях равно $M_{l'}$. Следовательно,

$$\sum_{l'=d^*}^{l} \binom{l}{l'} M_{l'} = q^{l-d^*+1} - 1.$$

Это неявное соотношение рекуррентно связывает M_{d^*+1} с M_{d^*}, M_{d^*+2} с M_{d^*} и M_{d^*+1} и т. д. Разрешив это рекуррентное соотношение, мы получим явную формулу для M_l.

Шаг 2. Запишем равенства в формулировке теоремы в более удобном для доказательства виде. Чтобы с суммой, входящей в последнее из этих равенств, можно было бы обращаться как с многочленом, будем рассматривать q как неопределенную переменную. Введем оператор $[\ ]$:

$$\left[\sum_{n=-N_1}^{N_2} a_n q^n \right] = \sum_{n=0}^{N_2} a_n q^n,$$

сохраняющий в сумме коэффициенты лишь при неотрицательных степенях n. Заметим, что он определяет линейную операцию. В этих обозначениях соотношение, которое должно быть доказано, можно переписать в виде

$$M_l = (q-1)\left[q^{-(d^*-1)} \sum_{j=0}^{l-1} (-1)^j \binom{l-1}{j} q^{l-1-j} \right].$$

Дополнительные слагаемые, включенные в эту сумму, соответствуют отрицательным степеням q и не вносят вклад в M_l. Свернем ее, используя формулу бинома Ньютона:

$$M_l = (q-1)\,[q^{-(d^*-1)}\,(q-1)^{l-1}].$$

Шаг 3. Покажем, что полученное на втором шаге выражение для M_l позволяет разрешить рекуррентное соотношение, выведенное на первом шаге:

$$\sum_{l'=d^*}^{l} \binom{l}{l'} M_{l'} = \sum_{l'=0}^{l} \binom{l}{l'} M_{l'} =$$

$$= (q-1) \sum_{l'=0}^{l} \binom{l}{l'} [q^{-(d^*-1)} (q-1)^{l'-1}] =$$

$$= (q-1) \left[q^{-(d^*-1)} (q-1)^{-1} \sum_{l'=0}^{l} \binom{l}{l'} (q-1)^{l'} \right] =$$

$$= (q-1) [q^{-d^*} (1-1/q)^{-1} q^l] =$$

$$= (q-1) \left[\sum_{i=0}^{\infty} q^{l-d^*-i} \right] =$$

$$= (q-1) \sum_{i=0}^{l-d^*} q^{l-d^*-i} = q^{l-d^*+1} - 1.$$

Это и завершает доказательство. □

Следствие 14.1.3. *Распределение весов (n, k)-кода над $GF(q)$ с максимальным расстоянием дается соотношениями $A_0 = 1$, $A_l = 0$ при $l = 1, \ldots, d^* - 1$ и*

$$A_l = \binom{n}{l} \sum_{j=0}^{l-d^*} (-1)^j \binom{l}{j} (q^{l-d^*+1-j} - 1)$$

при $l \geqslant d^*$.

Доказательство. Воспользуемся тождеством

$$\binom{l}{j} = \binom{l-1}{j} + \binom{l-1}{j-1}$$

и перепишем доказываемое равенство в виде

$$A_l = \binom{n}{l} \sum_{j=0}^{l-d^*} (-1)^j \left[\binom{l-1}{j} + \binom{l-1}{j-1} \right] (q^{l-d^*+1-j} - 1) =$$

$$= \binom{n}{l} \left[\sum_{j=0}^{l-d^*} (-1)^j \binom{l-1}{j} (q q^{l-d^*-j} - 1) - \right.$$

$$\left. - \sum_{j=1}^{l-d^*+1} (-1)^{j-1} \binom{l-1}{j-1} (q^{l-d^*+1-j} - 1) \right] =$$

$$= \binom{n}{l} \sum_{i=0}^{l-d^*} (-1)^i \binom{l-1}{i} (q-1) q^{l-d^*-i}.$$

Последнее равенство вытекает из теоремы 14.1.2. □

l	A_l
0	1
1	0
2	0
3	0
4	0
5	0
6	0
7	0
8	0
9	0
10	0
11	0
12	0
13	0
14	0
15	0
16	0
17	$8{,}22 \times 10^{9}$
18	$9{,}59 \times 10^{10}$
19	$2{,}62 \times 10^{12}$
20	$4{,}67 \times 10^{13}$
21	$7{,}64 \times 10^{14}$
22	$1{,}07 \times 10^{16}$
23	$1{,}30 \times 10^{17}$
24	$1{,}34 \times 10^{18}$
25	$1{,}17 \times 10^{19}$
26	$8{,}37 \times 10^{19}$
27	$4{,}81 \times 10^{20}$
28	$2{,}13 \times 10^{21}$
29	$6{,}83 \times 10^{21}$
30	$1{,}41 \times 10^{22}$
31	$1{,}41 \times 10^{22}$

Рис. 14.1. Распределение весов (31, 15)-кода Рида—Соломона.

Следствие 14.1.3 полезно при нахождении распределения весов кодов Рида — Соломона, например распределения весов (31, 15)-кода Рида—Соломона, приведенного на рис. 14.1. Даже для таких кодов Рида — Соломона небольшой мощности число кодовых слов веса l может быть очень большим. Вот почему, вообще говоря, практически невозможно найти распределение весов кода простым перебором кодовых слов.

В случае кодов, не являющихся кодами с максимальным расстоянием, не существует правила, аналогичного теореме 14.1.2.

При небольших n распределение весов может быть найдено поиском на ЭВМ, но с ростом n этот метод быстро становится непрактичным.

В настоящее время наиболее сильным инструментом являются выражения, устанавливающие связь между распределением весов линейного кода и распределением весов его дуального кода — так называемые *тождества Мак-Вильямс*. Тождества Мак-Вильямс справедливы для любого линейного кода; они вытекают из векторной структуры линейного кода и из того факта, что дуальный коду $\mathscr{C}$ код является ортогональным дополнением $\mathscr{C}$. Прежде чем переходить к выводу тождеств Мак-Вильямс, мы должны вернуться к изучению абстрактных конечномерных векторных пространств, начатому в § 2.6. Необходимо ввести понятия пересечения и прямой суммы двух подпространств и доказать их некоторые свойства.

Пусть U и V — подпространства пространства F^n. Тогда $U \cap V$ называется *пересечением* U и V и означает множество векторов, принадлежащих и U, и V; $U \oplus V$ называется *прямой суммой* U и V и означает множество всех линейных комбинаций $a\mathbf{u} + b\mathbf{v}$, где $\mathbf{u}$ и $\mathbf{v}$ принадлежат соответственно U и V, а a и b — скаляры. Как $U \cap V$, так и $U \oplus V$ являются подпространствами в F^n.

Теорема 14.1.4.

$$\dim [U \cap V] + \dim [U \oplus V] = \dim [U] + \dim [V].$$

Доказательство. Базис подпространства $U \cap V$ состоит из $\dim [U \cap V]$ векторов. Этот базис может быть расширен до базиса U добавлением $\dim [U] - \dim [U \cap V]$ базисных векторов и до базиса V добавлением $\dim [V] - \dim [U \cap V]$ базисных векторов. Все эти базисные векторы в совокупности образуют базис подпространства $U \oplus V$. Таким образом,

$$\dim [U \oplus V] = \dim [U \cap V] + \dim [U] - \dim [U \cap V] + \dim [V] - \dim [U \cap V],$$

откуда следует теорема. □

Теорема 14.1.5.

$$U^\perp \cap V^\perp = (U \oplus V)^\perp.$$

Доказательство. Подпространство U содержится в $U \oplus V$, и поэтому $(U \oplus V)^\perp$ содержится в $U^\perp$. Аналогично $(U \oplus V)^\perp$ содержится в $V^\perp$. Следовательно, $(U \oplus V)^\perp$ содержится в $U^\perp \cap V^\perp$. С другой стороны, запишем элемент из $U \oplus V$ как $a\mathbf{u} + b\mathbf{v}$ и обозначим через $\mathbf{w}$ любой элемент из $U^\perp \cap V^\perp$. Тогда $\mathbf{w} \cdot (a\mathbf{u} + b\mathbf{v}) = 0$, и, следовательно, $U^\perp \cap V^\perp$ содержится в $(U \oplus V)^\perp$. Отсюда следует, что эти два множества совпадают. □

Пусть A_l, $l = 0, \ldots, n$, и B_l, $l = 0, \ldots, n$, представляют собой распределения весов линейного кода и его дуального кода соответственно. Определим нумераторы весов кода равенствами

$$A(x) = \sum_{l=0}^{n} A_l x^l \quad \text{и} \quad B(x) = \sum_{l=0}^{n} B_l x^l.$$

Следующая теорема связывает эти два многочлена и позволяет вычислить один из них, если известен другой.

Теорема 14.1.6. *Нумератор весов $A(x)$ линейного (n, k)-кода над $GF(q)$ и нумератор весов $B(x)$ дуального ему кода связаны соотношением*

$$q^k B(x) = [1 + (q-1)x]^n A\left(\frac{1-x}{1+(q-1)x}\right).$$

Доказательство. Рассмотрим код $\mathscr{C}$ и дуальный ему код $\mathscr{C}^\perp$. Доказательство состоит из двух частей. В первой части мы докажем, что

$$\sum_{i=0}^{n} B_i \binom{n-i}{m} = q^{n-k-m} \sum_{j=0}^{n} A_j \binom{n-j}{n-m}$$

для $m = 0, \ldots, n$. Во второй части мы докажем, что это равенство эквивалентно утверждению теоремы.

Часть 1. Для заданного m разобъем целые числа от 0 до $n-1$ на два непересекающихся подмножества T_m и T_m^c, где подмножество T_m состоит из m элементов. Пусть V образует m-мерное подпространство векторного пространства $GF^n(q)$, состоящее из всех векторов с нулевыми компонентами в позициях с номерами из T_m. Тогда $V^\perp$ является $(n-m)$-мерным подпространством, состоящим из всех векторов с нулевыми компонентами в позициях с номерами из T_m.

В силу теоремы 14.1.5

$$(\mathscr{C} \cap V)^\perp = \mathscr{C}^\perp \oplus V^\perp;$$

поэтому

$$\dim[\mathscr{C}^\perp \oplus V^\perp] = n - \dim[\mathscr{C} \cap V].$$

С другой стороны, по теореме 14.1.4

$$\dim[\mathscr{C}^\perp \oplus V^\perp] = (n-k) + (n-m) - \dim[\mathscr{C}^\perp \cap V^\perp].$$

Из этих равенств следует, что

$$\dim[\mathscr{C}^\perp \cap V^\perp] = \dim[\mathscr{C} \cap V] + n - k - m.$$

Теперь заметим, что при каждом выборе T_m существует $q^{\dim[\mathscr{C} \cap V]}$ векторов в $\mathscr{C} \cap V$ и $q^{\dim[\mathscr{C}^\perp \cap V^\perp]}$ векторов в $\mathscr{C}^\perp \cap V^\perp$.

ассмотрим $\{T_m\}$ — совокупность всех таких T_m. Пересчитаем екторы в каждом $\mathscr{C} \cap V$, которые могут быть получены из некоторого подмножества T_m, принадлежащего совокупности $\{T_m\}$. Число пересчитанных векторов равно $\sum_{\{T_m\}} q^{\dim[\mathscr{C} \cap V]}$, причем многие из них входят в подсчет несколько раз. Аналогично пересчитав все векторы в каждом $\mathscr{C}^\perp \cap V^\perp$, которое получается из T_m, принадлежащего $\{T_m\}$, имеем

$$\sum_{\{T_m\}} q^{\dim[\mathscr{C}^\perp \cap V^\perp]} = q^{n-k-m} \sum_{\{T_m\}} q^{\dim[\mathscr{C} \cap V]}.$$

Чтобы завершить первую часть доказательства, необходимо оценить две суммы, входящие в это равенство. Для этого сосчитаем, сколько раз вектор веса j, принадлежащий $\mathscr{C}$, появится в множестве $\mathscr{C} \cap V$. Указанный вектор принадлежит $\mathscr{C} \cap V$ тогда и только тогда, когда эти его j позиций входят в число тех m позиций принадлежащих V векторов, в которых допускаются ненулевые компоненты, или, что эквивалентно, если $n - m$ позиций, в которых принадлежащие V векторы должны содержать нулевые компоненты, попадают в $n - j$ позиций кодового слова, содержащие нулевые компоненты. Выбрать $n - m$ позиций, которые содержат нулевые компоненты, можно $\binom{n-j}{n-m}$ способами; поэтому данное кодовое слово веса j входит в $\binom{n-j}{n-m}$ множеств. Существует A_j кодовых слов веса j. Поэтому

$$\sum_{\{T_m\}} q^{\dim[\mathscr{C} \cap V]} = \sum_{j=0}^{n} A_j \binom{n-j}{n-m}.$$

Аналогично можно сосчитать векторы в $\mathscr{C}^\perp \cap V^\perp$. Тогда предыдущее равенство переходит в равенство

$$\sum_{i=0}^{n} B_i \binom{n-i}{m} = q^{n-k-m} \sum_{j=0}^{n} A_j \binom{n-j}{n-m}.$$

Так как m произвольно, первая часть доказательства завершена.

Часть 2. Исходя из выводов первой части, запишем тождество для многочленов

$$\sum_{m=0}^{n} y^m \sum_{i=0}^{n} B_i \binom{n-i}{m} = \sum_{m=0}^{n} y^m q^{n-k-m} \sum_{j=0}^{n} A_j \binom{n-j}{n-m}.$$

Переменим порядок суммирования

$$\sum_{i=0}^{n} B_i \sum_{m=0}^{n-i} \binom{n-i}{m} y^m = q^{n-k} \sum_{j=0}^{n} A_i \sum_{m=0}^{n} \binom{n-j}{n-m} \left(\frac{y}{q}\right)^n \left(\frac{q}{y}\right)^{n-m},$$

вспомнив, что $\binom{n-i}{m} = 0$, если $m > n - i$. Используя формулу бинома Ньютона, имеем

$$\sum_{i=0}^{n} B_i (1+y)^{n-i} = q^{n-k} \sum_{j=0}^{n} A_j (y/q)^n (1+q/y)^{n-j}.$$

Наконец, сделав подстановку $y = 1/x - 1$, получим

$$q^k x^{-n} \sum_{i=0}^{n} B_i x^i = q^n \sum_{j=0}^{n} A_j \left(\frac{1-x}{xq}\right)^n \left(\frac{1+(q-1)x}{1-x}\right)^{n-j},$$

или

$$q^k \sum_{i=0}^{n} B_i x^i = (1+(q-1)x)^n \sum_{j=0}^{n} A_j \left(\frac{1-x}{1+(q-1)x}\right)^j,$$

что завершает доказательство теоремы. □

В заключение параграфа укажем на простое применение этой теоремы. Из табл. 1.1 следует, что распределение весов (7,4)-кода Хэмминга равно

$$(A_0, A_1, \ldots, A_7) = (1, 0, 0, 7, 7, 0, 0, 1),$$

так что нумератор весов имеет вид

$$A(x) = x^7 + 7x^4 + 7x^3 + 1.$$

Соответствующий дуальный код — это двоичный циклический код, известный под названием симплексного. Корнями его порождающего многочлена

$$g(x) = x^4 + x^3 + x^2 + 1$$

являются α^0 и α^1. Согласно теореме 14.1.6, нумератор весов $B(x)$ симплексного кода удовлетворяет равенству

$$2^4 B(x) = (1+x)^7 A[(1-x)/(1+x)] =$$

$$= (1-x)^7 + 7(1+x)^3 (1-x)^4 + 7(1+x)^4 (1-x)^3 + (1+x)^7,$$

откуда следует, что

$$B(x) = 7x^4 + 1.$$

Симплексный (7,3)-код имеет одно кодовое слово веса 0 и семь кодовых слов веса 4.

14.2. ВЕРОЯТНОСТИ ОШИБОЧНОГО ДЕКОДИРОВАНИЯ И НЕУДАЧНОГО ДЕКОДИРОВАНИЯ

Неполный декодер для корректирующего t ошибок кода исправляет в пределах радиуса упаковки все конфигурации ошибок веса, не превышающего t, и не исправляет ни одной конфигурации ошибок веса больше t. Если происходит более t ошибок, то декодер иногда заявляет, что сообщение не подлежит исправлению, а иногда делает ошибку при декодировании. Следовательно, на выходе декодера может появляться правильное сообщение, неправильное сообщение или декодирование объявляется неудачным (сообщение стирается). В общем случае неизвестно, как вычислить вероятности этих событий, но в некоторых частных случаях, представляющих практический интерес, получены удовлетворительные выражения.

Рассмотрим случай использования линейных кодов в каналах, вносящих в символы ошибки независимо и симметрично. Выражения для указанных вероятностей зависят от распределения весов $\{A_l\}$ кода и поэтому полезны лишь в случае, когда оно известно. В предыдущем параграфе получено распределение весов для кодов Рида—Соломона, так что для этих кодов мы можем вычислить вероятности ошибочного декодирования и неудачного декодирования.

Рассматриваемые каналы представляют собой q-ичные каналы, вносящие ошибки в передаваемые символы независимо с вероятностью P и передающие символы правильно с вероятностью $1 - P$. Каждое из $q - 1$ ошибочных значений принимается с вероятностью $P/(q - 1)$. Каждая конфигурация k ошибок наблюдается с вероятностью

$$p(k) = \left(\frac{P}{q-1}\right)^k (1 \cdot P)^{n-k}.$$

Мы рассмотрим лишь неполный декодер, который не принимает решения о декодировании вне радиуса упаковки кода. Каждое принятое слово, лежащее от ближайшего кодового слова на расстоянии, не превышающем t, декодируется в это кодовое слово; здесь t — фиксированное число, удовлетворяющее неравенству

$$2t + 1 \leqslant d^*.$$

Исследуем условные вероятности ошибочного и неудачного декодирования в случае линейного кода при условии, что передано слово, целиком состоящее из нулей. Каждое другое переданное слово приводит к тем же самым условным вероятностям, так что они являются также безусловными вероятностями. На рис. 14.2 изображены три области, в которые может попасть принятое слово.

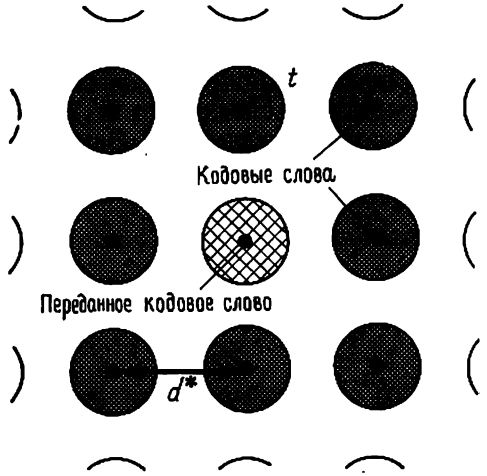


Рис. 14.2. Области декодирования.

Вероятность правильного декодирования — это вероятность того, что принятое слово попадает в область, покрытую редкой штриховкой. Вероятность неправильного декодирования — это вероятность того, что принятое слово попадет в область, покрытую частой штриховкой. Вероятность неудачного декодирования — это вероятность того, что принятое слово попадет в незаштрихованную область. Сумма этих трех вероятностей равна 1, поэтому достаточно найти формулы лишь для двух из них.

Начнем с самого простого.

Теорема 14.2.1. *Вероятность правильного декодирования декодера, исправляющего ошибки в пределах радиуса упаковки кода, равна*

$$p_c = \sum_{\nu=0}^{t} \binom{n}{\nu} P^{\nu}(1-P)^{n-\nu}.$$

Доказательство. Число способов, которыми можно разместить конфигурации из ν ошибок, равно $\binom{n}{\nu}$; каждая из них имеет место с вероятностью $P^{\nu}(1-P)^{n-\nu}$. Отсюда следует утверждение теоремы. □

Хотя теорема справедлива для любых объемов алфавита, в формулу входит лишь вероятность появления ошибочного символа. Не имеет значения, как она распадается на вероятности конкретных ошибочных значений. В следующей теореме, касающейся неудачного декодирования, необходимо подсчитать число способов, которыми могут быть сделаны ошибки. Некоторые кон-

фигурации ошибок, вероятности которых нужно просуммировать, приведены на рис. 14.3.

Пусть $N(l, h; s)$ означает число конфигураций ошибок веса h, находящихся на расстоянии s от кодового слова веса l. Ясно, что $N(l, h; s)$ не зависит от выбора кодового слова веса l. Заметим, что если $N(l, h; s) \neq 0$, то $l - s \leqslant h \leqslant l + s$. В приведенной ниже теореме это условие выполняется автоматически, так как мы условились считать $\binom{n}{m} = 0$ при $m < 0$ или $m > n$.

Теорема 14.2.2. *Число конфигураций ошибок веса h, находящихся на расстоянии s от конкретного кодового слова веса l, равно*

$$N(l, h; s) = \sum_{\substack{0 \leqslant i, j \leqslant n \\ i+2j+h=s+l}} \binom{n-l}{j+h-l} \binom{l}{i} \binom{l-i}{j} (q-1)^{j+h-l} (q-2)^i.$$

Доказательство. Равенству, которое требуется доказать, эквивалентно равенство

$$N(l, h; s) = \sum_{\substack{i, j, k \\ i+j+k=s \\ l+k-j=h}} \left[\binom{n-l}{k} (q-1)^k\right] \left[\binom{l}{i} (q-2)^i\right] \left[\binom{l-i}{j}\right],$$

что может быть проверено подстановкой $k = j + h - l$. В нем фигурируют три индекса суммирования и два ограничения. Суммируется число кодовых слов, которые могут быть получены заменой любых k из $n - l$ нулевых компонент кодового слова на любой из $q - 1$ ненулевых символов, любых i из l ненулевых

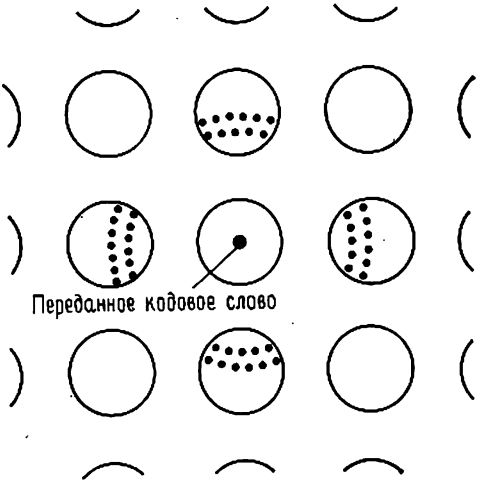


Рис. 14.3. Некоторые слова, вызывающие ошибку декодирования.

компонент на любой из оставшихся $q-2$ ненулевых символов и любых j из оставшихся ненулевых компонент на нули. Ограничение $i+j+k=s$ гарантирует, что результирующее слово находится на расстоянии s от кодового слова. Ограничение $l+k-j=h$ гарантирует, что результирующее слово имеет вес h. □

Теорема 14.2.3. *Если $2t+1 \leqslant d^*$ и декодер исправляет все конфигурации ошибок, вес которых не превышает t, то вероятность ошибочного декодирования равна*

$$p_e = \sum_{h=0}^{n} [P/(1-q)]^h (1-P)^{n-h} \sum_{s=0}^{t} \sum_{l=1}^{n} A_l N(l, h; s).$$

Доказательство. Число конфигураций ошибок веса h, которые приводят к ошибке декодирования, равно $\sum_{s=0}^{t} \sum_{l=1}^{n} A_l N(l, h; s)$; в силу условия $2t+1 \leqslant d^*$ ни одна из конфигураций ошибок не сосчитана дважды. Вероятность каждой из них равна $[P/(q-1)]^h (1-P)^{n-h}$, что доказывает теорему. □

Теоремы 14.2.2, 14.2.3 и 14.1.2 содержат все необходимое для нахождения вероятности ошибки кода Рида—Соломона. Эта вероятность может быть вычислена непосредственно на ЭВМ. Приведенные соотношения легко модифицировать применительно к декодеру, исправляющему ошибки и стирания. Если число стираний равно ρ, то нужно просто заменить n на $n-\rho$, а p_e на условную вероятность ошибки $p_{e|\rho}$. Тогда если распределение вероятностей $Q(\rho)$ числа стираний ρ известно, то p_e может быть вычислено по формуле $p_e = \sum_{\rho} p_{e|\rho} Q(\rho)$.

14.3. РАСПРЕДЕЛЕНИЕ ВЕСОВ СВЕРТОЧНЫХ КОДОВ

Вес бесконечно длинного кодового слова сверточного кода равен числу его ненулевых компонент. Свободное расстояние — это вес кодового слова минимального веса. Свободная длина — это длина сегмента кодового слова минимального веса до его последнего ненулевого кадра включительно. Если код настолько прост, что его решетку можно построить непосредственно, то все эти параметры можно найти по решетке. Например, на рис. 14.4 перерисована решетка сверточного (6,3)-кода, исследованного в гл. 12. Один из путей имеет вес 5, и этот вес является минимальным расстоянием кода. Длина этого пути равна трем кадрам, поэтому свободная длина кода равна 6. Информационная последовательность пути, определяющего свободное расстояние, содержит одну единицу. Изучая далее решетку, увидим, что вес 6 имеют два пути, вес 8 — один путь и вес 10 — один путь. Таким образом

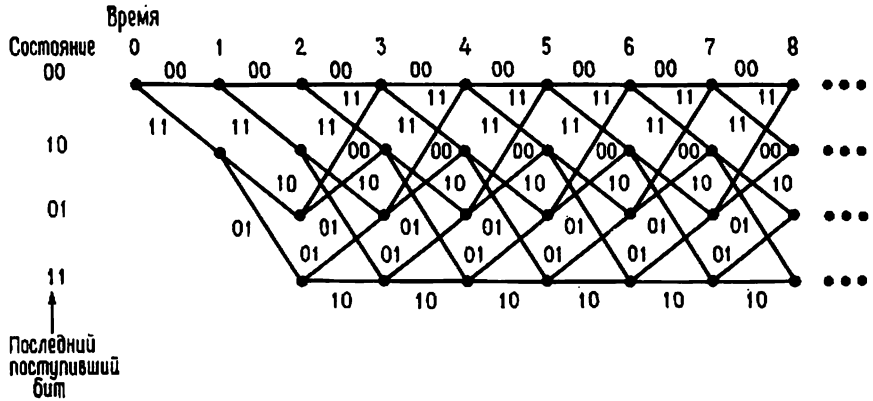


Рис. 14.4. Представление сверточного (6, 3)-кода с помощью решетки.

можно пересчитать число путей каждого веса, но это занятие быстро становится утомительным. В этом параграфе мы опишем более эффективную процедуру.

На рис. 14.5, *а* решетка после исключения временно́й оси сведена к диаграмме состояний. Для удобства эта диаграмма состояний перерисована на рис. 14.5, *б* с маркировкой путей. Так как мы интересуемся только путями, которые начинаются и заканчиваются 00, узел, маркированный символами 00, представлен дважды: один раз как вход, второй раз как выход. На этой диаграмме состояний легко проследить путь веса 5.

Вес пути можно найти проще, связав вклад каждого пути со степенью некоторой формальной переменной D и воспользовавшись методом, предложенным для анализа диаграмм состояний. Модифицированная диаграмма состояний приведена на рис. 14.6, где вес ребра представлен степенью D. Вес пути получается умножением вкладов всех ребер вдоль этого пути.

Вычислим общий вклад $T(D)$ совокупности всех путей между входом и выходом, решив следующую систему уравнений, найденных по рис. 14.6:

$$b = Dc + Db,$$

$$c = D^2a + b,$$

$$d = Dc + Dd,$$

$$e = D^2b.$$

Решение этих уравнений дает

$$e = \frac{D^5}{1 - 2D}a,$$

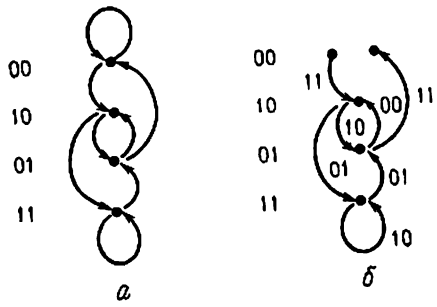


Рис. 14.5. Диаграмма состояний для сверточного (6, 3)-кода.

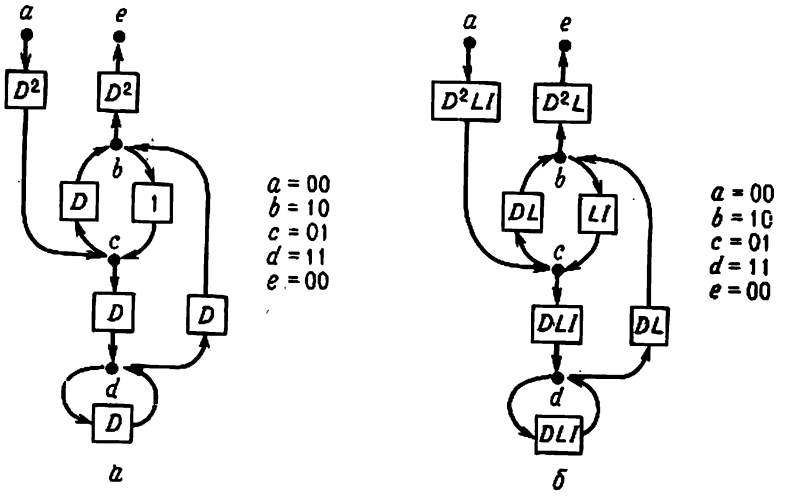


Рис. 14.6. Модифицированная диаграмма состояний для сверточного (6, 3)-кода. *а* — с переменной D, подсчитывающей задержку; *б* — с переменными D, L и I, подсчитывающими соответственно задержку, число кадров и число единиц на входе.

и поэтому передаточная функция равна

$$T(D) = \frac{D^5}{1 - 2D} = D^5 \cdot (2D^6 + 4D^7 + \ldots + 2^k D^{k+5} + \ldots).$$

Следовательно, вес $k + 5$ имеют 2^k путей.

Тот же метод можно использовать для изучения других свойств кода. Введем две новые формальные переменные, а именно L для подсчета кадров и I для подсчета числа единиц на входе. Вклад любого ребра равен одному L; если ребро соответствует поступлению единичного входного символа, то его вклад равен

одному I, а вклады остальных ребер не содержат I. Поэтому, как и ранее, получим

$$T(D, L, I) = \frac{D^5L^3I}{1 - DL(1+L)I} =$$
$$= D^5L^3I + D^6L^4(1+L)I^2 + D^7L^5(1+L)^2I^3 + \ldots$$
$$\ldots + D^{5+k}L^{3+k}(1+L)^kI^{1+k}.$$

Мы видим, что путь веса 5 имеет длину 3 и соответствует информационной последовательности с одним единичным символом. Вес 6 имеют два пути, каждому из которых соответствует информационная последовательность с двумя единицами; длина одного из этих путей равна 5, а другого 6.

Таким образом мы полностью определим свойства сверточного кода. Эти свойства полезно знать при сравнении структур двух сверточных кодов; кроме того, знание дистанционной структуры кода необходимо при оценке вероятности ошибки кода, используемого в конкретном канале.

14.4. ГРАНИЦЫ МИНИМАЛЬНОГО РАССТОЯНИЯ ДЛЯ БЛОКОВЫХ КОДОВ

О контролирующем ошибки коде с длиной n и скоростью R судят по его минимальному расстоянию d^*. Если имеются два кода с одинаковыми n и R, то в общем случае следует предпочесть код с бóльшим d^*. Для данного кода $\mathscr{C}$ хотелось бы знать, является ли его d^* максимально большим среди минимальных расстояний кодов с данными n и R. Вообще говоря, не считая случаев, когда n довольно мало, ответ на этот вопрос неизвестен, и можно указать лишь некоторые грубые границы для d^*. В частности, мы уже встречались с границей Синглтона, которая при n, много бóльших q, становится весьма грубой.

Мы проведем лишь асимптотический анализ. При фиксированном объеме алфавита q введем обозначение

$$d(n, R) = \max_{\mathscr{C}} d^*(\mathscr{C}),$$

где максимум берется по всем кодам с длиной n и скоростью R, а $d^*(\mathscr{C})$ равно

$$d^*(\mathscr{C}) = \min_{\substack{x, y \in \mathscr{C} \\ x \neq y}} d(x, y)$$

и представляет собой минимальное расстояние кода $\mathscr{C}$. Функция $d(n, R)$ определяет наибольшее минимальное расстояние среди всех кодов над $GF(q)$ со скоростью R и длиной n. Если не считать

области малых значений n, функция $\bar{d}(n, R)$ неизвестна. Определим, далее,

$$\bar{d}(R) = \lim_{n\to\infty} [\bar{d}(n, R)/n]$$

при условии, что этот предел существует. Если функция $\bar{d}(R)$ известна, то мы можем сказать, что при достаточно больших n наилучший блоковый код со скоростью R имеет минимальное расстояние d^*, приблизительно равное $n\bar{d}(R)$. Поэтому функция $\bar{d}(R)$, если она известна, дает нам критерий, по которому можно судить о существующих классах контролирующих ошибки кодов. Кроме того, знание $\bar{d}(R)$ может дать нам некоторый ключ к изучению структуры хороших кодов и указать путь их построения.

Хотя знать функции $\bar{d}(n, R)$ и $\bar{d}(R)$ было бы весьма желательно, в настоящее время они не известны. Все что мы знаем — это их нижние и верхние границы; некоторые из них выводятся в данном параграфе. Остановимся лишь на тех границах, которые можно получить, не прилагая больших усилий; в частности, мы получим границы, известные как *нижняя граница Гилберта* [1]) и *верхняя граница Элайса*.

Чтобы получить эти границы, воспользуемся методом исчерпывания и оценим композиции типичных кодовых слов. Композицией q-ичного кодового слова назовем q-мерный вектор $(n_0, \ldots, n_{q-1})$, $\sum_l n_l = n$, l-я компонента которого определяет, сколько раз в кодовом слове встречается l-я буква. Относительной частотой q-ичного кодового слова называется q-мерный вектор $(p_0, \ldots, p_{q-1})$, где $p_l = n_l/n$. Выразим границы минимального расстояния через полиномиальные коэффициенты $n!/\Pi_l n_l!$, где $\Pi_l n_l!$ означает произведение $\Pi_{l=0}^{q-1}(n_l!)$. Функция энтропии $H(p)$ определяется как

$$H(p) = -\sum_l p_l \log p_l.$$

Мы хотим доказать асимптотически справедливое при больших n равенство

$$\frac{n!}{\Pi_l (n_l!)} \approx e^{nH(p)}$$

и сделать это как можно короче. Этому условию удовлетворяет доказательство, использующее формулу Стирлинга

$$\sqrt{2n\pi}\left(\frac{n}{e}\right)^n < n! < \sqrt{2n\pi}\left(\frac{n}{e}\right)^n \left(1 + \frac{1}{12n-1}\right).$$

---

[1]) Эта граница известна также под названием границы Варшамова—Гилберта. — *Прим. ред.*

Теорема 14.4.1.

$$e^{n[H(p)-o_2(1)]} < \frac{n!}{\Pi_l (n_l!)} < e^{n[H(p)-o_1(1)]},$$

где

$$o_1(1) = \frac{1}{2n}(q-1)\log(2n\pi) + \frac{1}{2n}\sum_l \log p_l - \frac{1}{n}\log\left(1 + \frac{1}{12n-1}\right),$$

$$o_2(1) = \frac{1}{2n}(q-1)\log(2n\pi) + \frac{1}{2n}\sum_l \log p_l + \frac{1}{n}\sum_l \log\left(1 + \frac{1}{12np_l - 1}\right).$$

Доказательство.

$$\log \frac{n!}{\Pi_l (n_l!)} < \log \frac{\sqrt{2n\pi}\,(n/e)^n [1 + 1/(12n-1)]}{\Pi_l [\sqrt{2n_l\pi}\,(n_l/e)^{n_l}]} =$$

$$= \log \frac{\Pi_l n^{n_l}}{\Pi_l n_l^{n_l}} + \log \frac{\sqrt{2n\pi}\,[1 + 1/(12n-1)]}{\Pi_l \sqrt{2n_l\pi}}.$$

Замена $n_l = np_l$ доказывает правую часть неравенства. Левая часть доказывается тем же способом. □

Теперь можно приступить к доказательству границы Гилберта. Мы проведем его, подсчитав число кодовых слов внутри сферы с центром в произвольной точке пространства (не обязательно представляющей кодовое слово) и затем усреднив это число. Число точек внутри сферы радиуса d назовем *объемом* сферы; оно равно

$$V = \binom{n}{0} + (q-1)\binom{n}{1} + \ldots + (q-1)^d \binom{n}{d}.$$

Теорема 14.4.2. *Пусть V — число точек внутри сферы радиуса d с центром в некоторой точке векторного пространства $GF^n(q)$. Для заданного кода $\mathscr{C}$ мощности q^k среднее число кодовых слов внутри сферы радиуса d с центром в произвольной точке пространства равно*

$$\overline{T} = q^k V/q^n.$$

Доказательство. Опишем сферы радиуса d вокруг каждой точки пространства. Подсчитаем число кодовых слов внутри каждой сферы и затем просуммируем по всем сферам. Так как на расстоянии, не превышающем d, вокруг кодового слова находится V точек, это слово лежит внутри V таких сфер. Поэтому сумма числа кодовых слов внутри всех сфер равна $q^k V$, и, следовательно, в среднем на сферу приходится $q^k V/q^n$ кодовых слов. □

Теорема 14.4.3. *Над GF (q) существуют коды со скоростью R и минимальным расстоянием d, для которых*

$$R \geqslant 1 - (1/n) \log_q V,$$

где V — объем сферы радиуса d.

Доказательство. Воспользуемся теоремой 14.4.2 и будем считать, что $\overline{T} \geqslant 1$. Если это не так, то можно взять любую точку, у которой на расстоянии, не превышающем d, нет кодовых слов, и объявить ее кодовым словом, тем самым расширив код. Следовательно,

$$q^{k-n} V \geqslant 1,$$

или

$$n - k \leqslant \log_q V,$$

откуда следует утверждение теоремы.

Теорема 14.4.4 (граница Гилберта). *Функция $d(R) \geqslant \delta$ для всех значений δ, которые удовлетворяют неравенству*

$$R \geqslant 1 - \delta \log_q (q-1) + \delta \log_q \delta + (1-\delta) \log_q (1-\delta).$$

Доказательство. Объем V представляет сумму d слагаемых, из которых последнее является наибольшим [1]). Поэтому

$$V < d (q-1)^d \binom{n}{d}.$$

Используя теорему 14.4.1 и сделав подстановку $\delta = d/n$, получим

$$V < d (q-1)^{n\delta} q^{n [-\delta \log_q \delta - (1-\delta) \log_q (1-\delta) - o(1)]},$$

где $o(1)$ стремится к нулю при n, стремящемся к бесконечности. Далее,

$$\frac{1}{n} \log_q V < \delta \log_q (q-1) - \delta \log_q \delta - (1-\delta) \log_q (1-\delta) - o'(1).$$

Подставив это неравенство в неравенство, доказанное в теореме 14.4.3 и опустив в асимптотике член $o'(1)$, завершим доказательство. □

Для двоичных кодов граница Гилберта принимает вид

$$R \geqslant 1 - H_b(\bar{d}),$$

где

$$H_b(\bar{d}) = -\bar{d} \log_2 \bar{d} - (1-\bar{d}) \log_2 (1-\bar{d}).$$

Графически эта граница изображена на рис. 14.7.

---

[1]) При $d \leqslant n/2$. — *Прим. ред.*

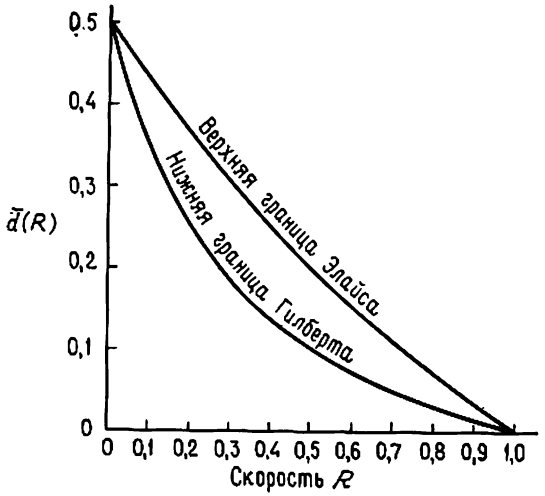


Рис. 14.7. Некоторые известные границы функции $d(R)$ при $q = 2$.

Теперь выведем верхнюю границу для $\bar{d}(R)$, называемую границей Элайса. Предположим, что код имеет $M = e^{nR}$ кодовых слов длины n. *Совместной композицией* кодовых слов $\mathbf{c}$ и $\mathbf{c}'$ называется таблица $\{n_{ll'}^{\mathbf{cc}'}\}$ размера $q \times q$, где элемент $n_{ll'}^{\mathbf{cc}'}$ определяет число позиций, в которых кодовое слово $\mathbf{c}$ содержит символ l, а кодовое слово $\mathbf{c}'$ — символ l'; следовательно,

$$d(\mathbf{c}, \mathbf{c}') = \sum_{\substack{l, l' \\ l \neq l'}} n_{ll'}^{\mathbf{cc}'}.$$

На расстоянии t от каждого кодового слова находится всего

$$A_t = (q-1)^t \binom{n}{t}$$

слов; это число равно количеству слов, расположенных на сфере радиуса t.

Теорема 14.4.5. *Пусть A_t — число кодовых слов на сфере радиуса t с центром в любой точке векторного пространства $GF^n(q)$. Для заданного кода $\mathscr{C}$ мощности q^k среднее число кодовых слов на сфере с центром в произвольной точке пространства равно*

$$\overline{T}_s = q^k A_t / q^n.$$

Доказательство. Так как на расстоянии t от кодового слова располагается A_t точек, оно лежит на A_t таких сферах. Поэтому сумма числа кодовых слов, расположенных на всех таких сферах, равна $q^k A_t$, и среднее число кодовых слов на сфере равно $q^k A_t / q^n$. □

Выберем сферу так, чтобы на ней число кодовых слов было бы не меньше среднего. Удобно представить себе, что центром такой сферы является слово, целиком состоящее из нулей. Для этого достаточно так отобразить на себя исходное пространство, включая кодовые слова, чтобы центр выбранной сферы совпал со словом, целиком состоящим из нулей. Это не повлияет на минимальное расстояние кода.

Таким образом получим сферу радиуса t с центром в начале координат, на которой лежит T кодовых слов, где T не меньше $q^{k-n}A_t$. В дальнейшем величина t будет выбрана так, чтобы T было достаточно большим и удовлетворяло нашим целям. Следующий шаг — оценка расстояния между двумя кодовыми словами, находящимися на сфере. Чтобы получить ее, оценим среднее расстояние между такими кодовыми словами

$$d_{\text{av}} = \frac{1}{T(T-1)} d_{\text{tot}},$$

где d_{tot} — общее расстояние, получаемое суммированием расстояний между неупорядоченными парами кодовых слов (не обязательно различных) на сфере:

$$d_{\text{tot}} = \sum_{\mathbf{c},\,\mathbf{c}'} d(\mathbf{c}, \mathbf{c}') = \sum_{\mathbf{c},\,\mathbf{c}'} \sum_{\substack{l,\,l' \\ l \neq l'}} n_{ll'}^{\mathbf{cc}'}.$$

Напомним, что $\{n_{ll'}^{\mathbf{cc}'}\}$ — это совместная композиция кодовых слов $\mathbf{c}$ и $\mathbf{c}'$. Упростим суммирование по $\mathbf{c}$ и $\mathbf{c}'$. Пусть $n_{ll'|i}^{\mathbf{cc}'}$ равно единице, если в i-й позиции $\mathbf{c}$ стоит l, а $\mathbf{c}'$ имеет в этой позиции l'; в противном случае оно равно нулю. Пусть $T_{l|i}$ — число кодовых слов на сфере, имеющих l в i-й позиции. Тогда

$$n_{ll'}^{\mathbf{cc}'} = \sum_{i=0}^{n-1} n_{ll'|i}^{\mathbf{cc}'},$$

$$d_{\text{tot}} = \sum_{\mathbf{c},\,\mathbf{c}'} \sum_{\substack{l,\,l' \\ l \neq l'}} \sum_{i} n_{ll'|i}^{\mathbf{cc}'} =$$

$$= \sum_{i} \sum_{\substack{l,\,l' \\ l \neq l'}} \sum_{\mathbf{c},\,\mathbf{c}'} n_{ll'|i}^{\mathbf{cc}'} = \sum_{i} \sum_{\substack{l,\,l' \\ l \neq l'}} T_{l|i} T_{l'|i}.$$

Введем теперь величину

$$\lambda_{l|i} = T_{l|i}/T,$$

равную доле кодовых слов на сфере, имеющих l в i-й позиции. Общее расстояние между такими кодовыми словами равно

$$d_{\text{tot}} = T^2 \sum_{i} \sum_{l,\,l'} \lambda_{l|i} \lambda_{l'|i} D_{ll'},$$

где $D_{ll'}$ равно единице, если $l \neq l'$, и нулю в противном случае. Значения $\lambda_{l\,|\,i}$ нам неизвестны, но мы знаем, что они удовлетворяют двум следующим ограничениям:

$$\sum_{l=0}^{q-1} \lambda_{l\,|\,i} = 1 \quad \text{и} \quad \sum_{l=1}^{q-1} \sum_{i=0}^{n-1} \lambda_{l\,|\,i} = t.$$

Так как $\lambda_{l|i}$ неизвестны, рассмотрим наихудший случай, выбрав их значения так, чтобы при одновременном выполнении указанных ограничений величина в правой части формулы для d_{tot} была бы как можно больше. Если предварительно доказано, что максимальная функция является выпуклой функцией вероятностного вектора $\boldsymbol{\lambda}$, то один из методов решения задачи состоит в использовании множителей Лагранжа. Чтобы упростить доказательство, воспользуемся вместо этого элементарным приемом: сначала укажем, какие значения $\lambda_{l\,|\,i}$ являются максимизирующими, а затем докажем, что они в самом деле таковы.

Лемма 14.4.6. *Пусть* $F(\boldsymbol{\lambda}) = \sum_i \sum_{l,\, l'} \lambda_{l\,|\,i} \lambda_{l'\,|\,i} D_{ll'}$, *где* $D_{ll'}$ *равно единице, если* $l \neq l'$, *и нулю в противном случае. Если для* $\boldsymbol{\lambda}$ *имеют место ограничения*

$$\sum_{l=0}^{q-1} \lambda_{l\,|\,i} = 1 \quad \text{и} \quad \sum_{l=1}^{q-1} \sum_{i=0}^{n-1} \lambda_{l\,|\,i} = t,$$

то

$$F(\boldsymbol{\lambda}) \leqslant t \left[\frac{q}{q-1} - \frac{1 + (q-1)^2}{(q-1)^2} \frac{t}{n} \right].$$

Доказательство. Прежде всего заметим, что из указанных ограничений вытекает соотношение

$$\sum_{i=0}^{n-1} \lambda_{0\,|\,i} = n - t.$$

Пусть

$$\lambda^*_{l\,|\,i} = \begin{cases} 1 - t/n, & l = 0, \\ (t/n)/(q-1), & l = 1, \ldots, q-1 \end{cases}$$

при всех i. Убедимся, что эти значения максимизируют $F(\boldsymbol{\lambda})$. Заметим, что

$$\sum_{l=0}^{q-1} \lambda^*_{l|i} D_{ll'} = \begin{cases} t/n, & l' = 0, \\ 1 - [(t/n)/(q-1)], & l' \neq 0. \end{cases}$$

Поэтому при всех λ, которые удовлетворяют нашим ограничениям, включая $\lambda = \lambda^*$,

$$\sum_{i=0}^{n-1}\sum_{l'=0}^{q-1}\lambda_{l'|i}\sum_{l=0}^{q-1}\lambda^*_{l|i}D_{ll'} =$$

$$= \left[\sum_{l'=1}^{q-1}\sum_{i=0}^{n-1}\lambda_{l'|i}\left(1-\frac{t/n}{q-1}\right)\right] + \left[\sum_{i=0}^{n-1}\lambda_{0|i}\frac{t}{n}\right] =$$

$$= t\left(1-\frac{t/n}{q-1}\right) + (n-t)\frac{t}{n}.$$

Раскрыв скобки и проведя упрощения в левой части этого равенства, получим

$$\sum_{i}\sum_{l,l'}(\lambda_{l|i} - \lambda^*_{l|i})(\lambda_{l'|i} - \lambda^*_{l'|i})D_{ll'} =$$

$$= \sum_{i}\sum_{l,l'}(\lambda_{l|i}\lambda_{l'|i} - \lambda^*_{l|i}\lambda^*_{l'|i})D_{ll'}.$$

Достаточно показать, что величина в левой части этого равенства неположительна. Введем обозначение

$$c_{l|i} = (\lambda_{l|i} - \lambda^*_{l|i})$$

и заметим, что $\sum_l c_{l|i} = 0$. Необходимо доказать, что при любом i

$$\sum_{l,l'}(c_{l|i}c_{l'|i})D_{ll'} \leqslant 0.$$

Но при любом i

$$\sum_{l,l'}c_{l|i}c_{l'|i}D_{ll'} = \sum_{l,l'}c_{l|i}c_{l'|i} - \sum_{l}c^2_{l|i} = 0 - \sum_{l}c^2_{l|i} \leqslant 0.$$

Доказательство леммы завершается вычислением значения $F(\lambda^*)$. □

Теорема 14.4.7 (граница Элайса). *Пусть*

$$T = q^{-(n-k)}(q-1)^t\binom{n}{t},$$

где t произвольно. Если $T > 1$, то любой код над $GF(q)$ со скоростью R и длиной n содержит пару кодовых слов, расстояние d между которыми удовлетворяет неравенству

$$d \leqslant \frac{T}{T-1}t\left[\frac{q}{q-1} - \frac{1+(q-1)^2}{(q-1)^2}\frac{t}{n}\right].$$

Доказательство. Выберем величину t в теореме 14.4.5 так, чтобы $\bar{T}_s$ было больше единицы. Тогда существует сфера, на

которой число кодовых слов T больше единицы. Среднее расстояние между этими кодовыми словами удовлетворяет неравенству

$$d_{\text{av}} \leqslant [T/(T-1)] \sum_{l} \sum_{l'} \lambda_{l\,|\,i} \lambda_{l'\,|\,i} D_{ll'}.$$

Для завершения доказательства необходимо применить лемму 14.4.6 и воспользоваться тем, что хотя бы два кодовых слова должны располагаться друг от друга на расстоянии, не меньшем среднего. □

Следствие 14.4.8 (граница Элайса). *В двоичных кодах со скоростью R относительное минимальное расстояние асимптотически удовлетворяет неравенству*

$$d\,(R) \leqslant 2\rho\,(1-\rho)$$

при любых ρ, для которых $1 - H_b(\rho) > R$.

Доказательство. Положим в теореме 14.4.7 $q = 2$, $t/n = \rho$ и перепишем неравенство теоремы 14.4.7 в виде

$$\frac{d^*}{n} \leqslant \frac{T}{T-1} 2\rho\,(1-\rho),$$

где

$$T = 2^{n\delta + o(1)} \quad \text{и} \quad \delta = H_b(\rho) - 1 + R.$$

По условиям следствия величина δ положительна; поэтому $T/(T-1)$ стремится к единице, когда n стремится к бесконечности. Следствие доказано. □

14.5. ГРАНИЦЫ МИНИМАЛЬНОГО РАССТОЯНИЯ ДЛЯ СВЕРТОЧНЫХ КОДОВ

Минимальное l-е расстояние d_l^* для сверточного кода с длиной блока $n = (m+1)\,n_0$ определяется как минимальное расстояние Хэмминга среди всех пар начальных сегментов кодовых слов длины $m+1$ кадров, у которых первые кадры различны. Минимальное расстояние d^* сверточного кода равно d_{m+1}^*, а свободное расстояние определяется соотношением

$$d_\infty = \max_l d_l^*.$$

В этом параграфе мы найдем границы типа границ Гилберта и Элайса для d^* и d_∞.

Очевидно, что любой сверточный код можно усечь до длины N и получить блоковый код длины N; этот блоковый код не может быть лучше наилучшего блокового кода длины N. Целью настоящего параграфа является изучение поведения расстояния в зависимости от других мер длины кода, таких, как длина блока и

свободная длина. Конечно, если несколькими способами определять и кодовое расстояние, и длину кода, то, комбинируя эти определения, можно получить различные границы.

Границы, которые будут получены в настоящем параграфе, можно сравнивать с границами для блоковых кодов. Однако делать какие-либо заключения опасно, так как не ясно, какой параметр сверточного кода соответствует длине блокового кода.

Пусть объем алфавита q фиксирован; определим

$$\bar{d}(n, R) = \max_{\mathscr{C}} d^*(\mathscr{C}),$$

где максимум берется по всем сверточным кодам с длиной блока n и скоростью R, а $d^*(\mathscr{C})$ означает минимальное расстояние для кода $\mathscr{C}$. Функция $\bar{d}(n, R)$ определяет наибольшее минимальное расстояние по всем сверточным кодам над $GF(q)$ со скоростью R и длиной блока n. Обозначим, далее,

$$\bar{d}(R) = \lim_{n\to\infty} [\bar{d}(n, R)/n]$$

при условии, что этот предел существует. Если функция $\bar{d}(R)$ известна, то мы можем сказать, что при достаточно больших n наилучший сверточный код со скоростью R имеет минимальное расстояние d^*, приблизительно равное $n\bar{d}(R)$. Поэтому функция $\bar{d}(R)$, если она известна, дает нам критерий, по которому можно оценивать сверточные коды.

Аналогичные определения можно дать и для свободного расстояния. В зависимости от того, что рассматривается: отношение свободного расстояния к длине блока или к свободной длине, можно получить различные функции. Определим

$$\bar{d}_\infty(n, R) = \max_{\mathscr{C}} d_\infty(\mathscr{C}),$$

где максимум берется по всем сверточным кодам с длиной блока n и скоростью R, а $d_\infty(\mathscr{C})$ означает свободное расстояние кода $\mathscr{C}$. Функция $\bar{d}_\infty(n, R)$ определяет наибольшее свободное расстояние по всем сверточным кодам над $GF(q)$ со скоростью R и длиной блока n.

Обозначим, далее,

$$\bar{d}_\infty(R) = \lim_{n\to\infty} [\bar{d}_\infty(n, R)/n]$$

при условии, что этот предел существует. Как и ранее, если функция $\bar{d}_\infty(R)$ известна, то мы можем сказать, что при достаточно больших n наилучший сверточный код со скоростью R и длиной блока n имеет свободное расстояние d_∞, приблизительно равное $n\bar{d}_\infty(R)$.

Так как обычно $d_\infty(\mathscr{C})$ больше $d^*(\mathscr{C})$, можно ожидать, что и $\delta_\infty(R)$ больше $\delta(R)$. В некоторых случаях полезно использовать также следующее альтернативное определение. Пусть $\bar{n}_\infty(n, R)$ означает свободную длину кода, на котором достигается $d_\infty(n, R)$. Свободная длина более тесно связана со свободным расстоянием и сложностью декодера, чем длина кода. Обозначим

$$\delta_\infty(R) = \lim_{n\to\infty} [d_\infty(n, R)/\bar{n}_\infty(n, R)]$$

при условии, что этот предел существует. Тогда если функция $\delta_\infty(R)$ известна, то можно сказать, что при достаточно большой свободной длине $\bar{n}$ наилучший сверточный код со скоростью R имеет свободное расстояние, приблизительно равное $\bar{n}\delta_\infty(R)$.

Мы ограничимся построением границы лишь для функции $\delta(R)$. Границу типа границы Элайса легко получить, воспользовавшись ранее выведенной границей для блокового кода. Эта граница будет приведена лишь для двоичных сверточных кодов.

Теорема 14.5.1 (граница Элайса). *Для двоичного сверточного кода со скоростью R функция $\delta(R)$ удовлетворяет условию*

$$\delta(R) \leqslant 2\rho(1-\rho)$$

при любом ρ, таком, что $1 - H_b(\rho) > R$.

Доказательство. Усечением сверточного кода с длиной блока $n = mn_0$, где m произвольно, получим блоковый код длины n. Скорость этого блокового кода также равна R, так как каждые k_0 информационных символов кодируются n_0 символами кодового слова. Минимальное расстояние этого усеченного кода не больше минимального расстояния наилучшего блокового кода. Следовательно, любая верхняя граница минимального расстояния блокового кода является также верхней границей минимального расстояния сверточного кода [1]). Такой границей является граница Элайса. □

Граница Гилберта не может быть получена использованием границы Гилберта для блоковых кодов; ее нужно доказывать непосредственно. Заметим также, что поскольку сверточные коды линейны, приведенная ниже граница Гилберта сильнее, чем граница Гилберта, ранее доказанная для блоковых кодов, которая лишь утверждала существование некоторого кода, не обязательно линейного.

Прежде чем доказывать границу Гилберта, необходимо доказать лемму. Лемма касается начального сегмента кодового слова

[1]) Это утверждение неточно. — *Прим. ред.*

сверточного кода, т. е. сегмента кодового слова, состоящего из первых n символов.

Лемма 14.5.2. *Начальный сегмент кодового слова, начинающийся с ненулевого кадра, входит точно в* $q^{(m+1)(n_0-k_0)(k_0-1)}$ *различных систематических сверточных* (n_0, k_0)*-кодов с длиной слова* $(m+1)k_0$.

Доказательство. Так как код систематический, задание первых n символов кодового слова определяет первые $(m+1)k_0$ информационных символов. Первые $(m+1)n_0$ символов кодового слова связаны с первыми $(m+1)k_0$ информационными символами через порождающую матрицу. Эта матрица описывается $m+1$ матрицами $\mathbf{P}_0, \ldots, \mathbf{P}_m$, каждая из которых имеет размер $k_0 \times (n_0 - k_0)$. Проверочные кадры $(\mathbf{p}_0, \mathbf{p}_1, \ldots, \mathbf{p}_m)$, число которых равно $m+1$, определяются следующим матричным равенством:

$$(\mathbf{p}_0, \mathbf{p}_1, \ldots, \mathbf{p}_m) = (\mathbf{i}_0, \mathbf{i}_1, \ldots, \mathbf{i}_m) \begin{bmatrix} \mathbf{P}_0 & \mathbf{P}_1 & \ldots & \mathbf{P}_m \\ 0 & \mathbf{P}_0 & \ldots & \mathbf{P}_{m-1} \\ 0 & 0 & \ldots & \mathbf{P}_{m-2} \\ & & & \vdots \\ & & & \mathbf{P}_0 \end{bmatrix}.$$

Поэтому $\mathbf{i}_0\mathbf{P}_0 = \mathbf{p}_0$; это равенство задает $n_0 - k_0$ ограничений на $k_0(n_0 - k_0)$ элементов $\mathbf{P}_0$; отсюда следует, что $(k_0 - 1)(n_0 - k_0)$ элементов могут быть выбраны независимо. Следовательно, при фиксированном первом кадре кодового слова матрица $\mathbf{P}_0$ может быть выбрана $q^{(k_0-1)(n_0-k_0)}$ способами. Аналогично $\mathbf{i}_0\mathbf{P}_1 = \mathbf{p}_1 - \mathbf{i}_0\mathbf{P}_0$ и матрица $\mathbf{P}_1$ может быть выбрана $q^{(k_0-1)(n_0-k_0)}$ способами. Продолжая процесс, убеждаемся, что каждая матрица $\mathbf{P}_i$ может быть выбрана столькими же способами, и поэтому существует $[q^{(k_0-1)(n_0-k_0)}]^{m+1}$ сверточных кодов, у которых начальное кодовое слово фиксировано. □

Теорема 14.5.3. *Пусть при заданной скорости* $R = k_0/n_0$ *и длине блока* $n = (m+1)n_0$ *величина* ρ *удовлетворяет условию*

$$\sum_{j=1}^{\rho} \binom{n}{j} (q-1)^j < q^{n(1-R)}.$$

Тогда существует хотя бы один систематический сверточный код над $GF(q)$, *минимальное расстояние* d^* *которого больше* ρ.

Доказательство. Минимальное расстояние сверточного кода больше ρ, если код не содержит начального сегмента кодового слова длины n и веса не больше ρ, у которого некоторый информационный символ первого кадра отличен от нуля. В множестве

всех сверточных кодов с длиной блока n можно сосчитать общее число начальных кодовых слов, вес которых не больше ρ и которые имеют ненулевой символ в первом кадре. Если их число меньше числа систематических сверточных кодов, то существует хотя бы один код, у которого минимальное расстояние больше ρ.

Так как систематический сверточный (n_0, k_0)-код описывается $(n_0 - k_0)\,k_0$ порождающими многочленами степени не выше m и каждый такой многочлен может быть выбран q^{m+1} различными способами, то число таких кодов равно $q^{(m+1)\,(n_0-k_0)\,k_0}$.

Ненулевые n-последовательности веса, не превышающего ρ, можно выбрать

$$\sum_{j=1}^{\rho} \binom{n}{j} (q-1)^j$$

способами. Согласно лемме 14.5.2, каждый начальный сегмент кодового слова, имеющий ненулевой символ в первом кадре, может войти не более чем в $q^{(m+1)\,(n_0-k_0)\,(k_0-1)}$ различных систематических сверточных (n_0, k_0)-кодов. Следовательно, если

$$\left[\sum_{j=1}^{\rho} \binom{n}{j} (q-1)^j\right] q^{(m+1)\,(n_0-k_0)\,(k_0-1)} < q^{(m+1)\,(n_0-k_0)\,k_0},$$

то должен существовать хотя бы один код с минимальным расстоянием, бо́льшим ρ. Теорема доказана. □

Следствие 14.5.4 (граница Гилберта). *Для двоичных сверточных кодов*

$$\bar{d}(R) \geqslant H_b^{-1}(1-R).$$

Доказательство. При $q = 2$ теорема утверждает, что

$$\sum_{j=1}^{\bar{d}(n,\,R)} \binom{n}{j} \geqslant 2^{n(1-R)}.$$

Следовательно,

$$(1/n) \log \sum_{j=1}^{\bar{d}(n,\,R)} \binom{n}{j} \geqslant 1 - R.$$

Отсюда, используя теорему 14.4.1 и переходя к пределу, получаем утверждение следствия. □

ЗАДАЧИ

14.1. Рассмотреть (31, 27)-код Рида—Соломона.
а. Сколько кодовых слов имеет вес 5?
б. Сколько конфигураций ошибок имеют вес 3?
в. Какая часть конфигураций ошибок веса 3 не обнаруживается?

г. Пусть слово выбрано случайно (с одинаковыми вероятностями для всех его значений). Какова вероятность того, что оно будет декодировано?

14.2. Используя тождество Мак-Вильямс и табл. 5.3, найти распределение весов кода, дуального расширенному (24, 12)-коду Голея. (В действительности расширенный (24, 12)-код Голея является дуальным самому себе; такие коды называются *самодуальными*.)

14.3. Доказать, что в линейном блоковом коде над $GF(2)$ либо не существует слов с нечетным весом, либо половина слов имеет нечетный вес.

14.4. Простейшим двумерным двоичным кодом с N строками и M столбцами является код, исправляющий одну ошибку. В каждой строке и в каждом столбце он имеет по одному проверочному символу. Минимальное расстояние кода равно 4. Предположим, что этот код используется для обнаружения ошибок в двоичном симметричном канале с переходной вероятностью q. Какова вероятность необнаруженной ошибки? При вычислениях необходимо учитывать лишь член с минимальной степенью q.

14.5. Исправляющий t ошибок (n, k)-код Рида—Соломона имеет минимальное расстояние $d^*=n-k+1=2t+1$ и число слов веса d^* равно $A_{d^*}=\binom{n}{d^*}(q-1)$.

а. Сколько существует конфигураций ошибок веса $t+1$?

б. Непосредственным подсчетом найти число конфигураций ошибок веса $t+1$, которые декодер, декодирующий в пределах радиуса упаковки, декодирует неправильно.

в. Какая доля конфигураций ошибок веса $t+1$ декодируется неправильно?

г. Дать численный ответ на вопрос п. в в случаях $q=32$, $t=5$ и $q=64$, $t=6$.

14.6. Оценить сумму компонент распределения весов $\sum_{l=0}^{n} A_l$.

14.7. Исправляющий t ошибок (n, k)-код Рида—Соломона имеет минимальное расстояние $d^*=n-k+1=2t+1$ и число слов веса d^* равно $A_{d^*}=\binom{n}{d^*}\times$ $\times(q-1)$.

а. Сколько существует конфигураций пакетов ошибок веса $t+1$ и длины $t+1$?

б. Найти число конфигураций пакетов ошибок веса $t+1$ и длины $t+1$, которые декодер, декодирующий в пределах радиуса упаковки, декодирует неправильно.

в. Какая доля конфигураций пакетов ошибок веса $t+1$ и длины $t+1$ декодируется неправильно?

г. Дать численный ответ на вопрос п. в в случаях $q=32$, $t=5$ и $q=64$, $t=6$.

14.8. Пусть код Рида—Соломона над $GF(2^m)$ используется как код, исправляющий многократные пакеты ошибок над $GF(2)$. Рассмотреть декодирование принятых слов, которые находятся вне радиуса упаковки кода.

14.9. Порождающие многочлены двоичного сверточного (8, 4)-кода равны

$$g_1(x)=x^3+x+1, \quad g_2(x)=x^3+x^2+x+1.$$

Найти его свободное расстояние, свободную длину и число единиц во входной последовательности, соответствующей пути с длиной, равной свободной.

ЗАМЕЧАНИЯ

Распределение весов кода с максимальным расстоянием было найдено независимо Ассмусом, Мэттсоном и Турином [1965], Форни [1966], Касами, Лином и Питерсоном [1966]. Тождество Мак-Вильямс было выведено ею в 1963 г. В этой же статье она дала свою трактовку связи между распределением весов и вероят-

ностью ошибки декодирования. Иная трактовка принадлежит Форни [1966] и Хунтуну и Михельсону [1977]. Использование диаграммы состояний для нахождения распределения весов сверточного кода предложил Витерби [1971].

Нижняя граница минимального расстояния блоковых кодов опубликована Гилбертом [1952]. Элайс никогда не публиковал свою верхнюю границу; впервые она появилась в его лекциях в 1960 г. Известны и лучшие верхние границы, но они доказываются гораздо сложнее. Наилучшая известная верхняя граница найдена Мак-Элисом, Родемичем, Рамсеем и Велчем [1977]. В недавно выполненной в СССР работе [1]) показано, что в GF (49) граница Гилберта может быть улучшена.

---

[1]) См. работу: Tsfasman M. A., Vladuts S. G., Zink Th. Modular curves, Shimura curves and Goppa codes, better than Varshamov—Gilbert bound. — Math. Nachr., 1982, Bd 104, S. 13—28. Она существенно опирается на работу: Гоппа В. Д. Коды на алгебраических кривых. — ДАН СССР, 1981, т. 259, № 6, с. 1289—1290. — *Прим. ред.*

ГЛАВА 15

ЭФФЕКТИВНАЯ ПЕРЕДАЧА СИГНАЛОВ ПО ЗАШУМЛЕННЫМ КАНАЛАМ

Сердцем любой цифровой системы связи обычно является аналоговый канал—канал непрерывный как по амплитуде, так и по времени. На вход канала подается аналоговый сигнал, представляющий собой непрерывную функцию времени с вещественными или комплексными значениями. Цифровая система связи для такого канала включает модулятор/демодулятор (модем), преобразующий аналоговый канал в дискретный, так, чтобы можно было использовать кодер/декодер (кодек).

Использование контролирующих ошибки кодов приносит наилучшие результаты при разумном согласовании устройства модема с устройством кодека. На этом уровне проектирования совместно используются теория информации, теория связи и теория контролирующих ошибки кодов. Чтобы представить контролирующие ошибки коды в более широком контексте, мы вкратце коснемся в этой главе некоторых разделов теории информации и теории связи. В таком контексте станет более ясной роль контролирующих ошибки кодов как одного из средств проектирования эффективных систем передачи сигналов по непрерывным зашумленным каналам.

В настоящей главе описаны алгоритмы декодирования, использующие дополнительную информацию, получаемую от демодулятора. При одном и том же коде эти декодеры с мягким решением, вообще говоря, имеют лучшие характеристики, чем декодеры с жестким решением, но в силу их большей сложности они полезны лишь для коротких кодов. Исключением является класс последовательных декодеров; эти декодеры могут без труда использовать информацию мягкого решения даже для кодов с большой длиной кодового ограничения, однако при этом возникает проблема переполнения буфера.

15.1. ОГРАНИЧЕННЫЙ ПО ПОЛОСЕ ГАУССОВСКИЙ КАНАЛ

Пусть на вход общего сигнального канала в течение некоторого длительного интервала T можно подавать любую непрерывную функцию $s(t)$, где t пробегает значения от 0 до T, и выполняется условие конечности интеграла

$$E_m = \int_0^T s^2(t)\,dt.$$

Этот интеграл от $s^2(t)$ называется *энергией сообщения* E_m для сигнала $s(t)$. Если T стремится к бесконечности, то *средняя мощность* S определяется как предел

$$S = \lim_{T\to\infty} [E_m(T)/T]$$

при условии, что он существует; здесь $E_m(T)$ — энергия сообщения на интервале длины T. Если сигнал выбирается случайно из ансамбля сигналов, то используются другие определения мощности, такие, как предел математического ожидания отношения $E_m(T)/T$ по ансамблю сигналов.

Каналы обычно накладывают ограничения на вид допустимых сигналов. Канал, ограниченный по полосе, — это канал, который пропускает лишь те сигналы с модулированной несущей, которые известны под названием *ограниченных по полосе сигналов*. Сигнал с модулированной несущей определяется выражением

$$s_c(t) = a(t)\cos(2\pi f_0 t + \theta(t)) =$$

$$= s_R(t)\cos 2\pi f_0 t + s_I \sin 2\pi f_0 t,$$

где $a(t)$ и $\theta(t)$ — вещественные функции, задающие так называемые амплитудную модуляцию и фазовую модуляцию соответственно, а $s_R(t)$ и $s_I(t)$ называются вещественной и мнимой компонентами модуляции соответственно. Последние два термина объясняются зачастую более удобным альтернативным комплексным описанием модуляции

$$s_c(t) = \mathrm{Re}\{s(t)\,e^{j2\pi f_0 t}\},$$

где $s(t) = s_R(t) + js_I(t)$. Комплексный сигнал $s(t)$, ограниченный по полосе, представляет собой лишь математическую фикцию; наблюдаемый сигнал $s_c(t)$ определен ранее. Величина f_0, называемая несущей частотой, постоянна и не играет существенной роли в изучаемых нами задачах, хотя и имеет главенствующее значение в передаваемом сигнале.

Спектр сигнала $s(t)$ определяется преобразованием Фурье

$$S(f) = \int_{-\infty}^{\infty} s(t)\, e^{-2j\pi ft}\, dt.$$

Ограниченный по полосе сигнал — это сигнал, спектр которого сосредоточен в малом интервале частотной оси около несущей частоты f_0. Длина этого интервала называется *шириной полосы* и обозначается W, причем W меньше $2f_0$. В идеале $S(f) = 0$ при $|f - f_0| > W/2$, но на практике приемлемо более слабое условие, такое, как $|S(f)| < \varepsilon$ при $|f - f_0| > W/2$.

После кодирования кодовое слово или последовательность кодовых слов должны быть отображены в модулированные сигналы $(a(t), \theta(t))$ или $(s_R(t), s_I(t))$. Эта операция отображения называется *модуляцией*, а обратная ей операция называется *демодуляцией*. Изучение этих операций составляет главный предмет теории цифровой связи. Исторически исследование контролирующих ошибки кодов велось независимо от исследования цифровой модуляции. В этой главе, однако, мы будем придерживаться такого подхода к системам модуляции и демодуляции, при котором вопросы, относящиеся к контролирующим ошибки кодам и к цифровой модуляции, излагаются совместно.

Выходной сигнал $v(t)$ ограниченного по полосе гауссовского канала с входным сигналом $s(t)$ определяется сверткой

$$v(t) = h(t) * (s(t) + n(t)),$$

где $h(t)$ — ограниченная по полосе функция, называемая *импульсным откликом* канала, а *шумовой сигнал* $n(t)$ — выборочная функция гауссовского случайного процесса.

Гауссовский шум является стандартом, который обычно используется для суждения о качестве схемы модуляции даже в том случае, когда допускается использование модема при другом шумовом процессе. Более того, в качестве стандартного гауссовского шумового процесса выбирается белый шум. Односторонняя спектральная плотность сигнала в виде белого шума равна N_0 Вт/Гц.

Теория цифровой связи утверждает, что если $2W\Delta T \ll 1$, то можно рассматривать входной и выходной сигналы изучаемого нами канала лишь в дискретные равноотстоящие моменты времени $k\Delta T$. Поэтому дискретный во времени входной сигнал будем представлять как совокупность отсчетов $s(k\,\Delta T) = s_R(k\,\Delta T) + js_I(k\,\Delta T)$, или, короче, $s_k = s_{Rk} + js_{Ik}$ при $k = 0, \pm 1, \pm 2, \ldots$, где s_{Rk} и s_{Ik} вещественны. Хорошо известная теорема отсчетов позволяет восстановить $s(t)$ по отсчетам.

Обычно удобно выбирать $h(t)$ (быть может, путем добавления к модулятору и демодулятору специальных выравнивающих

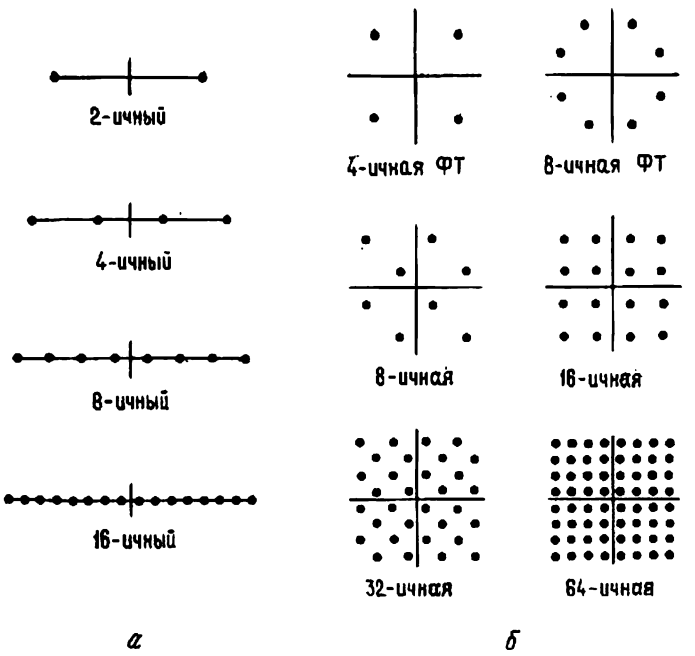


Рис. 15.1. Некоторые наборы сигналов. *а* — вещественные модуляционные алфавиты; *б* — комплексные модуляционные алфавиты (фазовая телеграфия).

фильтров) так, чтобы $h(k\,\Delta T)$ равнялось нулю при k, отличных от нуля, и равнялось единице при k, равном нулю. Тогда принятый сигнал $v(t)$ представляется дискретными отсчетами

$$v_k = s_k + n_k = (s_{Rk} + n_{Rk}) + j\,(s_{Ik} + n_{Ik}), \quad k = 0, \pm 1, \pm 2, \ldots,$$

где $\{n_k\}$ — отсчет вносимого аналоговым каналом шума в моменты $k\,\Delta T$. В случае когда k-й выходной отсчет зависит лишь от k-го входного отсчета, говорят, что межсимвольная интерференция отсутствует. Если $n(t)$ — гауссовский процесс, то n_k — комплексные гауссовские величины с нулевым средним. Более того, можно так организовать передачу по каналу, чтобы гауссовские случайные величины n_k можно было рассматривать как независимые. Этот случай изучается в первую очередь.

Очень часто при построении цифровой системы связи для представления m-битовых байтов выбирают совокупность 2^m вещественных или комплексных чисел, называемую *набором сигналов* (или *модуляционным алфавитом*). На рис. 15.1 приведены некоторые типичные наборы сигналов. Представлены четыре модуляционных алфавита с вещественными значениями, объеди-

няемые названием M-ичная амплитудная модуляция, и шесть модуляционных алфавитов с комплексными значениями, для обозначения которых используется несколько названий, а именно фазовая телеграфия (сокращенно ФТ), квадратурная амплитудная модуляция, амплитудно-фазовая модуляция (сокращенно АФМ). Не обсуждая выбор наборов сигналов, рассмотрим лишь их использование применительно к кодированию.

Как правило, кодовое слово длины n над $GF(q)$ отображается в последовательность элементов модуляционного алфавита длины n. (Возможны и другие варианты: например, модуляционный алфавит объема $2^{m/2}$ может использоваться с кодом над $GF(2^m)$ путем отображения символов кодового слова в пары букв алфавита.) Демодулятор, принимающий жесткое решение, вновь отображает множество вещественных или комплексных чисел, представляющее собой последовательность v_k, в множество символов модуляционного алфавита, а затем и в символы кодового слова.

15.2. ЭНЕРГИЯ НА БИТ И ЧАСТОТА ОШИБОК НА БИТ

Цифровая система связи включает кодер и модулятор на передающем конце и демодулятор и декодер на приемном конце. Качество такой системы частично характеризуется вероятностью ошибки на бит, которая также называется *частотой ошибок на бит* (ЧОБ). В фиксированной цифровой системе связи, передающей дискретные данные по каналу с аддитивным гауссовским шумом, частота ошибок на бит всегда может быть уменьшена путем увеличения мощности передатчика, которая также является одной из характеристик качества цифровой системы связи. Лучшей из двух цифровых систем связи считается та система, которая достигает желаемой частоты ошибок на бит при меньшей передаваемой мощности передатчика.

Для сообщений конечной длины, состоящих из K информационных битов и имеющих энергию сообщения E_m, энергия E_b, приходящаяся на бит, определяется отношением

$$E_b = E_m/K.$$

Вообще говоря, энергия на бит E_b — это не та энергия, которая может быть найдена по отдельной части сообщения, и не та энергия, которая может быть измерена. Она вычисляется по энергии всего сообщения. Определение E_b включает также число битов на входе блока кодер/модулятор. Если же рассматривать вход канала, то можно обнаружить сигналы, построенные так, что число символов в них, воспринимаемых как символы канала, будет больше. Эти дополнительные символы могут быть провероч-

ными символами, служащими для исправления ошибок, символами кадровой синхронизации или символами канального протокола. Эти символы не несут передаваемой информации. Для вычисления E_b используются лишь информационные биты.

Для сообщений бесконечной длины, передаваемых с постоянной скоростью R информационных бит/с, величина E_b определяется отношением

$$E_b = S/R,$$

где S — средняя мощность сообщения.

Кроме сообщения в приемник поступает и белый шум с односторонней спектральной плотностью N_0 Вт/Гц (Дж). Ясно, что условия приема сигнала не изменятся, если удвоить и сигнал, и шум: на частоту ошибок на бит влияют только отношения E_m/N_0 или E_b/N_0. Для сравнения двух различных схем передачи сигналов исследуются графики зависимостей их ЧОБ от требуемых отношений E_b/N_0.

Как это ни удивительно, можно точно сформулировать утверждение относительно значений E_b/N_0, при которых существуют хорошие сигналы. Исходным понятием для вывода границ E_b/N_0 является шенноновская пропускная способность канала. Нам необходима лишь формула пропускной способности канала с аддитивным гауссовским шумом. Она утверждает, что, каков бы ни был сигнал $s(t)$ мощности $S = E_bR$ со спектром $S(f)$, обращающимся в нуль при $|f| > W/2$, передача по каналу с аддитивным белым гауссовским шумом с произвольно малой частотой ошибок на бит возможна лишь в том случае, когда в секунду передается не более

$$C = W \log_2 [1 + S/(N_0W)]$$

битов. Обратно, всегда существует сигнал $s(t)$, который позволяет сколь угодно близко приблизиться к этой скорости передачи и к этому ограничению по полосе, как мала ни была бы требуемая частота ошибок на бит. Эта знаменитая формула, которая выводится в любом учебнике по теории информации, дает критерий для оценки цифровой системы связи.

Теорема 15.2.1. *В любой цифровой системе связи, передающей информацию на фоне белого гауссовского шума спектральной плотности N_0, необходимо, чтобы энергия на бит удовлетворяла неравенству*

$$E_b \geqslant N_0 \log_e 2 = 0{,}69N_0,$$

где E_b измеряется в джоулях, а N_0 — в ваттах на герц.

Доказательство. Ширина полосы W в теореме не ограничена. Заменим в формуле Шеннона для пропускной способности S на RE_b и устремим W к бесконечности. Тогда

$$R \leqslant (RE_b/N_0) \log_2 e,$$

откуда непосредственно следует утверждение теоремы. □

Если отношение E_b/N_0 измеряется в децибелах, то наше неравенство переписывается в виде

$$E_b/N_0 \geqslant -1{,}6 \text{ дБ},$$

так что значение $E_b/N_0 = -1{,}6$ дБ устанавливает нижний предел для допустимого отношения E_b/N_0. Экспериментально установлено, что при отношении E_b/N_0, приблизительно равном или большем 12 дБ, выбор сигнала, обеспечивающего малую частоту ошибки на бит, не представляет труда. Следовательно, проблема выбора сигнала возникает лишь в том случае, когда отношение E_b/N_0 лежит между этими двумя числами и ширина области составляет примерно 14 дБ. Для этой области и производится сравнение структур сигналов, выбираемых для канала с гауссовским шумом.

В случае когда предоставляется лишь ограниченная ширина полосы, требования на энергию становятся жестче. Определим спектральную скорость r в битах (измеряемую в битах в секунду на герц) как

$$r = R/W.$$

Спектральная скорость r в битах и E_b/N_0 представляют собой две важнейшие характеристики цифровой системы связи.

Теорема 15.2.2. *Пусть r — спектральная скорость в битах для некоторой цифровой системы связи. Тогда*

$$\frac{E_b}{N_0} \geqslant \frac{2^r - 1}{r}.$$

Более того, можно так сконструировать систему передачи, чтобы она обеспечивала надежную передачу при любом положительном ε и

$$\frac{E_b}{N_0} \leqslant \frac{2^r - 1}{r} + \varepsilon.$$

Доказательство. Из формулы Шеннона для пропускной способности вытекает, что при любом положительном ε' может быть

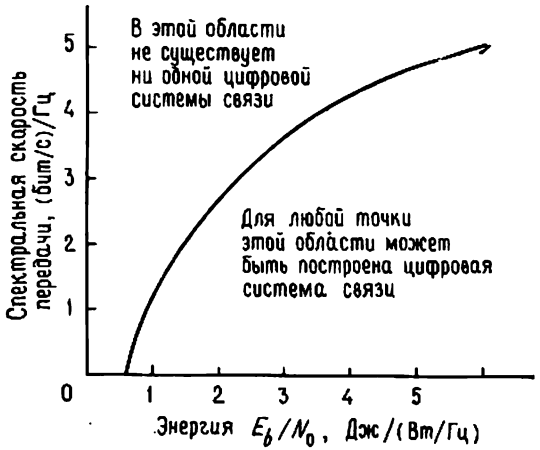


Рис. 15.2.

достигнута скорость передачи информации R, удовлетворяющая неравенству

$$R \leqslant W \log_2\left(1 + \frac{RE_b}{WN_0}\right) \leqslant R(1 - \varepsilon').$$

Отсюда непосредственно следует утверждение теоремы. □

Последняя теорема утверждает, что возрастание скорости передачи в битах на единицу полосы увеличивает требуемую энергию на один бит. Она является основой соотношений между энергией и шириной полосы в цифровой теории связи, где возрастание ширины полосы при фиксированной скорости передачи информации может ослабить энергетические требования.

Утверждение теоремы 15.2.2 иллюстрируется рис. 15.2. Любая система связи может быть описана точкой, лежащей ниже приведенной кривой, и для любой точки ниже этой кривой можно сконструировать систему связи, у которой частота ошибки на бит может быть сделана настолько малой, насколько это желательно. История цифровой связи в какой-то степени представляет собой серию попыток приблизить системы к этой предельной кривой, сохраняя очень низкую частоту ошибок на бит. Такие системы используют как модемную технику, так и методы исправления ошибок.

В некоторых цифровых системах связи водораздел между модемом и кодеком четко не обозначен. Демодулятор передает дополнительную информацию, которая помогает декодеру улучшить его характеристики. Это можно сделать многими способами. Например, в двоичном канале можно использовать приемник, у которого каждому принятому биту соответствует выходной

сигнал из трех бит. Он называется *обнаружителем с мягким решением*. Комбинация, состоящая из трех нулей, соответствует тому, что «уверенно» был принят нуль, а комбинация, состоящая из трех единиц, соответствует «уверенному» приему единицы. Другие комбинации трех двоичных символов соответствуют различным степеням неопределенности в приемнике. Для улучшения декодирования декодер может использовать эти мягкие решения. Мы уже знакомы с простейшим вариантом мягких решений — двоичным стирающим каналом. Три значения выходного сигнала приемника могут быть обозначены через 10 (жесткий нуль), 01 (жесткая единица) и 00 (стирание).

Другим примером мягкого решения является списочное декодирование в q-ичном канале. Вместо одного наиболее вероятного значения каждого символа демодулятор выдает два или три наиболее вероятных значения, иногда приписывая каждому из них некоторую вероятностную меру.

Конечно, если код фиксирован, то декодеры, использующие информацию мягкого решения, имеют, вообще говоря, лучшие характеристики; поэтому можно выбирать декодер, исходя из этих соображений. Однако такой подход не является бесспорным. Прежде всего системы с мягким решением достигают своих потенциальных возможностей только в том случае, когда статистика шума в канале достаточно стационарна и достаточно хорошо поддается моделированию. Если же шум прерывистый и переменный во времени или недостаточно хорошо известен, то следует предпочесть жесткое решение или жесткое решение со стиранием. Кроме того, декодер с мягким решением дороже. Если стоимость системы фиксирована, то длина кода, используемого с декодером с мягким решением, меньше, чем кода, используемого с декодером с жестким решением. Следовательно, декодер с мягким решением не обязательно является лучшим. Декодер с жестким решением может иметь лучшие характеристики, чем декодер с мягким решением, использующий код, у которого длина меньше. Иногда система с мягким решением лучше удовлетворяет практическим требованиям. В других случаях лучше использовать систему с жестким решением, так как она позволяет использовать более мощный код. Выбор той или иной возможности является трудной инженерной задачей, которую нельзя правильно решить, исходя лишь из теоретических соображений.

15.3. МЯГКОЕ ДЕКОДИРОВАНИЕ БЛОКОВЫХ КОДОВ

Вероятность ошибки декодирования минимизируется одновременным выбором демодуляции и декодирования. Модулятор отображает q^k кодовых слов в q^k сигналов $s_r(t)$, где $r = 1, \ldots, q^k$.

Выходной сигнал канала $v(t) = s_r(t) + n(t)$, где $s_r(t)$ — передаваемый сигнал, а $n(t)$ — реализация шума, представляется своими отсчетами $v_l = v(l\,\Delta T)$ при $l = 1, \ldots, N$. По этим отсчетам демодулятор и декодер должны восстановить кодовое слово, а затем и информационные символы.

Плотность распределения вероятностей вектора $\mathbf{v}$ при условии, что передан сигнал $s_r(t)$, дается выражением

$$Q(\mathbf{v} \mid r) = \prod_{l=1}^{N} [1/(\sqrt{2\pi}\sigma)]\, e^{-(v_l - s_{rl})^2/(2\sigma^2)},$$

причем предполагается, что отсчеты шума представляют собой независимые одинаково распределенные гауссовские случайные величины с дисперсией σ^2. По принятому сигналу $\mathbf{v}$ приемник максимального правдоподобия определяет значение r, при котором $Q(\mathbf{v} \mid r)$ максимально, и объявляет, что было передано r-е сообщение.

Однако поскольку шум предполагается гауссовским, приемник максимального правдоподобия идентичен приемнику минимального евклидова расстояния. Этот приемник находит значение r, при котором евклидово расстояние

$$d_E(\mathbf{v}, \mathbf{s}_r) = \sum_{l=1}^{N} (v_l - s_{rl})^2, \quad r = 1, \ldots, q^k,$$

минимально, и объявляет, что было передано r-е сообщение.

Далее, если энергия $s_r(t)$ одинакова для всех r, то приемник минимального евклидова расстояния может быть заменен корреляционным приемником. Он вычисляет совокупность q^k коэффициентов корреляции

$$\rho_r = \sum_{l=1}^{N} v_l^* s_{rl}, \quad r = 1, \ldots, q^k,$$

и затем принимает решение в пользу того кодового слова $\mathbf{c}_r$, у которого $\rho_r > \rho_{r'}$ при всех $r' \neq r$. После этого по $\mathbf{c}_r$ восстанавливаются информационные символы. Поскольку q^k очень велико, обычно даже этот приемник довольно непрактичен и поэтому переходят к более компактным схемам, схемам меньшей сложности.

На рис. 15.3 предлагается несколько вариантов приемника. Если модулятор и кодер фиксированы, то выбор самого верхнего варианта приводит к наименьшей частоте ошибок на символ, а самого нижнего — к наибольшей. Может создаться впечатление, что по мере возможности всегда желательно реализовать приемник максимального правдоподобия. Но это, вообще говоря, не так. Часто целесообразнее затратить имеющиеся ресурсы на более

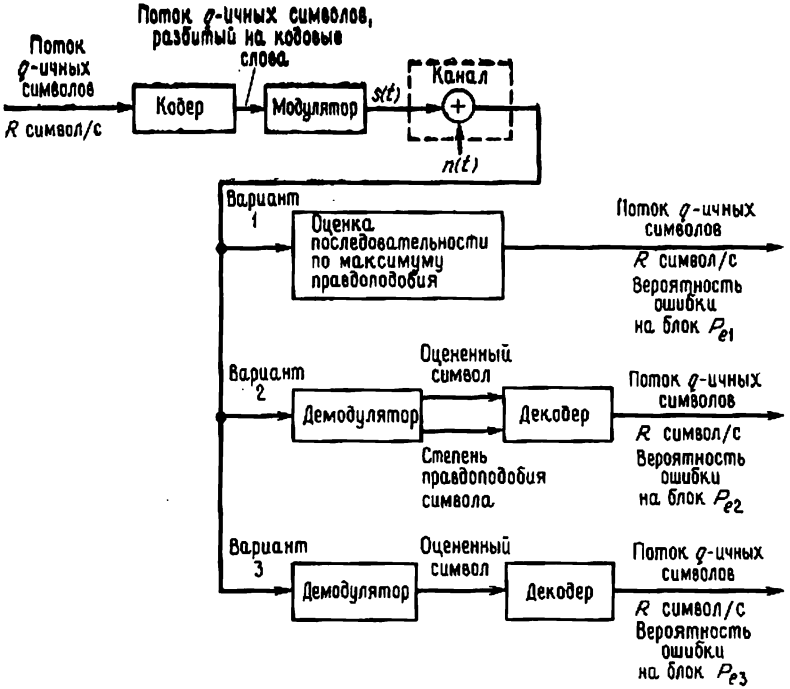


Рис. 15.3. Возможные варианты приемника.

мощный код и более мощный модулируемый сигнал, а для декодирования использовать подоптимальную процедуру.

В общем случае возможность использования в декодере мягкого решения приводит к необходимости выбора между передачей большего количества информации от демодулятора к декодеру при одновременном использовании более простого кода и передачей меньшего количества информации от демодулятора к декодеру при одновременном использовании более сильного кода. Первый экстремальный случай — демодулятор принимает жесткое решение; при этом последовательность вещественных чисел превращается демодулятором в последовательность q-ичных символов, к которой можно успешно применить мощную технику алгебраического декодирования. Не представляет трудностей использование той же техники в случае, когда демодулятор может производить стирания. Второй экстремальный случай — демодулятор вообще не принимает решения; при этом вся информация в принятом сигнале сохраняется и передается декодеру.

В хорошо сконструированной системе связи сложность демодулятора и сложность декодера сбалансированы. С другой стороны, демодулятор принимает окончательное решение по возмож-

ности на уровне символов; с другой стороны, он передает декодеру достаточно информации, чтобы качественные характеристики системы оставались достаточно высокими. Сохраняется также возможность использования каскадных кодов. Тогда соотношения между сложностью и качеством можно рассматривать на трех уровнях обработки в демодуляторе и на двух уровнях декодера. Используя мягкое решение в сочетании с малой длиной кода во внутреннем декодере и жесткое решение с исправлением ошибок и стираний во внешнем декодере, корректирующем оставшиеся ошибки и отказы внутреннего декодера, можно получить отличное соотношение между сложностью и качеством декодирования.

Изучим частный случай декодера мягкого решения, использующий алгоритм *обобщенного минимального расстояния* (ОМР-алгоритм). Этот алгоритм мы опишем дважды: сначала для двоичных кодов, используя некоторые геометрические построения, а затем для кодов над произвольным полем Галуа. В первом случае в демодулятор поступает последовательность двоичных чисел на фоне шумов. Принимаемый демодулятором i-й двоичный бит описывается вещественным числом $\tilde{v}_i \in [-1, 1]$, представляющим оценку принятого символа в сочетании с доверительным уровнем. Если $\tilde{v}_i$ равно -1, то i-й принятый бит равен 0 с максимальным доверительным уровнем; если $\tilde{v}_i = +1$, то i-й принятый бит равен единице с максимальным доверительным уровнем. Если $\tilde{v}_i$ представлено в виде числа, характеризуемого знаком и величиной, то можно считать, что его знак представляет оценку i-го бита на выходе демодулятора, а абсолютная величина $\tilde{v}_i$ — доверительный уровень оценки.

Мы уже рассмотрели два частных случая. Если величины $\tilde{v}_i$ на выходе демодулятора принимают значения ± 1, то демодулятор является демодулятором жесткого решения, а декодер исправляет лишь ошибки. Если на выходе демодулятора величины $\tilde{v}_i$ принимают лишь значения ± 1 или 0, то демодулятор является демодулятором жесткого решения со стиранием, а декодер исправляет ошибки и стирания. В данном параграфе мы рассмотрим ОМР-алгоритм, который оперирует с величинами $\tilde{v}_i$, принимающими любое значение между -1 и $+1$. Этот алгоритм декодирования может использоваться с любым демодулятором, выходные символы которого $\tilde{v}_i$ лежат в интервале $[-1, 1]$.

После обработки принятого сигнала демодулятором на его выход поступает принятый вектор

$$\tilde{\mathbf{v}} = (\tilde{v}_0, \ldots, \tilde{v}_{n-1}).$$

Если в канале не произошло ошибок и демодулятор приписал максимальный доверительный уровень каждому биту, то символы принятого вектора равны -1 в тех позициях, в которых сим-

волы переданного кодового слова $\mathbf{c}_r$ равны нулю, и равны $+1$ в тех позициях, в которых символы $\mathbf{c}_r$ равны единице.

Пусть $\tilde{\mathbf{c}}_r$ — вектор в n-мерном евклидовом пространстве, такой, что его компоненты определяются выражением

$$\tilde{c}_{ri} = \begin{cases} -1, & \text{если } c_{ri} = 0, \\ +1, & \text{если } c_{ri} = 1. \end{cases}$$

Это выражение описывает отображение кодовых слов из $GF^n(q)$ в $\mathbb{R}^n$. Сохраним за $\tilde{\mathbf{c}}_r$ название кодового слова. Евклидово расстояние между принятым словом $\tilde{\mathbf{v}}$ и кодовым словом $\tilde{\mathbf{c}}_r$ равно

$$d_E^2(\tilde{\mathbf{v}}, \tilde{\mathbf{c}}_r) = |\tilde{\mathbf{v}} - \tilde{\mathbf{c}}_r|^2 = \sum_{i=0}^{n-1} (\tilde{v}_i - \tilde{c}_{ri})^2.$$

Если $\tilde{\mathbf{v}}$ состоит лишь из символов жесткого решения $\tilde{v}_i = \pm 1$, то евклидово расстояние связано с хэммингловым расстоянием соотношением

$$d_E^2(\tilde{\mathbf{v}}, \tilde{\mathbf{c}}_r) = 4d_H(\mathbf{v}, \mathbf{c}_r),$$

где компоненты v_i вектора $\mathbf{v}$ равны 0, если $\tilde{v}_i = -1$, и равны 1, если $\tilde{v}_i = 1$. В частности, для любых двух кодовых слов

$$d_E^2(\tilde{\mathbf{c}}_r, \tilde{\mathbf{c}}_{r'}) = 4d_H(\mathbf{c}_r, \mathbf{c}_{r'});$$

Минимальное евклидово расстояние кода d_E^* и минимальное хэммингово расстояние d^* связаны соотношением

$$d_E^{*2} = 4d_H^*.$$

Опишем ОМР-декодер с помощью скалярного произведения $\tilde{\mathbf{v}} \cdot \tilde{\mathbf{c}}_r$, предварительно выразив евклидово расстояние кода через скалярное произведение. Мы можем переписать его в виде

$$d_E^2(\tilde{\mathbf{v}}, \tilde{\mathbf{c}}_r) = |\tilde{\mathbf{v}}|^2 - 2\tilde{\mathbf{v}} \cdot \tilde{\mathbf{c}}_r + |\tilde{\mathbf{c}}_r|^2.$$

Для фиксированного $\tilde{\mathbf{v}}$ минимизация $d_E(\tilde{\mathbf{v}}, \tilde{\mathbf{c}}_r)$ по r эквивалентна максимизации скалярного произведения $\tilde{\mathbf{v}} \cdot \tilde{\mathbf{c}}_r$ по r, поскольку $|\tilde{\mathbf{v}}|^2$ не зависит от r и $|\tilde{\mathbf{c}}_r|^2$ равно n для любого кодового слова. Минимизация евклидова расстояния интуитивно понятнее, но при вычислениях лучше максимизировать скалярное произведение.

Декодеру, исправляющему только ошибки, предшествует демодулятор, который принимает лишь жесткие решения. При этом $\tilde{v} = \pm 1$ и $d_E^*(\tilde{\mathbf{v}}, \mathbf{c}_r) = 4d_H(\mathbf{v}, \mathbf{c}_r)$. Следовательно, нахождение $\tilde{\mathbf{c}}_r$ на минимальном евклидовом расстоянии от $\tilde{\mathbf{v}}$ эквивалентно нахождению $\mathbf{c}_r$ на минимальном хэммингловом расстоянии от $\mathbf{v}$. Неравенству

$$d_H(\mathbf{v}, \mathbf{c}_r) \leqslant (d^* - 1)/2$$

удовлетворяет не более одного кодового слова $\mathbf{c}_r$. Следовательно, не более чем одно кодовое слово $\tilde{\mathbf{c}}_r$ удовлетворяет неравенству

$$d_E^2(\tilde{\mathbf{v}}, \tilde{\mathbf{c}}_r) \leqslant 2(d^* - 1).$$

Наконец, так как $|\tilde{\mathbf{v}}|^2$ равно n в случае демодулятора с жестким решением, то

$$d_E^2(\tilde{\mathbf{v}}, \tilde{\mathbf{c}}_r) = 2(n - \tilde{\mathbf{v}} \cdot \mathbf{c}_r)$$

и не более чем одно кодовое слово удовлетворяет неравенству

$$\tilde{\mathbf{v}} \cdot \tilde{\mathbf{c}}_r \geqslant n - (d^* - 1),$$

где d^* — минимальное хэммингово расстояние кода. Декодирование с исправлением только ошибок эквивалентно нахождению того единственного кодового слова $\tilde{\mathbf{c}}_r$, для которого

$$\tilde{\mathbf{v}} \cdot \tilde{\mathbf{c}}_r > n - d^*,$$

если такое кодовое слово существует. Возможно, что решения не существует и декодер не в состоянии найти кодовое слово. Такой декодер неполон, но он будет указывать, что соответствующие конфигурации ошибок являются неисправляемыми.

Если действия демодулятора сводятся к жесткому решению или стиранию, то $\tilde{v}_i = \pm 1$ или 0. По тем же причинам, что и выше, декодирование с исправлением ошибок и стираний эквивалентно нахождению того единственного кодового слова $\mathbf{c}_r$, для которого

$$\tilde{\mathbf{v}} \cdot \tilde{\mathbf{c}}_r > n - d^*,$$

если такое кодовое слово существует. Как и ранее, соответствующий декодер полон.

Следующая теорема утверждает, что те же условия существования справедливы и при мягком решении.

Теорема 15.3.1. *Каков бы ни был вектор $\tilde{\mathbf{v}}$ с компонентами, лежащими в интервале $[-1, 1]$, в двоичном коде с длиной n и минимальным хэмминговым расстоянием d^* существует не более одного кодового слова $\tilde{\mathbf{c}}_r$, для которого*

$$\tilde{\mathbf{v}} \cdot \tilde{\mathbf{c}}_r > n - d^*.$$

Доказательство. Пусть $\tilde{\mathbf{c}}_r$ — некоторое кодовое слово, удовлетворяющее приведенному выше неравенству, и пусть $\tilde{\mathbf{c}}_{r'}$ — любое другое кодовое слово. Тогда $\tilde{\mathbf{c}}_{r'}$ отличается от $\tilde{\mathbf{c}}_r$ по меньшей мере в d^* позициях. Пусть

$$S = \{i \mid \tilde{c}_{ri} \neq \tilde{c}_{r'i}\};$$

тогда

$$\tilde{\mathbf{v}}\cdot\tilde{\mathbf{c}}_r = \sum_{i\notin S}\tilde{v}_i\tilde{c}_{ri} + \sum_{i\in S}\tilde{v}_i\tilde{c}_{ri} = A_1 + A_2,$$

$$\tilde{\mathbf{v}}\cdot\tilde{\mathbf{c}}_{r'} = \sum_{i\notin S}\tilde{v}_i\tilde{c}_{r'i} + \sum_{i\in S}\tilde{v}_i\tilde{c}_{r'i} = A_1 - A_2.$$

Но величина A_1 удовлетворяет неравенству

$$A_1 = \sum_{i\notin S}\tilde{v}_i\tilde{c}_{ri} \leqslant n - d^*,$$

так как она равна сумме не более $n - d^*$ слагаемых, ни одно из которых не превышает единицу. Поэтому в силу неравенства $\tilde{\mathbf{v}}\cdot\tilde{\mathbf{c}}_r > n - d^*$ $A_2 > 0$, и, следовательно, $A_1 - A_2 \leqslant n - d^*$, что завершает доказательство теоремы. □

В качестве примера рассмотрим двоичный (3,2)-код $\mathscr{C} = \{000, 011, 110, 101\}$, у которого d^* равно 2. Этот пример достаточно прост для графической иллюстрации теоремы 15.3.1. На рис. 15.4 изображен куб в трехмерном евклидовом пространстве, содержащий возможные принятые векторы $\tilde{\mathbf{v}}$. В том же пространстве показаны четыре кодовых слова. Штриховкой в одном из углов выделена совокупность $\tilde{\mathbf{v}}$, удовлетворяющая условию $\tilde{v}_0\tilde{c}_{r0} + \tilde{v}_1\tilde{c}_{r1} + \tilde{v}_2\tilde{c}_{r2} > 1$, которая декодируется в кодовое слово, лежащее в этом углу. Теорема утверждает, что если построить аналогичные области для каждого из четырех кодовых слов, то эти области не будут пересекаться. Используя эти четыре области декодирования, можно построить ОМР-декодер. Заметим, что указанные четыре области не заполняют куб полностью. Остающаяся в центре область является тетраэдром, и если $\tilde{\mathbf{v}}$

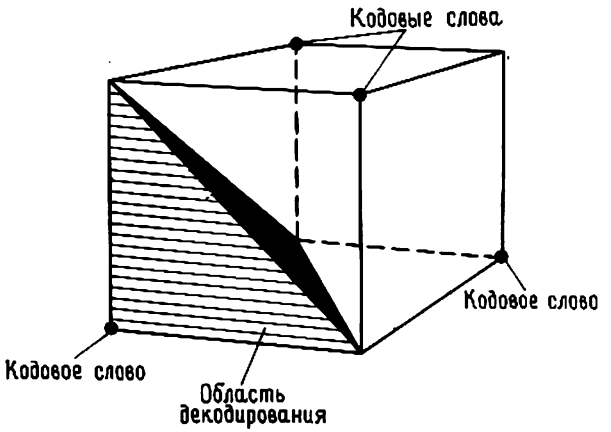


Рис. 15.4. Область декодирования по ОМР-алгоритму для двоичного (3, 2)-кода.

попадает в этот тетраэдр, то декодирование объявляется неудачным. Декодер является неполным, и его области декодирования лежат в евклидовом пространстве.

ОМР-декодер находит единственное кодовое слово, удовлетворяющее теореме 15.3.1, если оно существует; в противном случае декодирование объявляется неудачным. Конечно, находить это кодовое слово, вычисляя все 2^k скалярных произведений, непрактично. Опишем практический способ нахождения этого кодового слова. Идея состоит в следующем.

Мы уже знаем, как построить декодеры, исправляющие ошибки и стирания, полученные в результате жестких решений. Любой декодер, исправляющий ошибки и стирания, можно дополнить внешней петлей обработки данных. Основываясь на информации мягкого решения, этот модернизированный декодер вычисляет последовательность пробных векторов жесткого решения со стиранием. Для каждого l от 0 до $d^* - 1$ он стирает l компонент, для которых $|\tilde{v}_i|$ минимальны. (Если для некоторых l в силу равенства нескольких компонент выбор этих l компонент неоднозначен, то соответствующее l опускается.) Совокупность нестертых компонент отображается в нули или единицы в зависимости от знака $\tilde{v}_i$. При этом над *GF* (2) формируется вектор $\mathbf{v}^{(l)}$ со стираниями.

Декодируем $\mathbf{v}^{(l)}$, используя декодер, исправляющий ошибки и стирания. Если декодирование дало кодовое слово $\mathbf{c}_r$, то проверим выполнение неравенства $\tilde{\mathbf{v}} \cdot \tilde{\mathbf{c}}_r > n - d^*$. Если это неравенство выполняется, то $\mathbf{c}_r$ объявляется ОМР-декодированным кодовым словом. В противном случае l увеличивается и процедура повторяется до тех пор, пока l не превзойдет $d^* - 1$; в последнем случае декодирование объявляется неудачным.

На рис. 15.5 представлен ОМР-алгоритм в недвоичном случае. Теоремы 15.3.2 и 15.3.4 утверждают, что ОМР-алгоритм, представленный на рис. 15.5, в самом деле декодирует в единственное ОМР-кодовое слово. Заметим, что алгоритм, изображенный на рисунке, проверяет неравенство лишь при четных l. Этого достаточно, так как при изменении числа стираний на два число ошибок, которые можно исправить, изменяется на единицу.

Теорема 15.3.2. *Если* $\tilde{\mathbf{v}} \cdot \tilde{\mathbf{c}}_r > n - d^*$, *то хотя бы одно слово* $\mathbf{v}^{(l)}$, $l = 0, \ldots, d^* - 1$, *удовлетворяет неравенству*

$$\mathbf{v}^{(l)} \cdot \mathbf{c}_r > n - d^*.$$

Доказательство следует из доказательства теоремы 15.3.4, являющейся более общей формой нашей теоремы. □

Теперь обобщим ОМР-декодеры на недвоичные коды. Сложность ОМР-декодера приблизительно в $(d^* - 1)/2$ раз выше, чем декодера для кода, исправляющего ошибки и стирания. Поня-

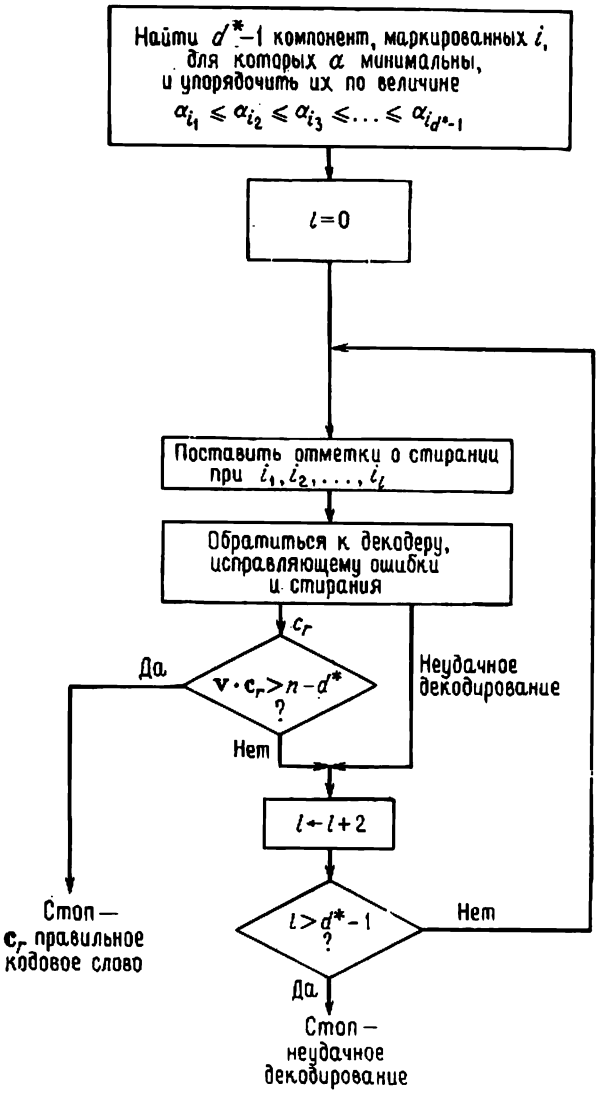


Рис. 15.5. ОМР-алгоритм.

тие обобщенного расстояния естественным образом распространяется на недвоичные сигналы точно так же, как распространяется хэммингово расстояние. Как при определении хэммингова расстояния, так и при определении обобщенного расстояния недвоичная природа символов игнорируется и задача сводится

к двоичной, при которой лишь учитывается, совпадают или нет два символа, а их величины не принимаются во внимание. ОМР-декодер в недвоичном случае почти не меняется.

На выход демодулятора поступают q-ичные символы, каждый из которых характеризуется своей оценкой v_i, элементом $GF(q)$ и доверительным уровнем α_i, $0 \leqslant \alpha_i \leqslant 1$, измеряющим надежность оценки. Если α_i равно единице, то принятый символ равен v_i с максимальным доверительным уровнем; если α_i равно 0, принятый символ равен v_i с минимальным доверительным уровнем.

Как и в двоичном случае, для алгоритма декодирования не важно, какое правило демодулятор использует для нахождения α_i. Он даже может всегда полагать α_i равным единице — в этом случае он является демодулятором с жестким решением; он может полагать α_i равным 0 или 1 — в этом случае он является демодулятором с жестким решением и со стиранием. Возможно также использование для нахождения α_i более сложной доверительной меры.

В двоичном случае мы смогли дать геометрическую интерпретацию декодирования, используя знак α_i для обозначения оценки переданного двоичного символа. В $GF(q)$-ичном случае такой геометрической интерпретации не существует. Для определения скалярного произведения введем обозначение для любого v':

$$\delta(v, v') = \begin{cases} +1, & v = v', \\ -1, & v \neq v'. \end{cases}$$

Затем определим произведение

$$\mathbf{v} \cdot \mathbf{c}_r = \sum_{i=0}^{n-1} \alpha_i \delta(v_i, c_{ri}),$$

использование которого позволяет доказать $GF(q)$-ичный вариант теоремы 15.3.1. Это определение $\mathbf{v} \cdot \mathbf{c}_r$ удобно, но неточно, поскольку $\mathbf{v} \cdot \mathbf{c}_r$ зависит и от α.

Теорема 15.3.3. *В коде над $GF(q)$ с длиной n и минимальным расстоянием d^* существует хотя бы одно кодовое слово $\mathbf{c}_r$, для которого*

$$\mathbf{v} \cdot \mathbf{c}_r > n - d^*.$$

Доказательство аналогично доказательству теоремы 15.3.1. □

Алгоритм декодирования в недвоичном случае таков же, как и в двоичном. Для любого l из интервала $[0, d^* - 1]$ сотрем компоненты, для которых α_i минимальны. При этом получим над $GF(q)$ слово $\mathbf{v}^{(l)}$ со стираниями. Начиная с l, равного 0, декодируем $\mathbf{v}^{(l)}$, используя декодер, исправляющий ошибки и стирания. Для декодирования $\mathbf{v}^{(l)}$ в $\mathbf{c}_r$ проверяется неравенство $\mathbf{v} \cdot \mathbf{c}_r > n - d^*$. Если оно удовлетворяется, то $\mathbf{c}_r$ объявляется ОМР-

декодированным кодовым словом. Если оно не удовлетворяется, т. е. если l-е декодирование оказывается неудачным, то увеличим l; будем повторять процедуру до тех пор, пока l не превысит $d^* - 1$ — в этом последнем случае декодирование объявляется неудачным. Следующая теорема обосновывает эту процедуру.

Теорема 15.3.4. *Если для кодового слова* $\mathbf{c}_r$ *выполняется неравенство* $\mathbf{v} \cdot \mathbf{c}_r > n - d^*$, *то хотя бы одно слово* $\mathbf{v}^{(l)}$, $l = 0, \ldots, d^* - 1$, *удовлетворяет неравенству*

$$\mathbf{v}^{(l)} \cdot \mathbf{c}_r > n - d^*.$$

Доказательство. Достаточно доказать теорему для l, пробегающих значения от 0 до n, поскольку ясно, что утверждение теоремы не может выполняться при $l \geqslant d^*$.

Упорядочим α_i по их величине. Пусть

$$\alpha_{i_1} \leqslant \alpha_{i_2} \leqslant \ldots \leqslant \alpha_{i_n},$$

$$\lambda_0 = \alpha_{i_1},$$

$$\lambda_l = \alpha_{i_{l+1}} - \alpha_{i_l}, \quad 1 \leqslant l \leqslant n-1,$$

и

$$\lambda_n = 1 - \alpha_{i_n}.$$

Тогда $0 \leqslant \lambda_l \leqslant 1$ и $\sum_{l=0}^{n} \lambda_l = 1$, так что вектор $\boldsymbol{\lambda}$ ведет себя как вероятностное распределение. Более того,

$$\sum_{l=0}^{l'-1} \lambda_l = \alpha_{i l'},$$

и поэтому

$$\mathbf{v} = \sum_{l=0}^{n} \lambda_l \mathbf{v}^{(l)}.$$

Допустим, что $\mathbf{v}^{(l)} \cdot \mathbf{c}_r \leqslant n - d^*$ при всех l. Тогда

$$\mathbf{v} \cdot \mathbf{c}_r = \sum_{l=0}^{n} \lambda_l \mathbf{v}^{(l)} \cdot \mathbf{c}_r \leqslant (n - d^*) \sum_{l=0}^{n} \lambda_l = n - d^*,$$

что противоречит предположению теоремы. Поэтому теорема доказана. □

15.4. МЯГКОЕ ДЕКОДИРОВАНИЕ СВЕРТОЧНЫХ КОДОВ

Использование информации мягкого решения может облегчить декодирование и сверточных кодов. В нашем распоряжении имеются вариант мягкого решения алгоритма Витерби и варианты

мягкого решения алгоритмов последовательного декодирования. Отложим изучение последовательного декодирования до следующего параграфа, а в настоящем параграфе займемся алгоритмом Витерби с демодулятором с мягким решением. Затем изучим сверточные коды Унгербёка, имеющие большое евклидово свободное расстояние; поэтому они полезны для каналов, ограниченных по полосе. Коды Унгербёка больше подходят для каналов, ограниченных по полосе, чем коды, построенные из соображений возможно большего хэммингова свободного расстояния.

Алгоритм Витерби отыскивает на решетке путь, ближайший к принятому слову. Расстояние принятого слова от пути на решетке определяется как сумма расстояний вдоль последовательных ребер. Мера расстояний выбирается произвольно, за исключением того, что она должна удовлетворять требованию аддитивности. Поэтому, хотя алгоритм Витерби, введенный в § 12.8, использует хэммингово расстояние, тот же алгоритм применим и при любом другом определении расстояния. Не является даже необходимым, чтобы расстояние было метрикой.

Алгоритм Витерби хорошо использовать в тех случаях, когда шум в канале стационарен и легко моделируется. Пусть $Q(v \mid c)$ — условная вероятность того, что на выходе демодулятора наблюдается символ v при условии, что на вход канала поступил символ c. Даже если c выбран из дискретного алфавита малого объема, демодулятор мягкого решения может иметь для v больший алфавит — быть может, непрерывный алфавит или алфавит двоичных чисел из m битов. Возьмем некоторый кадр принятого слова. Он состоит из n_0 компонент, обозначаемых через v_i, $i = 1, \ldots, n_0$. Выберем некоторое ребро решетки. Оно состоит из n_0 компонент кодового слова, обозначаемых через c_i, $i = 1, \ldots, n_0$. Расстояние кадра кодового слова $\mathbf{c}$ от соответствующего кадра принятого слова $\mathbf{v}$ определяется логарифмической функцией правдоподобия

$$d(\mathbf{v}, \mathbf{c}) = \sum_{i=1}^{n_0} \log Q(v_i \mid c_i).$$

Эта функция, называемая *метрикой ребра* (или метрикой пути), обладает свойствами расстояния, хотя и не является истинной метрикой.

Декодер Витерби с мягким решением аналогичен декодеру Витерби с жестким решением, но только вместо хэммингова расстояния он использует новую метрику пути. При каждой итерации каждый выживший путь, ведущий в каждый узел, продолжается всеми возможными способами в узлы следующего уровня с прибавлением метрик ребер к общей накопленной метрике пути. Выживший в каждом новом узле путь сохраняется, а остальные пути отбрасываются. Поэтому сложность декодера

Витерби возрастает очень мало при его перестройке на мягкое решение.

Если входные символы канала — вещественные или комплексные числа, а шум в канале — аддитивный гауссовский независимый, то

$$Q(\mathbf{v} \mid \mathbf{c}) = \prod_{i=1}^{n_0} \left\{ \frac{1}{\sqrt{2\pi}\sigma} \exp\left[-\frac{(c_i - v_i)^2}{2\sigma^2} \right] \right\}.$$

Для гауссовского канала максимизация логарифмической функции правдоподобия эквивалентна минимизации евклидова расстояния. В качестве метрик ребер используем евклидово расстояние

$$d_{\mathrm{E}}(\mathbf{v}, \mathbf{c})^2 = \sum_{i=1}^{n_0} (c_i - v_i)^2.$$

На практике во многих каналах с мягким решением, не являющихся гауссовскими, $Q(\mathbf{v} \mid \mathbf{c})$ неизвестно, но евклидово расстояние по-прежнему используется для вычисления метрик ребер. Соответствующий декодер является декодером по минимуму евклидова расстояния. Он не является декодером максимального правдоподобия, но практичен. Алгоритм Витерби с мягким решением выглядит совершенно аналогично алгоритму Витерби с жестким решением, но хэммингово расстояние заменено в нем евклидовым.

Рассмотрим сверточные коды, использующие вещественные или комплексные алфавиты из 2^{n_0} символов, где n_0 — некоторое целое число. Типичные сигнальные алфавиты представлены на рис. 15.1. Каждый алфавит — это дискретное множество символов, выбираемых или из поля вещественных чисел, или из поля комплексных чисел. Сигнал представляет собой последовательность этих вещественных или комплексных чисел.

Каждый сигнал из набора сигналов будет использоваться для представления одного из 2^{n_0} ребер двоичного сверточного (mn_0, mk_0)-кода. Наборы сигналов на рис. 15.1 обеспечивают кодирование байтов, в которые входят от 2 до 6 битов. Информационный байт меньше кодового байта. На протяжении кадра информационные байты поступают в кодер. Один из байтов кодового слова (какой именно — это зависит от текущего состояния кодера) передается по каналу в виде комплексного числа, а кодер переходит в новое состояние.

Пусть $\mathscr{C}$ — сверточный (mn_0, mk_0)-код над комплексным полем $\mathbb{C}$. Евклидово расстояние между двумя кодовыми словами $\mathbf{c}_r$ и $\mathbf{c}_{r'}$ (с комплексными компонентами c_{ri} и $c_{r'i}$, $i = 0, \ldots, \ldots$, соответственно) определяется формулой

$$d_{\mathrm{E}}(\mathbf{c}_r, \mathbf{c}_{r'})^2 = \sum_{i=0}^{\infty} |c_{ri} - c_{r'i}|^2.$$

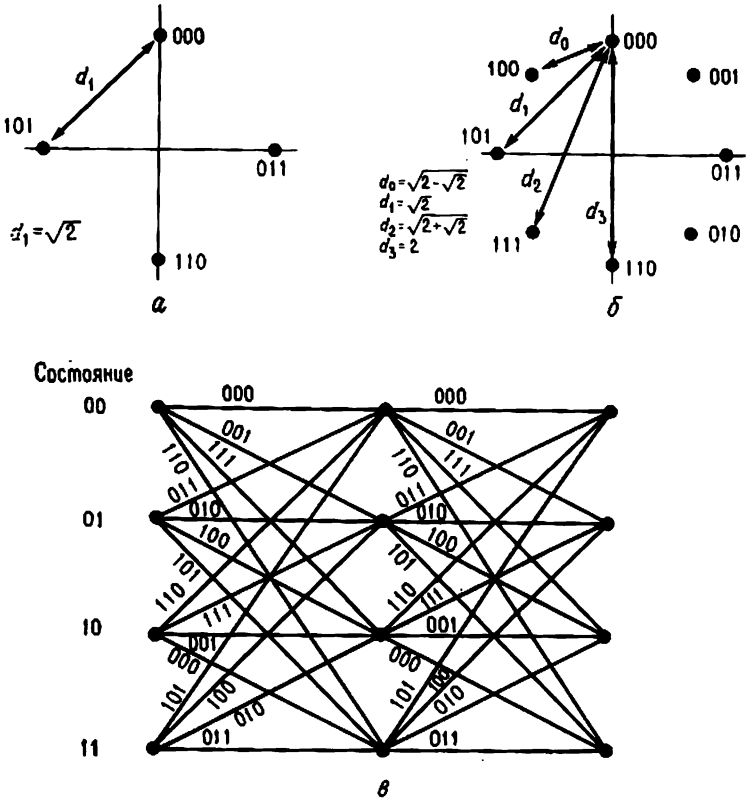


Рис. 15.6. Простой пример кодирования. *а* — четверичный набор ФТ-модулированных сигналов при отсутствии кодирования; *б* — восьмеричный набор ФТ-модулированных сигналов; *в* — решетка для сверточного (6, 4)-кода.

Евклидово свободное расстояние $\mathscr{C}$, обозначаемое через d_∞, определяется как минимум евклидовых расстояний всех возможных пар кодовых слов из $\mathscr{C}$. Евклидово свободное расстояние является подходящим расстоянием для измерения кодовых характеристик в гауссовском канале. Это следует из того, что в гауссовском канале вероятность ошибки асимптотически [1]) удовлетворяет неравенству

$$P_e \geqslant N_d \cdot Q\left(\frac{d_\infty}{2\sigma}\right),$$

где σ — среднеквадратичное отклонение шума, Q — интеграл гауссовского распределения, а N_d — число кодовых слов на

[1] При $\sigma \to 0$. — *Прим. ред.*

расстоянии d_∞ от слова, целиком состоящего из нулей. Следовательно, увеличение d_∞, грубо говоря, эквивалентно уменьшению σ. Вот почему увеличение свободного расстояния мы называем *выигрышем* сверточного кода.

Евклидово свободное расстояние часто измеряется в децибелах; эта мера определяется соотношением

$$d_\infty \text{ (дБ)} = 20 \log_{10} d_\infty.$$

Асимптотический выигрыш кодирования определяется формулой

$$G = 20 \log_{10} (d_\infty / d_{\text{ref}}),$$

где d_{ref} — минимальное евклидово расстояние между незакодированными соответствующими сигналами той же мощности.

На рис. 15.6 представлен простой пример. Сверточный код определяется восьмеричным набором ФТ-модулированных сигналов и решеткой в кодере. Он представляет собой (6,4)-код со скоростью 2/3. Пример несистематического кодера представлен на рис. 15.7, *а*. Соответствующий код может также использоваться как систематический, если рассматривать второй и третий биты в каждом кадре как информационные. Однако для реализации кода в систематической форме надо ввести в кодере обратную

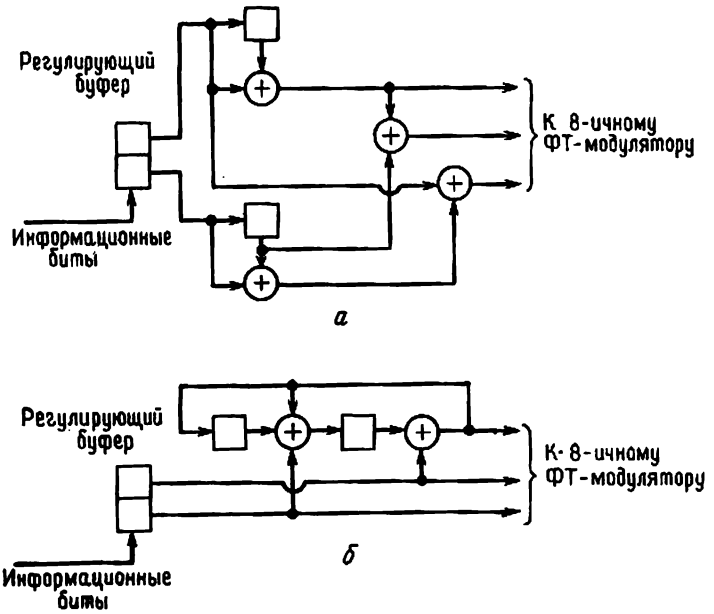


Рис. 15.7. Кодеры для сверточного (6, 4)-кода. *а* — кодер с регистром сдвига, не использующий обратную связь; *б* — кодер с регистром сдвига, использующий обратную связь.

связь, как показано на рис. 15.7, *б*. Символ кодового слова содержит лишь два бита информации, а может принимать восемь различных значений. На рис. 15.6, *в* легко видеть, что минимальный вес имеет путь 001 011 000; ему соответствует кодовое слово, находящееся на минимальном расстоянии от кодового слова,

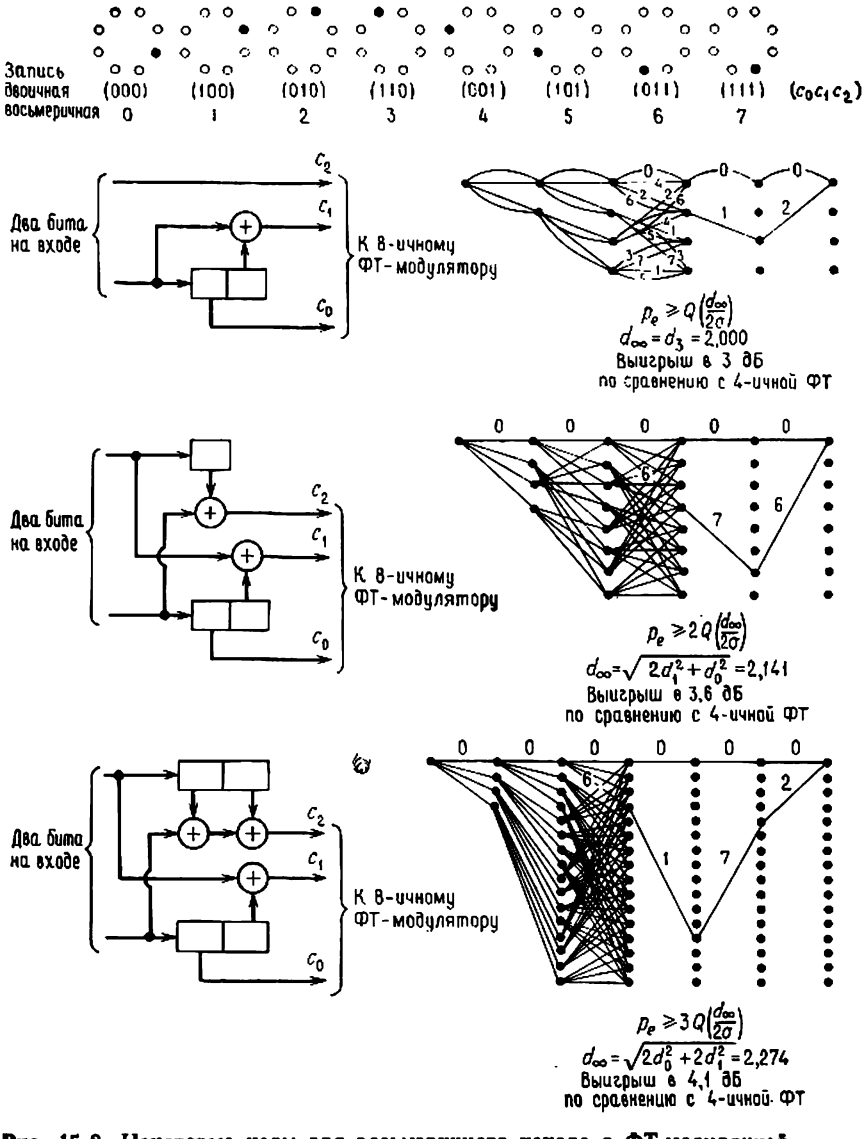


Рис. 15.8. Некоторые коды для восьмеричного канала с ФТ-модуляцией.

Коды для 8-ичного набора ФТ-сигналов

Длина кодового ограничения	$h_1(x)$	$h_2(x)$	$h_3(x)$	Выигрыш по сравнению с 4-ичной ФТ без кодирования (дБ)
2	x^2+1	x	0	~3.0
3	x^3+1	x	x^2	3.6
4	x^4+x+1	x^2	x^3+x^2+x	4.1
5	x^5+x^2+1	x^3+x^2+x	$x^4+x^3+x^2$	4.6
6	x^6+x^2+1	$x^4+x^3+x^2+x$	$x^5+x^4+x^3+x^2$	4.8
7	x^7+x+1	x^3+x^2	x^3+x^2+x	5.0
8	x^8+x^2+1	$x^7+x^5+x^3$	$x^6+x^5+x^4+x^3+x^2+x$	5.4
9	x^9+x^2+x+1	$x^6+x^5+x^4+x^2$	$x^7+x^5+x^4$	5.7

Коды для 8-ичного набора АФМ-сигналов

Длина кодового ограничения	$h_1(x)$	$h_2(x)$	$h_3(x)$	Выигрыш по сравнению с 4-ичной ФТ без кодирования (дБ)
2	x^2+1	x	0	~2.0
3	x^3+1	x	x^2	3.0
4	x^4+x+1	x^2	x^3+x^2+x	3.8
5	x^5+1	x^2+x	x^3	3.8
6	x^6+1	x^3+x^2+x	$x^5+x^4+x^2$	4.5
7	x^7+x+1	x^3+x^2	x^5+x	5.1
8	x^8+1	$x^5+x^3+x^2+x$	$x^7+x^6+x^5+x^3+x^2$	5.1
9	x^9+1	$x^7+x^6+x^5+x^2+x$	$x^8+x^7+x^3$	5.6

Рис. 15.9. Таблица кодов Унгербёка

целиком состоящего из нулей. Минимальное расстояние кода равно $d_0^2 + d_1^2$. На рис. 15.6, *а* представлен 4-ичный ФТ-модулированный набор для соответствующих незакодированных сигналов той же мощности. Выигрыш кодирования равен $G = 10 \log_{10} [(d_0^2 + d_1^2)/d_1^2]$, или 1,1 дБ, так как d_1 определяет евклидово свободное расстояние для незакодированных сигналов, изображенных на рис. 15.6, *а*. Как показывает выигрыш, кодирование приводит к небольшому улучшению, что является неожиданным для такого простого кода. Однако первый из изображенных на рис. 15.8 кодов, (9,6)-код, приводящий к выигрышу уже в 3 дБ, лишь немного сложнее, и поэтому следует отдать предпочтение ему.

Для нахождения более сложных кодов в общем случае требуется использовать ЭВМ, но даже при этом поиск должен быть разумно организован, иначе он станет практически невозможным. На рис. 15.8 приведены примеры трех кодов. Изображены решетки этих кодов, и в каждом случае выделен путь, на котором достигается евклидово свободное расстояние.

На рис. 15.9 представлена таблица кодов Унгербёка со скоростью 2/3 для восьмеричных наборов ФТ-сигналов и для восьмеричных наборов АФМ-сигналов. Сверточные коды описываются своими матрицами проверочных многочленов. Здесь удобнее использовать проверочные многочлены вместо порождающих, поскольку скорость кода больше 1/2 и систематический кодер с обратной связью легче строить, исходя из проверочных многочленов.

Любой из представленных на рис. 15.9 кодов Унгербёка может быть использован для замены обычно используемого четырехуровневого ФТ-модулятора. Информационная скорость по-прежнему остается равной 2 бита на символ. Посимвольная скорость не изменяется, и поэтому кодовая система использует ту же полосу, что и система без кодирования, и передает то же число битов на символ. Поэтому пользователь системы даже не знает о наличии кодирования. Однако система может использоваться при меньшем отношении сигнал/шум — код при длине кодового ограничения 9 дает выигрыш 5,7 дБ.

15.5. ПОСЛЕДОВАТЕЛЬНОЕ ДЕКОДИРОВАНИЕ

Используются два типа алгоритмов последовательного декодирования — алгоритм Фано, который уже изучался применительно к каналам с жестким решением, и стек-алгоритм. Они достаточно отличаются по структуре и аппаратурной реализации. Оба могут использоваться и в каналах с жестким решением, и в каналах с мягким решением. Относительные преимущества каждого из этих алгоритмов до сих пор обсуждаются. Начнем с обсуждения стек-алгоритма.

Класс алгоритмов последовательного декодирования, носящий общее название стек-алгоритма, был предложен для уменьшения вычислительной работы по сравнению с алгоритмом Фано. Стек-алгоритм сохраняет информацию о всех путях, которые уже были исследованы. В этом состоит его отличие от алгоритма Фано, который может продвинуться из данного узла, а затем вернуться на то же расстояние назад лишь для того, чтобы повторить обследование из того же узла. Стек-алгоритм хранит информацию лучше, так что не нужно повторять ненужную работу. С другой стороны, стек-алгоритм требует значительно большей памяти.

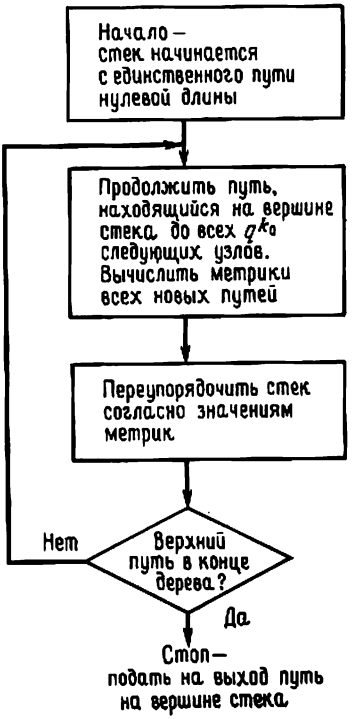


Рис. 15.10. Упрощенный стек-алгоритм

Стек-алгоритм легок для понимания; его упрощенная блок-схема приведена на рис. 15.10. Декодер содержит стек [1]) ранее исследованных путей различных длин. Стек может быть организован в виде списка в памяти случайного доступа.

Каждая запись в стеке соответствует некоторому пути и содержит три части: длину пути, последовательность информационных символов переменной длины, определяющую путь, и метрику пути. До начала декодирования стек содержит лишь тривиальный путь нулевой длины.

Метрика пути измеряет расхождение между соответствующим путем и начальным сегментом принятого слова той же длины. Она определяется как логарифмическая функция правдоподобия этого пути. Коротко объясним, как вычисляется эта метрика. Первые $N + 1$ кадров принятого слова можно записать в виде

$$\mathbf{v}^{(N)} = (v_0^1, \ldots, v_0^{n_0}, v_1^1, \ldots, v_1^{n_0}, \ldots, v_N^{(1)}, \ldots, v_N^{n_0}).$$

[1]) В данном случае стеком называется упорядоченная память. — *Прим. ред.*

Первые $m+1$ кадров кодового слова с имеют вид

$$\mathbf{c}^{(m)} = (c_0^1, \ldots, c_0^{n_0}, c_1^1, \ldots, c_1^{n_0}, \ldots . c_m^1, \ldots, c_m^{n_0}).$$

Мы хотим найти сегмент кодового слова, для которого $\log [\Pr(\mathbf{c}^{(m)} \mid \mathbf{v}^{(N)})]$ максимален. По определению последовательного декодера выбор начального сегмента кодового слова длины $m+1$ должен производиться без учета последующей структуры кодового дерева. Но по правилу Бейеса

$$\Pr(\mathbf{c}^{(m)} \mid \mathbf{v}^{(N)}) = \frac{\Pr(\mathbf{v}^{(N)} \mid \mathbf{c}^{(m)}) \Pr(\mathbf{c}^{(m)})}{\Pr(\mathbf{v}^{(N)})}.$$

По предположению кодовые слова используются с одинаковой вероятностью; поэтому

$$\Pr(\mathbf{c}^{(m)}) = \frac{1}{q^{(m+1)\,k_0}} = \frac{1}{q^{(m+1)\,n_0 R}}.$$

Множитель $\Pr(\mathbf{v}^{(N)} \mid \mathbf{c}^{(m)})$ можно переписать в виде

$$\Pr(\mathbf{v}^{(N)} \mid \mathbf{c}^{(m)}) = \prod_{i=0}^{m} \prod_{j=1}^{n_0} \Pr(v_i^j \mid c_i^j) \prod_{i=m+1}^{N} \prod_{j=1}^{n_0} \Pr(v_i^j).$$

Правая часть распадается на две части: произведение условных вероятностей по $m+1$ кадрам кодового слова и произведение безусловных вероятностей по тем кадрам, в которых кодовое слово еще не определено. Разбиение вероятности на две части, которое мы провели, приводит естественным образом к вероятностному описанию последовательного декодирования. Задача сводится к нахождению пути, для которого величина

$$\frac{\Pr(\mathbf{v}^{(N)} \mid \mathbf{c}^{(m)}) \Pr(\mathbf{c}^{(m)})}{\Pr(\mathbf{v}^{(N)})} = \frac{\prod_{i=0}^{m} \prod_{j=1}^{n_0} \Pr(v_i^j \mid c_i^j)}{\prod_{i=0}^{m} \prod_{j=1}^{n_0} \Pr(v_i^j)} \frac{1}{q^{(m+1)\,n_0 R}}$$

максимальна. Используемая нами в последовательном декодировании метрика пути, которая называется *метрикой Фано*, представляет собой логарифмическую функцию

$$\mu(\mathbf{c}^{(m)}) = \log_q \left[\frac{\Pr(\mathbf{v}^{(N)} \mid \mathbf{c}^{(m)}) \Pr(\mathbf{c}^{(m)})}{\Pr(\mathbf{v}^{(m)})} \right] =$$

$$= \sum_{i=0}^{m} \left[\sum_{j=1}^{n_0} \left\{ \log_q \frac{\Pr(v_i^j \mid c_i^j)}{\Pr(v_i^j)} - R \right\} \right].$$

В квадратную скобку заключен вклад в метрику Фано, вносимый i-м кадром. В каждом кадре каждого кодового слова необходимо

вычислять лишь этот член. Метрика пути $\mu(\mathbf{c}^{(m)})$ получается прибавлением соответствующего приращения к метрике Фано кодового слова, находившегося на вершине стека на предыдущей итерации.

Последовательное декодирование для жесткого решения, основанное на алгоритме Фано, уже обсуждалось в § 12.9; аналогичное обсуждение может быть проведено и для каналов с мягким решением. Единственно реальная разница между декодером для каналов с жестким решением и каналов с мягким решением состоит в используемой метрике пути. Соответствующая мягкому решению метрика пути — это метрика Фано. Единственное, что необходимо изменить в алгоритме Фано, описанном в § 12.9, — это ввести новую метрику. После такого изменения декодер Фано, описанный в § 12.9, станет пригодным и для каналов с мягким решением.

Любой алгоритм последовательного декодирования обладает следующими свойствами: этот декодер производит по меньшей мере одно вычисление в каждом посещаемом им узле; ребра исследуются последовательно, так что в любом узле выбор декодером ребра среди ранее не исследованных ребер не зависит от тех принятых ребер, которые расположены глубже на дереве. Это второе условие — критическое; оно приводит к характерным особенностям поведения последовательного декодирования. Декодеры блоковых кодов и синдромные декодеры сверточных кодов могут использовать последующую информацию, чтобы принять более раннее решение, и поэтому ведут себя иначе.

Число вычислений, производимых любым последовательным декодером для продвижения на один узел в глубь кодового дерева, является случайной величиной. Это обстоятельство в основном и влияет на сложность, требуемую для достижения заданного качественного уровня. Когда шум слаб, декодер обычно продвигается по правильному пути, используя лишь одно вычисление для продвижения в глубь кодового дерева на одно ребро. Однако когда шум становится сильным, декодер может проследовать вдоль неправильного пути, и прежде чем декодер найдет правильный путь, ему, возможно, потребуется произвести большое число вычислений. Непостоянство числа вычислений означает, что буферу входных данных может потребоваться большая память. Использование любого конечного буфера при последовательном декодировании приводит к ненулевой вероятности переполнения; это обстоятельство должно приниматься во внимание при вычислениях характеристик декодера.

ЗАДАЧИ

15.1. Привести простое выражение для метрики Фано в случаях q-ичного симметричного канала и гауссовского канала.

15.2. Построить блок-схему ОМР-алгоритма для недвоичных кодов.

15.3. Для сверточного кода с порождающей матрицей

$$\mathbf{G}(x) = \begin{bmatrix} x & x^2 & 1 \\ 0 & 1 & x \end{bmatrix}$$

построить систематический кодер, использующий в регистре сдвига обратную связь.

15.4. Разобрать случай декодирования, представленный на рис. 12.25, используя стек-алгоритм.

ЗАМЕЧАНИЯ

Возможность использования декодирования с мягким решением для улучшения качества декодирования стала ясной почти с того момента, когда были построены первые коды, однако хорошие декодеры для них тогда не были найдены. По существу первая попытка построить алгоритмический декодер мягкого решения для блоковых кодов была сделана Форни [1966], который ввел понятие обобщенного минимального расстояния. Несколько отличающийся алгоритм был предложен Чейзом [1972], который дополнил внешней петлей декодер двоичных кодов с жестким решением, исправляющий только ошибки. В работе Хартмана и Рудолфа [1976] предложен декодер мягкого решения совершенно иного типа.

Для сверточных кодов декодеры с мягким решением были введены легче и получили более широкое практическое распространение, чем для блоковых. Это произошло из-за того, что декодеры минимального расстояния, используемые для сверточных кодов, менее сложны, чем алгебраические декодеры, используемые для блоковых кодов, таких, как коды БЧХ. Хэммингово расстояние может быть заменено другими мерами разделения кодовых слов при весьма малом изменении структуры вычислений.

Унгербёк [1977, 1982] построил специально предназначенные для приемника мягкого решения коды, комбинируя выбор модуляции и выбор кода. Использование решетки для модуляции исследовали также Андерсон и Тейлор [1978], но вне связи с кодированием.

Последовательное декодирование было предложено Возенкрафтом [1957] и описано Возенкрафтом и Рейффеном [1961]. Оно было развито далее в работах Фано [1963], Зигангирова [1966] и Джелинека [1969]. Прогрессу в данной области способствовали также обзорные работы Джелинека [1968] и Форни [1974] и последующие разработки Шевийя и Костелло [1978] и Хаккоуна и Фергюсона [1975].

Нижние границы распределения числа вычислений последовательного декодирования получили Джекобс и Берлекэмп [1967], рассмотревшие также иллюстративный пример. Соответствующая верхняя граница была найдена Севиджем [1966].

ЛИТЕРАТУРА

Монографии

[1] Peterson W. W. Error-correcting codes. — Cambridge (Mass.): MIT Press, 1961. [Имеется перевод: Питерсон У. Коды, исправляющие ошибки. — М.: Мир, 1964.]

[2] Peterson W. W., Weldon E. J., Jr. Error-correcting codes. — 2nd ed. — Cambridge (Mass.): MIT Press, 1971. [Имеется перевод: Питерсон У., Уэлдон Э. Коды, исправляющие ошибки. — М.: Мир, 1976.]

[3] Berlekamp E. R. Algebraic coding theory. — New York: McGraw-Hill, 1968. [Имеется перевод: Берлекэмп Э. Алгебраическая теория кодирования. — М.: Мир, 1971.]

[4] Forney G. D. Concatenated codes. — Cambridge (Mass.): MIT Press, 1967. [Имеется перевод: Форни Д. Каскадные коды. — М.: Мир, 1970.]

[5] Gallager R. G. Information theory and reliable communication. — New York: McGraw-Hill, 1968, Ch. 6. [Имеется перевод: Галлагер Р. Теория информации и надежная связь. — М.: Сов. Радио, 1974.]

[6] Lin S. Introduction to error-correction codes. — Englewood Cliffs: Prentice-Hall, 1970.

[7] Lucky R. W., Salz J., Weldon E. J., Jr. Principles of data communication. — New York: McGraw-Hill, 1968, Ch. 11 and 12.

[8] Blake I. F., Mullin R. C. The mathematical theory of coding. — New York: Academic Press, 1975.

[9] Van Lint J. H. Coding theory. — New York: Springer, 1973.

[10] MacWilliams F. J., Sloane N. J. A. The theory of error-correcting codes. — New York: North-Holland, 1977. [Имеется перевод: Мак-Вильямс Ф. Дж., Слоэн Н. Дж. А. Теория кодов, исправляющих ошибки. — М.: Связь, 1979.]

[11] McEliece R. J. The theory of information and coding. — Reading: Addison-Wesley, 1977.

[12] Sloane N. J. A. A short course on error-correcting codes. — New York: Springer, 1975.

[13] Wiggert D. Error-control coding and applications. — Dedham: Artech House, 1979.

[14] Clark G. C., Bibb Cain J. Error-correction coding for digital communications. — New York: Plenum Press, 1981.

[15] Pless V. Introduction to the theory of error-correcting codes. — New York: Wiley-Interscience, 1982.

Обзоры

[1] Blake I. F. Algebraic coding theory. — Stroudsburg: Dowden, Hutchinson and Ross, 1972.

[2] Berlekamp E. R. Key papers in the development of coding theory. — New York: IEEE Press, 1974.

Глава 1

[1] Shannon C. E. A mathematical theory of communication. — Bell Syst. Tech. J., 1948, v. 27, p. 379—423 (Part I), p. 623—656 (Part II); reprinted in book form with postscript by W. Weaver. — Urbana: Univ. of Illinois Press, 1949. [Имеется перевод: Шеннон К. Работы по теории информации и кибернетике. — М.: ИЛ, 1963, с. 243—332.]
[2] Hamming R. W. Error detecting and error correcting codes. — Bell Syst. Tech. J., 1950, v. 29, p. 147—160. [Имеется перевод: Хэмминг Р. В. Коды с обнаружением и исправлением ошибок. — В кн.: Коды с обнаружением и исправлением ошибок. — М.: ИЛ, 1956, с. 7—22.]
[3] Bose R. C., Ray-Chaudhuri D. K. On a class of error-correcting binary group codes. — Inf. Contr., 1960, v. 3, p. 68—79. [Имеется перевод: Боуз Р. К., Рой-Чоудхури Д. К. Об одном классе двоичных групповых кодов с исправлением ошибок. — В кн.: Кибернетический сборник. Вып. 2. — М.: ИЛ, 1962, 83—94.]
[4] Hocquenghem A. Codes correcteurs d'erreurs. — Chiffres, 1959, t. 2, p. 147—156.
[5] Reed I. S., Solomon G. Polynomial codes over certain finite fields. — J. Soc. Indust. Appl. Math., 1960, v. 8, p. 300—304. [Имеется перевод: Рид И. С., Соломон Г. Полиномиальные коды над некоторыми конечными полями. — В кн.: Кибернетический сборник. Вып. 7. — М.: ИЛ, 1963, с. 74—79.]

Глава 2

[1] Birkhoff G., Mac Lane S. A survey of modern algebra. — Rev. ed. — New York: Macmillan, 1953.
[2] Van der Waerden B. L. Modern algebra. — 2 vols. — New York: Frederick Ungar, 1950, 1953. [Имеется перевод нем. изд. 1967 г.: Ван дер Варден Б. Л. Алгебра. — М.: Наука, 1976; 2-е изд. 1979.]
[3] Thrall R. M., Tornheim L. Vector spaces and matrices. — New York: Wiley, 1957.
[4] Fraleigh J. B. A first course in abstract algebra. — 2nd ed. — Reading: Addison-Wesley, 1976.
[5] Strang G. Linear algebra and its applications. — 2nd ed. — New York: Academic Press, 1980. [Имеется перевод изд. 1976 г.: Стренг Г. Линейная алгебра и ее применения. — М.: Мир, 1980.]
[6] Baumslag B., Chandler B. Theory and problems of group theory. — Schaum's Outline Series. — New York: McGraw-Hill, 1968.

Глава 3

[1] Hamming R. S. Error detecting and error correcting codes. — Bell Syst. Tech. J., 1950, v. 29, p. 147—160 [Имеется перевод: см. *гл. 1*, [2].]
[2] Golay M. J. E. Notes on digital coding. — Proc. IRE, 1949, v. 37, p. 657.
[3] Slepian D. A class of binary signalling alphabets. — Bell Syst. Tech. J., 1956, v. 35, p. 203—234. [Имеется перевод: Слепян Д. Класс двоичных сигнальных алфавитов. — В кн.: Теория передачи сообщений. — М.: ИЛ, 1957, с. 82—113.]
[4] Slepian D. Some further study of group codes. — Bell Syst. Tech. J., 1960, v. 39, p. 1219—1252.
[5] Kiyasu Z. Research and development data no. 4. — Electrical Communications Laboratory, Nippon Tele. Corp., Tokyo, 1953.
[6] Singleton R. C., Maximum distance q-nary codes. — IEEE Trans. Inf. Theor., 1964, v. IT-10, p. 116—118.

[7] Golay M. J. E. Notes on the penny-weighing problem, lossless symbol coding with nonprimes, etc. — IRE Trans. Inf. Theor., 1958, v. IT-4, p. 103—109.
[8] Cocke J. Lossless symbol coding with nonprimes. — IRE Trans. Inf. Theor., 1959, v. IT-5, p. 33—34.
[9] Tietäväinen A. A short proof for the nonexistence of unknown perfect codes over $GF(q)$, $q > 2$. — Annales Acad. Scient. Fennicae, Ser. A., 1974, No. 580, p. 1—6.
[10] Van Lint J. H. A survey of perfect codes. — Rocky Mountain J. Math., 1975, v. 5, p. 199—224.
[11] Васильев Ю. Л. О негрупповых плотно упакованных кодах. — В кн.: Проблемы кибернетики. Вып. 8. — М.: Физматгиз, 1962, с. 337—339.
[12] Schönheim J. On linear and nonlinear single-error-correcting q-nary perfect codes. — Inf. and Contr., 1968, v. 12, p. 23—26.
[13] Muller D. E. Application of Boolean algebra to switching circuit desing and to error detection. — IRE Trans. Electr. Comp., 1954, v. EC-3, p. 6—12.
[14] Reed I. S. A class of multiple-error-correcting codes and the decoding scheme. — IRE Trans. Inf. Theor., 1954, v. IT-4, p. 38—49. [Имеется перевод: Рид И. С. Класс кодов с исправлением нескольких ошибок и схема декодирования. — В кн.: Кибернетический сборник. Вып. 1. — М.: ИЛ, 1960, с. 189—205.]

Глава 4

[1] Birkhoff G., Mac Lane S. A survey of modern algebra. — Rev. ed. — New York: Macmillan, 1953.
[2] Van der Waerden B. L. Modern algebra. — 2 vols. — New York: Frederick Unger, 1950, 1953. [Имеется перевод: см. *гл. 2*, [2].]
[3] Berlekamp E. R. Algebraic coding theory. — New York: McGraw-Hill, 1968. [Имеется перевод: см. *монографии*, [3].]
[4] Fraleigh J. B. A first course in abstract algebra. — 2nd ed. — Reading: Addison-Wesley, 1976.

Глава 5

[1] Prange E. Cyclic error-correcting codes in two symbols. — AFCRC-TN-57-103, Air Force Cambridge Research Center, Cambridge (Mass.), Sept. 1957.
[2] Prange E. Some cyclic error-correcting codes with simple decoding algorithms. — AFCRC-TN-58-156, Air Force Cambridge Research Center, Bedford (Mass.), April 1958.
[3] Peterson W. W. Encoding and error-correction procedures for the Bose—Chaudhuri codes. — IRE Trans. Inf. Theor., 1960, v. IT-6, p. 459—470. [Имеется перевод: Питерсон У. Кодирование и исправление ошибок для кодов Боуза—Чоудхури. — В кн.: Кибернетический сборник. Вып. 6. — М.: ИЛ, 1963, с. 25—54.]
[4] Kasami T. Systematic codes using binary shift register sequences. — J. Inf. Process. Soc. Japan, 1960, v. 1, p. 198—200.
[5] Abramson N. A note on single error-correcting binary codes. — IRE Trans. Inf. Theor., 1960, v. IT-6, p. 502—503.
[6] Elspas B. A note on P-nary adjacent-error-correcting codes. — IRE Trans. Inf. Theor., 1960, v. IT-6, p. 13—15.
[7] Abramson N. A. Class of systematic codes for non-independent errors. — IRE Trans. Inf. Theor., 1959, v. IT-5, p. 150—157.
[8] Kasami T. Optimum shortened cyclic codes for burst-error correction. — IEEE Trans. Inf. Theor., 1963, v. IT-9, p. 105—109.
[9] Peterson W. W., Weldon E. J. Jr. Error correcting codes. — 2nd ed. — Cambridge (Mass.): MIT Press, 1971. [Имеется перевод: см. *монографии*, [2].]

[10] Fire P. A class of multiple-error correcting binary codes for non-independent errors. — Sylvania Report RSL-E-2, Sylvania Reconnaissance Systems Lab., Mountain View, Calif., 1959.
[11] Golay M. J. E. Notes on digital coding. — Proc. IRE, 1949, v. 37, p. 657.
[12] McEliece R. J. The theory of information and coding. — Reading: Addison-Wesley, 1977.
[13] Assmus E. F., Jr., Mattson H. F., Jr. Coding and combinatorics. — SIAM Rev., 1974, v. 16, p. 349—388.

Глава 6

[1] Peterson W. W. Encoding and error-correction procedures for the Bose—Chaudhuri codes. — IRE Trans. Inf. Theor., 1960, v. IT-6, p. 459—470. [Имеется перевод: см. *гл. 5*, [3].]
[2] Chien R. T. Cyclic decoding procedures for Bose—Chaudhuri—Hocquenghem codes. — IEEE Trans. Inf. Theor., 1964, v. IT-10, p. 357—363.
[3] Peterson W. W. Error-correcting codes. — Cambridge (Mass.): MIT Press; New York: Wiley, 1961. [Имеется перевод: см. *монографии*, [1].]
[4] Meggitt J. E. — Error-correcting codes for correcting bursts of errors. — IBM J. Res. Develop., 1960, v. 4, p. 329—334.
[5] Meggitt J. E. Error-correcting codes and their implementation. — IRE Trans. Inf. Theor., 1961, v. IT-7, p. 232—244.
[6] Kasami T. A decoding procedure for multiple-error-correcting cyclic codes. — IEEE Trans. Inf. Theor., 1964, v. IT-10, p. 134—139.
[7] MacWilliams F. J. Permutation decoding of systematic codes. — Bell Syst. Tech. J., 1964, v. 43, p. 485—505. [Имеется перевод: Мак-Вильямс Ф. Дж. Перестановочное декодирование систематических кодов. — В кн.: Кибернетический сборник. Новая серия. Вып. 1. — М.: Мир, 1966, с. 35—57.]
[8] Mitchell M. E. Error-trap decoding of cyclic codes. — G. E. Report No. 62MCD3, General Electric Military Communications Department, Oklahoma City, 1962.
[9] Rudolph L., Mitchell M. E. Implementation of decoders for cyclic codes. — IEEE Trans. Inf. Theor., 1964, v. IT-10, p. 259—260.

Глава 7

[1] Hochquenghem A. Codes correcteurs d'erreurs. — Chiffres, 1959, t. 2, p. 147—156.
[2] Bose R. C., Ray-Chaudhuri D. K. On a class of error correcting binary group codes. — Inf. and Contr., 1960, v. 3, p. 68—79. [Имеется перевод: см. *гл. 1*, [3].]
[3] Reed I. S., Solomon G. Polynomial codes over certain finite fields. — J. Soc. Indust. Appl. Math., 1960, v. 8, p. 300—304. [Имеется перевод: см. *гл. 1*, [5].]
[4] Kasami T., Tokura N. Some remarks on BCH bounds and minimum weights of binary primitive BCH codes. — IEEE Trans. Inf. Theor., 1969, v. IT-15, p. 408—412.
[5] Chen C. L. Computer results on the minimum distance of some binary cyclic codes. — IEEE Trans. Inf. Theor., 1970, v. IT-16, p. 359—360.
[6] Peterson W. W. Encoding and error-correction procedures for the Bose-Chaudhuri codes. — IEEE Trans. Inf. Theor., 1960, v. IT-6, p. 459—470. [Имеется перевод: см. *гл. 5*, [3].]
[7] Gorenstein D. C., Zierler N. A class of error-correcting codes in p^m symbols. — J. Soc. Indust. Appl. Math., 1961, v. 9, p. 207—214. [Имеется перевод: Горенстейн Д., Цирлер Н. Класс кодов из p^m символов с исправ-

лением ошибок. — В кн.: Кибернетический сборник. Вып. 7. — М.: ИЛ, 1963, с. 80—89.]
[8] Chien R. T. Cyclic decoding procedures for Bose—Chaudhuri—Hocquenghem codes. — IEEE Trans. Inf. Theor., 1964, v. IT-10, p. 357—363.
[9] Forney G. D., Jr. On decoding BCH codes. — IEEE Trans. Inf. Theor., 1965, v. IT-11, p. 549—557.
[10] Berlekamp E. R. Algebraic coding theory. — New York: McGraw-Hill, 1968. [Имеется перевод: см. *монографии*, [3].]
[11] Massey J. L. Shift-register syntesis and BCH decoding. — IEEE Trans. Inf. Theor., 1969, v. IT-15, p. 122—127.
[12] Burton H. O. Inversionless decoding of binary BCH codes. — IEEE Trans. Inf. Theor., 1971, v. IT-17, p. 464—466.
[13] Sugiyama Y., Kasahara M., Hirasawa S., Namekawa T. A method for solving key equation for decoding Goppa codes. — Inf. and Contr., 1975, v. 27, p. 87—99.
[14] Welch L. R., Scholtz R. A. Continued fractions and Berlekamp's algorithm. — IEEE Trans. Inf. Theor., 1979, v. IT-25, p. 19—27.
[15] Mandelbaum D. M. Decoding beyond the designed distance for certain algebraic codes. — Inf. and Contr., 1977, p. 209—228.

Глава 8

[1] Reed I. S., Solomon G. Polynomial codes over certain finite fields. — J. SIAM, 1960, v. 8, p. 300—304. [Имеется перевод: см. *гл. 1*, [5].]
[2] Mattson H. F., Solomon G. A new treatment of Bose—Chaudhuri codes. — J. Soc. Indust. Appl. Math., 1961, v. 9, p. 654—699. [Имеется перевод: Мэттсон Х., Соломон Г. Новая трактовка кодов Боуза—Чоудхури. — В кн.: Теория кодирования. — М.: Мир, 1964, с. 7—29.]
[3] Mandelbaum D. M., Construction of error-correcting codes by interpolation.— IEEE Trans. Inf. Theor., 1979, v. IT-25, p. 27—35.
[4] Pollard J. M. The fast Fourier transform in a finite field. — Math. Computat., 1971, v. 25, p. 365—374. [Имеется перевод: Поллард Дж. М. Быстрое преобразование Фурье в конечном поле. — В кн.: Макклеллан Дж. Х., Рейдер Ч. М. Применение теории чисел в цифровой обработке сигналов. — М.: Радио и связь, 1983, с. 147—155.]
[5] Gore W. C. Transmitting binary symbols with Reed—Solomon codes. — Proc. Princeton Conf. Inf. Sci. Syst., Princeton, 1973, p. 495—497.
[6] Chien R. T., Choy D. M. Algebraic generalization of BCH—Goppa—Helgert codes. — IEEE Trans. Inf. Theor., 1975, v. IT-21, p. 70—79.
[7] Lempel A., Winograd S. A new approach to error-correcting codes. — IEEE Trans. Inf. Theor., 1977, v. IT-23, p. 503—508.
[8] Chien R. T. A new proof of the BCH bound. — IEEE Trans. Inf. Theor., 1972, v. IT-18, p. 541.
[9] Wolf J. K. Adding two information symbols to certain nonbinary BCH codes and some applications. — Bell Syst. Tech. J., 1969, v. 48, p. 2405—2424.
[10] Андрианов В. И., Сасковец В. Н. Дециклические коды. — Кибернетика, 1966, № 1, с. 1—6.
[11] Sloane N. J. A., Reddy S. M., Chen C. L. New binary codes. — IEEE Trans. Inf. Theor., 1972, v. IT-18, p. 503—510.
[12] Kasahara M., Sugiyama Y., Hirasawa S., Namekawa T. A new class of binary codes constructed on the basis of BCH codes. — IEEE Trans. Inf. Theor., 1975, v. IT-21, p. 582—585.
[13] Blahut R. E. On extended BCH codes. — Proc. 18th Allerton Conf. Commun. Contr. Comput., Univ. of Illinois, Monticello, 1980, p. 50—59.
[14] Helgert H. H. Alternant codes. — Inf. and Contr., 1974, v. 26, p. 369—381.

[15] Гоппа В. Д. Новый класс линейных корректирующих кодов. — Проблемы передачи информации, 1970, вып. 3, с. 24—30.
[16] Delsarte P. On subfield subcodes of modified Reed—Solomon codes. — IEEE Trans. Inf. Theor., 1975, v. IT-21, p. 575—576.
[17] Preparata F. P. A class of optimum nonlinear double-error-correcting codes. — Inf. and Contr., 1968, v. 13, p. 378—400. [Имеется перевод: Препарата Ф. П. Класс оптимальных нелинейных кодов с исправлением двойных ошибок. — В кн.: Кибернетический сборник. Новая серия. Вып. 7. — М.: 1970, с. 18—42.]
[18] Nordstrom A. W., Robinson J. P. An optimum linear code. — Inf. and Contr., 1967, v. 11, p. 613—616.
[19] Nadler M. A 32-point n equals 12, d equals 5 code. — IRE Trans. Inf. Theor., 1962, v. IT-8, p. 58.
[20] Green M. W. Two heuristic techniques for block code construction. — IEEE Trans. Inf. Theor., 1966, v. IT-12, p. 273.
[21] Kerdock A. M. A class of low-rate nonlinear codes. — Inf. and Contr., 1972, v. 20, p. 182—187. [Имеется перевод: Кердок А. М. Класс нелинейных двоичных кодов с низкой скоростью передачи — В кн.: Кибернетический сборник. Новая серия. Вып. 10. — М.: Мир, 1973, с. 33—38.]
[22] Goethals J.-M. Nonlinear codes defined by quadratic forms over GF (2). — Inf. and Contr., 1976, v. 10, p. 43—74.

Глава 9

[1] Reed I. S., Solomon G. Polynomial codes over certain finite fields. — J. Soc. Indust. Appl. Math., 1960, v. 8, p. 300—304. [Имеется перевод: см. *гл. 1*, [5].]
[2] Mandelbaum D. On decoding of Reed-Solomon codes. — IEEE Trans. Inf. Theor., 1971, v. IT-17, p. 707—712.
[3] Paschburg R. H. Software implementation of error-correcting codes. — MS thesis. — Univ. of Illinois, Urbana, 1974.
[4] Gore W. C. Transmitting binary symbols with Reed—Solomon codes. — Proc. Princeton Conf. Inf. Sci. Syst., Princeton, 1973, p. 495—497.
[5] Michelson A. A fast transform in some Galois fields and an application to decoding Reed—Solomon codes. — IEEE Abstr. of Papers — IEEE Internat. Symp. Inf. Theor., Ronneby (Sweden), 1976.
[6] Blahut R. E. Transform techniques for error-control codes. — IBM J. Res. Develop., 1979, v. 23, p. 299—315.
[7] Blahut R. E. Algebraic decoding in the frequency domain. — In: Algebraic coding theory and practice. Ed. by G. Longo. — New York: Springer, 1979.
[8] Blahut R. E. On extended BCH codes. — Proc. 18th Allerton Conf. Circ. Syst. Theor., Univ. of Illinois, Monticello, 1980, p. 50—59.
[9] Mandelbaum D. Decoding beyond the designed distance for certain algebraic codes. — Inf. and Contr., 1977, v. 35, p. 209—228.
[10] Elias P. Error-free coding. — IRE Trans. Inf. Theor., 1954, v. IT-4, p. 29—37.
[11] Blum R. A., Weiss A. D. Further results in error-correcting codes. — SM thes. — MIT, Cambridge, 1960.
[12] Forney G. D., Jr. On decoding BCH codes. — IEEE Trans. Inf. Theor., 1965, v. IT-11, p. 549—557.
[13] Wolf J. K. Adding two information symbols to certain nonbinary BCH codes and some applications. — Bell Syst. Tech. J., 1969, v. 48, p. 2405—2424.
[14] Kasahara M., Sugiyama Y., Hirasawa S., Namekawa T., A new class of binary codes constructed on the basis of BCH codes. — IEEE Trans. Inf. Theor., 1975, v. IT-21, p. 582—585.
[15] Patterson N. J. The algebraic decoding of Goppa codes. — IEEE Trans. Inf. Theor., 1975, v. IT-21, p. 203—207.

[16] Helgert H. J. Decoding of alternant codes. — IEEE Trans. Inf. Theor., 1977, v. IT-23, p. 513—514.
[17] Delsarte P. On subfield-subcodes of modified Reed—Solomon codes. — IEEE Trans. Inf. Theor., 1975, v. IT-21, p. 575—576.
[18] Berlekamp E. R. Algebraic coding theory. — New York: McGraw-Hill, 1968. [Имеется перевод: см. *монографии*, [3].]
[19] Hartmann C. R. P. Decoding beyond the BCH bound. — IEEE Trans. Inf. Theor., 1972, v. IT-18, p. 441—444.
[20] Vanderhorst J., Berger T. Complete decoding of triple-error-correcting binary BCH codes. — IEEE Trans. Inf. Theor., 1976, v. IT-22, p. 138—147.
[21] Bluestein L. I. Linear filtering approach to the computation of discrete Fourier transforms. — IEEE Trans. Audio Electroacoust., 1970, v. AU-18, p. 451—455.
[22] Rader C. M. Discrete Fourier transforms when the number of data samples is prime. — Proc. IEEE, 1968, v. 56, p. 1107—1108. [Имеется перевод: Рейдер Ч. М. Дискретное преобразование Фурье, когда число отсчетов простое. — В кн.: Макклеллан Дж. Х., Рейдер Ч. М. Применение теории чисел в цифровой обработке сигналов. — М.: Радио и связь, 1983, с. 89—91.]
[23] Goertzel G. — An algorithm for the evaluation of finite trigonometric series. — Amer. Math. Mon., 1968, v. 65, p. 34—35.
[24] Oppenheim A. V., Schafer R. W. Digital signal processing. — Englewood Cliffs: Prentice-Hall, 1975.
[25] Rabiner L. R., Gold B. Theory and application of digital signal processing. — Englewood Cliffs: Prentice-Hall, 1975. [Имеется перевод: Рабинер Р. Л., Гоулд Б. Теория и применение цифровой обработки сигналов. — М.: Мир, 1978.]
[26] Sarwate D. V. Semi-fast Fourier transforms over $GF(2^m)$. — IEEE Trans. Comput., 1978, v. C-27, p. 283—284.

Глава 10

[1] Elias P. Error-free coding. — IRE Trans. Inf. Theor., 1954. v. IT-4, p. 29—37.
[2] Burton H. O., Weldon E. J., Jr. Cyclic product codes. — IEEE Trans. Inf. Theor., 1965, v. IT-11, p. 433—439.
[3] Lin S., Weldon E. J., Jr. Further results on cyclic product codes. — IEEE Trans. Inf. Theor., 1970, v. IT-16, p. 452—459.
[4] Reddy S. M., Robinson J. P. Random-error and burst correction by iterated codes. — IEEE Trans. Inf. Theor., 1972, v. IT-18, p. 182—185.
[5] Weldon E. J., Jr. Decoding binary block codes on Q-ary output channels. — IEEE Trans. Inf. Theor., 1971, v. IT-17, p. 713—718.
[6] Blahut R. E. Transform techniques for error-control codes. — IBM J. Res. Develop., 1979, v. 23, p. 299—315.
[7] Chien R. T., Ng S. W. Dual product codes for correction of multiple low-density burst errors. — IEEE Trans. Inf. Theor., 1973, v. IT-19, p. 672—678.

Глава 11

[1] Cooley J. W., Tukey J. W. An algorithm for the machine computation of complex Fourier series. — Math. Comp., 1965, v. 19, p. 297—301.
[2] Good I. J. The interaction algorithm and practical Fourier analysis. — J. Roy. Statist. Soc., 1958, v. 20, p. 361—375; addendum, 1960, v. 22, p. 372—375.
[3] Thomas L. H. Using a computer to solve problems in physics. — In: Applications of digital computers. — Boston: Ginn, 1963.

[4] Justesen J. On the complexity of decoding Reed—Solomon codes. — IEEE Trans. Inf. Theor., 1976, v. IT-22, p. 237—238.
[5] Sarwate D. V. On the complexity of decoding Goppa codes. — IEEE Trans. Inf. Theor., 1977, v. IT-23, p. 515—516.
[6] Agarwal R., Cooley J. W. Algorithms for digital convolution. — IEEE Trans. Acoust. Speech Signal Process, 1977, v. ASSP-25, p. 392—410. [Имеется перевод: Агарвал Р. К., Кули Дж. У. Новые алгоритмы для цифровой свертки. — В кн.: Макклеллан Дж. Х., Рейдер Ч. М. Применение теории чисел в цифровой обработке сигналов. — М.: Радио и связь, 1983, с. 91—117.]
[7] Winograd S. On computing the discrete Fourier transform. — Math. Comp., 1978, v. 32, p. 175—199. [Имеется перевод: Виноград С. О вычислении дискретного преобразования Фурье. — В кн.: Макклеллан Дж. Х., Рейдер Ч. М. Применение теории чисел в цифровой обработке сигналов. — М.: Радио и связь, 1983, с. 117—136.]
[8] Nussbaumer H. J. Fast Fourier transform and convolution algorithms. — Berlin: Springer, 1981. [Имеется перевод: Нуссбаумер Г. Быстрое преобразование Фурье и алгоритмы вычисления свертки. — М.: Радио и связь, 1985.]
[9] Miller R. L., Truong T. K., Reed I. S. Efficient program for decoding the (255, 223) Reed—Solomon code over $GF(2^8)$ mith both errors and erasures, using transform decoding. — IEEE Proc., 1980, v. 127, p. 136—142.
[10] Blahut R. E. Efficient decoder algorithms based on spectral techniques. — IEEE Abstr. of Papers — IEEE Internat. Sympos. Inf. Theor. — Santa Monica, 1981.
[11] Preparata F. P., Sarwate D. V. Computational complexity of Fourier transforms over finite fields. — Math. Comp., 1977, v. 31, p. 740—751.

Глава 12

[1] Elias P. Error-free coding. — IRE Trans. Inf. Theor., 1954, v. IT-4, p. 29—37.
[2] Wozencraft J. M. Sequential decoding for reliable communication. — 1957 Nat. IRE Conv. Rec., v. 5, p. 11—25.
[3] Wyner A. D., Ash R. B. Analysis of recurrent codes. — IEEE Trans. Inf. Theor., 1963, v. IT-9, p. 143—156. [Имеется перевод: Вайнер А. Д., Эш Р. Б. Анализ рекурреитных кодов. — В кн.: Кибернетический сборник. Новая серия. Вып. 5.— М.: Мир, 1968, с. 5—33.]
[4] Massey J. L., Threshold decoding. — Cambridge (Mass.): MIT Press, 1963. [Имеется перевод: Месси Дж. Пороговое декодирование. — М.: Мир, 1966.]
[5] Costello D. J., Jr. A construction technique for random-error-correcting codes. — IEEE Trans. Inf. Theor., 1969, v. IT-15, p. 631—636.
[6] Bussgang J. J. Some properties of binary convolutional code generators. — IEEE Trans. Inf. Theor., 1965, v. IT-11, p. 90—100.
[7] Odenwalder J. P. Optimal decoding of convolutional codes. — Ph. D. diss.— Univ. of California,Los Angeles, 1970.
[8] Bahl L. R., Jelinek F. Rate 1/2 convolutional codes with complementary generators. — IEEE Trans. Inf. Theor., 1971, v. IT-17, p. 718—727.
[9] Larsen K. J., Short convolutional codes with maximal free distance for rates 1/2, 1/3 and 1/4. — IEEE Trans. Inf. Theor., 1973, v. IT-19, p. 371—372.
[10] Paaske E. Short binary convolutional codes with maximal free distance for rates 2/3 and 3/4. — IEEE Trans. Inf. Theor., 1974, v. IT-20, p. 683—689.
[11] Johannesson R. Robustly optimal rate one-half binary convolutional codes.— IEEE Trans. Inf. Theor., 1975, v. IT-21, p. 464—468.
[12] Massey J. L., Sain M. K. Inverses of linear sequential circuits. — IEEE Trans. Comp., 1968, v. C-17, p. 330—337.

[13] Forney G. D., Jr. Convolutional codes I: Algebraic structure. — IEEE Trans. Inf. Theor., 1970, v. IT-16, p. 720—738.
[14] Lindner R., Staiger L. Algebraische Codierungstheorie. — Berlin: Akademie-Verlag, 1977.
[15] Fano R. M. A heuristic discussion of probabilistic decoding. — IEEE Trans. Inf. Theor., 1963, v. IT-9, p. 64—74. [Имеется перевод: Фано Р. М. Эвристическое обсуждение вероятностного декодирования. — В кн.: Теория кодирования. — М.: Мир, 1964, с. 166—198.]
[16] Gallager R. G. Information theory and reliable communication. — New York: Wiley, 1968. [Имеется перевод: см. *монографии*, [5].]
[17] Viterbi A. J. Error bounds for convolutional codes and an asymptotically optimum decoding algorithm. — IEEE Trans. Inf. Theor., 1967, v. IT-13, p. 260—269. [Имеется перевод: Витерби А. Границы ошибок для сверточных кодов и асимптотически оптимальный алгоритм декодирования. — В кн.: Некоторые вопросы теории кодирования. — М.: Мир, 1970, с. 142—165.]
[18] Heller J. A. Short constraint length convolutional codes. — Jet Propulsion Labs. Space Prog. Sum. 37-54 III, 1968, p. 171—177.
[19] Viterbi A. J., Omura J. K. Principles of digital communication and coding. — McGraw-Hill, 1979. [Имеется перевод: Витерби А. Д., Омура Дж. К. Принципы цифровой связи и кодирования. — М.: Радио и связь, 1982.]
[20] Massey J. L. Error bounds for tree codes, trellis codes and convolutional codes with encoding and decoding procedures. — In: Coding and complexity. Ed. by G. Longo. — New York: Springer, 1975.
[21] Forney G. D., Jr. Convolutional codes II: Maximum-likelihood decoding and convolutional codes; III: Sequential decoding. — Inf. and Contr., 1974, v. 25, p. 222—297.

Глава 13

[1] Massey J. L. Threshold decoding. — Cambridge (Mass.): MIT Press, 1963. [Имеется перевод: см. *гл. 14*, [4].]
[2] Reed I. S. A class of multiple-error-correcting codes and the decoding scheme. — IRE Trans. Inf. Theor., 1954, v. IT-4, p. 38—49. [Имеется перевод: см. *гл. 3*, [14].]
[3] Robinson J. P., Bernstein A. J. A class of recurrent codes with limited error propagation. — IEEE Trans. Inf. Theor., 1967, v. IT-13, p. 106—113.
[4] Wu W. W. New convolutional codes. — IEEE Trans. Communicat. Theor., part I, 1975, v. COM-23, p. 142—156; part II, 1976, v. COM-24, p. 19—33; part III, 1976, v. COM-24, p. 946—955.
[5] Rudolph L. D. Geometric configuration and majority-logic decodable codes. — M. E. E. thesis. — Univ. of Oklahoma, Norman, 1964.
[6] Rudolph L. D. A class of majority-logic decodable codes. — IEEE Trans. Inf. Theor., 1967, v. IT-14, p. 305—307.
[7] Kasami T., Lin S., Peterson W. W. New generalizations of the Reed—Muller codes. Part I: Primitive codes. — IEEE Trans. Inf. Theor., 1968, v. IT-14, p. 189—199.
[8] Колесник В. Д., Мирончиков Е. Т. Циклические коды Рида—Маллера и их декодирование. — Проблемы передачи информации, 1968, вып. 4, с. 15—19.
[9] Weldon E. J., Jr. Euclidean geometry cyclic codes. — Proc. Sympos. Combinatorial Math., Univ. of North Carolina, Chapel Hill, 1967.
[10] Weldon E. J., Jr. New generalizations of the Reed—Muller codes. Part II: Non-primitive codes. — IEEE Trans. Inf. Theor., 1968, v. IT-14, p. 199—206.

[11] Kasami T., Lin S., Peterson W. W. Polynomial codes. — IEEE Trans. Inf. Theor., 1968, v. IT-14, p. 807—814.
[12] Goethals J. M., Delsarte P. On a class of majority-logic decodable cyclic codes. — IEEE Trans. Inf. Theor., 1968, v. IT-14, p. 182—189. [Имеется перевод: Гетале Дж. М., Дейзарт П. Один класс циклических кодов с мажоритарным декодированием. — В кн.: Кибернетический сборник. Новая серия. Вып. 6. — М.: Мир, 1969, с. 7—23.]
[13] Delsarte P. J., Goethals J. M., MacWilliams F. J. On generalized Reed—Muller codes and their relatives. — Inf. and Contr., 1970, v. 16, p. 402—442.
[14] Lin S., Markowsky G. On a class of one-step majority-logic decodable cyclic codes. — IBM J. Res. Develop., 1980, v. 24, p. 56—63.
[15] Goethals J. M. Threshold decoding—a tentative survey. — In: Coding and complexity. Ed. by G. Longo. — New York: Springer, 1975.

Глава 14

[1] Assmus E. F., Jr., Mattson H. F., Jr., Turyn R. J. Cyclic codes. — AFCRL-65-332, Air Force Cambridge Research Labs, Bedford, Mass., 1965.
[2] Forney G. D., Jr. Concatenated codes. — Cambridge (Mass.): MIT Press, 1966. [Имеется перевод: см. *монографии*, [4].]
[3] Kasami T., Lin S., Peterson W. W. Some results on weight distributions of BCH codes. — IEEE Trans. Inf. Theor., 1966, v. IT-12, p. 274.
[4] MacWilliams F. J. A theorem on the distribution of weights in a systematic code. — Bell Syst. Tech. J., 1963, p. 79—94.
[5] Huntoon Z. McC., Michelson A. M. On the computation of the probability of post-decoding error events for block codes. — IEEE Trans. Inf. Theor., 1977, v. IT-23, p. 399—403.
[6] Viterbi A. J. Convolutional codes and their performance in communication systems. — IEEE Trans. Communicat. Tech., 1971, v. COM-19, p. 751—772.
[7] Gilbert E. N. A comparison of signalling alphabets. — Bell Syst. Tech. J., 1952, v. 31, p. 504—522.
[8] McEliece R. J., Rodemich E. R., Rumsey H., Jr., Welch L. R. New upper bounds on the rate of a code via the Delsarte—MacWilliams inequalities.— IEEE Trans. Inf. Theor., 1977, v. IT-23, p. 157—166. [Имеется перевод: Мак-Элис Р. Д., Родемич Е. Р., Рамсей Г., мл., Велч Л. Р. Новые верхние границы для скорости кода, основанные на неравенствах Дельсарта—Мак-Вильямс. — В кн.: Кибернетический сборник. Новая серия. Вып. 17.— М.: Мир, 1980, с. 6—27.]
[9] Costello D. J. Free distance bounds for convolutional codes. — IEEE Trans. Inf. Theor., 1974, v. IT-20, p. 356—365.
[10] Savage J. E. Minimum distance estimates of the performance of sequential decoding. — IEEE Trans. Inf. Theor., 1969, v. IT-15, p. 128—140.

Глава 15

[1] Forney G. D., Jr. Generalized minimum distance decoding. — IEEE Trans. Inf. Theor., 1966, v. IT-12, p. 125—131.
[2] Chase D. A class of algorithms for decoding block codes with channel measurement information. — IEEE Trans. Inf. Theor., 1972, v. IT-18, p. 170—182.
[3] Hartmann C. R. P., Rudolph L. D. An optimum symbol-by-symbol decoding rule for linear codes. — IEEE Trans. Inf. Theor., 1976. v. IT-22, p. 514—517.
[4] Ungerboeck G. Trellis coding with expanded channel signal sets. — Abstr.— 1977 IEEE Internat. Sympos. Inf. Theor., Ithaca, 1977.
[5] Ungerboeck G. Channel coding with multilevel phase signals. — IEEE Trans. Inf. Theor., 1982, v. IT-28, p. 55—67.

[6] Anderson J. B., Taylor D. P. A bandwidth-efficient class of signal-space codes. — IEEE Trans. Inf. Theor., 1978, v. IT-24, p. 703—712.
[7] Wozencraft J. M. Sequential decoding for reliable communication. — 1957 Nat. IRE Conv. Rec., v. 5, part 2, 1957, p. 11—25.
[8] Wozencraft J. M., Reiffen B. Sequential decoding. — Cambridge (Mass.): MIT Press, 1961. [Имеется перевод: Возенкрафт Дж., Рейффен Б. Последовательное декодирование. — М.: ИЛ, 1963.]
[9] Fano R. M. A heuristic discussion of probabilistic decoding. — IEEE Trans. Inf. Theor., 1963, v. IT-9, p. 64—74. [Имеется перевод: см. *гл. 12*, [15].]
[10] Зигангиров К. Ш. Некоторые процедуры последовательного декодирования. — Проблемы передачи информации, 1966, вып. 2, с. 13—25.
[11] Jelinek F. A fast sequential decoding algorithm using a stack. — IBM J. Res. Develop., 1969, v. 13, p. 675—685.
[12] Forney G. D., Jr. Convolutional codes III: Sequential decoding. — Inform. Contr., 1974, v. 25, p. 267—297.
[13] Jelinek F. Probabilistic information theory. — New York: McGraw-Hill, 1968.
[14] Chevillat P. R., Costello D. J., Jr. An analysis of sequential decoding for specific time-invariant convolutional codes. — IEEE Trans Inf. Theor., 1978, v. IT-24, p. 443—451.
[15] Haccoun D., Ferguson M. J. Generalized stack algorithms for decoding convolutional codes. — IEEE Trans. Inf. Theor., 1975, v. IT-21, p. 638—651.
[16] Savage J. E. Sequential decoding — The computation problem. — Bell Syst. Tech. J., 1966, v. 45, p. 149—175.
[17] Jacobs I. M., Berlekamp E. R. A lower bound to the distribution of computations for sequential decoding. — IEEE Trans. Inf. Theor., 1967, v. IT-13, p. 167—174. [Имеется перевод: Джекобс И., Берлекэмп Е. Нижняя граница количества вычислений для последовательного декодирования. — В кн.: Некоторые вопросы теории кодирования. — М.: Мир, 1970, с. 230—248.]
[18] Jelinek F. An upper bound on moments of sequential decoding effort. — IEEE Trans. Inf. Theor., 1969, v. IT-15, p. 140—149.

ИМЕННОЙ УКАЗАТЕЛЬ

Абель (Abel N. H.) 60
Агарвал (Agarwal R.) 398
Андерсон (Anderson J. B.) 552
Андрианов В. Н. 284
Ассмус (Assmus E. F.) 153, 521
Афанасьев В. Б. 397

Бартон (Burton H. O.) 238, 353
Бергер (Berger T.) 10, 324
Берлекэмп (Berlecamp E. R.) 14, 111, 203, 214, 238, 324, 552
Бернстайн (Bernstein A. J.) 491
Бест (Best M. R.) 10
Биркгоф (Birkhoff A. G.) 60, 111
Блейхут Б. (Blahut B.) 10
Блейхут Р. Э. (Blahut R. E.) 284, 324, 353, 398
Блейхут Э. Дж. (Blahut E. J.) 10
Блечман (Blachman N. M.) 10
Блюм (Blum R. A.) 324
Блюстейн (Bluestein L. I.) 324
Бол (Bahl L. R.) 447
Боуз (Bose R. C.) 14, 238
Буссганг (Bussgang J. J.) 447

Вайнер (Wyner A. D.) 447
Ван-дер-Варден (Van der Waerden B.L.) 60, 111
Вандеркулк (Vanderkulk W.) 10
Вандерхорст (Vanderhorst J.) 324
Ван Линт (Van Lint J. H.) 83
Васильев Ю. Л. 83
Вейс (Weiss A. D.) 324
Велч (Welch R. L.) 238, 522
Вест (West S. C.) 10
Виноград (Winograd S.) 10, 284, 398
Витерби (Viterbi A. J.) 10, 447, 522
Владус С. Г. (Vladuts S. G.) 522
Возенкрафт (Wozencraft J. M.) 447, 552
Вулф (Wolf J. K.) 284, 324
Ву (Wu W. W.) 491

Галлагер (Gallager R. G.) 447
Галуа (Galois E.) 60
Герцель (Goertzel G.) 324
Геталс (Goethals J. M) 284, 491
Гилберт (Gilbert E. N.) 522
Голд (Gold B.) 324
Голей (Golay M. J. E.) 83, 153
Головина Л. И. 60
Гоппа В. Н. 14, 284, 522
Гор (Gore W. .C) 284, 324
Горенстейн (Gorenstein D. C.) 193, 238
Грин (Green M. W.) 284
Гуд (Good I. J.) 397

Дельсарт (Delsarte P.) 284, 324, 491
Джелинек (Jelinek F.) 447, 552
Джекобс (Jacobs I. M.) 552

Зигангиров К. Ш. 552
Зиновьев В. А. 83
Зяблов В. В. 236

Йоханнессон (Johannesson R.) 447

Касами (Kasami T.) 153, 186, 238, 491, 521
Касахара (Kasahara M.) 238, 284, 324
Кок (Cocke J.) 83
Колесник В. Д. 491
Костелло (Costello D. J., Jr.) 447
Кердок (Kerdock A. M.) 284
Киясу (Kiyasu Z.) 83
Кобаяси (Kobajashi K.) 10
Краченфелс (Krachenfels C. A.) 10
Кули (Cooley J. W.) 397, 398
Курош А. Г. 60

Ларсен (Larsen K. J.) 447
Лемпель (Lempel A.) 284
Лин (Lin S.) 491, 521
Линднер (Lindner R.) 447

Мак-Вильямс (MacWilliams F. J.) 186, 491, 521
Маклейн (MacLane S.) 60, 111
Мак-Элис (McEliece R. J.) 153, 552
Маллер ((Muller D. E.) 83
Мальцев А. И. 60
Мандельбаум (Mandelbaum D.) 238, 284, 324
Марковский (Markowsky G.) 491
Меггитт (Meggitt J. E.) 186
Месси (Massey J. L.)14, 203, 213, 238, 447, 490, 491
Миллер (Miller R. L.) 398
Мирончиков Е. Т. 491
Митчелл (Mitchell M. E.) 186
Михельсон (Michelson A.) 324, 522
Мэттсон (Mattson H. F., Jr.) 153, 284, 521

Намекава (Namekawa T.) 238, 284
Нг (Ng S. W.) 353
Нейдлер (Nadler M.) 284
Нордстром (Nordstrom A. W.) 284
Нуссбаумер (Nussbaumer H. J.) 398

Оденвальдер (Odenwalder J. P.) 447
Омура (Omura J. K.) 447
Оппенгейм (Oppenheim A. V.) 324

Пааске (Paaske E.) 447
Паттерсон (Petterson N. J.) 324
Пашбург (Paschburg R. N.) 324
Питерсон (Peterson W. W.) 14, 153, 186, 193, 238, 491, 521
Поллард (Pollard J. M.) 284
Прейндж (Prange E.) 153, 186
Препарата (Preparata F. P.) 284, 398
Поллард (Pollard J. M.) 284

Рабинер (Rabiner L. R.) 324
Рамсей (Rumsey H., Jr.) 522
Редди (Reddy S. M.) 284, 353
Рейдер (Rader C. M.) 324
Рейффен (Reiffen B.) 552
Рид (Reed I. S.) 14, 83, 238, 284, 324, 398, 491
Робинсон (Robinson J. P.) 284, 353, 491
Родемич (Rodemich E. R.) 522
Рой-Чоудхури (Ray-Chaudhuri D. K.) 14, 238
Рудолф (Rudolph L. D.) 186, 491, 552

Сайн (Sain M. K.) 447
Сасковец В. Н. 284
Севидж (Savage J. E.) 552
Семаков Н. В. 83
Сервейт (Sarwate D. V.) 10, 324, 397, 398
Симада (Shimada M.) 10
Синглтон (Singleton R. C.) 66, 83
Слепян (Slepian D.) 83
Слоэн (Sloane N. J. A.) 284
Соломон (Solomon G.) 14, 238, 284, 324
Сугияма (Sugiyama Y.) 238, 284

Тейлор (Taylor D. P.) 552
Тиетявяйнен (Tietäväinen A.) 83
Токура (Tokura N.) 238
Томас (Thomas L. H. 397
Торнгейм (Tornheim L.) 60
Тралль (Thrall R. M.) 60
Труонг (Truong T. K.) 398
Турин (Turyn R. J.) 521
Тьюки (Tukey J. W.) 397

Унгербёк (Ungerboeck G.) 10, 552
Уэлдон (Weldon E. J.) 153, 353, 491

Фано (Fano R. M.) 447, 552
Файр (Fire P.) 153
Фергюсон (Ferguson M. J.) 552
Форни (Forney G. D.) 238, 324, 447, 521, 552

Хаккоун (Haccoun D.) 552
Хартман (Hartmann C. R. P.) 324, 552
Хасимото (Hashimoto T.) 10
Хеллер (Heller J. A.) 447
Хельгерт (Helgert H. J.) 284, 324
Хирасава (Hirasawa S.) 238, 284
Хоквингем (Hochquenghem A.) 14, 238
Хунтун (Huntoon Z. McC.) 522
Хэмминг (Hamming R. W.) 13, 83

Цинк (Zink Th.) 522
Цирлер (Zierler N.) 193, 238
Цфасман М. А. (Tsfasman M. A.) 522

Чамер (Chamer A. S.) 10
Чейз (Chase D. A.) 552
Чень (Chien R. T.) 186, 238, 284, 353
Чинь (Chen C. L.) 10, 238, 284
Чой (Choy D. M.) 284

Шафер (Schafer R. W.) 324
Шевийя (Chevillat P. R.) 552
Шеннон (Shannon C. E.) 13, 14
Шёнхейм (Schönheim J.) 83
Шольц (Scholtz R. A.) 238
Штайгер (Staiger L.) 447

Эйбрамсон (Abramson N.) 153
Элайс (Elias P.) 324, 353, 447, 522
Элспас (Elspas B.) 153
Эльгамаль (ElGamal A.) 10
Эш (Ash R. B.) 447

Юстесен (Justesen J.) 14, 232, 236, 397

ПРЕДМЕТНЫЙ УКАЗАТЕЛЬ

Абелева группа 32
Авторегрессионный фильтр 158
Алгебраическое дополнение 51
Алгоритм *Агарвала—Кули* 371—374
— *Берлекэмпа* 214
— — рекуррентный 389
— — ускоренный 385
— *Берлекэмпа—Месси* 208—215, 287, 293, 302, 308
— *Блюстейна* 318
— БПФ 364—371
— быстрый для свертки 357
— *Винограда* для свертки 357
— — БПФ 370—381
— *Витерби* 433—438
— *Герцеля* 322
— *Гуда—Томаса* 365, 370
— декодирования *см.* Декодирование
— деления для многочленов 90
— — — целых чисел 85
— *Евклида* для многочленов 92, 224
— — — целых чисел 86
— *Кули—Тьюки* 366—367
— *Месси* 213—214
— поиска по решетке 438—446
— *Рейдера* 318—319
— *Рида* 79
— синдромного декодирования сверточного кода 423—428
— стек 549
— *Фано* 440—446
— *Форни* 213
Амплитудная модуляция 525
Асимптотический выигрыш кодирования 545
Ассоциативности закон 32, 38
Аффинная перестановка 457
Аффинное подпространство 477

Базис 44
— пополнение 45
Белый шум 525
Бит 18
— информационный 25
— проверочный 25
— частота ошибки 528
— энергия 528
Берлекэмпа алгоритм *см.* Алгоритм
Берлекэмпа—Месси алгоритм *см.* Алгоритм
Блоковый код 13, 18
Блюстейна алгоритм 318
БПФ-алгоритмы 364—371
Буфера переполнение 444
БЧХ-граница 248
БЧХ-коды *см.* Коды БЧХ
Быстрое преобразование Фурье (БПФ) 317
Быстрые алгоритмы свертки 357—364
— коды БЧХ 342—344

Вайнера—Эша код 418
Варшамова—Гилберта граница *см.* Граница *Гилберта*
Вектор 42
— внутренний 254
— инцидентности 480
— ортогональный 46
— сигнальный 240
— синдромных многочленов 49
— сложение 42
— умножение на скаляр 42
— частотный 240
Векторное пространство 42
Вероятность декодирования неудачного 502
— — ошибочного 502
— — правильного 502
Вес q-ичный целого числа 469
— распределение 493
— — в коде *Голея* 147
— — — — с максимальным расстоянием 496
— *Хэмминга* 62
— — кодового слова 65
Винограда алгоритм БПФ 370—381
— — для свертки 367
Витерби алгоритм 433—438
Внешний код 229
Внутренний код 229
Выбрасывание 22
Вылавливание ошибок 173—179
Вычет квадратичный 148
— по модулю многочлена 91
— — — целого числа 85

Гауссовский шум 525
Геометрия евклидова 476
— конечная 474
— проективная 486
Герцеля алгоритм 322
Геталса код 284
Голея код 142—147
— — распределение весов 147
Гоппы код 269—279

Гоппы код в узком смысле 269
— — сепарабельный 272
— многочлен 269
Граница БЧХ 248
— *Гилберта* 269, 511
— — для сверточных кодов 520
— квадратного корня 150
— *Рейггра* 137
— *Сингхтона* 66
— *Хэмминга* 26
— *Элайса* 515
— — для сверточных кодов 518
Группа 32
— абелева 29, 32
— дважды транзитивная 457
— конечная 32
— перестановок 34
— — аффинная 456
— порядок 32
— транзитивная 457
— циклическая 36
Гуда—Томаса алгоритм 365, 370

Двоичный код *Голея* 142—147
Двойной транзитивности свойство 457
Декодер 12
— во временно́й области 308
— — — — для поля характеристики 2 309
— для альтернантного кода 316
— — двоичного ОРМ-кода 535—538
— — кода-произведения 332
— — q-ичного ОРМ-кода 538—541
— мажоритарный 449
— — L-шаговый 451
— *Меггитта* 165—172
— на регистрах сдвига 162—165
— неполный 22
— несистематический 169
— *Питерсона—Горенстейна — Цирлера* 193—201
— полный 22
— с вылавливанием ошибок 173—179
— синдромный 164, 423—428
— систематический 162
— с обратной связью 413
— частотный для кодов БЧХ 287
— — — — — исправляющий ошибки и стирания 298—299
— — — — — основанный на алгоритме *Евклида* 293
— — — — — схемные реализации 289
— — — двоичных кодов БЧХ 306
— — — расширенных кодов БЧХ 303—304
Декодирование альтернантных кодов 314—317
— быстрое кодов БЧХ 211—221
— во временно́й области 303—309
— двоичных кодов БЧХ 221—223
— за границей БЧХ 309—314
— мажоритарное 449
— мягкое блоковых кодов 531—541
— — сверточных кодов 541—548
— неполное 22
— поиском по решетке 438—446
— полное 22
— по минимуму расстояния 21, 434
— — обобщенному минимальному расстоянию 534—537
— последовательное 548—551
— просмотром таблицы 168
— расширенных кодов РС 299—303
— с исправлением t ошибок 285—287
— — — ошибок и стираний 294—299
— — помощью алгоритма *Евклида* 223—227
Делимость 38
Демодуляция 526
Дерево 406
Диаграмма состояний 506
Дискретное преобразование *Фурье* (ДПФ) в поле *Галуа* 240
— — — — — — быстрое 317, 364—371
— — — — — $GF(p)$, p — простое число *Ферма* 368
— — — — — комплексных чисел 240, 265
— — — двумерное 338
— — — полубыстрое 322
Дистанционный профиль 414
Дистрибутивности закон 38, 42
Длина кода 17
— — примитивная 122
— кодового блока 401, 408
— — ограничения 400, 408
— — слова информационная 401
— преобразования *Фурье* 240
— свободная 414
ДПФ *см.* Дискретное преобразование *Фурье*
Древовидный код 399, 401
— — свойства 401—402
Дуальное подпространство 63
Дуальный код 63

Закон ассоциативности 32, 38
— дистрибутивности 38, 42
Замкнутости свойство 32, 37

Евклида алгоритм 86, 92, 224
Евклидова геометрия 476
Евклидово-геометрический код 476, 481
Единица 32
— кольца 39

Ивадаре код 429
Идеал кольца 117
Идемпотент 246
Изоморфизм полей 104
Интерференция межсимвольная 525
Информационная длина кодового слова 401, 408
Информационный бит 25
— кадр 400
— символ 25
Информационное слово 63
Инцидентности вектор 480

Кадр информационных символов 400
— кодового слова 401
Канал 12
— гауссовский 524
— ограниченный по полосе 523
— пропускная способность 13, 528
Квадратично-вычетный код 149
Квадратичный вычет 148
КИО-фильтр 158
Китайские теоремы об остатках 328—332
Класс сопряженных элементов 244
Классификация древовидных кодов 403
Код 13
— альтернантный 264
— — в частотной области 265
— блоковый 18
— — скользящий 402
— БЧХ 187—236
— — быстрый 343
— — непримитивный 192
— — примитивный 187.
— — расширенный 258
— *Вайнера—Эша* 418
— внешний 229
— внутренний 229
— выкалывание 77
— *Геталса* 284
— *Голея* 142—147
— *Гоппы* 269—279
— — в узком смысле 269, 271
— — двоичный 273
— — расширенный 270
— — сепарабельный 272
— двоичный 17
— двумерный 339
— древовидный 399, 401
Код дуальный 63, 342
— евклидово-геометрический 476, 481
— *Ивадаре* 429
— каскадный 228—232
— катастрофический 412
— квадратично-вычетный 148—152
— квазисовершенный 74
— *Кердока* 284
— контролирующий ошибки 13, 18
— линейный 61
— — нумератор весов 499
— многомерный 325
— *Нейдлера* 284
— некатастрофический 412
— несистематический 66, 402
— *Нордстрома—Робинсона* 284
— основанный на спектральных методах 239—284
— подкод над подполем 77
— *Препараты* 280
— проективно-геометрический 485
— произведение 326—342
— расширенный 75
— решетчатый 401
— *Рида—Маллера* (РМ) 76
— — порядка r 76
— — — — обобщенный 469
— — — — циклический 469
— *Рида—Соломона* (РС) 201—203
— — непримитивный 475
— — расширенный 254
— самодуальный 63, 521
— сверточный 14, 399
— — исправляющий пакеты ошибок 428—433
— — — t ошибок 414
— — катастрофический 412
— — некатастрофический 412
— с выбрасыванием 77
— — повторением 23
— симплексный 453
— систематический 66, 402
— совершенный 74
— *Унгербёка* 547
— *Файра* 139—142
— *Хэмминга* 23, 83
— — недвоичный 71, 83
— циклический 112—153
— — исправляющий две ошибки 134
— — непримитивный 475
— — примитивный 122
— — расширенный 254
— — укороченный 179—183
— — частотное определение 248
— *Юстесена* 232—236

Кодер 12
— на регистрах сдвига 162—165
— несистематический 162
— систематический 162
— — для кодов *Гоппы* 270
— — частотный для кодов БЧХ 251—253
— — — — кода РС 249
— — — — расширенного кода БЧХ 261
— — — — — — РС 256—257
— — — — циклического кода 247—253
Кодовая длина блока 408
Кодовое слово канала 12, 18
— — источника 12
Кольцо 29—37
— коммутативное 38
— многочленов 88—95
— отношений 87
— с единицей 38
— целых чисел 84—88
Конструктивное расстояние 188
Контролирующий ошибки код 12
Класс сопряженных элементов 244
Кронекеровское произведение 378
q-ичный потомок 458
Кули—Тьюки алгоритм 366—367

Левый обратный 30
— смежный класс 36
Лидер смежного класса 36, 68
Линейная зависимость 44
— комбинация 44
— независимость 42
— свертка 354
Линейность сверточного кода 402
Линейный код 61
Логические цепи 154—157
Локатор 134, 457
Локаторный многочлен 249
Лукаса теорема 458
L-шаговое мажоритарное декодирование 451

Мажоритарное декодирование 449
Матрица *Вандермонда* 144, 195—196
— вырожденная 50
— главная диагональ 48
— единичная 48
— канонический ступенчатый вид 52
— квадратная 48
— кронекеровское произведение 378
— невырожденная 50
— нулевое подпространство 52
— определитель 62
Матрица порождающая 62
— — в систематическом виде 65
— — из многочленов 409
— пространство столбцов 52
— — строк 52
— транспонированная 50
— элементарная 50
Меггитта декодер 165—172
Метрика ребра 544
— *Хэмминга* 27
Минимальное расстояние 20, 414
Минимальный многочлен 104
Минор 51
Многомерная циклическая свертка 371
Многомерное ДПФ 338
Многомерные спектральные методы 325—352
Многомерный код 321
— спектр 339
Многочлен 88
— алгоритм деления 90
— — *Евклида* 92, 229
— ассоциированный 242
— взаимно простой 89
— взаимный 130
— — к локаторному 212
— *Гоппы* 269
— значений ошибок 212
— информационный 119, 120
— истинно-локаторный 290
— кодовый 113, 120
— кольцо 88
— — по модулю приведенного многочлена 95
— корень 94
— локаторов 249
— — ошибок 194
— — — и стираний 296, 298
— минимальный 104
— — элемента 122
— неприводимый 89
— НОД 89
— НОК 89
— ошибок 119, 120
— порождающий 117, 406
— приведенный 88
— примитивный 103
— принятый 119, 120
— проверочный 118
— простой 89
— синдромный 119, 120, 212
— степень 88
Множество ортогональных проверок 449
— согласующихся проверок 449, 451
— сопряженных элементов 125

Модулятор/демодулятор 525
Модуляционный алфавит 526
Мягкое декодирование блоковых кодов 531—541
— — сверточных кодов 541—548

Набор сигналов 526
Невязка 205
Недвоичный код *Хэмминга* 71, 83
Неисправляемая конфигурация ошибок 309
Нейдлера код 284
Некатастрофический код 412
Неполный декодер 22
Неприводимый многочлен 89
Нерекуррентный фильтр 158
Несистематический код 66, 402
Несистематический кодер 162
НОД многочленов 89
— целых чисел 85
НОК многочленов 90
— целых чисел 84
Нордстрома—Робинсона код 284
Нулевое подпространство матрицы 52
Нумератор весов 499

Область целостности 42
Обнаружитель с мягким решением 531
Обобщенный код *Рида—Маллера* (ОРМ-код) 469—475
Обратный элемент 32
— — левый 32
— — правый 32
Ограничения (условия) сопряженности 243
Ограниченный по полосе сигнал 524
ОРМ-код порядка r 469
— — — циклический 469
Ортогональное дополнение 47
Ошибок вылавливание 173—179
— исправляемая конфигурация 309
— неисправляемая конфигурация 309
— пакет 136, 428
— — циклический 136
— распространение 469

Память перемежающаяся 232
Переполнение буфера 444
Перестановка 34
— аффинная 456
Питерсона — Горенстейна — Цирлера декодер 193—201
Подгруппа 37
— разложение по 37
Подкод над подполем 77
Подпространство 45
— аффинное 477
— дуальное 63
— пересечение 498
— прямая сумма 498
— размерность 45
Поле 39
— изоморфизм 104
— конечное 40, 95—111
— — основанное на кольце многочленов 95—99
— — — — — целых чисел 87—88
— примитивный элемент 99
— разложения многочлена 106
— расширение 41
— характеристика 99, 103
— целые числа 99
Полный декодер 22
Пополнение базиса 45
— кода 77
Порождающая матрица 62, 65
— — из многочленов 408
Порождающий многочлен 117, 120, 406
Порядок группы 32
— элемента 36
Последовательное декодирование 548—551
Потомок q-ичный 458
Преобразование *Фурье* в поле *Галуа* 240
— — — — — быстрые алгоритмы 317, 364—371
— — — — $GF(p)$, p — простое число *Ферма* 368
— — — — комплексных чисел 240, 265
— — многомерное 338
Препараты код 280
Приведенный многочлен 88
Примитивная длина 122
Примитивный многочлен 103
— циклический код 122
— элемент 99
Проверка на четность 24
Проверочная матрица 64, 65
— — из многочленов 409
Проверочный бит 25
Проверочный многочлен 118, 120
Проективная геометрия 486
Проективно-геометрический код 485
Произведение кронекеровское 378
Производная формальная 90
Пропускная способность канала 13, 528
Простое число 84
— — *Ферма* 368
Простой многочлен 89
Пространство векторов 42

Профиль дистанционный 414
Процедура БерМес 389
— *Ченя* 199

Радиус покрытия 74
— сферической упаковки 74
Разложение по подгруппе 37
Размерность 45
Ранг матрицы по столбцам 52
— — — строкам 52
Распределение весов 493
— — в коде *Голея* 147
— — — — — с максимальным расстоянием 496
— — сверточных кодов 505—508
Распространение ошибок 413, 469
— — катастрофическое 413
— — обычное 413
Расстояние евклидово 543
— конструктивное 189
— минимальное 20, 414
— — обобщенное 534
— по *Хэммингу* 20
— свободное 414
Расширение кода 28
— поля 75
Расширенный код БЧХ 258
— — РС 254
Ребра метрика 544
Регистр сдвига 154
— — разряды 154
— — с линейной обратной связью 158
Рейгера алгоритм 318
Рейдера граница 137
Рекуррентный фильтр 158
Решетка сверточного кода 505
Решетчатый код 401
Рида алгоритм 79

Самодуальный код 63, 521
Свертка линейная 356
— — метод перекрытия с накоплением 356
— циклическая 354
— — алгоритм *Агарвала—Кули* 365—366
— — — *Винограда* 357
— — в суррогатном поле 394
Сверточный код 14, 399
— — исправляющий t ошибок 414
— — — пакеты ошибок 428—433
— — катастрофический 412
— — некатастрофический 412
Свободная длина 414
Свободное расстояние 414
Свойство двойной транзитивности 457
Сепарабельный код *Гоппы* 272
Сигнал, мощность 524
— ограниченный по полосе 524
— спектр 525
— ширина полосы 525
— энергия 525
Симплексный код 453
Синглтона граница 66
Синдром 25, 70
Синдромное декодирование блоковых кодов 164—184
— — сверточных кодов 423—428
Синдромный многочлен 119, 120, 212
Систематический код 66, 402
— кодер 162
Скаляр 39, 88
— умножитель 155
Скалярное произведение 16
Скорость кода 18, 400
След q-ичный 244
Слово, вес *Хэмминга* 62
— внутреннее 254
— кодовое источника 12
— — канала 12, 18
— принятое
Сложение векторное 42
— по модулю 2 29
Спектральные методы 239—324
— — многомерные 325, 352
Совершенный код 74
Согласующиеся проверки 449
Сопряженный элемент 125
Спектр 240
— двумерный 339
— профильтрованный 265
Спектральная (частотная) компонента 242
Спектральное описание циклических кодов 247—253
Спектральный многочлен 242
Сравнения в кольце многочленов 91
— — — целых чисел 85
Стандартное расположение 37
Стирание 22
— помеченное 298
Сумматор по модулю 2 29
Суперканал 229
Суррогатное поле 394
Сферы декодирования 21

Таблица значений синдрома 120
Такт 154
Теорема, алгоритм *Берлекэмпа—Месси* 208
— — деления 85, 90
— — *Евклида* 86, 92, 229

Теорема, алгоритм *Форни* 213
— граница БЧХ 248
— — *Гилберта* 511, 520
— — *Рейгера* 137
— — *Синглтона* 66
— — — квадратного корня 150
— *Лукаса* 458
— *Меггитта* 265
— о свертке 241
— свойство сдвига 242
Тождество *Мак-Вильямс* 498
Транзитивная группа перестановок 457
Транспонирование матрицы 50
Троичный канал 237

Удлинение кода 75
Укорочение кода 75
Укороченные циклические коды 179—183
Умножение вектора на скаляр 42
— по модулю 2 29
Умножитель 155
— на скаляр 156
Унгербёка код 547
Ускоренное декодирование кодов БЧХ 391—393
Ускоренный алгоритм *Берлекэмпа—Месси* 385
Условия (ограничения) сопряженности 243

Файра код 139—142
Фано алгоритм 440—446
Фильтр 157
— авторегрессионный 158
— нерекуррентный 158
— рекуррентный 158
— с конечным импульсным откликом (КИО-фильтр) 158
— — линейной обратной связью 157—158
— цифровой 157
Форни алгоритм 213
Формальная производная 90

Характеристика поля 103
Хорда 244
Хэмминга вес 20
— метрика 27
— расстояние 20

Целые числа поля 99
Цепи логические 154—157
Цепь умножения на многочлен 159
Циклическая свертка 354
— — алгоритм *Агарвала —Кули* 365—366
— — — *Винограда* 357
— — быстрые алгоритмы 357
— — в суррогатном поле 394
Циклический код 112—153
— — исправляющий две ошибки 134
— — непримитивный 475
— — примитивный 122
— — расширенный 254
— — укороченный 179—183
— — частотное определение 248

Частота 240
— граничная 255
— информационная 252
— ошибки на бит 528
— проверочная 248
— свободная 253
— связанная 253
Частотная функция 240
Число взаимно простое 84
— поля целое 99
— простое 84
— составное 84

Шаблон во временной области 264
— в частотной области 264
Ширина полосы сигнала 525

Элемент примитивный 99
— сопряженный 125
Элементарные операции над матрицами 52
Элементы устройств 155
Энергия сигнала 525

Юстесена код 232—236

ОГЛАВЛЕНИЕ

От редактора перевода 5
Предисловие к русскому изданию 7
Предисловие 8

1. ВВЕДЕНИЕ 11
1.1. Дискретный канал связи 11
1.2. История кодирования, контролирующего ошибки 13
1.3. Приложения 15
1.4. Основные понятия 16
1.5. Простейшие коды 23
Задачи 26

2. ВВЕДЕНИЕ В АЛГЕБРУ 28
2.1. 2-поле и 6-10-поле 28
2.2. Группы 31
2.3. Кольца 37
2.4. Поля 39
2.5. Векторные пространства 42
2.6. Линейная алгебра 48
Задачи 57
Замечания 60

3. ЛИНЕЙНЫЕ БЛОКОВЫЕ КОДЫ 61
3.1. Структура линейных блоковых кодов 61
3.2. Матричное описание линейных блоковых кодов 62
3.3. Стандартное расположение 65
3.4. Коды Хэмминга 71
3.5. Совершенные и квазисовершенные коды 73
3.6. Простые преобразования линейного кода 74
3.7. Коды Рида—Маллера 76
Задачи 81
Замечания 83

4. АРИФМЕТИКА ПОЛЕЙ ГАЛУА 84
4.1. Кольцо целых чисел 84
4.2. Конечные поля, основанные на кольце целых чисел 87
4.3. Кольца многочленов 88
4.4. Конечные поля, основанные на кольцах многочленов 95
4.5. Примитивные элементы 99
4.6. Структура конечного поля 103
Задачи 110
Замечания 111

5. ЦИКЛИЧЕСКИЕ КОДЫ 112
5.1. Код с точки зрения расширения поля 112
5.2. Полиномиальное описание циклических кодов 116
5.3. Минимальные многочлены и сопряжения 121
5.4. Матричное описание циклических кодов 128
5.5. Коды Хэмминга как циклические коды 131
5.6. Циклические коды, исправляющие две ошибки 134
5.7. Циклические коды, исправляющие пакеты ошибок 136
5.8. Двоичный код Голея 142
5.9. Квадратично-вычетные коды 148
Задачи 152
Замечания 153

6. СХЕМНАЯ РЕАЛИЗАЦИЯ ЦИКЛИЧЕСКОГО КОДИРОВАНИЯ 154
6.1. Логические цепи для арифметики конечного поля 154
6.2. Цифровые фильтры 157
6.3. Кодеры и декодеры на регистрах сдвига 162
6.4. Декодер Меггитта 165
6.5. Вылавливание ошибок 173
6.6. Укороченные циклические коды 179
6.7. Декодер Меггитта для кода Голея 183
Задачи 184
Замечания 186

7. КОДЫ БОУЗА — ЧОУДХУРИ — ХОКВИНГЕМА 187
7.1. Определение кодов БЧХ 187
7.2 Декодер Питерсона—Горенстейна—Цирлера 193
7.3 Коды Рида—Соломона 201
7.4 Синтез авторегрессионных фильтров 203
7.5 Быстрое декодирование кодов БЧХ 211
7.6 Декодирование двоичных кодов БЧХ 221
7.7. Декодирование с помощью алгоритма Евклида 223
7.8. Каскадные (гнездовые) коды 228
7.9. Коды Юстесена 232
Задачи 236
Замечания 238

8. КОДЫ, ОСНОВАННЫЕ НА СПЕКТРАЛЬНЫХ МЕТОДАХ 239
8.1. Преобразования Фурье в поле Галуа 239
8.2. Ограничения сопряженности и идемпотенты 243
8.3. Спектральное описание циклических кодов 247
8.4. Расширенные коды Рида—Соломона 254
8.5. Расширенные коды БЧХ 258
8.6. Альтернантные коды 263
8.7. Характеристики альтернантных кодов 266
8.8. Коды Гоппы 269
8.9. Коды Препараты 279
Задачи 283
Замечания 284

9. АЛГОРИТМЫ, ОСНОВАННЫЕ НА СПЕКТРАЛЬНЫХ МЕТОДАХ 285
9.1. Спектральные методы декодирования 285
9.2. Исправление стираний и ошибок 294
9.3. Декодирование расширенных кодов Рида—Соломона 299
9.4. Декодирование расширенных кодов БЧХ 303
9.5. Декодирование во временной области 304
9.6. Декодирование за границей БЧХ 309
9.7. Декодирование альтернантных кодов 314
9.8. Вычисление преобразований в конечных полях 317
Задачи 323
Замечания 324

10. МНОГОМЕРНЫЕ СПЕКТРАЛЬНЫЕ МЕТОДЫ 325
10.1. Коды-произведения 325
10.2. Китайские теоремы об остатках 328
10.3. Декодирование кода-произведения 332
10.4. Многомерные спектры 338
10.5. Быстрые коды БЧХ 342
10.6. Декодирование многомерных кодов 346

10.7. Длинные коды над малыми полями 347
Задачи 352
Замечания 353

11. БЫСТРЫЕ АЛГОРИТМЫ 354
11.1. Линейная свертка и циклическая свертка 354
11.2. Быстрые алгоритмы свертки 357
11.3. Быстрые преобразования Фурье 364
11.4. Алгоритмы Агарвала—Кули вычисления сверток 371
11.5. Алгоритм Винограда быстрого преобразования Фурье . . . 374
11.6. Ускоренный алгоритм Берлекэмпа—Месси 381
11.7. Рекуррентный алгоритм Берлекэмпа—Месси 387
11.8. Ускоренное декодирование кодов БЧХ 391
11.9. Свертка в суррогатных полях 394
Задачи 396
Замечания 397

12. СВЕРТОЧНЫЕ КОДЫ 399
12.1. Древовидные и решетчатые коды 399
12.2. Описание сверточных кодов с помощью многочленов 406
12.3. Исправление ошибок; понятия расстояния 412
12.4. Матричное описание сверточных кодов 415
12.5. Некоторые простые сверточные коды 418
12.6. Алгоритмы синдромного декодирования 423
12.7. Сверточные коды для исправления пакетов ошибок 428
12.8. Алгоритм декодирования Витерби 433
12.9. Алгоритмы поиска по решетке 438
Задачи 446
Замечания 447

13. КОДЫ И АЛГОРИТМЫ ДЛЯ ДЕКОДИРОВАНИЯ МАЖОРИТАРНЫМ МЕТОДОМ 448
13.1. Декодирование мажоритарным методом 448
13.2. Схемы мажоритарного декодирования 452
13.3. Аффинные перестановки для циклических кодов 456
13.4. Циклические коды, основанные на перестановках 461
13.5. Сверточные коды с мажоритарным декодированием 465
13.6. Обобщенные коды Рида—Маллера 469
13.7. Евклидово-геометрические коды 475
13.8. Проективно-геометрические коды 485
Задачи 490
Замечания 490

14. КОМПОЗИЦИЯ И ХАРАКТЕРИСТИКИ КОНТРОЛИРУЮЩИХ ОШИБКИ КОДОВ 492
14.1. Распределения весов 492
14.2. Вероятности ошибочного декодирования и неудачного декодирования 502
14.3. Распределение весов сверточных кодов 505
14.4. Границы минимального расстояния для блоковых кодов . . . 508
14.5. Границы минимального расстояния для сверточных кодов 516
Задачи 520
Замечания 521

15. ЭФФЕКТИВНАЯ ПЕРЕДАЧА СИГНАЛОВ ПО ЗАШУМЛЕННЫМ КАНАЛАМ 523
15.1. Ограниченный по полосе гауссовский канал 524
15.2. Энергия на бит и частота ошибок на бит 527
15.3. Мягкое декодирование блоковых кодов 531
15.4. Мягкое декодирование сверточных кодов 541
15.5. Последовательное декодирование 548
Задачи 551
Замечания 552

Литература 553

Именной указатель 564

Предметный указатель 566

УВАЖАЕМЫЙ ЧИТАТЕЛЬ!

Ваши замечания о содержании книги, ее оформлении, качестве перевода и другие просим присылать по адресу: 129820, Москва, И-110, ГСП, 1-й Рижский пер., д. 2, издательство «Мир».

Ричард Э. Блейхут

ТЕОРИЯ И ПРАКТИКА КОДОВ, КОНТРОЛИРУЮЩИХ ОШИБКИ

Ст. научн. редактор Г. М. Ильичева
Мл. научн. редакторы: Р. И. Пяткина, Л. А. Королева
Художник Л. М. Муратова
Художественный редактор В. И. Шаповалов
Технический редактор В. П. Сизова
Корректор В. С. Соколов

ИБ № 5595

Сдано в набор 3.07.85. Подписано к печати 13.02.86.
Формат 60×90¹/₁₆. Бумага типографская № 1.Печать высокая.
Гарнитура литературная. Объем бум. л. 18,00. Усл. печ. л. 36,00. Усл. кр.-отт. 36,00.
Уч.-изд. л. 32,21. Изд. № 1/4310. Тираж 10000 экз. Зак. 247. Цена 2 р. 60 к.

ИЗДАТЕЛЬСТВО «МИР»
129820, ГСП, Москва, И-110, 1-й Рижский пер., 2.

Ленинградская типография № 6 ордена Трудового Красного Знамени
Ленинградского объединения «Техническая книга» им. Евгении Соколовой
Союзполиграфпрома при Государственном комитете СССР
по делам издательств, полиграфии и книжной торговли.
193144, г. Ленинград, ул. Моисеенко, 10.